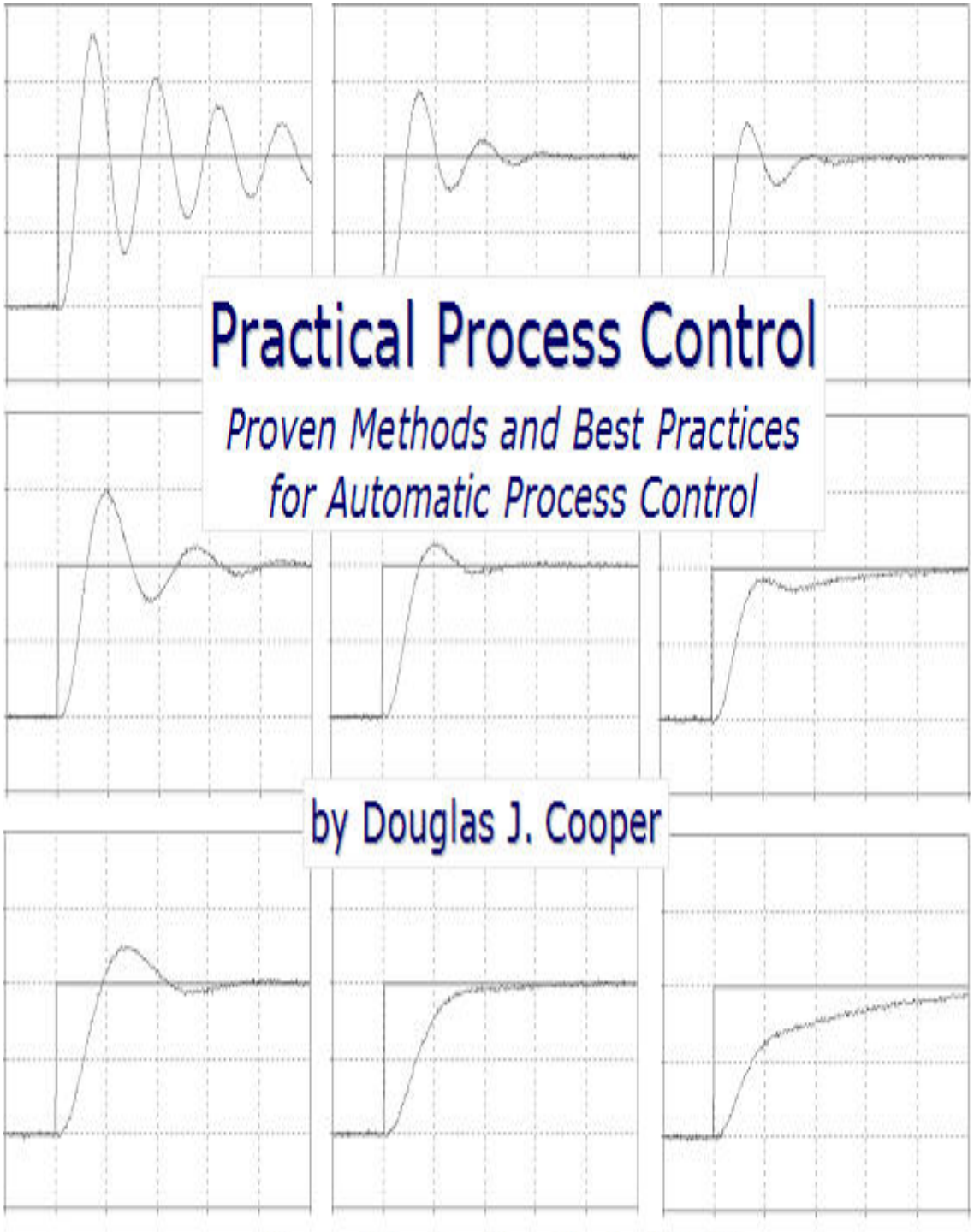


Practical Process Control

Proven Methods and Best Practices for Automatic Process Control

by Douglas J. Cooper

co-author articles by Allen Houtz, Bob Rice

guest articles by Jim Riggs, Peter Nachtwey, Fred Thomasson

Practical Process Control

Proven Methods and Best Practices for Automatic PID Control

I.Modern Control is Based on Process Dynamic Behavior 4
1- Fundamental Principles of Process Control 4
Motivation and Terminology of Automatic Process Control 4
The Components of a Control Loop 8
Process Data, Dynamic Modeling and a Recipe for Profitable Control 13
Sample Time Impacts Controller Performance 16
2) Graphical Modeling of Process Dynamics: Heat Exchanger Case Study 18
Step Test Data From the Heat Exchanger Process 18
Process Gain Is The "How Far" Variable 22
Process Time Constant is The "How Fast" Variable 26
Dead Time Is The "How Much Delay" Variable 28
Validating Our Heat Exchanger FOPDT Model 32
3) Modeling Process Dynamics: Gravity Drained Tanks Case Study 35
The Gravity Drained Tanks Process 35
Dynamic "Bump" Testing Of The Gravity Drained Tanks Process 39
Graphical Modeling of Gravity Drained Tanks Step Test 43
Modeling Gravity Drained Tanks Data Using Software 50
4) Software Modeling of Process Dynamics: Jacketed Stirred Reactor Case Study 56
Design Level of Operation for the Jacketed Stirred Reactor Process 56
Modeling the Dynamics of the Jacketed Stirred Reactor with Software 60
Exploring the FOPDT Model With a Parameter Sensitivity Study 64
II. PID Controller Design and Tuning (by Doug Cooper) 72
5) Process Control Preliminaries 72
Design and Tuning Recipe Must Consider Nonlinear Process Behavior 72
A Controller's "Process" Goes From Wire Out to Wire In 79
The Normal or Standard PID Algorithm 82
6) Proportional Control - The Simplest PID Controller 87
The P-Only Control Algorithm 87
P-Only Control of the Heat Exchanger Shows Offset 91
P-Only Disturbance Rejection Of The Gravity Drained Tanks 96
7) Caution: Pay Attention to Units and Scaling 102
Controller Gain Is Dimensionless in Commercial Systems 102
8) Integral Action and PI Control 108
Integral Action and PI Control 108
PI Control of the Heat Exchanger 114
PI Disturbance Rejection Of The Gravity Drained Tanks 120
The Challenge of Interacting Tuning Parameters 125
PI Disturbance Rejection in the Jacketed Stirred Reactor 130
Integral (Reset) Windup, Jacketing Logic and the Velocity PI Form 136
9) Derivative Action and PID Control 145
PID Control and Derivative on Measurement 145
The Chaos of Commercial PID Control 150
PID Control of the Heat Exchanger 155
Measurement Noise Degrades Derivative Action 160
PID Disturbance Rejection Of The Gravity Drained Tanks 164
10) Signal Filters and the PID with Controller Output Filter Algorithm 171

Using Signal Filters In Our PID Loop 171
PID with Controller Output (CO) Filter 176
PID with CO Filter Control of the Heat Exchanger 181
PID with CO Filter Disturbance Rejection in the Jacketed Stirred Reactor 186
III. Additional PID Design and Tuning Concepts (by Doug Cooper) 193
11) Exploring Deeper: Sample Time, Parameter Scheduling, Plant-Wide Control 193
Sample Time is a Fundamental Design and Tuning Specification 193
Parameter Scheduling and Adaptive Control of Nonlinear Processes 205
Plant-Wide Control Requires a Strong PID Foundation 215
12) Controller Tuning Using Closed-Loop (Automatic Mode) Data 218
Ziegler-Nichols Closed-Loop Method a Poor Choice for Production Processes 218
Controller Tuning Using Set Point Driven Data 221
Do Not Model Disturbance Driven Data for Controller Tuning 225
13) Evaluating Controller Performance 231
Comparing Controller Performance Using Plot Data 231
IV. Control of Integrating Processes (by Doug Cooper & Bob Rice) 237
14) Integrating (Non-Self Regulating) Processes 237
Recognizing Integrating (Non-Self Regulating) Process Behavior 237
A Design and Tuning Recipe for Integrating Processes 243
Analyzing Pumped Tank Dynamics with a FOPDT Integrating Model 247
PI Control of the Integrating Pumped Tank Process 257
V. Advanced Classical Control Architectures (by Doug Cooper & Allen Houtz) 266
15) Cascade Control For Improved Disturbance Rejection 266
The Cascade Control Architecture 266
An Implementation Recipe for Cascade Control 274
A Cascade Control Architecture for the Jacketed Stirred Reactor 279
Cascade Disturbance Rejection in the Jacketed Stirred Reactor 285
16) Feed Forward with Feedback Trim For Improved Disturbance Rejection 297
The Feed Forward Controller 297
Feed Forward Uses Models Within the Controller Architecture 303
Static Feed Forward and Disturbance Rejection in the Jacketed Reactor 310
17) Ratio, Override and Cross-Limiting Control 321
The Ratio Control Architecture 321
Ratio Control and Metered-Air Combustion Processes 325
Override (Select) Elements and Their Use in Ratio Control 331
Ratio with Cross-Limiting Override Control of a Combustion Process 342
18) Cascade, Feed Forward and Three-Element Control 350
Cascade, Feed Forward and Boiler Level Control 350
Dynamic Shrink/Swell and Boiler Level Control 353
VI. Process Applications in Control 357
19) Distillation Column Control 357
Distillation: Introduction to Control 357
Distillation: Major Disturbances & First-Level Control 361
Distillation: Inferential Temperature Control & Single-Ended Control 366
Distillation: Dual Composition Control & Constraint Control 370
20) Discrete Time Modeling of Dynamic Systems (by Peter Nachtwey) 379
A Discrete Time Linear Model of the Heat Exchanger 379
21) Fuzzy Logic and Process Control (by Fred Thomassom) 385
Envelope Optimization and Control Using Fuzzy Logic 385

I.Modern Control is Based on Process Dynamic Behavior

1- Fundamental Principles of Process Control

Motivation and Terminology of Automatic Process Control

Automatic control systems enable us to operate our processes in a safe and profitable manner. Consider, as on this site, processes with streams comprised of gases, liquids, powders, slurries and melts. Control systems achieve this "safe and profitable" objective by continually measuring ***process variables*** such as temperature, pressure, level, flow and concentration - and taking actions such as opening valves, slowing down pumps and turning up heaters - all so that the measured process variables are maintained at operator specified set point values.

Safety First
The overriding motivation for automatic control is safety, which encompasses the safety of people, the environment and equipment.

The safety of plant personnel and people in the community are the highest priority in any plant operation. The design of a process and associated control system must always make human safety the primary objective.

The tradeoff between safety of the environment and safety of equipment is considered on a case by case basis. At the extremes, the control system of a multi-billion dollar nuclear power facility will permit the entire plant to become ruined rather than allow significant radiation to be leaked to the environment.

On the other hand, the control system of a coal-fired power plant may permit a large cloud of smoke to be released to the environment rather than allowing damage to occur to, say, a single pump or compressor worth a few thousand dollars.

The Profit Motive
When people, the environment and plant equipment are properly protected, our control objectives can focus on the profit motive. Automatic control systems offer strong benefits in this regard.

Plant-level control objectives motivated by profit include:
- meeting final product specifications
- minimizing waste production
- minimizing environmental impact

- minimizing energy use
- maximizing overall production rate

It can be most profitable to operate as close as possible to these minimum or maximum objectives. For example, our customers often set our product specifications, and it is essential that we meet them if failing to do so means losing a sale.

Suppose we are making a film or sheet product. It takes more raw material to make a product thicker than the minimum our customers will accept on delivery. Consequently, the closer we can operate to the minimum permitted thickness constraint without going under, the less material we use and the greater our profit.

Or perhaps we sell a product that tends to be contaminated with an impurity and our customers have set a maximum acceptable value for this contaminant. It takes more processing effort (more money) to remove impurities, so the closer we can operate to the maximum permitted impurity constraint without going over, the greater the profit.

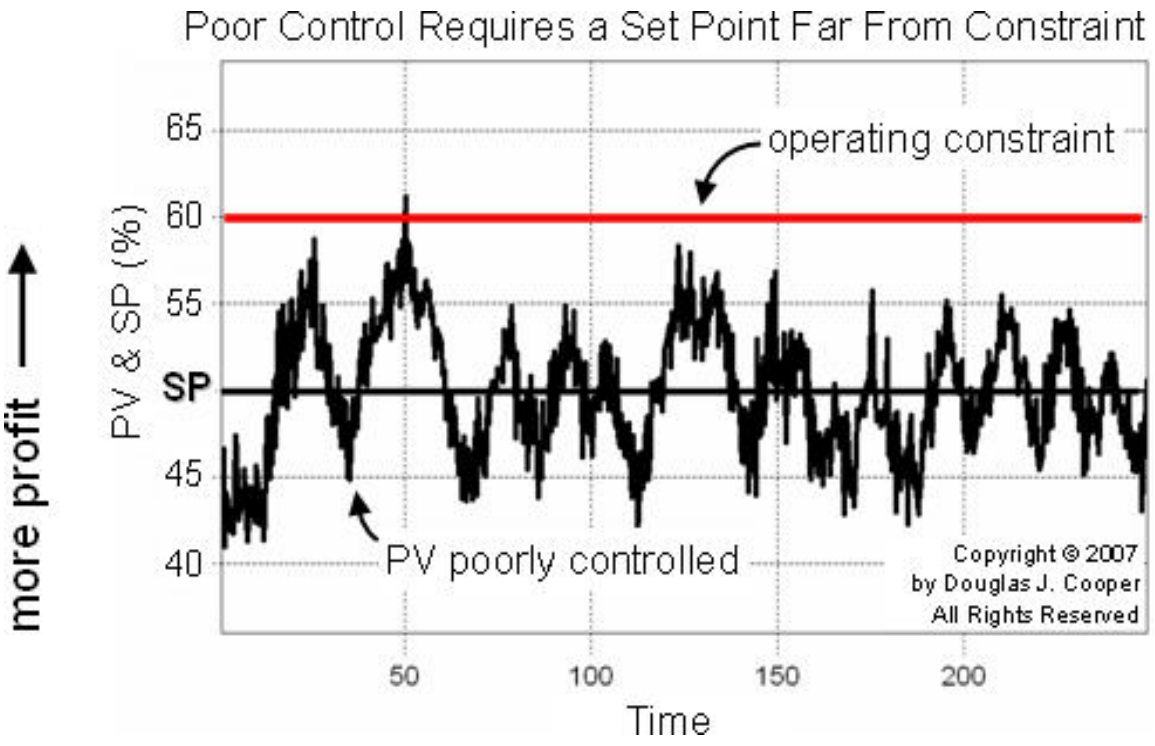
Whether it is a product specification, energy usage, production rate, or other objective, approaching these targets ultimately translates into operating the individual process units within the plant as close as possible to predetermined set point values for temperature, pressure, level, flow, concentration and the other measured process variables.

Controllers Reduce Variability

As shown in the plot below, a poorly controlled process can exhibit large variability in a measured process variable (e.g., temperature, pressure, level, flow, concentration) over time.

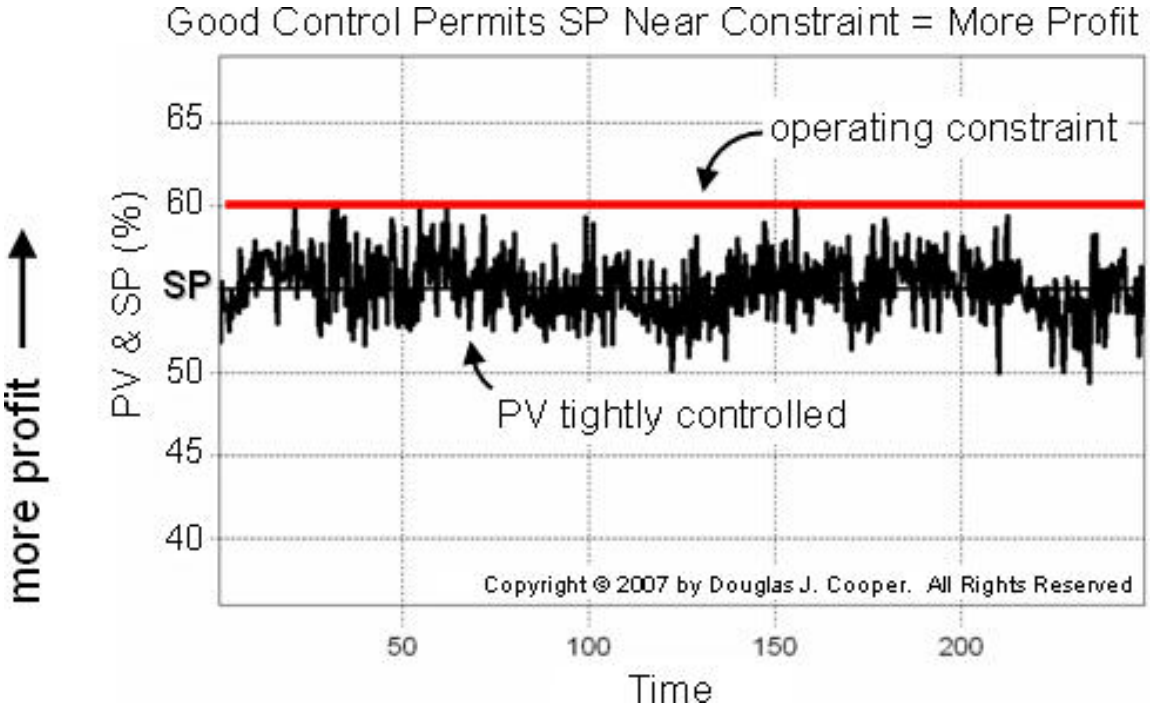
Suppose, as in this example, the measured process variable (PV) must not exceed a maximum value. And as is often the case, the closer we can run to this operating constraint, the greater our profit (note the vertical axis label on the plot).

To ensure our operating constraint limit is not exceeded, the operator-specified set point (SP), that is, the point where we want the control system to maintain our PV, must be set far from the constraint to ensure it is never violated. Note in the plot that SP is set at 50% when our PV is poorly controlled.



Below we see the same process with improved control. There is significantly less variability in the measured PV, and as a result, the SP can be moved closer to the operating constraint.

With the SP in the plot below moved to 55%, the average PV is maintained closer to the specification limit while still remaining below the maximum allowed value. The result is increased profitability of our operation.

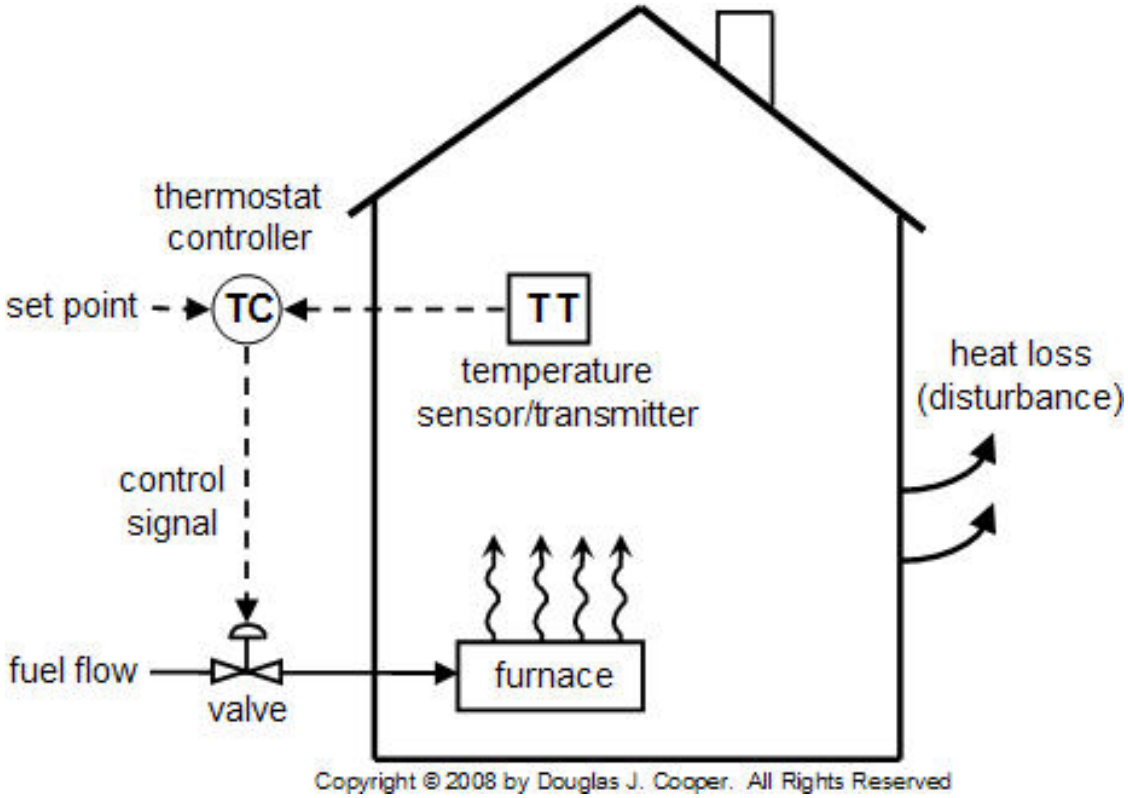


Terminology of Control
We establish the jargon for this site by discussing a home heating control system as illustrated below.

This is a simplistic example because a home furnace is either on or off. Most control challenges have a ***final control element*** (FCE), such as a valve, pump or compressor, that can receive and respond to a complete range of ***controller output*** (CO) signals between full on and full off. This would include, for example, a valve that can be open 37% or a pump that can be running at 73%.

For our home heating process, the ***control objective*** is to keep the ***measured process variable*** (PV) at the ***set point value*** (SP) in spite of ***unmeasured disturbances*** (D).

For our home heating system:
PV = process variable is house temperature
CO = controller output signal from thermostat to furnace valve
SP = set point is the desired temperature set on the thermostat by the home owner
D = heat loss disturbances from doors, walls and windows; changing outdoor temperature; sunrise and sunset; rain...



Copyright © 2008 by Douglas J. Cooper. All Rights Reserved

To achieve this control objective, the measured process variable is compared to the thermostat set point. The difference between the two is the ***controller error***, which is used in a ***control algorithm*** such as a PID (proportional-integral-derivative) controller to compute a CO signal to the final control element (FCE).

The change in the controller output (CO) signal causes a response in the final control element (fuel flow valve), which subsequently causes a change in the ***manipulated process variable*** (flow of fuel to the furnace). If the manipulated process variable is

moved in the right direction and by the right amount, the measured process variable will be maintained at set point, thus satisfying the control objective.

This example, like all in process control, involves a measurement, computation and action:
- is the measured temp colder than set point (SP – PV > 0)? Then open the valve.
- is the measured temp hotter than set point (SP – PV < 0)? Then close the valve.

Note that computing the necessary controller action is based on ***controller error***, or the difference between the set point and the measured process variable, i.e.

e(t) = SP – PV (error = set point – measured process variable)

In a home heating process, control is an on/off or open/close decision. And as outlined above, it is a straightforward decision to make. The price of such simplicity, however, is that the capability to tightly regulate our measured PV is rather limited.

One situation not addressed above is the action to take when PV = SP (i.e., e(t) = 0). And in industrial practice, we are concerned with ***variable position*** final control elements, so the challenge elevates to computing:
- the direction to move the valve, pump, compressor, heating element...
- how far to move it at this moment
- how long to wait before moving it again
- whether there should be a delay between measurement and action

This site
This site offers information and discussion on proven methods and practices for PID (proportional-integral-derivative) control and related architectures such as cascade, feed forward, Smith predictors, multivariable decoupling, and similar traditional and advanced classical strategies.

Applications focus on processes with streams comprised of gases, liquids, powders, slurries and melts. As stated above, final control elements for these applications tend to be valves, variable speed pumps and compressors, and cooling and heating elements.

Industries that operate such processes include chemical, bio-pharma, oil and gas, paints and coatings, food and beverages, cement and coal, polymers and plastics, metals and materials, pulp and paper, personal care products, and more.

The Components of a Control Loop

Components of a Control Loop
A controller seeks to maintain the measured process variable (PV) at set point (SP) in spite of unmeasured disturbances (D). The major components of a control system include a sensor, a controller and a final control element. To design and implement a controller, we must:

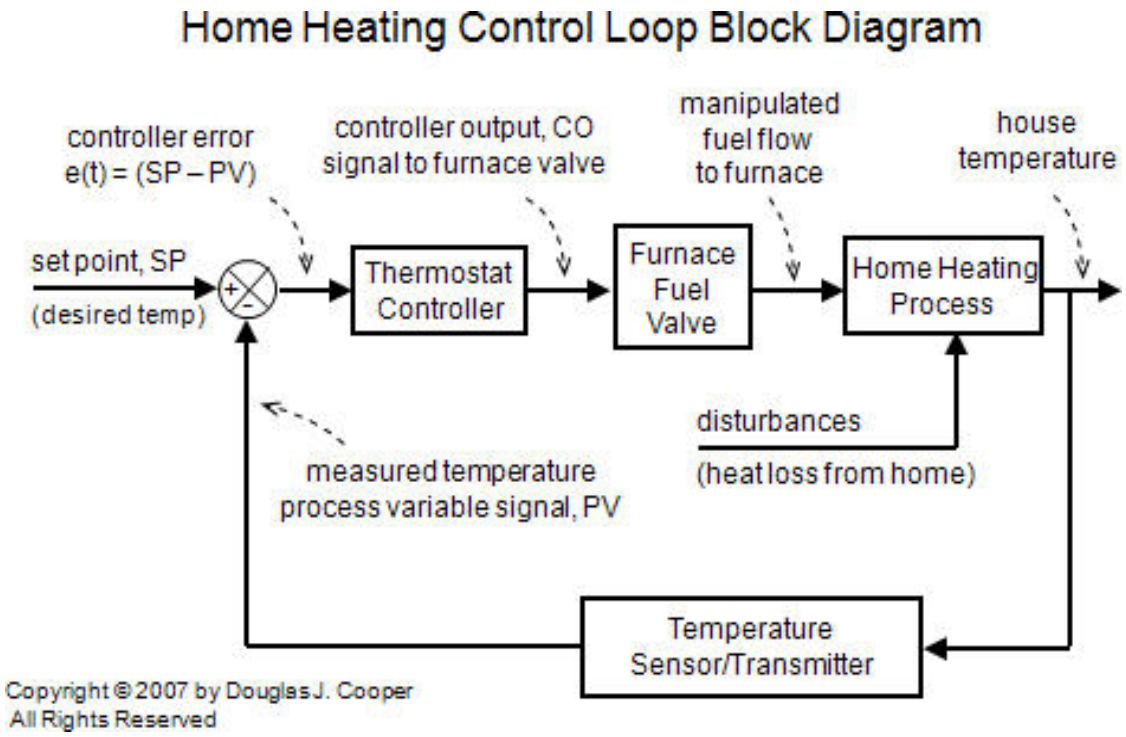
1) have identified a process variable we seek to regulate, be able to measure it (or something directly related to it) with a sensor, and be able to transmit that measurement as an electrical signal back to our controller, and
2) have a final control element (FCE) that can receive the controller output (CO) signal, react in some fashion to impact the process (e.g., a valve moves), and as a result

cause the process variable to respond in a consistent and predictable fashion.

Home Temperature Control

As shown below, the home heating control system described in this article can be organized as a traditional control loop block diagram. Block diagrams help us visualize the components of a loop and see how the pieces are connected.

A home heating system is simple on/off control with many of the components contained in a small box mounted on our wall. Nevertheless, we introduce the idea of control loop diagrams by presenting a home heating system in the same way we would a more sophisticated commercial control application.



Starting from the far right in the diagram above, our process variable of interest is house temperature. A sensor, such as a thermistor in a modern digital thermostat, measures temperature and transmits a signal to the controller.

The measured temperature PV signal is subtracted from set point to compute controller error, e(t) = SP – PV. The action of the controller is based on this error, e(t).

In our home heating system, the controller output (CO) signal is limited to open/close for the fuel flow solenoid valve (our FCE). So in this example, if e(t) = SP – PV > 0, the controller signals to open the valve. If e(t) = SP – PV < 0, it signals to close the valve. As an aside, note that there also must be a safety interlock to ensure that the furnace burner switches on and off as the fuel flow valve opens and closes.

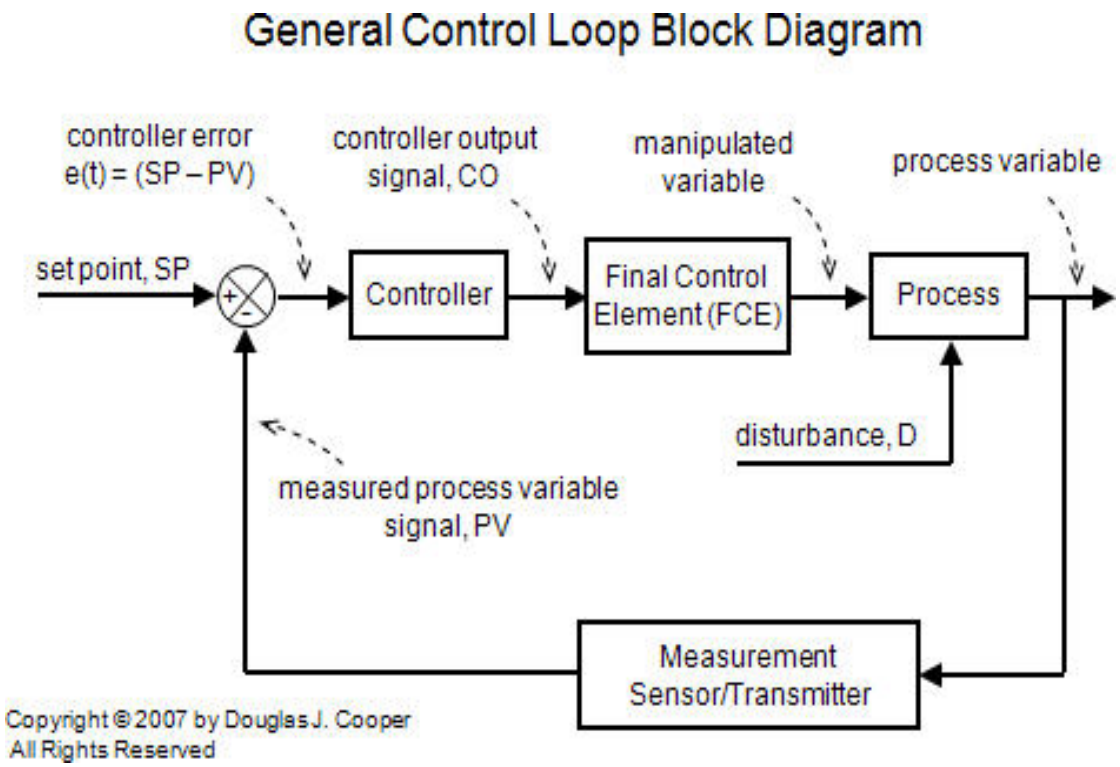
As the energy output of the furnace rises or falls, the temperature of our house increases or decreases and a feedback loop is complete. The important elements of a home heating control system can be organized like any commercial application:

- Control Objective: *maintain house temperature at SP in spite of disturbances*
- Process Variable: *house temperature*

- Measurement Sensor: *thermistor; or bimetallic strip coil on analog models*
- Measured Process Variable (PV) Signal: *signal transmitted from the thermistor*
- Set Point (SP): *desired house temperature*
- Controller Output (CO): *signal to fuel valve actuator and furnace burner*
- Final Control Element (FCE): *solenoid valve for fuel flow to furnace*
- Manipulated Variable: *fuel flow rate to furnace*
- Disturbances (D): *heat loss from doors, walls and windows; changing outdoor temperature; sunrise and sunset; rain...*

A General Control Loop and Intermediate Value Control

The home heating control loop above can be generalized into a block diagram pertinent to all feedback control loops as shown below.



Both diagrams above show a closed loop system based on negative feedback. That is, the controller takes actions that counteract or oppose any drift in the measured PV signal from set point.

While the home heating system is on/off, our focus going forward shifts to intermediate value control loops. An intermediate value controller can generate a full range of CO signals anywhere between full on/off or open/closed. The PI algorithm and PID algorithm are examples of popular intermediate value controllers.

To implement intermediate value control, we require a sensor that can measure a full range of our process variable, and a final control element that can receive and assume a full range of intermediate positions between full on/off or open/closed. This might include, for example, a process valve, variable speed pump or compressor, or heating or cooling element.

Note from the loop diagram that the process variable becomes our official PV only after it

has been measured by a sensor and transmitted as an electrical signal to the controller. In industrial applications. these are most often implemented as 4-20 milliamps signals, though commercial instruments are available that have been calibrated in a host of amperage and voltage units.

With the loop closed as shown in the diagrams, we are said to be in automatic mode and the controller is making all adjustments to the FCE. If we were to open the loop and switch to manual mode, then we would be able to issue CO commands through buttons or a keyboard directly to the FCE. Hence:

- open loop = manual mode
- closed loop = automatic mode

Cruise Control and Measuring Our PV

Cruise control in a car is a reasonably common intermediate value control system. For those who are unfamiliar with cruise control, here is how it works.

We first enable the control system with a button on the car instrument panel. Once on the open road and at our desired cruising speed, we press a second button that switches the controller from manual mode (where car speed is adjusted by our foot) to automatic mode (where car speed is adjusted by the controller).

The speed of the car at the moment we close the loop and switch from manual to automatic becomes the set point. The controller then continually computes and transmits corrective actions to the gas pedal (throttle) to maintain measured speed at set point.

It is often cheaper and easier to measure and control a variable directly related to the process variable of interest. This idea is central to control system design and maintenance. And this is why the loop diagrams above distinguish between our "process variable" and our "measured PV signal."

Cruise control serves to illustrate this idea. Actual car speed is challenging to measure. But transmission rotational speed can be measured reliably and inexpensively. The transmission connects the engine to the wheels, so as it spins faster or slower, the car speed directly increases or decreases.

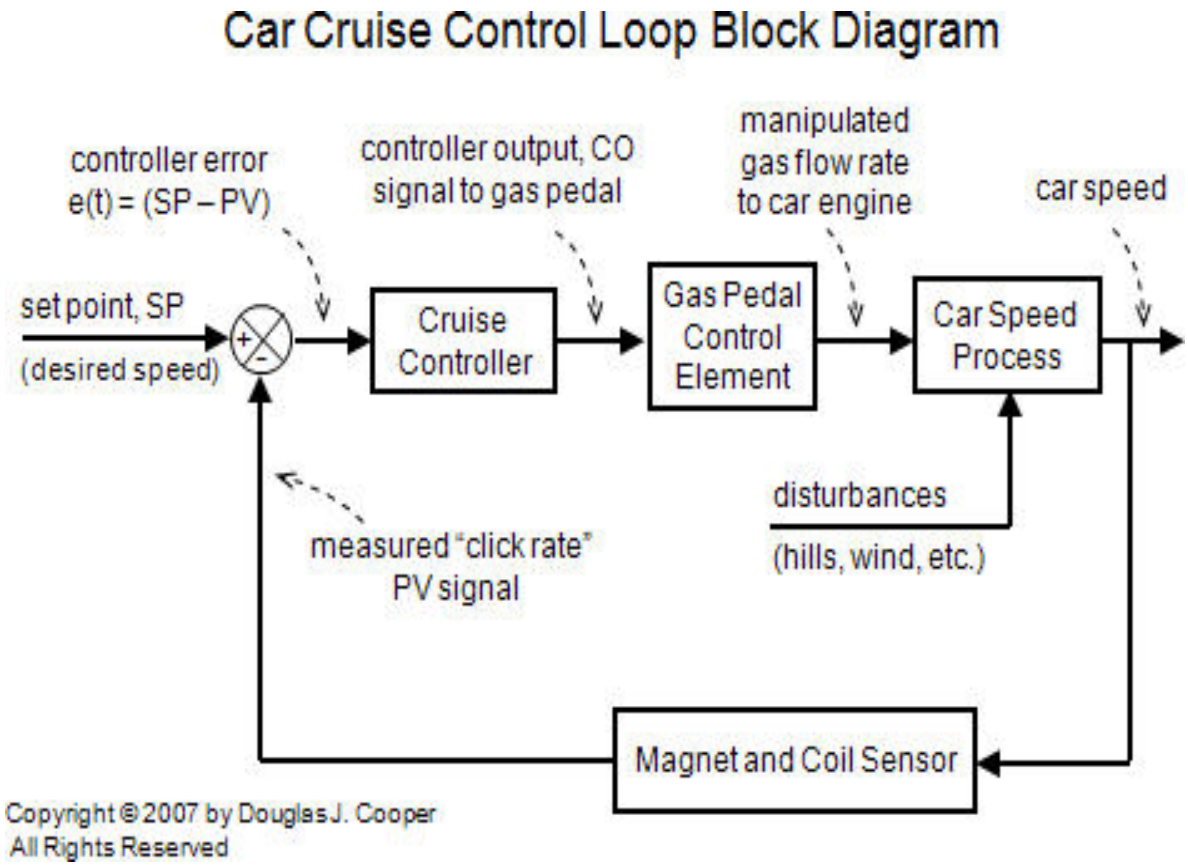
Thus, we attach a small magnet to the rotating output shaft of the car transmission and a magnetic field detector (loops of wire and a simple circuit) to the body of the car above the magnet. With each rotation, the magnet passes by the detector and the event is registered by the circuitry as a “click.” As the drive shaft spins faster or slower, the click rate and car speed increase or decrease proportionally.

So a cruise control system really adjusts fuel flow rate to maintain click rate at the set point value. With this knowledge, we can organize cruise control into the essential design elements:

- Control Objective: *maintain car speed at SP in spite of disturbances*
- Process Variable: *car speed*
- Measurement Sensor: *magnet and coil to clock drive shaft rotation*
- Measured Process Variable (PV) Signal: *"click rate" signal from the magnet and coil*
- Set Point (SP): *desired car speed, recast in the controller as a desired click rate*
- Controller Output (CO): *signal to actuator that adjusts gas pedal (throttle)*
- Final Control Element (FCE): *gas pedal position*

- Manipulated Variable: *fuel flow rate*
- Disturbances (D): *hills, wind, curves, passing trucks…*

The traditional block diagram for cruise control is thus:



Instruments Should be Fast, Cheap and Easy

The above magnet and coil "click rate = car speed" example introduces the idea that when purchasing an instrument for process control, there are wider considerations that can make a loop faster, easier and cheaper to implement and maintain. Here is a "best practice" checklist to use when considering an instrument purchase:

- Low cost
- Easy to install and wire
- Compatible with existing instrument interface
- Low maintenance
- Rugged and robust
- Reliable and long lasting
- Sufficiently accurate and precise
- Fast to respond (small time constant and dead time)
- Consistent with similar instrumentation already in the plant

Process Data, Dynamic Modeling and a Recipe for Profitable Control

It is best practice to follow a formal procedure or "recipe" when designing and tuning a PID (proportional-integral-derivative) controller. A recipe-based approach is the fastest method for moving a controller into operation. And perhaps most important, the performance of the controller will be superior to a controller tuned using a guess-and-test or trial-and-error method.

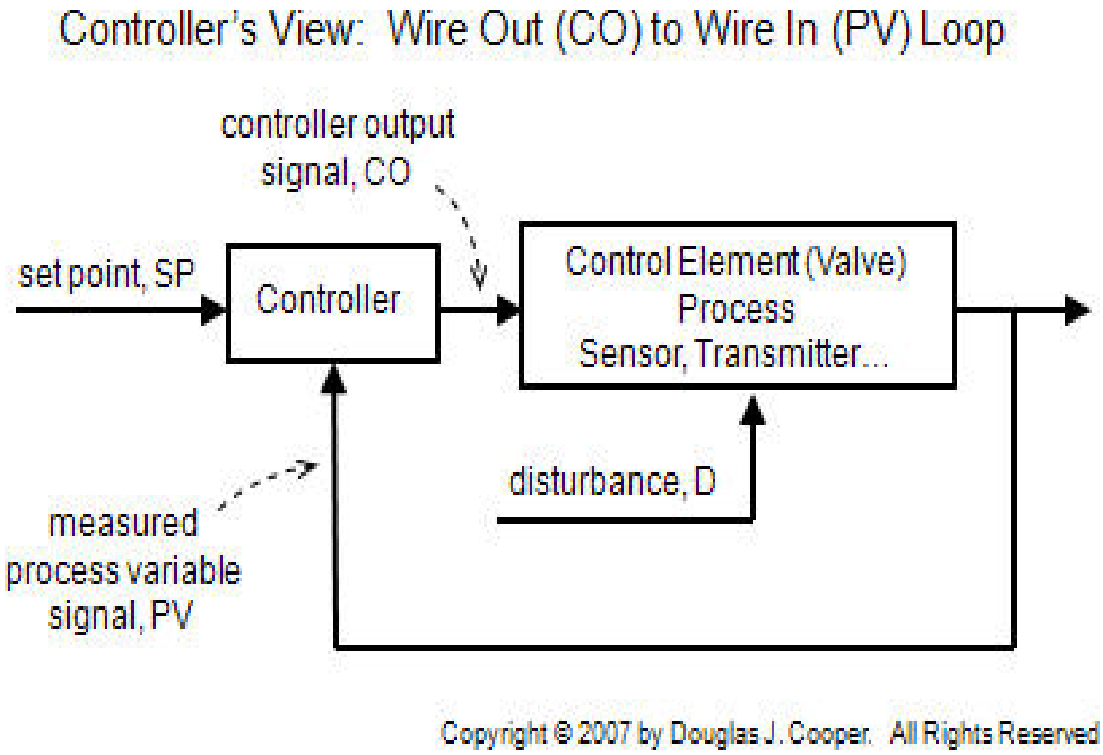
Additionally, a recipe-based approach overcomes many of the concerns that make control projects challenging in a commercial operating environment. Specifically, the recipe-based method causes less disruption to the production schedule, wastes less raw material and utilities, requires less personnel time, and generates less off-spec product.

The recipe for success is short:

1. Establish the design level of operation (DLO), defined as the expected values for set point and major disturbances during normal operation
2. Bump the process and collect controller output (CO) to process variable (PV) dynamic process data around this design level
3. Approximate the process data behavior with a first order plus dead time (FOPDT) dynamic model
4. Use the model parameters from step 3 in rules and correlations to complete the controller design and tuning.

We explore each step of this recipe in detail in other articles on this site. For now, we introduce some initial thoughts about steps 2 and 4.

Step 2: Bumping Our Process and Collecting CO to PV Data
From a controller's view, a complete control loop goes from wire out to wire in as shown below. Whenever we mention controller output (CO) or process variable (PV) data anywhere on this site, we are specifically referring to the data signals exiting and entering our controller at the wire termination interface.



To generate CO to PV data, we bump our process. That is, we step or pulse the CO (or the set point if in automatic mode as discussed here) and record PV data as the process responds. Here are three basic rules we follow in all of our examples:

• *Start with the process at steady state and record everything*
The point of bumping the CO is to learn about the cause and effect relationship between it and the PV. With the plant initially at steady state, we are starting with a clean slate. The dynamic behavior of the process is then clearly isolated as the PV responds. It is important that we start capturing data before we make the initial CO bump and then sample and record quickly as the PV responds.

• *Make sure the PV response dominates the process noise*
When performing a bump test, it is important that the CO moves far enough and fast enough to force a response that clearly dominates any noise or random error in the measured PV signal. If the CO to PV cause and effect response is clear enough to see by eye on a data plot, we can be confident that modern software can model it.

• *The disturbances should be quiet during the bump test*
We desire that the dynamic test data contain PV response data that has been clearly, and in the ideal world exclusively, forced by changes in the CO.

Data that has been corrupted by unmeasured disturbances is of little value for controller design and tuning. The model (see below) will then incorrectly describe the CO to PV cause and effect relationship. And as a result, the controller will not perform correctly. If we are concerned that a disturbance event has corrupted test data, it is conservative to rerun the test.

Step 4: Using Model Parameters For Design and Tuning
The final step of the recipe states that once we have obtained model parameters that approximate the dynamic behavior of our process, we can complete the design and tuning of our PID controller.

We look ahead at this last step because this is where the payoff of the recipe-based approach is clear. To establish the merit, we assume for now that we have determined the design level of operation for our process (step 1), we have collected a proper data set rich in dynamic process information around this design level (step 2), and we have approximated the behavior revealed in the process data with a first order plus dead time (FOPDT) dynamic model (step 3).

Thankfully, we do not need to know what a FOPDT model is or even what it looks like. But we do need to know about the three model parameters that result when we fit this approximating model to process data.

The FOPDT (first order plus dead time) model parameters, listed below, tell us important information about the measured process variable (PV) behavior whenever there is a change in the controller output (CO) signal:

- process gain, Kp (tells the direction and how far PV will travel)
- process time constant, *T*p (tells how fast PV moves after it begins its response)
- process dead time, Θp (tells how much delay before PV first begins to respond)

Aside: we do not need to understand differential equations to appreciate the articles on this site. But for those interested, we note that the first order plus dead time (FOPDT) dynamic model has the form:

$$Tp\frac{dPV(t)}{dt} + PV(t) = Kp \cdot CO(t - \theta p)$$

Where:
PV(t) = measured process variable as a function of time
CO(t – Θp) = controller output signal as a function of time and shifted by Θp
Θp = process dead time
t = time

The other variables are as listed above this box. It is a first order differential equation because it has one derivative with one time constant, *T*p. It is called a first order plus dead time equation because it also directly accounts for a delay or dead time, Θp, in the CO(t) to PV(t) behavior.

We study what these three model parameters are and how to compute them in other articles, but here is why process gain, Kp, process time constant, *T*p, and process dead time, Θp, are all important:

• *Tuning*
These three model parameters can be plugged into proven correlations to directly compute P-Only, PI, PID, and PID with CO Filter tuning values. No more trial and error.

No more tweaking our way to acceptable control. Great performance can be readily achieved with the step by step recipe listed above.

• *Controller Action*
Before implementing our controller, we must input the proper direction our controller should move to correct for growing errors. Some vendors use the term “reverse acting” and “direct acting.” Others use terms like “up-up” and “up-down” (as CO goes up, then PV goes up or down). This specification is determined solely by the sign of the process gain, Kp.

• *Loop Sample Time,* T
Process time constant, *T*p, is the clock of a process. The size of *T*p indicates the maximum desirable loop sample time. Best practice is to set loop sample time, T, at 10 times per time constant or faster (T ≤ 0.1*T*p). Sampling faster will not necessarily provide better performance, but it is a safer direction to move if we have any doubts. Sampling too slowly will have a negative impact on controller performance. Sampling slower than five times per time constant will lead to degraded performance.

• *Dead Time Problems*
As dead time grows larger than the process time constant (Θp > *T*p), the control loop can benefit greatly from a model based dead time compensator such as a Smith predictor. The only way we know if Θp > *T*p is if we have followed the recipe and computed the parameters of a FOPDT model.

• *Model Based Control*
If we choose to employ a Smith predictor, a dynamic feed forward element, a multivariable decoupler, or any other model based controller, we need a dynamic model of the process to enter into the control computer. The FOPDT model from step 2 of the recipe is often appropriate for this task.

Fundamental to Success
With tuning values, loop specifications, performance diagnostics and advanced control all dependent on knowledge of a dynamic model, we begin to see that process gain, Kp; process time constant, *T*p; and process dead time, Θp; are parameters of fundamental importance to success in process control.

Sample Time Impacts Controller Performance
There are two sample times, T, used in process controller design and tuning.

One is the control loop sample time that specifies how often the controller samples the measured process variable (PV) and then computes and transmits a new controller output (CO) signal.

The other is the rate at which CO and PV data are sampled and recorded during a bump test of our process. Bump test data is used to design and tune our controller prior to implementation.

In both cases, sampling too slow will have a negative impact on controller performance. Sampling faster will not necessarily provide better performance, but it is a safer direction

to move if we have any doubts.

Fast and slow are relative terms defined by the process time constant, *T*p. Best practice for both control loop sample time and bump test data collection are the same:

Best Practice:
Sample time should be 10 times per process time constant or faster (T ≤ 0.1*T*p).

We explore this "best practice" rule in a detailed study here. This study employs some fairly advanced concepts, so it is placed further down in the Table of Contents.

Yet perhaps we can gain an appreciation for how sample time impacts controller design and tuning with this thought experiment:

Suppose you see me standing on your left. You close your eyes for a time, open them, and now I am standing on your right. Do you know how long I have been at my new spot? Did I just arrive or have I been there for a while? What path did I take to get there? Did I move around in front or in back of you? Maybe I even jumped over you?

Now suppose your challenge is to keep your hands at your side until I pass by, and just as I do, you are to reach out and touch me. What are your chances with your eyes closed (and loud music is playing so you cannot hear me)?

Now lets say you are permitted to blink open your eyes briefly once per minute. Do you think you will have a better chance of touching me? How about blinking once every ten seconds? Clearly, as you start blinking say, two or three times a second, the task of touching me becomes easy. That's because you are sampling fast enough to see my "process" behavior fully and completely.

Based on this thought experiment, sampling too slow is problematic and sampling faster is generally better.

Keep in mind the "T ≤ 0.1*T*p" rule as we study PID control. This applies both to sampling during data collection, and the "measure and act" loop sample time when we implement our controller.

2) Graphical Modeling of Process Dynamics: Heat Exchanger Case Study

Step Test Data From the Heat Exchanger Process

A previous article presented the first order plus dead time (FOPDT) dynamic model and discussed how this model, when used to approximate the controller output (CO) to process variable (PV) behavior of proper data from our process, yields the all-important model parameters:

- process gain, Kp (*tells the direction and how far PV will travel*)
- process time constant, *T*p (*tells how fast PV moves after it begins its response*)
- process dead time, Θp (*tells how much delay before PV first begins to respond*)

The previous articles also mentioned that these FOPDT model parameters can be used to determine PID tuning values, proper sample time, whether the controller should be direct or reverse acting, whether dead time is large enough to cause concern, and more.

A Hands-On Study
There is an old saying (a Google search shows a host of attributed authors) that goes something like this: *I hear and I forget, I see and I remember, I do and I understand*.

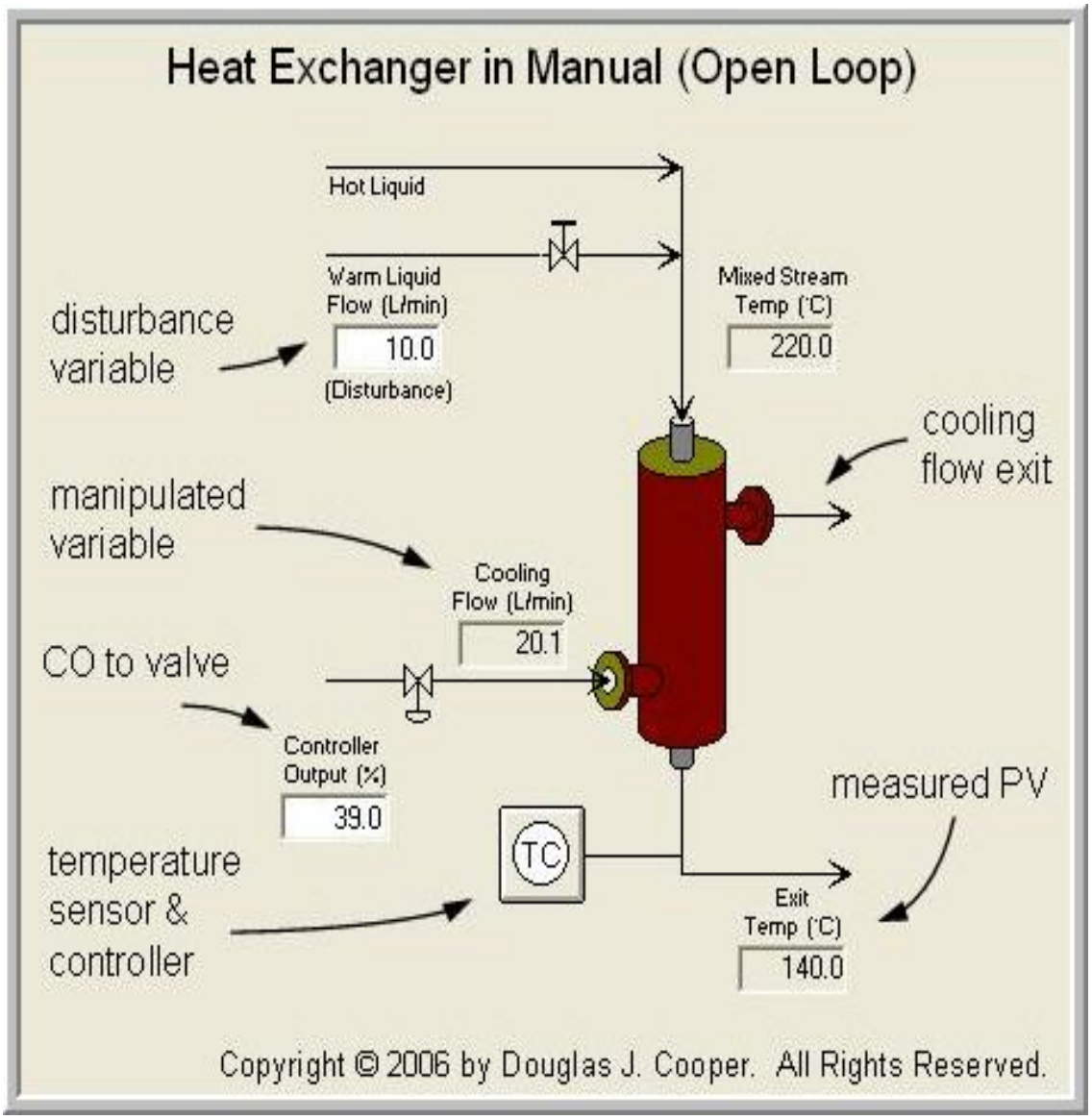
Since our goal is to understand, this means we must "do." To that end, we take a hands-on approach in this case study that will help us appreciate what each FOPDT model parameter is telling us about our process and empower us to act accordingly as we explore best practices for controller design and tuning.

To proceed, we require a process we can manipulate freely. We start with a heat exchanger because they are common to a great many industries.

Heat Exchanger Process
The heat exchanger we will study is really a process simulation from commercial software. The simulation is developed from first-principles theory, so its response behavior is realistic. The benefit of a simulation is that we can manipulate process variables whenever and however we desire without risk to people or profit.

The heat exchanger is shown below in manual mode (also called open loop). Its behavior is that of a counter-current, shell and tube, hot liquid cooler.

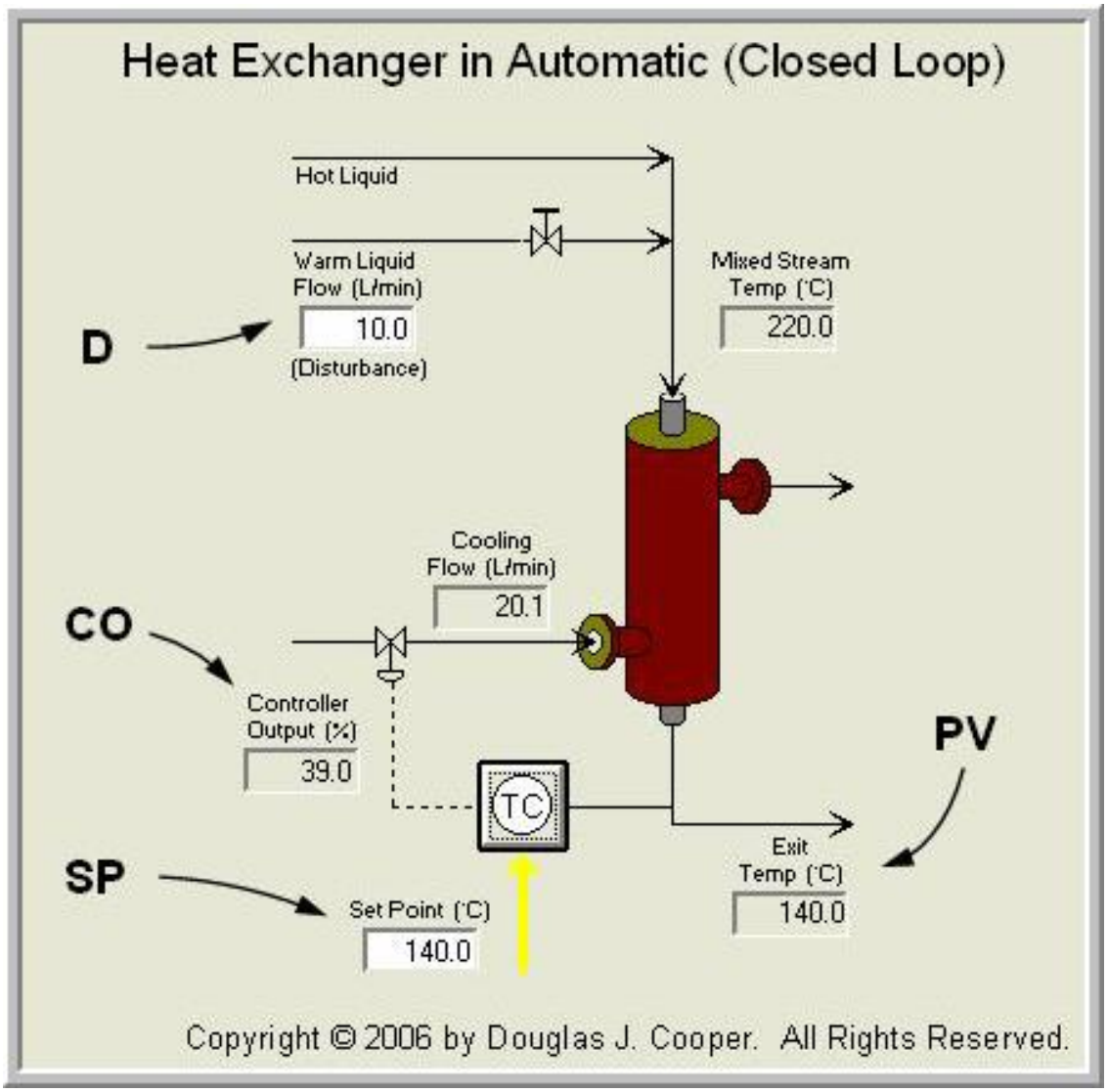
The measured process variable is the hot liquid temperature exiting the exchanger on the tube side. To regulate this hot exit temperature, the controller moves a valve to manipulate the flow rate of a cooling liquid entering on the shell side.



The hot tube side and cool shell side liquids do not mix. Rather, the cooling liquid surrounds the hot tubes and pulls off heat energy as it passes through the exchanger. As the flow rate of cooling liquid around the tubes increases (as the valve opens), more heat is removed and the temperature of the exiting hot liquid decreases.

A side stream of warm liquid combines with the hot liquid entering the exchanger and acts as a disturbance to our process in this case study. As the warm stream flow rate increases, the mixed stream temperature decreases (and vice versa).

Shown below is the heat exchanger in automatic mode (also called closed loop) using the standard nomenclature for this site.



The measured process variable (PV) is the hot liquid temperature exiting the exchanger. The controller output (CO) signal moves a valve to manipulate the flow rate of cooling liquid on the shell side to maintain the PV at set point (SP). The warm liquid flow acts as a disturbance (D) to the process.

Generating Step Test Data

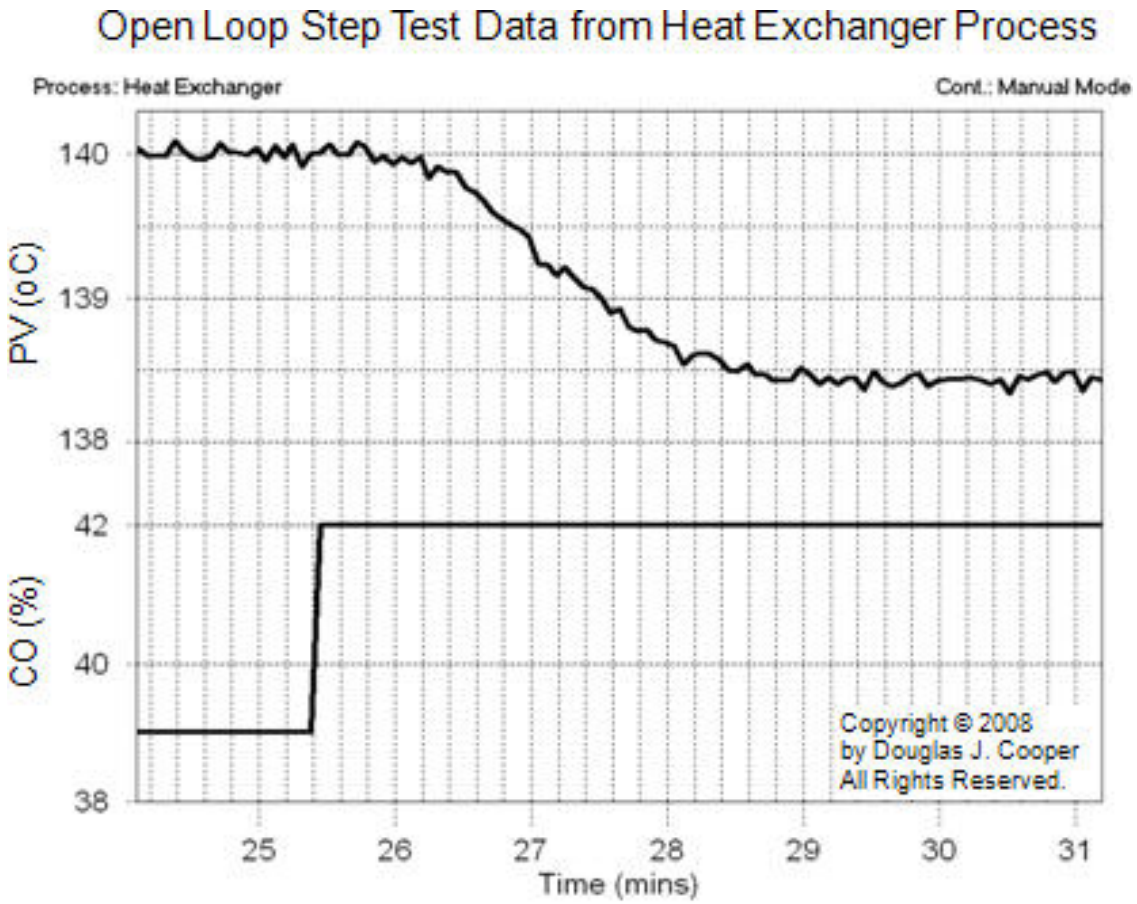
To fit a FOPDT (first order plus dead time) model to dynamic process data using hand calculations, we will be reading numbers off of a plot. Such a graphical analysis technique can only be performed on step test data collected in manual mode (open loop).

Practitioner's Note: operations personnel can find switching to manual mode and performing step tests to be unacceptably disruptive, especially when the production schedule is tight. It is sometimes easier to convince them to perform a closed loop (automatic mode) pulse test, but such data must be analyzed by software and this reduces our "doing" to simply "seeing" software provide answers.

To generate our dynamic process step test data, wait until the CO and PV appear to be as steady as is reasonable for the process under study. Then, after confirming we are in manual mode, step the CO to a new value.

The CO step must be large enough and sudden enough to cause the PV to move in a clear response that dominates all noise in the measurement signal. Data collection must begin before the CO step is implemented and continue until the PV reaches a new steady state.

The plot below shows dynamic step test data from the heat exchanger. Note that the PV signal (the upper trace in the plot) includes a small amount of random measurement noise. This is added in the simulation to create a more realistic process behavior.



As shown in the plot, the CO is initially constant at 39% while the exit temperature PV is steady at about 140 °C. The CO is then stepped from 39% up to 42%.

The step increase in CO causes the valve to open, increasing the flow rate of cooling liquid into the shell side of the exchanger. The additional cooling liquid causes the measured PV (exit temperature on the tube side) to decrease from its initial steady state value of 140 °C down to a new value of about 138.4 °C.

We will refer back to the this dynamic process test data in future articles as we work through the details of computing process gain, Kp; process time constant, *T*p; and process dead time, Θp.

Practitioner's Note: the heat exchanger graphic shows that this process has one disturbance variable, D. It is a side stream of warm liquid that mixes with the hot liquid on the tube side. When generating the step test data above, disturbance D is held constant.

Yet real processes can have many disturbances, and by their very nature, disturbances

are often beyond our ability to monitor, let alone control. While quiet disturbances are something we can guarantee in a simulation, we may not be so lucky in the plant. Yet to accurately model the dynamics of a process, it is essential that the influential disturbances remain quiet when generating dynamic process test data.

Whether the disturbances have remained quiet during a dynamic test is something you must “know” about your process. Otherwise, you should not be adjusting any controller settings.

To appreciate this sentiment, recognize that you “know” when your car is acting up. You can sense when it shows a slight but clearly different behavior that needs attention. Someone planning to adjust the controllers in an industrial operation should have this same level of familiarity with their process.

Process Gain Is The "How Far" Variable

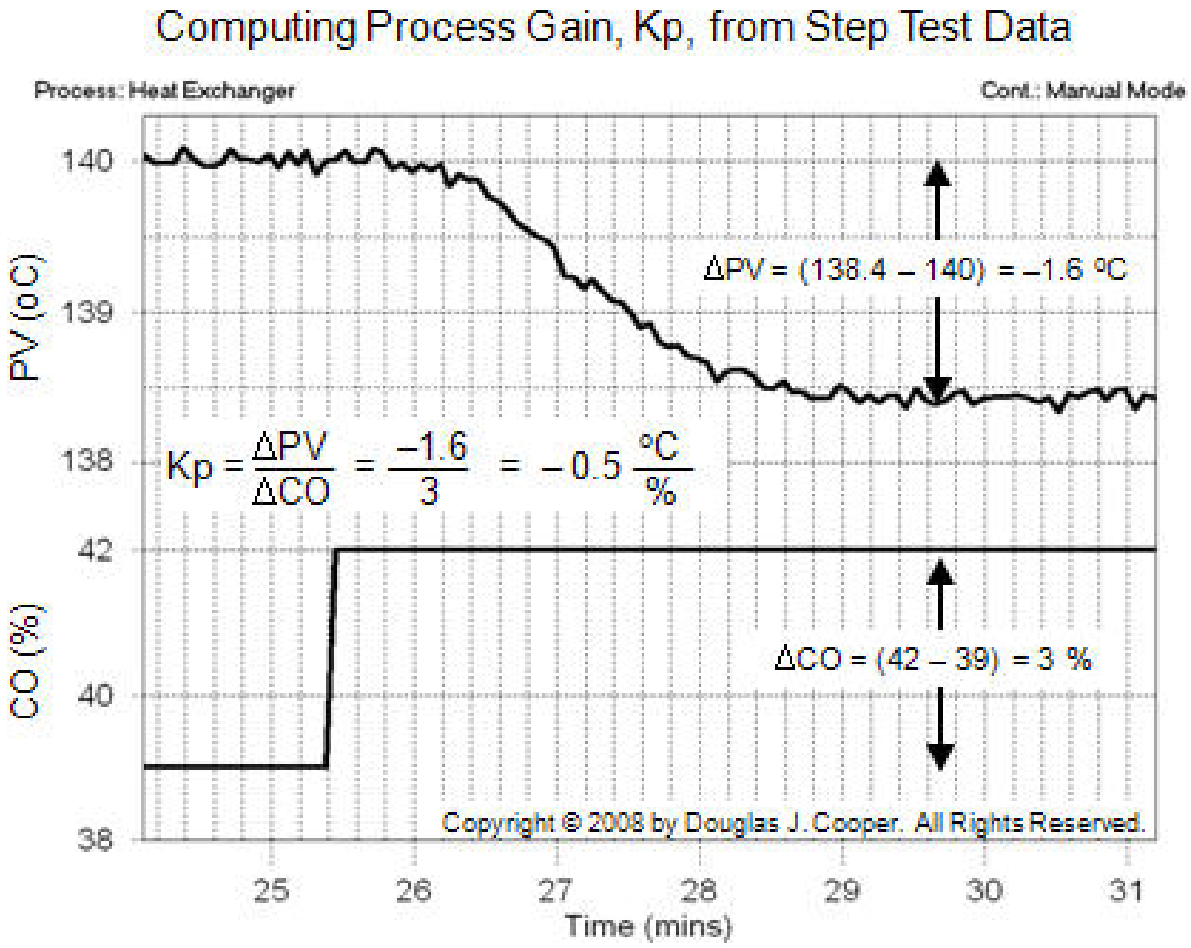
Step 3 of our controller design and tuning recipe is to approximate the often complex behavior contained in our dynamic process test data with a simple first order plus dead time (FOPDT) dynamic model.

In this article we focus on process gain, Kp, and seek to understand what it is, how it is computed, and what it implies for controller design and tuning. Corresponding articles present details of the other two FOPDT model parameters: process time constant, *T*p; and process dead time, Θp.

Heat Exchanger Step Test Data

We explore Kp by analyzing step test data from a heat exchanger. The heat exchanger is a realistic simulation where the measured process variable (PV) is the temperature of hot liquid exiting the exchanger. To regulate this PV, the controller output (CO) signal moves a valve to manipulate the flow rate of a cooling liquid into the exchanger.

The step test data below was generated by moving the process from one steady state to another. As shown, the CO was stepped from 39% up to 42%, causing the measured PV to decrease from 140 °C down to approximately 138.4 °C.



Computing Process Gain
Kp describes the direction PV moves and how far it travels in response to a change in CO. It is based on the difference in steady state values. The path or length of time the PV takes to get to its new steady state does not enter into the Kp calculation.

Thus, Kp is computed:

$$Kp = \frac{\text{Steady State Change in Measured Process Variable, } \Delta PV}{\text{Steady State Change in Controller Output, } \Delta CO}$$

where ΔPV and ΔCO represent the total change from initial to final steady state.

Aside: the assumptions implicit in the discussion above include that:
- the process is properly instrumented as a CO to PV pair,
- major disturbances remained reasonably quiet during the test, and
- the process itself is self regulating. That is, it naturally seeks to run at a steady state if left uncontrolled and disturbances remain quiet. Most, but certainly not all, processes are self regulating. Certain configurations of something as simple as liquid level in a pumped tank can be non-self regulating.

Reading numbers off the above plot:

- The CO was stepped from 39% up to 42%, so the ΔCO = 3%.

- The PV was initially steady at 140 °C and moved down to a new steady state value of 138.4 °C. Since it decreased, the ΔPV = –1.6 °C.

Using these ΔCO and ΔPV values in the Kp equation above, the process gain for the heat exchanger is computed:

$$Kp = \frac{\Delta PV}{\Delta CO} = \frac{138.4 - 140°C}{42 - 39\%} = -0.53 \frac{°C}{\%}$$

Practitioner's Note: Real plant data is rarely as clean as that shown in the plot above and we should be cautious not to try and extract more information from our data than it actually contains. When used in tuning correlations, rounding the Kp value to –0.5 °C/% will provide virtually the same performance.

Kp Impacts Control

Process gain, Kp, is the "how far" variable because it describes how far the PV will travel for a given change in CO. It is sometimes called the sensitivity of the process.

If a process has a large Kp, then a small change in the CO will cause the PV to move a large amount. If a process has a small Kp, the same CO change will move the PV a small amount.

As a thought experiment, let's suppose a disturbance moves our measured PV away from set point (SP). If the process has a large Kp, then the PV is very sensitive to CO changes and the controller should make small CO moves to correct the error. Conversely, if the process has a small Kp, then the controller needs to make large CO actions to correct the same error.

This is the same as saying that a process with a large process gain, Kp, should have a controller with a small controller gain, Kc (and vice versa).

Looking ahead to the PI tuning correlations we will use in our case studies:

$$Kc = \frac{1}{Kp} \frac{Tp}{(\theta p + Tc)} \quad \text{and} \quad Ti = Tp$$

Where:

Kc = controller gain, a tuning parameter
*T*i = reset time, a tuning parameter

Notice that in the Kc correlation, a large Kp in the denominator will yield a small Kc value (that is, Kc is inversely proportional to Kp). Thus, the tuning correlation tells us the same thing as our thought experiment above.

Sign of Kp Tells Direction

The sign of Kp tells us the direction the PV moves relative to the CO change. The negative value found above means that as the CO goes up, the PV goes down. We see this "up-down" relationship in the plot. For a process where a CO increase causes the PV to move up, the Kp would be positive and this would be an "up-up" process.

When implementing a controller, we need to know if our process is up-up or up-down. If we tell the controller the wrong relationship between CO actions and the direction of the PV responses, our mistake may prove costly. Rather than correcting for errors, the controller will quickly amplify them as it drives the CO signal, and thus the valve, pump or other final control element (FCE), to the maximum or minimum value.

Units of Kp

If we are computing Kp and want the results to be meaningful for control, then we must be analyzing wire out to wire in CO to PV data as used by the controller.

The heat exchanger data plot indicates that the data arriving on the PV wire into the controller has been scaled (or is being scaled in the controller) into units of temperature. And this means the the controller gain, Kc, needs to reflect the units of temperature as well.

This may be confusing, at least initially, since most commercial controllers do not require that units be entered. The good news is that, as long as our computations use the same "wire out to wire in" data as collected and displayed by our controller, the units will be consistent and we need not dwell on this issue.

> Aside: for the Kc tuning correlation above, the units of time (dead time and time constants) cancel out. Hence, the controller gain, Kc, will have the reciprocal or inverse units of Kp. For the heat exchanger, this means Kc has units of %/°C.
>
> With modern computer control systems, scaling for unit conversions is becoming more common in the controller signal path. Sometimes a display has been scaled but the signal in the loop path has not. You must pay attention to this detail and make sure you are using the correct units in your computations. There is more discussion in this article.
>
> It has been suggested that the gain of some controllers do not have units since both the CO and PV are in units of %. Actually, the Kc will have units of "% of CO signal" divided by "% of PV signal," which mathematically do not cancel out.

Practitioner's Note: Step test data is practical in the sense that all model fitting computations can be performed by reading numbers off of a plot. However, when dealing with production processes, operations personnel tend to prefer quick "bumps" rather than complete step tests.

Step tests move the plant from one steady state to another. This takes a long time, may have safety implications, and can create expensive off-spec product. Pulse and doublet tests are examples of quick bumps that returns our plant to desired operating conditions as soon as the process data shows a clear response to a controller output (CO) signal change.

Getting our plant back to a safe, profitable operation as quickly as possible is a popular concept at all levels of operation and management. Using pulse tests requires the use of inexpensive commercial software to analyze the bump test results, however.

Process Time Constant is The "How Fast" Variable

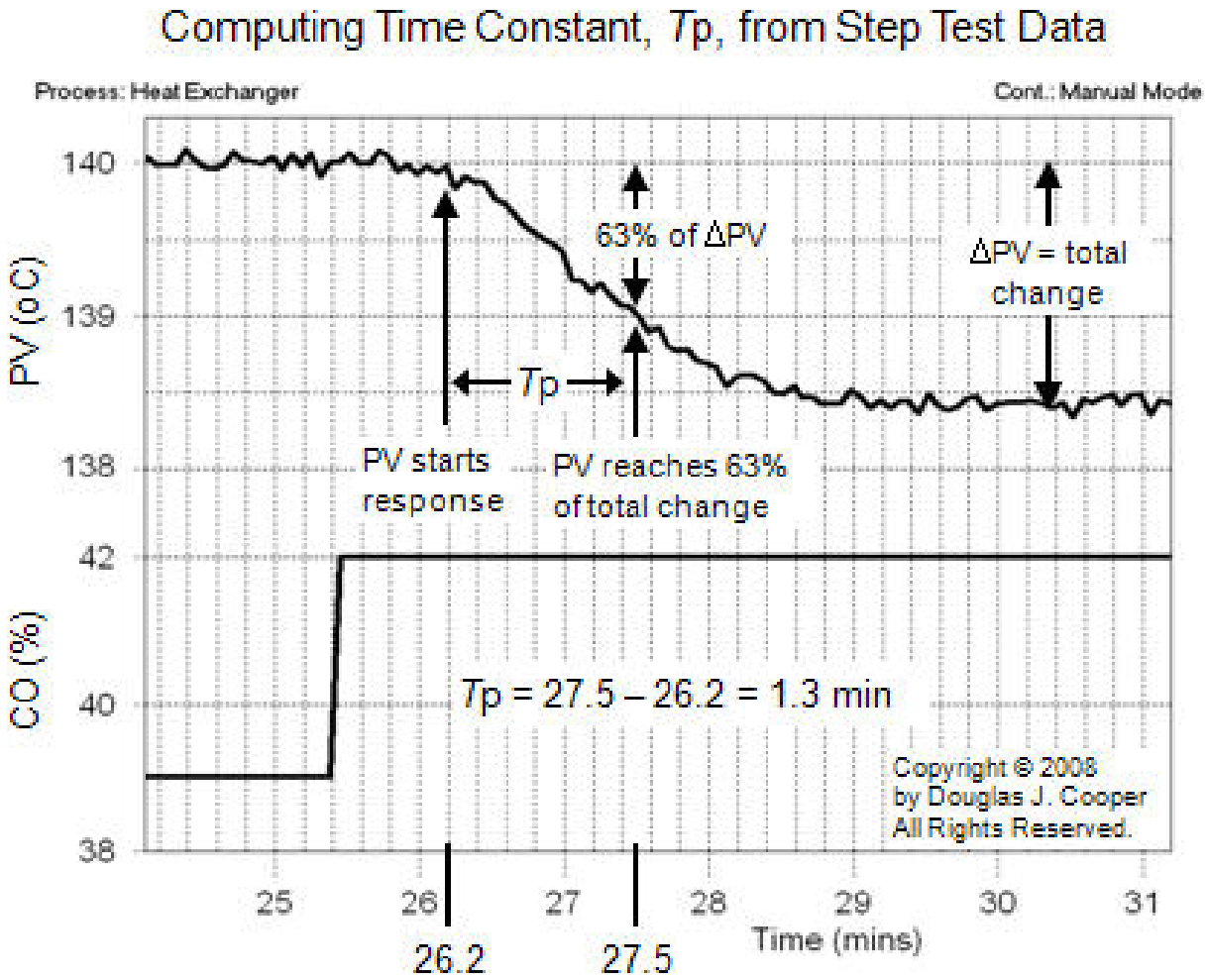
Step 3 of our controller design and tuning recipe is to approximate the often complex behavior contained in our dynamic process test data with a simple first order plus dead time (FOPDT) dynamic model.

In this article we focus on process time constant, *T*p, and seek to understand what it is, how it is computed, and what it implies for controller design and tuning. Corresponding articles present details of the other two FOPDT model parameters: process gain, Kp; and process dead time, Θp.

Heat Exchanger Step Test Data

We seek to understand *T*p by analyzing step test data from a heat exchanger. The heat exchanger is a realistic simulation where the measured process variable (PV) is the temperature of hot liquid exiting the exchanger. To regulate this PV, the controller output (CO) moves a valve to manipulate the flow rate of a cooling liquid into the exchanger.

The step test data below was generated by moving the process from one steady state to another. As shown, the CO was stepped from 39% up to 42%, causing the measured PV to decrease from 140 °C down to approximately 138.4 °C.



Time Constant in Words

In general terms, the time constant, *T*p, describes how fast the PV moves in response to a change in the CO.

The time constant must be positive and it must have units of time. For controllers used on processes comprised of gases, liquids, powders, slurries and melts, *T*p most often has units of minutes or seconds.

We can be more precise in our word definition if we restrict ourselves to step test data such as that shown in the plot above. Please recognize that while it is easier to describe *T*p in words using step test data, it is a parameter that always describes "how fast" PV moves in response to any sort of CO change.

Step test data implies that the process is in manual mode (open loop) and initially at steady state. A step in the CO has forced a response in the PV, which moves from its original steady state value to a final steady state.

With these restrictions, we compute Tp in five steps:

1. Determine ΔPV, the total change that is going to occur in PV, computed as "final minus initial steady state"

2. Compute the value of the PV that is 63% of the total change that is going to occur, or "initial steady state PV + 0.63(ΔPV)"

3. Note the time when the PV passes through the 63% point of "initial steady state PV + 0.63(ΔPV)"

4. Subtract from it the time when the "PV starts a clear response" to the step change in the CO

5. The passage of time from step 4 minus step 3 is the process time constant, *T*p.

Summarizing in one sentence, for step test data, *T*p is the time that passes from when the PV shows its first response to the CO step, until when the PV reaches 63% of the total □PV change that is going to occur.

Computing *T*p for the Heat Exchanger

Following the steps above for the heat exchanger step test data:

1. The PV was initially steady at 140 °C and moved down to a final steady state of 138.4 °C. The total change, ΔPV, is "final minus initial steady state" or:

ΔPV = 138.4 – 140 = –1.6 °C

2. The value of the PV that is 63% of this total change is "initial steady state PV + 0.63ΔPV" or:

initial PV + 0.63(ΔPV) = 140 + 0.63(–1.6)
= 140 – 1.0
= 139 °C

3. From the plot, the time when the PV passes through the "initial steady state PV + 0.63ΔPV" point of 139 °C is:

Time to 0.63(ΔPV) = Time to 139 °C
= 27.5 min

4. From the plot (see the following Practitioner's Note), the time when the "PV starts a response" to the CO step is:

Time PV response starts = 26.2 min

5. The time constant is “time to 63%(ΔPV)” minus “time PV response starts” or:
***T*p** = 27.5 – 26.2 **= 1.3 min**

Practitioner’s Note: the time when PV shows its “first clear response to the CO step” is a judgment call. Reasonable values could range from, say, 26 min to 26.4 min. Some thoughts:

▪ Dead time, Θp, is the time from when the CO step is made until the time when the “PV starts a clear response.” If your judgment says that the “clear response” is sooner in time (say 26.0 min for this case), then the value of *T*p increases, but dead time, Θp, then decreases by the same amount (and vice versa).

▪ It is generally conservative in controller design and tuning to overestimate Θp. Based on the previous statement, this implies that it is safer to pick a “clear response” that is further along in time (say 26.4 min for this case). Please understand the being more conservative is simply an attitude, it is not more right or wrong.

▪ Since *T*p and Θp are used in tuning correlations and design rules, your judgment will have some impact on controller performance.

▪ Reading data plots by eye is better than blind trial and error tuning. But an even better approach is to use a software tool for model fitting. This makes the procedure a quick and repeatable calculation, it eliminates personal judgment calls from the plot analysis, and consequently, different engineers will get the same answers from the same data set. The benefit is a consistent and predictable controller design and tuning result.

Aside: the “*T*p = 63% of response” rule is derived by solving the linear first order plus dead time (FOPDT) dynamic model that we use to approximate dynamic process behavior.

While no real process is exactly described by this FOPDT model, the form reasonably describes the direction, how far, how fast, and with how much delay the PV will respond when forced by a change in CO. And this provides a sufficient basis for controller design and tuning that has had proven success in industrial practice time and again.

A detailed derivation of the 63% rule is provided in this pdf.

Dead Time Is The "How Much Delay" Variable

Step 3 of our controller design and tuning recipe is to approximate the often complex behavior contained in our dynamic process test data with a simple first order plus dead time (FOPDT) dynamic model.

In this article we focus on process dead time, Θp, and seek to understand what it is, how it is computed, and what it implies for controller design and tuning. Corresponding articles present details of the other two FOPDT model parameters: process gain, Kp; and process time constant, *T*p.

Dead Time is the Killer of Control

Dead time is the delay from when a controller output (CO) signal is issued until when the

measured process variable (PV) first begins to respond. The presence of dead time, Θp, is never a good thing in a control loop.

Think about driving your car with a dead time between the steering wheel and the tires. Every time you turn the steering wheel, the tires do not respond for, say, two seconds. Yikes.

For any process, as Θp becomes larger, the control challenge becomes greater and tight performance becomes more difficult to achieve. “Large” is a relative term and this is discussed later in this article.

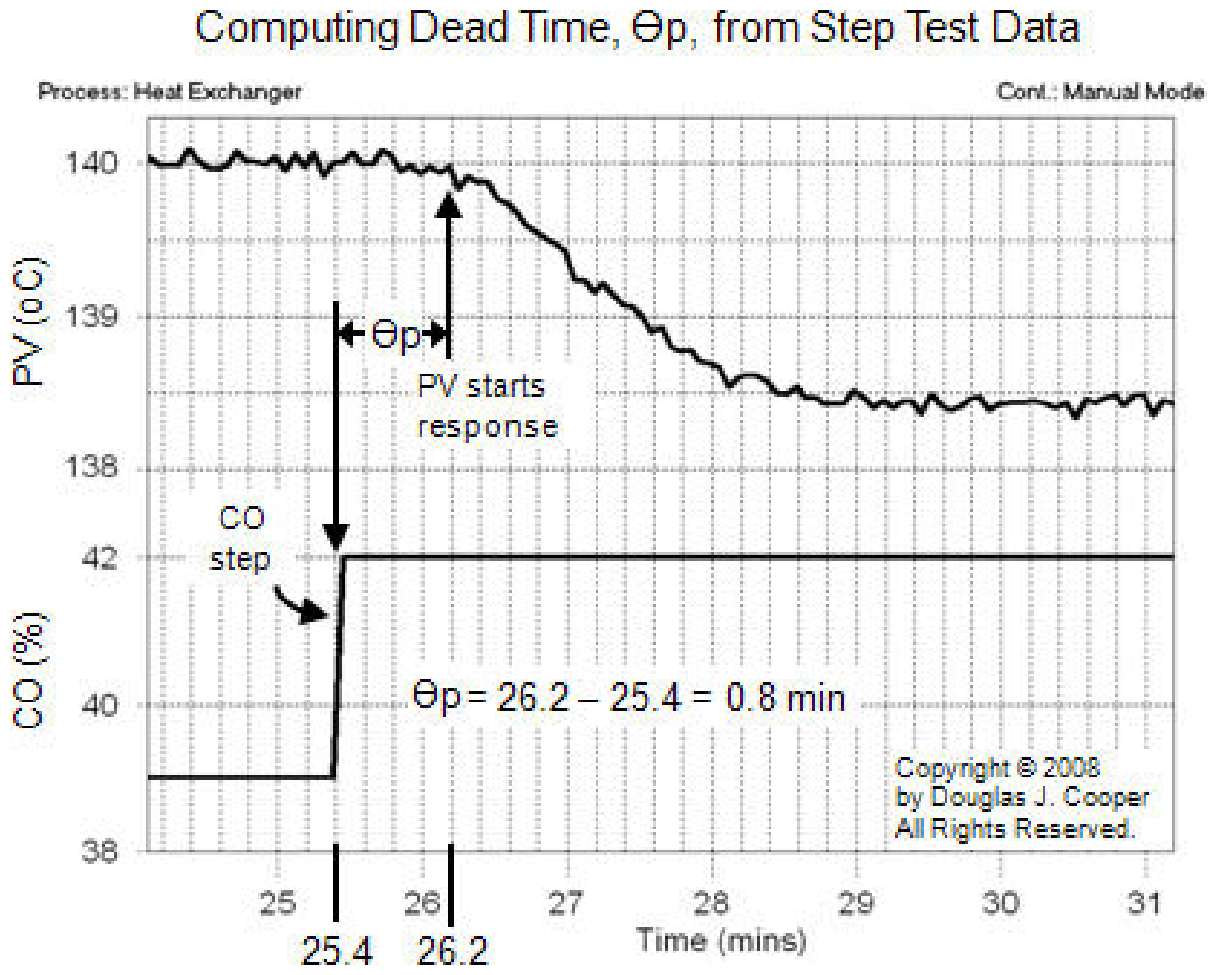
Causes for Dead Time
Dead time can arise in a control loop for a number of reasons:

- Control loops typically have "sample and hold" measurement instrumentation that introduces a minimum dead time of one sample time, T, into every loop. This is rarely an issue for tuning, but indicates that every loop has at least some dead time.
- The time it takes for material to travel from one point to another can add dead time to a loop. If a property (e.g. a concentration or temperature) is changed at one end of a pipe and the sensor is located at the other end, the change will not be detected until the material has moved down the length of the pipe. The travel time is dead time. This is not a problem that occurs only in big plants with long pipes. A bench top process can have fluid creeping along a tube. The distance may only be an arm's length, but a low enough flow velocity can translate into a meaningful delay.
- Sensors and analyzer can take precious time to yield their measurement results. For example, suppose a thermocouple is heavily shielded so it can survive in a harsh environment. The mass of the shield can add troublesome delay to the detection of temperature changes in the fluid being measured.
- Higher order processes have an inflection point that can be reasonably approximated as dead time for the purpose of controller design and tuning. Note that modeling for tuning with the simple FOPDT form is different from modeling for simulation, where process complexities should be addressed with more sophisticated model forms (all subjects for future articles).

Sometimes dead time issues can be addressed through a simple design change. It might be possible to locate a sensor closer to the action, or perhaps switch to a faster responding device. Other times, the dead time is a permanent feature of the control loop and can only be addressed through detuning or implementation of a dead time compensator (e.g. Smith predictor).

Heat Exchanger Test Data
We seek to understand Kp by analyzing step test data from a heat exchanger. The heat exchanger is a realistic simulation where the measured process variable (PV) is the temperature of hot liquid exiting the exchanger. To regulate this PV, the controller output (CO) moves a valve to manipulate the flow rate of a cooling liquid into the exchanger.

The step test data below was generated by moving the process from one steady state to another. In particular, CO was stepped from 39% up to 42%, causing the measured PV to decrease from 140 °C down to approximately 138.4 °C.



Computing Dead Time

Estimating dead time, Θp, from step test data is a three step procedure:

1. Locate the point in time when the "PV starts a clear response" to the step change in the CO. This is the same point we identified when we computed *T*p in the previous article.

2. Locate the point in time when the CO was stepped from its original value to its new value.

3. Dead time, Θp, is the difference in time of step 1 minus step 2.

Applying the three step procedure to the step test plot above:

1. As we had determined in the previous *T*p article, the PV starts a clear response to the CO step at 26.2 min.
2. Reading off the plot, the CO step occurred at 25.4 min, and thus,
3. **Θp** = 26.2 – 25.4 = **0.8 min**

We analyze step test data here to make the computation straightforward, but please recognize that dead time describes "how much delay" occurs from when any sort of CO change is made until when the PV first responds to that change.

Like a time constant, dead time has units of time and must always be positive. For the types of processes explored on this site (streams comprised of gasses, liquids, powders, slurries and melts), dead time is most often expressed in minutes or seconds.

During a dynamic analysis study, it is best practice to express *T*p and Θp in the same units (e.g. both in minutes or both in seconds). The tuning correlations and design rules

assume consistent units. Control is challenging enough without adding computational error to our problems.

Implications for Control

• Dead time, Θp, is large or small only in comparison to *T*p, the clock of the process. Tight control becomes more challenging when Θp > *T*p. As dead time becomes much greater than *T*p, a dead time compensator such as a Smith predictor offers benefit. A Smith predictor employs a dynamic process model (such as an FOPDT model) directly within the architecture of the controller. It requires additional engineering time to design, implement and maintain, so be sure the loop is important to safety or profitability before undertaking such a project.

• It is more conservative to overestimate dead time when the goal is tuning. Computing Θp requires a judgment of when the “PV starts a clear response.” If your judgment says that the “clear response” is sooner in time (maybe you choose 26.0 min for this case), then *T*p increases, but dead time, Θp, decreases by the same amount (and vice versa). We can see the impact this has by looking ahead to the PI tuning correlations:

$$Kc = \frac{1}{Kp}\frac{Tp}{(\theta p + Tc)} \quad \text{and} \quad Ti = Tp$$

Where:

Kc = controller gain, a tuning parameter

*T*i = reset time, a tuning parameter

Since Θp is in the denominator of the Kc correlation, as dead time gets larger, the controller gain gets smaller. A smaller Kc implies a less active controller. Overly aggressive controllers cause more trouble than sluggish controllers, at least in the first moments after being put into automatic. Hence, a larger dead time estimate is a more cautious or conservative estimate.

Practitioner's Note on the "Θp,min = T" Rule for Controller Tuning:

Consider that all controllers measure, act, then wait until next sample time; measure, act, then wait until next sample time. This "measure, act, wait" procedure has a delay (or dead time) of one sample time, T, built naturally into its structure.

Thus, the minimum dead time, Θp, in any real control implementation is the loop sample time, T. Dead time can certainly be larger than T (and it usually is), but it cannot be smaller.

Thus, if our model fit yields Θp < T (a dead time that is less than the controller sample time), we must recognize that this is an impossible outcome. Best practice in such a situation is to substitute Θp = T everywhere when using our controller tuning correlations and other design rules.

This is the "Θp,min = T" rule for controller tuning. Read more details about the impact of sample time on controller design and tuning in this article.

Validating Our Heat Exchanger FOPDT Model

Over a series of articles, we generated step test data from a heat exchanger process simulation and then explored details of how to perform a graphical analysis of the plot data to compute values for a first order plus dead time (FOPDT) dynamic model.

The claim has been that the parameters from this FOPDT model will offer us a good approximation of the complex dynamic behavior of our process. And this, in turn, will provide us information critical for the design and tuning of a PID controller.

So appropriate questions at this point might be:

- How well does our FOPDT model fit the heat exchanger data?
- How does this help us design and tune our PID controller?

Comparing Graphical Model to Data

Here are the FOPDT model parameter estimates we computed from the graphical analysis of the step test data (each variable provides a link back to the article with details):

- Process gain (how far), Kp = – 0.53 °C/%
- Time constant (how fast), *T*p = 1.3 min
- Dead time (how much delay), Θp = 0.8 min

Aside: the FOPDT dynamic model has the general controller output (CO) to process variable (PV) form:

$$Tp\frac{dPV(t)}{dt} + PV(t) = Kp \cdot CO(t - \theta p)$$

And this means we are claiming that the dynamic behavior of the heat exchanger can be reasonably approximated as:

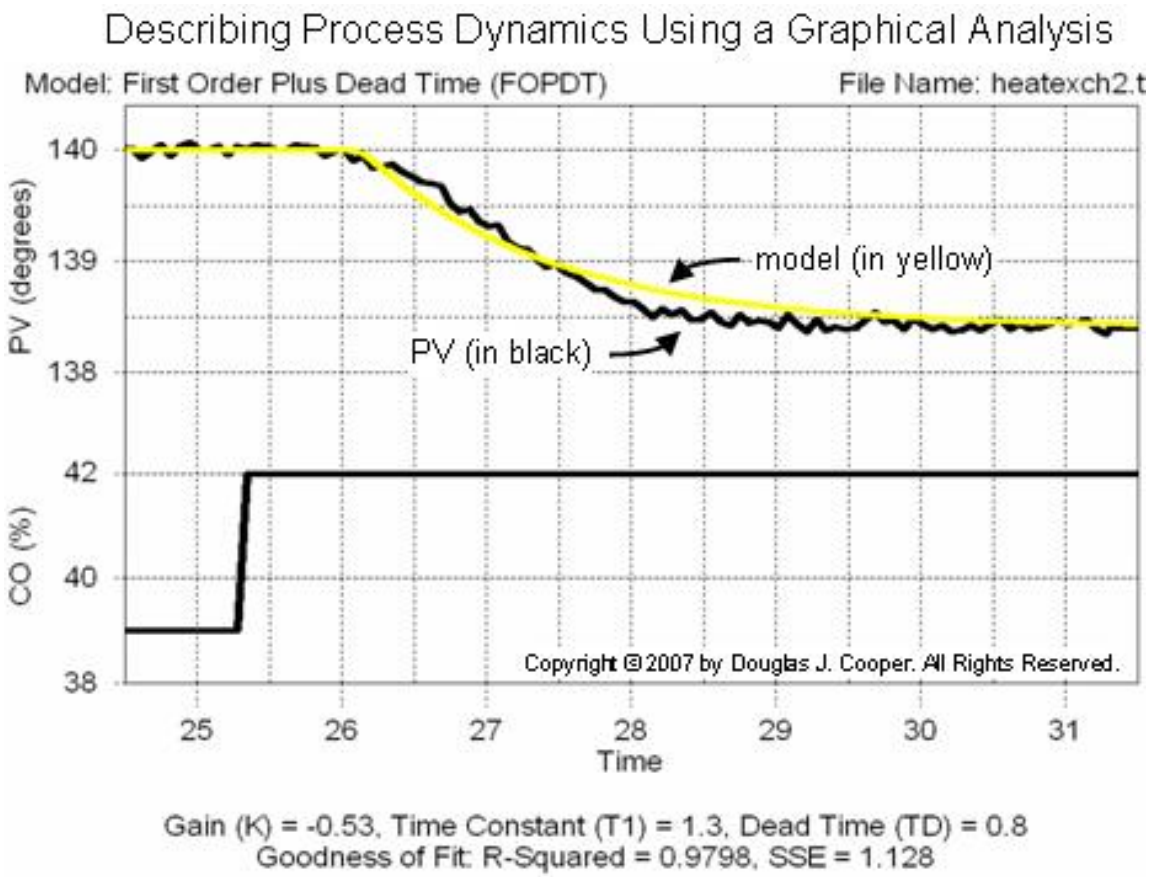
$$1.3\frac{dPV(t)}{dt} + PV(t) = -0.53 \cdot CO(t - 0.8)$$

With units: t [=] min, PV(t) [=] °C, CO(t – Θp) [=] %

The FOPDT model prediction for PV in the plot below was generated by solving the above differential equation using the actual CO signal trace as shown in the plot.

The plot below compares step test data from the heat exchanger process to the FOPDT model using the parameters we computed in the previous articles.

Visual inspection reveals that the simple FOPDT model provides a very good approximation of the dynamic response behavior between the controller output (CO) signal and the measured process variable (PV) for this process.

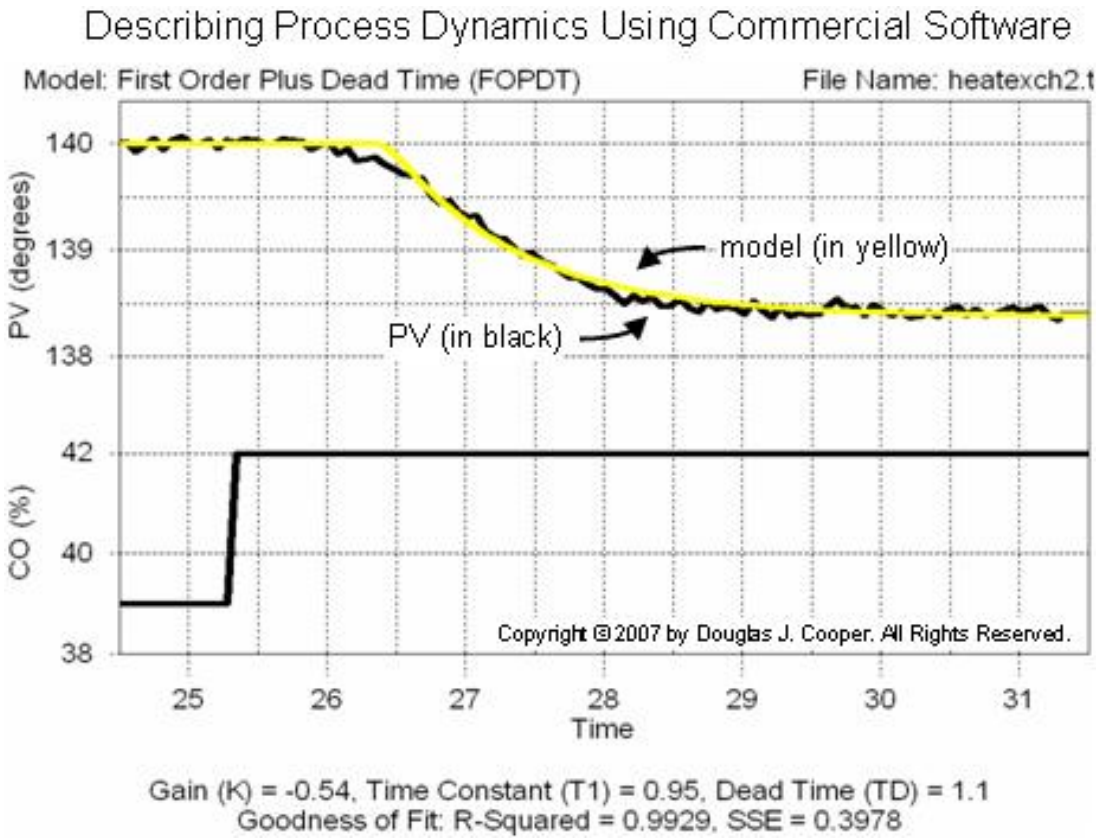


This validates that, for the heat exchanger process at this design level of operation, we have good knowledge of:

- the direction the PV moves given a change in the CO
- how far the PV ultimately travels for a given change in the CO
- how fast the PV moves as it heads toward its new steady state
- how much delay occurs between when the CO changes and the PV first begins to respond

Modeling Using Software

For comparison, we show a FOPDT model fit of the same heat exchanger data using a software modeling tool (more discussion here). The model parameters are somewhat different and the model appears to match the PV data a little better based on a visual inspection.



We will leave it to later articles to discuss the benefits of commercial software for controller design and tuning. For now, it is enough to know that:

- the option is available,
- FOPDT modeling can occur with just a few mouse clicks instead of using graph paper and hand calculations, and
- the results are more descriptive when compared to the graphical method.

With that said, graphical modeling is important to understand because it helps us isolate the role of each FOPDT model parameter, and in particular, appreciate what each says about the controller output (CO) to process variable (PV) relationship.

Using the Model in Controller Design

With a FOPDT model in hand, we can complete the controller design and tuning recipe to implement P-Only, PI, PID and PID with CO Filter control of the heat exchanger process.

The exciting result is that we achieve our desired controller performance using the information from this single step test. Our results are achieved quickly, there is no trial and error, we produce minimal off-spec product, and thus, there is no wasted time or expense.

The method of approximating complex behavior with a FOPDT model and then following a recipe for controller design and tuning has been proven on a wide variety of processes. In fact, it is an approach being used to improve profitability and safety in many plants today.

3) Modeling Process Dynamics: Gravity Drained Tanks Case Study

The Gravity Drained Tanks Process

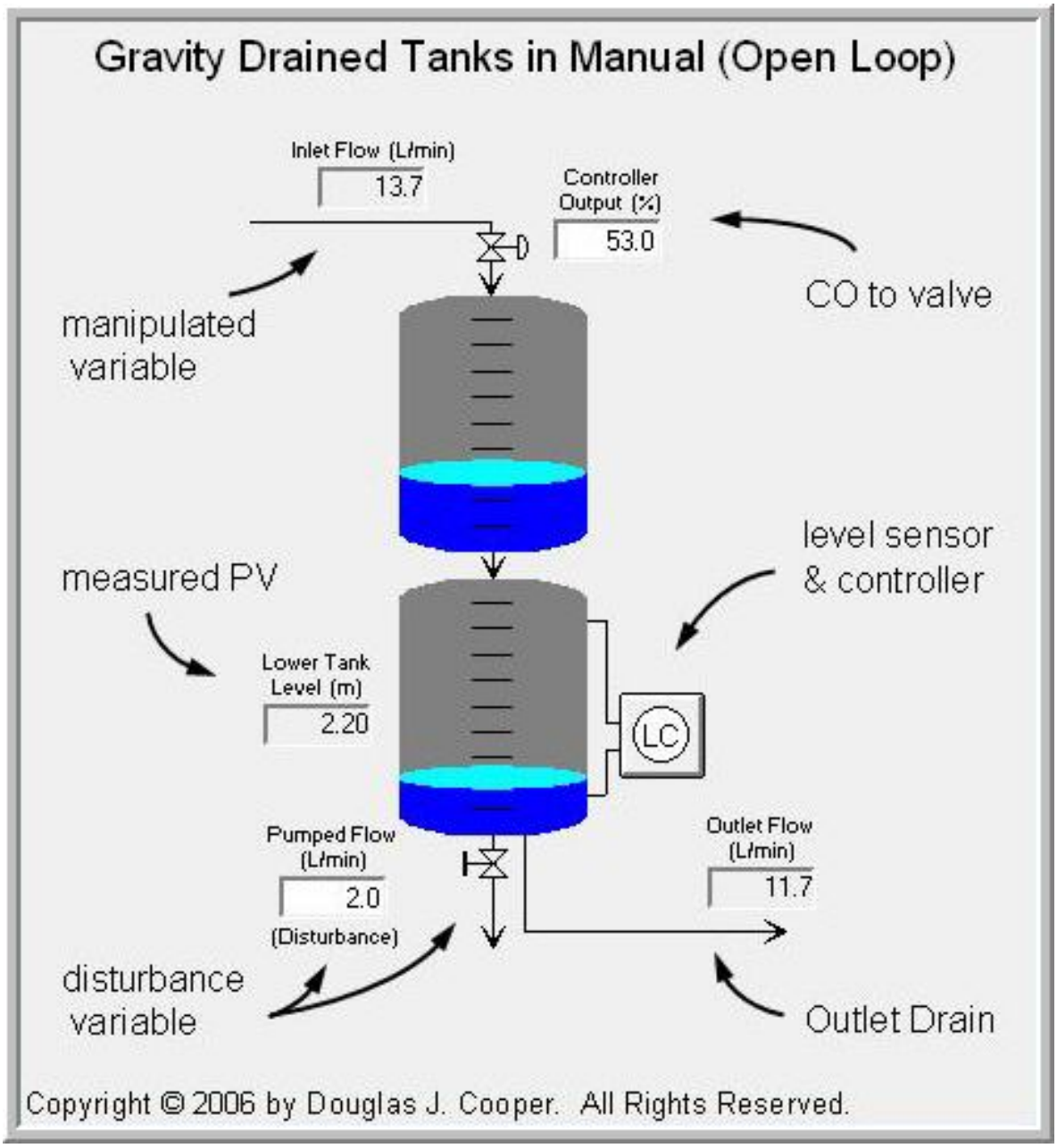
Self Regulating vs Integrating Process Behavior
This case study considers the control of liquid level in a gravity drained tanks process. Like the heat exchanger, the gravity drained tanks displays a typical self regulating process behavior. That is, the measured process variable (PV) naturally seeks a steady operating level if the controller output (CO) and major disturbances are held constant for a sufficient length of time.

It is important to recognize that not all processes, and especially not all liquid level processes, exhibit a self regulating behavior. Liquid level control of a pumped tank process, for example, displays a classical integrating (or on-self-regulating) behavior.

The control of integrating processes presents unique challenges that we will explore in later articles. For now, it is enough to recognize that controller design and tuning for integrating processes has special considerations.

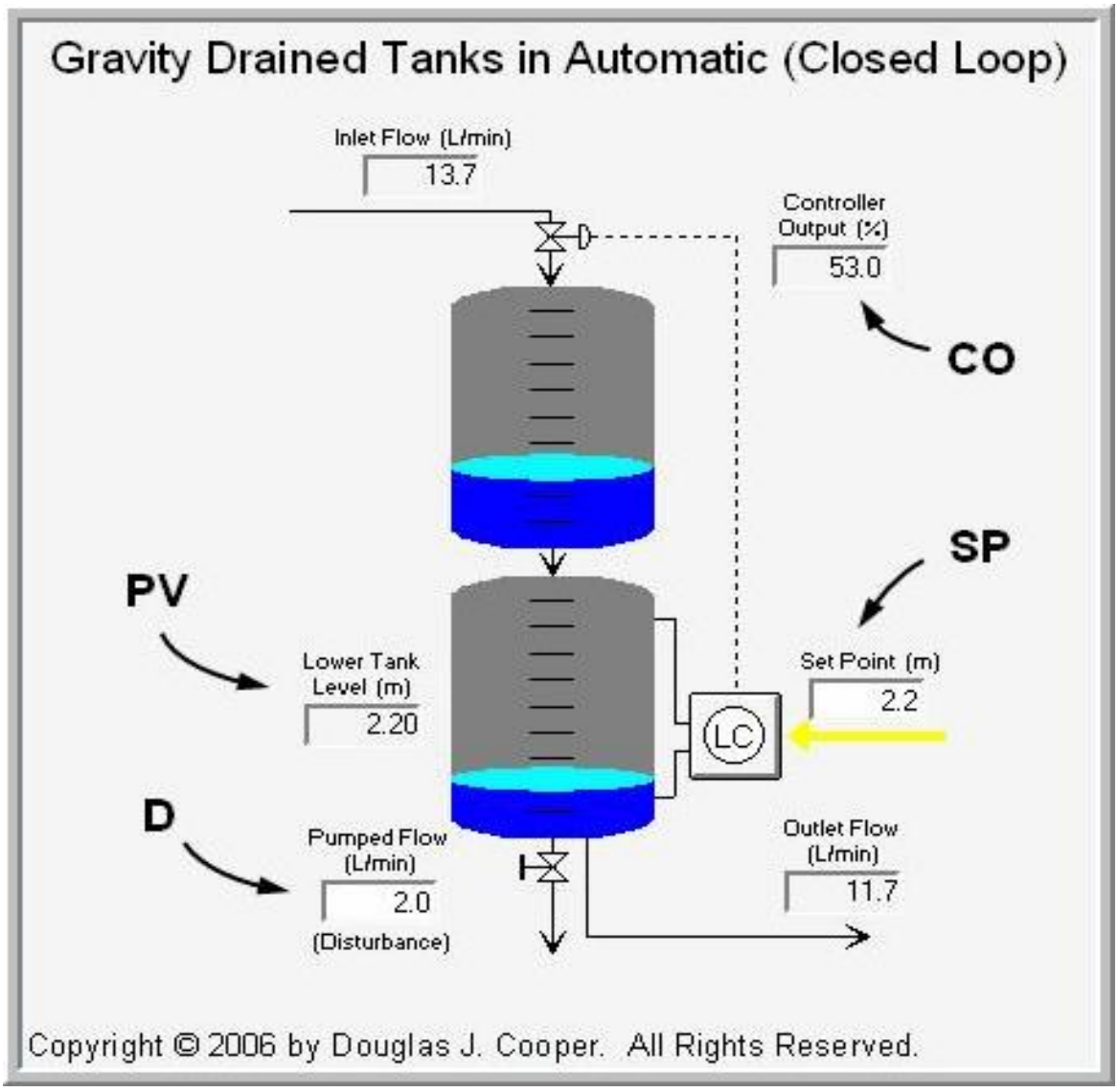
We note that, like the heat exchanger and pumped tank process, the gravity drained tanks case study is a sophisticated simulation derived from first-principles theory and available in commercial software. Simulations let us study different ideas without risking safety or profit. Yet the rules and procedures we develop here are directly applicable to the broad world of real processes with streams comprised of liquids, gases, powders, slurries and melts.

Gravity Drained Tanks Process
The gravity drained tanks process, shown below in manual mode (), is comprised of two tanks stacked one above the other. They are essentially two drums or barrels with holes punched in the bottom.



A variable position control valve manipulates the inlet flow rate feeding the upper tank. The liquid drains freely out through the hole in the bottom of the upper tank to feed the lower tank. From there, the liquid exits either by an outlet drain (another free-draining hole) or by a pumped flow stream.

The next graphic shows the process in automatic mode using our standard nomenclature.



The measured process variable (PV) is liquid level in the lower tank. The controller output (CO) adjusts the valve to maintain the PV at set point (SP).

Aside: The graphic shows tap lines out of the top and bottom of the lower tank and entering the level sensor/controller. This configuration hints at the use of pressure drop as the level measurement method. As we discuss here, we often choose to use sensors that are inexpensive to purchase, install and maintain to monitor parameters related to the actual variable of interest. While our trend plots in this case study show liquid level, our true measured variable is pressure drop across the liquid inventory in the tank. A simple multiplier block translates the weight of liquid pushing on the bottom tap into this level measurement display.

So, for example, if the liquid level in the lower tank is below set point:

- the controller opens the valve some amount,
- increasing the flow rate into the upper tank,
- raising the liquid level in the upper tank,
- increasing the pressure near the drain hole,
- raising the liquid drain rate into the lower tank,

- thus increasing the liquid level in the lower tank.

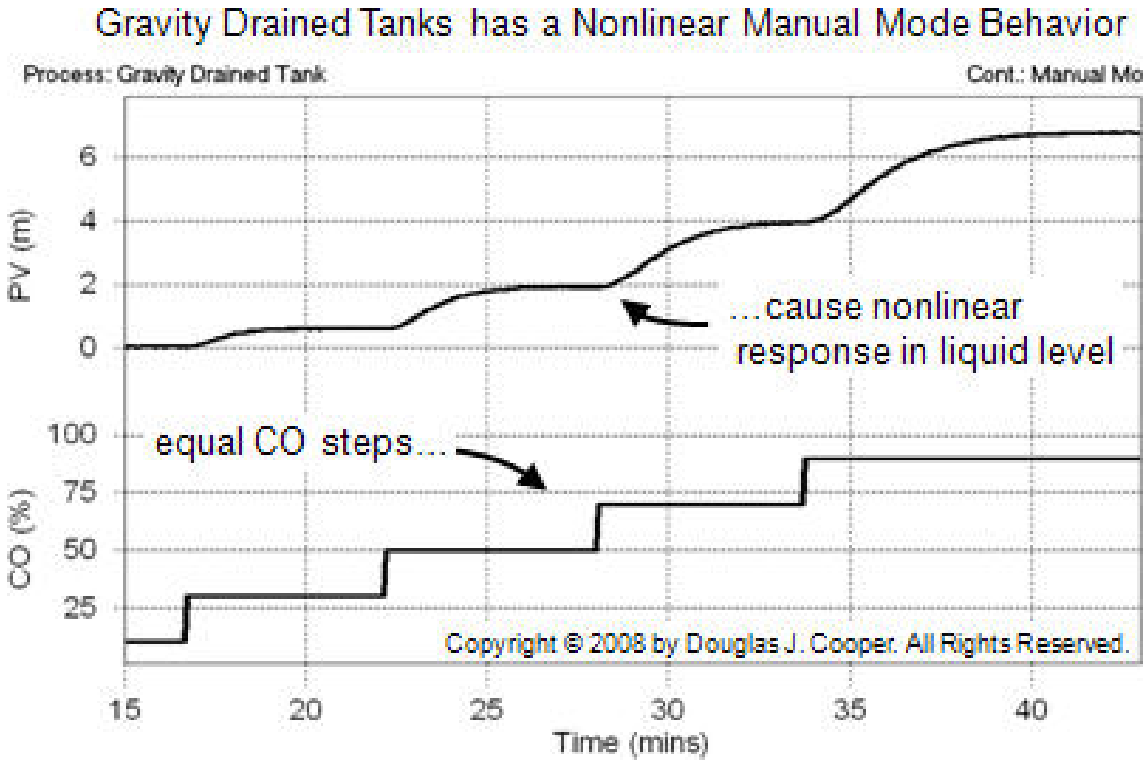
The Disturbance Stream
As shown in the above graphic, the pumped flow stream out of the lower tank acts as a disturbance to this process. The disturbance flow (D) is controlled independently, as if by another process (which is why it is a disturbance to our process).

Because the pumped flow rate, D, runs through a positive displacement pump, it is not affected by liquid level, though it drops to zero if the tank empties.

When D increases (or decreases), the measured PV level quickly falls (or rises) in response.

Process Behavior is Nonlinear
The dynamic behavior of this process is reasonably intuitive. Increase the inlet flow rate into the upper tank and the liquid level in the lower tank eventually rises to a new value. Decrease the inlet flow rate and the liquid level falls.

Gravity driven flows are proportional to the square root of the hydrostatic head, or height of liquid in a tank. As a result, the dynamic behavior of the process is modestly nonlinear. This is evident in the open loop response plot below ().



As shown above, the CO is stepped in equal increments, yet the response shape of the PV clearly changes as the level in the tank rises. The consequence of this nonlinear behavior is that a controller designed to give desirable performance at one operating level may not give desirable performance at another level.

Modeling Dynamic Process Behavior
We next explore dynamic modeling of process behavior for the gravity drained tanks.

Dynamic "Bump" Testing Of The Gravity Drained Tanks Process

We introduced the gravity drained tanks process in a previous article and established that it displays a self regulating behavior. We also learned that it exhibits a nonlinear behavior, though to a lesser degree than that of the heat exchanger.

Our control objective is to maintain liquid level in the lower tank at set point in spite of unplanned and unmeasured disturbances. The controller will achieve this by manipulating the inlet flow rate into the upper tank.

To proceed, we follow our controller design and tuning recipe:

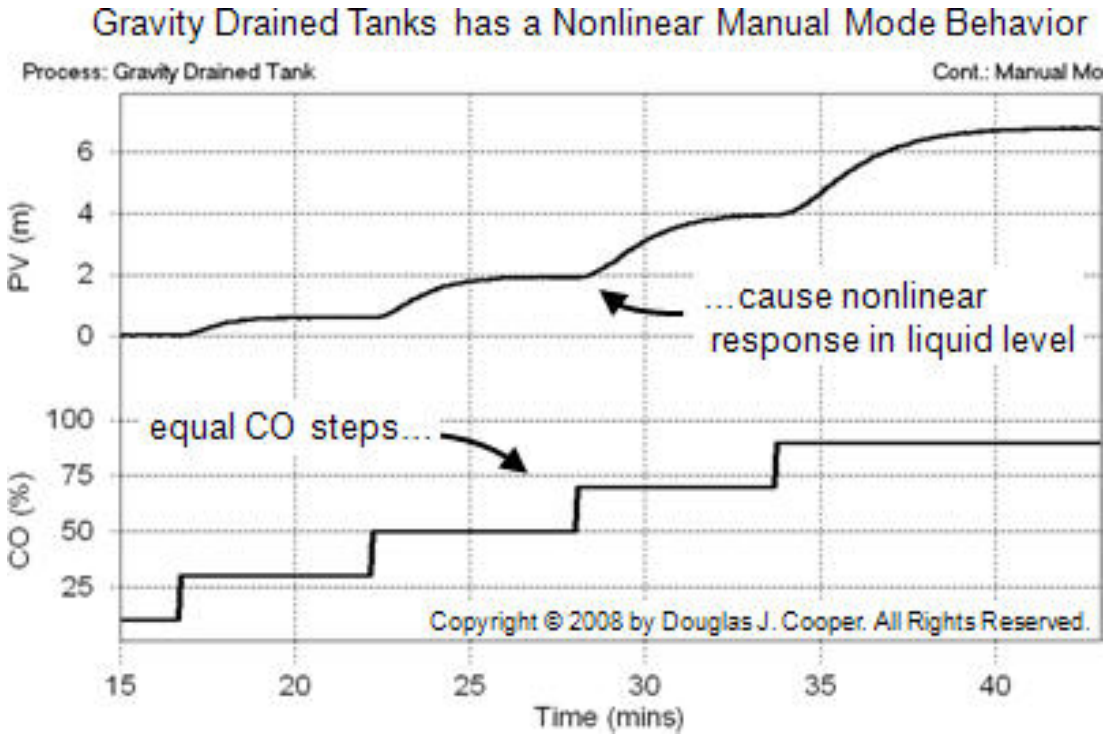
1. Establish the design level of operation (DLO), defined as the expected values for set point and major disturbances during normal operation
2. Bump the process and collect controller output (CO) to process variable (PV) dynamic process data around this design level
3. Approximate the process data behavior with a first order plus dead time (FOPDT) dynamic model to obtain estimates for process gain, Kp (how far variable), process time constant, *T*p (how fast variable), and the process dead time, Θp (with how much delay variable).
4. Use the model parameters from step 3 in rules and correlations to complete the controller design and tuning.

Step 1: Design Level of Operation (DLO)

Nonlinear behavior is a common characteristic of processes with streams comprised of liquids, gases, powders, slurries and melts. Nonlinear behavior implies that Kp, *T*p and/or Θp changes as operating level changes.

Since we use Kp, *T*p and Θp values in correlations to complete the controller design and tuning, the fact that they change gives us pause. It implies that a controller tuned to provide a desired performance at one operating level will not give that same performance at another level. In fact, we demonstrated this on the heat exchanger.

The nonlinear nature of the gravity drained tanks process is evident in the manual mode (open loop) response plot below .



As shown, the CO is stepped in equal increments, yet the response (and thus the Kp, *T*p, and/or Θp) changes as the level in the tank rises.

We address this concern by specifying a design level of operation (DLO) as the first step of our controller design and tuning recipe. If we are careful about how and where we collect our test data, we heighten the probability that the recipe will yield a controller with our desired performance.

The DLO includes where we expect the set point, SP, and measured process variable, PV, to be during normal operation, and the range of values the SP and PV might assume so we can explore the nature of the process across that range.

For the gravity drained tanks, the PV is liquid level in the lower tank. For this control study, we choose:

- Design PV and SP = 2.2 m with range of 2.0 to 2.4 m

The DLO also considers our major disturbances. We should know the normal or typical values for our major disturbances and be reasonably confident that they are quiet so we may proceed with a dynamic (bump) test.

The gravity drained tanks process has one major disturbance variable, the pumped flow disturbance, D. For this study, D is normally steady at about 2 L/min, but certain operations in the plant cause it to momentarily spike up to 5 L/min for brief periods.

Rejecting this disturbance is a major objective in our controller design. For this study, then:

- Design D = 2 L/min with occasional spikes up to 5 L/min

Step 2: Collect Data at the DLO (Design Level of Operation)
The next step in our recipe is to collect dynamic process data as near as practical to our design level of operation. We do this with a **bump test**, where we step or pulse the CO and collect data as the PV responds.

It is important to wait until the CO, PV and D have settled out and are as near to constant values as is possible for our particular operation before we start a bump test. The point of bumping a process is to learn about the cause and effect relationship between the CO and PV.

With the process steady, we are starting with a clean slate and as the PV responds to the CO bumps, the dynamic cause and effect behavior is isolated and evident in the data. On a practical note, be sure the data capture routine is enabled before the initial bump so all relevant data is collected.

While closed loop testing is an option, here we consider two open loop (manual mode) methods: the step test and the doublet test.

For either method, the CO must be moved far enough and fast enough to force a response in the PV that dominates the measurement noise.

Also, our bump should move the PV both above and below the DLO during testing. With data from each side of the DLO, the FOPDT model will be able to average out the nonlinear effects.

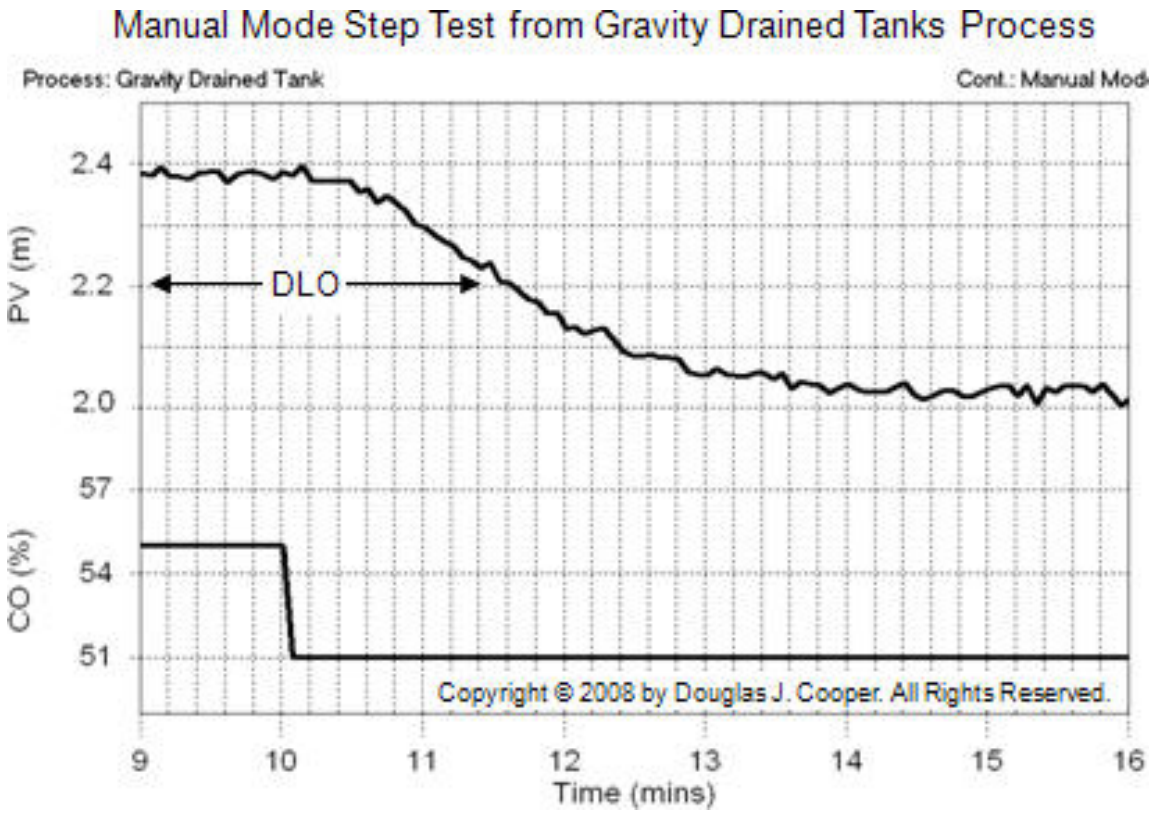
● *Step Test*
To collect data that will “average out” to our design level of operation, we start the test with the PV on one side of (either above or below) the DLO. Then, we step the CO so that the measurement moves across to settle on the other side of the DLO.

We acknowledge that it may be unrealistic to attempt such a precise step test in some production environments. But we should understand why we propose this ideal approach (answer: to average nonlinear process effects).

Recall that our DLO is a PV = 2.2 m and D = 2 L/min (though not shown on the plots, disturbance D remains constant at 2 L/min throughout the test).

Below, we set CO = 55% and the process steadies at a PV = 2.4 L.

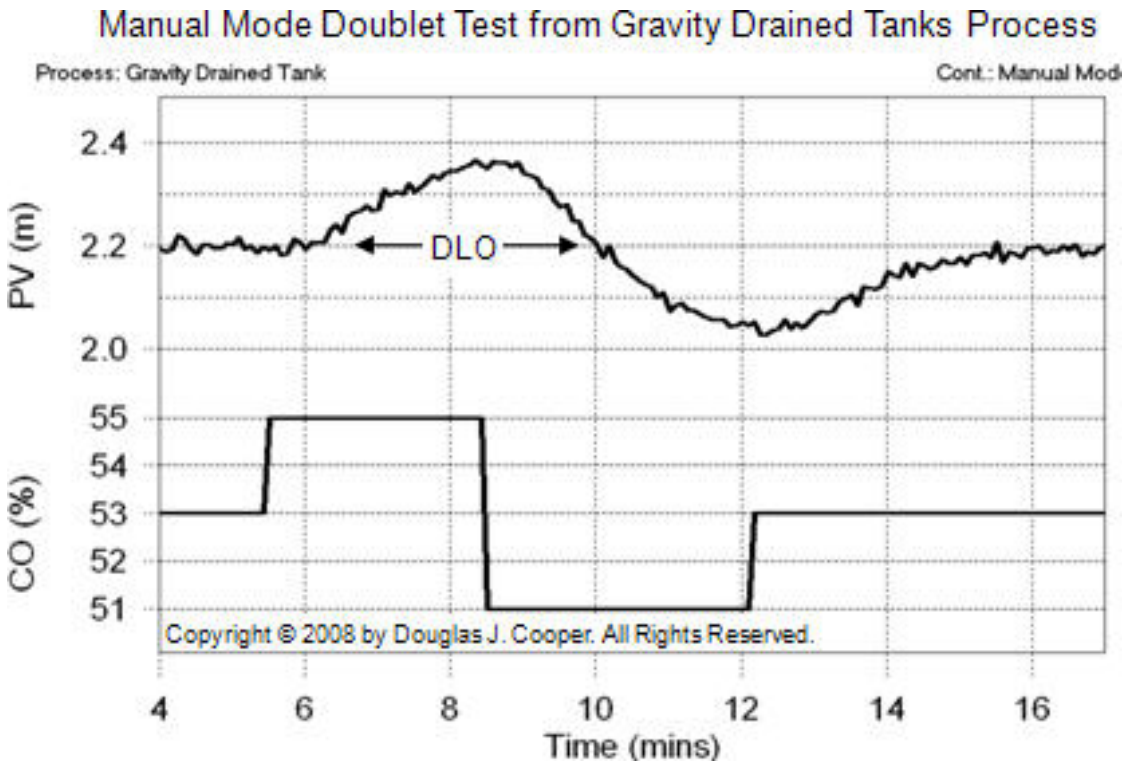
Then, as shown below (), we step the CO to 51% and the PV settles at abut 2.0 L/min. Thus, we have collected data “around” our design level of operation.



Note that we can start high and step the CO down (as above), or start low and step the CO up. Both methods produce dynamic data of equal value for our design and tuning recipe.

● *Doublet Test*
A doublet test, as shown below (), is two CO pulses performed in rapid succession and in opposite direction.

The second pulse is implemented as soon as the process has shown a clear response to the first pulse that dominates the noise in the PV. It is not necessary to wait for the process to respond to steady state for either pulse.



The doublet test offers important benefits. Specifically, it:

- starts from and quickly returns to the design level of operation,
- produces data both above and below the design level to "average out" the nonlinear effects, and
- the PV always stays close to the DLO, thus minimizing off-spec production.

For these reasons, many industrial practitioners find the doublet to be the preferred method for generating open loop dynamic process data, though it does require that we use a commercial software tool for the model fitting task.

Modeling Process Dynamics
Next, we model the dynamics of the gravity drained tanks process and use the result in a series of control studies.

Graphical Modeling of Gravity Drained Tanks Step Test

We have explored the manual mode (open loop) operation and behavior of the gravity drained tanks process and have worked through the first two steps of the controller design and tuning recipe.

As those articles discuss:

- Our measured process variable, PV, is liquid level in the lower tank,
- The set point, SP, is held constant during normal production operation,
- Our primary control challenge is rejecting unexpected disruptions from D, the pumped flow disturbance.

We have generated process data from both a step test and a doublet test around our design level of operation (DLO), which for this study is:

- Design PV and SP = 2.2 m with range of 2.0 to 2.4 m
- Design D = 2 L/min with occasional spikes up to 5 L/min

Here we present step 3 of our recipe and focus on a graphical analysis of the step test data. Next we will explore modeling of the doublet test data using software.

Data Accuracy
We should understand that real plant data is rarely as perfect as that shown in the plots below. As such, we should not seek to extract more information from our data than it actually contains.

In the analyses presented here, we display extra decimals of accuracy only because we will be comparing different modeling methods over several articles. The extra accuracy will help when we make side-by-side comparisons of the results.

Step 3: Fit a FOPDT Model to the Data
The third step of the recipe is to describe the overall dynamic behavior of the process with an approximating first order plus dead time (FOPDT) dynamic model.

We will move quickly through the graphical analysis of step response data as we already presented details of the procedure in the heat exchanger study.

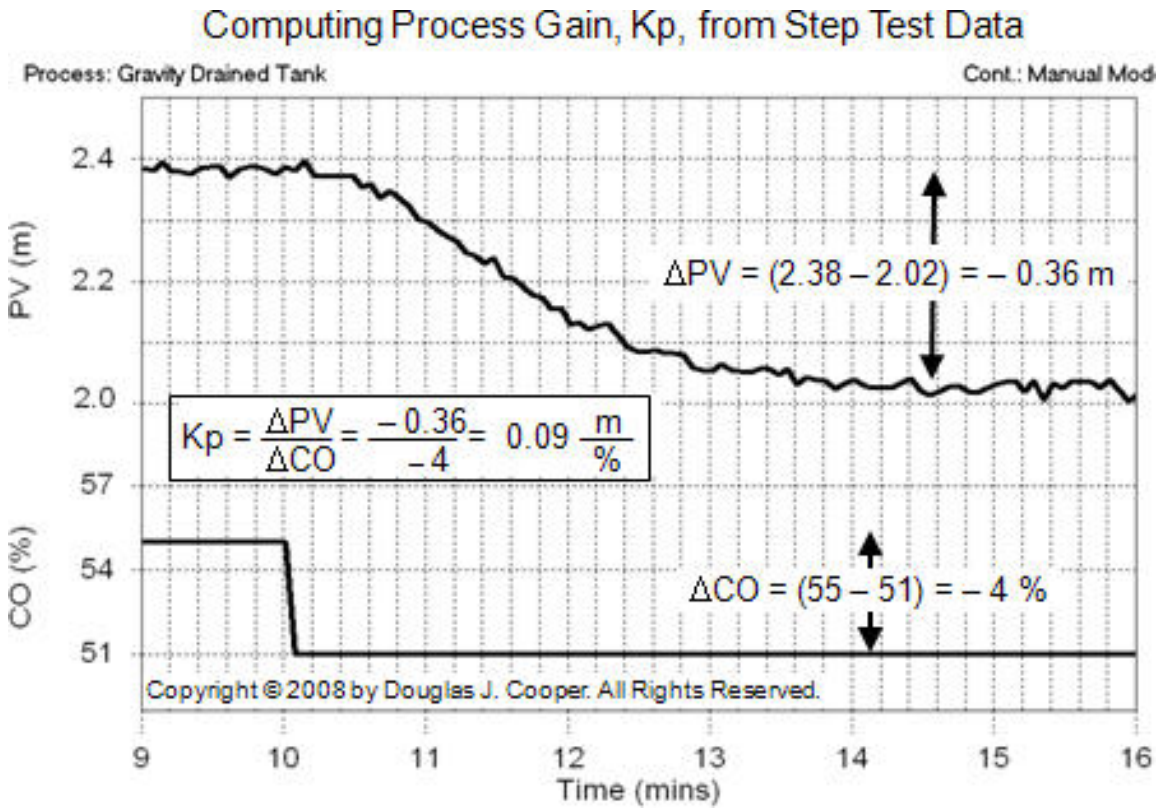
• *Process Gain – The "Which Direction and How Far" Variable*
Process gain, Kp, describes how far the PV moves in response to a change in controller output (CO). It is computed:

$$Kp = \frac{\text{Steady State Change in Measured Process Variable, } \Delta PV}{\text{Steady State Change in Controller Output, } \Delta CO}$$

where ΔPV and ΔCO represent the total change from initial to final steady state. The path or length of time the PV takes to get to its new steady state does not enter into the Kp calculation.

Reading the numbers off of our step test plot below (), the CO was stepped from a steady value of 55% down to 51%.

The PV was initially steady at 2.38 m, and in response to the CO step, moved down to a new steady value of 2.02 m.



Using these in the Kp equation above, the process gain for the gravity drained tanks process around a DLO (design level of operation) PV of about 2.2 m when the pumped disturbance (not shown in the plot) is constant at 2 L/min is:

$$\mathbf{Kp} = \frac{\Delta PV}{\Delta CO} = \frac{2.38 - 2.02\ \text{m}}{55 - 51\%} = \mathbf{0.09\ m/\%}$$

For further discussion and details, another example of a process gain calculation from step test data for the heat exchanger is presented here.

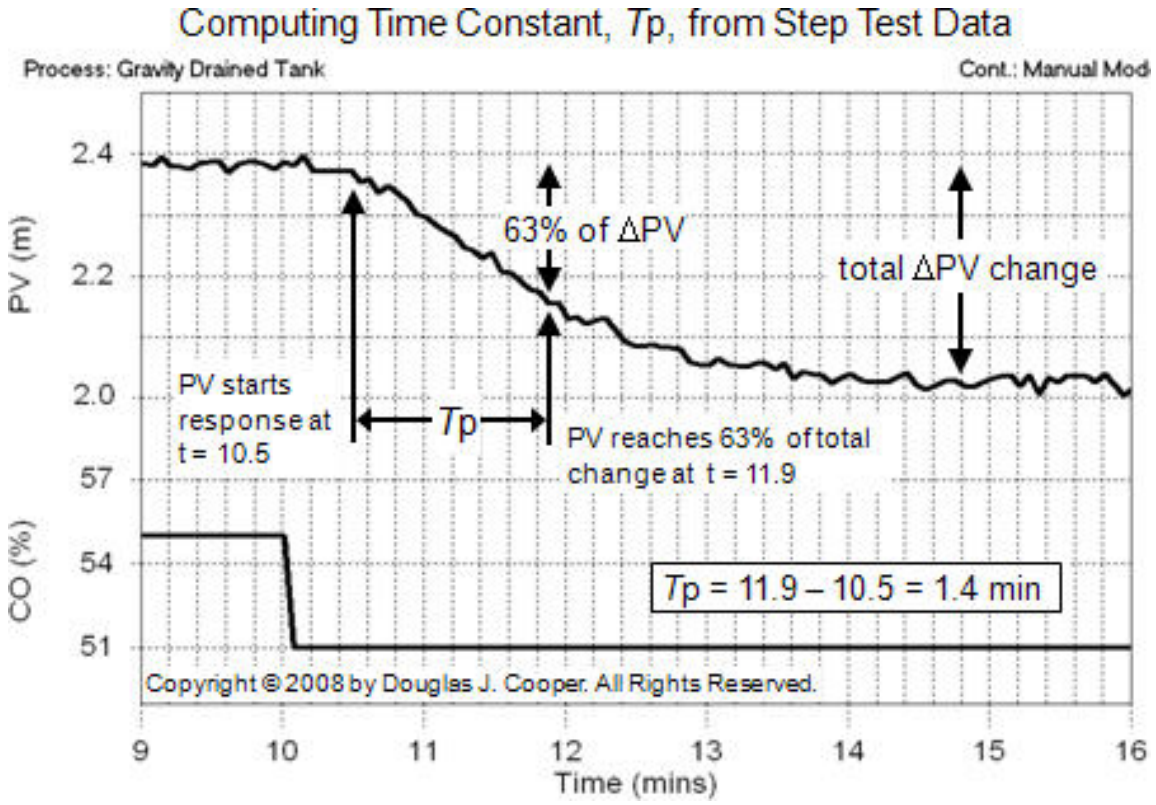
• *Time Constant - The "How Fast" Variable*
The time constant, *T*p, describes how fast the PV moves in response to a change in the CO.

For step test data, *T*p can be computed as the time that passes from when the PV shows its first response to the CO step, until when the PV reaches 63% of the total ΔPV change that is going to occur.

The time constant must be positive and have units of time. For controllers used on processes comprised of gases, liquids, powders, slurries and melts, *T*p most often has units of minutes or seconds.

For step test data, we compute *T*p in five steps (see plot below):
1. Determine ΔPV, the total change in PV from final steady state minus initial steady state
2. Compute the value of the PV that is 63% of the way toward the total ΔPV change, or "initial steady state + 0.63(ΔPV)"

3. Note the time when the PV passes through the “initial steady state + 0.63(ΔPV)” value
4. Subtract from it the time when the “PV starts a first clear response” to the step change in the CO
5. The passage of time from step 4 minus step 3 is the process time constant, *T*p.



Following the procedure for the plot above ():

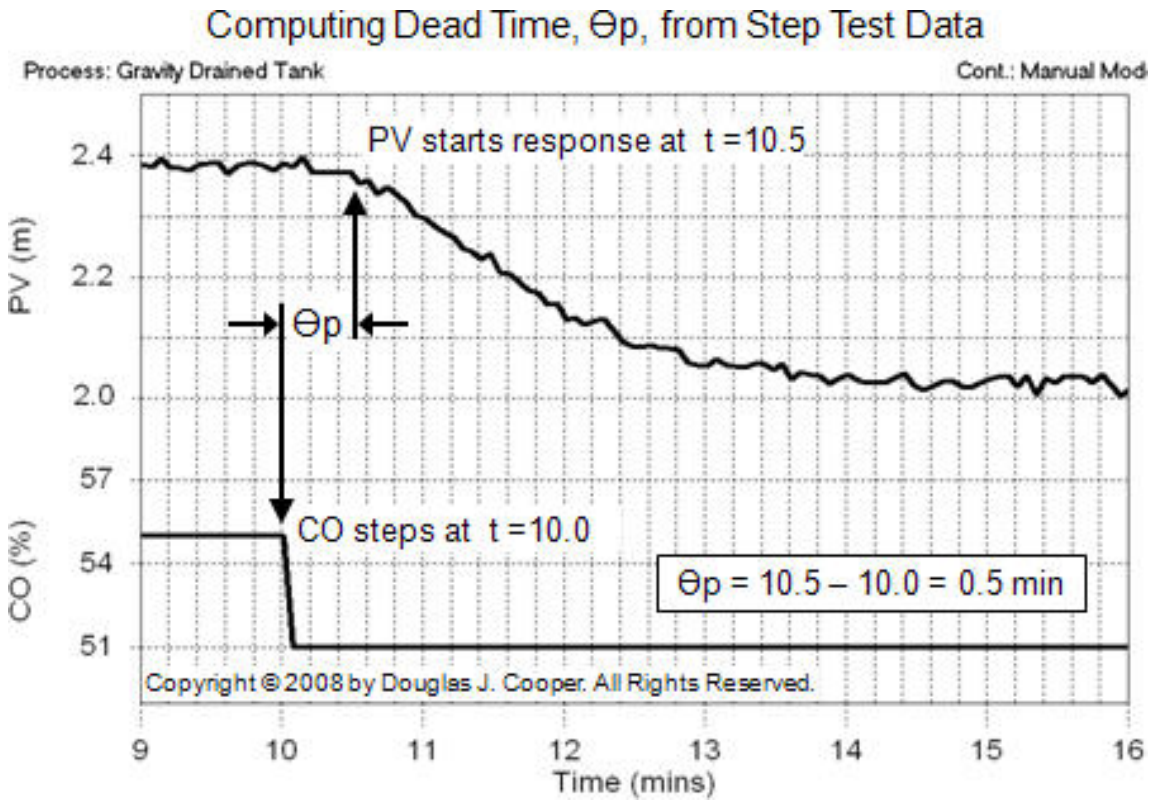
1. The PV was initially steady at 2.38 m and moved to a final steady state of 2.02m. The total change, ΔPV, is “final minus initial steady state” or:
   ΔPV = 2.02 – 2.38 = –0.36 m
2. The value of the PV that is 63% of the way toward this total change is “initial steady state + 0.63(ΔPV)” or:
   Initial PV + 0.63(ΔPV) = 2.38 + 0.63(–0.36)
   = 2.38 – 0.23
   = 2.15 m
3. The time when the PV passes through the “initial steady state PV + 0.63(ΔPV)” point of 2.15 m is:
   Time to 0.63(ΔPV) = Time to 2.15
   = 11.9 min
4. The time when the “PV starts a first response” to the CO step is:
   Time PV response starts = 10.5 min
5. The time constant is “time to 63%(ΔPV)” minus “time PV response starts” or:
   ***T*p** = 11.9 – 10.5 = **1.4 min**

There are further details and discussion on the process time constant and its calculation from step test data in the heat exchanger example presented in another article.

• *Dead Time - The "How Much Delay" Variable*
Dead time, Θp, is the time delay that passes from when a CO action is made and the measured PV shows its first clear response to that action.

Like a time constant, dead time has units of time, must always be positive, and for processes with streams comprised of gasses, liquids, powders, slurries and melts, is most often expressed in minutes or seconds.

Estimating dead time, Θp, from step test data is a three step procedure:
1. Locate the time when the "PV starts a first clear response" to the step change in the CO. We already identified this point when we computed *T*p above.
2. Locate the point in time when the CO was stepped from its original value to its new value.
3. Dead time, Θp, is the difference in time of step 1 minus step 2.



Applying this procedure to the step test plot above ():

1. As identified in the plot above, the PV starts a first clear response to the CO step at 10.5 min (this is the same point we identified in the *T*p analysis),
2. The CO step occurred at 10.0 min, and thus,
3. **Θp** = 10.0 – 10.5 = **0.5 min**

Additional details and discussion on process dead time and its calculation from step test data can be found in the heat exchanger example presented here.

Note on Units
During a dynamic analysis study, it is best practice to express *T*p and Θp in the same

units (e.g. both in minutes or both in seconds). The tuning correlations and design rules assume consistent units.

Also, the process gain, Kp, should be expressed in the same (though inverse) units of the controller gain, Kc, (or proportional band, PB) used by our manufacturer.

Control is challenging enough without adding computational error to our problems.

Validating Our FOPDT Model

It is good practice to validate our FOPDT model before proceeding with design and tuning. If our model describes the dynamic data, and the data is reflective of the process behavior, then the last step of the recipe follows smoothly.

The FOPDT model parameters we computed from the analysis of the step test data are:

- Process gain (how far), Kp = 0.09 m/%
- Time constant (how fast), τp = 1.4 min
- Dead time (how much delay), Θp = 0.5 min

Recall that, using the nomenclature of this site, the FOPDT dynamic model has the general form:

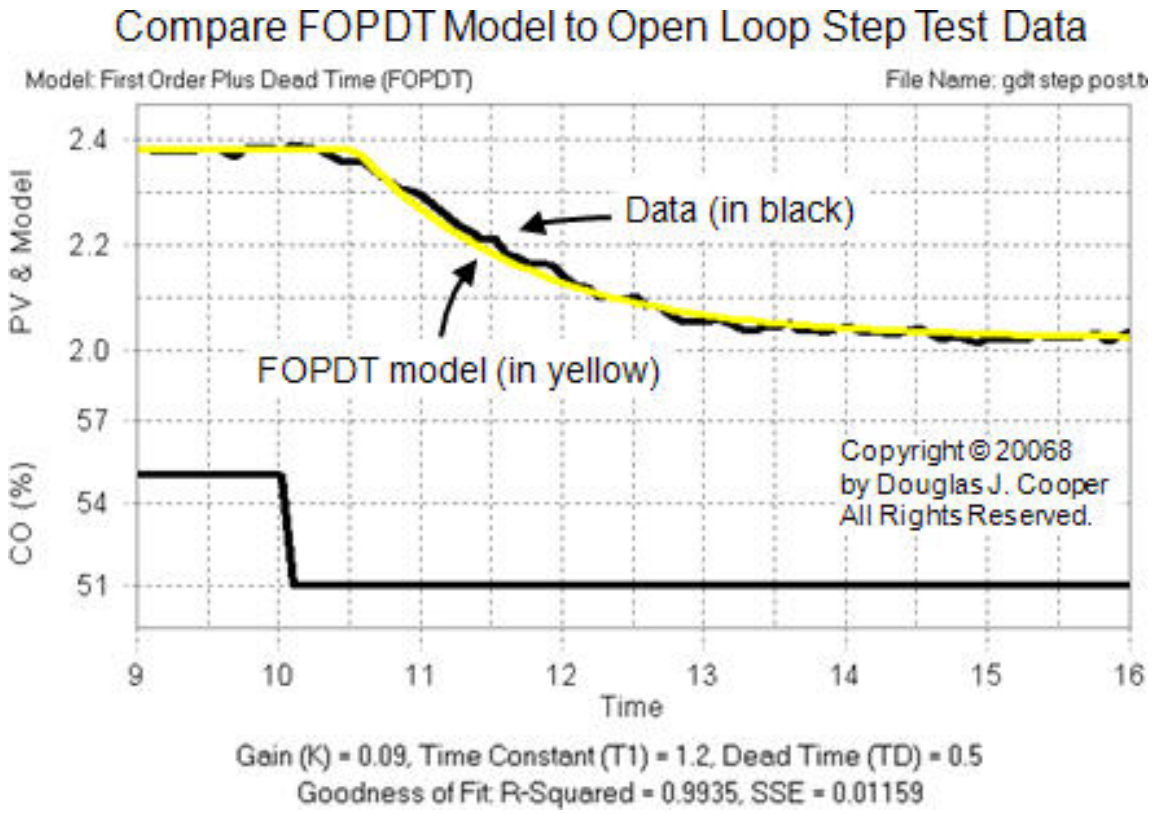
$$\tau p \frac{dPV(t)}{dt} + PV(t) = Kp \cdot CO(t - \theta p)$$

And this means that the dynamic behavior of the gravity drained tanks can be reasonably approximated around our DLO as:

$$1.4 \frac{dPV(t)}{dt} + PV(t) = 0.09 \cdot CO(t - 0.5)$$

Where: t [=] min, PV(t) [=] m, $CO(t - \Theta p)$[=] %

The plot below compares step test data from the gravity drained tanks process to this FOPDT model.



Visual inspection confirms that the simple FOPDT model provides a very good approximation for the behavior of this process.

Specifically, our graphical analysis tells us that for the gravity drained tanks process, with a DLO PV of about 2.2 m when the pumped disturbance is constant at 2 L/min:

- the direction PV moves given a change in CO
- how far PV ultimately travels for a given change in CO
- How fast PV moves as it heads toward its new steady state
- how much delay occurs between when CO changes and PV first begins to respond

This is precisely the information we need to proceed with confidence to step 4 of the design and tuning recipe.

Modeling Doublet Test Data

We had suggested in a previous article that a doublet test offers benefits as an open loop method for generating dynamic process data. These include that the process:

- starts from and quickly returns to the DLO,
- yields data both above and below the design level to “average out” the nonlinear effects, and
- the PV always stays close to the DLO, thus minimizing off-spec production.

We explore using commercial software to fit a FOPDT model to doublet test data for the gravity drained tanks in this article.

Modeling Gravity Drained Tanks Data Using Software

We have investigated a graphical analysis method for fitting a first order plus dead time (FOPDT) dynamic model to step test data for both the heat exchanger and the gravity drained tanks processes in previous articles.

Describing process behavior with an approximating FOPDT dynamic model is the third step of our controller design and tuning recipe. Thus, it is a critical step for quickly achieving desired controller performance while avoiding time consuming and expensive trial and error methods.

The reason we studied graphical modeling is because it is a useful way to isolate the role of each FOPDT model parameter, and in particular, appreciate what each says about a controller output (CO) to process variable (PV) relationship.

As we learned in these investigations, for a change in CO:

- Process gain, Kp, describes the direction and how far the PV moves,
- Time constant, *T*p, describes how fast the PV responds,
- Dead time, Θp, describes how much delay occurs before the PV first begins to move.

But in industrial practice, graphical modeling methods are very limiting for (at least) two reasons.

First, they restrict us to often-impractical step test data. With software, we can fit models to a broad range of dynamic data sets, including closed loop (automatic mode) set point response data.

And instead of using pencil, paper, calculator and ruler to analyze a step test, software can produce a reliable fit and present the results for our inspection almost as fast as we can click on the program icons.

Software Requires Electronic Data

One requirement for using a commercial software package for dynamic modeling and controller design is that the process data must be available in some sort of electronic form, ranging from a simple text file to an Excel spreadsheet format.

If the controller for the loop being modeled is software based (such as on a DCS), if the hardware is OPC enabled, if the controller is connected to a data historian, or if we have capable in-house tech support, then we should have access to process data in a file f Practitioner's note: If you believe process control is important to plant safety and profitability, yet your process data is not available in an electronic format, then perhaps your management is not convinced that process control is important to plant safety and profitability.ormat.

The file must contain the sampled CO signal paired with the corresponding PV measurement for the entire bump test. If the data was collected at a constant sample rate, T, then we also must know this value. Otherwise, the data file must match each CO and PV pair with a sample time stamp.

Model Fitting of Doublet Data

For our gravity drained tanks study, we have previously discussed that the PV is liquid level in the lower tank, that the set point, SP, is held constant during production, and that

our main objective is rejecting disruptions from D, the pumped flow disturbance.

We also presented process data from a doublet test around our design level of operation (DLO), which for this study is:

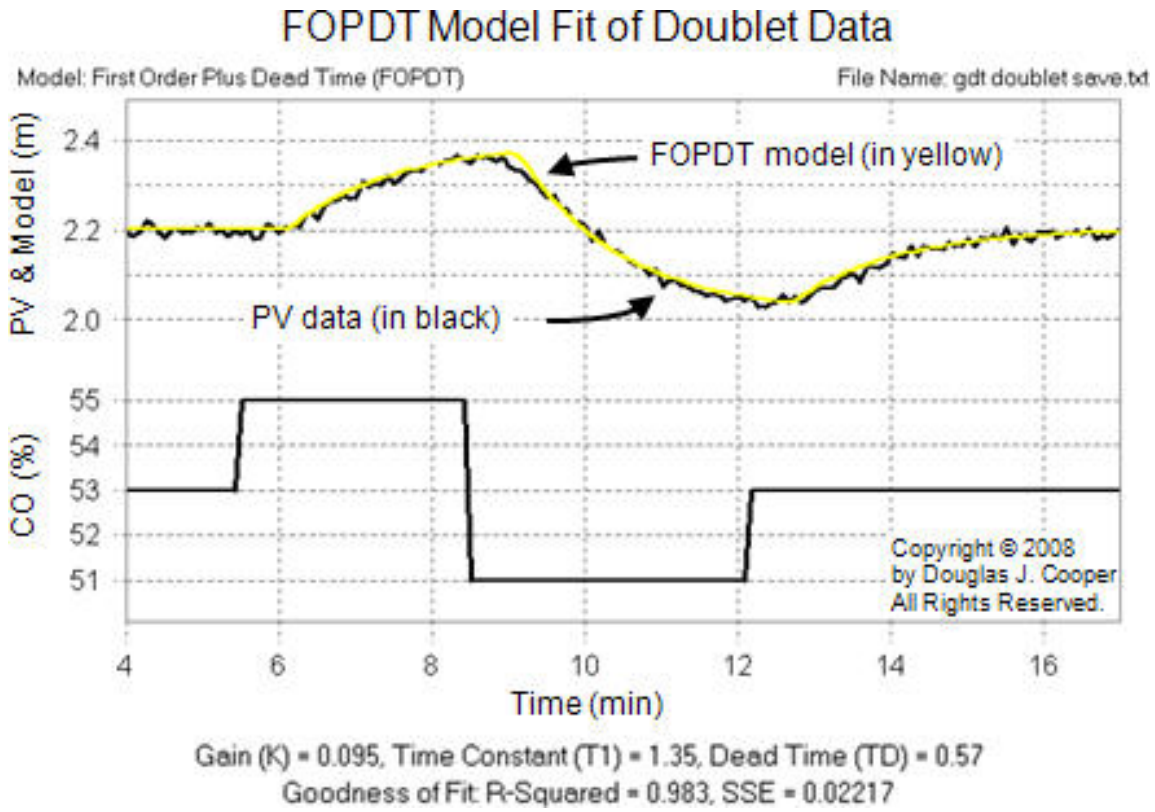
- design PV and SP = 2.2 m with range of 2.0 to 2.4 m
- design D = 2 L/min with occasional spikes up to 5 L/min

A doublet test, as shown below, is two CO pulses performed in rapid succession and in opposite direction. The second pulse is implemented as soon as the process has shown a response to the first pulse that clearly dominates the noise in the PV. It is not necessary to wait for the process to respond to steady state for either pulse.

The doublet test offers important benefits as a testing method, including that it starts from and quickly returns to the DLO. It also produces data both above and below the design level to “average out” the nonlinear effects. And the PV always stays close to the design level of operation, thus minimizing off-spec production.

Using the commercial software offered by Control Station, Inc, we read our data file into the software, select “First Order Plus Dead Time” from the model library, and then click “fit model.”

The results of the automated model fit are displayed below ():



The sampled data is the black trace in the above plot and the FOPDT model is displayed in yellow. The model parameters from the doublet test model fit (shown below the plot) are listed in the table below.

For comparison, the results from our previous step analysis are also listed:

	Step	Doublet
Process gain, Kp (m/%)	0.09	0.095
Time constant, *Tp* (min)	1.4	1.35
Dead time, Θp (min)	0.5	0.57

The software model fit is consistent with the step test graphical analysis. The extra accuracy of the computer output, though displayed by the software, does not necessarily hold significance as process data rarely contains such precise dynamic information. For real processes, these numbers are essentially equal.

Model Fit Minimizes SSE

The model fitting software performs a systematic search for a combination of model parameters that minimizes the sum of squared errors (SSE), computed as:

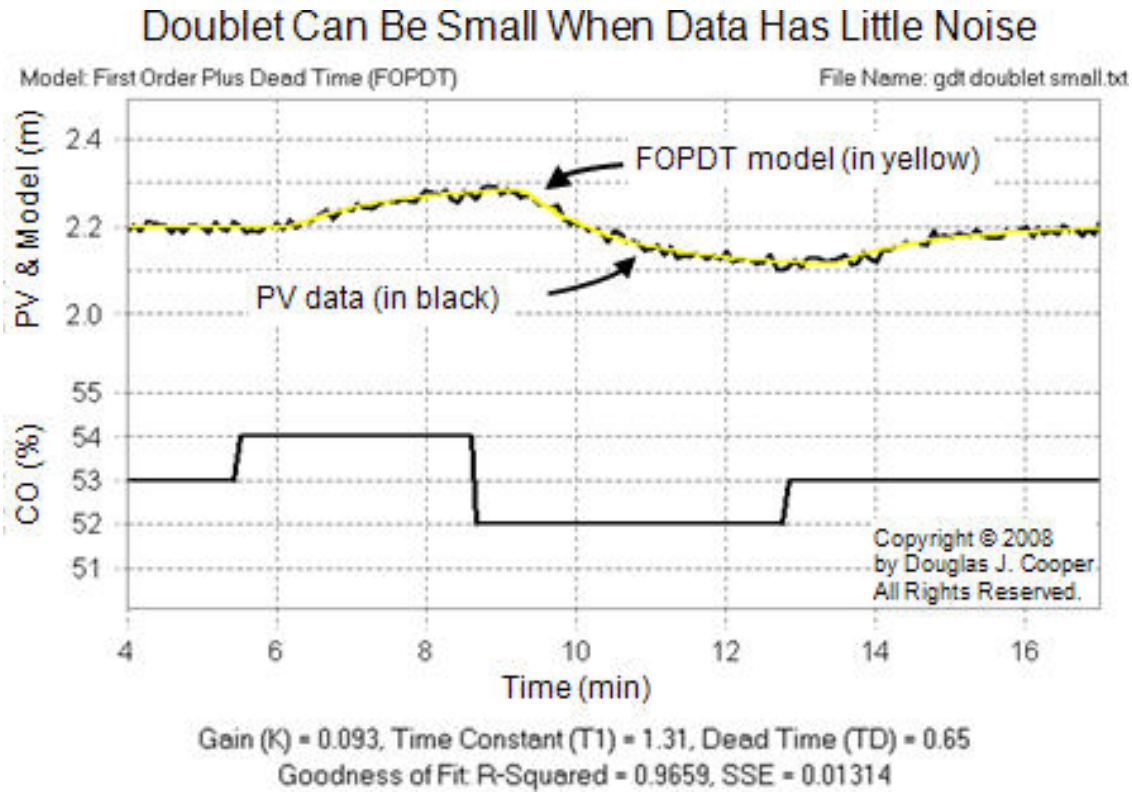
$$SSE = \sum_{i=1}^{N} [\text{Measured PV}_i - \text{Model PV}_i]^2$$

The Measured PV is the actual data collected from our process. The Model PV is computed using the model parameters from the search routine and the actual CO data from the file. N is the total number of samples in the file.

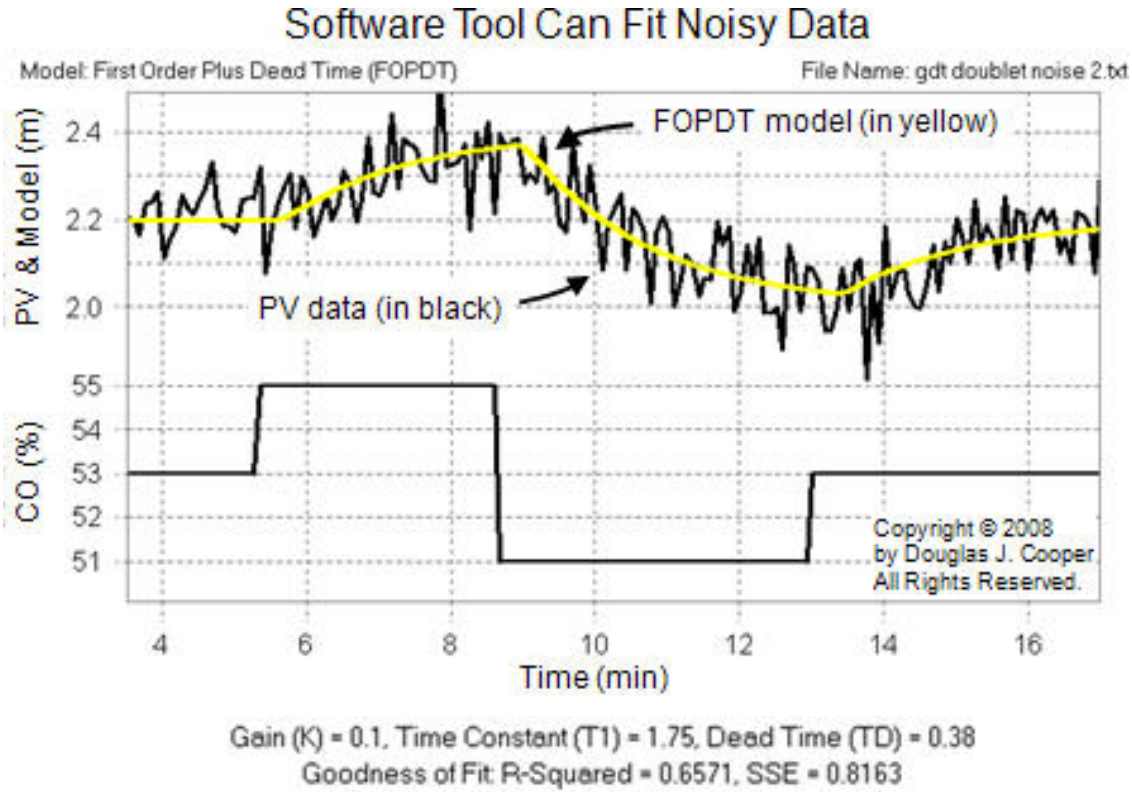
In general, the smaller the SSE, the better the model describes the data.

Software Offers Benefits

When the external disturbances and noise in the PV signal are small, the doublet can be quite modest in size yet still yield data for a meaningful fit as shown below ():



As shown below (), software can also model data that contains significant noise in the PV signal, as long as the external disturbances are quiet.



For comparison, the model parameters from all of the above fits are summarized below:

	Step	Doublet	Small	Noisy
Process gain, Kp (m/%)	0.09	0.095	0.093	0.10
Time constant, *Tp* (min)	1.4	1.35	1.31	1.75
Dead time, Θp (min)	0.5	0.57	0.65	0.38

From a controller design and tuning perspective, each set of model parameters are similar enough that each will yield a controller with virtually identical performance and capability.

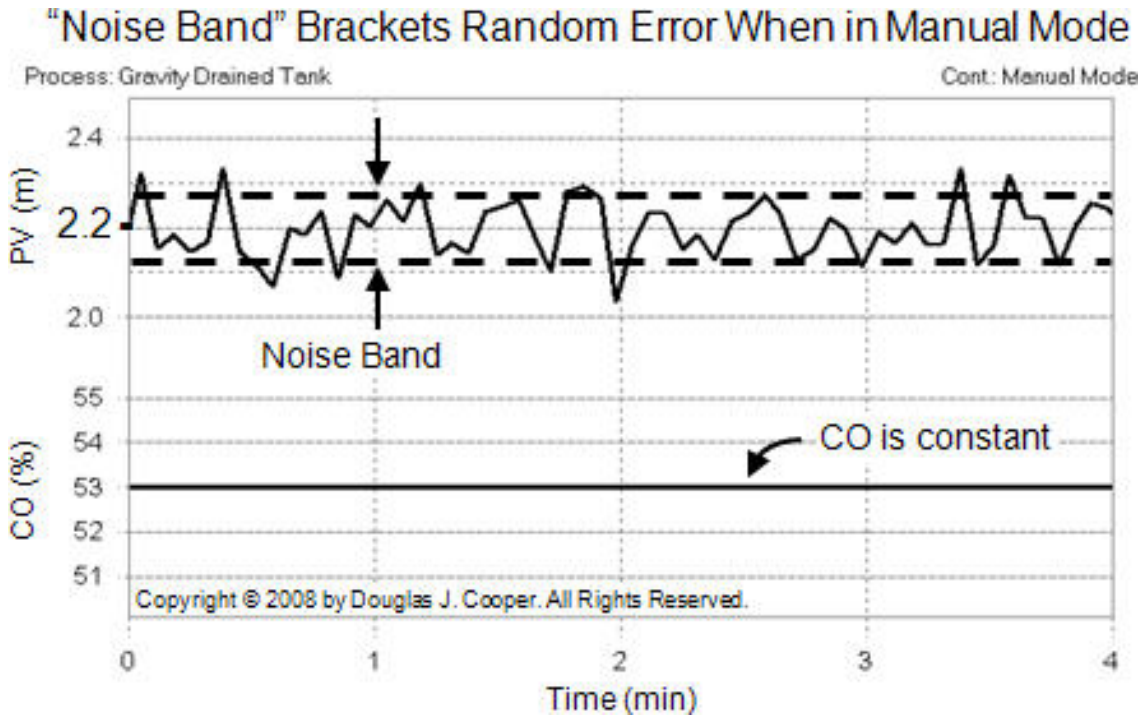
Noise Band Guides Test

When generating dynamic process data, it is important that the CO change is large enough and fast enough to force a response in the measured PV that clearly dominates the higher-frequency signal noise and lower-frequency random process variations.

One way to quantify the amount of noise and random variation for a process is with a noise band.

While there are formal approaches to defining a noise band, a simple approach as illustrated below () is to:

- collect data for a period of time when the CO is held constant (i.e. the controller is in manual).
- draw lines that bracket most of the data.
- the separation in the brackets is the “noise band.”



If the data is to be used for modeling, it is best practice to make changes in the CO that force the PV to move *at least* 5 times the noise band. In fact, some experts recommend that the PV moves 10 times the noise band to ensure a reliable result.

The noisy doublet example above did not meet this noise band rule, yet the fit was still reasonable. This is true in part because the process is a simulation and we could be

certain that no process disturbances occurred to corrupt the data.

Advances in software also enable us to extract more information from a data set, which is why the noise band rule mentioned above has grown smaller in recent years.

Step 4: Controller Tuning and Testing
In later articles, we use the above model data and move quickly through the range of PID controllers. We focus on disturbance rejection and highlight the differences and similarities with the set point tracking studies we presented for the heat exchanger process.

4) Software Modeling of Process Dynamics: Jacketed Stirred Reactor Case Study

Design Level of Operation for the Jacketed Stirred Reactor Process

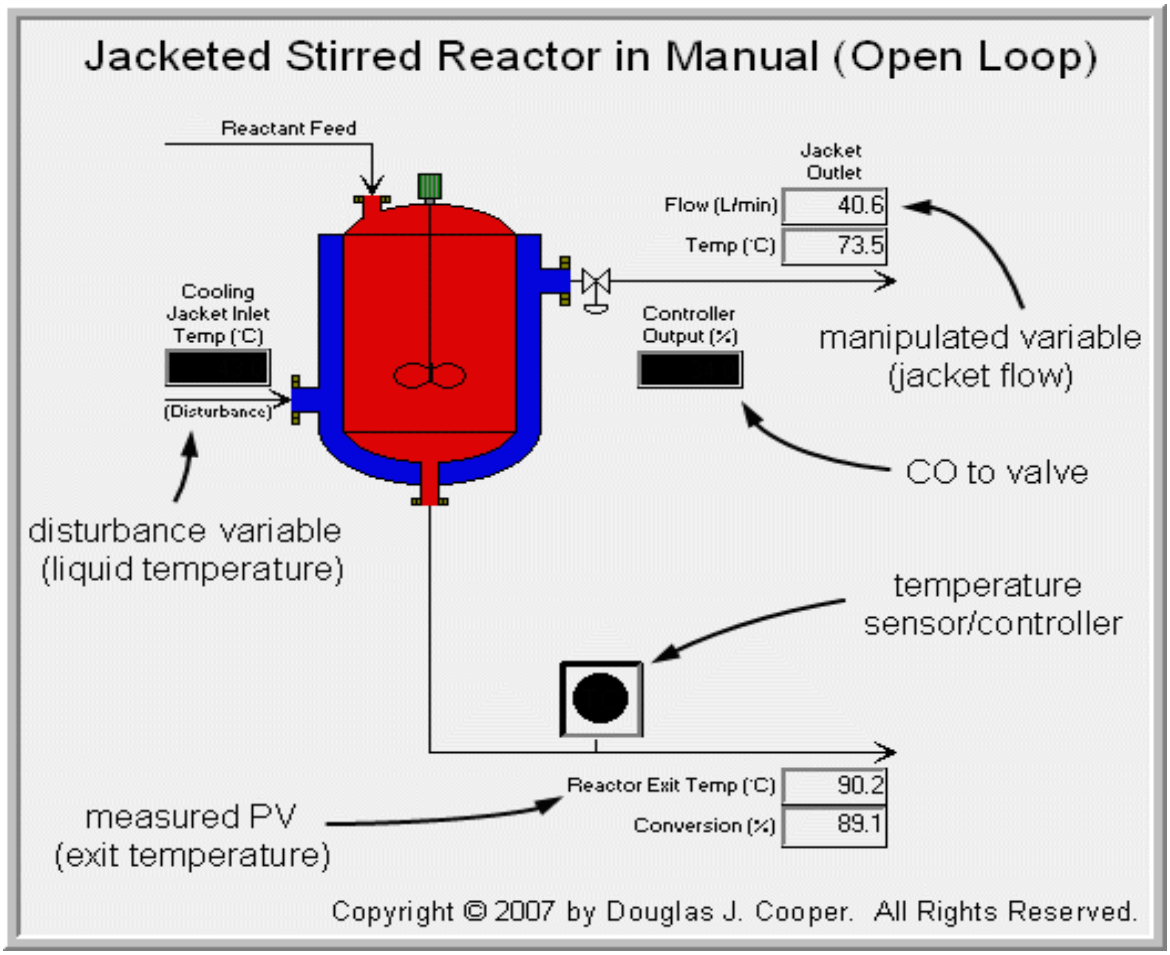
Like the heat exchanger and gravity drained tanks case studies, the jacketed stirred reactor is a self regulating processes. That is, the measured process variable (PV) naturally seeks a steady operating level if the controller output (CO) and major disturbance (D) are held constant for a sufficient length of time.

And like the heat exchanger and the gravity drained tanks, the jacketed stirred reactor process is actually a sophisticated simulation derived from first-principles theory and available in commercial software. Nevertheless, the methods and procedures we establish during these investigations are directly applicable to a broad range of industrial processes with streams comprised of liquids, gases, powders, slurries and melts.

The Jacketed Stirred Reactor

The process, shown below in manual mode, is often called a continuously stirred tank reactor (CSTR).

As labeled in the figure, a reactant feed stream enters the top of the vessel. A chemical reaction converts most of this feed into the desired product as the material passes through what is essentially a stirred tank. The stream exiting the bottom of the vessel includes the newly created product plus that portion of the feed that did not convert while in the vessel.



This well mixed reactor has additional considerations we use later in the discussion:

- The residence time, or overall flow rate of reactant feed plus product through the vessel, is constant.
- The chemical reaction that occurs is exothermic, which means that heat energy is released as feed converts to product.

The Function of the Cooling Jacket

The chemical reaction releases heat and this energy causes the temperature of the material in the vessel to rise. As temperature rises, the conversion of feed to product proceeds faster, leading to the release of even more heat.

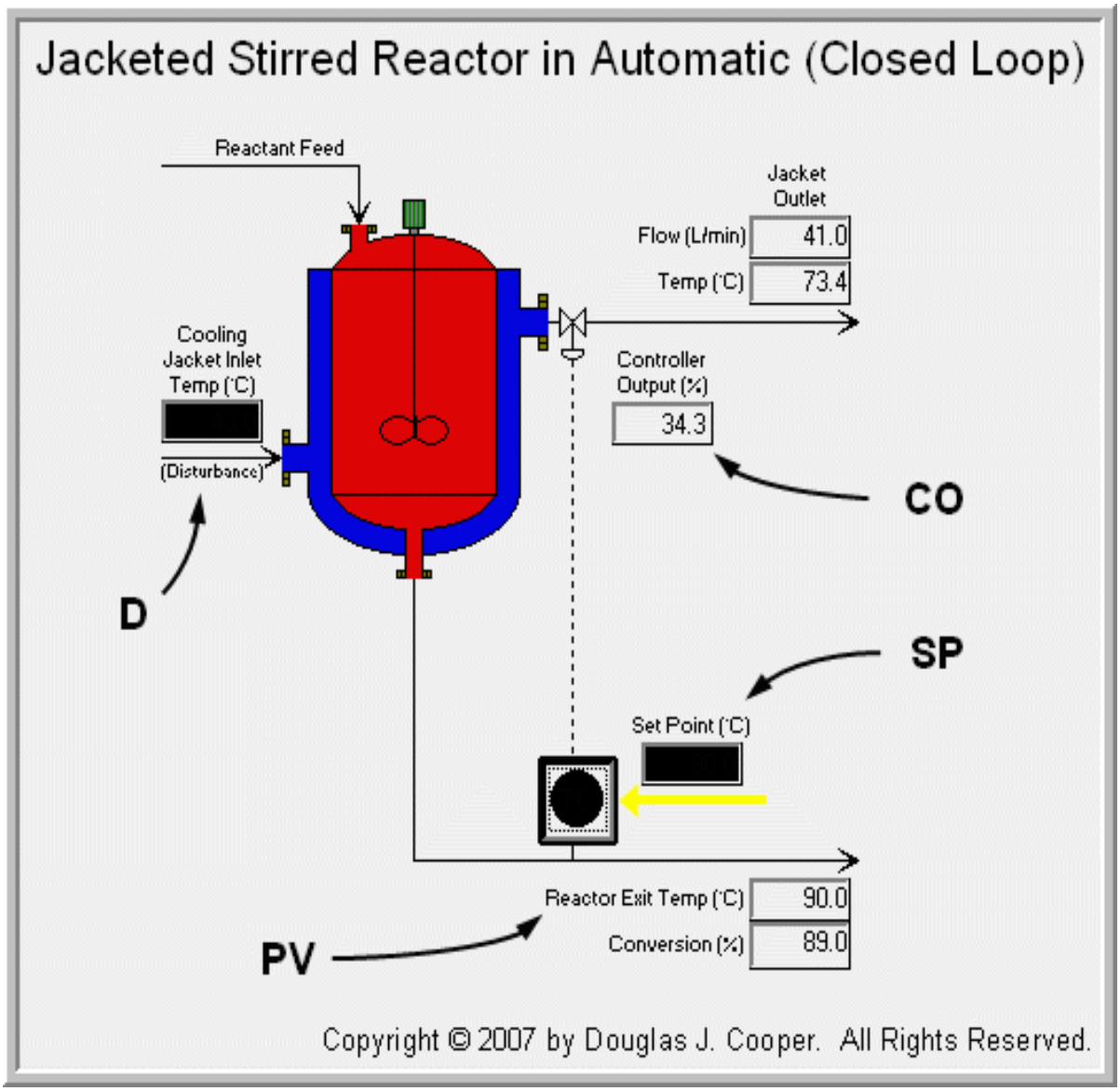
To stop the upward spiral of hotter temperatures increasing the rate of reaction that produces even more heat, the vessel is enclosed with a jacket (or outer shell). A cooling liquid flows through the jacket, collecting heat energy from the outer surface of the reactor vessel and carrying it away as the cooling liquid exits at the jacket outlet.

When the flow of cooling liquid through the jacket increases, more heat is removed. This lowers the reactor temperature, slowing the rate of the reaction, and thus decreasing the amount of feed converted to product during passage through the reactor.

When the flow of cooling liquid through the jacket decreases, some of the energy from the heat-producing reaction, rather than being carried away with the cooling liquid, accumulates in the vessel and drives the reactor temperature higher. The result is an

increased conversion of reactant feed to product.

As shown in the figure below, the flow rate of cooling liquid is adjusted with a valve on the cooling jacket outlet stream.



Control Conversion by Controlling Temperature
In this case study, we do not seek 100% conversion of reactant feed to product. Rather, our operating specification is a precise 89% conversion.

Because the reactor has a constant residence time, the amount of heat energy released inside the vessel is directly related to the percent of feed converted to product. By controlling the temperature in the reactor, we can maintain the percent conversion to the desired value.

The vessel is well mixed, so the bulk temperature inside the reactor is about the same as the temperature flowing out the exit stream. Thus, as shown in the process graphic above, we place a temperature sensor in the stream at the bottom of the vessel and our me Practitioner's Note: as discussed in this article, there can be benefits to measuring and

controlling a related variable that is linked to the actual process variable of interest. In this reactor application, for example, a temperature sensor is significantly less expensive to purchase, install and maintain relative to the alternative of an online analyzer that directly measures chemical composition. Temperature sensors are also rugged and reliable, have a rapid response time, and provide a high accuracy and precision compared to process analyzers. If it provides sufficiently useful information, a "fast, cheap and easy" sensor is the smart choice.asured process variable (PV) becomes reactor exit temperature.

As the figure reveals, we can achieve our desired 89% conversion by maintaining the reactor exit stream temperature at 90 ^{o}C. From an operational view, this means the reactor exit temperature will have a fixed set point (SP) value of 90 ^{o}C.

During bump testing, operations personnel tell us that we may briefly move the reactor exit temperature up and down by 2 ^{o}C, but they strongly discourage anything more. Thus:

- design PV and SP = 90 ^{o}C with approval for brief dynamic testing of ±2 ^{o}C

The Disturbance
Because we seek to hold conversion to a constant 89% at all times (which is achieved by holding reactor exit stream temperature at 90 ^{o}C), disturbance rejection becomes our main controller design concern.

The major disturbance in this jacketed stirred reactor is the result of an unfortunate design. Specifically, the temperature of the cooling liquid entering the jacket changes over time (this situation is surprisingly more common in industrial installations than one might first believe).

As the temperature of the cooling liquid entering the jacket changes, so does its ability to remove heat energy. Warm liquid removes less energy than cool liquid when flowing through the jacket at the same rate.

So "disturbance rejection" in this case study means minimizing the impact of cooling liquid temperature changes on reactor operation.

As labeled in the process graphic, the temperature of the cooling liquid is normally at about 43 ^{o}C. On occasion, however, this temperature can climb, sometimes rather rapidly, to as high as 50 ^{o}C.

We will design for the worst-case scenario and test our controller when the cooling liquid temperature (our disturbance, D) spikes from 43 ^{o}C up to 50 ^{o}C in a single step. Thus:

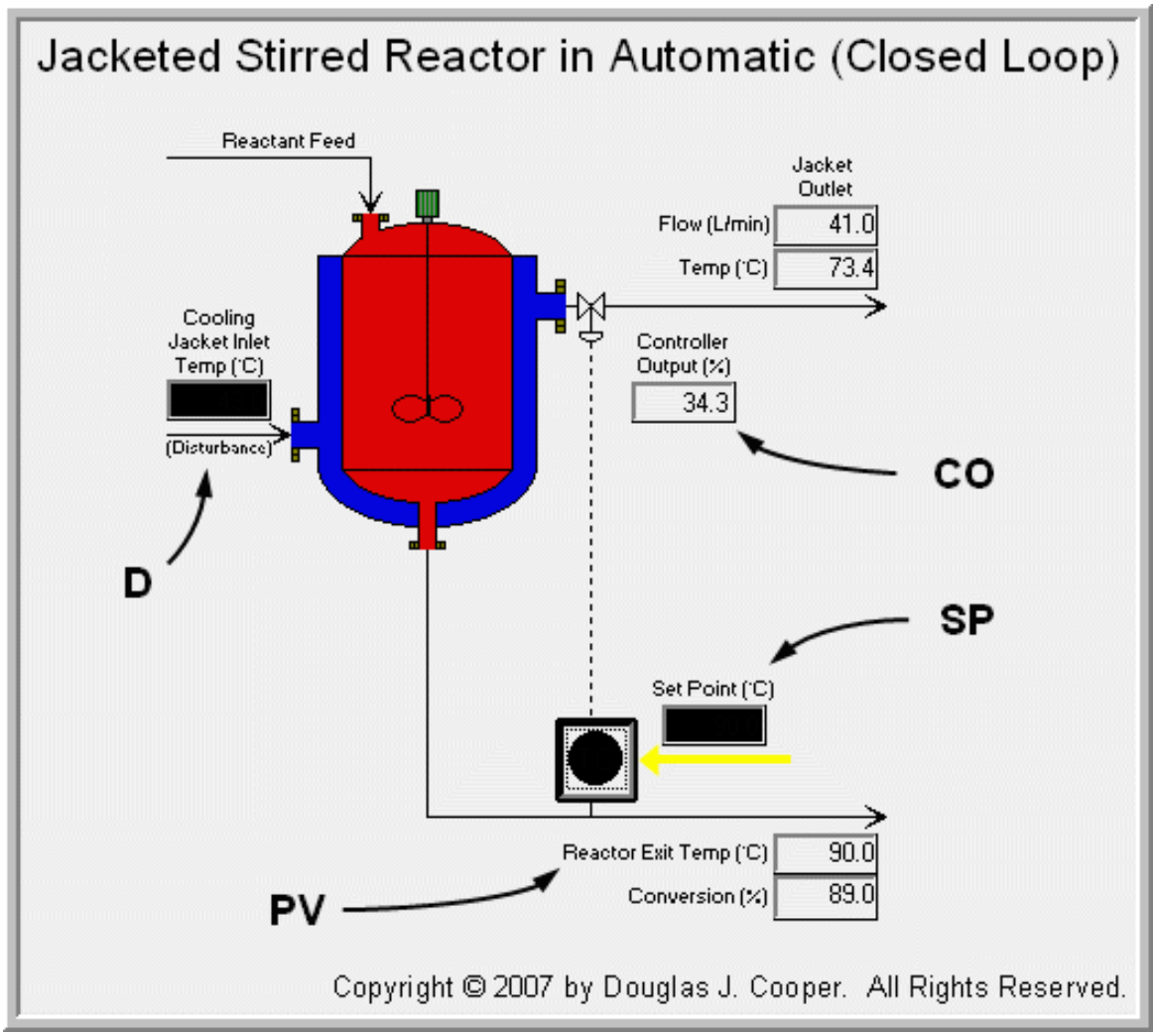
- design D = 43 ^{o}C with spikes up to 50 ^{o}C

The Design Level of Operation (DLO)
The first step of the our four step design and tuning recipe is to establish the design level of operation. In the discussion above, we have completed step 1 by establishing the DLO as:

- Design PV and SP = 90 ^{o}C with approval for brief dynamic testing of ±2 ^{o}C
- Design D = 43 ^{o}C with spikes up to 50 ^{o}C

Modeling the Dynamics of the Jacketed Stirred Reactor with Software

The control objective of the jacketed reactor case study, shown below and discussed in detailed in this article, is disturbance rejection. More specifically, we seek a controller design that will minimize the impact on reactor operation when the temperature of the liquid entering the cooling jacket changes.



As labeled in the graphic, the important variables for this case study include:

CO = signal to valve that adjusts cooling jacket liquid flow rate (controller output, %)
PV = reactor exit stream temperature (measured process variable, ^{o}C)
SP = desired reactor exit stream temperature (set point, ^{o}C)
D = temperature of cooling liquid entering the jacket (major disturbance, ^{o}C)

Controller Design and Tuning Recipe

As with any control project, we follow our controller design and tuning recipe:

1. Establish the design level of operation (the normal or expected values for set point and major disturbances)

2. Bump the process and collect controller output (CO) to process variable (PV) dynamic process data around this design level
3. Approximate the process data behavior with a first order plus dead time (FOPDT) dynamic model
4. Use the model parameters from step 3 in rules and correlations to complete the controller design and tuning.

Step 1: Design Level of Operation (DLO)
The details and discussion for our DLO are presented in this article and are summarized:
- Design PV and SP = 90 ^{o}C with approval for brief dynamic testing of ±2 ^{o}C
- Design D = 43 ^{o}C with spikes up to 50 ^{o}C

Step 2: Collect Data at the DLO
The point of bumping a process is to generate and collect dynamic process data. To be of value, this data should be collected near our DLO and must clearly reveal the cause and effect relationship between how changes in the CO signal force a response in the measured PV.

• *Data Must Be Wire Out to Wire In*
The data must be collected from the controller's viewpoint, since the controller will be making all decisions once in automatic mode. The controller only knows about the state of the process from the PV signal arriving on the "wire in" from the sensor. It can only impact the process with the CO signal it sends on the "wire out" to the final control element (e.g., a valve). All devices, mechanisms and instruments that affect the signal in the complete "wire-out to wire-in" CO to PV loop must be accounted for in the recorded data.

• *Process Should Be Steady*
To further isolate the pure cause and effect relationship, we should wait until the CO, PV and major disturbances have settled out and are reasonably constant before bumping the process. This provides us confidence that observed PV responses during the dynamic test are a direct result of the CO bumps. In the perfect world, this steady state will be at (or at least near) our DLO.

• *Center Data Around the DLO*
Like most processes with streams comprised of gases, liquids, powders, slurries and melts, the jacketed stirred reactor displays a nonlinear or changing behavior as operating level changes. In fact, it is this nonlinear character that leads us to specify a design level of operation in the first place. To the extent possible in our manufacturing environment, we should collect our process data centered around the DLO. A model fit of such data (step 3) will then average out the nonlinear effects and provide a controller equally balanced to address process movement both up and down.

• *The PV Response Should Dominate the Noise*
The CO bump must be far enough and fast enough to force a clear "cause and effect" response in the PV. And this PV response must clearly dominate the measurement noise.

Step 3: Fit a FOPDT Dynamic Model to Process Data Using Software
Two popular open loop (manual mode) methods for generating dynamic process response (bump test) data around the DLO include the step test and the doublet test. The primary disturbance (D) of interest in this study is cooling jacket inlet temperature. Not shown in

the plots below is that D is steady at its design value of 43 ^{o}C during the bump tests.

Practitioner's Note: While the process graphic above shows the jacketed stirred reactor in automatic mode (closed loop), the step and doublet tests presented below are performed when the controller is in manual mode (open loop).

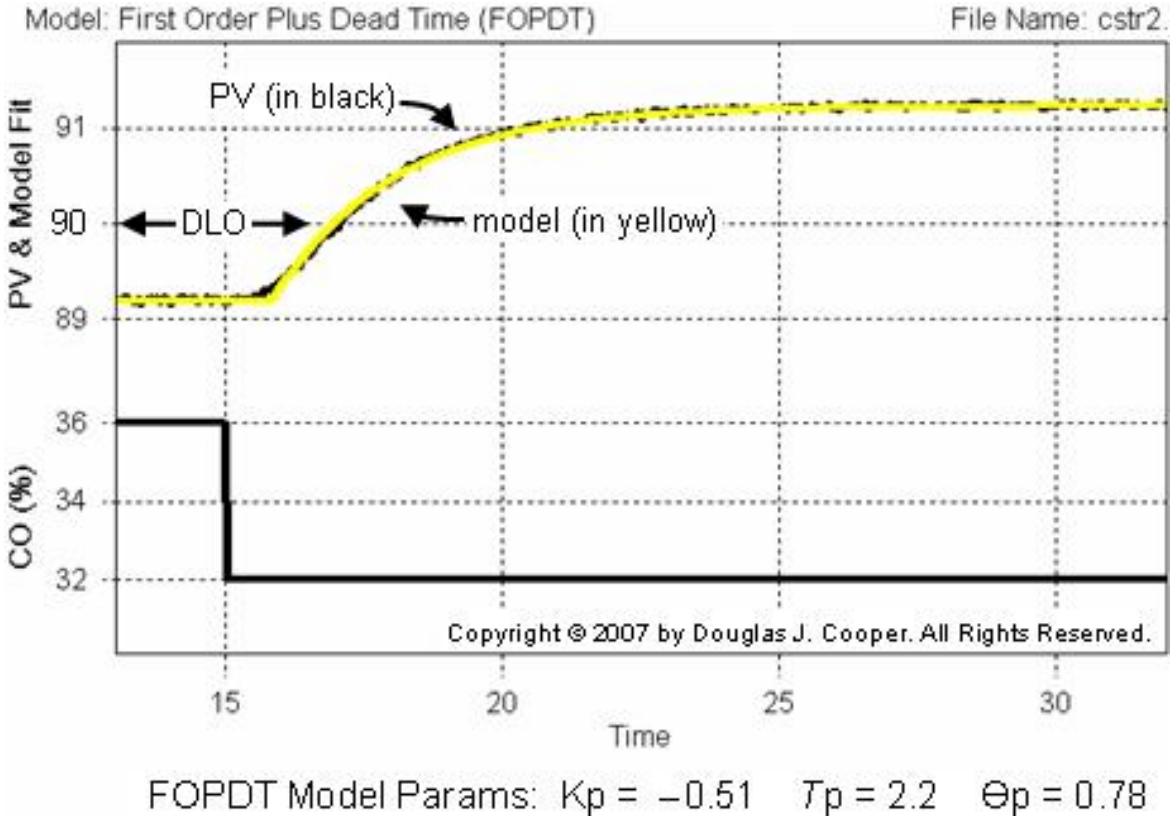
◊ ***Step Test***

Step tests have value because we can analyze the plot data by hand to compute the first order plus dead time (FOPDT) model parameters Kp, *T*p and Θp. It is important that we have proper values for these model parameters because they are used in the rules and correlations of step 4 from the recipe to complete the controller design and tuning (examples here and here).

One disadvantage of a step test is that it is conducted in manual mode. As shown in the plot below, the PV is away from the DLO for an extended period of time. This can create profitability concerns from off-spec production and perhaps even safety concerns if constraints become violated.

To collect process data in manual mode that will "average out" the nonlinear effects around our design level of operation, we start the test at steady state with the PV on one side of the DLO. Then, as shown in the plot below, we step the CO so that the measured PV moves across to settle on the other side of the DLO.

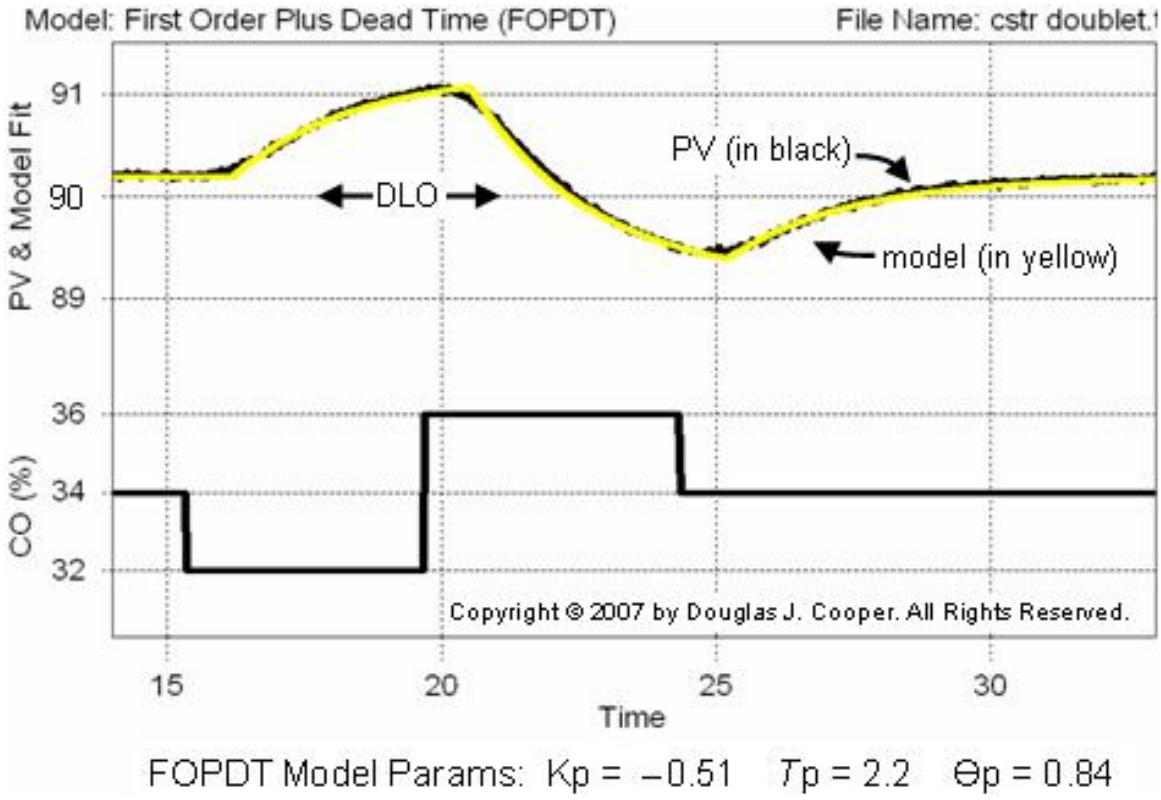
We acknowledge that it may be unrealistic to attempt such a precise step test in some production environments. But we should understand that the motivation is to obtain an approximating FOPDT model of the dynamic response behavior that averages out the changing nature of the process as it moves across the expected range of operation.



A FOPDT model fit of the CO to PV data using commercial software is shown as the yellow trace in the plot above. It clearly tracks the measured PV data quite closely. The computed model parameters are listed below the plot.

Such results can be obtained in a few mouse clicks, and the visual confirmation that "model equals data" gives us confidence that we indeed have a meaningful description of the dynamic process behavior. This, in turn, gives us confidence that the subsequent controller design will provide the performance we desire.

◊ ***Doublet Test***
A doublet test, again performed below in manual mode, is two CO pulses made in rapid succession and in opposite direction. The second pulse is implemented as soon as the process has shown a clear response to the first pulse that dominates the noise in the PV. It is not necessary to wait for the process to respond to steady state for either pulse.



A FOPDT model fit of the CO to PV data using commercial software is shown as the yellow trace in the plot above. It also tracks the measured PV data quite closely.

A doublet test offers attractive benefits, including that:
- it starts from and quickly returns to the DLO,
- it produces data above and below the DLO to "average out" the nonlinear effects,
- the PV stays close to the DLO, minimizing off-spec production and safety concerns.

For these reasons, a doublet is preferred by many practitioners for open loop testing.

The FOPDT Model
By approximating the dynamic behavior of the jacketed reactor process with a first order plus dead time (FOPDT) model, we quantify those essential features that are fundamental

to control.

Aside: the FOPDT dynamic model is a linear, ordinary differential equation describing how PV(t) responds over time to changes in CO(t):

$$Tp\frac{dPV(t)}{dt} + PV(t) = Kp \cdot CO(t - \theta p)$$

Where in this case study: t [=] min, PV(t) [=] °C, CO(t – Θp) [=] %

The FOPDT model describes how the PV will respond to a change in CO with the:
- Process gain, Kp, that describes the direction and how far the PV will travel,
- Time constant, *T*p, that states how fast the PV moves after it begins its response,
- Dead time, Θp, that is the delay that occurs from when CO is changed until when the PV begins its response.

Based on both the step and doublet test model fits shown above, we conclude that, when operating near the DLO, the jacketed reactor process dynamics are described:
- Kp = – 0.5 ^{o}C/%
- *T*p = 2.2 min
- Θp = 0.8 min

We will use these parameter values in our subsequent control studies of the jacketed stirred reactor.

Exploring the FOPDT Model With a Parameter Sensitivity Study

Quantifying Dynamic Process Behavior

Step 3 of the PID controller design and tuning recipe is to approximate process bump test data by fitting it with a first order plus dead time (FOPDT) dynamic model.

Data from a proper bump test is rich in dynamic information that is characteristic of our controller output (CO) to measured process variable (PV) relationship. A FOPDT model is a convenient way to quantify (assign numerical values to) key aspects of this CO to PV relationship for use in controller design and tuning.

In particular, when the CO changes:
- Process gain, Kp, describes the direction and how far the PV moves
- Time constant, *T*p, describes how fast the PV responds
- Dead time, Θp, describes how much delay occurs before the PV first begins to move

In the investigation below, we isolate and study each FOPDT parameter individually to establish its contribution to the model response curve. By the end of the study, we (hopefully) will have established the power and utility of this model in describing a broad range of common process behaviors.

Aside: the FOPDT dynamic model is a linear, ordinary differential equation describing how PV(t) responds over time to changes in CO(t):

$$Tp\frac{dPV(t)}{dt} + PV(t) = Kp \cdot CO(t - \theta p)$$

Where in this case study: t [=] min, PV(t) [=] °C, CO(t – Θp) [=] %

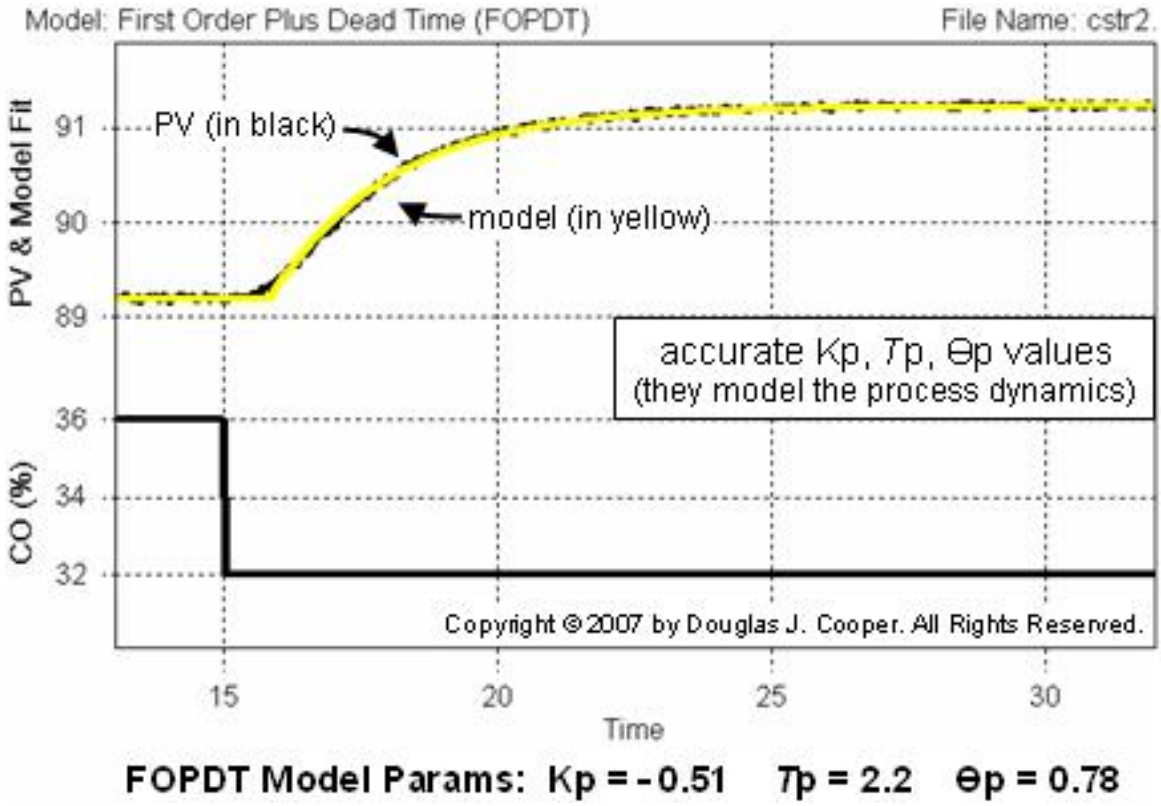
The Base Case Model

Each plot in this article shows the identical bump test data that was used in the jacketed stirred reactor study presented here. The CO step and PV response data are shown as black curves. The response of the computed FOPDT model is shown as a yellow curve.

The parameter values used to compute all of the FOPDT model responses are listed at the bottom of each plot. Throughout the study, the model parameter units are:

Kp [=] ^{o}C/%, *T*p [=] min, Θp [=] min

The plot below shows a FOPDT model fit as computed by commercial software. We call this fit "good" or "descriptive" because we can visually see that the computed yellow model closely tracks the measured PV data.

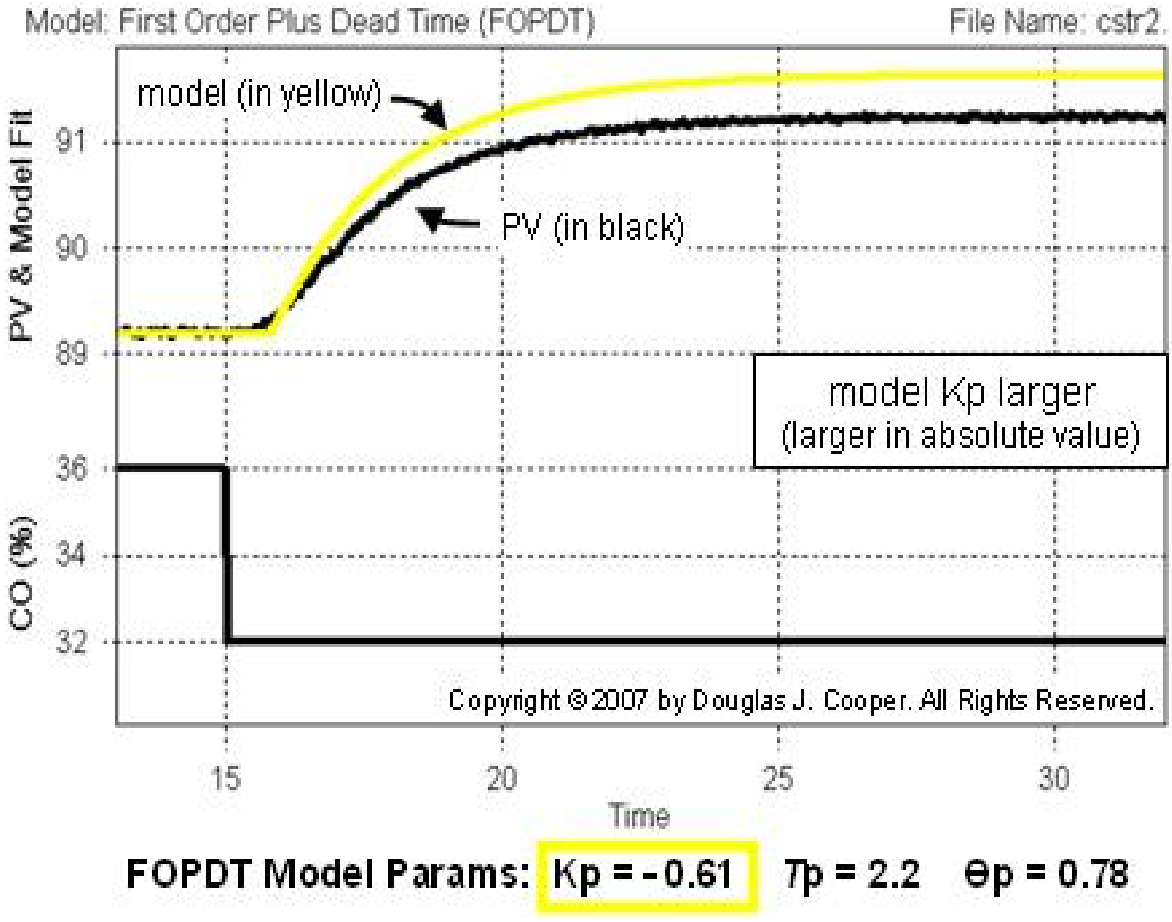


If no significant disturbances occurred during the bump test to corrupt the response, then we can have confidence in our data set. And if the FOPDT model reasonably describes the data, as it does above, then a controller designed and tuned using the parameters from the model will perform as we expect. This, in fact, is the very reason we fit the model in the first place.

Kp is the "How Far" Variable
Process gain, Kp, describes the direction and how far the PV moves in response to a change in CO.

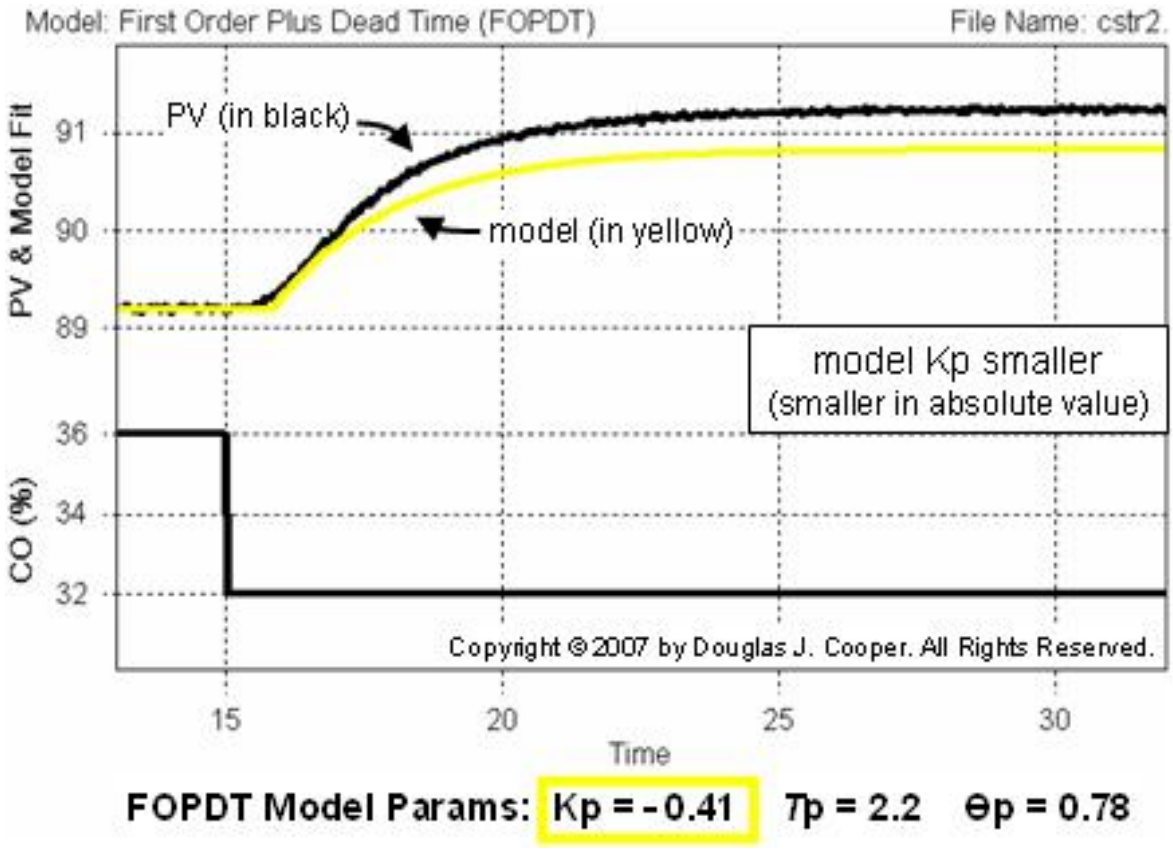
The direction is established with the sign of Kp. We recognize that in this case, Kp must be negative because when the CO moves in one direction, the PV moves in the other. We keep the negative sign throughout this study and focus on how the absolute size of Kp impacts model response behavior.

Below we compute and display the FOPDT model response using the base case time constant, *T*p, and dead time, Θp. The yellow response curve below is different because the process gain used in the model has been increased in absolute size from the base case value of Kp = – 0.51 up to Kp = – 0.61.



With a larger model Kp, the "how far" response of the yellow curve clearly increases (more discussion on computing Kp here). Though perhaps not easy to see, the "how fast" and "how much delay" aspects of the response have not been impacted by the change in Kp.

Below we compute the yellow FOPDT model response using the base case *T*p and Θp, only this time, the process gain used in the model has been decreased from the base case Kp = – 0.51 down to Kp = – 0.41 (decreased in absolute size).

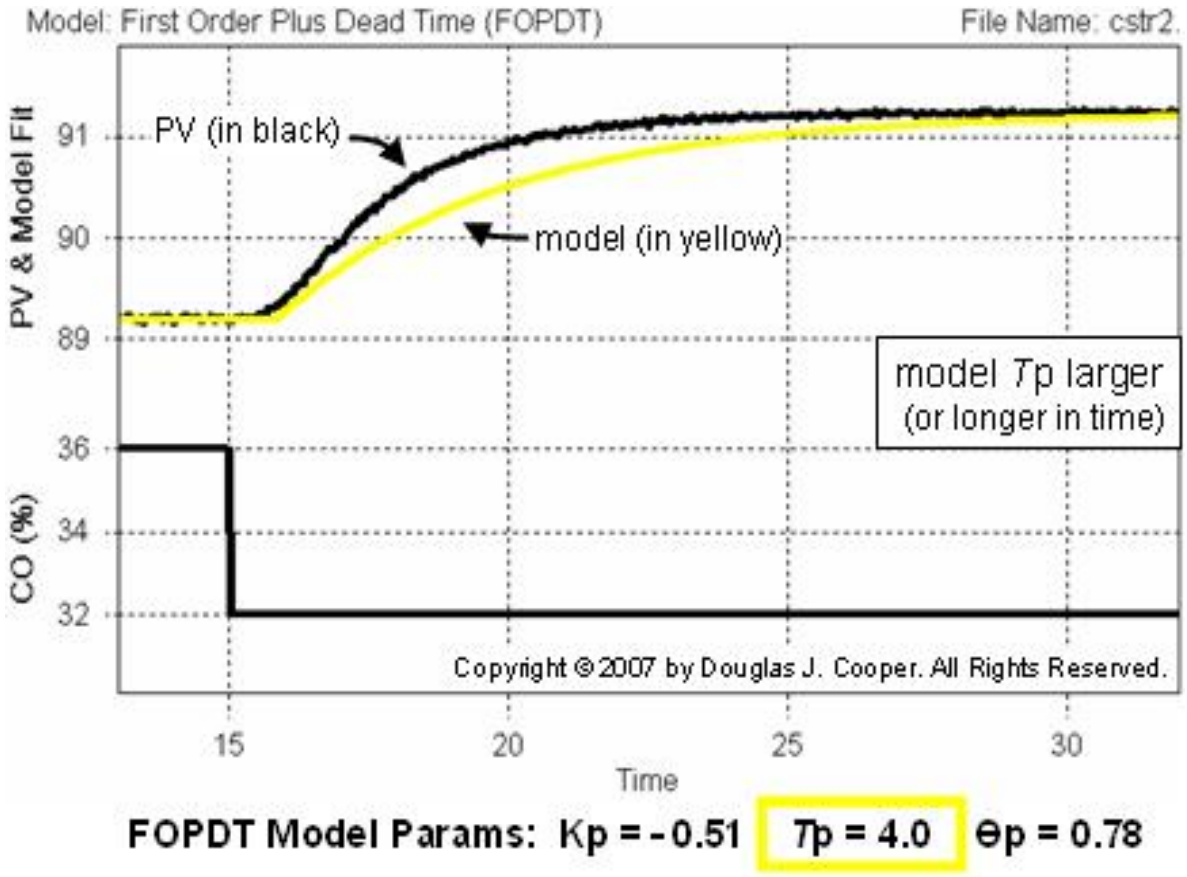


With a smaller Kp, we see that the computed model curve undershoots the PV data. We thus establish that process gain, Kp, dictates how far the PV travels in response to changes in CO.

***T*p is the "How Fast" Variable**

Process time constant, *T*p, describes how fast the PV moves in response to changes in CO. The plot at the top of this article establishes that when *T*p = 2.2, the speed of response of the model matches that displayed by the PV data.

Below we compute and display the FOPDT model response curve using the base case gain, Kp, and dead time, Θp. The curve is different because the time constant used to compute the yellow model response has been increased from the base case *T*p = 2.2 up to *T*p = 4.0.



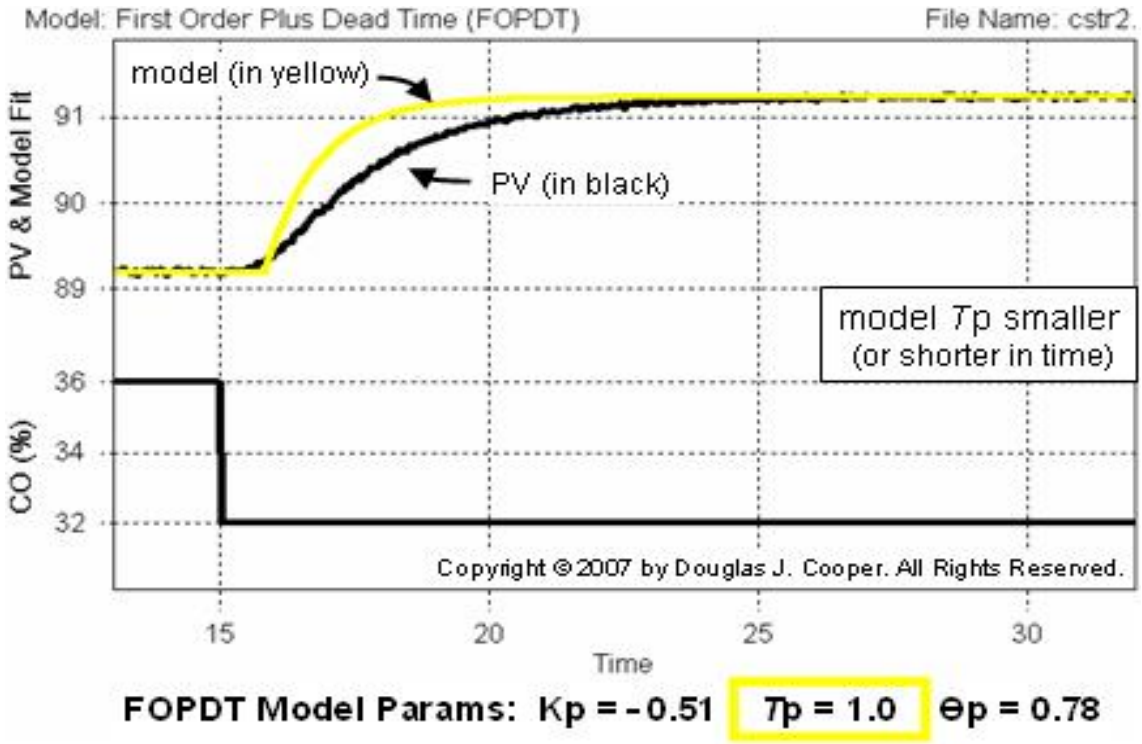
Recall that for step test data like that above, *T*p is the time that passes from when the PV shows its first clear movement until when it reaches 63% of the total □PV change that is going to occur (more discussion on computing *T*p here).

Since *T*p marks the passage of time, a larger time constant describes a process that will take longer to complete its response. Put another way, a process with a larger *T*p moves slower in response to changes in CO.

A very important observation in the above plot is that the yellow model curve, even though it moves more slowly than in the base case, ultimately reaches and levels out at the black PV data line. This is because the model response was generated using the base case Kp = – 0.51.

If we accept that the size of Kp alone dictates "how far" a response will travel, then this result is consistent with our understanding.

Below we show the yellow FOPDT model response using the base case Kp and Θp. The response curve is different because the model computation uses a process time constant that has been decreased from the base case *T*p = 2.2 down to *T*p = 1.0.



A smaller *T*p means a process will take a shorter amount of time to complete its response. Thus, a process with a smaller time constant is one that moves faster in response to CO changes.

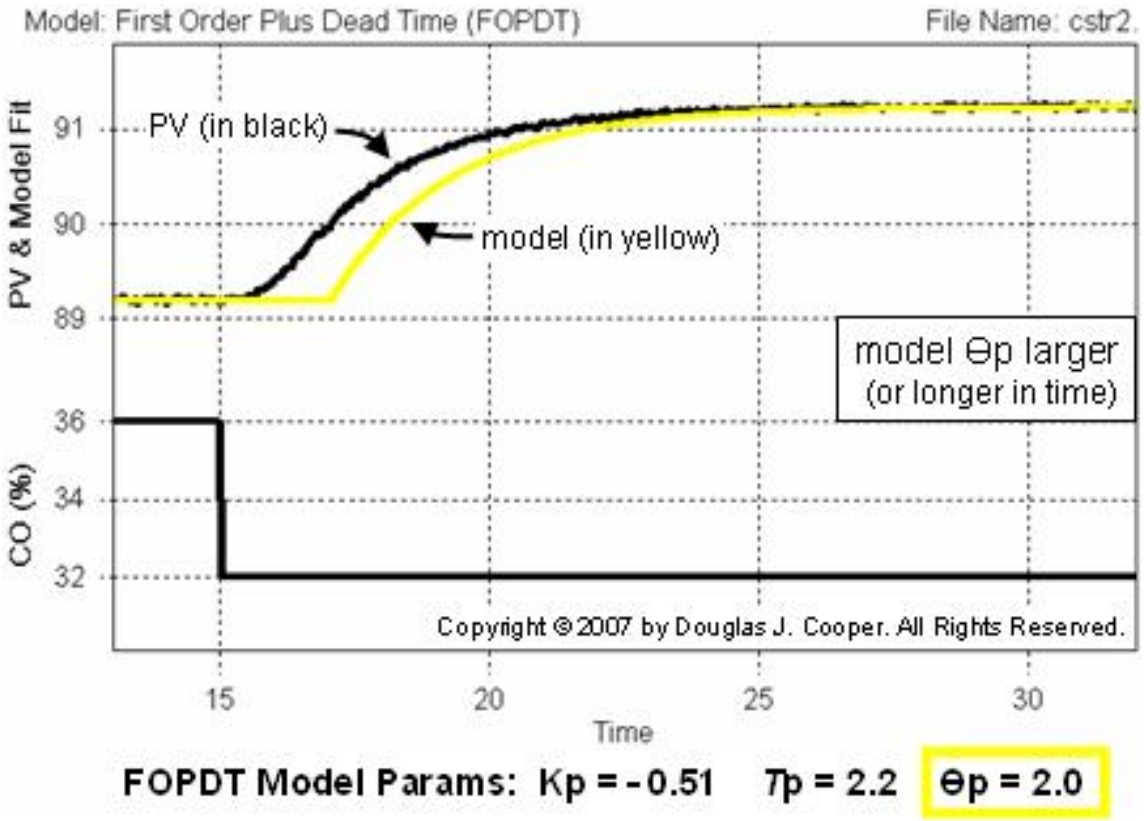
Again note that the yellow model response curve levels out at the black PV data line because the base case value of Kp = – 0.51 was used in computing "how far" the model response will travel.

Θp is the "With How Much Delay" Variable
If we examine all of the plots above we can observe that the "how much delay" behavior of the yellow model response curves are all the same, regardless of changes in Kp or *T*p.

That is, the time delay that passes from when the CO step is made until when the model shows its first clear response to that action is the same regardless of the process gain and/or time constant values used (more discussion on computing Θp here).

This is because it is Θp alone that dictates the "how much delay" model response behavior.

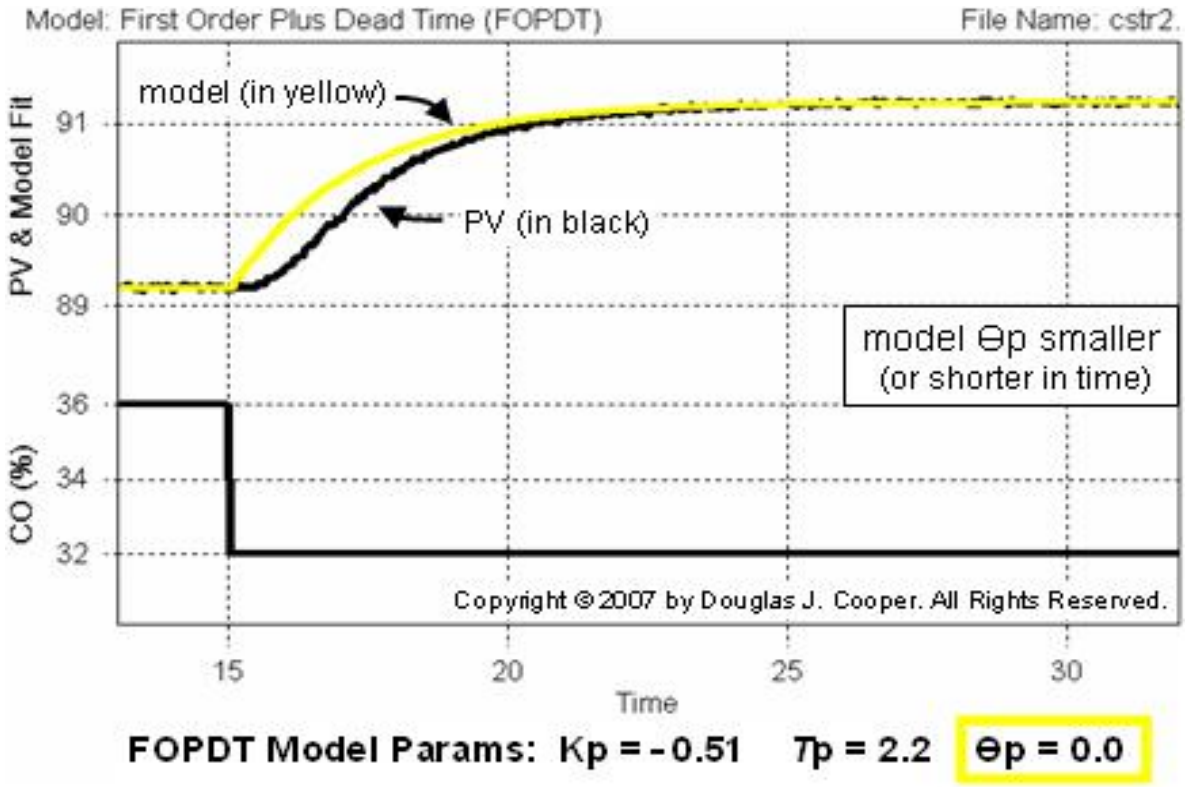
To illustrate, below we compute and display the FOPDT model response curve using the base case process gain, Kp, and time constant, *T*p. The yellow response is different in the plot below because the model dead time has been increased from the base case value of Θp = 0.78 up to Θp = 2.0.



As the plot above shows, increasing dead time simply shifts the yellow model response curve without changing its shape in any fashion. A larger dead time means a longer delay before the response first begins.

The "how far" and "how fast" shape remains identical in the model response curve above because it is Kp and *T*p that dictate these behaviors.

Below we show the yellow FOPDT model response using the base case Kp and *T*p. The response curve is different because the model computation uses a dead time value that has been decreased from the base case Θp = 0.78 down to Θp = 0.



As shown in the plot above, if we specify no delay (i.e., Θp = 0), then the computed yellow model response curve starts to respond the instant that the CO step is made.

Conclusions

The above study illustrates that each parameter of the FOPDT model describes a unique characteristic of dynamic process behavior:

- Process gain, Kp, describes the direction and how far the PV moves.
- Time constant, Tp, describes how fast the PV responds.
- Dead time, Θp, describes how much delay occurs before the PV first begins to move.

Knowing how a PV will behave in response to a change in CO is fundamental to controller design and tuning.

For example, a controller must be tuned to make small corrective actions if every CO change causes a large PV response (if Kp is large). And if Kp is small, then the controller must be tuned to make large CO corrective actions whenever the PV starts to drift from set point.

If the time constant of a process is long (if a process responds slowly), then the controller must not be issuing new corrective actions in rapid fire. Rather, the controller must permit previous CO actions to show their impact before computing further actions.

We see exactly how the FOPDT model parameters are used as a basis for controller design and tuning in a host of articles in this e-book.

II. PID Controller Design and Tuning (by Doug Cooper)

5) Process Control Preliminaries

Design and Tuning Recipe Must Consider Nonlinear Process Behavior

Processes with streams comprised of gases, liquids, powders, slurries and melts tend to exhibit variations in behavior as operating level changes. This, in fact, is the very nature of a nonlinear process. For this reason, our recipe for controller design and tuning begins by specifying our design level of operation.

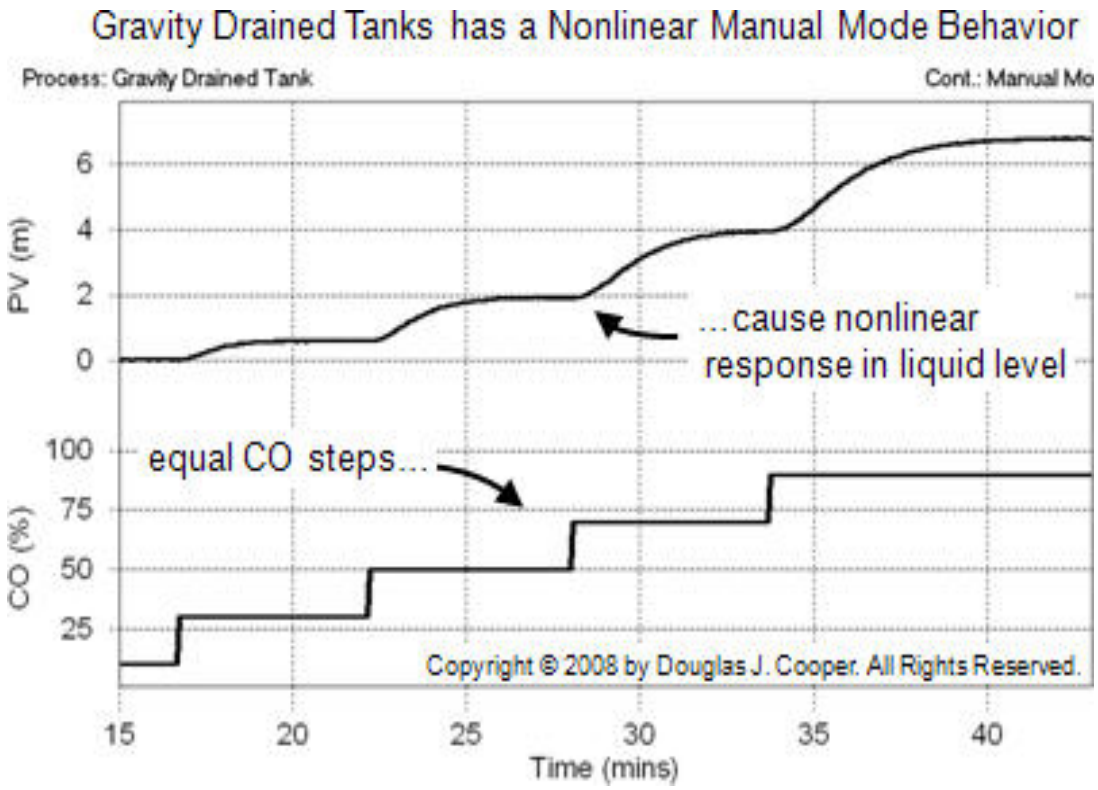
Controller Design and Tuning Recipe:

1. Establish the design level of operation (DLO), which is the normal or expected values for set point and major disturbances.
2. Bump the process and collect controller output (CO) to process variable (PV) dynamic process data around this design level.
3. Approximate the process data behavior with a first order plus dead time (FOPDT) dynamic model.
4. Use the model parameters from step 3 in rules and correlations to complete the controller design and tuning.

Nonlinear Behavior of the Gravity Drained Tanks

The dynamic behavior of the gravity drained tanks process is reasonably intuitive. Increase or decrease the inlet flow rate into the upper tank and the liquid level in the lower tank rises or falls in response.

One challenge this process presents is that its dynamic behavior is nonlinear. That is, the process gain, Kp; time constant, τp; and/or dead time, Θp; changes as operating level changes. This is evident in the open loop response plot below.

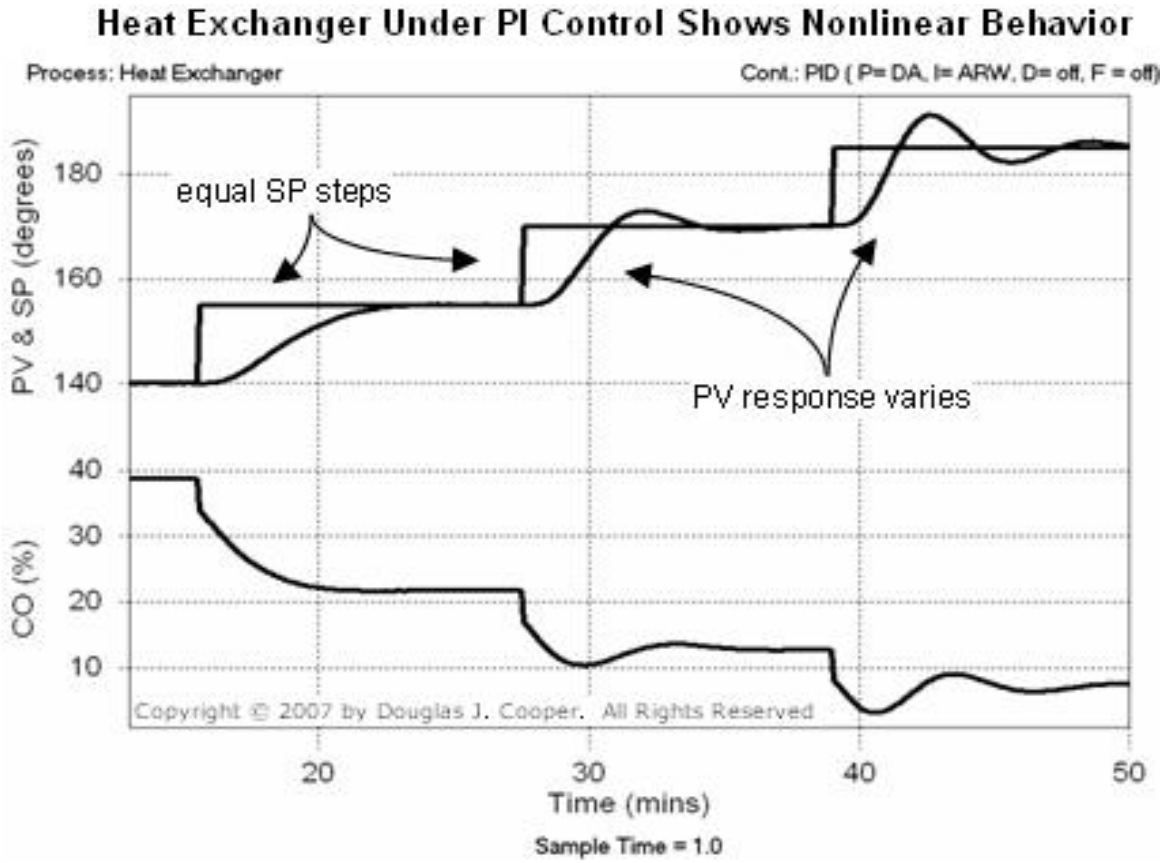


As shown above, the CO is stepped in equal increments, yet the response behavior of the PV changes as the level in the tank rises. The consequence of nonlinear behavior is that a controller designed to give desirable performance at one operating level may not give desirable performance at another level.

Nonlinear Behavior of the Heat Exchanger

Nonlinear process behavior has important implications for controller design and tuning. Consider, for example, our heat exchanger process under PI control.

When tuned for a moderate response as shown in the first set point step from 140 °C to 155 °C in the plot below, the process variable (PV) responds in a manner consistent with our design goals. That is, the PV moves to the new set point (SP) reasonably quickly but does not overshoot the set point.



The consequence of a nonlinear process character is apparent as the set point steps continue to higher temperatures. In the third set point step from 170 °C to 185 °C, the same controller that had given a desired moderate performance now produces a PV response with a clear overshoot and some oscillation.

Such a change in performance with operating level may be tolerable in some applications and unacceptable in others. As we discuss in this article, “best” performance is something we judge for ourselves based on the goals of production, capabilities of the process, impact on down stream units and the desires of management

Nonlinear behavior should not catch us by surprise. It is something we can know about our process in advance. And this is why we should choose a design level of operation as a first step in our controller design and tuning procedure.

Step 1: Establish the Design Level of Operation (DLO)
Because, as shown in the examples above, processes have process gain, Kp; time constant, *T*p; and/or dead time, Θp values that change as operating level changes, and these FOPDT model parameter values are used to complete the controller design and tuning procedure, it is important that dynamic process test data be collected at a pre-determined level of operation.

Defining this design level of operation (DLO) includes specifying where we expect the set point (SP) and measured process variable (PV) to be during normal operation, and the range of values the SP and PV might typically assume. This way we know where to explore the dynamic process behavior during controller design and tuning.

The DLO also considers our major disturbances (D). We should know the normal or typical values for our major disturbances. And we should be reasonably confident that the disturbances are quiet so we may proceed with a bump test to generate and record dynamic process data.

Step 2. Collect Dynamic Process Data Around the DLO
The next step in our recipe is to collect dynamic process data as near as practical to our design level of operation. We do this with a bump test, where we step or pulse the CO and collect data as the PV responds.

It is important to wait until the CO, PV and D have settled out and are as near to constant values as is possible for our particular operation before we start a bump test. The point of bumping a process is to learn about the cause and effect relationship between the CO and PV.

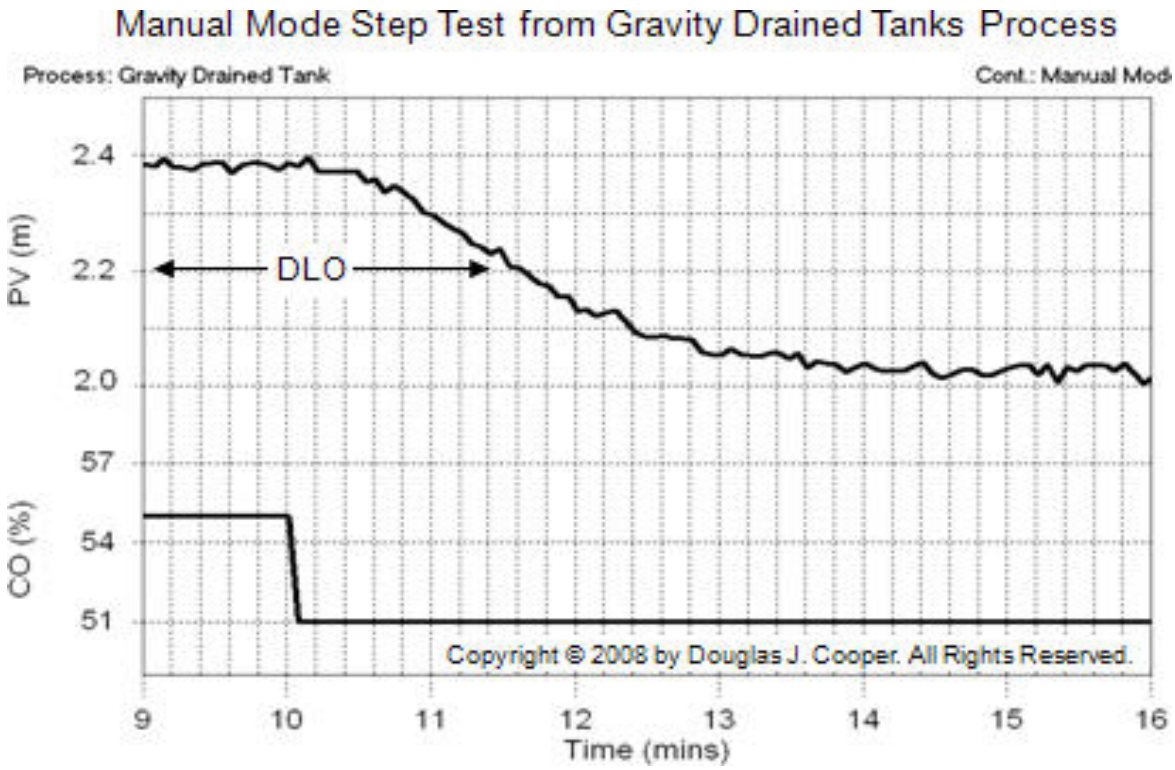
With the process at steady state, we are starting with a clean slate. As the PV responds to the CO bumps, the dynamic cause and effect behavior is isolated and evident in the data. On a practical note, be sure the data capture routine is enabled before the initial bump is implemented so all relevant data is collected.

Two popular open loop (manual mode) methods are the step test and the doublet test.

For either method, the CO must be moved far enough and fast enough to force a response in the PV that dominates the measurement noise.

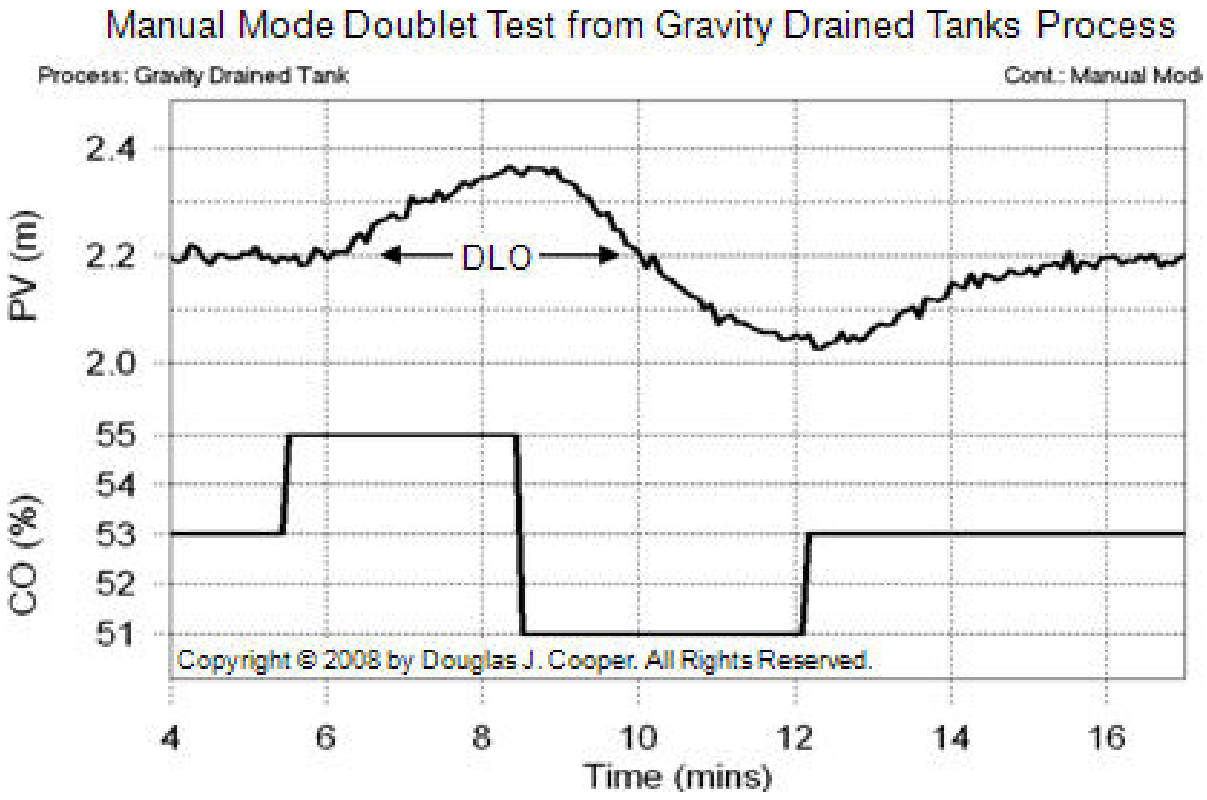
Also, our bump should move the PV both above and below the DLO during testing. With data from each side of the DLO, the model (step 3) will be able to average out the nonlinear effects as discussed above.

• *Step Test*
To collect data that will “average out” to our design level of operation, we start the test at steady state with the PV on one side of (either above or below) the DLO. Then, as shown in the plot below, we step the CO so that the measured PV moves across to settle on the other side of the DLO.



We can either start high and step the CO down (as shown above), or start low and step the CO up. Both methods produce dynamic data of equal value for our design and tuning recipe.

• *Doublet Test*
A doublet test, as shown below, is two CO pulses performed in rapid succession and in opposite direction. The second pulse is implemented as soon as the process has shown a clear response to the first pulse that dominates the noise in the PV. It is not necessary to wait for the process to respond to steady state for either pulse.



The doublet test offers attractive benefits, including that it starts from and quickly returns to the DLO, it produces data both above and below the design level to "average out" the nonlinear effects, and the PV always stays close to the DLO, thus minimizing off-spec production. Such data does require commercial software for model fitting, however.

Step 3: Fit a FOPDT dynamic model to Process Data

In fitting a first order plus dead time (FOPDT) model, we approximate those essential features of the dynamic process behavior that are fundamental to control. We need not understand differential equations to appreciate the articles on on this site, but for completeness, the first order plus dead time (FOPDT) dynamic model has the form:

$$Tp\frac{dPV(t)}{dt} + PV(t) = Kp \cdot CO(t - \theta p)$$

Where:

PV(t) = measured process variable as a function of time
CO(t – Θp) = controller output signal as a function of time and shifted by Θp
Θp = process dead time
t = time

When the FOPDT dynamic model is fit to process data, the results describe how PV will respond to a change in CO via the model parameters. In particular:

- Process gain, Kp, describes the direction and how far PV will travel,
- Time constant, *T*p, states how fast PV moves after it begins its response,
- Dead time, Θp, is the delay from when CO changes until when PV begins to respond.

An example study that compares dynamic process data from the heat exchanger with a

FOPDT model prediction can be found here. Comparisons between data and model for the gravity drained tanks can be found here and here.

Step 4: Use the model parameters to complete the design and tuning
In step 4, the three FOPDT model parameters are used in correlations to compute controller tuning values. For example, the chart below lists internal model control (IMC) tuning correlations for the PI controller and dependent ideal PID controller, and dependent ideal PID with CO filter forms:

	Controller Gain Kc	Reset Time T_i	Deriv Time T_d	Filter Const α
PI	$\frac{1}{K_p}\frac{T_p}{(\theta_p+T_c)}$	T_p		
Ideal PID	$\frac{1}{K_p}\left(\frac{T_p+0.5\theta_p}{T_c+0.5\theta_p}\right)$	$T_p+0.5\theta_p$	$\frac{T_p\theta_p}{2T_p+\theta_p}$	
PID w/ CO Filter	$\frac{1}{K_p}\left(\frac{T_p+0.5\theta_p}{T_c+\theta_p}\right)$	$T_p+0.5\theta_p$	$\frac{T_p\theta_p}{2T_p+\theta_p}$	$\frac{T_c(T_p+0.5\theta_p)}{T_p(T_c+\theta_p)}$

The closed loop time constant, T_c, in the IMC correlations is used to specify the desired speed or quickness of our controller in responding to a set point change or rejecting a disturbance. The closed loop time constant is computed:

- aggressive performance: T_c is the larger of $0.1 \cdot T_p$ or $0.8 \cdot \Theta_p$
- moderate performance: T_c is the larger of $1 \cdot T_p$ or $8 \cdot \Theta_p$
- conservative performance: T_c is the larger of $10 \cdot T_p$ or $80 \cdot \Theta_p$

Use the Recipe - It is Best Practice
The FOPDT dynamic model of step 3 also provides us the information we need to decide other controller design issues, including:

• *Controller Action*
Before implementing our controller, we must input the proper direction our controller should move to correct for growing errors. Some vendors use the term “reverse acting” and “direct acting.” Others use terms like “up-up” and “up-down” (as CO goes up, then PV goes up or down). This specification is determined solely by the sign of the process gain, Kp.

• *Loop Sample Time,* T
Process time constant, T_p, is the clock of a process. The size of T_p indicates the

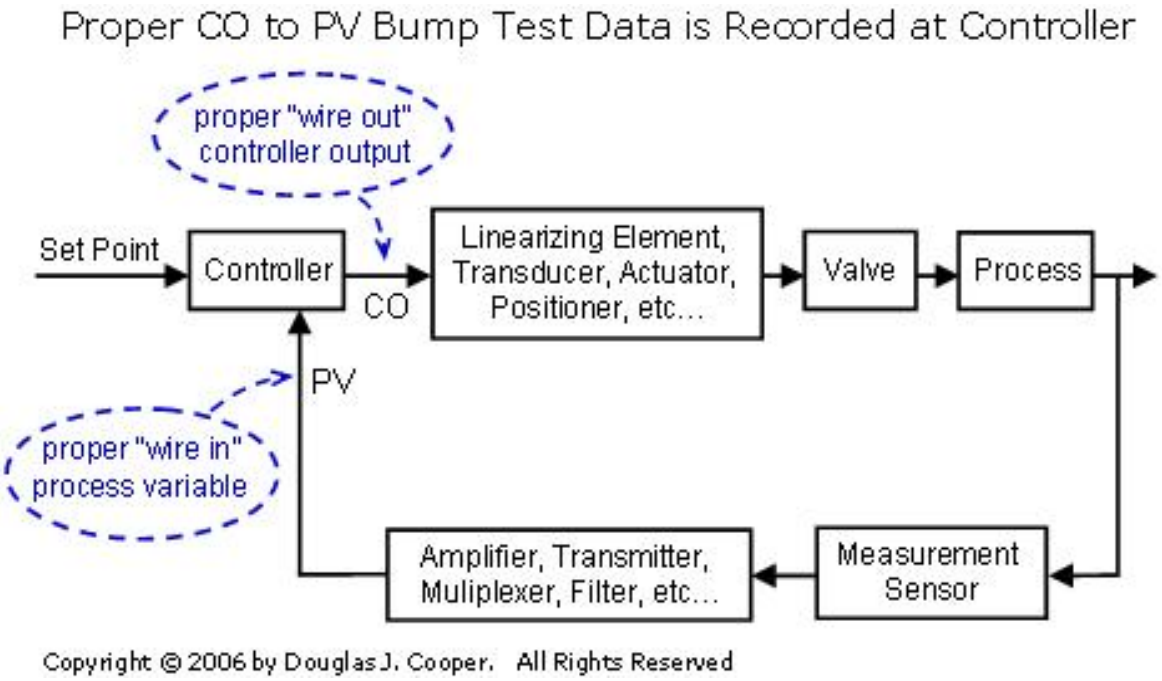
maximum desirable loop sample time. Best practice is to set loop sample time, T, at 10 times per time constant or faster (T ≤ 0.1*T*p). Faster may provide modestly improved performance. Slower than five times per time constant leads to significantly degraded performance.

• *Dead Time Problems*
As dead time grows greater than the process time constant (when Θp > *T*p), controller performance can benefit from a model based dead time compensator such as the Smith predictor.

• *Model Based Control*
If we choose to employ a Smith predictor, or perhaps a dynamic feed forward element, a multivariable decoupler, or any other model based controller, we need a dynamic model of the process to enter into the control computer. The FOPDT model from step 3 of the recipe is usually appropriate for this task.

A Controller's "Process" Goes From Wire Out to Wire In

A controller seeks to maintain the measured process variable (PV) at set point (SP) in spite of unplanned and unmeasured disturbances. Since e(t) = SP — PV, this is equivalent to saying that a controller seeks to maintain controller error, e(t), equal to zero.



A controller repeats a measurement-computation-action procedure at every loop sample time, T. Starting at the far right of the control loop block diagram above:

- A sensor measures a temperature, pressure, concentration or other property of interest from our process.
- The sensor signal is transmitted to the controller. The pathway from sensor to controller might include: a transducer, an amplifier, a scaling element, quantization, a

signal filter, a multiplexer, and other operations that can add delay and change the size, sign, and/or units of the measurement.

- After all electronic and digital operations, the result terminates at our controller as the **"wire in"** measured process variable (PV) signal.
- This "wire in" process variable is subtracted from set point in the controller to compute error, e(t) = SP — PV, which is then used in an algorithm (examples here and here) to compute a controller output (CO) signal.
- The computed CO signal is transmitted on the **"wire out"** from the controller on a path to the final control element (FCE).
- Similar to the measurement path, the signal from the controller to FCE might include filtering, scaling, linearization, amplification, multiplexing, transducing and other operations that can add delay and change the size, sign, and/or units of our original CO signal.
- After any electronic and digital operations, the signal reaches the valve, pump, compressor or other FCE, causing a change in the manipulated variable (a liquid or gas stream flow rate, for example).
- The change in the manipulated variable causes a change in our temperature, pressure, concentration or other process property of interest, all with the goal of making e(t) = 0.

Design Based on CO to PV Dynamics

The steps of the controller design and tuning recipe include: bumping the CO signal to generate CO to PV dynamic process data, approximating this test data with a first order plus dead time (FOPDT) model, and then using the model parameters in rules and correlations to complete the controller design and tuning.

The recipe provides a proven basis for controller design and tuning that avoids wasteful and expensive trial-and-error experiments. But for success, controller design and tuning must be based on process data as the controller sees it.

The controller only knows about the state of the process from the PV signal arriving on the "wire in" *after* all operations in the signal path from the sensor. It can only impact the state of the process with the CO signal it sends on the "wire out" *before* any such operations are made in the path to the final control element.

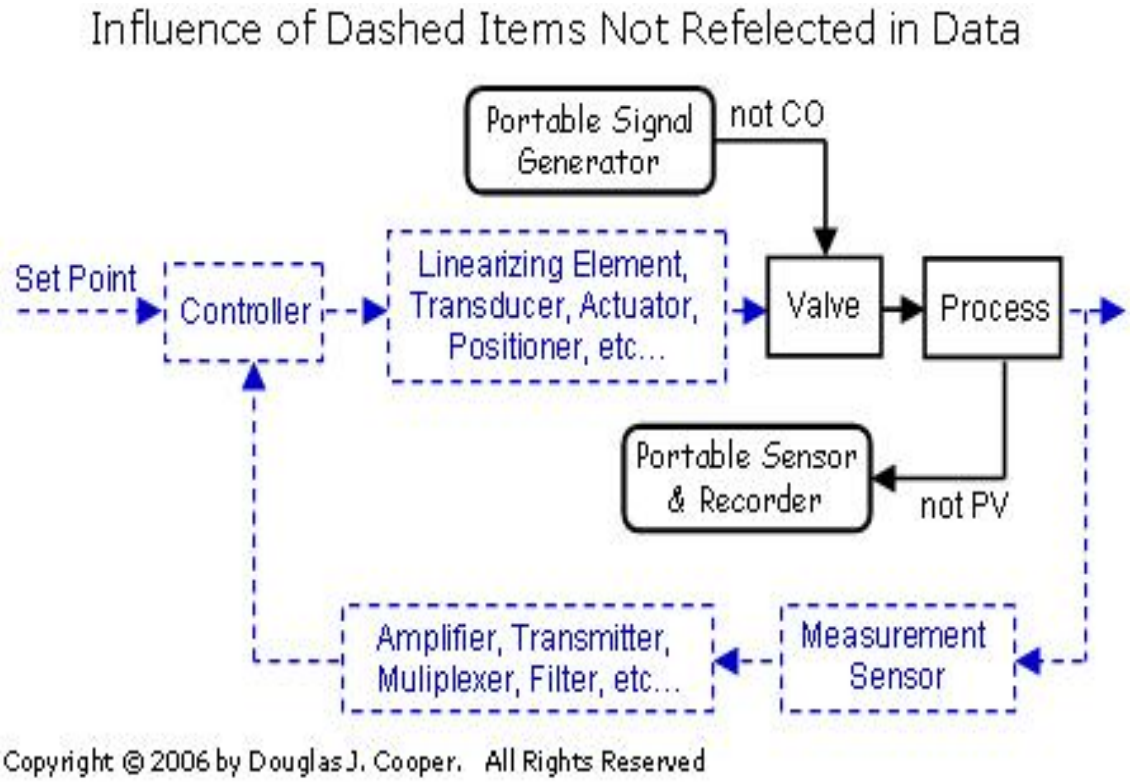
As indicated in the diagram at the top of this article, the proper signals that describe our complete "process" from the controller's view is the "wire out" CO and the "wire in" PV.

Complete the Circuit

Sometimes we find ourselves unable to proceed with an orderly controller design and tuning. Perhaps our controller interface does not make it convenient to directly record process data. Maybe we find a vendor's documentation to be so poorly written as to be all but worthless. There are a host of complications that can hinder progress.

Being resourceful, we may be tempted to move the project forward by using portable instrumentation. It seems reasonable to collect, say, temperature in a vessel during a bump test by inserting a spare thermocouple into the liquid. Or maybe we feel we can be more precise by standing right at the valve and using a portable signal generator to bump the process rather than doing so from a remote control panel.

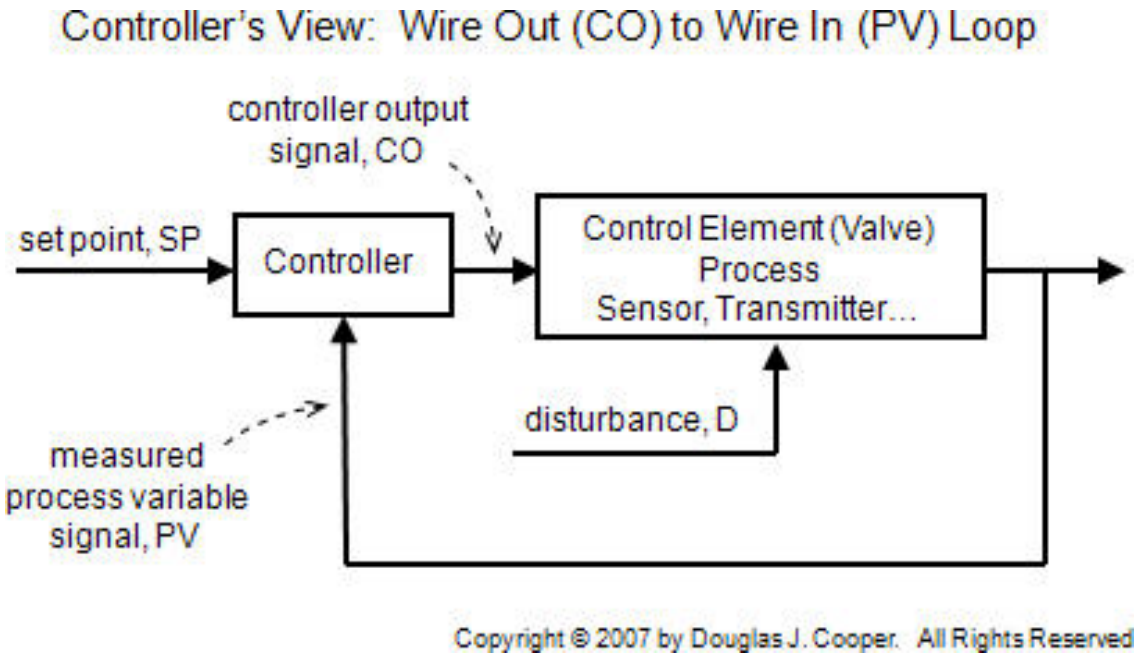
As shown below, such an approach cuts out or short circuits the complete control loop pathway. External or portable instrumentation will not be recording the actual CO or PV as the controller sees it, and the data will not be appropriate for controller design or tuning.



Every Item Counts
The illustration above is extreme in that it shows many items that are not included in the control loop. But please recognize that it can be problematic to leave out even a single step in the complete signal pathway.

A simple scaling element that multiplies the signal by a constant value, for example, may seem reasonably unimportant to the overall loop dynamics. But this alone can change the size and even the sign of Kp, thus having dramatic impact on best tuning and final controller performance.

From a controller's view, the complete loop goes from "wire out" to "wire in" as shown below.



Every item in the loop counts. Always use the complete CO to PV data for process control analysis, design and tuning.

Pay Attention to Units

As detailed in this related article, signals can appear in a control loop in electronic units (e.g., volts, mA), in engineering units (e.g. ^{o}C, Lb/hr), as percent of scale (e.g., 0% to 100%), or as discrete or digital counts (e.g. 0 to 4095 counts).

It is critical that we remain aware of the units of a signal when working with a particular instrument or device. All values entered and computations performed must be consistent with the form of the data at that point in the loop.

Beyond the theory and methods discussed in this e-book, such "accounting confusion" can be one of the biggest challenges for the process control practitioner.

The Normal or Standard PID Algorithm

The question arises quite often, "What is the normal or standard form of the PID (proportional-integral-derivative) algorithm?"

ıe answer is both simple and complex. Before we explore the answer, consider the screen displays shown below (of example 1 or example 2):

Example 1 – The three PID forms compute CO based on:
controller gain (Kc); proportional on error; reset time (*Ti*); derivative on error

Step 1 - Select Proportional Parameter:
(•) Controller Gain ( ) Proportional Band

Step 2 - Proportional On:
(•) Error ( ) Measurement (PV)

Step 3 - Select Integral Parameter:
(•) Reset Time ( ) Reset Rate

Step 4 - Derivative On:
(•) Error ( ) Measurement (PV)

Legend

CO	[=]	Controller Output	PV	[=]	Process Variable
SP	[=]	Set Point	e	[=]	Error (SP - PV)
K_C	[=]	Controller Gain	PB	[=]	Proportional Band
τ_I	[=]	Reset Time	τ_R	[=]	Reset Rate
K_I	[=]	Integral Gain			
τ_D	[=]	Derivative Time	K_D	[=]	Derivative Gain

Step 5 - Select PID Controller: [ ] Show Filter Term

(•) Dependent, Ideal PID

$$CO = bias + K_C e + \frac{K_C}{\tau_I}\int e dt + K_C \tau_D \frac{de}{dt}$$

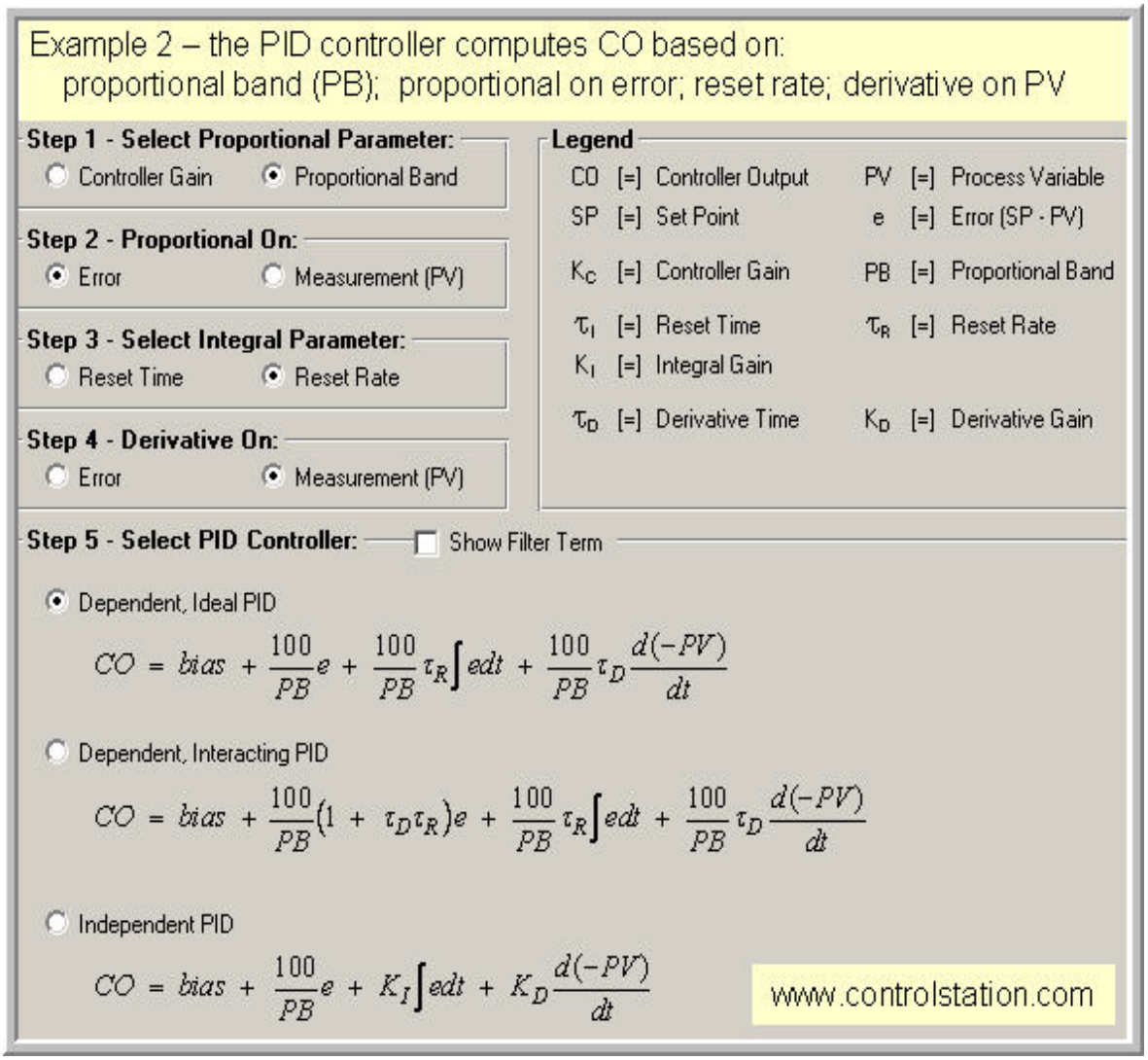
( ) Dependent, Interacting PID

$$CO = bias + K_C\left(1 + \frac{\tau_D}{\tau_I}\right)e + \frac{K_C}{\tau_I}\int e dt + K_C \tau_D \frac{de}{dt}$$

( ) Independent PID

$$CO = bias + K_C e + K_I\int e dt + K_D \frac{de}{dt}$$

www.controlstation.com



As shown in the screen displays:
- There are three popular PID algorithm forms (see step 5 in the large image views).
- Each of the three algorithms has tuning parameters and algorithm variables that can be cast in different ways (see steps 1 – 4 in the large image views).

So your vendor might be using one of dozens of possible algorithm forms. And if you add a filter term to your controller, the number of possibilities increases substantially.

The Simple Answer
Any of the algorithms can deliver the same performance as any of the others. There is no control benefit from choosing one form over another. They are all standard or normal in that sense.

If you are considering a purchase, select the vendor that serves your needs the best and don't dwell on the specifics of the algorithm. Some things to consider include:
- compatibility with existing controllers and associated hardware and software
- cost
- ease of installation and maintenance

- reliability
- your operating environment (is it clean? cool? dry?)

A More Complete Answer
Most of the different controller algorithm forms can be found in one vendor's product or another. Some vendors even use different forms within their own product lines. More information can be found in this article.

And while the various forms are equally capable, each must be tuned (values for the adjustable parameters must be specified) using tuning correlations specifically designed for that particular control algorithm.

Commercial software makes it straightforward to get desired performance from any of them. But it is essential that you know your vendor and controller model number to ensure a correct match between controller form and computed tuning values.

The alternative to an orderly design methodology is a "guess and test" approach. While used by some practitioners, such trial and error tuning squanders valuable production time, consumes more feedstock and utilities than is necessary, generates additional waste and off-spec product, and can even present safety concerns.

In most articles on Controlguru.com, we use some variation of the dependent, ideal PID controller form:

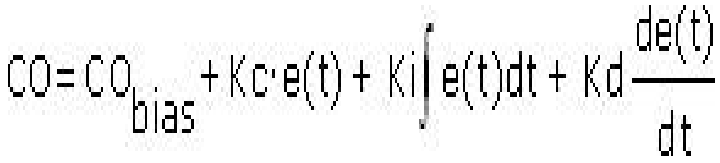
$$CO = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{Ti}\int e(t)dt + Kc \cdot Td\frac{de(t)}{dt}$$

Where:
CO = controller output signal
CO_{bias} = controller bias
e(t) = current controller error, defined as SP – PV
SP = set point
PV = measured process variable
Kc = controller gain, a tuning parameter
*T*i = reset time, a tuning parameter
*T*d = derivative time, a tuning parameter

To reinforce that the controllers all are equally capable, we occasionally use variations of the dependent, interacting form:

$$CO = CO_{bias} + Kc\left(1 + \frac{Td}{Ti}\right)e(t) + \frac{Kc}{Ti}\int e(t)dt + Kc \cdot Td\frac{de(t)}{dt}$$

or variations of the independent PID form:



Final Thoughts
The discussion above glosses over some of the subtle differences in algorithm form that we can exploit to improve control performance. We will learn about these details as we progress in our learning.

For example, derivative on error behaves different from derivative on measured PV. This is true for all of the algorithms. Derivative on error can “kick” after set point steps and this is rarely considered desirable behavior. Thus, derivative on PV is recommended for industrial applications.

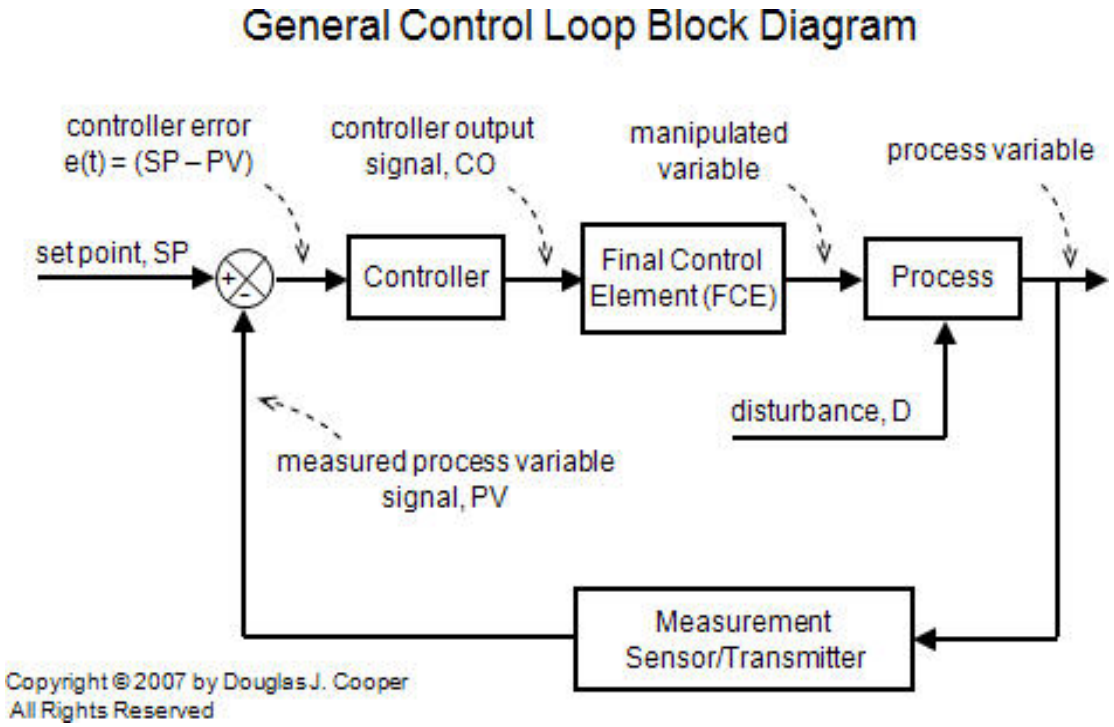
And if you are considering programming the controller yourself, it is not the algorithm form that is the challenge. The big hurdle is properly accounting for the anti-reset windup and jacketing logic to allow bumpless transition between operating modes.

There is much more to consider than we can possibly address in this one article. Return to the Table of Contents to learn more.

6) Proportional Control - The Simplest PID Controller

The P-Only Control Algorithm

The simplest algorithm in the PID family is a proportional or P-Only controller. Like all automatic controllers, it repeats a measurement-computation-action procedure at every loop sample time, T, following the logic flow shown in the block diagram below:



Starting at the far right of the control loop block diagram above:

- A sensor measures and transmits the current value of the process variable, PV, back to the controller (the 'controller wire in')
- Controller error at current time t is computed as set point minus measured process variable, or e(t) = SP – PV
- The controller uses this e(t) in a control algorithm to compute a new controller output signal, CO
- The CO signal is sent to the final control element (e.g. valve, pump, heater, fan) causing it to change (the 'controller wire out')
- The change in the final control element (FCE) causes a change in a manipulated variable
- The change in the manipulated variable (e.g. flow rate of liquid or gas) causes a change in the PV

The goal of the controller is to make e(t) = 0 in spite of unplanned and unmeasured

disturbances. Since e(t) = SP – PV, this is the same as saying a controller seeks to make PV = SP.

The P-Only Algorithm
The P-Only controller computes a CO action every loop sample time T as:

$$CO = CO_{bias} + Kc \cdot e(t)$$

Where:
CO_{bias} = controller bias or null value
Kc = controller gain, *a tuning parameter*
e(t) = controller error = SP – PV
SP = set point
PV = measured process variable

Design Level of Operation
Real processes display a nonlinear behavior, which means their apparent process gain, time constant and/or dead time changes as operating level changes and as major disturbances change. Since controller design and tuning is based on these Kp, *T*p and Θp values, controllers should be designed and tuned for a pre-defined level of operation.

When designing a cruise control system for a car, for example, would it make sense for us to perform bump tests to generate dynamic data when the car is traveling twice the normal speed limit while going down hill on a windy day? Of course not.

Bump test data should be collected as close as practical to the design PV when the disturbances are quiet and near their typical values. Thus, the design level of operation for a cruise control system is when the car is traveling at highway speed on flat ground on a calm day.

Definition: the design level of operation (DLO) is where we expect the SP and PV will be during normal operation while the important disturbances are quiet and at their expected or typical values.

Understanding Controller Bias
Let's suppose the P-Only control algorithm shown above is used for cruise control in an automobile and CO is the throttle signal adjusting the flow of fuel to the engine.

Let's also suppose that the speed SP is 70 and the measured PV is also 70 (units can be mph or kph depending on where you live in the world). Since PV = SP, then e(t) = 0 and the algorithm reduces to:

$$CO = CO_{bias} + Kc \cdot (0) = CO_{bias}$$

If CO_{bias} is zero, then when set point equals measurement, the above equation says that the throttle signal, CO, is also zero. This makes no sense. Clearly if the car is traveling 70 kph, then some baseline flow of fuel is going to the engine.

This baseline value of the CO is called the bias or null value. In this example, CO_{bias} is the flow of fuel that, in manual mode, causes the car to travel the design speed of 70 kph

when on flat ground on a calm day.

Definition: CO_{bias} is the value of the CO that, in manual mode, causes the PV to steady at the DLO while the major disturbances are quiet and at their normal or expected values.

A P-Only controller bias (sometimes called null value) is assigned a value as part of the controller design and remains fixed once the controller is put in automatic.

Controller Gain, Kc
The P-Only controller has the advantage of having only one adjustable or tuning parameter, Kc, that defines how active or aggressive the CO will move in response to changes in controller error, e(t).

For a given value of e(t) in the P-Only algorithm above, if Kc is small, then the amount added to CO_{bias} is small and the controller response will be slow or sluggish. If Kc is large, then the amount added to CO_{bias} is large and the controller response will be fast or aggressive.

Thus, Kc can be adjusted or tuned for each process to make the controller more or less active in its actions when measurement does not equal set point.

P-Only Controller Design
All controllers from the family of PID algorithms (P-Only, PI, PID) should be designed and tuned using our proven recipe:

1. Establish the design level of operation (the normal or expected values for set point and major disturbances).
2. Bump the process and collect controller output (CO) to process variable (PV) dynamic process data around this design level.
3. Approximate the process data behavior with a first order plus dead time (FOPDT) dynamic model.
4. Use the model parameters from step 3 in rules and correlations to complete the controller design and tuning.

The Internal Model Control (IMC) tuning correlations that work so well for PI and PID controllers cannot be derived for the simple P-Only controller form. The next best choice is to use the widely-published integral of time-weighted absolute error (ITAE) tuning correlation:

Moderate P-Only: $$Kc = \frac{0.2}{Kp}\left(\frac{Tp}{\theta p}\right)^{1.22}$$

This correlation is useful in that it reliably yields a moderate Kc value. In fact, some practitioners find that the ITAE Kc value provides a response performance so predictably modest that they automatically start with an aggressive P-Only tuning, defined here as two and a half times the ITAE value:

Aggressive P-Only: Kc = 2.5 (Moderate Kc)

Reverse Acting, Direct Acting and Control Action
Time constant, *T*p, and dead time, Θp, cannot affect the sign of Kc because they mark the passage of time and must always be positive. The above tuning correlation thus implies that Kc must always have the same sign as the process gain, Kp.

When CO increases on a process that has a positive Kp, the PV will increase in response. The process is direct acting. Given this CO to PV relationship, when in automatic mode (closed loop), if the PV starts drifting too high above set point, the controller must decrease CO to correct the error.

This "opposite to the problem" reaction is called ***negative feedback*** and forms the basis of stable control.

A process with a positive Kp is direct acting. With negative feedback, the controller must be reverse acting for stable control. Conversely, when Kp is negative (a reverse acting process), the controller must be direct acting for stable control.

Since Kp and Kc always have the same sign for a particular process and stable control requires negative feedback, then:

- direct acting process (Kp and Kc positive) −› use a reverse acting controller
- reverse acting process (Kp and Kc negative) −› use a direct acting controller

In most commercial controllers, a positive value of the Kc is always entered. The sign (or action) of the controller is then assigned by specifying that the controller is either reverse or direct acting to indicate a positive or negative Kc respectively.

If the wrong control action is entered, the controller will quickly drive the final control element (e.g., valve, pump, compressor) to full on/open or full off/closed and remain there until the proper control action entry is made.

Proportional Band
Some manufacturers use different forms for the same tuning parameter. The popular alternative to Kc found in the marketplace is proportional band, PB.

In many industry applications, both the CO and PV are expressed in units of percent. Given that a controller output signal ranges from a minimum (CO_{min}) to maximum (CO_{max}) value, then:

$$PB = (CO_{max} - CO_{min})/Kc$$

When CO and PV have units of percent and both range from 0% to 100%, the much published conversion between controller gain and proportional band results:

$$PB = 100/Kc$$

Many case studies on this site assign engineering units to the measured PV because plant software has made the task of unit conversions straightforward. If this is true in your plant, take care when using these conversion formula.

Implementation Issues

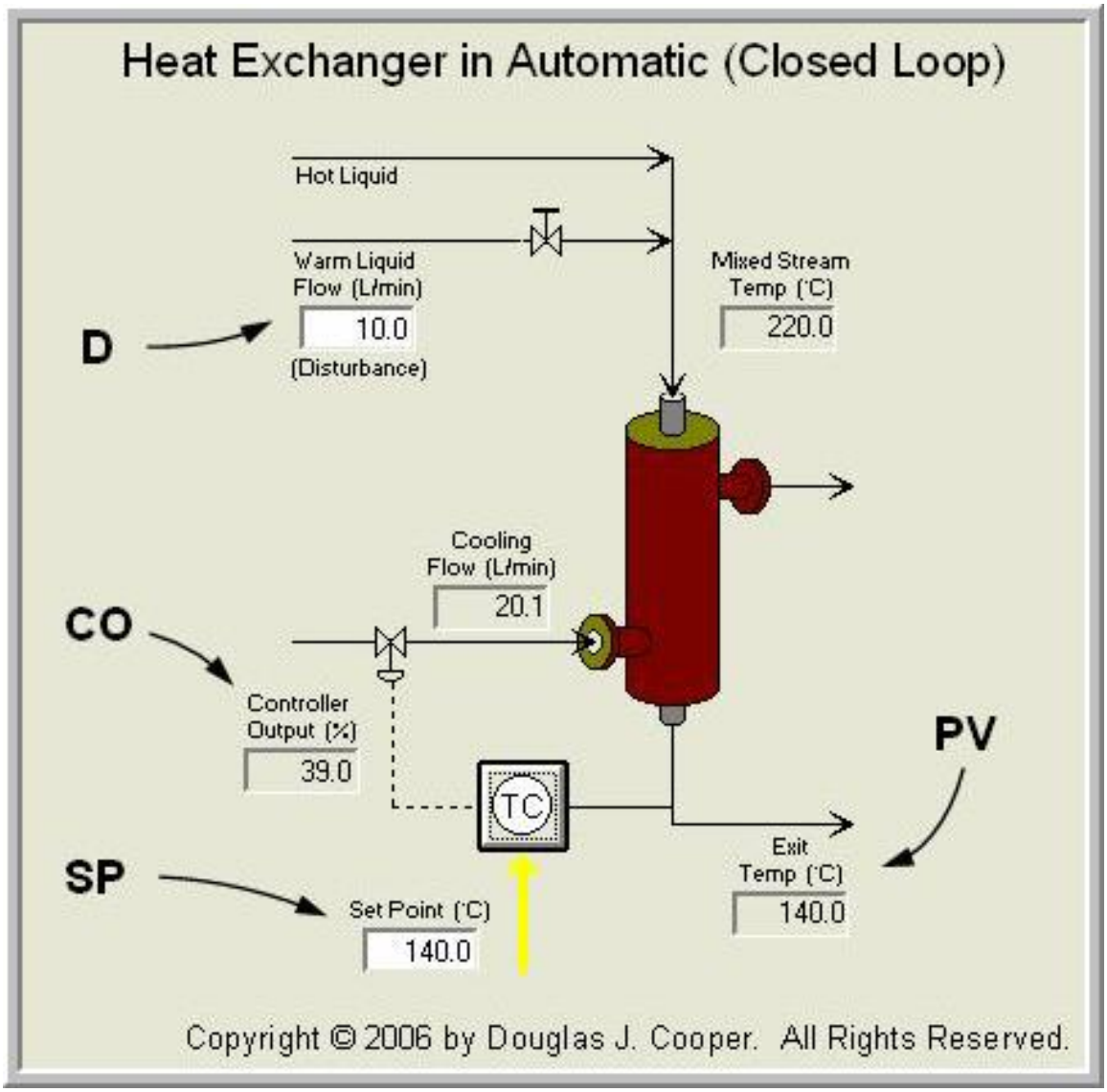
Implementation of a P-Only controller is reasonably straightforward, but this simple algorithm exhibits a phenomenon called "offset." In most industrial applications, offset is considered an unacceptable weakness. We explore P-Only control, offset and other issues for the heat exchanger and the gravity drained tanks processes.

P-Only Control of the Heat Exchanger Shows Offset

We have discussed the general proportional only (P-Only) algorithm structure and considered important design and tuning issues associated with implementation.

Here we investigate the capabilities of the P-Only controller on our heat exchanger process and highlight some key features and weaknesses of this simple algorithm.

The heat exchanger process used in this study is shown below () and discussed in more detail here.



As with all controller implementations, best practice is to follow the four-step design and tuning recipe as we proceed with the study:

Step 1: Design Level of Operation (DLO)
Real processes display a nonlinear behavior. That is, their process gain (Kp), time constant (*T*p) and/or dead time (Θp) changes as operating level changes and as major disturbances change. Since the rules and correlations we use are based on these Kp, *T*p and Θp values, controllers should be designed and tuned for a specific level of operation.

The first step in the controller design recipe is to specify our design level of operation (DLO). This includes stating where we expect the set point, SP, and measured process variable, PV, to be during normal operation. Hopefully, these will be the same values as this is the point of a controller.

We also should have some sense of the range of values the SP and PV might assume so we can explore the nature of the process dynamics across that range.

For the heat exchanger, we specify that the SP and PV will normally be at 138 °C, and during production, they may range from 138 to 140 °C. Thus, we can state:

- Design PV and SP = 138 °C with range of 138 to 140 °C

We also should know normal or typical values for our major disturbances and be reasonably confident that they are quiet so we may proceed with a bump test. As shown in the graphic above, the heat exchanger process has only one major disturbance variable (D), a side stream labeled Warm Liquid Flow. We specify that the expected or design value for this stream is:

- Expected warm liquid flow disturbance, D = 10 L/min

We assume that D remains quiet and at this normal design value throughout the study.

Step 2: Collect Data at the DLO
The next step in the design recipe is to collect dynamic process data as near as practical to our design level of operation. We have previously collected and documented heat exchanger step test data that matches our design conditions.

Step 3: Fit an FOPDT Model to the Design Data
Here we document a first order plus dead time (FOPDT) model approximation of the heat exchanger step test data from step 2:
- Process gain (how far), Kp = – 0.53 °C/%
- Time constant (how fast), *T*p = 1.3 min
- Dead time (how much delay), Θp = 0.8 min

Step 4: Use the Parameters to Complete the Design
The P-Only controller computes a controller output (CO) action every loop sample time T as:

$$CO = CO_{bias} + Kc \cdot e(t)$$

Where:

CO_{bias} = controller bias or null value
Kc = controller gain, a tuning parameter
e(t) = controller error defined as SP – PV

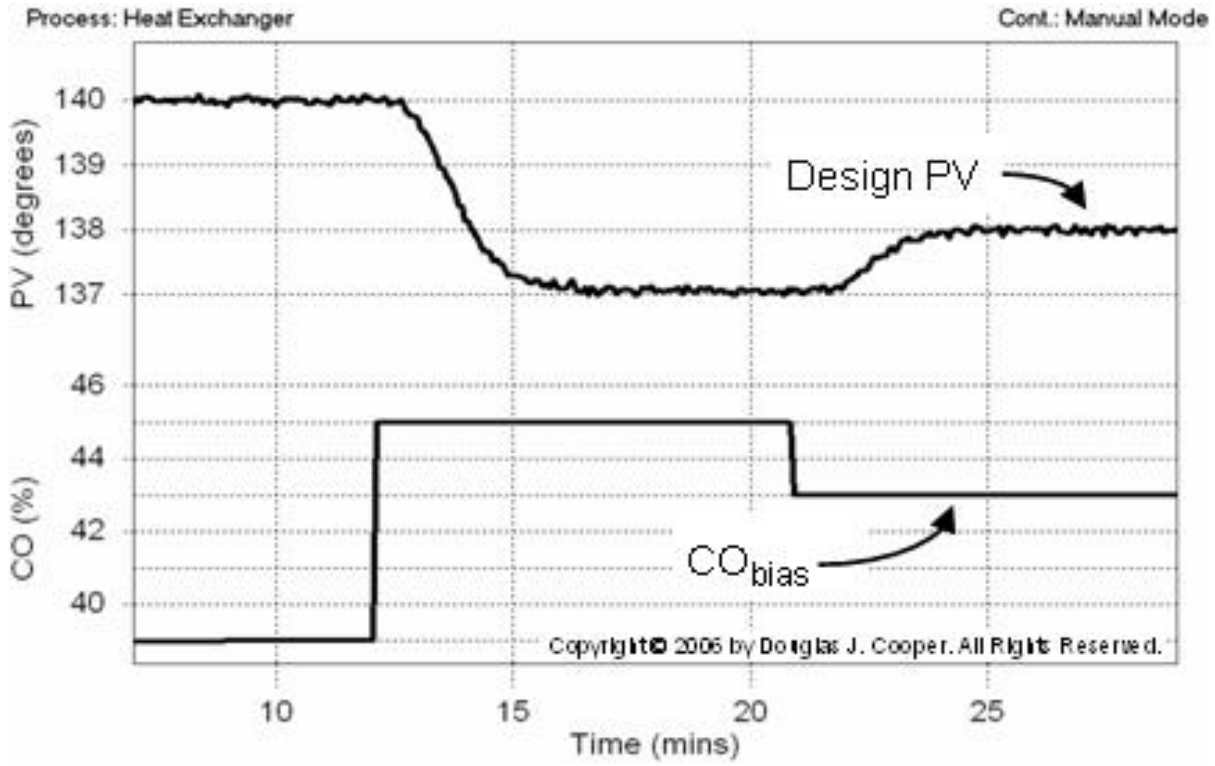
• *Computing controller error, e(t)*
Set point (SP) is something we enter into the controller. The PV measurement comes from our sensor (our wire in). With SP and PV values known, controller error can be computed at every loop sample time T as: e(t) = SP – PV.

• *Determining Bias Value*
CO_{bias} is the value of the CO that, in manual mode, causes the PV to steady at the DLO while the major disturbances are quiet and at their normal or expected values.

The plot below () shows that CO_{bias} can be located with an ordered search. That is, we move CO up and down while in manual mode until the PV settles at the design value of 138 °C while the major disturbances (trace not shown) are quiet and at their normal or expected values.

Such a manipulation of our process may be impractical or impossible in production situations. The plot is useful, however, because it helps us visualize how the baseline (bias) value of the CO is linked to the design PV.

When we explore PI control of the heat exchanger, we will discuss how commercial controllers use a bumpless transfer method to automatically provide a value for CO_{bias}.



The plot above shows that when CO is held constant at 43% with the disturbances at their normal values, the PV steadies at the design value of 138 °C. Thus:

- CO_{bias} = 43%

• *Computing Controller Gain*
For the simple P-Only controller, we compute Kc with the integral of time-weighted absolute error (ITAE) tuning correlation:

Moderate P-Only: $$Kc = \frac{0.2}{Kp}\left(\frac{\tau p}{\theta p}\right)^{1.22}$$

This correlation is useful in that it reliably yields a moderate Kc value.

Aside: Dead time, Θp, is in the denominator in the correlation, so it cannot equal zero. Otherwise, Kc will approach infinity, a fairly useless result.

Consider that all controllers measure, act, then wait until next sample time before measuring again. This "measure, act, wait" procedure has a delay (or dead time) of one sample time, T, built naturally into its structure.

Thus, by definition, the minimum dead time (Θp,min) in a control loop is the loop sample time, T. Dead time can certainly be larger than sample time and it usually is, but it cannot be smaller.

Whether by software or graphical analysis, if we compute a Θp that is less than T, we must set Θp = T everywhere in our tuning correlations. More information about the importance of sample time to controller design and tuning can be found in this other article.

☐Best Practice Rule:
when using the FOPDT model for controller tuning, Θp,min = T

(In the unreal world of pure theory, a true first order process with zero dead time is unconditionally stable under P-Only control. It would not even oscillate, let alone go unstable, at infinite Kc. The tuning correlation is therefore valid even at the theoretical extreme.)

Using our FOPDT model values from step 3, we compute:

$$Kc = \frac{0.2}{-0.53}\left(\frac{1.3}{0.8}\right)^{1.22} = -0.7\ \%/^{o}C$$

And our moderate P-Only controller becomes:

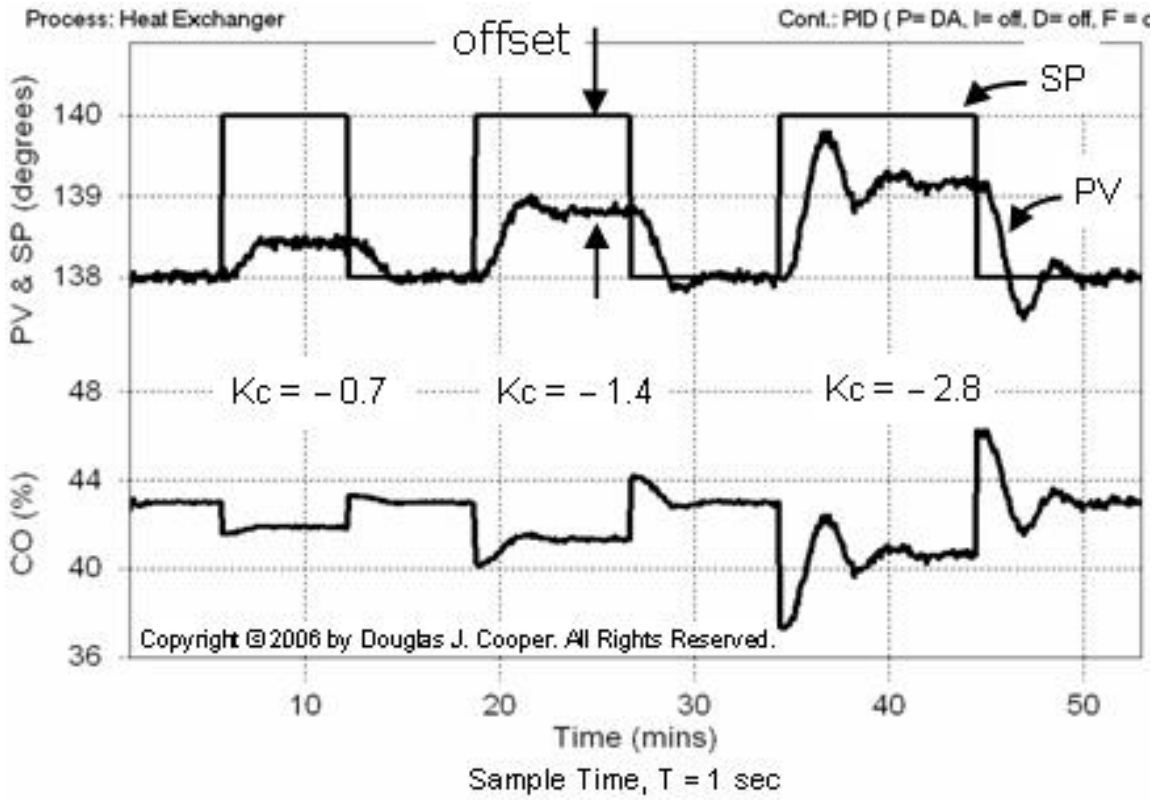
- CO = 43% – 0.7·e(t)

Implement and Test
To explore how controller gain, Kc, impacts P-Only controller behavior, we test the controller with this ITAE controller gain value. Since the Kc value tends to be moderate, we also study more active or aggressive controller behavior when we double Kc and then

double it again:

- 2Kc = – 1.4 %/°C
- 4Kc = – 2.8 %/°C

The performance of the P-Only controller in tracking set point changes is pictured below () for the ITAE Kc and its multiples. Note that the warm liquid disturbance flow, though not shown, remains constant at 10 L/min throughout the study.



As shown in the figure, whenever the set point is at the design level of 138 °C, then PV equals SP.

Each of the three times the SP is stepped away from the DLO, however, the PV settles out at a value short of the set point. The simple P-Only controller is not able to eliminate this "offset," or sustained error between the PV and SP. We talk more about offset below.

• *Kc and Controller Activity*
The plot above shows the performance of the P-Only controller using three different values of Kc.

One point of this study is to highlight that as Kc increases, the activity of the controller output, CO, increases. The CO trace at the bottom of the plot shows this increasingly active behavior, seen as more dramatic moves in response to the same set point step, as Kc increases across the plot.

Thus, we establish that controller gain, Kc, is responsible for the general, and especially the initial, activity in a controller response. This “response activity related to Kc” behavior

carries over to PI and PID controllers.

We also see that as Kc increases across the plot, the offset (difference between SP and final PV) decreases but the oscillatory nature of the response increases.

Offset - The Big Disadvantage of P-Only Control
The biggest advantage of P-Only control is that there is only one tuning parameter to adjust, so it is relatively easy to achieve a “best” final tuning. The disadvantage is that this simple control algorithm permits offset.

Offset occurs in most processes under P-Only control when the set point and/or disturbances are at any value other than that used to determine CO_{bias}.

To understand why offset occurs, let's work our way through the P-Only equation:

$$CO = 43\% – 0.7 \cdot e(t)$$

and recognize that:
- when PV equals SP, then error is zero: e(t) = 0
- if e(t) is zero, then CO equals the CO_{bias} value of 43%
- if CO is steady at 43%, then the PV settles to 138 °C. We know this is true because that’s how CO_{bias} was determined in the first place. The first plot in this article shows us this.

Continuing our reasoning:
- the only way CO can be different from the CO_{bias} value of 43% is if something is added or subtracted from the 43%
- the only way we have something to add or subtract from the 43% is if the error e(t) is not zero
- if e(t) is not zero, then PV cannot equal SP, and we have offset.

Possible Applications?
If P-Only controllers permit offset, do they have any place in the process world? Actually, yes.

One example is a surge or swing tank designed to smooth flows between two units. It does not matter what specific liquid level the tank maintains. The level can be at 63% or 36% and we are happy. Just as long as the tank never empties completely or fills so much that it overflows.

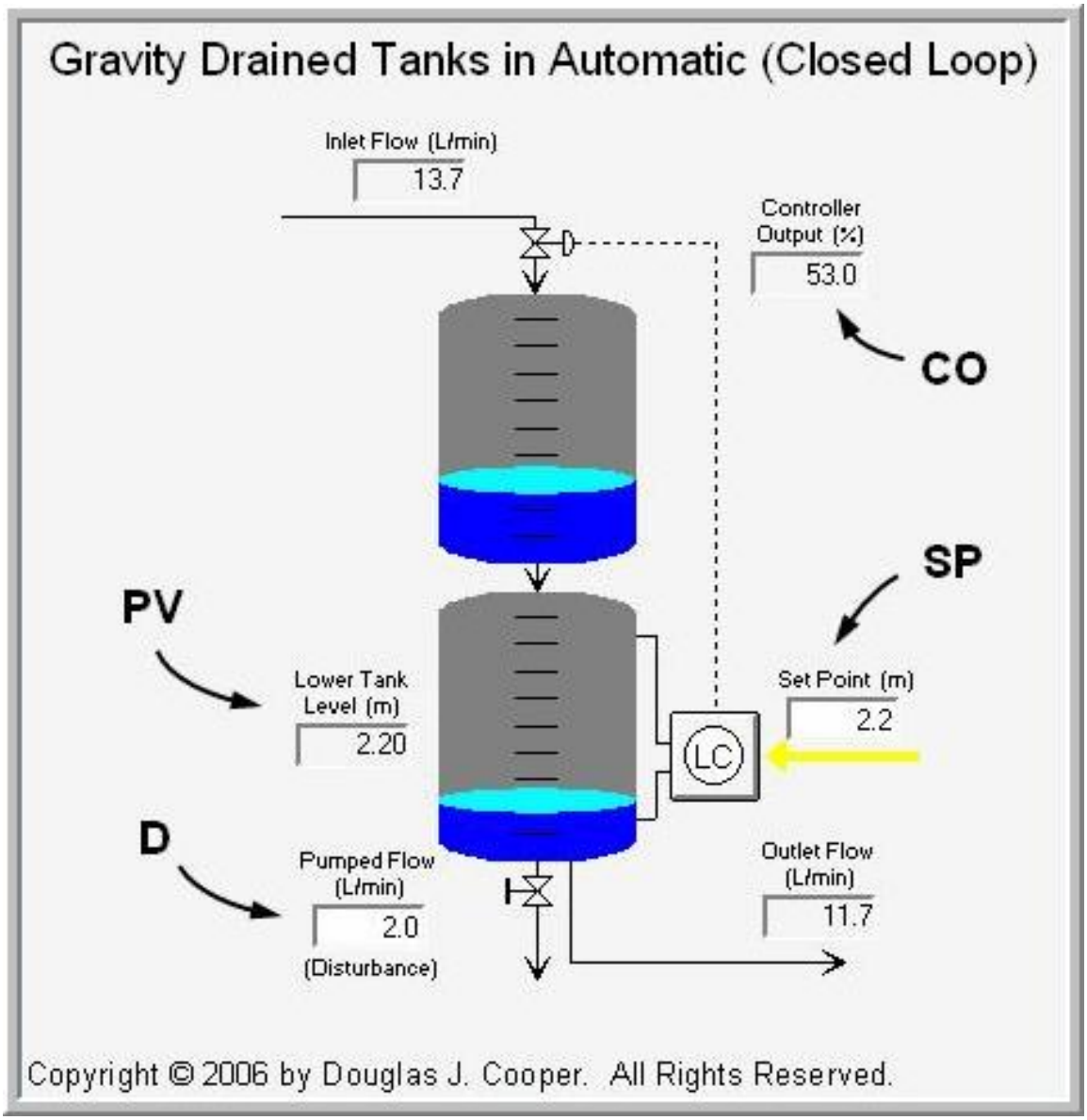
A P-Only controller can serve this function. Put the set point at a level of 50% and let the offset happen. We can have the controller implemented quickly and keep it tuned with little effort.

P-Only Disturbance Rejection Of The Gravity Drained Tanks

In a previous article, we looked at the structure of the P-Only algorithm and we considered some design issues associated with implementation. We also studied the set point tracking (or servo) performance of this simple controller for the heat exchanger process.

Here we investigate the capabilities of the P-Only controller for liquid level control of the gravity drained tanks process. Our objective in this study is disturbance rejection (or regulatory control) performance.

Gravity Drained Tanks Process
A graphic of the gravity drained tanks process is shown below ():



The measured process variable (PV) is liquid level in the lower tank. The controller output (CO) adjusts the flow into the upper tank to maintain the PV at set point (SP).

The disturbance (D) is a pumped flow out of the lower tank. It's draw rate is adjusted by a different process and is thus beyond our control. Because it runs through a pump, D is not affected by liquid level, though the pumped flow rate drops to zero if the tank empties.

We begin by summarizing the previously discussed results of steps 1 through 3 of our design and tuning recipe as we proceed with our P-Only control investigation:

Step 1: Determine the Design Level of Operation (DLO)
Our primary objective is to reject disturbances as we control liquid level in the lower tank. As discussed here, our design level of operation (DLO) for this study is:

- design PV and SP = 2.2 m with range of 2.0 to 2.4 m
- design D = 2 L/min with occasional spikes up to 5 L/min

Step 2: Collect Process Data around the DLO
When CO, PV and D are steady near the design level of operation, we bump the CO far enough and fast enough to force a clear dynamic response in the PV that dominates the signal and process noise. As detailed here, we performed two different open loop (manual mode) dynamic tests, a step test and a doublet test.

Step 3: Fit a FOPDT Model to the Dynamic Process Data
The third step of the recipe is to describe the overall dynamic behavior of the process with an approximating first order plus dead time (FOPDT) dynamic model. We define the model parameters and present details of the model fit of step test data here. A model fit of doublet test data using commercial software confirms these values:

- process gain (how far), Kp = 0.09 m/%
- time constant (how fast), Tp = 1.4 min
- dead time (how much delay), Θp = 0.5 min

Step 4: Use the FOPDT Parameters to Complete the Design
Following the heat exchanger P-Only study, the P-Only control algorithm computes a CO action every loop sample time T as:

$$CO = CO_{bias} + Kc \cdot e(t)$$

Where:
CO_{bias} = controller bias or null value
Kc = controller gain, a tuning parameter
e(t) = controller error, defined as SP – PV

☐*Sample Time,* T
Best practice is to set the loop sample time, T, at one-tenth the time constant or faster (i.e., T ≤ 0.1*T*p). Faster sampling may provide modestly improved performance. Slower sampling can lead to significantly degraded performance.

In this study, T ≤ (0.1)(1.4 min), so T should be 8 seconds or less. We meet this specification with the common vendor sample time option:

- sample time, T = 1 sec

☐*Control Action (Direct/Reverse)*
The gravity drained tanks has a positive Kp. That is, when CO increases, PV increases in response. When in automatic mode (closed loop), if the PV is too high, the controller must decrease the CO to correct the error (read more here). Since the controller must move in the direction opposite of the problem, we specify:

- controller is reverse acting

☐*Dead Time Issues*
If dead time is greater than the process time constant (Θp > Tp), control becomes increasingly problematic and a Smith predictor can offer benefit. For this process, the

dead time is smaller than the time constant, so:

- dead time is small and thus not a concern

☐*Computing Controller Error, e(t)*

Set point, SP, is manually entered into a controller. The measured PV comes from the sensor (our wire in). Since SP and PV are known values, then at every loop sample time, T, controller error can be directly computed as:

- error, e(t) = SP – PV

☐*Determining Bias Value,* CO_{bias}

CO_{bias} is the value of CO that, in manual mode, causes the PV to remain steady at the DLO when the major disturbances are quiet and at their normal or expected values. Our doublet plots establish that when CO is at 53%, the PV is steady at the design value of 2.2 m, thus:

- controller bias, CO_{bias} = 53%

☐*Controller Gain,* Kc

For the simple P-Only controller form, we use the integral of time-weighted absolute error (ITAE) tuning correlation:

Moderate P-Only: $$Kc = \frac{0.2}{Kp}\left(\frac{Tp}{\theta p}\right)^{1.22}$$

Aside: Regardless of the values computed in the FOPDT fit, best practice is to set Θp no smaller than sample time, T (or Θp ≥ T) in the control rules and correlations (more discussion here). In this gravity drained tanks study, our FOPDT fit produced a Θp much larger than T, so the "dead time greater than sample time" rule is met.

Using our FOPDT model values from step 3, we compute:

$$Kc = \frac{0.2}{0.09}\left(\frac{1.4}{0.5}\right)^{1.22} = 8\ \%/m$$

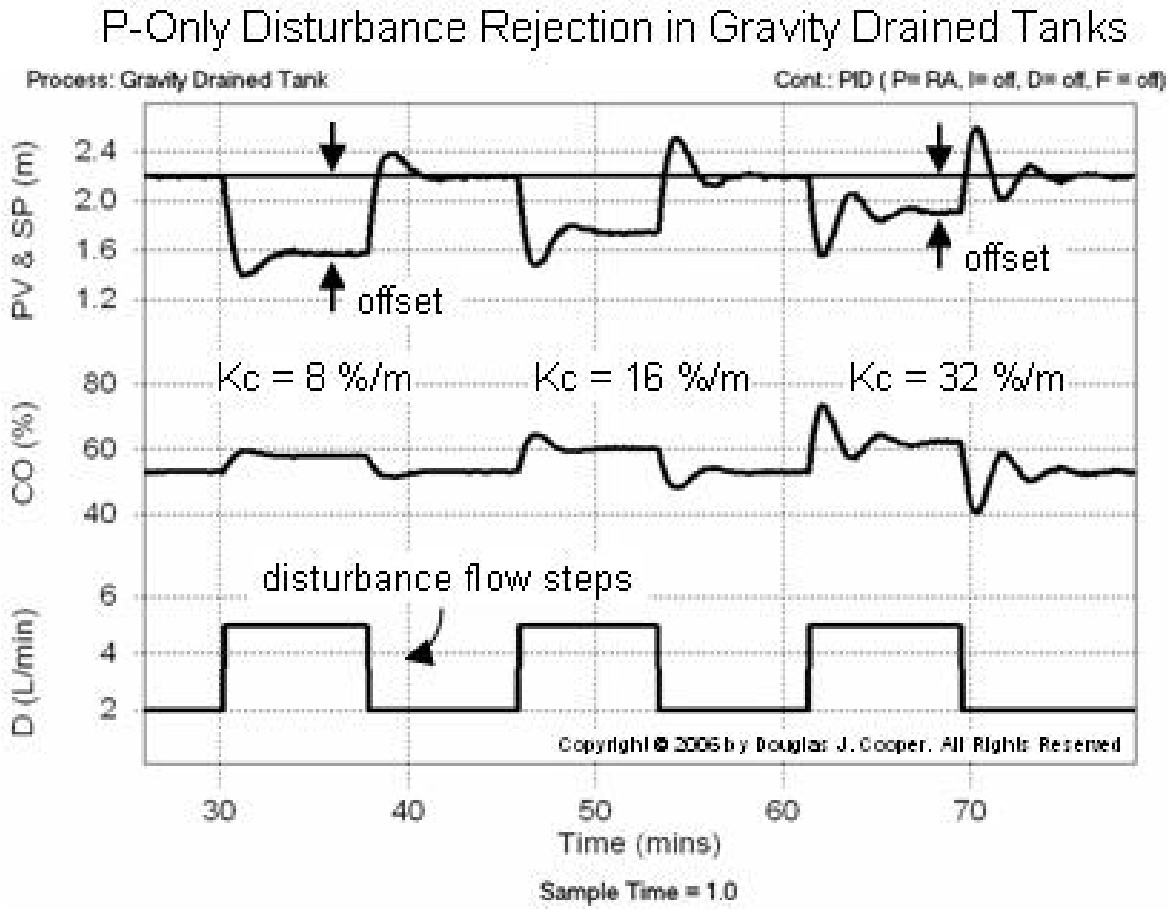
And our moderate P-Only controller becomes:

- P-Only controller: CO = 53% + 8·e(t)

Implement and Test

To explore how controller gain impacts P-Only performance, we test the controller with the above Kc = 8 %/m. Since the correlation tends to produce moderate performance values, we also explore increasingly aggressive or active P-Only tuning by doubling Kc (2Kc = 16 %/m) and then doubling it again (4Kc = 32 %/m).

The ability of the P-Only controller to reject step changes in the pumped flow disturbance, D, is pictured below () for the ITAE value of Kc and its multiples. Note that the set point remains constant at 2.2 m throughout the study.



As shown in the figure above, whenever the pumped flow disturbance, D, is at the design level of 2 L/min (e.g., when time is less than 30 min) then PV equals SP.

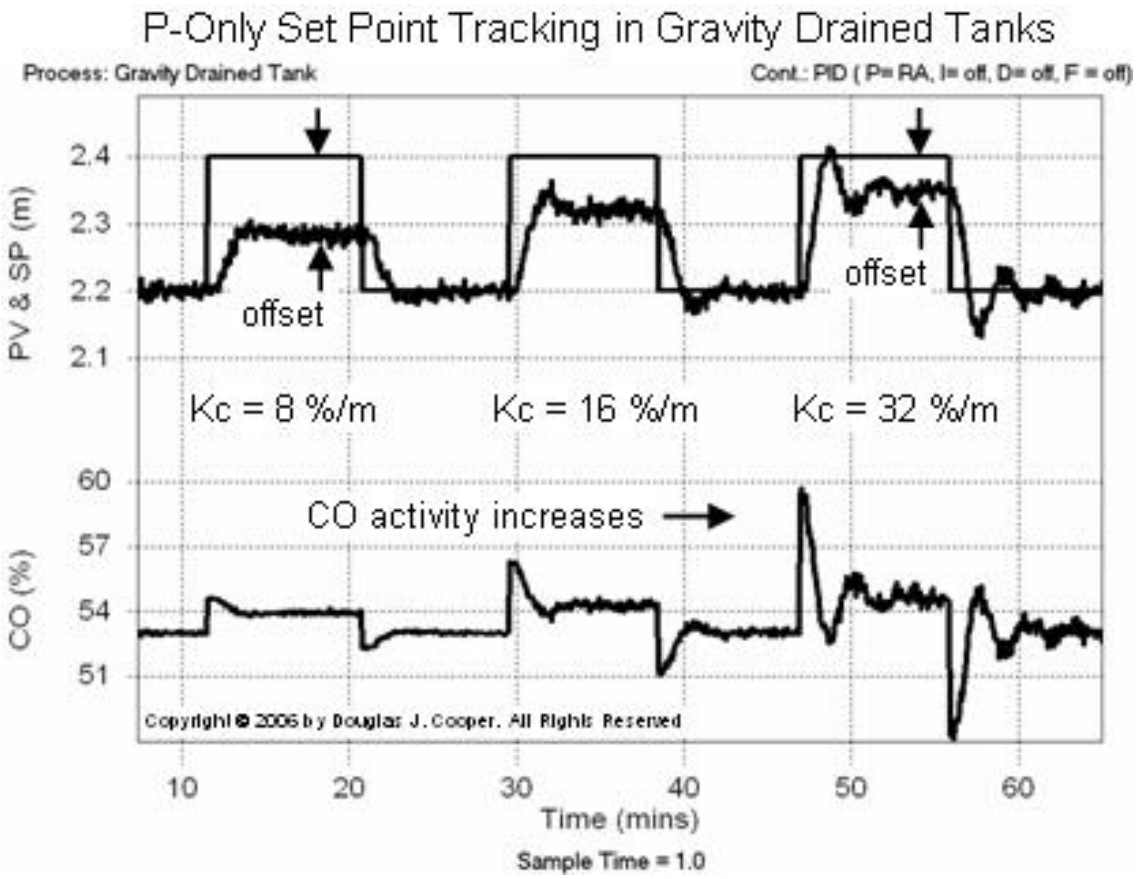
The three times that D is stepped away from the DLO, however, the PV shifts away from the set point. The simple P-Only controller is not able to eliminate this “offset,” or sustained error between the PV and SP. This behavior reinforces that both set point and disturbances contribute to defining the design level of operation for a process.

The figure shows that as Kc increases across the plot:

- the activity of the controller output, CO, increases,
- the offset (difference between SP and final PV) decreases, and
- the oscillatory nature of the response increases.

Offset, or the sustained error between SP and PV when the process moves away from the DLO, is a big disadvantage of P-Only control. Yet there are appropriate uses for this simple controller (more discussion here).

While not our design objective, presented below is the set point tracking ability of the controller () when the disturbance flow is held constant:



The figure shows that as Kc increases across the plot, the same performance observations made above apply here: the activity of CO increases, the offset decreases, and the oscillatory nature of the response increases.

Aside: it may appear that the random noise in the PV measurement signal is different in the two plots above, but it is indeed the same. Note that the span of the PV axis in the two plots differs by a factor of four. The narrow span of the set point tracking plot greatly magnifies the signal traces, making the noise more visible.

Proportional Band

Different manufacturers use different forms for the same tuning parameter. The popular alternative to controller gain found in the marketplace is proportional band, PB.

If the CO and PV have units of percent and both can range from 0 to 100%, then the conversion between controller gain and proportional band is: $PB = 100/Kc$

Thus, as Kc increases, PB decreases. This reverse thinking can challenge our intuition when switching among manufacturers.

Many examples on this site assign engineering units to the measured PV because plant software has made the task of unit conversions straightforward. If this is true in your plant, take care when using this formula.

Integral Action

Integral action has the benefit of eliminating offset but presents greater design challenges.

7) Caution: Pay Attention to Units and Scaling

Controller Gain Is Dimensionless in Commercial Systems

In modern plants, process variable (PV) measurement signals are typically scaled to engineering units before they are displayed on the control room HMI computer screen or archived for storage by a process data historian. This is done for good reasons.

When operations staff walk through the plant, the assorted field gauges display the local measurements in engineering units to show that a vessel is operating, for example, at a pressure of 25 psig (1.7 barg) and a temperature of 140 ^{o}C (284 ^{o}F).

It makes sense, then, that the computer screens in the control room display the set point (SP) and PV values in these same familiar engineering units because:

- It helps the operations staff translate their knowledge and intuition from their field experience over to the abstract world of crowded HMI computer displays.
- Familiar units will facilitate the instinctive reactions and rapid decision making that prevents an unusual occurrence from escalating into a crisis situation.
- The process was originally designed in engineering units, so this is how the plant documentation will list the operating specifications.

Controlguru.com Articles Compute Kc With Units

Like a control room display, the Controlguru.com e-book presents PV values in engineering units. In most articles, these PVs are used directly in tuning correlations to compute controller gains, Kc. As a result, the Kc values also carry engineering units.

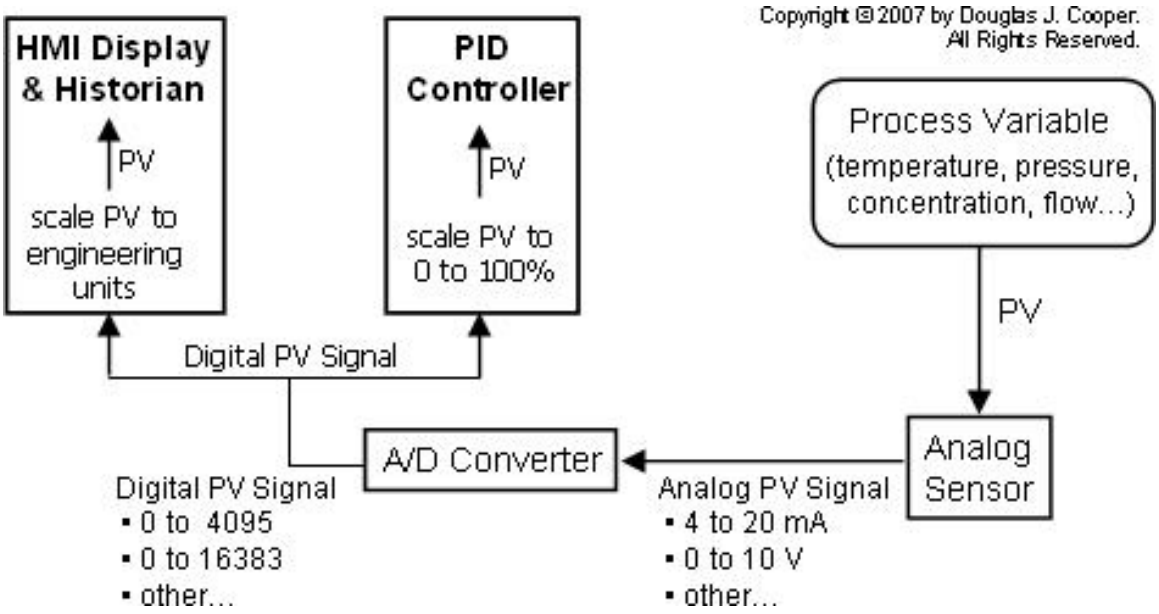
The benefit of this approach is that controller gain maintains the intuitive familiarity that engineering units provide. The difficulty is that commercial controllers are normally configured to use a dimensionless Kc (or dimensionless proportional band, PB).

To address this issue, we explore below how to convert a Kc with engineering units into the standard dimensionless (%/%) form.

The conversion formula presented at the end of this article is reasonably straightforward to use. But it is derived from several subtle concepts that might benefit from explanation. Thus, we begin with a background discussion on units and scaling, and work our way toward our Kc conversion formula goal.

From Analog Sensor to Digital Signal

There are many ways to measure a process variable and move the signal into the digital world for use in a computer based control system. Below is a simplified sketch of one approach ().



Other operations in the pathway from sensor to control system not shown in the simplified sketch might include a transducer, an amplifier, a transmitter, a scaling element, a linearizing element, a signal filter, a multiplexer, and more.

The central issue for this discussion is that the PV signal arrives at the computers and controllers in a raw digital form. The continuous analog PV measurement has been quantized (broken into) a range of discrete increments or digital integer "counts" by an A/D (analog to digital) converter.

More counts dividing the span of a measurement signal increases the resolution of the measurement when expressed as a digital value. The ranges offered by most vendors result from the computer binary 2^n form where *n* is the number of bits of resolution used by the A/D converter.

Example: a 12 bit A/D converter digitizes an analog signal into 2^{12} = 4096 discrete increments normally expressed to range from 0 to 4095 counts.

A 13 bit A/D converter digitizes an analog signal into 2^{13} = 8192 discrete increments normally expressed to range from 0 to 8191 counts.

A 14 bit A/D converter digitizes an analog signal into 2^{14} = 16384 discrete increments normally expressed to range from 0 to 16383 counts.

Example: if a 4 to 20 mA (milliamp) analog signal range is digitized by a 12 bit A/D converter into 0 to 4095 counts, then the resolution is:

(20 – 4 mA) ÷ 4095 counts = 0.00391 mA/count

A signal of 7 mA from an analog range of 4 to 20 mA changes to digital counts from the 12 bit A/D converter as:

(7 – 4 mA) ÷ 0.00391 mA/count = 767 counts

A signal of 1250 counts from a 12 bit A/D converter corresponds to an input signal of 8.89 mA from an analog range of 4 to 20 mA as:

4 mA + (1250 counts)·(0.00391 mA/count) = 8.89 mA

Scaling the Digital PV Signal to Engineering Units for Display
During the configuration phase of a control project, the minimum and maximum (or zero and span) of the PV measurement must be entered. These values are used to scale the digital PV signal to engineering units for display and storage.

> ***Example:*** if a temperature range of 100 ^{o}C to 500 ^{o}C is digitized into 0 to 8191 counts by a 13 bit A/D converter, the signal is scaled for display and storage by setting the minimum digital value of 0 counts = 100 ^{o}C, and maximum digital value of 8191 counts = 500 ^{o}C
>
> Each digital count from the 13 bit A/D converter gives a resolution of:
> (500 – 100 ^{o}C) ÷ 8191 counts = 0.0488 ^{o}C/count
>
> A signal of 175 ^{o}C from an analog range of 100 ^{o}C to 500 ^{o}C changes to digital counts from the 13 bit A/D converter as:
> (175 – 100 ^{o}C) ÷ 0.0488 ^{o}C/count = 1537 counts
>
> A signal of 1250 counts from the 13 bit A/D converter corresponds to an input signal of 161 ^{o}C from an analog range of 100 ^{o}C to 500 ^{o}C as:
> 100 ^{o}C + (1250 counts)·(0.0488 ^{o}C/count) = 161 ^{o}C

As discussed at the top of this article, the intuition and field knowledge of the operations staff is maintained by using engineering units in control room displays and when storing data to a historian.

For this same reason, modern control software uses engineering units when passing variables between the function blocks used for calculations and decision-making. Calculation and decision functions are easier to understand, document and debug when the logic is written using floating point values in common engineering units.

Scaling the Digital PV Signal for Use by the PID Controller
Most commercial PID controllers use a controller gain, Kc (or proportional band, PB) that is expressed as a standard dimensionless %/%.

> Note: Controller gain in commercial controllers is often said to be unitless or dimensionless, but Kc actually has units of (% of CO signal)/(% of PV signal). In a precise mathematical world, these units do not cancel, though there is little harm in speaking as though they do.

Prior to executing the PID controller calculation, the PV signal must be scaled to a standard 0% to 100% to match the "dimensionless" Kc. This happens every loop sample time, T, regardless of whether we are measuring temperature, pressure, flow, or any other process variable.

To perform this scaling, the minimum and maximum PV values in engineering units corresponding to the 0% to 100% standard PV range must be entered during setup and loop configuration.

> ***Example:*** if a temperature range of 100 ^{o}C to 500 ^{o}C is digitized into 0 to 8191 counts by a 13 bit A/D converter, the signal is scaled for the PID control calculation

by setting the minimum digital value of 0 counts = 0%, and the maximum digital value of 8191 counts = 100%.

Each digital count from the 13 bit A/D converter gives a resolution of:
(100 – 0%) ÷ 8191 counts = 0.0122%/count

A signal of 1537 counts (175 ^{o}C) from a 13 bit A/D converter would translate to a signal of 18.75% as:
0% + (1537)·(0.0122%/value) = 18.75%

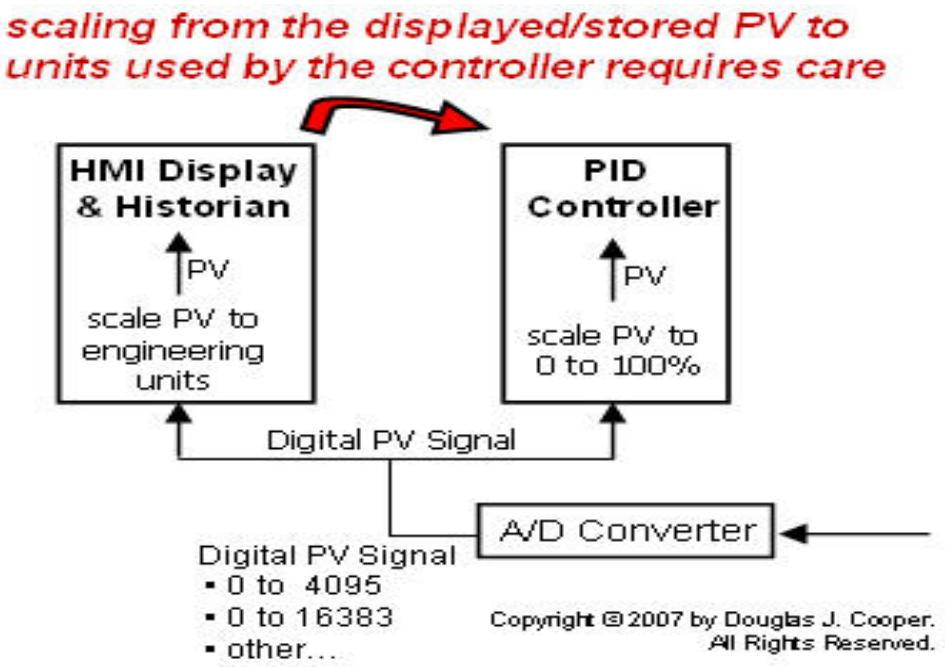
A signal of 1250 counts (161 ^{o}C) from a 13 bit A/D converter would translate to a signal of 15.25% as:
0% + (1250)·(0.0122%/value) = 15.25%

Control Output is 0% to 100%
The controller output (CO) from commercial controllers normally default to a 0% to 100% digital signal as well. Digital to analog (D/A) converters begin the transition of moving the digital CO values into the appropriate electrical current and voltage required by the valve, pump or other final control element (FCE) in the loop.

Note: While CO commonly defaults to a 0% to 100% signal, this may not be appropriate when implementing the outer primary controller in a cascade. The outer primary CO1 becomes the set point of the inner secondary controller, and signal scaling must match. For example, if SP2 is in engineering units, the CO1 signal must be scaled accordingly.

Care Required When Using Engineering Units For Controller Tuning
It is quite common to analyze and design controllers using data retrieved from our process historian or captured from our computer display. Just as with the articles in this e-book, this means the computed Kc values will likely be scaled in engineering units.

The sketch below highlights () that scaling from engineering units to a standard 0% to 100% range used in commercial controllers requires careful attention to detail.



The conversion of PV in engineering units to a standard 0% to 100% range requires knowledge of the maximum and minimum PV values in engineering units. These are the same values that are entered into our PID controller software function block during setup and loop configuration. The general conversion formula is:

$$PV(\%) = \frac{PV - PV_{min}}{PV_{max} - PV_{min}}(100\% - 0\%)$$

where:

PVmax = maximum PV value in engineering units
PVmin = minimum PV value in engineering units
PV = current PV value in engineering units

Example: a temperature signal ranges from 100 ^{o}C to 500 ^{o}C and we seek to scale it to a range of 0% to 100% for use in a PID controller. We set:

PVmin = 100 ^{o}C and PVmax = 500 ^{o}C

A temperature of 175 ^{o}C converts to a standard 0% to 100% range as:

[(175 – 100 ^{o}C) ÷ (500 – 100 ^{o}C)]·(100 – 0%) = 18.75%

A temperature of 161 ^{o}C converts to a standard 0% to 100% range as:

[(161 – 100 ^{o}C) ÷ (500 – 100 ^{o}C)]·(100 – 0%) = 15.25%

Applying Conversion to Controller Gain, Kc

The discussion to this point provides the basis for the formula used to convert Kc from engineering units into dimensionless (%/%):

$$Kc(\%/\%) = Kc(\%/units)\left(\frac{PV_{max}\ units - PV_{min}\ units}{100\% - 0\%}\right)$$

Example: the moderate Kc value in our P-Only control of the heat exchanger study

is Kc = – 0.7 %/ ^{o}C. For this process, PVmax = 250 ^{o}C and PVmin = 0 ^{o}C

Kc = (– 0.7 %/ ^{o}C)·[(250 – 0 ^{o}C) ÷ (100 – 0%)]
= – 1.75 %/%

Example: the moderate value for Kc in our P-Only control of the gravity drained tanks study is Kc = 8 %/ ^{o}C For this process, PVmax = 10 m and PVmin = 0 m

Kc = (8 %/ m)·[(10 – 0 m) ÷ (100 – 0%)]
= 0.8 %/%

Final Thoughts

Textbooks are full of rule-of-thumb guidelines for estimating initial Kc values for a controller depending on whether, for example, it is a flow loop, a temperature loop or a liquid level loop. While we have great reservations with such a "guess and test" approach to tuning, it is important to recognize that such rules are based on a Kc that is expressed in a dimensionless (%/%) form.

8) Integral Action and PI Control

Integral Action and PI Control

Like the P-Only controller, the Proportional-Integral (PI) algorithm computes and transmits a controller output (CO) signal every sample time, T, to the final control element (e.g., valve, variable speed pump). The computed CO from the PI algorithm is influenced by the controller tuning parameters and the controller error, e(t).

PI controllers have two tuning parameters to adjust. While this makes them more challenging to tune than a P-Only controller, they are not as complex as the three parameter PID controller.

Integral action enables PI controllers to eliminate offset, a major weakness of a P-only controller. Thus, PI controllers provide a balance of complexity and capability that makes them by far the most widely used algorithm in process control applications.

The PI Algorithm
While different vendors cast what is essentially the same algorithm in different forms, here we explore what is variously described as the dependent, ideal, continuous, position form:

$$CO = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{Ti}\int e(t)dt$$

Where:
CO = controller output signal (the wire out)
CO_{bias} = controller bias or null value; set by bumpless transfer as explained below
e(t) = current controller error, defined as SP – PV
SP = set point
PV = measured process variable (the wire in)
Kc = controller gain, a tuning parameter
*T*i = reset time, a tuning parameter

The first two terms to the right of the equal sign are identical to the P-Only controller referenced at the top of this article.

The integral mode of the controller is the last term of the equation. Its function is to integrate or continually sum the controller error, e(t), over time.

Some things we should know about the reset time tuning parameter, *T*i:

- It provides a separate weight to the integral term so the influence of integral action can be independently adjusted.
- It is in the denominator so smaller values provide a larger weight to (i.e. increase the influence of) the integral term.
- It has units of time so it is always positive.

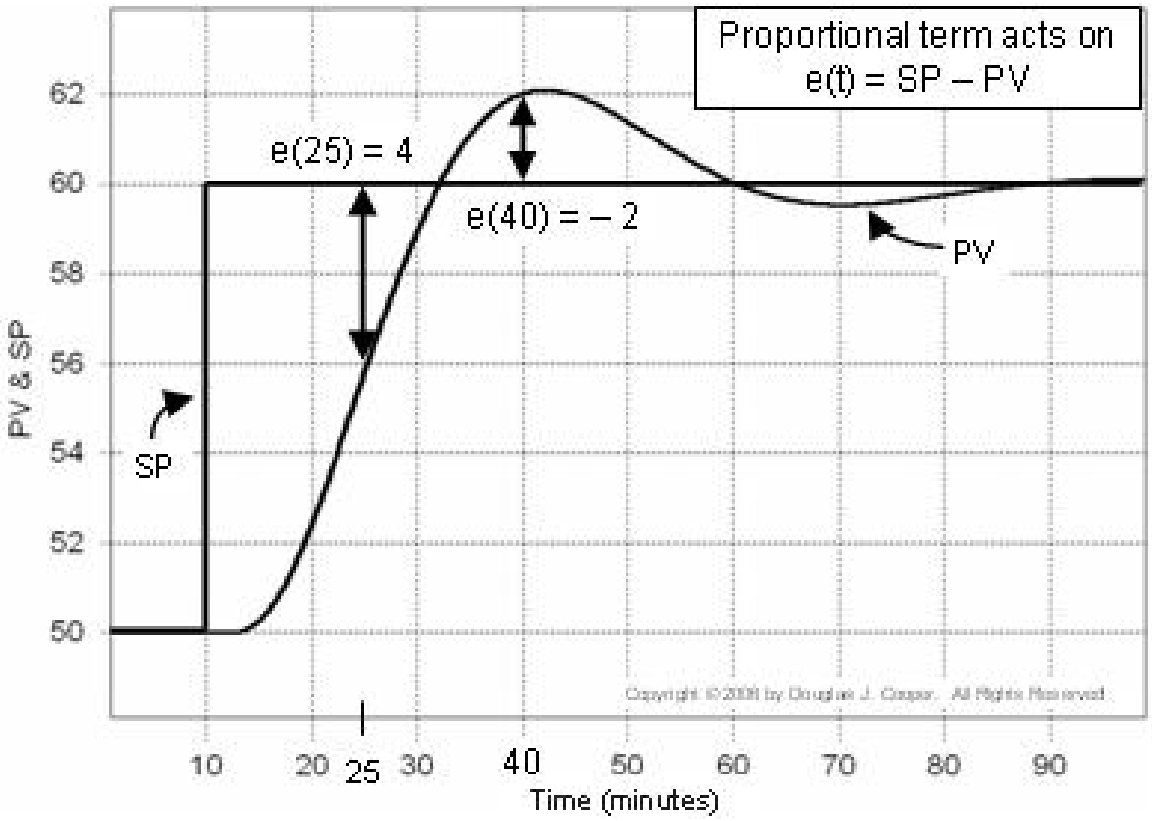
Function of the Proportional Term

As with the P-Only controller, the proportional term of the PI controller, $K_c \cdot e(t)$, adds or subtracts from CO_{bias} based on the size of controller error e(t) at each time t.

As e(t) grows or shrinks, the amount added to CO_{bias} grows or shrinks immediately and proportionately. The past history and current trajectory of the controller error have no influence on the proportional term computation.

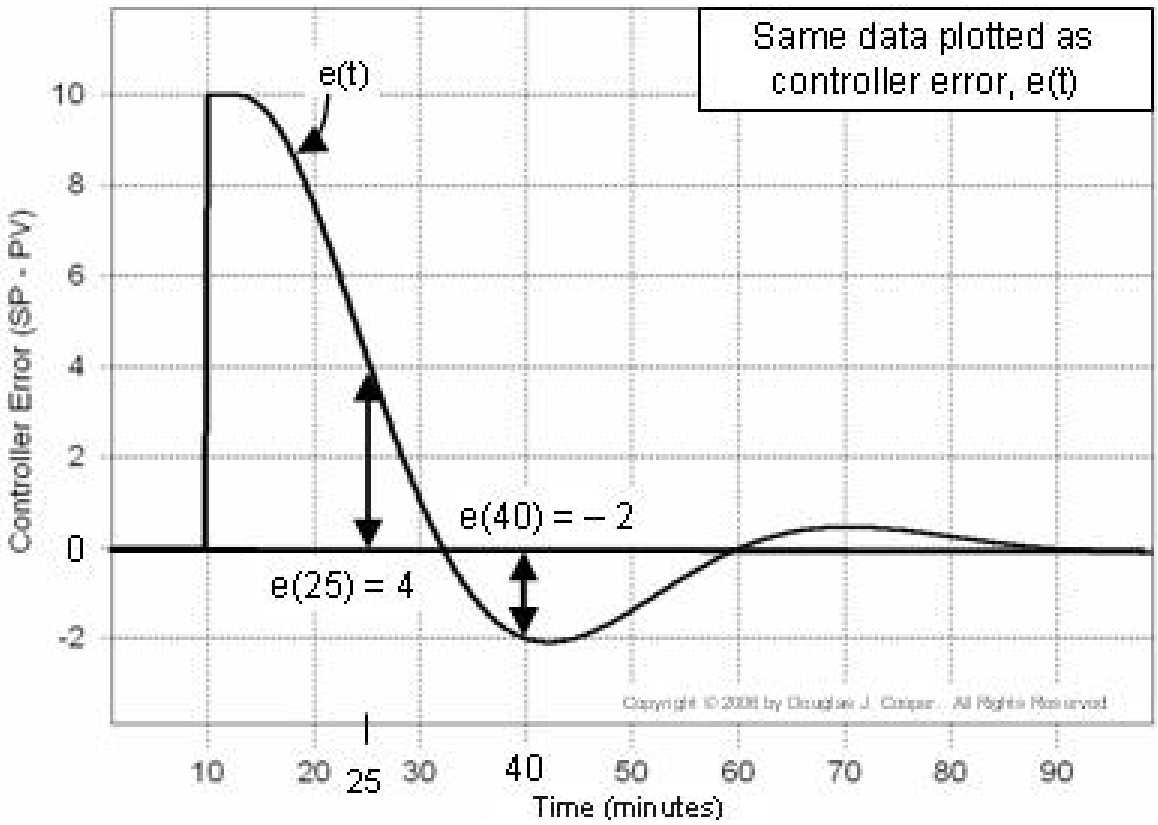
The plot below () illustrates this idea for a set point response. The error used in the proportional calculation is shown on the plot:

- At time t = 25 min, e(25) = 60–56 = 4
- At time t = 40 min, e(40) = 60–62 = –2



Recalling that controller error e(t) = SP – PV, rather than viewing PV and SP as separate traces as we do above, we can compute and plot e(t) at each point in time t.

Below () is the identical data to that above only it is recast as a plot of e(t) itself. Notice that in the plot above, PV = SP = 50 for the first 10 min, while in the error plot below, e(t) = 0 for the same time period.



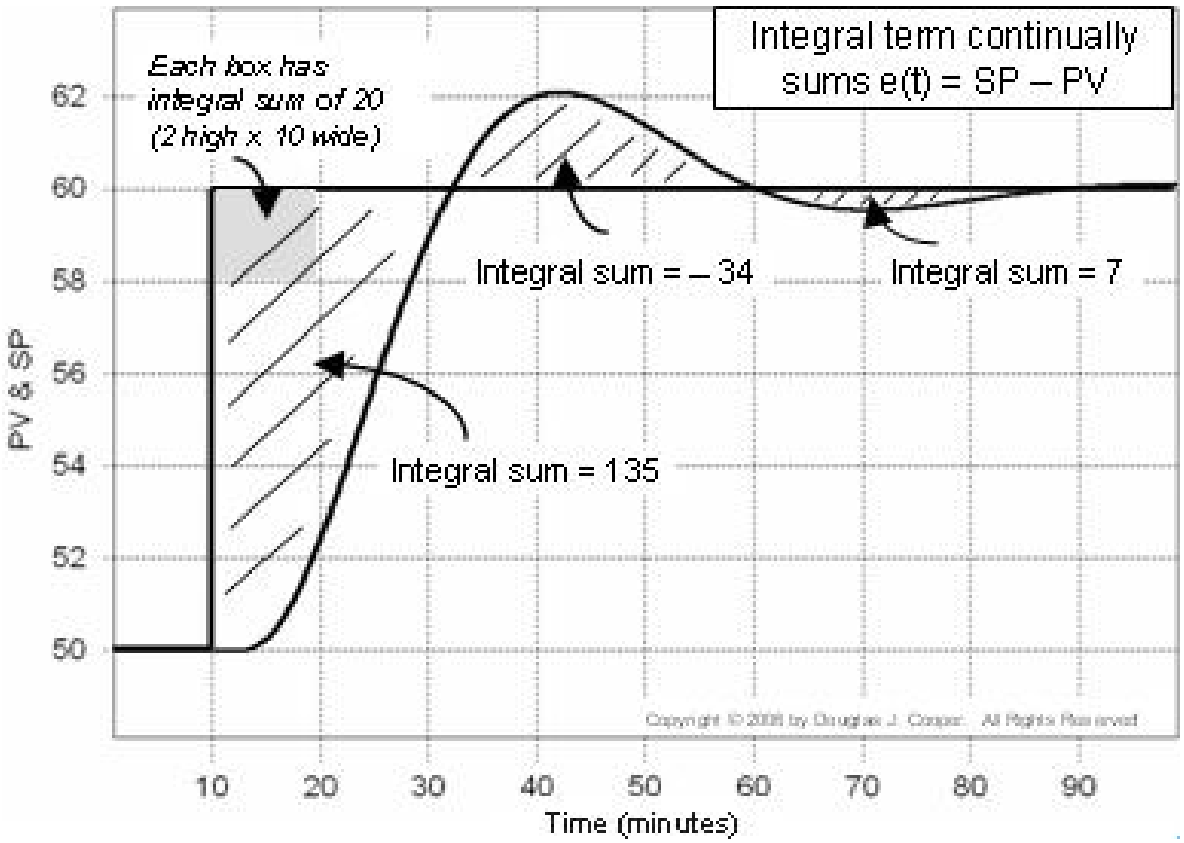
This plot is useful as it helps us visualize how controller error continually changes size and sign as time passes.

Function of the Integral Term

While the proportional term considers the current size of e(t) only at the time of the controller calculation, the integral term considers the history of the error, or how long and how far the measured process variable has been from the set point over time.

Integration is a continual summing. Integration of error over time means that we sum up the complete controller error history up to the present time, starting from when the controller was first switched to automatic.

Controller error is e(t) = SP – PV. In the plot below (), the integral sum of error is computed as the shaded areas between the SP and PV traces.

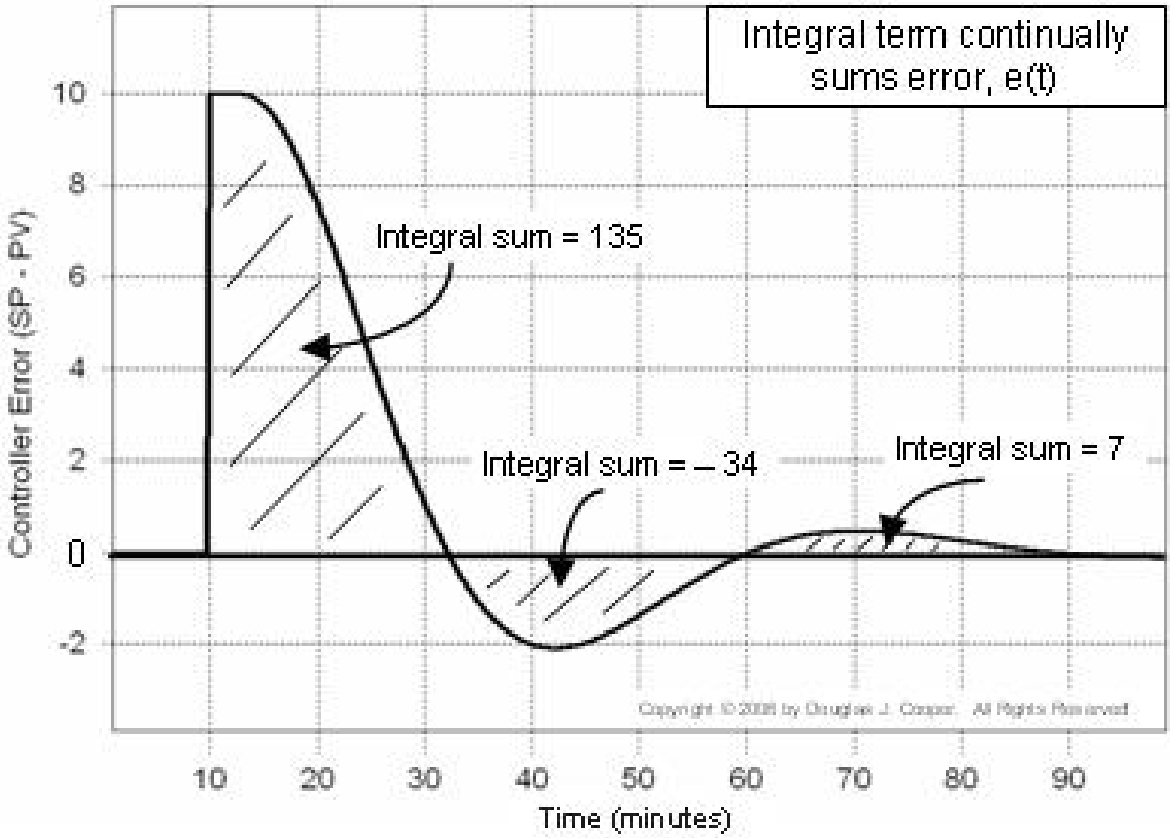


Each box in the plot has an integral sum of 20 (2 high by 10 wide). If we count the number of boxes (including fractions of boxes) contained in the shaded areas, we can compute the integral sum of error.

So when the PV first crosses the set point at around t = 32, the integral sum has grown to about 135. We write the integral term of the PI controller as:

$$\frac{Kc}{Ti}\int_0^{32} e(t)dt = \frac{Kc}{Ti}(135)$$

Since it is controller error that drives the calculation, we get a direct view the situation from a controller error plot as shown below ():



Note that the integral of each shaded portion has the same sign as the error. Since the integral sum starts accumulating when the controller is first put in automatic, the total integral sum grows as long as e(t) is positive and shrinks when it is negative.

At time t = 60 min on the plots, the integral sum is 135 – 34 = 101. The response is largely settled out at t = 90 min, and the integral sum is then 135 – 34 + 7 = 108.

Integral Action Eliminates Offset

The previous sentence makes a subtle yet very important observation. The response is largely complete at time t = 90 min, yet the integral sum of all error is not zero.

In this example, the integral sum has a final or residual value of 108. It is this residual value that enables integral action of the PI controller to eliminate offset.

As discussed in a previous article, most processes under P-only control experience offset during normal operation. Offset is a sustained value for controller error (i.e., PV does not equal SP at steady state).

We recognize from the P-Only controller:

$$CO = CO_{bias} + Kc \cdot e(t)$$

that CO will always equal CO_{bias} unless we add or subtract something from it.

The only way we have something to add or subtract from CO_{bias} in the P-Only equation above is if e(t) is not zero. It e(t) is not steady at zero, then PV does not equal SP and we

have offset.

However, with the PI controller:

$$CO = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{Ti} \int e(t)dt$$

we now know that the integral sum of error can have a final or residual value after a response is complete. This is important because it means that e(t) can be zero, yet we can still have something to add or subtract from CO_{bias} to form the final controller output, CO.

So as long as there is any error (as long as e(t) is not zero), the integral term will grow or shrink in size to impact CO. The changes in CO will only cease when PV equals SP (when e(t) = 0) for a sustained period of time.

At that point, the integral term can have a residual value as just discussed. This residual value from integration, when added to CO_{bias}, essentially creates a new overall bias value that corresponds to the new level of operation.

In effect, integral action continually resets the bias value to eliminate offset as operating level changes.

Challenges of PI Control
There are challenges in employing the PI algorithm:

- The two tuning parameters interact with each other and their influence must be balanced by the designer.
- The integral term tends to increase the oscillatory or rolling behavior of the process response.

Because the two tuning parameters interact with each other, it can be challenging to arrive at “best” tuning values. The value and importance of our design and tuning recipe increases as the controller becomes more complex.

Initializing the Controller for Bumpless Transfer
When we switch any controller from manual mode to automatic (from open loop to closed loop), we want the result to be uneventful. That is, we do not want the switchover to cause abrupt control actions that impact or disrupt our process

We achieve this desired outcome at switchover by initializing the controller integral sum of error to zero. Also, the set point and controller bias value are initialized by setting:
- SP equal to the current PV
- CO_{bias} equal to the current CO

With the integral sum of error set to zero, there is nothing to add or subtract from CO_{bias} that would cause a sudden change in the current controller output. With the set point equal to the measured process variable, there is no error to drive a change in our CO. And with the controller bias set to our current CO value, we are prepared by default to maintain

current operation.

Thus, when we switch from manual mode to automatic, we have “bumpless transfer” with no surprises. This is a result everyone appreciates.

Reset Time Versus Reset Rate
Different vendors cast their control algorithms in slightly different forms. Some use proportional band rather than controller gain. Also, some use reset rate, *T*r, instead of reset time. These are simply the inverse of each other:

$$T_r = 1/T_i$$

No matter how the tuning parameters are expressed, the PI algorithms are all equally capable.

But it is critical to know your manufacturer before you start tuning your controller because parameter values must be matched to your particular algorithm form. Commercial software for controller design and tuning will automatically address this problem for you.

Implementing a PI controller
We explore PI controller design, tuning and implementation on the heat exchanger in this article and the gravity drained tanks in this article.

PI Control of the Heat Exchanger

We investigated P-Only control of the heat exchanger process and learned that while P-Only is an algorithm that is easy to tune and maintain, it has a severe limitation. Specifically, its simple form permits steady state error, called offset, in most processes during normal operation.

Then we moved on to integral action and PI control. We focused in that article on the structure of the algorithm and explored the mathematics of how the proportional and integral terms worked together to eliminate offset.

Here we test the capabilities of the PI controller on the heat exchanger process. Our focus is on design, implementation and basic performance issues. Along the way we will highlight some strengths and weaknesses of this popular algorithm.

As with all controller implementations, best practice is to follow our proven four-step design and tuning recipe as we proceed with this case study.

Step 1: Design Level of Operation (DLO)
Real processes display a nonlinear behavior. That is, their process gain, time constant and/or dead time changes as operating level changes and as major disturbances change. Since controller design and tuning is based on these process Kp, *T*p and Θp values, controllers should be designed and tuned for a specific level of operation.

Thus, the first step in our controller design recipe is to specify our design level of operation

(DLO). This includes stating:

- Where we expect the set point, SP, and measured process variable, PV, to be during normal operation.
- The range of values the SP and PV might assume so we can explore the nature of the process dynamics across that range.

We will track along with the same design conditions used in the P-Only control study to permit a direct comparison of performance and capability. As in that study, we specify:

- Design PV and SP = 138 °C with range of 138 to 140 °C

We also should know normal or typical values for our major disturbances and be reasonably confident that they are quiet so we may proceed with a bump test. The heat exchanger process has only one major disturbance variable, and consistent with the previous study:

- Expected warm liquid flow disturbance = 10 L/min

Step 2: Collect Data at the DLO
The next step in the design recipe is to collect dynamic process data as near as practical to our design level of operation. We have previously collected and documented heat exchanger step test data that matches our design conditions.

Step 3: Fit an FOPDT Model to the Design Data
Here we document a first order plus dead time (FOPDT) model approximation of the step test data from step 2:
- Process gain (how far), Kp = –0.53 °C/%
- Time constant (how fast), τp = 1.3 min
- Dead time (how much delay), Θp = 0.8 min

Step 4: Use the Parameters to Complete the Design
One common form of the PI controller computes a controller output (CO) action every loop sample time T as:

$$CO = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{\tau i} \int e(t)dt$$

Where:
CO = controller output signal (the wire out)
CO_{bias} = controller bias or null value; set by bumpless transfer as explained below
e(t) = current controller error, defined as SP – PV
SP = set point
PV = measured process variable (the wire in)
Kc = controller gain, a tuning parameter
τi = reset time, a tuning parameter

• *Loop Sample Time, T*
Best practice is to specify loop sample time, T, at 10 times per time constant or faster (T ≤ 0.1τp). For this study, T ≤ 0.13 min = 8 sec. Faster sampling may provide modestly improved performance, while slower sampling can lead to significantly degraded performance. Most commercial controllers offer an option of T = 1.0 sec, and since this

meets our design rule, we use that here.

• *Computing controller error, e(t)*
Set point, SP, is something we enter into the controller. The PV measurement comes from our sensor (our wire in). With SP and PV known, controller error, e(t) = SP – PV, can be directly computed at every loop sample time T.

• *Determining Bias Value*
Strictly speaking, CO_{bias} is the value of the CO that, in manual mode, causes the PV to steady at the DLO while the major disturbances are quiet and at their normal or expected values.

• *Bumpless Transfer*
A desirable feature of the PI algorithm is that it is able to eliminate the offset that can occur under P-Only control. The integral term of the PI controller provides this capability by providing updated information that, when combined with the controller bias, keeps the process centered as conditions change.

Since integral action acts to update (or reset) our bias value over time, CO_{bias} can be initialized in a straightforward fashion to a value that produces no abrupt control actions when we switch to automatic. Most commercial controllers do this with a simple "bumpless transfer" feature. When switching to automatic, they initialize:
- SP equal to the current PV
- CO_{bias} equal to the current CO

With the set point equal to the measured process variable, there is no error to drive a change in our controller output. And with the controller bias set to our current controller output, we are prepared by default to maintain current operation.

We will use a controller that employs these bumpless transfer rules when we switch to automatic. Hence, we need not specify any value for CO_{bias} as part of our design.

• *Computing Controller Gain and Reset Time*
Here we use the industry-proven Internal Model Control (IMC) tuning correlations. The first step in using the IMC correlations is to compute *T*c, the closed loop time constant. All time constants describe the speed or quickness of a response. The closed loop time constant describes the desired speed or quickness of a controller in responding to a set point change or rejecting a disturbance.

If we want an active or quickly responding controller and can tolerate some overshoot and oscillation as the PV settles out, we want a small *T*c (a short response time) and should choose *aggressive* tuning:
- *aggressive:* *T*c is the larger of $0.1 \cdot T_p$ or $0.8 \cdot \Theta_p$

Moderate tuning is for a controller that will move the PV reasonably fast while producing little to no overshoot.
- *moderate*: *T*c is the larger of $1 \cdot T_p$ or $8 \cdot \Theta_p$

If we seek a more sluggish controller that will move things in the proper direction, but quite slowly, we choose *conservative* tuning (a big or long *T*c).

▪ *conservative*: *T*c is the larger of $10 \cdot T$p or $80 \cdot \Theta$p

Once we have decided on our desired performance and computed the closed loop time constant, *T*c, with the above rules, then the PI correlations for controller gain, Kc, and reset time, *T*i, are:

$$Kc = \frac{1}{Kp}\frac{Tp}{(\theta p + Tc)} \quad \text{and} \quad Ti = Tp$$

Notice that reset time, *T*i, is always set equal to the time constant of the process, regardless of desired controller activity.

a) *Moderate Response Tuning:*
For a controller that will move the PV reasonably fast while producing little to no overshoot, choose:

Moderate *T*c = the larger of $1 \cdot T$p or $8 \cdot \Theta$p
= larger of 1(1.3 min) or 8(0.8 min)
= 6.4 min

Using this *T*c and our model parameters in the tuning correlations above, we arrive at the moderate tuning values:

$$Kc = \frac{1}{-0.53}\frac{1.3}{(0.8 + 6.4)} = -0.34\,\frac{\%}{^{\circ}C} \quad \text{and} \quad Ti = 1.3 \text{ min}$$

b) *Aggressive Response Tuning:*
For an active or quickly responding controller where we can tolerate some overshoot and oscillation as the PV settles out, specify

Aggressive *T*c = the larger of $0.1 \cdot T$p or $0.8 \cdot \Theta$p
= larger of 0.1(1.3 min) or 0.8(0.8 min)
= 0.64 min

and the aggressive tuning values are:

$$Kc = \frac{1}{-0.53}\frac{1.3}{(0.8 + 0.64)} = -1.7\,\frac{\%}{^{\circ}C} \quad \text{and} \quad Ti = 1.3 \text{ min}$$

Practitioner's Note: The FOPDT model parameters used in the tuning correlations above have engineering units, so the Kc values we compute also have engineering units. In commercial control systems, controller gain (or proportional band) is normally entered as a dimensionless (%/%) value.

For commercial implementations, we could:
▪ Scale the process data before fitting our FOPDT dynamic model so we directly compute a dimensionless Kc.
▪ Convert the model Kp to dimensionless %/% after fitting the model but before using the FOPDT parameters in the tuning correlations.
▪ Convert Kc from engineering units into dimensionless %/% after using the tuning

correlations.

CO is already scaled from 0 – 100% in the above example. Thus, we convert Kc from engineering units into dimensionless %/% using the formula:

$$Kc(\%/\%) = Kc(\%/\text{units})\left(\frac{PV_{max}\ \text{units} - PV_{min}\ \text{units}}{100\% - 0\%}\right)$$

For the heat exchanger, PVmax = 250 ^{o}C and PVmin = 0 ^{o}C. The dimensionless Kc values are thus computed:

- moderate Kc = (– 0.34 %/ ^{o}C)·[(250 – 0 ^{o}C) ÷ (100 – 0%)] = – 0.85 %/%
- aggressive Kc = (– 1.7%/ ^{o}C)·[(250 – 0 ^{o}C) ÷ (100 – 0%)] = – 4.2 %/%

We use Kc with engineering units in the remainder of this article and are careful that our PI controller is formulated to accept such values. We would be mindful if we were using a commercial control system, however, to ensure our tuning parameters are cast in the form appropriate for our equipment.

• *Controller Action*

The process gain, Kp, is negative for the heat exchanger, indicating that when CO increases, the PV decreases in response. This behavior is characteristic of a reverse acting process. Given this CO to PV relationship, when in automatic mode (closed loop), if the PV starts drifting above set point, the controller must increase CO to correct the error. Such negative feedback is an essential component of stable controller design.

A process that is naturally reverse acting requires a controller that is direct acting to remain stable. In spite of the opposite labels (reverse acting process and direct acting controller), the details presented above show that both Kp and Kc are negative values.

In most commercial controllers, only positive Kc values can be entered. The sign (or action) of the controller is then assigned by specifying that the controller is either reverse acting or direct acting to indicate a positive or negative Kc, respectively.

If the wrong control action is entered, the controller will quickly drive the final control element (FCE) to full on/open or full off/closed and remain there until a proper control action entry is made.

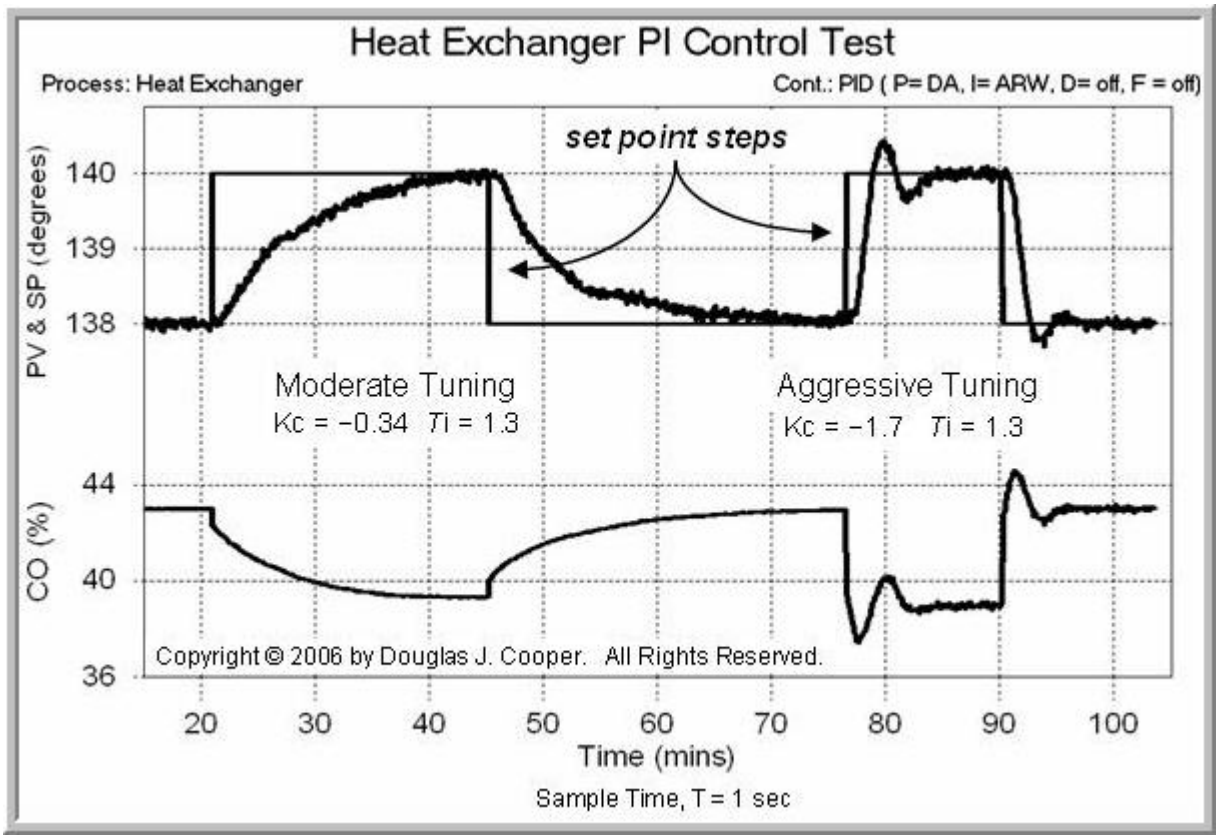
Implement and Test

Below we test our two PI controllers on the heat exchanger process simulation. Shown are two set points step pairs from 138 °C up to 140 °C and back again.

The first set point steps to the left show the PI controller performance using the moderate tuning values computed above. The second set point steps to the right show the controller performance using the aggressive tuning values. Note that the warm liquid disturbance flow, though not shown, remains constant at 10 L/min throughout the study.

(For comparison, the performance of the P-Only controller in tracking these set point

changes is pictured here).



The asymmetrical behavior of the PV for the set point steps up compared to the steps down is due to the very nonlinear character of the heat exchanger.

If we seek tuning between moderate and aggressive performance, we would average the Kc values from the tuning rules above.

But if we believe we had collected good bump test data (we saw a clear response in the PV when we stepped the CO and the major disturbances were quiet during the test), and the FOPDT model fit appears to be visually descriptive of the data, then we have a good value for *T*p and that means a good value for *T*i.

If we are going to fiddle with the tuning, we can tweak Kc and we should leave the reset time alone.

Tuning Recipe Saves Time and Money
The exciting result is that we achieved our desired controller performance based on one bump test and following a controller design recipe. No trial and error was involved. Little off-spec product was produced. No time was wasted.

Soon we will see how software tools help us achieve such results with even less disruption to the process.

The method of approximating complex behavior with a FOPDT model and then following a recipe for controller design and tuning has been used successfully on a broad spectrum of processes with streams composed of gases, liquids, powders, slurries and melts. It is a

reliable approach that has been proven time and again at diverse plants from a wide range of companies.

PI Disturbance Rejection Of The Gravity Drained Tanks

When exploring the capabilities of the P-Only controller in rejecting disturbances for the gravity drained tanks process, we confirmed the observations we had made during the the P-Only set point tracking study for the heat exchanger.

In particular, the P-Only algorithm is easy to tune and maintain, but whenever the set point or a major disturbance moves the process from the design level of operation, a sustained error between the process variable (PV) and set point (SP), called offset, results.

Further, we saw in both case studies that as controller gain, Kc, increases (or as proportional band, PB, decreases):

- the activity of the controller output, CO, increases
- the oscillatory nature of the response increases
- the offset (sustained error) decreases

In this article, we explore the benefits of integral action and the capabilities of the PI controller for rejecting disturbances in the gravity drained tanks process. We have previously presented the fundamentals behind PI control and its application to set point tracking in the heat exchanger.

As with all controller implementations, best practice is to follow our proven four-step design and tuning recipe. One benefit of the recipe is that steps 1-3, summarized below from our P-Only study, remain the same regardless of the control algorithm being employed. After summarizing steps 1-3, we complete the PI controller design and tuning in step 4.

Step 1: Determine the Design Level of Operation (DLO)
The control objective is to reject disturbances as we control liquid level in the lower tank. Our design level of operation (DLO), detailed here for this study is:

- design PV and SP = 2.2 m with range of 2.0 to 2.4 m
- design D = 2 L/min with occasional spikes up to 5 L/min

Step 2: Collect Process Data around the DLO
When CO, PV and D are steady near the design level of operation, we bump the CO as detailed here and force a clear response in the PV that dominates the noise.

Step 3: Fit a FOPDT Model to the Dynamic Process Data
We then describe the process behavior by fitting an approximating first order plus dead time (FOPDT) dynamic model to the test data from step 2. We define the model parameters and present details of the model fit of step test data here. A model fit of doublet test data using commercial software confirms these values:

- process gain (how far), Kp = 0.09 m/%
- time constant (how fast), *T*p = 1.4 min

▪ dead time (how much delay), Θp = 0.5 min

Step 4: Use the FOPDT Parameters to Complete the Design
Following the heat exchanger PI control study, we explore what is often called the dependent, ideal form of the PI control algorithm:

$$CO = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{Ti}\int e(t)dt$$

Where:
CO = controller output signal (the wire out)
CO_{bias} = controller bias or null value; set by bumpless transfer
e(t) = current controller error, defined as SP – PV
SP = set point
PV = measured process variable (the wire in)
Kc = controller gain, a tuning parameter
*T*i = reset time, a tuning parameter

Aside: our observations using the dependent ideal PI algorithm directly apply to the other popular PI controller forms. For example, the integral gain, Ki, in the independent algorithm form:

$$CO = CO_{bias} + Kc \cdot e(t) + Ki\int e(t)dt$$

can be computed directly from controller gain and reset time as: Ki = Kc/*T*i.

In the P-Only study, we established that for the gravity drained tanks process:
▪ sample time, T = 1 sec
▪ the controller is reverse acting
▪ dead time is small compared to *T*p and thus not a concern in the design

• *Controller Gain,* Kc*, and Reset Time, T*i
We use our FOPDT model parameters in the industry-proven Internal Model Control (IMC) tuning correlations to compute PI tuning values.

The first step in using the IMC correlations is to compute *T*c, the closed loop time constant. All time constants describe the speed or quickness of a response. *T*c describes the desired speed or quickness of a controller in responding to a set point change or rejecting a disturbance.

If we want an active or quickly responding controller and can tolerate some overshoot and oscillation as the PV settles out, we want a small *T*c (a short response time) and should choose aggressive tuning:
▪ *Aggressive Response*: *T*c is the larger of $0.1 \cdot T$p or $0.8 \cdot \Theta$p

If we seek a sluggish controller that will move things in the proper direction, but quite slowly, we choose conservative tuning (a big or long *T*c).
▪ *Conservative Response*: *T*c is the larger of $10 \cdot T$p or $80 \cdot \Theta$p

Moderate tuning is for a controller that will move the PV reasonably fast while producing little to no overshoot.

▪ *Moderate Response*: *T*c is the larger of $1 \cdot T_p$ or $8 \cdot \Theta_p$

With *T*c computed, the PI controller gain, Kc, and reset time, *T*i, are computed as:

$$K_c = \frac{1}{K_p}\frac{T_p}{(\theta_p + T_c)} \quad \text{and} \quad T_i = T_p$$

Notice that reset time, *T*i, is always equal to the process time constant, *T*p, regardless of desired controller activity.

a) *Moderate Response Tuning:*
For a controller that will move the PV reasonably fast while producing little to no overshoot, choose:

Moderate *T*c = the larger of $1 \cdot T_p$ or $8 \cdot \Theta_p$
= larger of 1(1.4 min) or 8(0.5 min)
= 4 min

Using this *T*c and our model parameters in the tuning correlations above, we arrive at the moderate tuning values:

$$K_c = \frac{1}{0.09}\frac{1.4}{(0.5+4)} = 3.5\frac{\%}{m} \quad \text{and} \quad T_i = 1.4 \text{ min}$$

b) *Aggressive Response Tuning:*
For an active or quickly responding controller where we can tolerate some overshoot and oscillation as the PV settles out, specify:

Aggressive *T*c = the larger of $0.1 \cdot T_p$ or $0.8 \cdot \Theta_p$
= larger of 0.1(1.4 min) or 0.8(0.5 min)
= 0.4 min

and the aggressive tuning values are:

$$K_c = \frac{1}{0.09}\frac{1.4}{(0.5+0.4)} = 17\frac{\%}{m} \quad \text{and} \quad T_i = 1.4 \text{ min}$$

Practitioner's Note: The FOPDT model parameters used in the tuning correlations above have engineering units, so the Kc values we compute also have engineering units. In commercial control systems, controller gain (or proportional band) is normally entered as a dimensionless (%/%) value.

To address this, we could:
▪ Scale the process data before fitting our FOPDT dynamic model so we directly compute a dimensionless Kc.

▪ Convert the model Kp to dimensionless %/% after fitting the model but before using the FOPDT parameters in the tuning correlations.
▪ Convert Kc from engineering units into dimensionless %/% after using the tuning correlations.

Since we already have Kc in engineering units, we employ the third option. CO is already scaled from 0 – 100% in the above example. Thus, we convert Kc from engineering units into dimensionless %/% using the formula:

$$Kc(\%/\%) = Kc(\%/\text{units})\left(\frac{PV_{max}\ \text{units} - PV_{min}\ \text{units}}{100\% - 0\%}\right)$$

For the gravity drained tanks, PVmax = 10 m and PVmin = 0 m. The dimensionless Kc values are thus computed:

- moderate Kc = (3.5 %/m)·[(10 – 0 m) ÷ (100 – 0%)] = 0.35 %/%
- aggressive Kc = (17 %/m)·[(10 – 0 m) ÷ (100 – 0%)] = 1.7 %/%

We use the Kc with engineering units in the remainder of this article and are careful that our PI controller is formulated to accept such values. If we were using these results in a commercial control system, we would be careful to ensure our tuning parameters are cast in the form appropriate for our equipment.

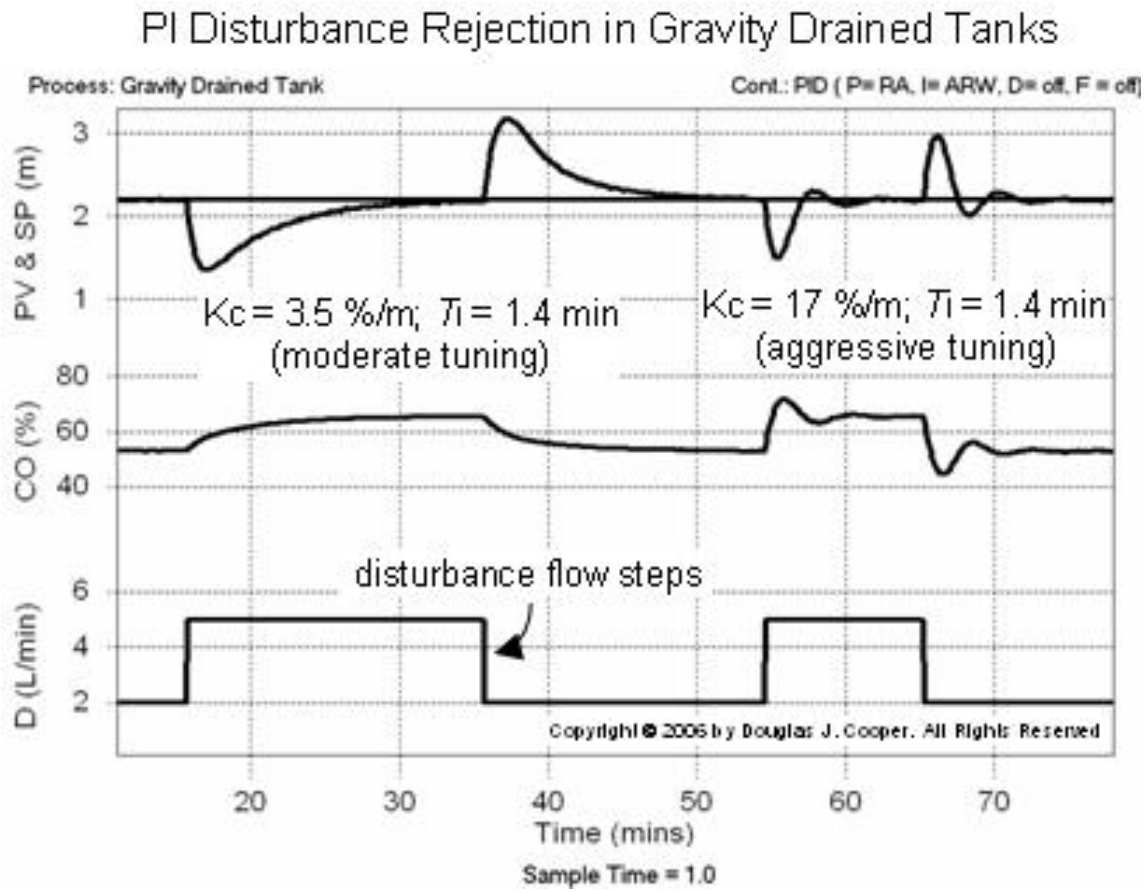
• *Controller Action*
The process gain, Kp, is positive for the gravity drained tanks, indicating that when CO increases, the PV increases in response. This behavior is characteristic of a direct acting process. Given this CO to PV relationship, when in automatic mode (closed loop), if the PV starts drifting above set point, the controller must decrease CO to correct the error. Such negative feedback is an essential component of stable controller design.

A process that is naturally direct acting requires a controller that is reverse acting to remain stable. In spite of the opposite labels (direct acting process and reverse acting controller), the details presented above show that both Kp and Kc are positive values.

In most commercial controllers, only positive Kc values can be entered. The sign (or action) of the controller is then assigned by specifying that the controller is either reverse acting or direct acting to indicate a positive or negative Kc, respectively.

If the wrong control action is entered, the controller will quickly drive the final control element (FCE) to full on/open or full off/closed and remain there until a proper control action entry is made.

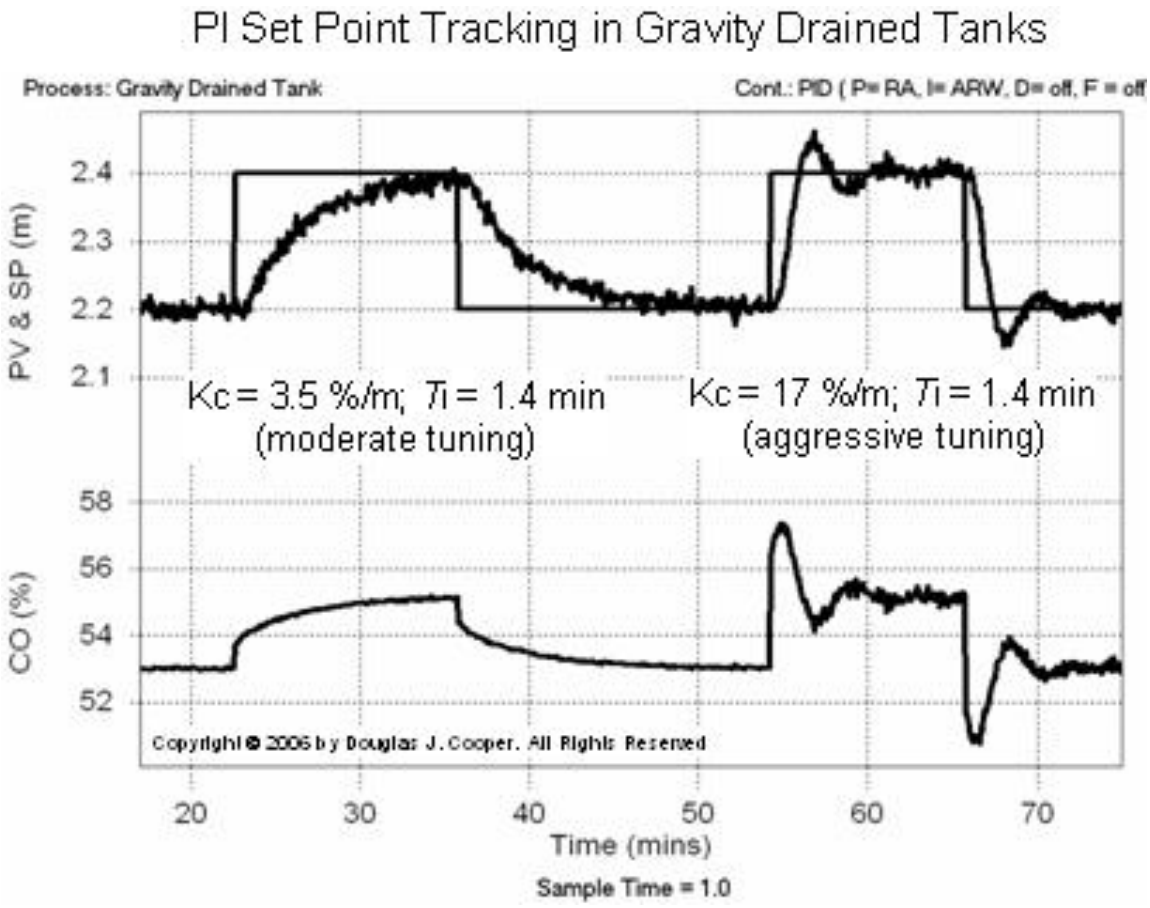
Implement and Test
The ability of the PI controller to reject changes in the pumped flow disturbance, D, is pictured below () for the moderate and aggressive tuning values computed above. Note that the set point remains constant at 2.2 m throughout the study.



The aggressive controller shows a more energetic CO action, and thus, a more active PV response. As shown above, however, the penalty for this increased activity is some overshoot and oscillation in the process response.

Please be aware that the terms "moderate" and "aggressive" hold no magic. If we desire a control performance between the two, we need only average the Kc values from the tuning rules above. Note, however, that these rules provide a constant reset time, *T*i, regardless of our desired performance. So if we believe we have collected a good process data set, and the FOPDT model fit looks like a reasonable approximation of this data, then *T*i = *T*p always.

While not our design objective, presented below is the set point tracking ability of the PI controller () when the disturbance flow is held constant:



Again, the aggressive tuning values provide for a more active response.

Aside: it may appear that the random noise in the PV measurement signal is different in the two plots above, but it is indeed the same. Note that the span of the PV axis in each plot differs by a factor of four. The narrow span of the set point tracking plot greatly magnifies the signal traces, making the noise more visible.

Comparison With P-Only Control

The performance of a P-Only controller in addressing the same disturbance rejection and set point tracking challenge is shown here. A comparison of that study with the results presented here reveals that PI controllers:

- can eliminate the offset associated with P-Only control,
- have integral action that increases the tendency for the PV to roll (or oscillate),
- have two tuning parameters that interact, increasing the challenge to correct tuning when performance is not acceptable.

Derivative Action

The addition of the derivative term to complete the PID algorithm provides modest benefit yet significant challenges.

The Challenge of Interacting Tuning Parameters

Many process control practitioners tune by "intuition," fiddling their way to final tuning by a

combination of experience and trial-and-error.

Some are quite good at approaching process control as art. Since they are the ones who define “best” performance based on the goals of production, the capabilities of the process, the impact on down stream units, and the desires of management, it can be difficult to challenge any claims of success.

To explore the pitfalls of a trial and error approach and reinforce that there is science to controller tuning, we consider the common dependent, ideal form of the PI controller:

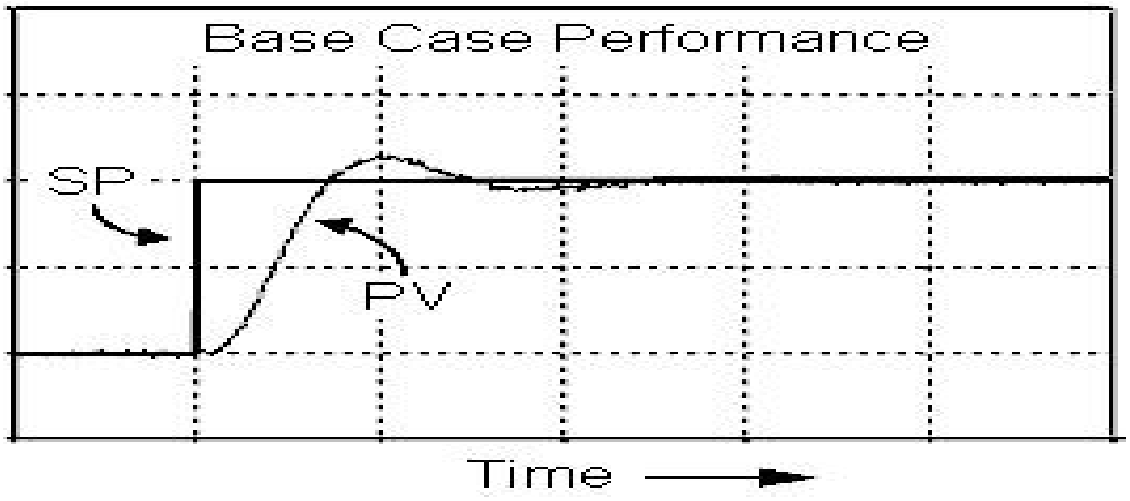
$$CO = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{Ti} \int e(t)dt$$

Where:
CO = controller output
e(t) = controller error = set point – process variable = SP – PV
Kc = controller gain, a tuning parameter
*T*i = reset time, a tuning parameter

For this form, controller activity or aggressiveness increases as Kc increases and as *T*i decreases (*T*i is in the denominator, so smaller values increase the weighting on the integral action term, thus increasing controller activity).

Since Kc and *T*i individually can make a controller more or less aggressive in its response, the two tuning parameters interact with each other. If current controller performance is not what we desire, it is not always clear which value to raise or lower, or by how much.

Example of Interaction Confusion
To illustrate, consider a case where we seek to balance a fairly rapid response to a set point change (a short rise time) against a small overshoot. While every process application is different, we choose to call the response plot below our desired or base case performance.



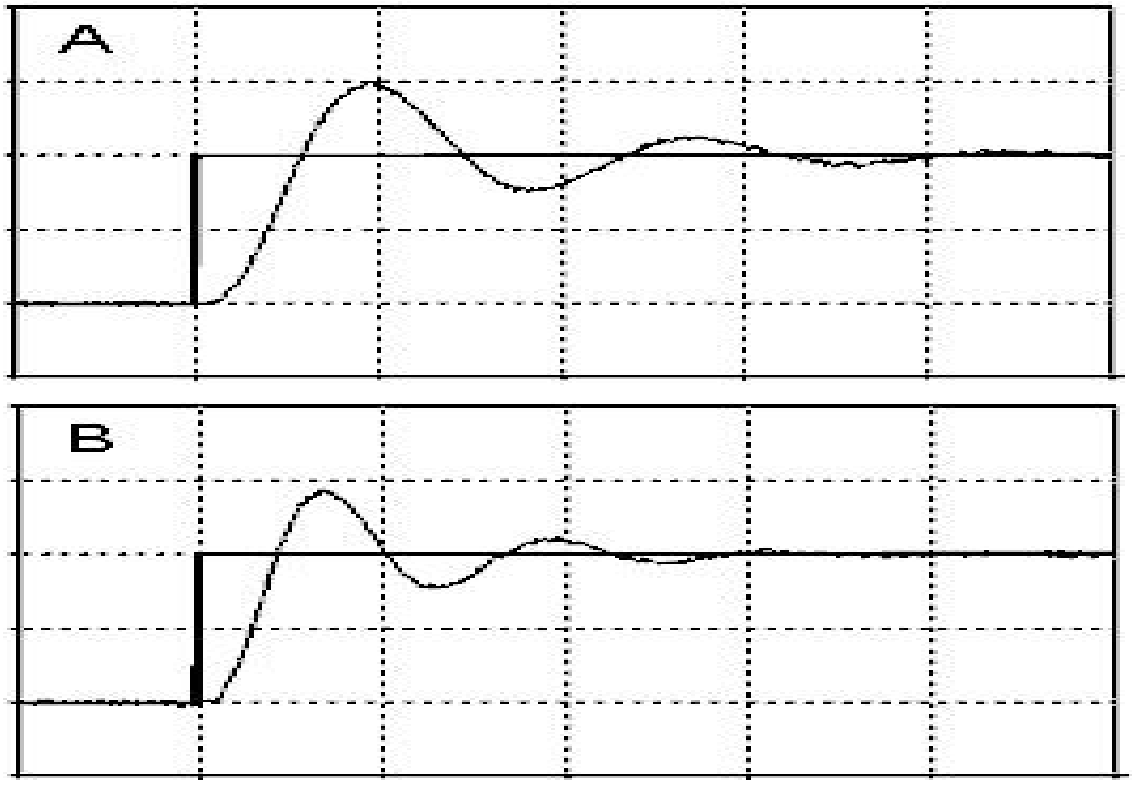
Now consider the two response plots below. These were made using the identical process and controller to that above. The only difference between the base case response above

and plot A and plot B below is that different Kc and *T*i tuning values were used in each one.

And now the question: what tuning adjustments are required to restore the desired base case performance above starting from each plot below? Or alternatively: how has the tuning been changed from base case performance to produce these different behaviors?

There are no tricks in this question. The "process" is a simple linear second order system with modest dead time. Controller output is not hitting any limits. The scales on the plot are identical. Everything is as it seems, except PI controller tuning is different in each case.

Study the plots for a moment before reading ahead and see if you can figure it out. Each plot has a very different answer.



Some Hints

Before we reveal the answer, here is a hint. One plot has been made more active or aggressive in its response by doubling Kc while keeping *T*i constant at the original base case value.

The other cuts *T*i in half (remember, decreasing *T*i makes this PI form more active) while keeping Kc at the base case value:

So we have:

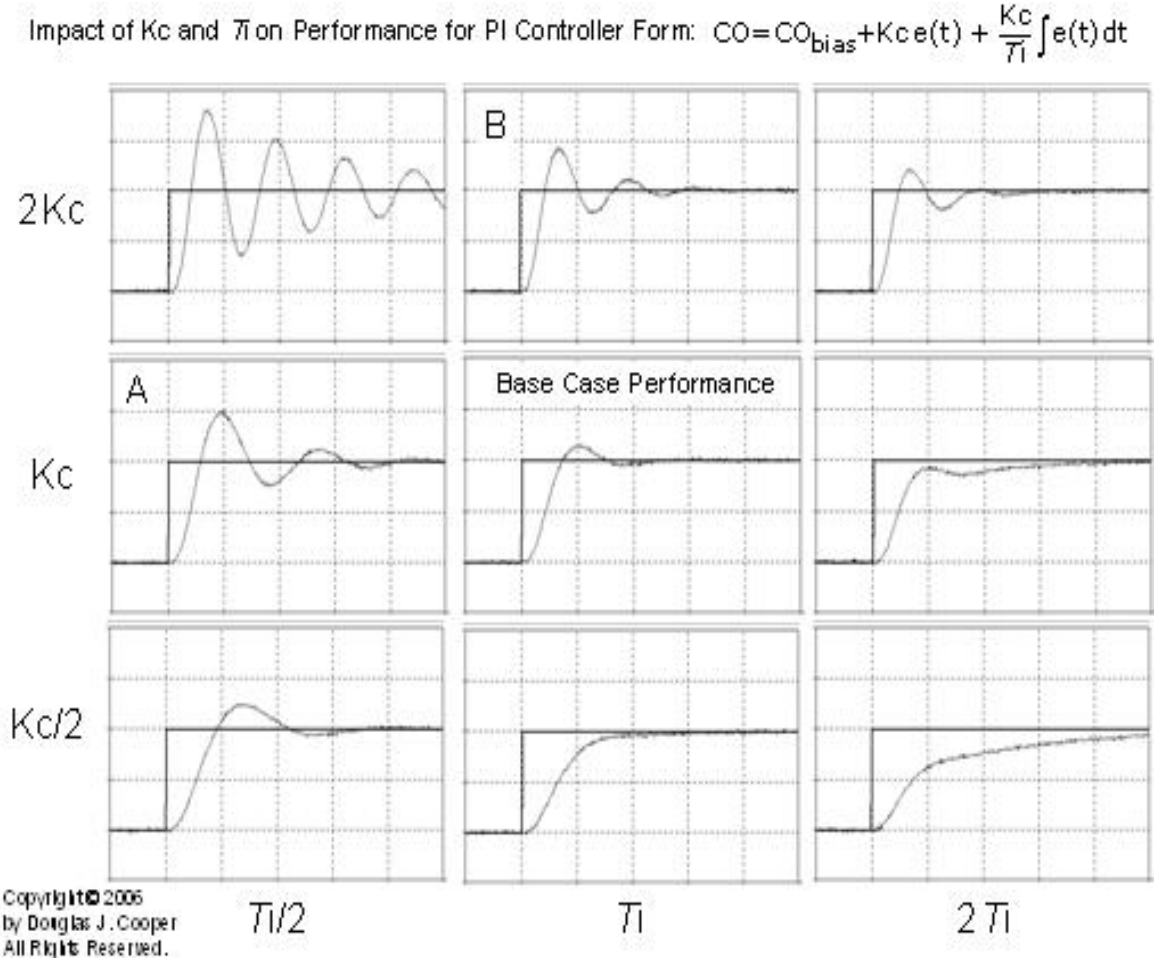
- Base case = Kc and *T*i
- Plot A or B = 2Kc and *T*i
- Other Plot B or A = Kc and *T*i/2

Still not sure? Here is a final hint: remember from our previous discussions that

proportional action is largely responsible for the first movements in a response. We also discussed that integral action tends to increase the oscillatory or cycling behavior in the PV.It is not easy to know the answer, even with these huge hints, and that is the point of this article.

The Answer

Below is a complete tuning map () with the base case performance from our challenge problem in the center. The plot shows how performance changes as Kc and *T*i are doubled and halved from the base case for the dependent, ideal PI controller form.



Starting from the center and moving up on the map from the base case performance brings us to plot B. As indicated on the tuning map axis, this direction increases (doubles) controller gain, Kc, thus making the controller more active or aggressive. Moving down on the map from the base case decreases (halves) Kc, making the controller more sluggish in its response.

Moving left on the map from the base case brings us to plot A. As indicated on the tuning map axis, this direction decreases reset time (cuts it in half), again making the controller more active or aggressive. Moving right on the map from the base case increases (doubles) reset time, making the controller more sluggish in its response.

It is clear from the tuning map that the controller is more active or aggressive in its

response when Kc increases and *T*i decreases, and more sluggish or conservative when Kc decreases and *T*i increases.

Building on this observation, it is not surprising that the upper left most plot (2Kc and *T*i/2) shows the most active controller response, and the lower right most plot (Kc/2 and 2*T*i) is the most conservative or sluggish response.

Back to the question. With what we now know, the answer:
- Base case = Kc and *T*i
- Plot B = 2Kc and *T*i
- Plot A = Kc and *T*i/2

Interacting Parameters Makes Tuning Problematic
The PI controller has only two tuning parameters, yet it produces very similar looking performance plots located in different places on a tuning map.

If our instincts lead us to believe that we are at plot A when we really are at plot B, then the corrective action we make based on this instinct will compound our problem rather than solve it. This is strong evidence that trial and error is not an efficient or appropriate approach to tuning.

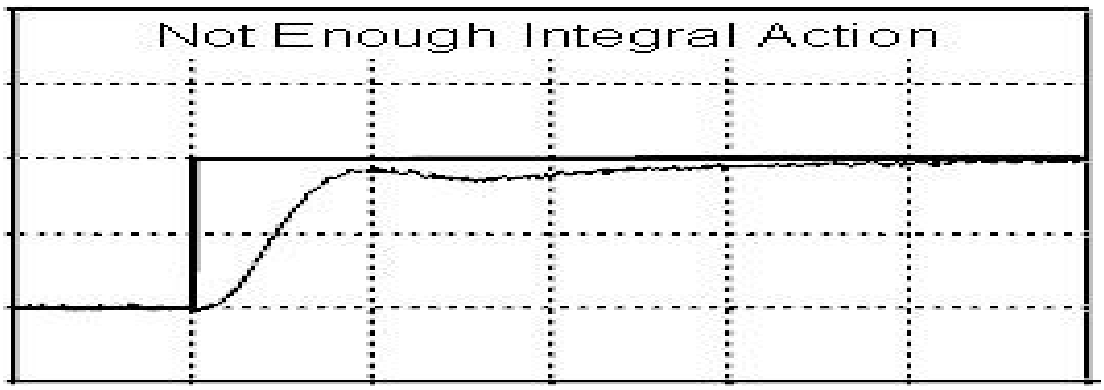
When we consider a PID controller with three tuning parameters, the number of similar looking plots in what would be a three dimensional tuning map increases dramatically. Trial and error tuning becomes almost futile.

We have been exploring a step by step tuning recipe approach that produces desired results without the wasted time and off-spec product that results from trial and error tuning.

If we follow this industry-proven methodology, we will improve the safety and profitability of our operation.

Interesting Observation
Before leaving this subject, we make one more very useful observation from the tuning map. This will help build our intuition and may help one day when we are out in the plant.

The right most plot in the center row (Kc, 2*T*i) of the tuning map above is reproduced below.



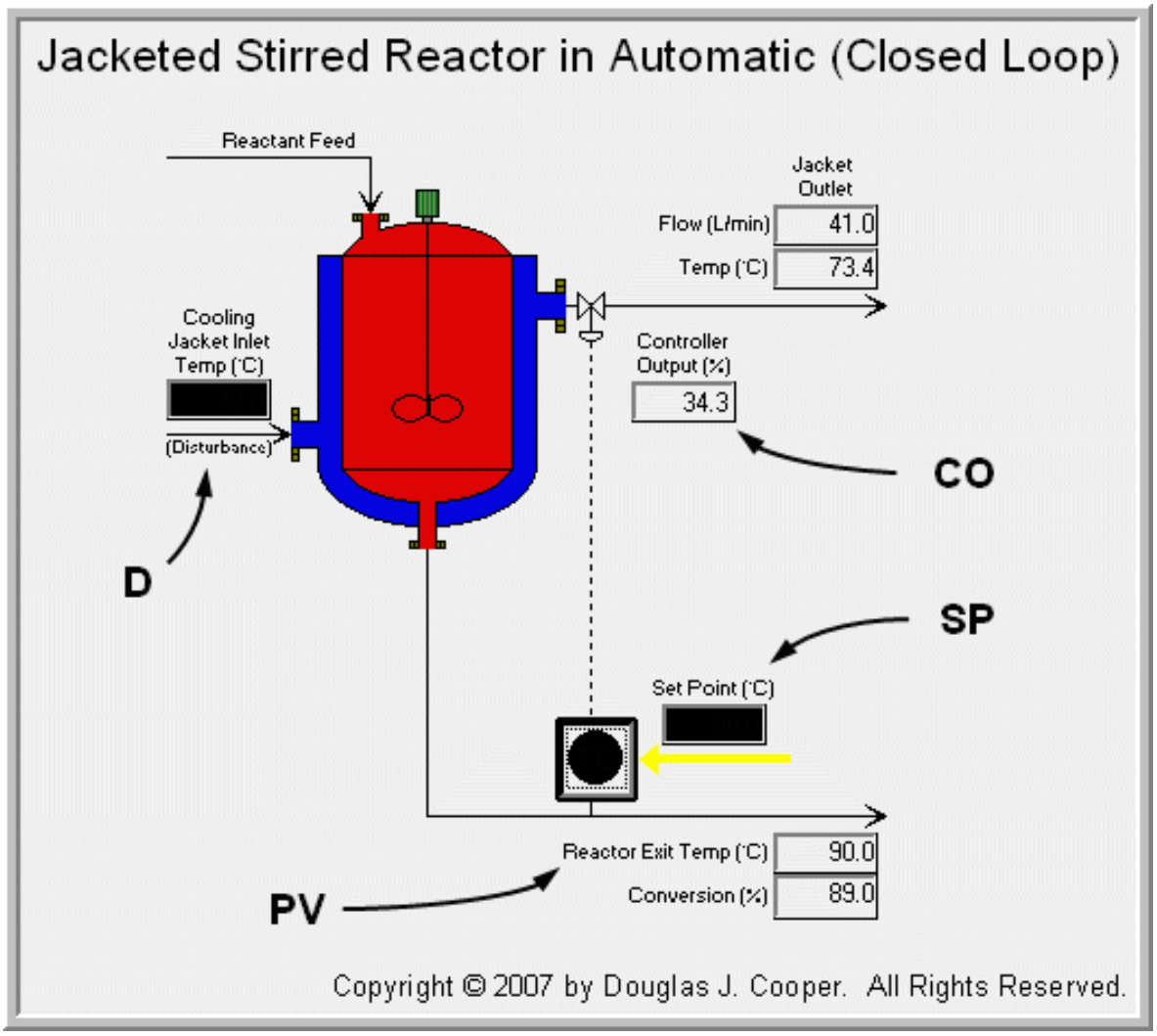
Notice how the PV shows a dip or brief oscillation on its way up to the set point? This is a classic indication that the proportional term is reasonable but the integral term is not getting enough weight in the calculation. For the PI form used in this article, that would mean that the reset time, *T*i, is too large since it is in the denominator.

If we cover the right half of the "not enough integral action" plot, the response looks like it is going to settle out with some offset, as would be expected with a P-Only controller. When we consider the plot as a whole, we see that as enough time passes, the response completes. This is because the weak integral action finally accumulates enough weight in the calculation to move the PV up to set point.

This “oscillates on the way” pattern is a useful marker for diagnosing a lack of sufficient integral action.

PI Disturbance Rejection in the Jacketed Stirred Reactor

The control objective for the jacketed reactor is to minimize the impact on reactor operation when the temperature of the liquid entering the cooling jacket changes (detailed discussion here). As a base case study, we establish here the performance capabilities of a PI controller in achieving this objective.



The important variables for this study are labeled in the graphic:
CO = signal to valve that adjusts cooling jacket liquid flow rate (controller output, %)
PV = reactor exit stream temperature (measured process variable, ^{o}C)
SP = desired reactor exit stream temperature (set point, ^{o}C)
D = temperature of cooling liquid entering the jacket (major disturbance, ^{o}C)

We follow our industry proven recipe to design and tune our PI controller:

Step 1: Design Level of Operation (DLO)
The details of expected process operation and how this leads to our DLO are presented in this article and are summarized:
- Design PV and SP = 90 ^{o}C with approval for brief dynamic (bump) testing of ±2 ^{o}C.
- Design D = 43 ^{o}C with occasional spikes up to 50 ^{o}C.

Step 2: Collect Process Data around the DLO
When CO, PV and D are steady near the design level of operation, we bump the process as detailed here to generate CO-to-PV cause and effect response data.

Step 3: Fit a FOPDT Model to the Dynamic Process Data
We approximate the dynamic behavior of the process by fitting a first order plus dead time

(FOPDT) dynamic model to the test data from step 2. The results of the modeling study are presented in detail here and are summarized:

- Process gain (direction and how far), Kp = – 0.5 ^{o}C/%
- Time constant (how fast), *T*p = 2.2 min
- Dead time (how much delay), Θp = 0.8 min

Step 4: Use the FOPDT Parameters to Complete the Design

As in the heat exchanger PI control study, we explore what is often called the dependent, ideal form of the PI control algorithm:

$$CO = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{Ti} \int e(t)dt$$

Where:

CO = controller output signal (the wire out)
CO_{bias} = controller bias or null value; set by bumpless transfer
e(t) = current controller error, defined as SP – PV
SP = set point
PV = measured process variable (the wire in)
Kc = controller gain, a tuning parameter
*T*i = reset time, a tuning parameter

Aside: our observations using the dependent ideal PI algorithm directly apply to the other popular PI controller forms. For example, the integral gain for the independent algorithm form, written as:

$$CO = CO_{bias} + Kc \cdot e(t) + Ki \int e(t)dt$$

can be computed as: Ki = Kc/*T*i. The Kc is the same for both forms, though it is more commonly called the proportional gain for the independent algorithm.

• *Sample Time,* T

Best practice is to set the loop sample time, T, at one-tenth the time constant or faster (i.e., T ≤ 0.1*T*p). Faster sampling may provide modestly improved performance, while slower sampling can lead to significantly degraded performance.

In this study, T ≤ 0.1(2.2 min), so T should be 13 seconds or less. We meet this with the sample time option available from most commercial vendors:

◊ sample time, T = 1 sec

• *Control Action (Direct/Reverse)*

The jacketed stirred reactor process has a negative Kp. That is, when CO increases, PV decreases in response. Since a controller must provide negative feedback, if the process is reverse acting, the controller must be direct acting. That is, when in automatic mode (closed loop), if the PV is too high, the controller must increase the CO to correct the error. Since the controller must move in the same direction as the problem, we specify:

◊ controller is direct acting

• *Dead Time Issues*

If dead time is greater than the process time constant (Θp > Tp), control becomes increasingly problematic and a Smith predictor can offer benefit. For this process, the dead time is smaller than the time constant, so:
◊ dead time is small and not a concern

• *Computing Controller Error, e(t)*
Set point, SP, is manually entered into a controller. The measured PV comes from the sensor (our wire in). Since SP and PV are known values, then at every loop sample time, T, controller error can be directly computed as:
◊ error, e(t) = SP - PV

• *Determining Bias Value,* CO_{bias}
CO_{bias} is the value of CO that, in manual mode, causes the PV to steady at the DLO when the major disturbances are quiet and at their normal or expected values. When integral action is enabled, commercial controllers determine the bias value with a bumpless transfer procedure.

That is, when switching to automatic, the controller initializes the SP to the current value of PV, and CO_{bias} to the current value of CO. By choosing our current operation as our design state (at least temporarily at switchover), there is no corrective action needed by the controller that will bump the process. Thus,
◊ controller bias, CO_{bias} = current CO for a bumpless transfer

• *Controller Gain,* Kc*, and Reset Time, T*i
We use our FOPDT model parameters in the industry-proven Internal Model Control (IMC) tuning correlations to compute PI tuning values.

The first step in using the IMC correlations is to compute *T*c, the closed loop time constant. *T*c describes how active our controller should be in responding to a set point change or in rejecting a disturbance.

The performance implications of choosing *T*c have been explored previously for PI control of the heat exchanger and the gravity drained tanks case studies.

In short, the closed loop time constant, *T*c, is computed based on whether we seek:
▪ aggressive action and can tolerate some overshoot and oscillation in the PV response,
▪ moderate action where the PV will move reasonably fast but show little overshoot,
▪ conservative action where the PV will move in the proper direction, but quite slowly.

Once this decision is made, we compute *T*c with these rules:
▪ *Aggressive Response*: *T*c is the larger of $0.1 \cdot T$p or $0.8 \cdot \Theta$p
▪ *Moderate Response*: *T*c is the larger of $1 \cdot T$p or $8 \cdot \Theta$p
▪ *Conservative Response*: *T*c is the larger of $10 \cdot T$p or $80 \cdot \Theta$p

With *T*c computed, the PI controller gain, Kc, and reset time, *T*i, are computed as:

$$Kc = \frac{1}{Kp} \frac{Tp}{(\theta p + Tc)} \quad \text{and} \quad Ti = Tp$$

Notice that reset time, *T*i, is always equal to the process time constant, *T*p, regardless of

desired controller activity.

a) *Moderate Response Tuning:*
For a controller that will move the PV reasonably fast while producing little to no overshoot, choose:

Moderate T_c = the larger of $1 \cdot T_p$ or $8 \cdot \Theta_p$
= larger of 1(2.2 min) or 8(0.8 min)
= 6.4 min

Using this T_c and our model parameters in the tuning correlations above, we arrive at the moderate tuning values:

$$K_c = \frac{1}{-0.5}\frac{2.2}{(0.8 + 6.4)} = -0.6\,\frac{\%}{^{\circ}C} \quad \text{and} \quad T_i = 2.2 \text{ min}$$

b) *Aggressive Response Tuning:*
For an active or quickly responding controller where we can tolerate some overshoot and oscillation as the PV settles out, specify:

Aggressive T_c = the larger of $0.1 \cdot T_p$ or $0.8 \cdot \Theta_p$
= larger of 0.1(2.2 min) or 0.8(0.8 min)
= 0.64 min

and the aggressive tuning values are:

$$K_c = \frac{1}{-0.5}\frac{2.2}{(0.8 + 0.64)} = -3.1\,\frac{\%}{^{\circ}C} \quad \text{and} \quad T_i = 2.2 \text{ min}$$

Practitioner's Note: The FOPDT model parameters used in the tuning correlations above have engineering units, so the Kc values we compute also have engineering units. In commercial control systems, controller gain (or proportional band) is normally entered as a dimensionless (%/%) value.

For commercial implementations, we could:
▪ Scale the process data before fitting our FOPDT dynamic model so we directly compute a dimensionless Kc.
▪ Convert the model Kp to dimensionless %/% after fitting the model but before using the FOPDT parameters in the tuning correlations.
▪ Convert Kc from engineering units into dimensionless %/% after using the tuning correlations.

CO is already scaled from 0 – 100% in the above example. Thus, we convert Kc from engineering units into dimensionless %/% using the formula:

$$Kc(\%/\%) = Kc(\%/\text{units})\left(\frac{PV_{max}\text{ units} - PV_{min}\text{ units}}{100\% - 0\%}\right)$$

For the jacketed stirred reactor, PVmax = 250 oC and PVmin = 0 oC. The dimensionless

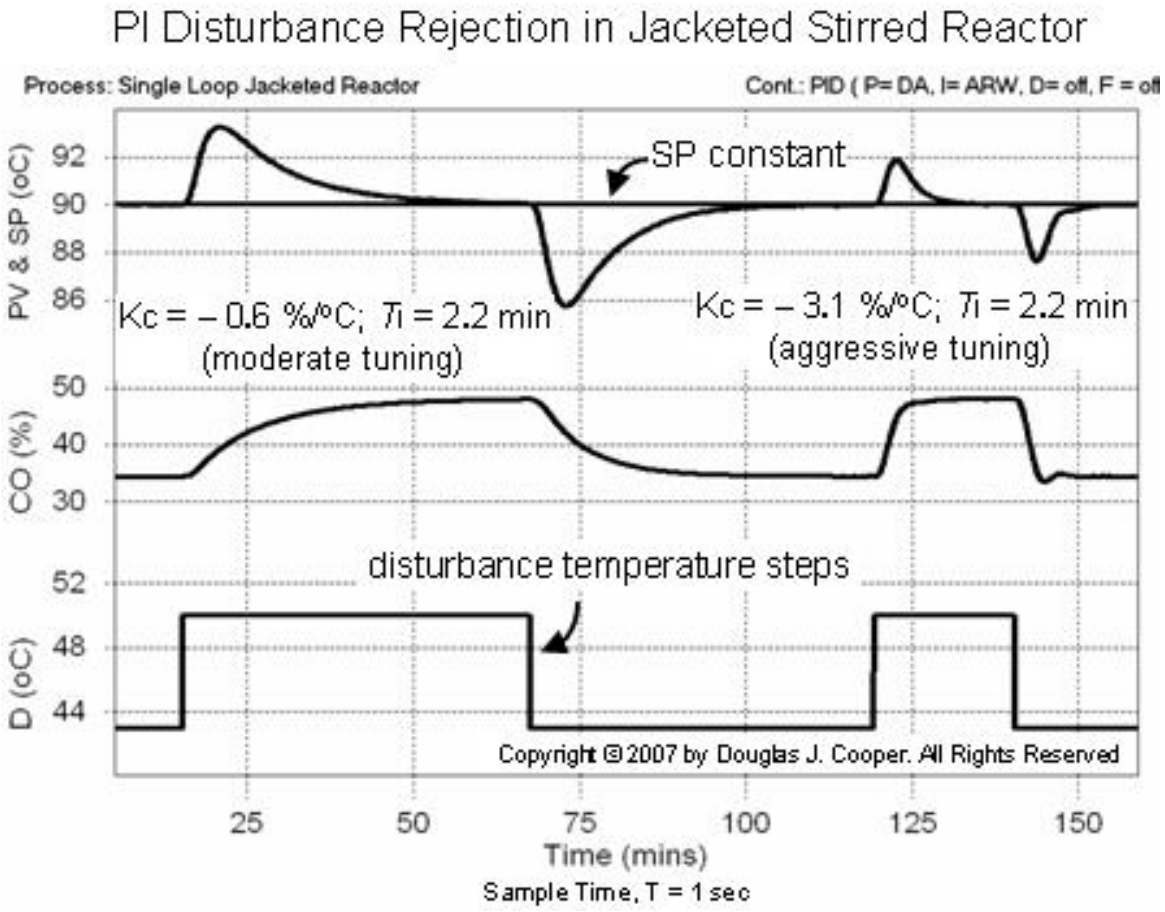
Kc values are thus computed:

- moderate Kc = (– 0.6 %/ oC)·[(250 – 0 oC) ÷ (100 – 0%)] = – 1.5 %/%
- aggressive Kc = (– 3.1%/ oC)·[(250 – 0 oC) ÷ (100 – 0%)] = – 7.8 %/%

We use Kc with engineering units in the remainder of this article and are careful that our PI controller is formulated to accept such values. We would be mindful if we were using a commercial control system, however, to ensure our tuning parameters are cast in the form appropriate for our equipment.

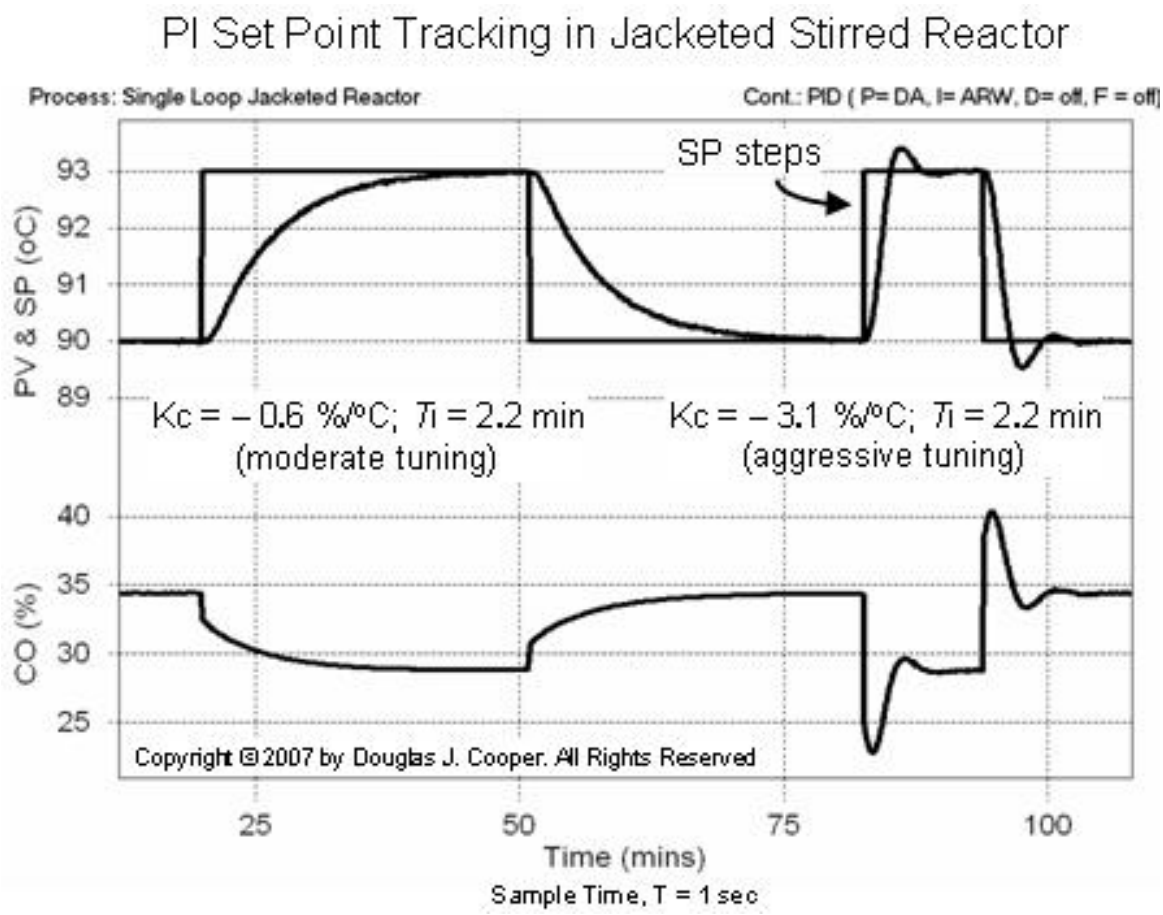
Implement and Test

The ability of the PI controller to reject changes in the cooling jacket inlet temperature, D, is pictured below () for the moderate and aggressive tuning values computed above. Note that the set point remains constant at 90 ^{o}C throughout the study.



As expected, the aggressive controller shows a more energetic CO action, and thus, a more active PV response.

While not our design objective, presented below is the set point tracking ability of the PI controller () when the disturbance temperature is held constant.



The plot shows that set point tracking performance matches the descriptions used above for choosing *T*c:

- Use aggressive action if we seek a fast response and can tolerate some overshoot and oscillation in the PV response.
- Use moderate action if we seek a reasonably fast response but seek little to no overshoot in the PV response.

Important => *T*i Always Equals *T*p
As stated above, the rules provide a constant reset time, *T*i, regardless of our desired performance. So if we believe we have collected a good process data set, and the FOPDT model fit looks like a reasonable approximation of this data, then we have a good estimate of the process time constant and *T*i = *T*p regardless of desired performance.

If we are going to tweak the tuning, Kc should be the only value we adjust. For example, if we seek a performance between moderate and aggressive, we average the Kc values while *T*i remains constant.

Integral (Reset) Windup, Jacketing Logic and the Velocity PI Form

A valve cannot open more than all the way. A pump cannot go slower than stopped. Yet an improperly programmed control algorithm can issue such commands.

Herein lies the problem of integral windup (also referred to as reset windup or integral saturation). It is a problem that has been around for decades and was solved long ago. We discuss why it occurs and how to prevent it to help those who choose to write their own control algorithm.

The PI Algorithm

To increase our comfort level with the idea that different vendors cast the same PI algorithm in different forms, we choose the independent, continuous, position PI form for this discussion:

$$CO = CO_{bias} + Kc \cdot e(t) + Ki \int e(t)dt$$

Where:

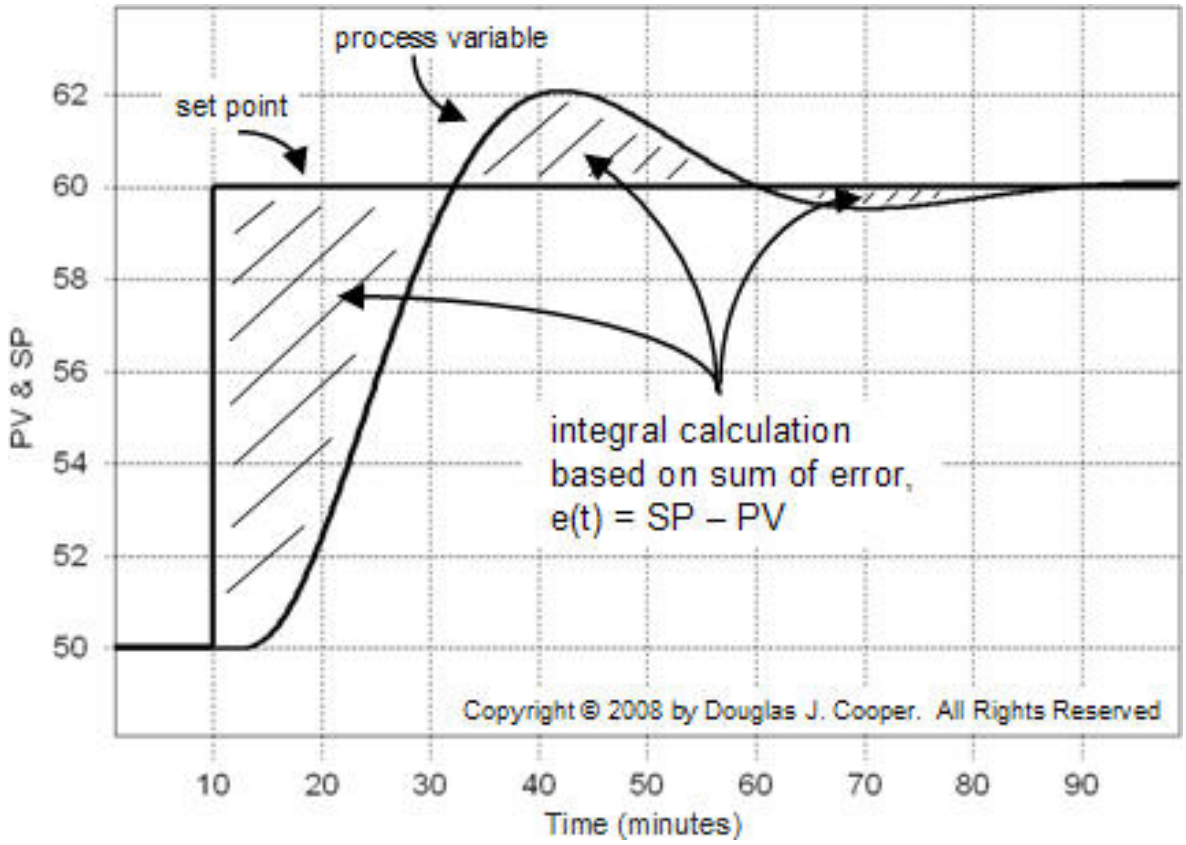
CO = controller output signal (the wire out)
CO_{bias} = controller bias or null value
e(t) = current controller error, defined as SP – PV
SP = set point
PV = measured process variable (the wire in)
Kc = proportional gain, a tuning parameter
Ki = integral gain, a tuning parameter

Note that Kc is the same parameter in both the dependent and independent forms, though it is more typically called controller gain in the dependent form.

Every procedure and observation we have previously discussed about PI controllers applies to both forms. Both even use the same tuning correlations. To tune Ki, we compute Kc and *T*i for the dependent form and then divide (Ki = Kc/*T*i).

Integral (Reset) Windup

Our previous discussion of integral action noted that integration is a continual summing. As shown below (), integration of error means that we continually sum controller error up to the present time.



The integral sum starts accumulating when the controller is first put in automatic and continues to change as long as controller error exists.

If an error is large enough and/or persists long enough, it is mathematically possible for the integral term to grow very large (either positive or negative):

$$CO = CO_{bias} + Kc \cdot e(t) + Ki \underbrace{\int e(t)dt}_{\text{can grow very large}}$$

This large integral, when combined with the other terms in the equation, can produce a CO value that causes the final control element (FCE) to saturate. That is, the CO drives the FCE (e.g. valve, pump, compressor) to its physical limit of fully open/on/maximum or fully closed/off/minimum.

And if this extreme value is still not sufficient to eliminate the error, the simple mathematics of the controller algorithm, if not jacketed with protective logic, permits the integral term to continue growing.

If the integral term grows unchecked, the equation above can command the valve, pump or compressor to move to 110%, then 120% and more. Clearly, however, when an an FCE reaches its full 100% value, these last commands have no physical meaning and consequently, no impact on the process.

Control is Lost

Once we cross over to a “no physical meaning” computation, the controller has lost the

ability to regulate the process.

When the computed CO exceeds the physical capabilities of the FCE because the integral term has reached a large positive or negative value, the controller is suffering from *windup*. Because windup is associated with the integral term, it is often referred to as *integral windup* or *reset windup*.

To prevent windup from occurring, modern controllers are protected by either:

- Employing extra "jacketing logic" in the software to halt integration when the CO reaches a maximum or minimum value.
- Recasting the controller into a discrete velocity form that, by its very formulation, naturally avoids windup.

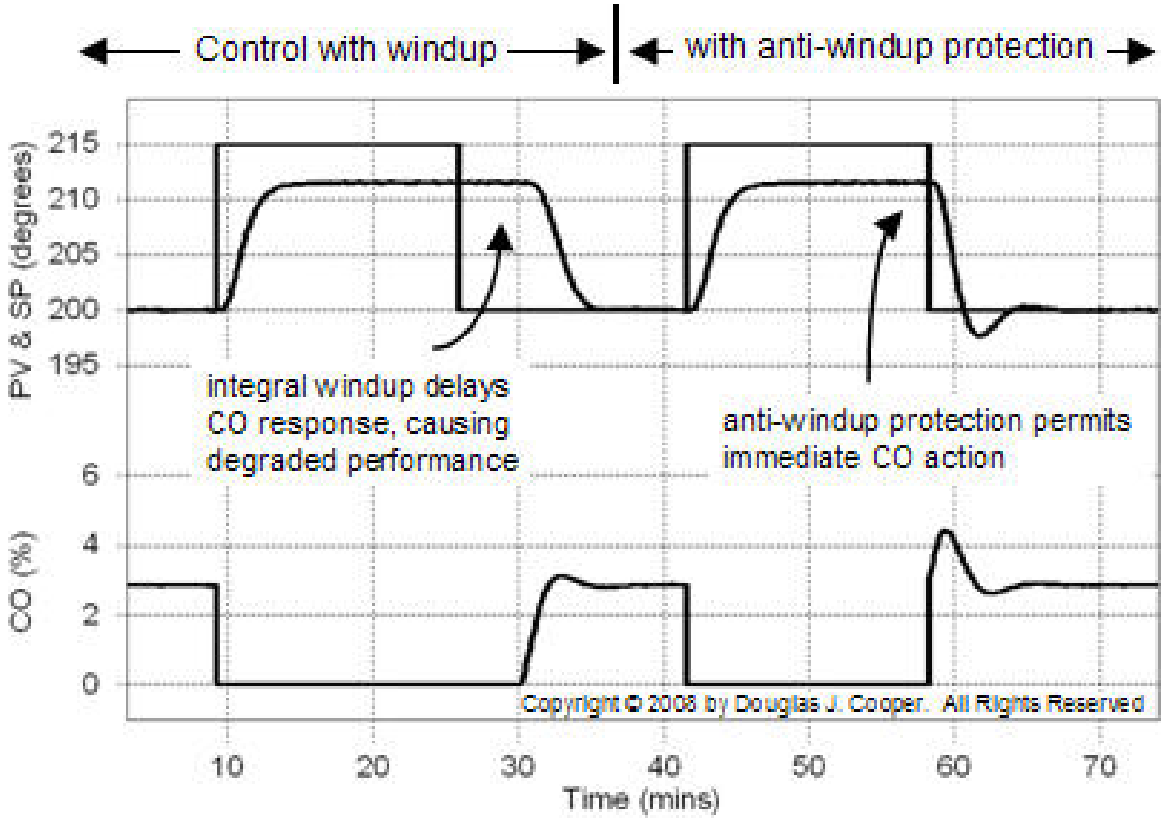
Both alternatives offer benefits but possess some fairly subtle drawbacks that we discuss below.

Visualizing Windup

To better visualize the problem of windup and the benefit of anti-windup protection, consider the plot from our heat exchanger process below ().

To the left is the performance of a PI controller with no windup protection. To the right is the performance of the same controller protected by an anti-windup strategy.

For both controllers, the set point is stepped from 200 °C up to 215 °C and back again. As shown in the lower trace on the plot, the controller moves the CO to 0%, closing the valve completely, yet this is not sufficient to move the PV up to the new set point.



To the left in the plot, the impact of windup is a degraded controller performance. When the set point is stepped back to its original value of 200 °C , the windup condition causes a delay in the CO action. This in turn causes a delay in the PV response.

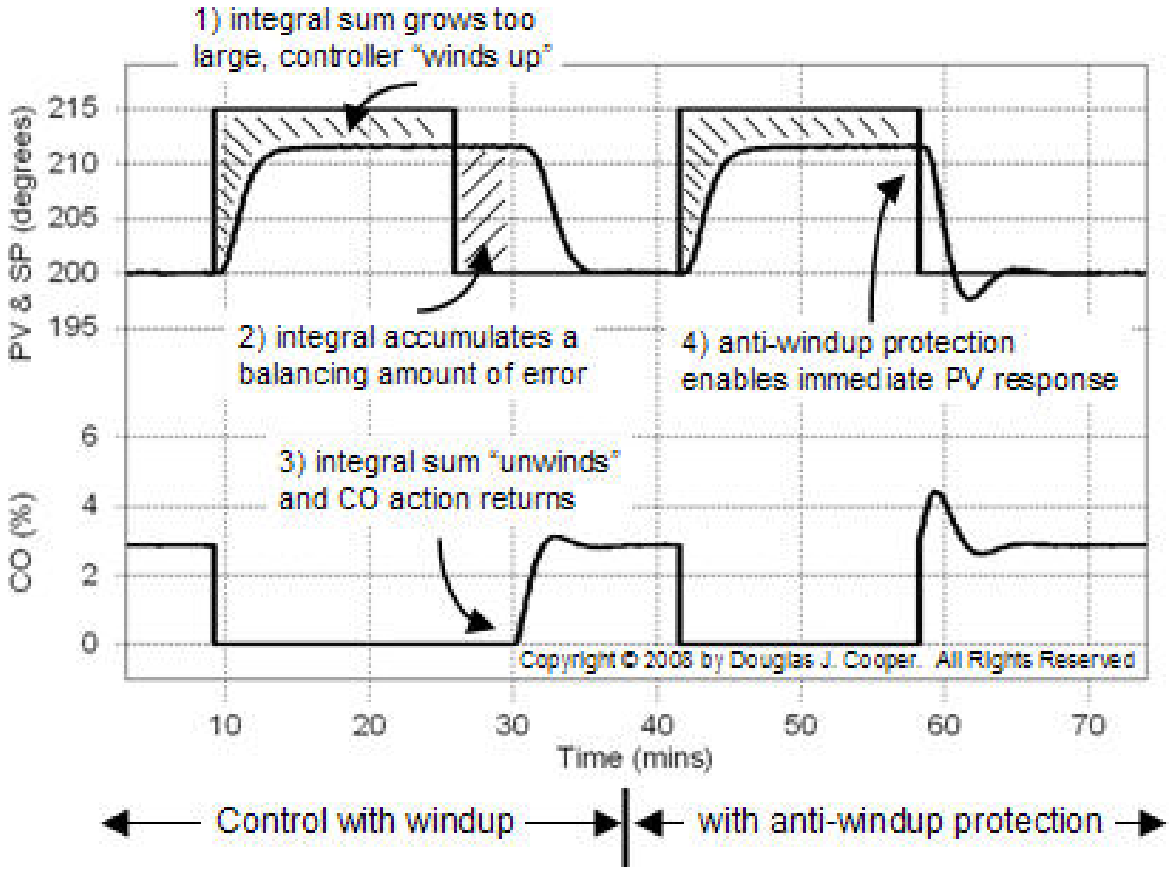
To the right in the plot, anti-windup protection permits the CO, and thus PV, to respond promptly to the command to return to the original SP value of 200 °C.

More Details on Windup
The plot below offers more detail. As labeled on the plot:

1) To the left for the Controller with Wind-up case, the SP is stepped up to 215 °C. The valve closes completely but is not able to move the PV all the way to the high set point value. Integration is a summing of controller error, and since error persists, the integration term grows very large.

The sustained error permits the controller to windup (saturate). While it is not obvious from the plot, the PI algorithm is computing values for CO that ask the valve to be open –5%, –8% and more. The control algorithm is just simple math with no ability to recognize that a valve cannot be open to a negative value.

Note that the chart shows the CO signal bottoming out at 0% while the controller algorithm is computing negative CO values. This misleading information is one reason why windup can be difficult to diagnose as the root cause of a problem from visual inspection of process data trend plots.



2) When the SP is stepped back to 200 °C, it seems as if the CO does not move at first.

In reality, the control algorithm started moving the CO when the SP changed, but the values remain in the physically meaningless range of negative numbers.

So while the valve remains fully closed at 0%, the integral sum is accumulating controller errors of opposite sign. As time passes, the integral term shrinks or "unwinds" as the running sum of errors balance out.

3) When the integral sum of errors shrinks enough, it no longer dominates the CO computation. The CO signal returns from the physically meaningless world of negative values. The valve can finally move in response.

4) To the right in the plot above, the controller is protected from windup. As a result, when the set point is stepped back to 200 °C, the CO immediately reacts with a change that is proportional to the size of the SP change. The PV moves quickly in response to the CO actions as it tracks the SP back to 200 °C.

◊ Solution 1: Jacketing Logic on the Position Algorithm

The PI controller at the top of this article is called the position form because the computed CO is a specific intermediate value between full on/open/maximum and closed/off/minimum. The continuous PI algorithm is specifying the actual position (e.g., 27% open, 64% of maximum) that the final control element (FCE) should assume.

● *Simple Logic Creates Additional Problems*
It is not enough to have logic that simply limits or clips the CO if it reaches a maximum (COmax) or minimum (COmin) value because this does nothing to check the growth of the integral sum of errors term.

In fact, such simple logic was used in the "control with windup" plots just discussed. The CO seems stuck at 0% and we are unaware that the algorithm is actually computing negative valve positions as described in item 1 above.

● *Anti-Windup Logic Outline*
When we switch from manual mode to automatic, we assume that we have initialized the controller using a bumpless transfer procedure. That is, at switchover, the integral sum of error is set to zero, the SP is set equal to the current PV, and the controller bias is set equal to the current CO (implying that COmin < CO_{bias} < COmax).

Thus, there is nothing to cause CO to immediately change and "bump" our process at switchover.

One approach to creating anti-windup jacketing logic is to artificially manipulate the integral sum of error itself. With our controller properly initialized, the approach is to flip the algorithm around and ***back-calculate a value for the integral sum of error*** that will provide a desired controller output value (COdesired), or:

$$\int e(t)dt = \frac{1}{Ki}\left\{CO_{desired} - CO_{bias} - Kc \cdot e(t)\right\}$$

Note that COdesired can be different in different situations. For example,

- We do not want tuning parameter adjustments to cause sudden CO movements that bump our process. So if tuning values have changed, COdesired is the value of CO from the previous loop calculation cycle.
- If the PI controller computes CO values that are above COmax or below COmin, then we must be concerned about windup and COdesired is set equal to the limiting COmax or COmin value.

The anti-windup logic followed at every loop sample time, T, is thus:

1) If tuning parameters have changed since the last loop calculation cycle, then COdesired = current CO. Back calculate the integral sum of error so CO remains unchanged from the previous sample time. This prevents sudden CO bumps due to tuning changes.

2) Update SP and PV for this loop calculation cycle.

3) compute: $$CO = CO_{bias} + Kc \cdot e(t) + Ki \int e(t)dt$$

4) If CO > COmax or if CO < COmin, then the anti-windup (integral desaturation) logic of step 5 is required. Otherwise, proceed to step 6.

5) If CO > COmax, then CO = COdesired = COmax. if CO < COmin, then CO = COdesired = COmin. Back calculate the integral sum of error using our selected COdesired and save it for use in the next control loop calculation cycle.

6) Implement CO

◊ **Solution 2 - Use the Velocity (Discrete) Controller Form**

Rather than computing a CO signal indicating a specific position for our final control element, an alternative is to compute a signal that specifies a change, ΔCO, from current position for the FCE. As explained below, this is called the velocity or discrete controller form.

We employ the dependent algorithm for this presentation, but the derivation that follows can be applied in an analogous fashion to the independent PI form. To derive the discrete velocity form, we must first write the continuous, position form of the PI controller to include the independent variable on the controller output, showing it properly as CO(t) to reflect that it changes with time:

$$CO(t) = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{Ti} \int e(t)dt$$

Please note that this controller is identical to all dependent PI forms as presented in other articles in this e-book. The only difference is we are being more mathematically precise in our expression of CO(t).

● *Deriving the Discrete Velocity Form*

The first step in deriving the discrete form is to take the time derivative of the continuous form. In physics, the time derivative (rate of change) of a position is a velocity. This is why the final form of the PI controller we derive is often called the velocity form.

Taking the derivative of the continuous PI controller above with respect to time yields:

$$\frac{dCO(t)}{dt} = \frac{dCO_{bias}}{dt} + Kc\frac{de(t)}{dt} + \frac{Kc}{Ti}e(t)$$

Since CO_{bias} is a constant and the derivative of a constant is zero, then:

$$\frac{dCO_{bias}}{dt} = 0$$

Removing this term from the equation results in:

$$\frac{dCO(t)}{dt} = Kc\frac{de(t)}{dt} + \frac{Kc}{Ti}e(t)$$

If we assume discrete or finite difference approximations for the continuous derivatives, then the controller becomes:

$$\frac{\Delta CO}{\Delta t} = Kc\frac{\Delta e_i}{\Delta t} + \frac{Kc}{Ti}e_i$$

where e_i is the current controller error, e_{i-1} is the controller error at the previous sample time, T, and $\Delta e_i = e_i - e_{i-1}$.

Recognizing that loop sample time is $T = \Delta t$, then the PI controller becomes:

$$\frac{\Delta CO}{T} = Kc\left(\frac{e_i - e_{i-1}}{T}\right) + \frac{Kc}{Ti}e_i$$

Rearranging, we arrive at the ***discrete velocity form*** of the PI controller:

$$\Delta CO = Kc\left(1 + \frac{T}{Ti}\right)e_i - Kc \cdot e_{i-1}$$

● *Reason for Anti-Windup Protection*

Discrete velocity algorithms compute a ΔCO that signals the FCE to move a specific distance and direction from its current position. As we can see from the PI controller form above, the computation does not keep track of the current FCE position, nor does it mathematically accumulate any integral sums.

In a sense, the accumulation of integration is stored in the final control element itself. If a long series of ΔCO moves are all positive, for example, the valve, pump or compressor will move toward its maximum value. And once the FCE reaches its maximum limit, any ΔCO

commands to move further will have no impact because, as stated in the first sentences of this article, a valve cannot open more than all the way and a pump cannot go slower than stopped. It is the physical nature of the FCE itself that provides protection from over-accumulation (i.e., windup).

As long as the CO never reaches COmax or COmin, the continuous position and discrete velocity forms of the PI controller provide identical performance. A properly jacketed continuous position PI controller will also provide windup protection equal to the discrete velocity form. Implicit in these statements is that sample time,T, is reasonably fast and that T and the tuning values (Kc and *T*i) are the same when comparing implementations.

● *Concerns with Discrete Velocity PID*
Unfortunately, the usefulness of the discrete velocity form is limited because the method suffers problems when derivative action is included. We find that we must take the derivative of a derivative, yielding a second derivative. A second derivative applied to data that contains even modest noise can produce nonsense results.

Some vendors implement this form anyway and include a signal filter and additional logic sequences to address the problem. Thus, even with the anti-windup benefits of a discrete velocity algorithm, we find the need to jacket the algorithm with protective logic.

9) Derivative Action and PID Control

PID Control and Derivative on Measurement

Like the PI controller, the Proportional-Integral-Derivative (PID) controller computes a controller output (CO) signal for the final control element every sample time T.

The PID controller is a “three mode” controller. That is, its activity and performance is based on the values chosen for three tuning parameters, one each nominally associated with the proportional, integral and derivative terms.

As we had discussed previously, the PI controller is a reasonably straightforward equation with two adjustable tuning parameters. The number of different ways that commercial vendors can implement the PI form is fairly limited, and they all provide the same performance if properly tuned.

With the addition of a third adjustable tuning parameter, the number of algorithm permutations increases markedly. And there are even different forms of the PID equation itself. This creates added challenges for controller design and tuning.

Here we focus on what a derivative is, how it is computed, and what it means for control. We also explore why derivative on measurement is widely recommended for industrial practice.

We narrow our world in this article and focus on the dependent, ideal form of the controller. In later articles we will circle back and talk about the different algorithm forms, methods for design and tuning, algorithm limitations, and other practical issues.

The Dependent, Ideal PID Form
A popular way vendors express the dependent, ideal PID controller is:

$$CO = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{Ti}\int e(t)dt + Kc \cdot Td\frac{de(t)}{dt}$$

Where:
- CO = controller output signal (the wire out)
- CO_{bias} = controller bias; set by bumpless transfer
- e(t) = current controller error, defined as SP – PV
- SP = set point
- PV = measured process variable (the wire in)
- Kc = controller gain, a tuning parameter
- *T*i = reset time, a tuning parameter
- *T*d = derivative time, a tuning parameter

The first three terms to the right of the equal sign are identical to the PI controller we have already explored in some detail.

The derivative mode of the PID controller is an additional and separate term added to the

end of the equation that considers the derivative (or rate of change) of the error as it varies over time.

The Contribution of the Derivative Term

The proportional term considers *how far* PV is from SP at any instant in time. Its contribution to the CO is based on the size of e(t) only at time t. As e(t) grows or shrinks, the influence of the proportional term grows or shrinks immediately and proportionately.

The integral term addresses *how long* and how far PV has been away from SP. The integral term is continually summing e(t). Thus, even a small error, if it persists, will have a sum total that grows over time and the influence of the integral term will similarly grow.

A derivative describes how steep a curve is. More properly, a derivative describes the slope or the rate of change of a signal trace at a particular point in time. Accordingly, the derivative term in the PID equation above considers *how fast*, or the rate at which, error (or PV as we discuss next) is changing at the current moment.

Derivative on PV is Opposite but Equal

While the proportional and integral terms of the PID equation are driven by the controller error, e(t), the derivative computation in many commercial implementations should be based on the value of PV itself.

The derivative of e(t) is mathematically identical to the negative of the derivative of PV everywhere except when set point changes. And when set point changes, derivative on error results in an undesirable control action called *derivative kick*.

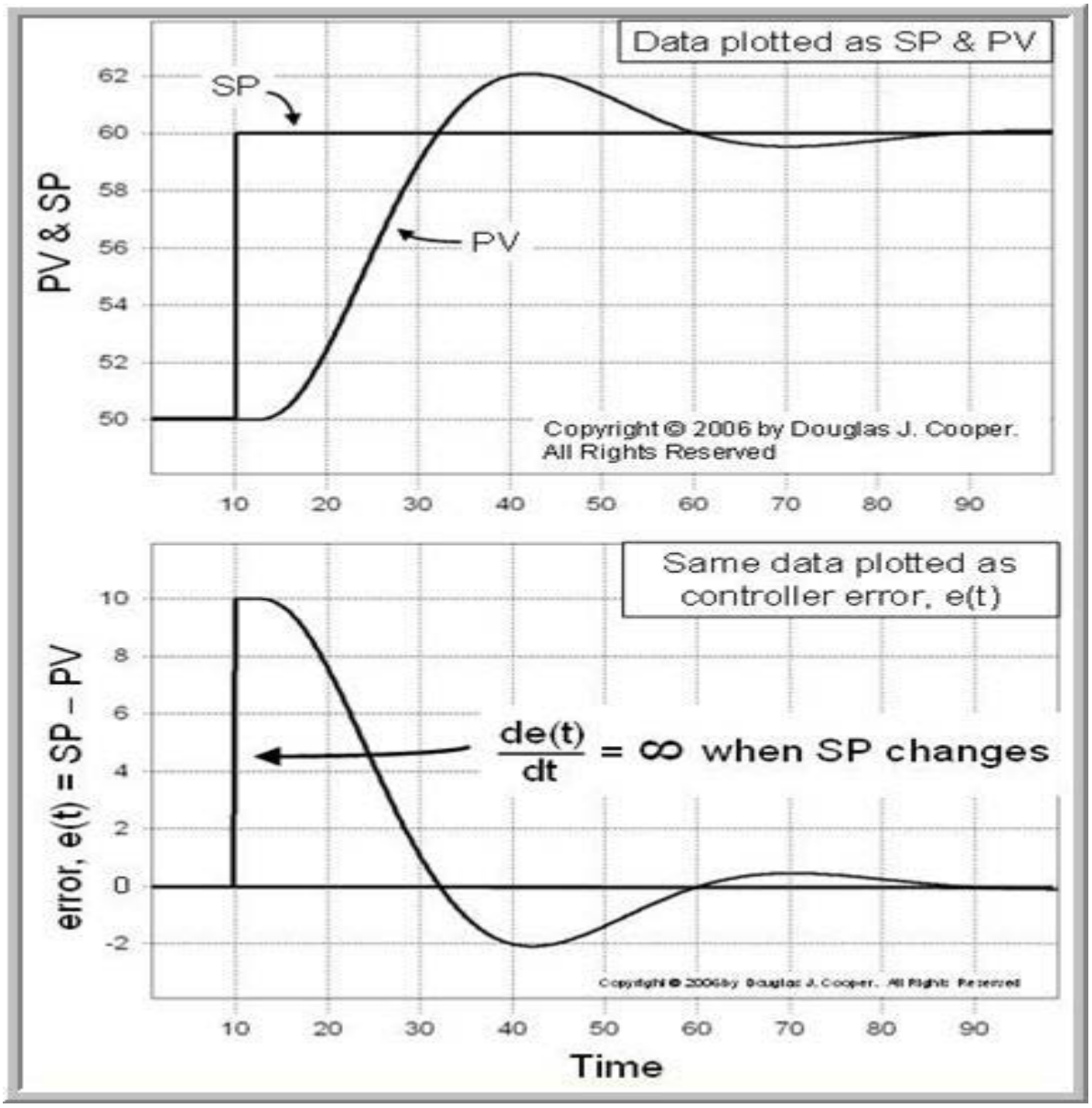
Math Note: the mathematical defense that “derivative of e(t) equals the negative derivative of PV when SP is constant" considers that, since e(t) = SP – PV, the equation below follows. That is, derivative of error equals derivative of set point minus process variable.

The derivative of a constant is zero, so when SP is constant, mathematically, the derivative (or slope or rate of change) of the controller error equals the derivative (or slope or rate of change) of the measured process variable, PV, except the sign is opposite.

$$\frac{de(t)}{dt} = \frac{d(\overset{0}{\cancel{SP}} - PV)}{dt} = -\frac{dPV}{dt}$$

The figures below () provide a visual appreciation that the derivative of e(t) is the negative of the derivative of PV.

The top plot shows the measured PV trace after a set point step. The bottom plot shows the e(t) = SP – PV trace for the same event.



If we compare the two plots after the SP step at time t = 10, we see that the PV trace in the upper plot is an exact reflection of the e(t) trace in the lower plot. The PV trace ascends, peaks and then settles, while in a reflected pattern, the e(t) trace descends, dips and then settles.Mathematically, this “mirror image” of trace shapes means that the derivatives (or slopes or rates of change) are the same everywhere after the SP step, except they are opposite in sign.

Derivative on PV Used in Practice

While the shape of e(t) and PV are opposite but equal everywhere after the set point step, there is an important difference at the moment the SP changes. The lower plot shows a vertical spike in e(t) at this moment. There is no corresponding spike in the PV plot.

The derivative (or slope) of a vertical spike in the theoretical world approaches infinity. In the real world it is at least a very big number. If *T*d is large enough to provide any meaningful weight to the derivative term, this huge derivative value will cause a large and sudden manipulation in CO. This large manipulation in CO, referred to as *derivative kick*,

is almost always undesirable.

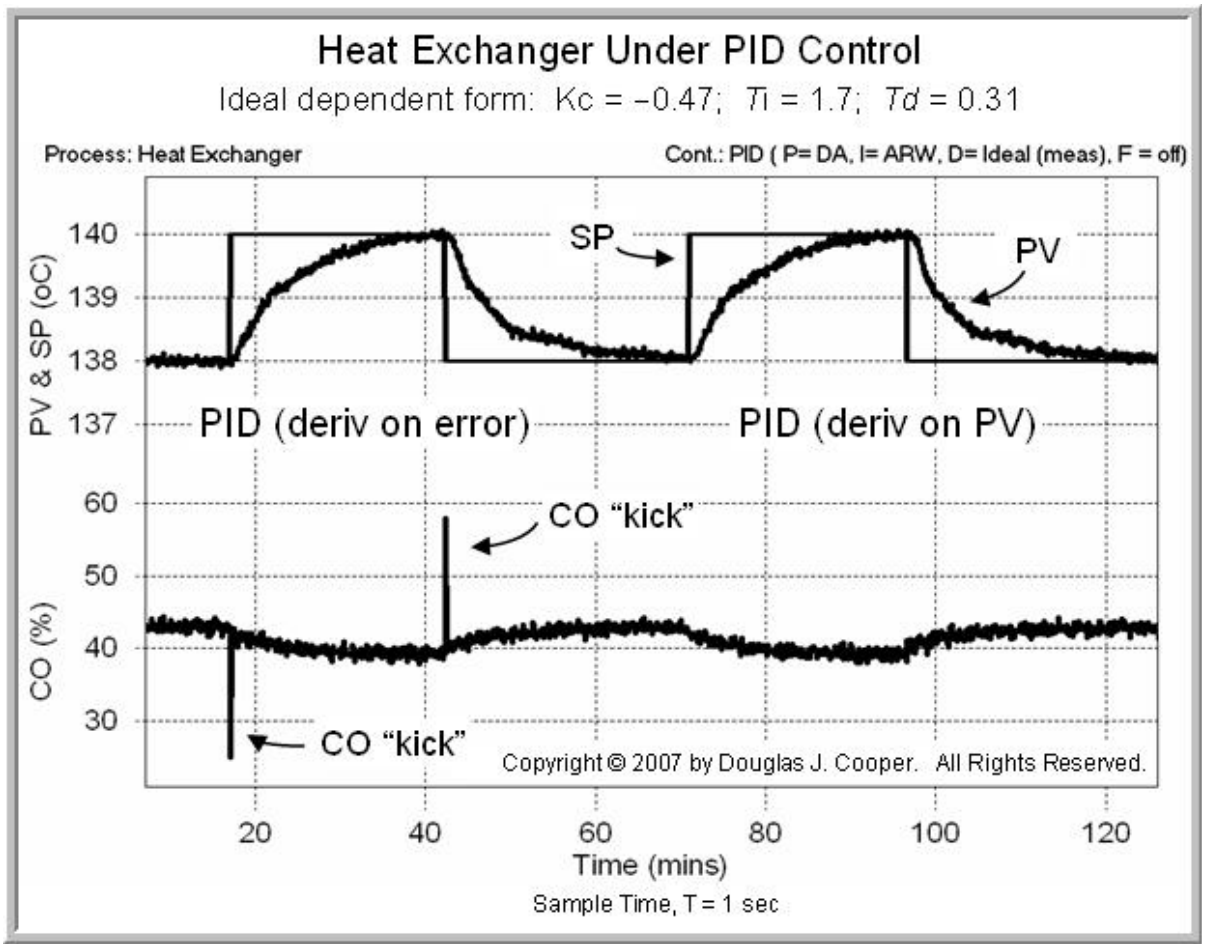
As long as loop sample time, T, is properly specified, the PV trace will follow a gradual and continuous response, avoiding the dramatic vertical spike evident in the e(t) trace.

Because derivative on e(t) is identical to derivative on PV at all times except when the SP changes, and when the set point does change, derivative on error provides information we don't want our controller to use, we substitute the "math note" equation in the yellow box above to obtain the PID with derivative on measurement controller:

$$CO = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{Ti}\int e(t)dt - Kc \cdot Td\frac{dPV}{dt}$$

Derivative on PV Does Not "Kick"
Below we show the heat exchanger case study under PID control using the dependent, ideal algorithm form and moderate tuning values as computed in this article.

The first set point steps to the left in the plot below () show loop performance when PID with derivative on error is used. The set points steps to the right present the identical scenario except that PID with derivative on measurement is used.



The "kick" that dominates the CO trace when derivative on error is used is rather dramatic and somewhat unsettling. While it exists for only a brief moment and does not impact performance in this example, we should not assume this will always be true. In any event,

such action will eventually take a toll on mechanical final control elements.

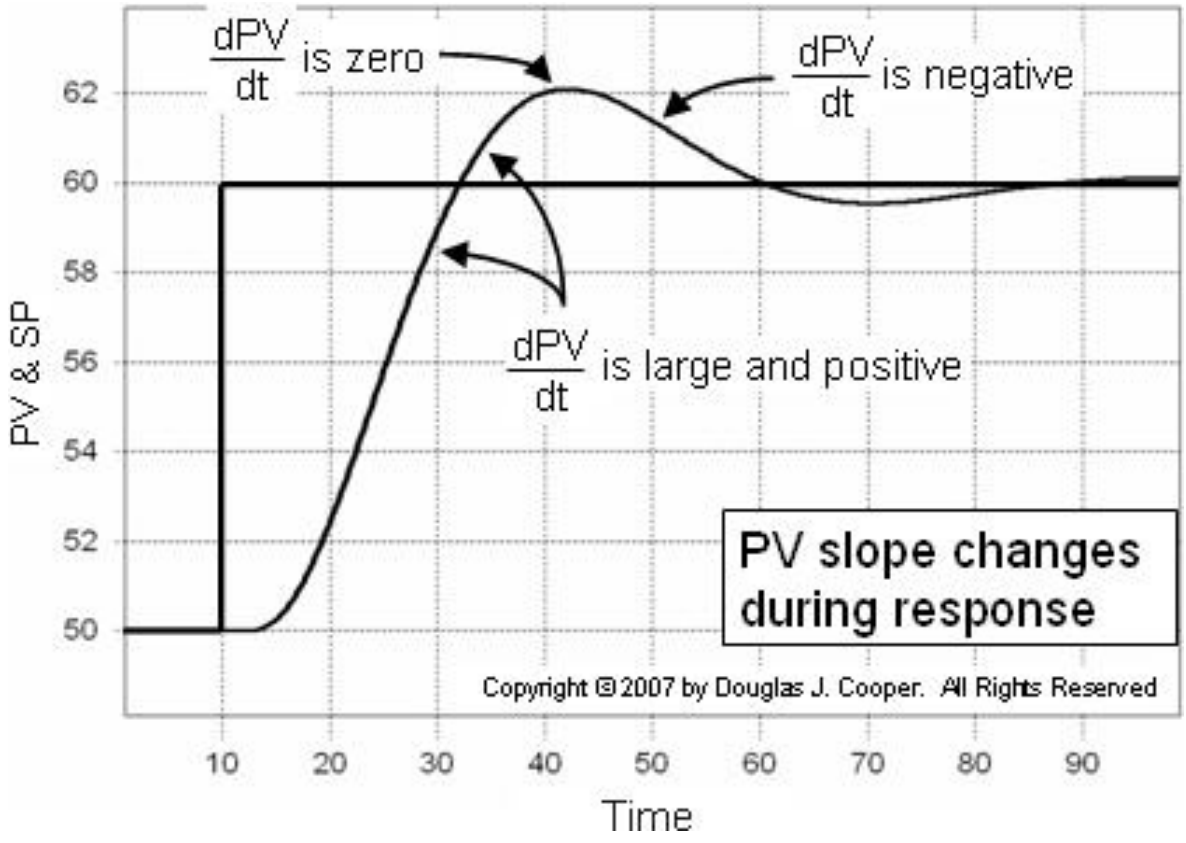
Based on the preceding discussion, we recommend that derivative on measured PV be used if our vendor provides the option (fortunately most do). The tuning values remain the same for both algorithms.

Understanding Derivative Action

A rapidly changing PV has a steep slope and this yields a large derivative. This is true regardless of whether a dynamic event has just begun or if it has been underway for some time.

In the plot below (), the derivative dPV/dt describes the slope or “steepness” of PV during a process response.

Early in the response, the slope is large and positive when the PV trace is increasing rapidly. When PV is decreasing, the derivative (slope) is negative. And when the PV goes through a peak or a trough, there is a moment in time when the derivative is zero.



To understand the impact of this changing derivative, let's assume for discussion that:

- Controller gain, Kc, is positive.
- Derivative time, *T*d (always positive) is large enough to provide meaningful weight to the derivative term. After all, if *T*d is very small, the derivative term has little influence, regardless of the slope of the PV.

The negative sign in front of the derivative term of the PID with derivative on measurement controller (and given the above assumptions) means that the impact on CO from the

derivative term will be opposite to the sign of the slope:

$$CO = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{\tau i}\int e(t)dt - Kc \cdot \tau d \frac{dPV}{dt}$$

Thus, when dPV/dt is large and positive, the derivative term has a large influence and seeks to decrease CO.

Conversely, when dPV/dt is negative, the derivative term seeks to increase CO.

It is interesting to note that the derivative term does not consider whether PV is heading toward or away from the set point (whether e(t) is positive or negative). The only consideration is whether PV is heading up or down and how quickly.

The result is that derivative action seeks to inhibit rapid movements in the PV. This could be an especially useful characteristic when seeking to dampen the oscillations in PV that integral action tends to magnify. Unfortunately, as we will discuss, the potential benefit comes with a price.

The Chaos of Commercial PID Control

The design and tuning of a three mode PID controller follows the proven recipe we have used with success for P-Only control (e.g., here and here) and PI Control (e.g., here, here and here). The decisions and procedures we established for steps 1-3 of the design and tuning recipe in these previous studies remain unchanged as we move on to the PID algorithm.

Step 4 of the recipe remains the same as well. But it is essential in this step that we match the rules and correlations of step 4 with the particular controller algorithm form we are using.

The challenge arises because the number of PID algorithm forms available from hardware vendors increases markedly when derivative action is included. And unfortunately, these PID algorithms are implemented in many different forms across the commercial market.

The potential for confusion by even a careful practitioner is significant. For example:
- there are three popular PID algorithm forms, and
- each of these three forms have multiple parameters that are cast in different ways.

As a result, there are literally dozens of possible PID algorithm forms. Matching each controller form with its proper design rules and correlations requires careful attention if performed without the help of software tools.

Common Algorithm Forms
Listed below are the three common PID controller forms. If offered as an option by our vendor (most do offer it), derivative on measured process variable (PV) is the recommended PID form:

□□ Dependent, ideal PID controller form (derivative on measurement):

$$CO = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{Ti}\int e(t)dt - Kc \cdot Td\frac{dPV}{dt}$$

□□ Dependent, interacting form (derivative on measurement):

$$CO = CO_{bias} + Kc\left(1 + \frac{Td}{Ti}\right)e(t) + \frac{Kc}{Ti}\int e(t)dt - KcTd\frac{dPV}{dt}$$

□□ Independent PID form (derivative on measurement):

$$CO = CO_{bias} + Kc \cdot e(t) + Ki\int e(t)dt - Kd\frac{dPV}{dt}$$

Where for the above:
CO = controller output signal (the wire out)
CO_{bias} = controller bias; set by bumpless transfer
e(t) = current controller error, defined as SP – PV
SP = set point
PV = measured process variable (the wire in)
Kc = controller gain (also called proportional gain), a tuning parameter
Ki = integral gain, a tuning parameter
Kd = derivative gain, a tuning parameter
*T*i = reset time, a tuning parameter
*T*d = derivative time, a tuning parameter

Tuning parameters
Because there has been little standardization on nomenclature, the same tuning parameters can appear under different names in the commercial market. Perhaps more unfortunate, the same parameter can even have a different name within a single company's product line.

We will not attempt to list all of the different names here, though we will look at a solution to this issue later in this article. A few notes to consider:

1) The dependent forms appear most in products commonly used in the process industries, but the independent form is not uncommon.
2) The majority of DCS and PLC systems now use controller gain, Kc, for their dependent PID algorithms. There are notable exceptions, however, such as Foxboro who uses proportional band (PB = 100/Kc assuming PV and CO both range from 0 to 100%).
3) Reset time, *T*i, is slightly more common for the dependent PID algorithms, though it is rarely called that in product documentation. Reset rate, defined as *T*r = 1/*T*i, comes in a close second. Again, the name for this parameter changes with product.
4) Most vendors use derivative time, *T*d, for their dependent PID algorithms, though few

refer to it by that name in their product documentation.

Tune One, Tune Them All

Some good news in all this confusion is that the different forms, if tuned with the proper correlations, will perform exactly the same. No one form is better than another, it is just expressed differently.

In fact, we can show equivalence among the parameters, and thus algorithms, with these relations.

- *Proportional*

$$K_{c,\,ideal} = K_{c,\,interact}\left(1 + \frac{T_{d,\,interact}}{T_{i,\,interact}}\right)$$

$$K_{c,\,independent} = K_{c,\,ideal}$$

- *Integral*

$$T_{i,\,ideal} = T_{i,\,interact}\left(1 + \frac{T_{d,\,interact}}{T_{i,\,interact}}\right)$$

$$K_{i,\,independent} = \frac{K_{c,\,ideal}}{T_{i,\,ideal}}$$

- *Derivative*

$$T_{d,\,ideal} = \frac{T_{d,\,interact}}{\left(1 + \frac{T_{d,\,interact}}{T_{i,\,interact}}\right)}$$

$$K_{d,\,independent} = (K_{c,\,ideal})(T_{d,\,ideal})$$

Though not presented here, analogous conversion relations can be developed for forms expressed using proportional band and/or reset rate.

Clarity in the Chaos

It is perhaps reasonable to hope that industrial practitioners will have an intuitive understanding of proportional, integral and derivative action. They might know the benefits each term offers and problems each presents. And experienced practitioners will know how design, tune and validate a PID implementation.

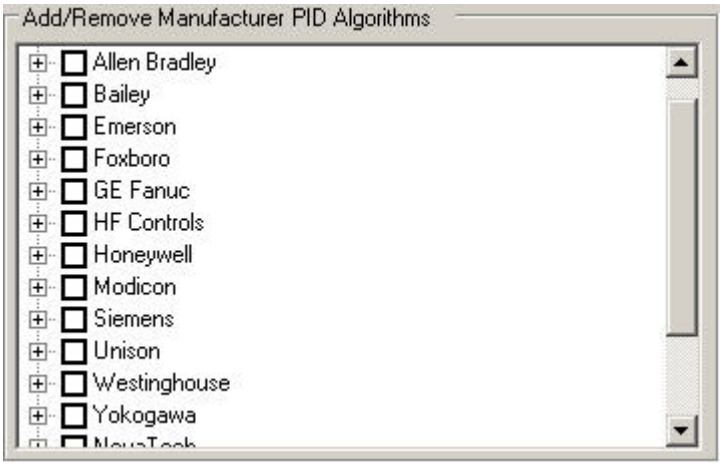
Expecting a practitioner to convert that knowledge and intuition over into the confusion of the commercial PID marketplace might not be so reasonable.

Given this, the best solution for those in the real world is to use software that lets us focus on the big picture while the software ensures that details are properly addressed.

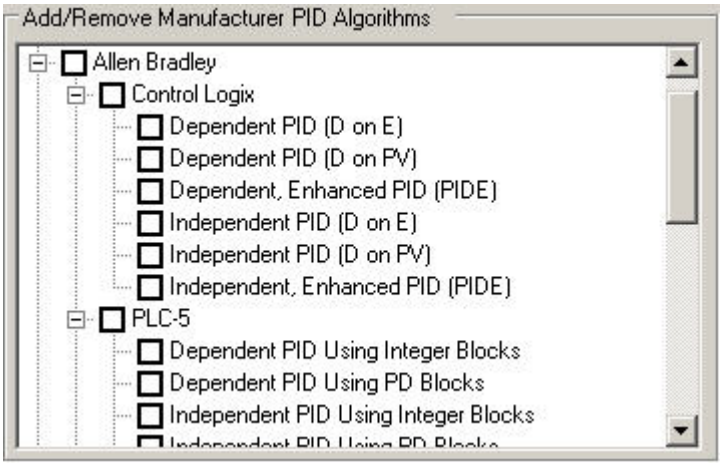
Such productivity software should not only provide a "click and go" approach to algorithm and tuning parameter selection, but should also provide this information simply based on our choice of equipment manufacturer and product line.

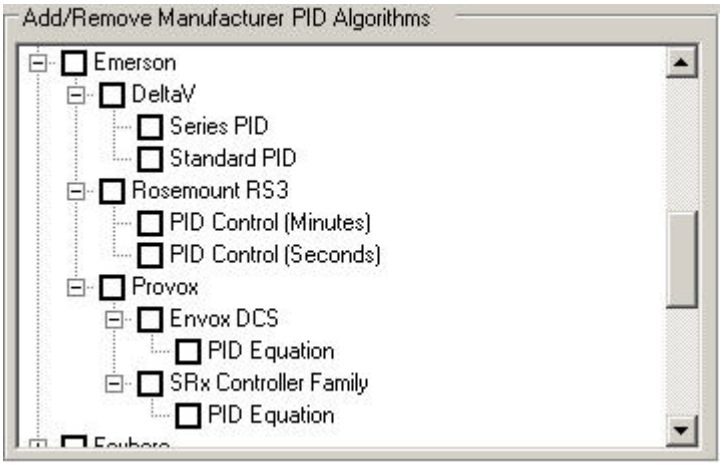
For example, below is a portion of the controller manufacturer selection available in one

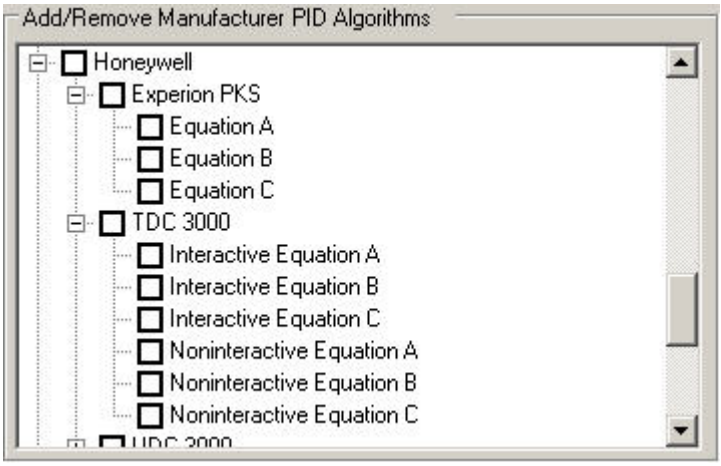
commercial software package:



If you select Allen Bradley, Emerson, and Honeywell in the above list, the choice of PID controllers for each company is shown in the next three images:







It is clear from these displays that there are different terms and many options for us to select from, all for PID control. And it may not be obvious that the different terms above refer to some version of our "basic three" PID forms.

Too much is at stake in a plant to ask a practitioner to keep track of it all. Software can get us past the details during PID controller design and tuning so we can focus on mission-critical control tasks like improving safety, performance and profitability.

Note: the Laplace domain is a subject that most control practitioners can avoid their entire careers, but it provides is a certain mathematical “elegance.”

Below, for example, are the three controller forms assuming derivative on error. Even without familiarity with Laplace, perhaps you will agree the three PID forms indeed look like part of the same family:

- *Dependent, Ideal (Non-interacting) Form*

$$\frac{CO(s)}{E(s)} = Kc\left(1 + \frac{1}{Ti\,s} + Td\,s\right)$$

- *Dependent, Interacting Form*

$$\frac{CO(s)}{E(s)} = Kc\left(1 + \frac{1}{Ti\,s}\right)(Td\,s + 1)$$

- *Independent Form*

$$\frac{CO(s)}{E(s)} = Kc + \frac{Ki}{s} + Kd\,s$$

PID Control of the Heat Exchanger

In recent articles, we investigated P-Only control and then PI control of a heat exchanger. Here we explore the benefits and challenges of derivative action with a PID control study of this process. Our focus is on basic design, implementation and performance issues.

We follow the same four-step design and tuning recipe we use for all control implementations. A benefit of the recipe, beyond the fact that it is easy to use, widely applicable, and reliable in industrial applications, is that steps 1-3 of the recipe remain the same regardless of the control algorithm being employed.

Summary results of steps 1-3 from the previous heat exchanger control studies are presented below. Details for these steps are presented with discussion in the PI control article (nomenclature for this article is listed in step 4).

Step 1: Specify the Design Level of Operation (DLO)
- **Design PV and SP = 138 °C with operation ranging from 138 to 140 °C**
- **Expected warm liquid flow disturbance = 10 L/min**

Step 2: Collect Process Data around the DLO
See the PI control article referenced above for a summary, or go to this article to see details of the data collection experiment.

Step 3: Fit an FOPDT Model to the Dynamic Data
The first order plus dead time (FOPDT) model approximation of the heat exchanger data from step 2 is:
- **Process gain (how far), Kp = –0.53 °C/%**
- **Time constant (how fast), *T*p = 1.3 min**
- **Dead time (how much delay), Θp = 0.8 min**

Step 4: Use the Parameters to Complete the Design
Vendors market the PID algorithm in a number of different forms, creating a confusing array of choices for the practitioner.

The preferred algorithm in industrial practice is PID with derivative on PV. The three most common of these forms each have their own tuning correlations, and they all provide identical performance as long as we take care to match each algorithm with its proper correlations during implementation.

Because the three common forms are identical in capability and performance if properly tuned, the observations and conclusions we draw from any one of these algorithms applies to the other forms.

Among the most widely used is the Dependent, Ideal (Non-interacting) form:

$$CO = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{\tau i}\int e(t)dt - Kc \cdot \tau d \frac{dPV}{dt}$$

Where:
CO = controller output signal (the wire out)
CO_{bias} = controller bias; set by bumpless transfer
e(t) = current controller error, defined as SP – PV
SP = set point
PV = measured process variable (the wire in)
Kc = controller gain, a tuning parameter
***T*i = reset time, a tuning parameter**
***T*d = derivative time, a tuning parameter**

As explained in the PI control study, best practice is to set loop sample time, T, at 10 times per time constant or faster (T ≤ 0.1*T*p). For this process, controller sample time, T = 1.0 sec. Also, like most commercial controllers, we employ bumpless transfer. Thus, when switching to automatic, SP is set equal to the current PV and CO_{bias} is set equal to the current CO.

• *Controller Gain, Reset Time & Derivative Time*
We use the industry-proven Internal Model Control (IMC) tuning correlations in this study. These require specifying the closed loop time constant, *T*c, that describes the desired speed or quickness of our controller in responding to a set point change or rejecting a disturbance.

Guidance for computing *T*c for an aggressive, moderate or conservative controller action are listed in our PI control study and summarized as:

- ***aggressive:*** ***T*c is the larger of 0.1·*T*p or 0.8·Θp**
- ***moderate*:** ***T*c is the larger of 1·*T*p or 8·Θp**
- ***conservative*: *T*c is the larger of 10·*T*p or 80·Θp**

With *T*c determined, the IMC tuning correlations for the Dependent, Ideal (Non-Interacting) PID form are:

$$Kc = \frac{1}{Kp}\left(\frac{Tp + 0.5\theta p}{Tc + 0.5\theta p}\right); \quad Ti = Tp + 0.5\theta p; \quad Td = \frac{Tp\,\theta p}{2Tp + \theta p}$$

Note that, similar to the PI controller tuning correlations, only controller gain contains *T*c, and thus, only Kc changes based on a desire for a more active or less active controller.

We start our study by choosing an aggressive response tuning:

Aggressive *T*c = the larger of 0.1·*T*p or 0.8·Θp
= larger of 0.1 (1.3 min) or 0.8 (0.8 min)
= 0.64 min

Using this *T*c and our Kp, *T*p and Θp from Step 3 in the correlations above, we compute these aggressive PID tuning values:

Aggressive Ideal PID: Kc = –3.1 %/°C; *T*i = 1.7 min; *T*d = 0.31 min

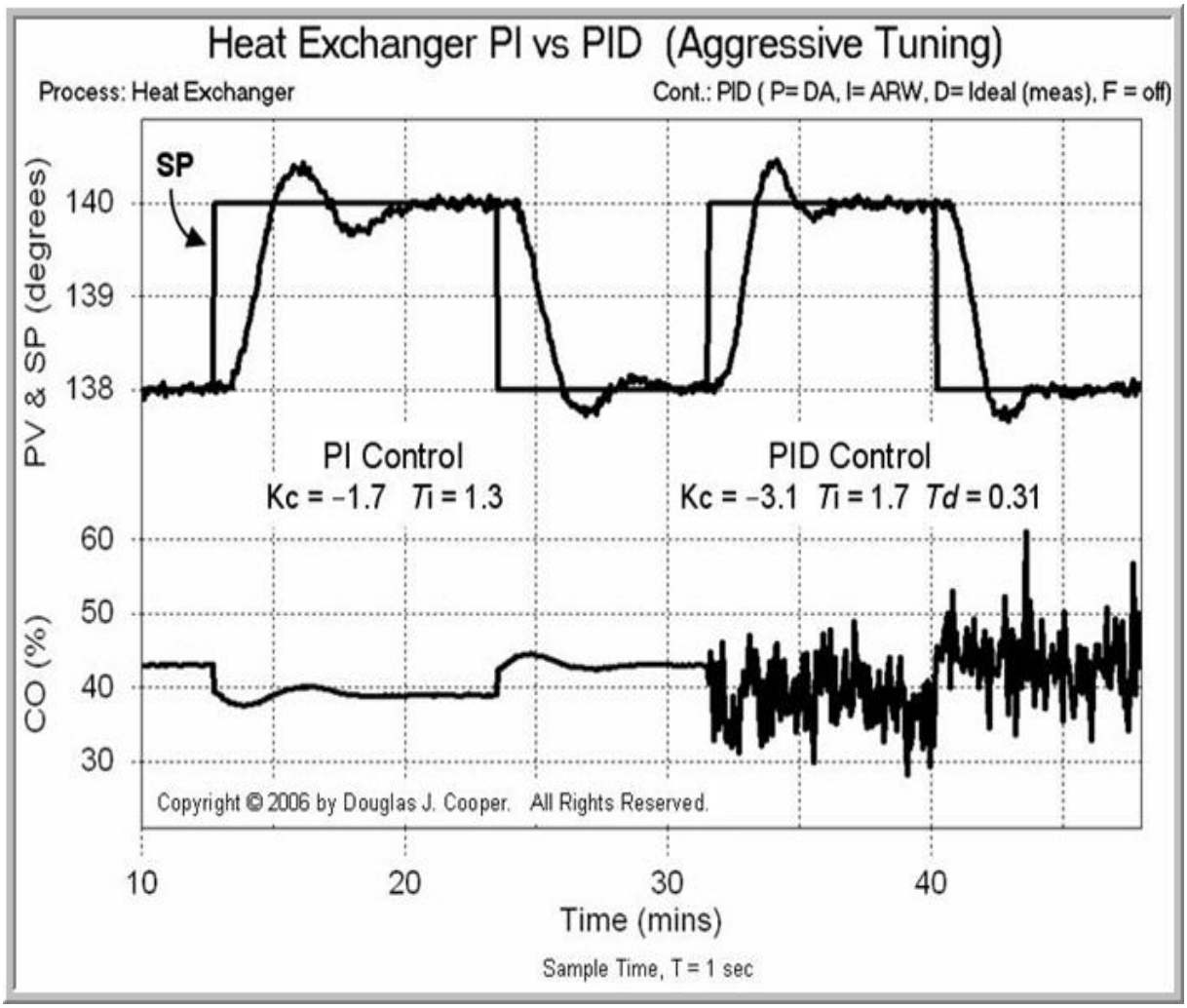
• *Controller Action*

Controller gain is negative for the heat exchanger, yet most commercial controllers require that a positive value of Kc be entered. The way we indicate a negative sign is to choose the *direct acting* controller option during implementation. If the wrong control action is entered, the controller will quickly drive the final control element to full on/open or full off/closed and remain there until a proper control action entry is made.

Practitioner's Note: Controller gain, Kc, always has the same sign as the process gain, Kp. For a process with a negative Kp such as our heat exchanger, when the CO increases, the PV decreases (sometimes called up-down behavior). With the controller in automatic, if the PV is too high, the controller has to increase CO to correct for the error. The CO acts directly toward the problem, and thus, is said to be direct acting. If Kp (and thus Kc) are positive or up-up, when the PV is too high, the controller has to decrease CO to correct for the error. The CO acts in reverse of the problem and is said to be reverse acting.

Implement and Test

Below we compare the performance of two controllers side-by-side for the heat exchanger process simulation. Shown are two set point step pairs from 138 °C up to 140 °C and back again. Though not shown, the disturbance flow rate remains constant at 10 L/min throughout the study.To the left is set point tracking performance for an aggressively tuned PI controller from our previous study. To the right is the set point tracking performance of our aggressively tuned PID controller.



The PID controller not only displays a faster rise time (because Kc is bigger), but also a faster settling time. This is because derivative action seeks to inhibit rapid movements in the process variable. One result of this is a decrease in the rolling or oscillatory behavior in the PV trace.

Perhaps more significant, however, is the obvious difference in the CO signal trace for the two controllers. Derivative action causes the noise (random error) in the PV signal to be amplified and reflected in the control output. Such extreme control action will wear a mechanical final control element, requiring increased maintenance.

This unfortunate consequence of noise in the measured PV is a serious disadvantage with PID control. We will discuss this more in the next article.

Tune One, Tune Them All
To complete this study, we compare the Dependent, Ideal (Non-interacting) form above to the performance of the Dependent, Interacting form:

$$CO = CO_{bias} + Kc\left(1 + \frac{Td}{Ti}\right)e(t) + \frac{Kc}{Ti}\int e(t)dt - Kc\,Td\frac{dPV}{dt}$$

The IMC tuning correlations for this form are:

$$Kc = \frac{1}{Kp}\left(\frac{\tau p}{\tau c + 0.5\theta p}\right); \quad \tau i = \tau p; \quad \tau d = 0.5\,\theta p$$

For variety, we explore moderate response tuning:

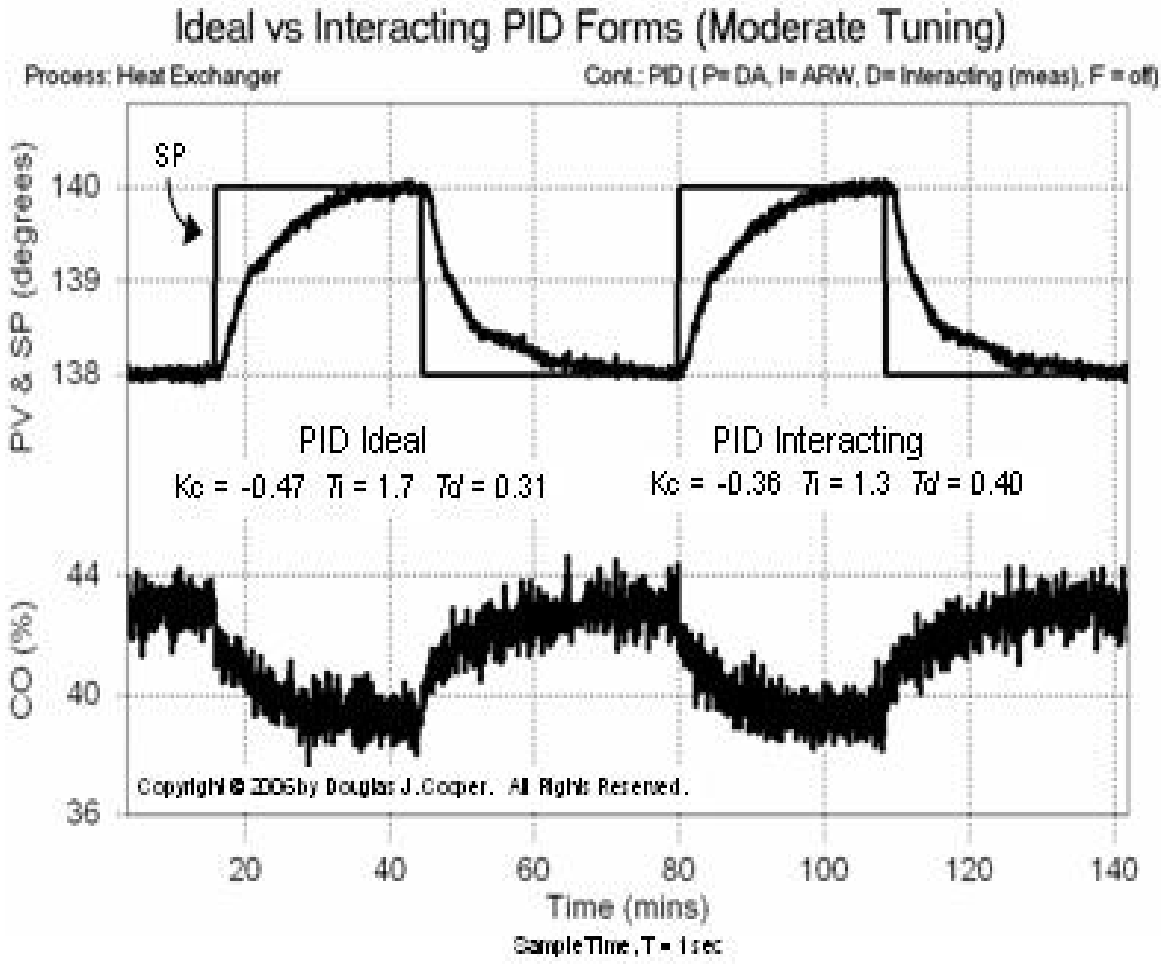
Moderate τc = the larger of $1 \cdot \tau p$ or $8 \cdot \Theta p$
= larger of 1 (1.3 min) or 8 (0.8 min)
= 6.4 min

Using this τc and our model parameters in the proper tuning correlations, we arrive at these moderate tuning values:

Dependent, Interacting PID: Kc = –0.36 %/°C; τi = 1.3 min; τd = 0.40 min

Dependent, Ideal PID: Kc = –0.47 %/°C; τi = 1.7 min; τd = 0.31 min

As shown in the plot below (), we see that moderate tuning provides a reasonably fast PV response while producing no overshoot.



But more important, we establish that the interacting PID form and the ideal PID

form provide identical performance when tuned with their own correlations.

Measurement Noise Degrades Derivative Action

At the start of a recent Practical Process Control workshop, I asked the attendees what the "D" in PID stood for. One fellow immediately shouted from the back of the room, "Disaster?" Another piped in, "How about Danger?" When the laughter died down, another emphatically stated, "D is for Do not use." This one got a good laugh out of me. I had not heard it before and thought it was perfect. And here's why…

Benefits and Drawbacks
Derivative action has its largest influence when the measured process variable (PV) is changing rapidly (when the slope of the PV trace is steep).

The three terms of a properly tuned PID controller thus work together to provide a rapid response to error (proportional term), to eliminate offset (integral term), and to minimize oscillations in the PV (derivative term).

While this sounds great in theory, unfortunately, there are serious drawbacks to including derivative action in our controller.

We have discussed how challenging it can be to balance *two* interacting tuning parameters for PI control.

A PID controller has *three* tuning parameters (three modes) that all interact and must be balanced to achieve a desired performance. It is often not at all obvious which of the three tuning parameters must be adjusted to correct behavior if performance is considered to be undesirable.

Trial and error tuning is hopeless for any but the most skilled practitioner. Fortunately, our tuning recipe provides a quick route to a safe and profitable PID performance.

A second disadvantage relates to the uncertainty in the derivative computation for processes that have noise in the PV signal.

PID Controller Form
For discussion purposes in this article, we use the Dependent, Ideal (Non-interacting) form:

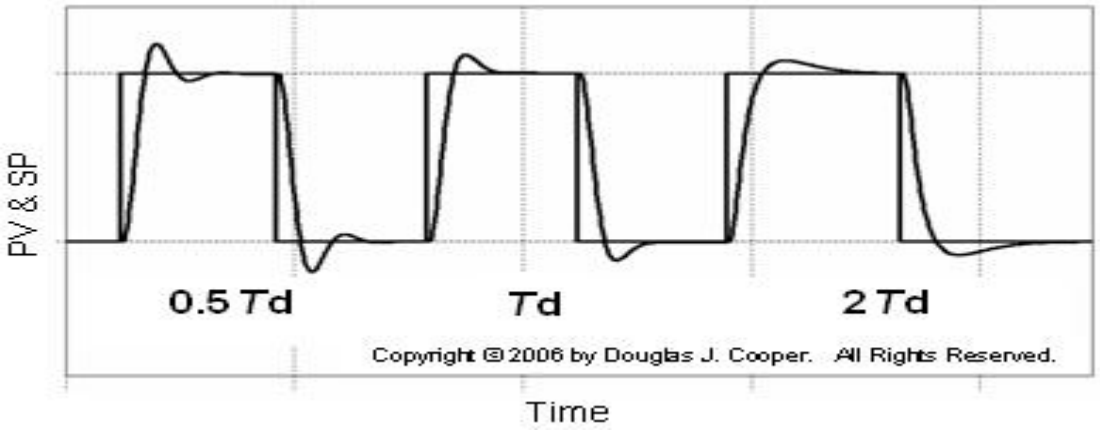
$$CO = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{Ti}\int e(t)dt - Kc \cdot Td\frac{dPV}{dt}$$

Nomenclature and tuning correlations for conservative, moderate and aggressive performance are presented here.

As discussed in this article, the various PID algorithms forms provide identical performance if each algorithm is matched to its proper tuning correlations. Hence, the observations and conclusions presented below are general in nature and are not specific

to a particular algorithm form.

Derivative Action Dampens Oscillations
The plot below shows the impact of derivative action on set point response performance. Because noise in the PV signal can impact performance, an idealized noise-free simulation was used to create the plot.



The plot shows the PV response to three pairs of set point (SP) steps. The middle response shows the base case performance of a PID controller tuned using the aggressive correlations referenced above.

For the set point steps to the right and left of the base case, the derivative time, *T*d, is adjusted while the controller gain, Kc, and reset time, *T*i, are kept constant. This let's us isolate the impact of derivative time on performance.

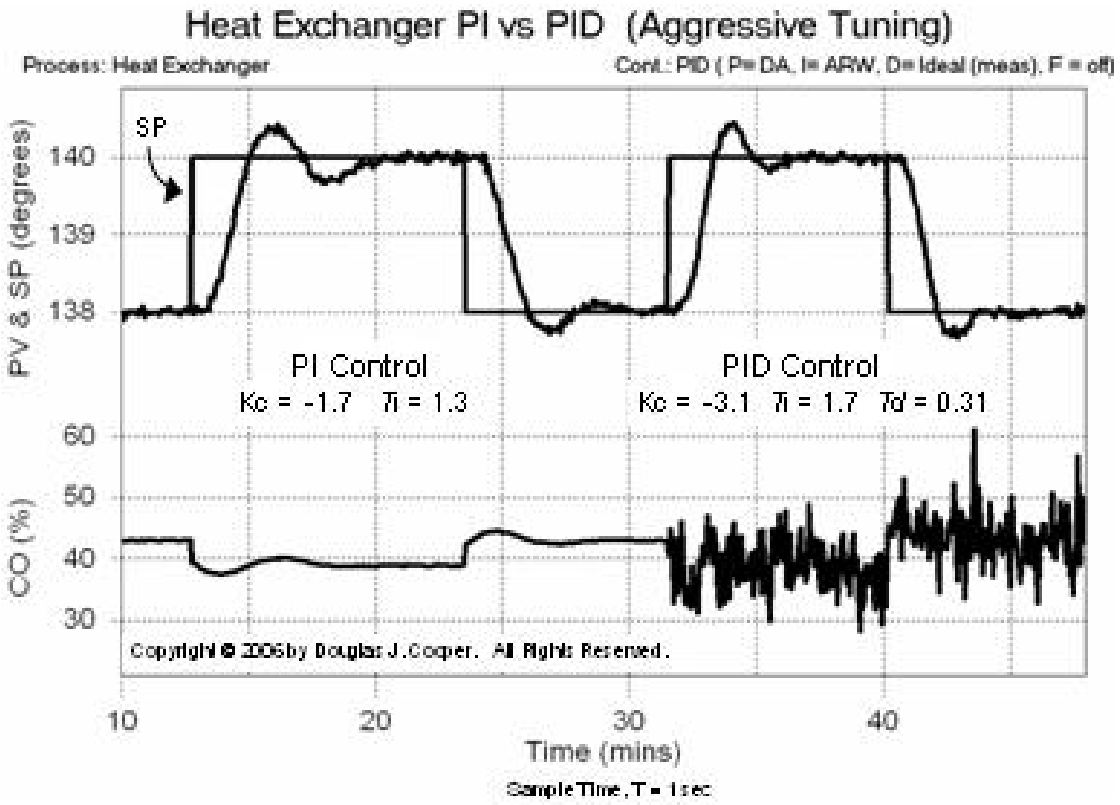
It is apparent that when derivative action is cut in half to the left in the plot, the oscillating nature of the response increases. And when *T*d is doubled to the right, the increased derivative action inhibits rapid movement in the PV, causing the rise time and settling time to lengthen.

We saw in this tuning map from the article on interacting tuning parameters for PI controllers that with only two tuning parameters, performance response plots could look similar.

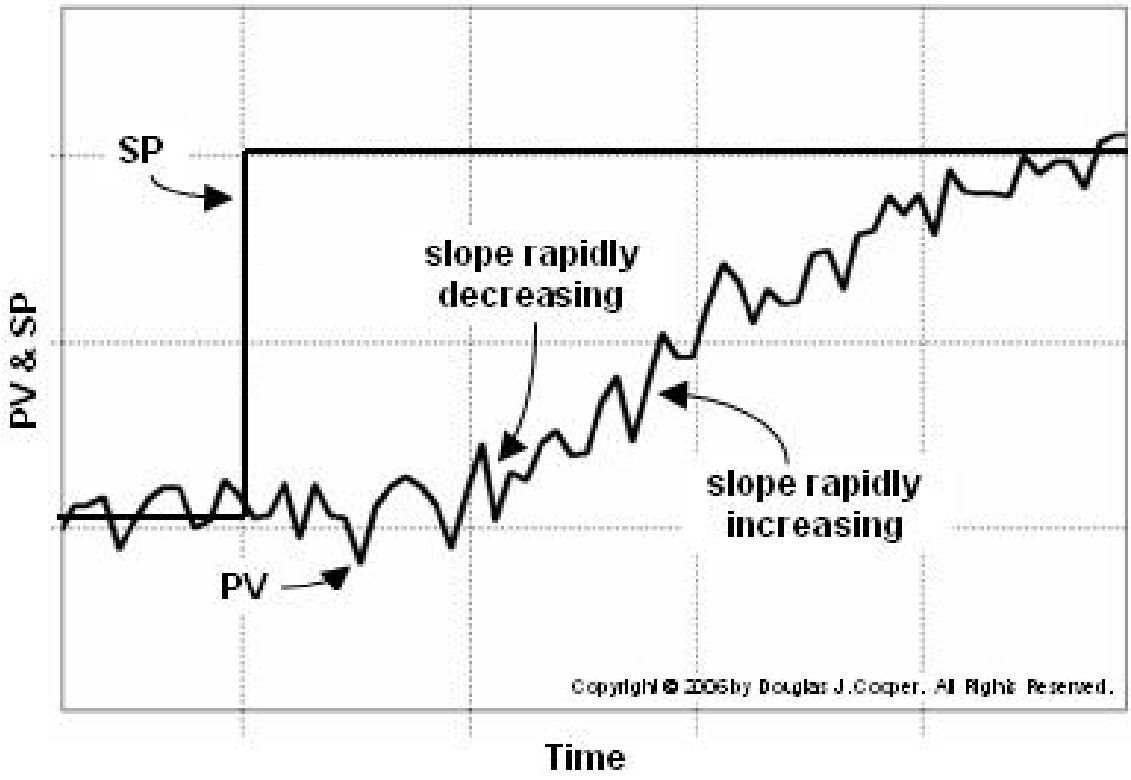
With the addition of *T*d, we now have a three dimensional tuning map with a great many similar-looking plots. If we are unhappy with our controller performance, knowing which parameter to adjust and by how much borders on the impossible. With three mode PID control, the orderly approach of our tuning recipe becomes fundamental to success.

Measurement Noise Leads to Controller Output "Chatter"
As discussed in more detail in the PID control of the heat exchanger study and summarized here, the side-by-side comparison of PI vs PID control shown below illustrates one unwelcome result of adding derivative action to our controller.

The CO signal trace along the bottom of the plot clearly changes when the derivative term is added. Specifically, derivative action causes the noise (random error) in the PV signal to be amplified and reflected in the controller output.



The reason for this extreme CO action or "chatter" is illustrated below. As indicated in the plot, a noisy PV signal produces conflicting derivatives as the slope appears to dramatically alternate direction at every sample.



The consequence of a PV that repeatedly changes from “rapidly increasing slope” to “rapidly decreasing slope” is a derivative term that computes a series of large, alternating

CO actions.

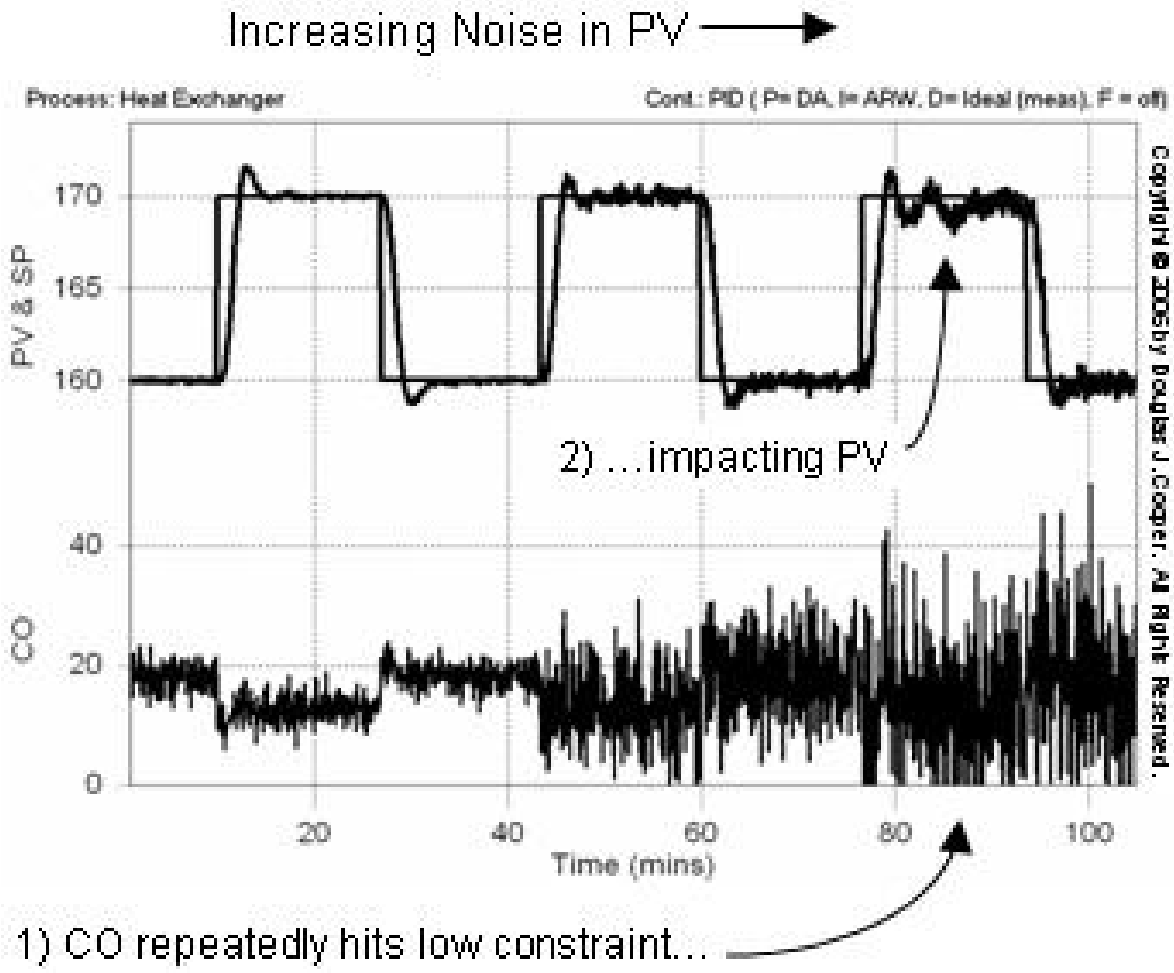
The ultimate impact of this alternating derivative computation on the total CO depends on the size of *T*d (the weight given to the derivative term). As *T*d grows larger, the “chatter” in the CO signal grows in response.

In any event, extreme control action will increase the wear on a mechanical final control element and lead to an increase in maintenance costs.

Larger Noise Means Larger Problems

The plot below shows a more subtle problem that measurement noise can cause with derivative action. In particular, this problem arises when the level of operation is near a controller output constraint (either the maximum or minimum CO).

The PID tuning values in the plot are constant throughout the experiment. As indicated on the plot, measurement noise is increased in increments across three set point tracking tests.



As long as measurement noise causes the derivative to alternate equally between suddenly increasing and suddenly decreasing, and the controller output can reflect this “equality in randomness” unimpeded, then controller performance is reasonably consistent in spite of increasing noise in the PV signal.

If a constraint inhibits the controller output from the “equality in randomness” symmetry, causing it to become skewed or off center, then controller performance degrades.

This is illustrated in the right most set point steps above. For this case, the controller output signal becomes so active that it repeatedly hits the minimum CO value. By constraining CO, the controller output loses its symmetry, causing the PV to wander.

What's the Solution?
One solution is to include a signal filter somewhere in the PID loop. There are several possible locations, a half dozen candidate filter algorithms, and choice of a hardware or software implementation. We explore filtering enough to see big-picture concepts and the potential benefits, though we will by no means exhaust the topic.

PID Disturbance Rejection Of The Gravity Drained Tanks

We have explored disturbance rejection in the gravity drained tanks process using P-Only and then PI control. In the PI study, we confirmed the observations we had made in the PI control of the heat exchanger investigation.

In particular, we learned that PI controllers:
- can eliminate the offset associated with P-Only control,
- have integral action that increases the tendency for the PV to roll (or oscillate),
- have two tuning parameters that interact, making it challenging to correct tuning when performance is not acceptable.

Here we investigate the benefits and challenges of derivative action and PID control when disturbance rejection remains our control objective.

As with all controller implementations, we follow our four-step design and tuning recipe. A benefit of this recipe is that steps 1-3 are independent of the controller used, so our previous results from steps 1 and 2 (detailed here) and step 3 (detailed here and here) can be used in this PID study.

We summarize those previous results before proceeding to step 4 and the design and tuning of a PID controller (nomenclature for this article is listed in step 4).

Step 1: Determine the Design Level of Operation (DLO)
The control objective is to reject disturbances as we control liquid level in the lower tank. Our DLO for this study is:
- design PV and SP = 2.2 m with range of 2.0 to 2.4 m
- design D = 2 L/min with occasional spikes up to 5 L/min

Step 2: Collect Process Data around the DLO
When CO, PV and D are steady near the design level of operation (DLO), we bump the CO as detailed here and force a clear response in the PV that dominates the noise.

Step 3: Fit a FOPDT Model to the Dynamic Process Data
We approximate the dynamic behavior of the process by fitting test data with a first order

plus dead time (FOPDT) dynamic model. A fit of step test data and doublet test data yields these values:

- process gain (how far), Kp = 0.09 m/%
- time constant (how fast), Tp = 1.4 min
- dead time (how much delay), Θp = 0.5 min

Step 4: Use the FOPDT Parameters to Complete the Design

The preferred PID algorithm in industrial practice employs derivative on PV, and vendors market this controller in several different forms. Each algorithm form has its own tuning correlations, and if we take care to match algorithm with correlation, they all provide identical capability and performance.

For tuning, we rely on the industry-proven Internal Model Control (IMC) tuning correlations. These require only one specification, the closed loop time constant (*T*c), that describes the desired speed or quickness of our controller in responding to a set point (SP) change or rejecting a disturbance (D).

Our PI control study describes what to expect from an aggressive, moderate or conservative controller. Once our desired performance is chosen, the closed loop time constant is computed:

- *aggressive:* *T*c is the larger of 0.1·*T*p or 0.8·Θp
- *moderate*: *T*c is the larger of 1·*T*p or 8·Θp
- *conservative*: *T*c is the larger of 10·*T*p or 80·Θp

Because the popular PID forms perform the same if properly tuned, the observations and conclusions we draw from any one algorithms applies to the other forms.

Dependent Ideal PID

Among the most widely used algorithms is the Dependent Ideal (Non-interacting) PID form:

$$CO = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{Ti}\int e(t)dt - Kc \cdot Td\frac{dPV}{dt}$$

Where:

CO = controller output signal (the wire out)
CO_{bias} = controller bias; set by bumpless transfer
e(t) = current controller error, defined as SP – PV
SP = set point
PV = measured process variable (the wire in)
Kc = controller gain, a tuning parameter
*T*i = reset time, a tuning parameter
*T*d = derivative time, a tuning parameter

• *Design and Tune*

In the P-Only study, we had established that for the gravity drained tanks process:

- sample time, T = 1 sec
- the controller is reverse acting

▪ dead time is small compared to *T*p and thus not a concern in the design

After we choose a *T*c based on our desired performance, the tuning correlations for the Dependent Ideal PID form are:

$$Kc = \frac{1}{Kp}\left(\frac{Tp + 0.5\theta p}{Tc + 0.5\theta p}\right); \quad Ti = Tp + 0.5\theta p; \quad Td = \frac{Tp\,\theta p}{2Tp + \theta p}$$

Similar to the PI controller tuning correlations, only controller gain contains *T*c, and thus, only Kc changes based on the need for a more or less active controller.

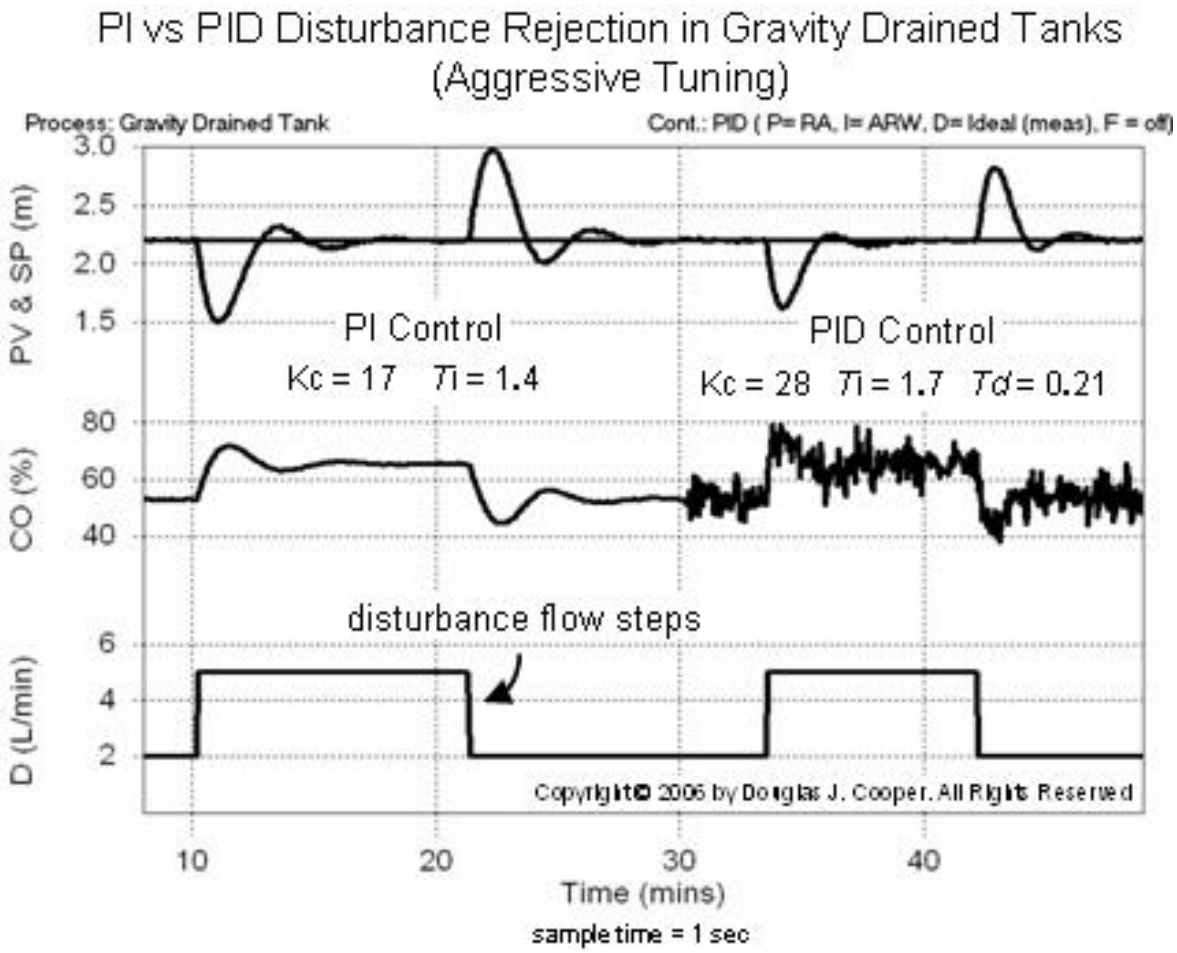
• *Implement and Test*
We first explore an aggressive response tuning for our ideal PID controller:

Aggressive *T*c = the larger of 0.1·*T*p or 0.8·Θp
= larger of 0.1 (1.4 min) or 0.8 (0.5 min)
= 0.4 min

Using this *T*c and our Kp, *T*p and Θp from Step 3 in the tuning correlations above, we compute these aggressive controller gain, reset time and derivative time tuning values:

Aggressive Ideal PID: Kc = 28 %/m; *T*i = 1.7 min; *T*d = 0.21 min

The performance of this controller in rejecting changes in the pumped flow disturbance (D) for the gravity drained tanks is shown to the right in plot below (). For comparison, the performance of an aggressive PI controller is shown in the plot to the left (design details here). Note that the set point (SP) remains constant at 2.2 m throughout the study.



The maximum deviation of the PV from set point during the disturbance rejection event is smaller for the PID controller relative to the PI controller. The PID controller also provides a faster settling time because derivative action tends to reduce the rolling or oscillatory behavior in the PV trace.

Like the heat exchanger PID study, there is an obvious difference in the CO signal trace for the PI vs PID controllers. Derivative action causes the noise (random error) in the PV signal to be amplified and reflected in the control output (CO) signal.

Such extreme control action will cause excessive wear in a valve or other mechanical final control element, requiring increased maintenance. This consequence of noise in the measured PV can be a serious disadvantage with PID control.

Ideal vs Interacting PID

We compare the Dependent Ideal PID form above to the performance of the Dependent Interacting PID form and establish that they are identical in performance if properly tuned. The Dependent Interacting form is written:

$$CO = CO_{bias} + Kc\left(1 + \frac{Td}{Ti}\right)e(t) + \frac{Kc}{Ti}\int e(t)dt - Kc\,Td\frac{dPV}{dt}$$

• *Design and Tune*

We use the same rules above to choose a *T*c that reflects our desired performance. The

IMC tuning correlations for the Dependent, Interacting form are then:

$$Kc = \frac{1}{Kp}\left(\frac{\tau p}{\tau c + 0.5\theta p}\right); \quad \tau i = \tau p; \quad \tau d = 0.5\,\theta p$$

As before, only controller gain contains τc, and thus, only Kc changes based on a desire for a more or less active controller. Sample time remains for this implementation at T = 1 sec and the controller remains as reverse acting.

• *Implement and Test*
We choose a moderate response tuning in this example:

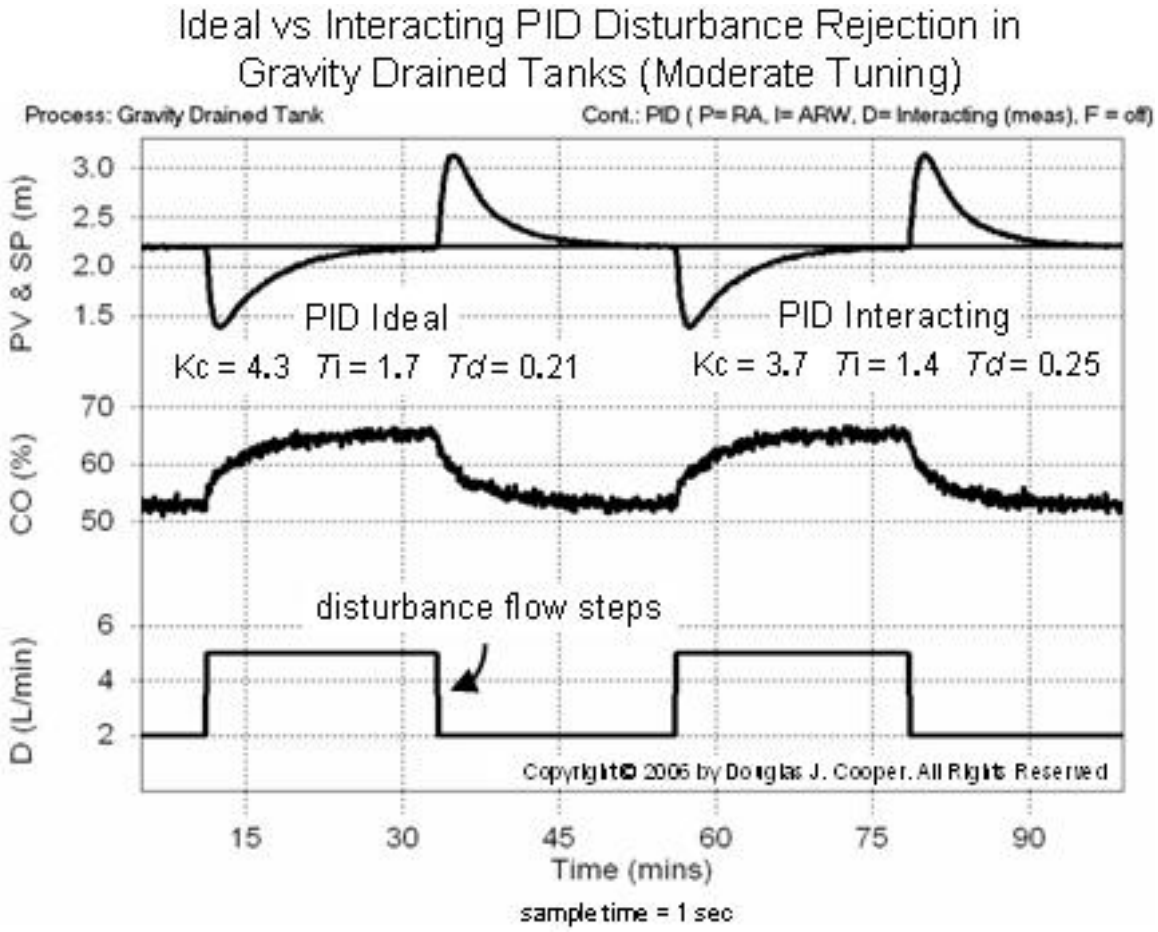
Moderate τc = the larger of $1 \cdot \tau p$ or $8 \cdot \Theta p$
= larger of 1.0 (1.4 min) or 8 (0.5 min)
= 0.4 min

Using this τc and our model parameters in the proper tuning correlations (ideal or interacting), we arrive at these moderate tuning values:

Moderate Ideal PID: Kc = 4.3 %/m; τi = 1.7 min; τd = 0.21 min

Moderate Interacting PID: Kc = 3.7 %/m; τi = 1.4 min; τd = 0.25 min

As shown in the plot below (), moderate tuning provides a reasonably fast disturbance rejection response while producing little or no oscillations as the PV settles.



The indistinguishable behavior confirms that the two controllers indeed are identical in capability and performance if tuned with their own correlations.

Aside: Our observations using the dependent ideal and dependent interacting PID algorithms directly apply to the other popular PID controller forms.

For example, the independent PID algorithm form is written:

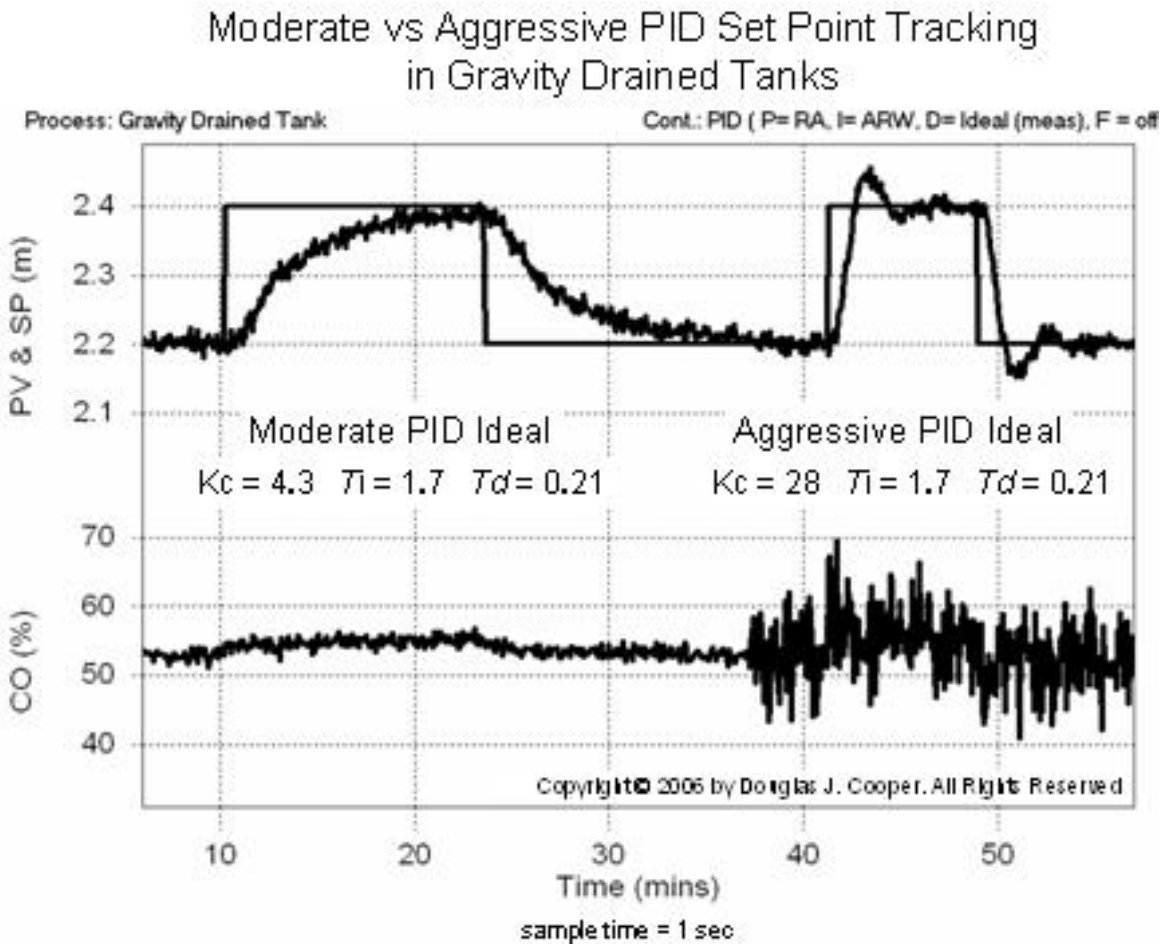
$$CO = CO_{bias} + Kc \cdot e(t) + Ki \int e(t)dt - Kd \frac{dPV}{dt}$$

The integral and derivative gains in the above independent form can be computed, for example, using the ideal PID correlations as: Ki = Kc/*T*i and Kd = Kc□*T*d.

Because of these mathematical identities, performance and capability observations drawn about one algorithm will apply directly to the other.

Ideal Moderate vs Ideal Aggressive

As shown in the plot below (), we compare moderate tuning side-by-side with aggressive tuning for the dependent ideal PID controller. For a different perspective, we make this comparison using a set point tracking objective.



The performance of the two controllers matches the design descriptions provided here. That is, a controller tuned with:

- a moderate *T*c will move the PV reasonably fast while producing little to no overshoot.
- an aggressive *T*c will move the PV quickly enough to produce some overshoot and then oscillation as the PV settles out.

Need for CO Filtering

The excessive activity in the CO signal can be a problem, and a controller output (CO) signal filter is one solution. An interesting observation from the above plot is that the degree of "chatter" in the CO signal grows as controller gain, Kc, increases.

10) Signal Filters and the PID with Controller Output Filter Algorithm

Using Signal Filters In Our PID Loop

In our study of the derivative mode of a PID controller, we explored how noise or random error in the measured process variable (PV) can degrade controller performance.

As discussed in that article, derivative action can cause the noise in the PV measurement to be reflected and amplified in the controller output (CO) signal, producing "chatter" in the final control element (FCE). This extreme control action will increase the wear on a mechanical FCE (e.g., a valve) and lead to increased maintenance needs.

Sources of Noise
Random behavior in the PV measurement arises because of signal noise and process noise.

Signal noise tends to have higher frequency relative to the characteristic dynamics of process control applications (i.e., processes with streams comprised of liquids, gases, powders, slurries and melts). Sources of signal noise include:

- electrical interference
- jitter (clock related irregularities such as variations in sample spacing)
- quantizing of signal samples into overly-broad discrete "buckets" from low resolution or improperly specified instrumentation (e.g. too-large measurement span relative to operating range).

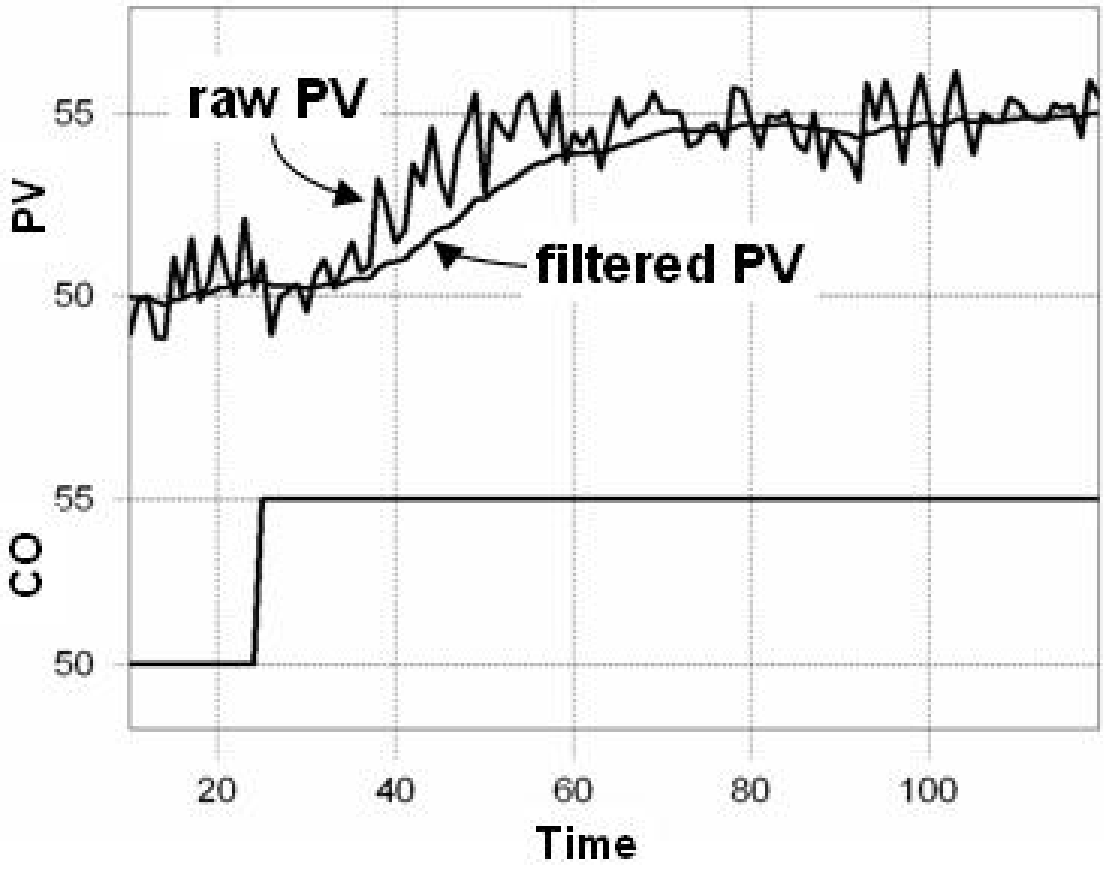
Process noise tends to be lower in frequency. This category borders on the philosophical as to what constitutes a disturbance to be controlled versus noise to be filtered.

Bubbles and splashing that randomly corrupts liquid pressure drop measurements is an example of process noise that might benefit from filtering.

A less clear candidate for filtering is a temperature measurement in a poorly-mixed vessel. The mixing patterns can cause lower-frequency random variations in the temperature signal that are unrelated to changes in the bulk vessel temperature.

It is important to emphasize that before we try to filter away a problem, we should first work to understand the source of the random error. Rather than "fix" the noise by hiding it with additional or modified algorithms, we should attempt to reduce or eliminate the problem through normal engineering and maintenance practices.

The Filtered Signal
The plot below shows the random behavior of a raw (unfiltered) PV signal and the smoother trace of a filtered PV signal.



As the above plot illustrates, a filter is able to receive a noisy signal and yield a signal with reduced random variation. A “better” filter design is one that decreases the random variation while retaining more of the true dynamic information of the original signal.

Filters can be analog (hardware) or digital (software); high, low or band pass; linear or nonlinear; designed in the time, Z-transform or frequency domain; and much more. Filters can collect and process data at a rate faster than the control loop sample time, so many data points can go into a single PV sample forwarded to the controller.

Clearly, filter design is a substantial topic. In these articles we offer only an introduction to the basic methods and ideas.

Filters Add Delay

The filtered signal in the plot above, though perhaps visually appealing, clearly lags behind the actual dynamic response of the unfiltered signal. More specifically, the filtered signal has an increased dead time and time constant relative to the behavior of the actual process.

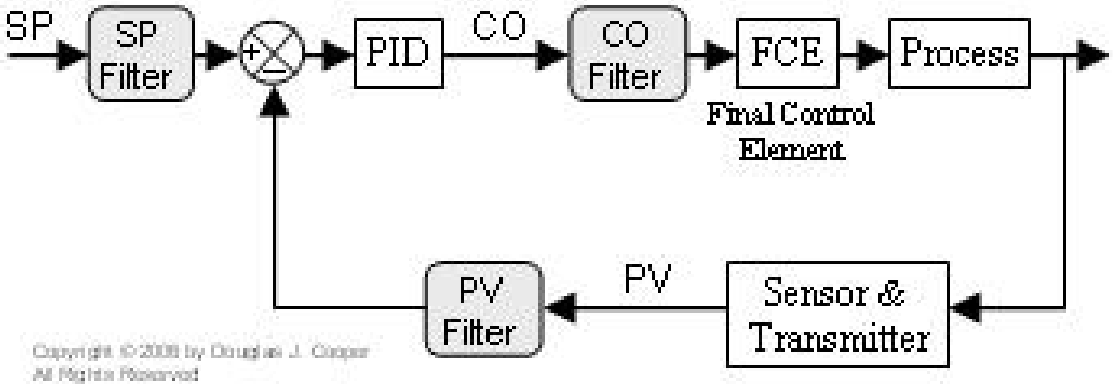
Signal filters offer benefit in process control applications because they can temper the large CO moves caused by derivative action of noisy PV measurements.

Yet they add delay in sensing the true state of a process, and this has negative consequences in that as delay increases, the best achievable control performance decreases.

The design challenge is to find the careful balance between signal smoothing and

information delay to achieve the controller performance we desire for our application.

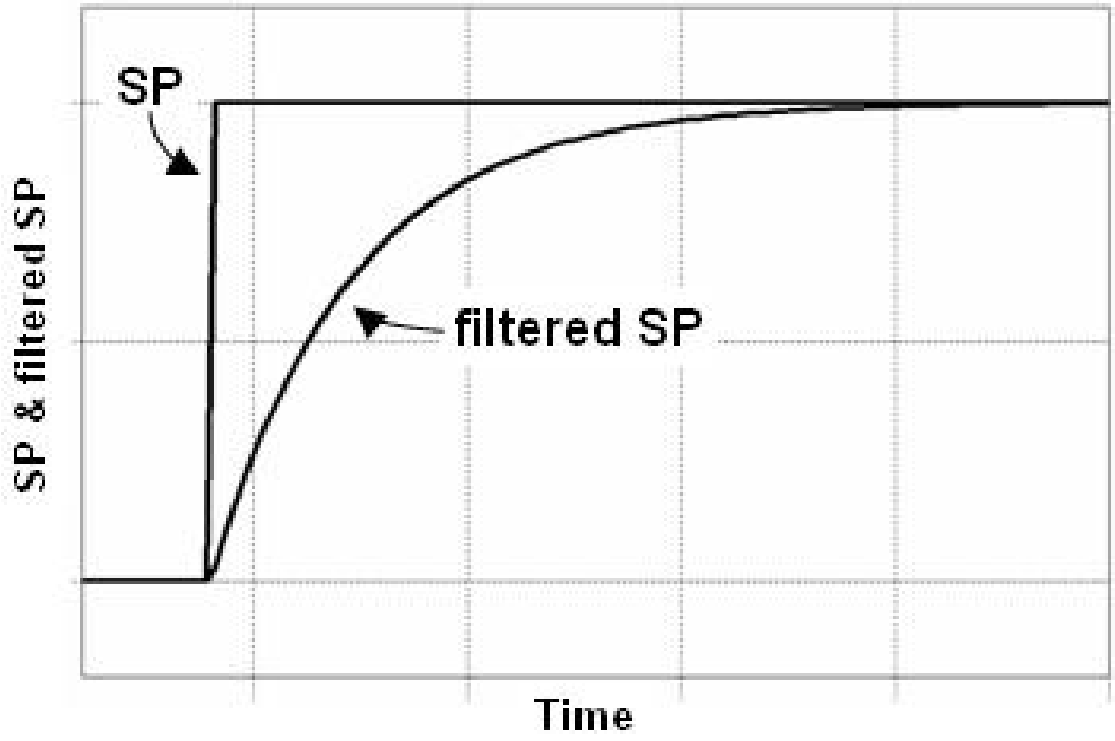
External Filters in Control
As shown below, there are three popular places to put external filters in the feedback loop. By “external,” we mean that the filters are designed, installed and maintained separately from the controller.



Internal filters that are part of the controller architecture itself are introduced later in this article.

• *Set Point Filters*
Set point filters are not associated with the noisy PV problem, but are included here to make the discussion general.

A set point filter takes a step change in SP, and as shown below, forwards a smooth transition signal to the controller.



SP filters do not influence the disturbance rejection (regulatory) performance of a

controller. Hence, these filters permit a controller to be tuned aggressively to reject disturbances, yet the smoothed SP transition results in a moderate set point tracking (servo) performance from this same aggressive controller.

SP filters are also used to limit overshoot at the top of a set point ramp or step change. If this is our design objective, an alternative is to eliminate the filter and employ a controller that uses proportional on PV rather than proportional on error. For example (compare to PI with proportional on error here):

$$CO = CO_{bias} - Kc\,PV + \frac{Kc}{Ti}\int e(t)dt$$

We will explore methods to limit overshoot in more detail in a future article.

Finally, and unfortunately, SP filters are occasionally used as a bandage to mask the fact that a controller is simply poorly designed and/or tuned. We all recognize that this is a practice to be avoided.

• *PV Filters*
Signal filters are frequently placed between the sensor transmitter and the controller (or more likely, the multiplexer feeding the controller). In process control applications, these filters should be analog (hardware) devices designed specifically to minimize high frequency electrical interference.

While measurement noise does degrade derivative action in that it leads to chatter in the CO signal, this “noise leads to CO chatter” effect is very modest for proportional action. And interestingly, integral action is unaffected by noise because the constant summing of error literally averages the random variations in the signal.

Since filtering adds delay and this hurts best possible control performance, and because noise is not an issue for proportional and integral action, it is generally poor practice to filter the PV signal external to the controller for anything beyond electrical interference.

The preferred approach is to selectively filter only that signal destined for the derivative computation. This moves the filter inside the controller architecture as discussed below.

• *CO Filters*
While PV filters smooth the signal feeding the controller, CO filters smooth the noise or “chatter” in the CO signal sent to the final control element. Even if PV signal noise does not appears to cause performance problems, a CO filter can offer potential benefits as it reduces fluctuations in the controller output and this reduces wear on the FCE.

If a noisy PV is an issue in our controller, our first attempts should be to locate and correct the problem. If after that exercise, our decision is to design and implement a filter in our feedback loop, CO filters are attractive alternatives.

The design of a CO filter can be integrated as part of the loop tuning exercise. We will see in a later article how we can integrate an external CO filter into a unified "PID with CO Filter" form, and then use our controller design recipe to tune what is essentially a four

mode controller.

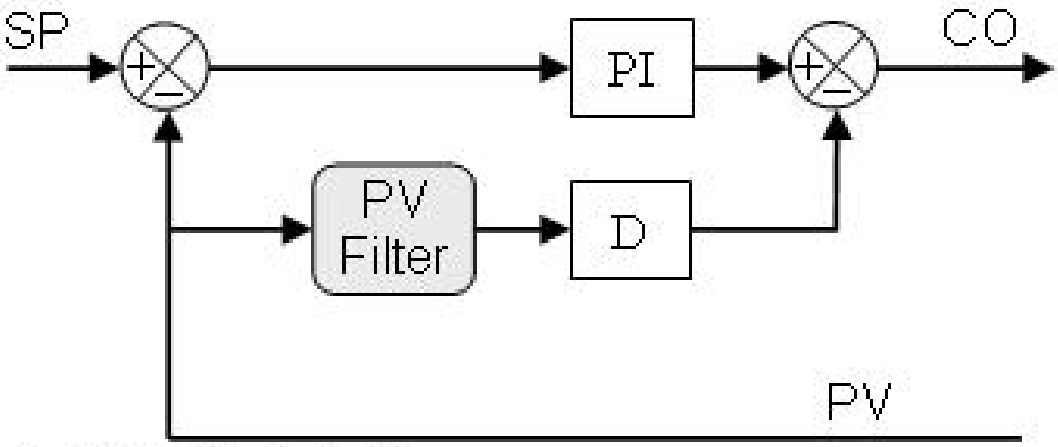
Internal Filters in Control

For feedback control, filtering need only be applied to the signal feeding the derivative term. As stated before, noise does not present a problem for proportional and integral action. These elements will perform best without the delay introduced from a signal filter.

When we selectively filter just that signal feeding the derivative calculation, the filter becomes part of the controller architecture. Hence, we can still use our design recipe, though the correlations for tuning this four mode form are different from the four mode "PID with CO Filter" form mentioned above.

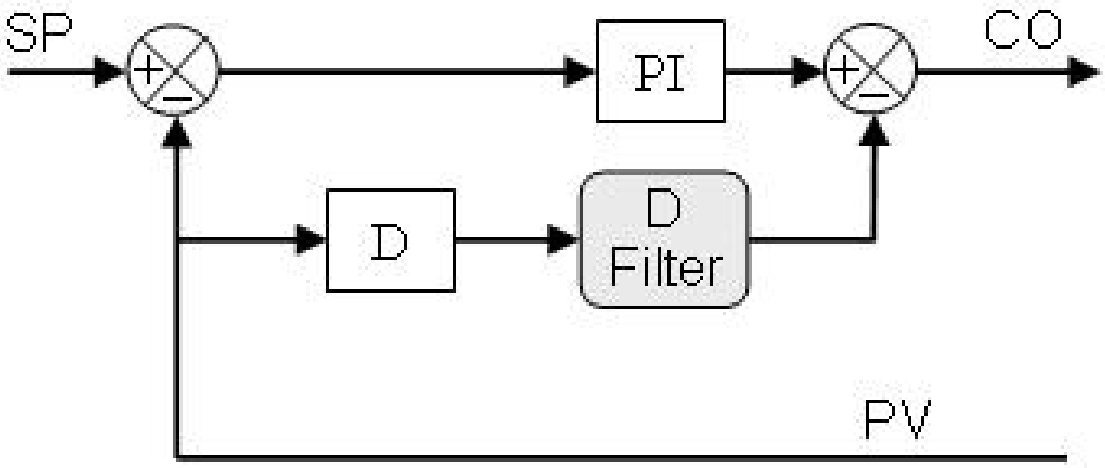
There are two common architectures that are identical in capability, though different in presentation.

As shown below, we filter the PV signal before feeding it to the derivative mode of the PID algorithm for computation:



Copyright © 2006 by Douglas J. Cooper.
All Rights Reserved

We can also compute the derivative action with the noisy signal and then filter the computed result:



Copyright © 2006 by Douglas J. Cooper.
All Rights Reserved

If the same filter form is used (e.g., first-order), we can show mathematically that both options above are identical.

As it turns out, many commercial controllers use this internal derivative filtering form where they implement a first-order filter and fix the filter time at one-tenth of the derivative time value.

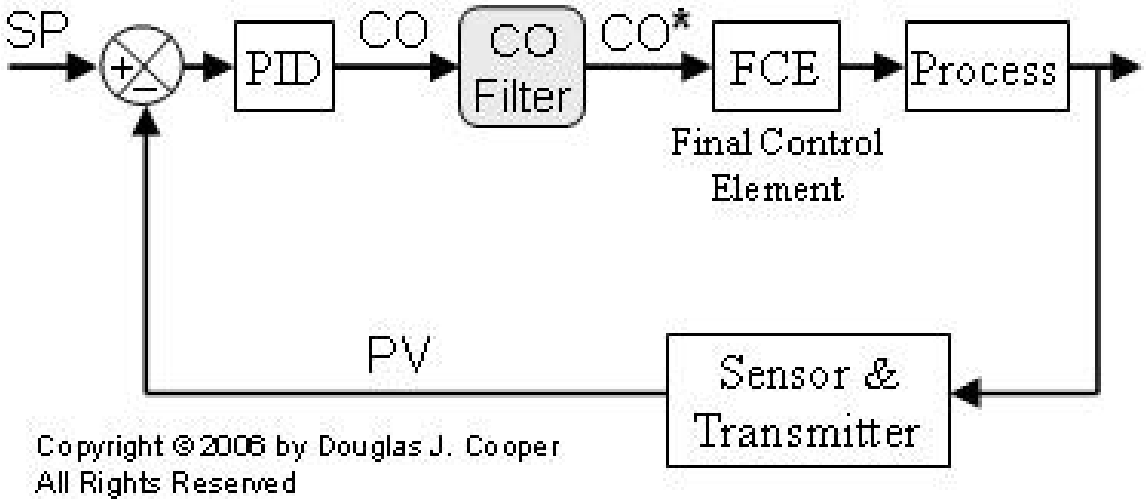
PID with Controller Output (CO) Filter

The derivative action of a PID controller can cause noise in the measured process variable (PV) to be amplified and reflected as "chatter" in the controller output (CO) signal. Signal filters, implemented as either analog hardware or digital software, offer a popular solution to this problem.

If noise is impacting controller performance, our first attempts should be to locate and correct the underlying fault. Filters are poor cures for a bad design or failing equipment.

If we decide to employ a filter in our loop, an algorithm designed to smooth the controller output signal holds some allure.

The CO Filter Architecture
Below is a loop architecture with a PID controller followed by a controller output filter.



The benefits of this architecture include:

- A CO filter works to limit large controller output moves regardless of the underlying cause. As a result, CO filters can reduce persistent controller output fluctuations that cause wear in a mechanical final control element (FCE).
- A CO filter is a single solution that addresses both a noisy PV measurement problem and computational oddities that may exist in our vendor's particular PID algorithm,
- Perhaps most important, the tuning recipe we have employed so successfully with PI and PID algorithms can be directly applied to a PID with CO filter architecture.

PID Plus External Filter

As shown above, the filter computation is performed after the PID controller has computed the CO. Below, the PID controller output is computed using the non-interacting, dependent, ideal form, but any of the popular algorithms can be used with this "PID plus filter" architecture:

$$CO = CO_{bias} + Kc\,e(t) + \frac{Kc}{\tau i}\int e(t)dt + Kc\,\tau d\frac{de(t)}{dt}$$

Where:

CO = controller output signal (the wire out)
CO_{bias} = controller bias; set by bumpless transfer
e(t) = current controller error, defined as SP – PV
SP = set point
PV = measured process variable (the wire in)
Kc = controller gain, a tuning parameter
τi = reset time, a tuning parameter
τd = derivative time, a tuning parameter

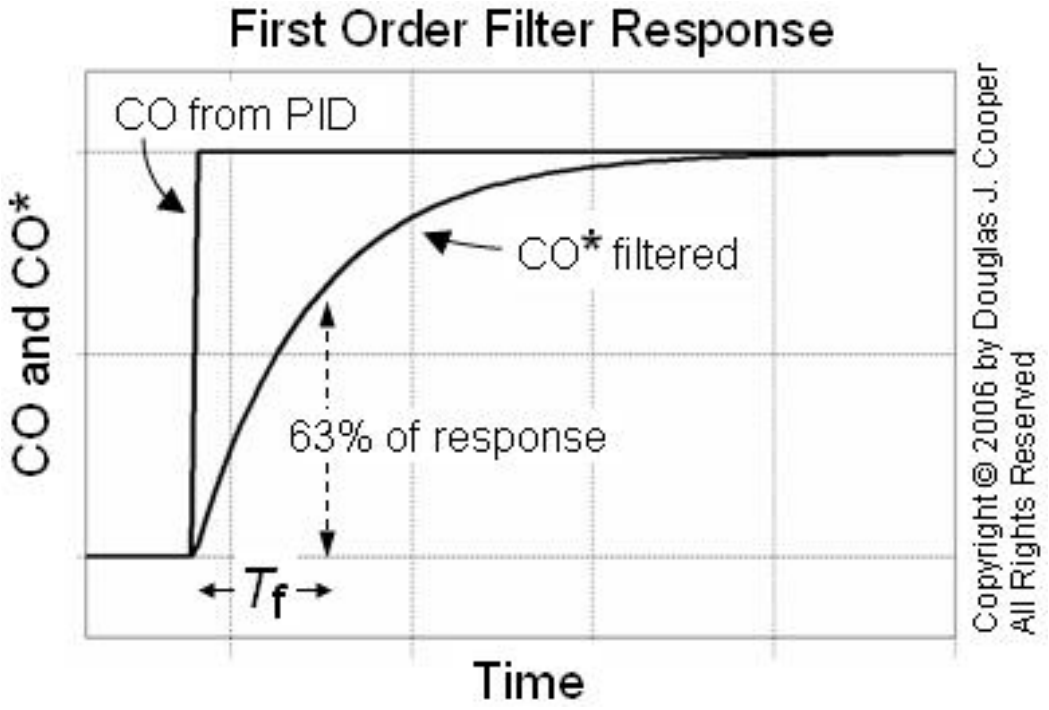
The Filtering Algorithm
A first order filter yields a smoothed CO* value as:

$$T_f \frac{dCO^*}{dt} + CO^* = CO$$

Where:
CO = raw PID controller output signal
CO* = filtered CO signal sent to FCE (e.g., valve)
T_f = filter time constant, a tuning parameter

A comparison of the first order filter above to a general first order plus dead time (FOPDT) model form reveals that:

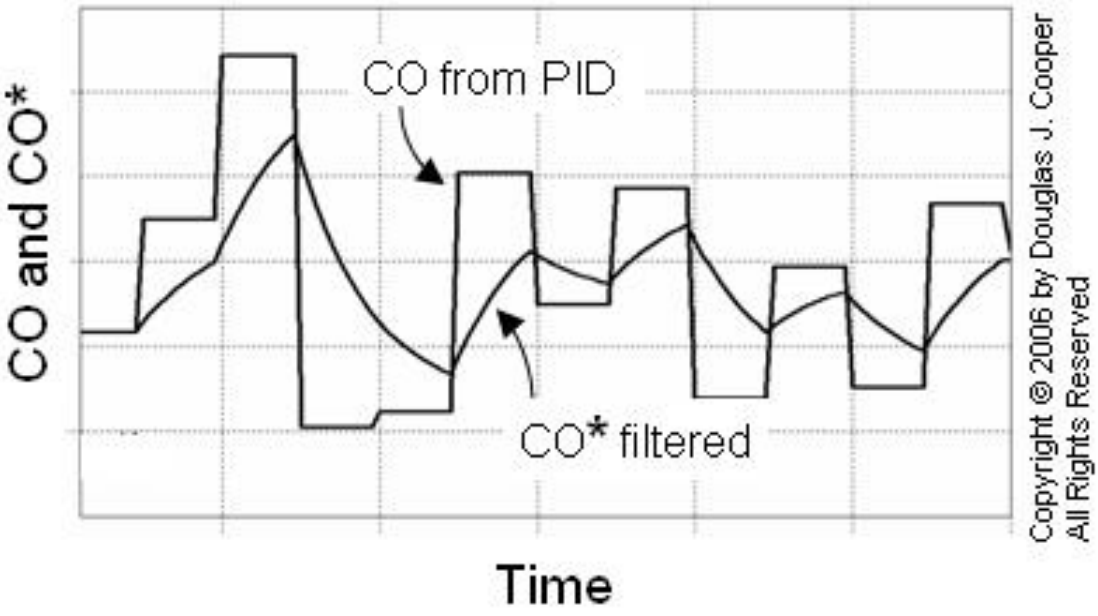
- The gain (or scaling factor) of the filter is one. That is, the filtered CO* has the same zero, span and units as the CO value from the PID algorithm.
- There is no dead time built into the filter. The CO* forwarded to the FCE is computed immediately after the PID algorithm yields the raw (unfiltered) CO signal.
- The degree of filtering, or how quickly CO* moves toward the unfiltered CO value, is set by T_f , the filter time constant.



Filtering Adds Delay
The degree of smoothing depends on the size of the filter time constant, T_f :

- A smaller T_f means CO* moves quickly and follows closer to changes in CO, so there is little filtering or smoothing of the signal.
- A larger T_f means CO* responds more slowly to changes in CO, so the filtering or smoothing is greater.

Shown below is a series of CO signals from a PID controller. Also shown is the filtered CO* trace using the same T_f as in the plot above.



As smoothing (or filter time, T_f) increases, the filtered signal may become more visually appealing, but more filtering means additional information delay in the control loop computation.

As delay increases in a control loop, the best achievable control performance decreases. The design challenge is to find the careful balance between signal smoothing and information delay to achieve the controller performance we desire for our process.

Combining Controller Plus Filter

To use our tuning recipe, we must first combine the controller and filter into a single unified equation. Since both equations above have CO isolated on one side of the equal sign, we can set the two equations equal to yield:

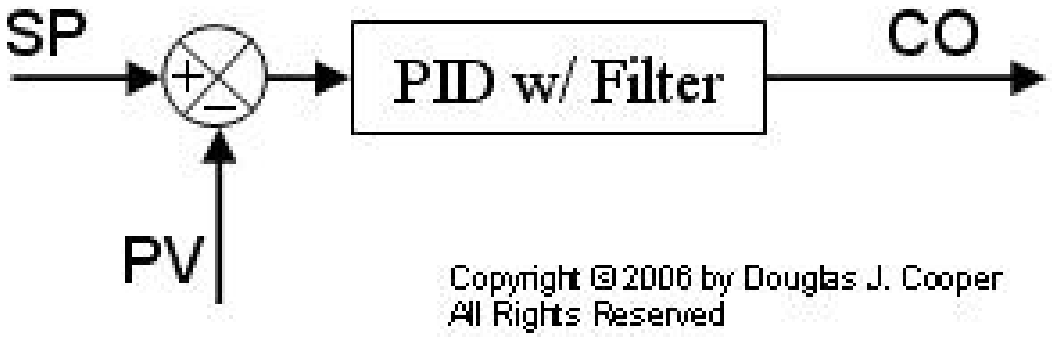
$$T_f \frac{dCO^*}{dt} + CO^* = CO_{bias} + K_c e(t) + \frac{K_c}{T_i}\int e(t)dt + K_c T_d \frac{de(t)}{dt}$$

Moving the left-most term to the right-hand side produces the unified PID with Filter equation:

$$CO^* = CO_{bias} + K_c e(t) + \frac{K_c}{T_i}\int e(t)dt + K_c T_d \frac{de(t)}{dt} - T_f \frac{dCO^*}{dt}$$

Unified PID With Filter Form

For design and tuning purposes going forward, we will use the unified PID with Filter equation form, as if it were represented by the schematic:



As implied by this diagram, we will drop the CO versus CO* nomenclature and simply write the unified PID with Filter equation as:

$$CO = CO_{bias} + Kc\,e(t) + \frac{Kc}{Ti}\int e(t)dt + Kc\,Td\frac{de(t)}{dt} - Tf\frac{dCO}{dt}$$

Please be aware that this is identical in every way to the PID with external filter algorithm derived earlier in this article. We recast it only so the tuning correlations are consistent in appearance and application with those of the PI and PID forms presented in earlier articles.

Filter Time Constant
Many commercial controllers that include some form of a PID with Filter algorithm cast the filter time constant as a fraction of the controller derivative time, or:

$$Tf = \alpha\,Td$$

The unified PID with Filter algorithm then becomes:

$$CO = CO_{bias} + Kc\,e(t) + \frac{Kc}{Ti}\int e(t)dt + Kc\,Td\frac{de(t)}{dt} - \alpha\,Td\frac{dCO}{dt}$$

We will use this αTd form in the example that follows. If your controller output filter uses T_f, the conversion is computed: $T_f = \alpha Td$.

Discrete-Time Implementation
While a CO filter offers potential for benefit in loops with noise and/or delicate mechanical FCEs, if our vendor does not offer the option, we must program the filter ourselves. Fortunately, the code is straightforward.

As above, the first order filtering algorithm is expressed:

$$Tf\frac{dCO^*}{dt} + CO^* = CO$$

In discrete-time form, we write:

$$T_f \frac{(CO^*new - CO^*old)}{T} + CO^*old = CO$$

or

$$CO^*new = CO^*old + (T/T_f)(CO - CO^*old)$$

Where T is the loop sample time and CO is the unfiltered PID signal.

In a computer program, the "new" CO* is computed directly from the "old" CO* value at each loop, so we do not need to keep track of these labels. The filter computation can be programmed in one line as:

$$COstar = COstar + (T/T_f)(CO - COstar)$$

Applying the CO Filter
We explore an example of PID control with CO filtering applied to the heat exchanger and jacketed stirred reactor process.

PID with CO Filter Control of the Heat Exchanger

The same tuning recipe we successfully demonstrated for PI control and PID control design and tuning can be used when a controller output (CO) filter is added to the heat exchanger process control loop.

Here we explore PID with CO Filter control using the unified (controller with internal filter) form. We showed in this previous article that the unified form is identical to a PID with external first-order CO filter implementation. Hence, the methods we use and observations we make apply equally to both internal and external filter architectures.

We follow the same four-step design and tuning recipe used for all control implementations.

Steps 1-3 of the PID with CO Filter design are identical to our previous PI and PID control case studies. The details for steps 1-3, stated below as summary conclusions, are presented with discussion in the PI control study.

Step 1: Design Level of Operation (DLO)
- Design PV and SP = 138 °C with operation ranging from 138 to 140 °C
- Expected warm liquid flow disturbance = 10 L/min

Step 2: Collect Data at the DLO
See the PI control article referenced above for a summary, or go to this article to see details of the data collection experiment.

Step 3: Fit an FOPDT Model to the Dynamic Data
The first order plus dead time (FOPDT) model approximation of the heat exchanger data from step 2 is:

- Process gain (how far), Kp = –0.53 °C/%
- Time constant (how fast), τp = 1.3 min
- Dead time (how much delay), Θp = 0.8 min

Step 4: Use the Parameters to Complete the Design
Vendors market the PID algorithm in a number of different forms, creating a confusing array of choices for the practitioner. The addition of a CO filter makes a bad situation worse. A filter adds another adjustable (tuning) parameter and significantly increases the number of possible algorithm forms.

Trial and error tuning of a four mode (four tuning parameter) controller with filter while our process is making product is a sure path to waste and expense. With so many interacting variables, we will likely settle for an operation that "isn't horrible" rather than a performance that is near optimal.

The dilemma is real and our tuning recipe is the answer. Yet for success in this final step, it is critical that we match our algorithm with its proper tuning correlations.

The way to do this reliably is with loop tuning software. In fact, a good package will help with all of the steps, from data collection and model fitting through vendor algorithm selection and final performance analysis. When our task list includes maintaining and tuning loops during production, commercial software will pay for itself in days.

• *Sample Time and Bumpless Transfer*
As explained in the PI control study, best practice is to set loop sample time T ≤ 0.1τp (10 times per time constant or faster). For this example, T = 1.0 sec.

Also, like most commercial controllers, we employ bumpless transfer. Thus, when switching to automatic, SP is set equal to the current PV and CO_{bias} is set equal to the current CO.

• *Controller Action*
Controller gain is negative for the heat exchanger, yet most commercial controllers require that a positive value of Kc be entered (more discussion here). The way we indicate a negative sign is to choose the direct acting option during implementation. If the wrong control action is entered, the controller will quickly drive the final control element to full on/open or full off/closed and remain there until a proper control action entry is made.

• *Specify Desired Performance*
We use the industry-proven Internal Model Control (IMC) tuning correlations in this study. IMC correlations employ a closed loop time constant, τc, that describes the desired speed or quickness of our controller in responding to a set point change or rejecting a disturbance. We must decide whether we seek:

- An *aggressive* controller with a rapid response and some overshoot:
  τc is the larger of $0.1 \cdot \tau p$ or $0.8 \cdot \Theta p$

- A *moderate* controller that will move the PV reasonably fast yet produce little to no overshoot in a set point response:
  T_c is the larger of $1 \cdot T_p$ or $8 \cdot \Theta_p$
- A *conservative* controller that will move the PV in the proper direction, but quite slowly:
  T_c is the larger of $10 \cdot T_p$ or $80 \cdot \Theta_p$

Ideal PID With Filter Example

A previous article presents details of how to combine an external first-order filter into a unified dependent, ideal, non-interacting PID with internal filter form:

$$CO = CO_{bias} + K_c e(t) + \frac{K_c}{T_i}\int e(t)dt + K_c T_d \frac{de(t)}{dt} - \alpha T_d \frac{dCO}{dt}$$

If our vendor offers the option, the preferred algorithm in industrial practice is PID with derivative on measurement (derivative on PV).

While a CO filter can largely address derivative kick, the filter term must be made larger than otherwise necessary to do so. Thus, there remains a performance benefit to derivative on measurement even when using a CO filter.

$$CO = CO_{bias} + K_c e(t) + \frac{K_c}{T_i}\int e(t)dt - K_c T_d \frac{dPV}{dt} - \alpha T_d \frac{dCO}{dt}$$

The IMC tuning correlations for either of the above PID with CO Filter forms are:

K_c	T_i	T_d	α
$\frac{1}{K_p}\left(\frac{T_p + 0.5\theta_p}{T_c + \theta_p}\right)$	$T_p + 0.5\theta_p$	$\frac{T_p \theta_p}{2T_p + \theta_p}$	$\frac{T_c(T_p + 0.5\theta_p)}{T_p(T_c + \theta_p)}$

We start our study by choosing an aggressive response tuning:

Aggressive T_c = the larger of $0.1 \cdot T_p$ or $0.8 \cdot \Theta_p$
= larger of 0.1(1.3 min) or 0.8(0.8 min)
= 0.64 min

Using this T_c and our K_p, T_p and Θ_p from Step 3 in the tuning correlations above yields the aggressive PID w/ Filter tuning values below. Also listed are the PID Ideal and PI controller tuning values from earlier studies:

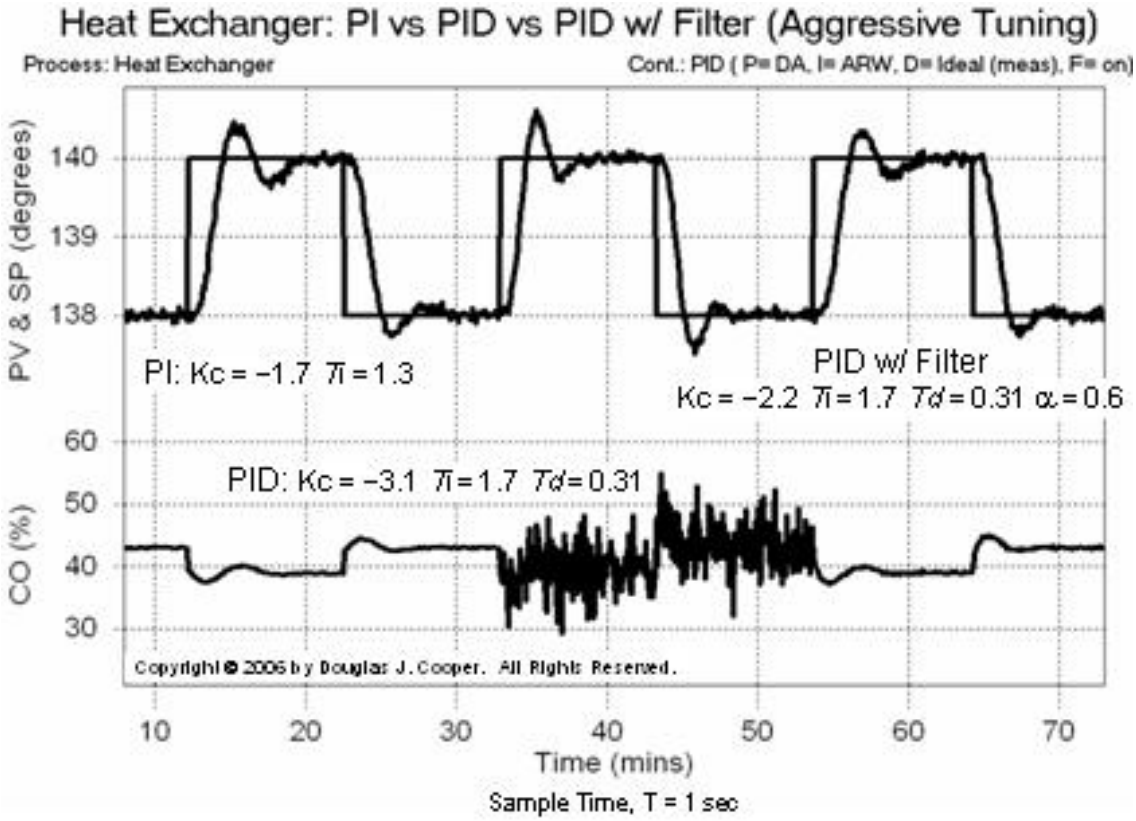
PID w/ Filter:
K_c = –2.2 %/°C T_i = 1.7 min T_d = 0.31 min □= 0.6

PID:
K_c = –3.1 %/°C T_i = 1.7 min T_d = 0.31 min

PI:

Kc = –1.7 %/°C T_i = 1.3 min

Below we compare the performance of these three aggressively tuned controllers side-by-side for the heat exchanger process simulation. Shown are three set point step pairs from 138 °C up to 140 °C and back again. Though not shown, the disturbance flow rate remains constant at 10 L/min throughout the study.



To the left is set point tracking performance for the PI controller. The middle set point steps show the performance of the PID controller. Derivative action enables a slightly faster rise time and settling time, but the derivative action causes the noise in the PV signal to be amplified and reflected as "chatter" in the control output signal.

To the right is the set point tracking performance of the PID w/ CO Filter controller. Indeed, the filter does an impressive job of cleaning up the chatter in the controller output signal without degrading performance.

In truth, however, the four tuning parameter PID w/ Filter performs similar to the two tuning parameter PI controller.

Tune One, Tune Them All

To complete this study, we compare the dependent, ideal, non-interacting form above to the performance of the dependent, interacting form:

$$CO = CO_{bias} + Kc\left(1 + \frac{Td}{Ti}\right)e(t) + \frac{Kc}{Ti}\int e(t)dt - KcTd\frac{dPV}{dt} - \alpha Td\frac{dCO}{dt}$$

The tuning correlations for the dependent, interacting form are:

Kc	T_i	T_d	α
$\frac{1}{K_p}\left(\frac{T_p}{T_c+\theta_p}\right)$	T_p	$0.5\,\theta_p$	$\frac{T_c}{T_c+\theta_p}$

For variety, we choose moderate response tuning for this comparison:

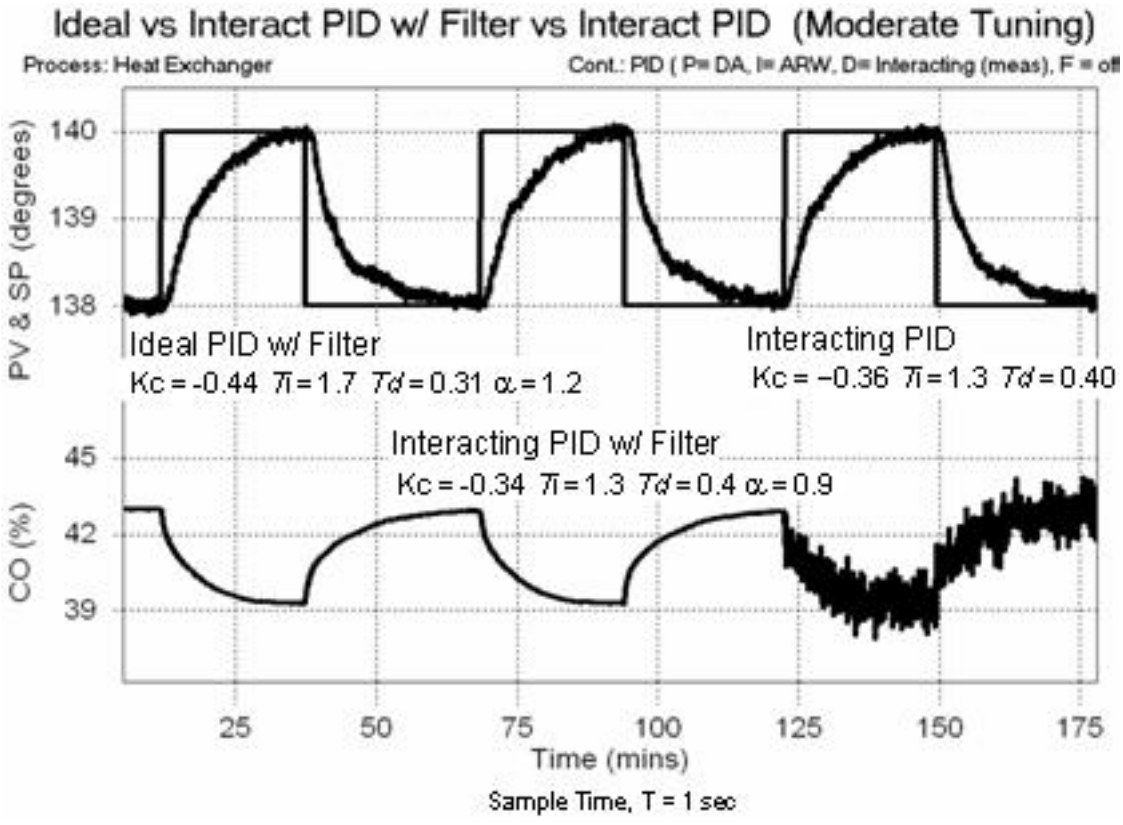
Moderate T_c = the larger of $1\cdot T_p$ or $8\cdot\Theta_p$
= larger of 1(1.3 min) or 8(0.8 min)
= 6.4 min

Using this T_c and our model parameters in the proper tuning correlations, we arrive at these moderate tuning values:

Dependent, Interacting:
Kc = –0.34 %/°C T_i = 1.3 min T_d = 0.40 min □= 0.9

Dependent, Ideal:
Kc = –0.44 %/°C T_i = 1.7 min T_d = 0.31 min □= 1.2

As shown in the plot below (), we see that moderate tuning provides a reasonably fast PV response while producing no overshoot. But more important, we establish that the interacting form and the ideal form provide identical performance when tuned with their own correlations.



The third set point step shows the performance of a straight PID with no filter. This

reinforces the benefits of a CO filter if derivative action is being contemplated.

Observations
Our study of the heat exchanger process has shown that PID controllers provide minor performance improvements over PI controllers. Yet derivative action causes noise in the PV to be reflected as chatter in the CO signal, and this counterbalances the small benefits of the derivative term.

In this article, we explore CO filters and learn that they can correct the chatter problem. But now we have elevated a difficult two tuning parameter PI controller design into an extremely challenging four parameter PID w/ Filter controller design. And at best, this extra effort still provides only modest performance benefits.

Unless the economic impact of a loop is substantial, many practitioners conclude that the PI controller is the best choice. It is faster to implement, easier to maintain, and provides performance approaching that of the PID w/ Filter controller.

For the heat exchanger case study, this conclusion appears to be reasonable. Return to the Table of Contents to learn more.

PID with CO Filter Disturbance Rejection in the Jacketed Stirred Reactor

The control objective for the jacketed stirred reactor process is to minimize the impact on reactor operation when the temperature of the liquid entering the cooling jacket changes. We have previously established the performance capabilities of a PI controller in rejecting the impact of this disturbance.

Here we explore the performance of a PID with controller output (CO) filter algorithm in meeting this same disturbance rejection objective. We use the unified PID with CO filter controller in this study. As detailed in a prior article, the unified form is identical to a PID with external first-order CO filter implementation. Thus, the methods and observations from this investigation apply equally to both controller architectures.

The important variables for the jacketed reactor are (view a process graphic):
CO = signal to valve that adjusts cooling jacket liquid flow rate (controller output, %)
PV = reactor exit stream temperature (measured process variable, ^{o}C)
SP = desired reactor exit stream temperature (set point, ^{o}C)
D = temperature of cooling liquid entering the jacket (major disturbance, ^{o}C)

We follow our industry proven recipe to design and tune our PID with CO filter controller. Recall that steps 1-3 of a design remain the same regardless of the controller used. For this process and objective, the results of steps 1-3 are summarized from previous investigations:

Step 1: Design Level of Operation (DLO)
DLO details are presented in this article and are summarized:
- Design PV and SP = 90 ^{o}C with approval for brief dynamic (bump) testing of ±2 ^{o}C

- Design D = 43 ^{o}C with occasional spikes up to 50 ^{o}C

Step 2: Collect Process Data around the DLO
When CO, PV and D are steady near the design level of operation, we bump the jacketed stirred reactor to generate CO-to-PV cause and effect process response data.

Step 3: Fit a FOPDT Model to the Dynamic Process Data
We approximate the dynamic behavior of the process by fitting a first order plus dead time (FOPDT) dynamic model to the test data from step 2. The results of the modeling study are summarized:

- process gain (direction and how far), Kp = – 0.5 ^{o}C/%
- time constant (how fast), Tp = 2.2 min
- dead time (how much delay), Θp = 0.8 min

Step 4: Use the FOPDT Parameters to Complete the Design

• *Algorithm Form*
A PID with CO filter controller, regardless of whether the filter is internal or external, presents us with a "four adjustable tuning parameter" challenge. As more parameters are included in our controller, the array of vendor algorithm forms increases.

The various controller forms are all capable of delivering a similar, predictable performance, as long as we match our algorithm with its proper tuning correlations. Certainly, a "guess and test" approach to tuning a four-mode controller while our process is making product is a sure path to wasting feedstock and utilities, creating safety and environmental concerns, and putting plant profitability at risk.

Modern loop tuning software will not only guide data analysis and model fitting, but will ensure our tuning matches our vendor's algorithm. Such software will even display expected final performance prior to implementation. If our task list includes maintaining and tuning control loops during production, such software will pay for itself in days.

• *Sample Time*
As discussed here, best practice is to set loop sample time to T ≤ 0.1Tp (10 times per time constant or faster). We meet this design criterion with the widely-available vendor option of T = 1.0 sec.

• *Controller Action*
Kp, and thus Kc, are negative for our jacketed stirred reactor process. Most commercial controllers have us specify a negative Kc by entering a positive value into the controller and then choosing the "direct acting" option (more discussion in this article). If the wrong control action is entered, the controller will drive the final control element to full on/open or full off/closed and remain there until a proper control action entry is made.

• *Specify Desired Performance*
We use the industry-proven Internal Model Control (IMC) tuning correlations in this study. IMC correlations employ a closed loop time constant, Tc, that describes the desired speed or quickness of our controller in responding to a set point change or rejecting a

disturbance.

Our PI control study describes what to expect from an aggressive, moderate or conservative controller. Once our desired performance is chosen, the closed loop time constant is computed:

- aggressive performance: T_c is the larger of $0.1 \cdot T_p$ or $0.8 \cdot \Theta_p$
- moderate performance: T_c is the larger of $1 \cdot T_p$ or $8 \cdot \Theta_p$
- conservative performance: T_c is the larger of $10 \cdot T_p$ or $80 \cdot \Theta_p$

• *The Tuning Correlations*

A previous article presents details of how to combine an external first-order filter into the unified ideal PID with internal filter form:

$$CO = CO_{bias} + K_c e(t) + \frac{K_c}{T_i}\int e(t)dt + K_c T_d \frac{de(t)}{dt} - \alpha T_d \frac{dCO}{dt}$$

If our vendor offers the option, the preferred algorithm in industrial practice is PID with derivative on measurement, PV:

$$CO = CO_{bias} + K_c e(t) + \frac{K_c}{T_i}\int e(t)dt - K_c T_d \frac{dPV}{dt} - \alpha T_d \frac{dCO}{dt}$$

The IMC tuning correlations for either of the above PID with CO filter forms are the same and listed in the chart below. The chart also lists the tuning correlations as discussed in previous articles for the PI controller and ideal PID controller forms:

	Controller Gain K_c	Reset Time T_i	Deriv Time T_d	Filter Const α
PI	$\frac{1}{K_p}\frac{T_p}{(\theta_p + T_c)}$	T_p		
Ideal PID	$\frac{1}{K_p}\left(\frac{T_p + 0.5\theta_p}{T_c + 0.5\theta_p}\right)$	$T_p + 0.5\theta_p$	$\frac{T_p \theta_p}{2T_p + \theta_p}$	
PID w/ CO Filter	$\frac{1}{K_p}\left(\frac{T_p + 0.5\theta_p}{T_c + \theta_p}\right)$	$T_p + 0.5\theta_p$	$\frac{T_p \theta_p}{2T_p + \theta_p}$	$\frac{T_c(T_p + 0.5\theta_p)}{T_p(T_c + \theta_p)}$

PID With CO Filter Disturbance Rejection Study

In the plots below, we compare the performance of the PID with CO filter controller side-

be-side with that of the PI controller and the ideal (unfiltered) PID controller.

Our objective is rejecting the the impact on reactor operation when the temperature of cooling liquid entering the reactor jacket changes. We test both moderate and aggressive response tuning for the three controllers.

a) *Moderate Response Tuning:*
For a controller that will move the PV reasonably fast while producing little to no overshoot, choose:

Moderate T_c = the larger of $1 \cdot T_p$ or $8 \cdot \Theta_p$
= larger of 1(2.2 min) or 8(0.8 min)
= 6.4 min

Using this T_c and the Kp, T_p and Θ_p values listed in step 3 at the top of this article, the moderate IMC tuning values are:

PI: Kc = –0.61 %/°C T_i = 2.2 min

PID: Kc = –0.77 %/°C T_i = 2.6 min T_d = 0.34 min

PID w/ Filter: Kc = –0.72 %/°C T_i = 2.6 min T_d = 0.34 min □= 1.1

b) *Aggressive Response Tuning:*
For an active or quickly responding controller where we can tolerate some overshoot and oscillation as the PV settles out, specify:

Aggressive T_c = the larger of $0.1 \cdot T_p$ or $0.8 \cdot \Theta_p$
= larger of 0.1(2.2 min) or 0.8(0.8 min)
= 0.64 min

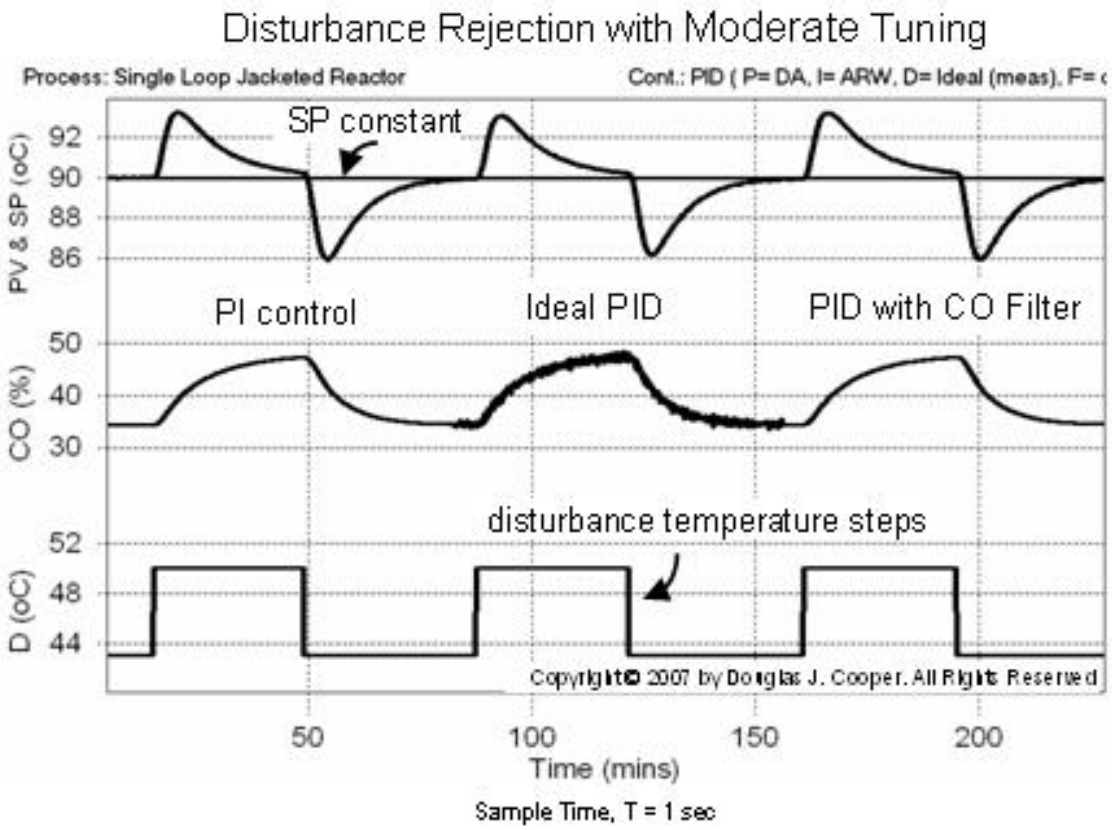
and the aggressive IMC tuning values are:

PI: Kc = –3.1 %/°C T_i = 2.2 min

PID: Kc = –5.0 %/°C T_i = 2.6 min T_d = 0.34 min

PID w/ Filter: Kc = –3.6 %/°C T_i = 2.6 min T_d = 0.34 min □= 0.5

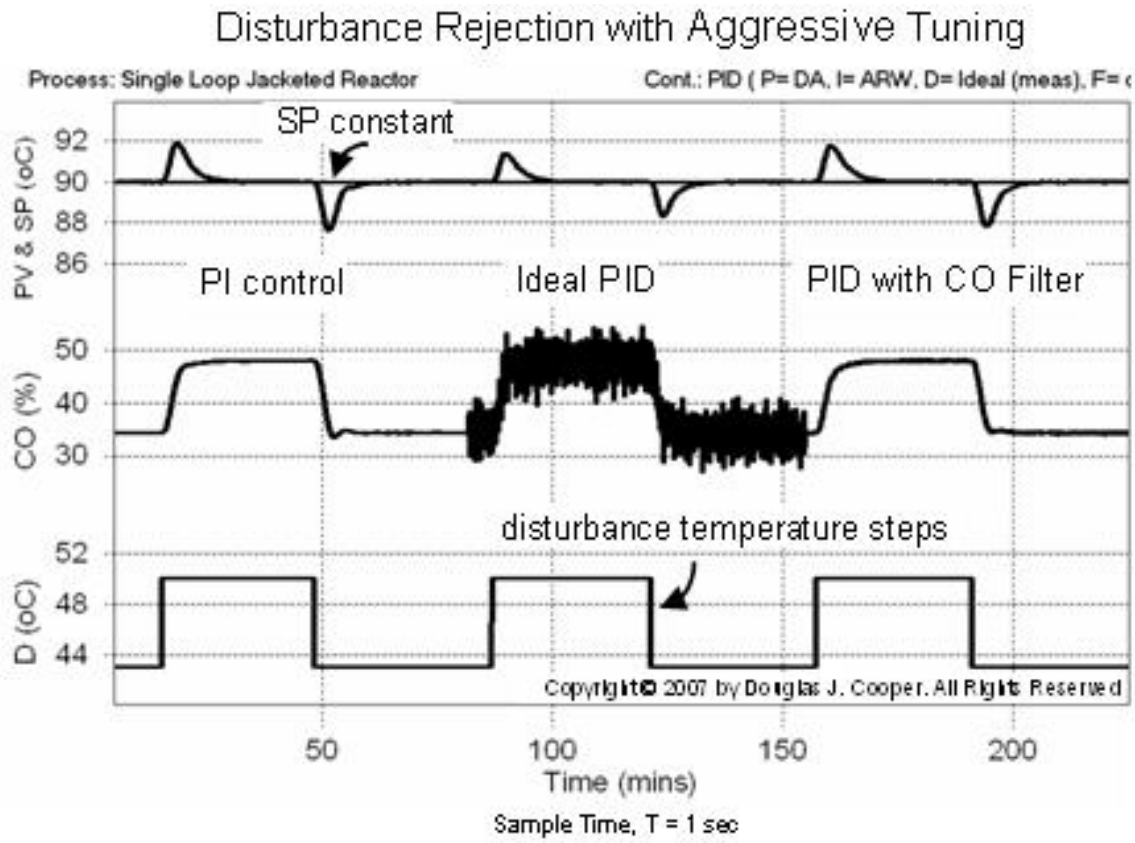
• *Implement and Test*
A comparison of the three controllers in rejecting a disturbance change in the cooling jacket inlet temperature, D, is shown below (). This plot shows controller performance when using the moderate tuning values computed above. Note that the set point remains constant at 90 ^{o}C throughout the study.



The PI controller performance is shown to the left in the plot above. The ideal PID performance is in the middle. The plot reveals that the benefit of derivative action is marginal at best. There is a clear penalty, however, in that derivative action causes the modest noise in the PV signal to be amplified and reflected as "chatter" in the CO signal.

To the right in the plot above is the performance of the PID with CO filter controller. The filter is effective in reducing the controller output chatter caused by the derivative action without degrading performance. In truth, however, the four tuning parameter PID with filter performs similar to the two tuning parameter PI controller.

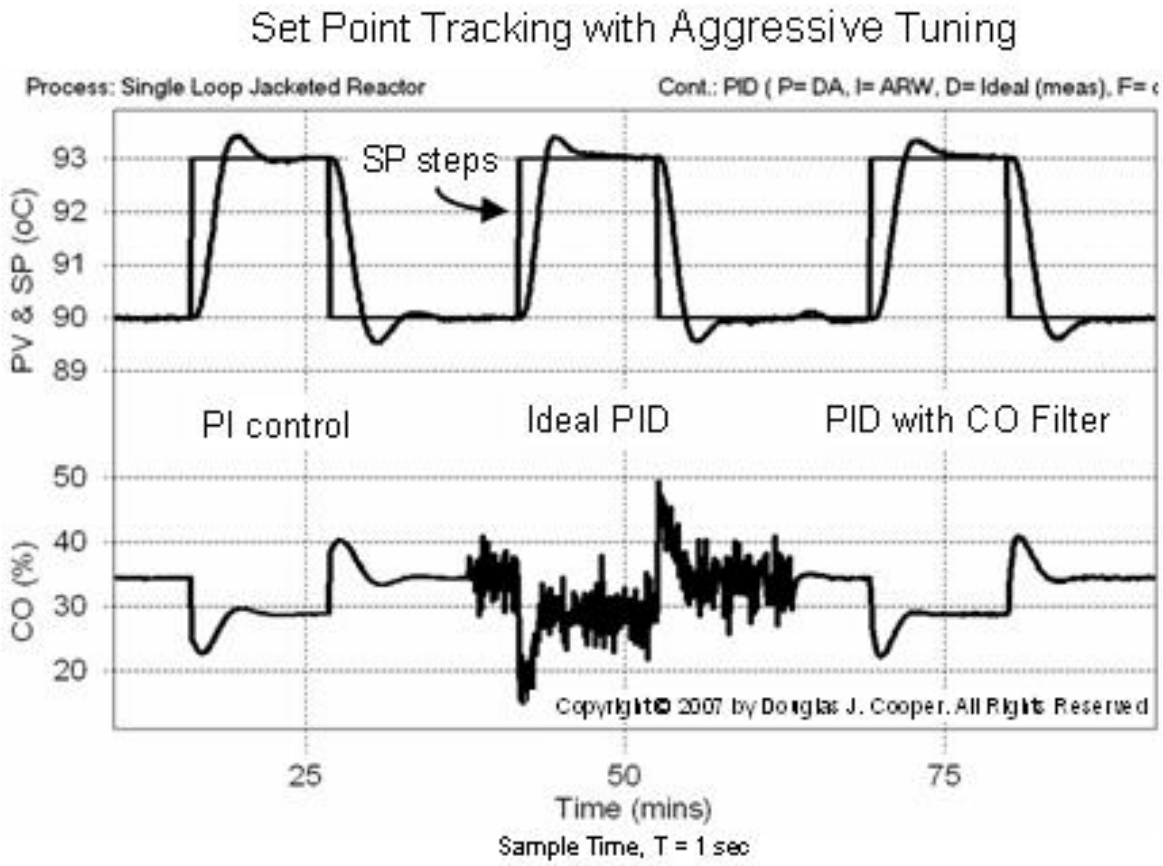
The disturbance rejection performance of the controllers when tuned for aggressive action is shown below (). Note that the axis scales for the plots both above and below are the same to permit a visual comparison.



The aggressive tuning provides a smaller maximum deviation from set point and a faster settling time relative to the moderate tuning performance. The only obvious difference is that as a PID controller (middle of plot) becomes more aggressive in its actions, the CO chatter grows as a problem and filtering solutions become increasingly beneficial.

But ultimately, just as with the moderate tuning case, the two mode (or two tuning parameter) PI controller compares favorably with the four mode PID with CO filter controller.

While not our design objective, presented below () is the set point tracking ability of the aggressively tuned controllers when the disturbance temperature is held constant:



The set point tracking response of the ideal PID controller is marginally better in that it shows a slightly shorter rise time, smaller overshoot and faster settling time. The CO chatter that comes as a price for these minor benefits will likely increase maintenance costs as our final control element (e.g., valve, pump or compressor) wears from this excessive activity.

The four mode PID with CO filter addresses the chatter, but it is not clear that the added complexity is worth the marginal performance benefits.

Thus, many practitioners conclude that the PI controller provides the best balance of complexity and performance. It is faster to implement, easier to maintain, and provides performance approaching that of the PID with CO filter controller.

III. Additional PID Design and Tuning Concepts (by Doug Cooper)

11) Exploring Deeper: Sample Time, Parameter Scheduling, Plant-Wide Control

Sample Time is a Fundamental Design and Tuning Specification

There are two sample times, T, used in process controller design and tuning.

One is the control loop sample time (step 4 of the design and tuning recipe) that specifies how often the controller samples the measured process variable (PV) and computes and transmits a new controller output (CO) signal.

The other is the rate at which CO and PV data are sampled and recorded during a bump test (step 2 of the recipe).

In both cases, sampling too slow will have a negative impact on performance. Sampling fast will not necessarily provide better performance, though it may lead us to spend more than necessary on high-end instrumentation and computing resources.

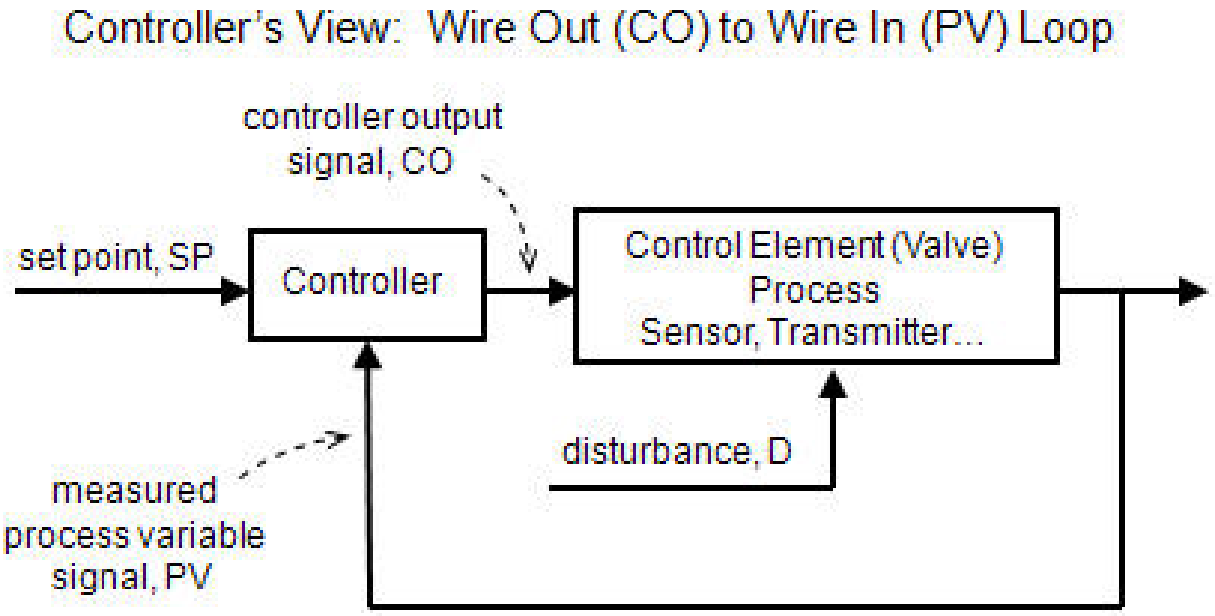
Fast and slow are relative terms defined by the process time constant, T_p. Best practice for both control loop sample time and bump test data collection are the same:

> *Best Practice*:
> Sample time should be 10 times per process time constant or faster ($T \leq 0.1T_p$).

In this article we explore both sample time issues. Specifically, we study:

1) The impact on performance when we adjust control loop sample time while keeping the tuning of a PI controller fixed, and
2) How performance is affected when we sample a process at different rates during a bump test and then complete the controller design and tuning using this same T.

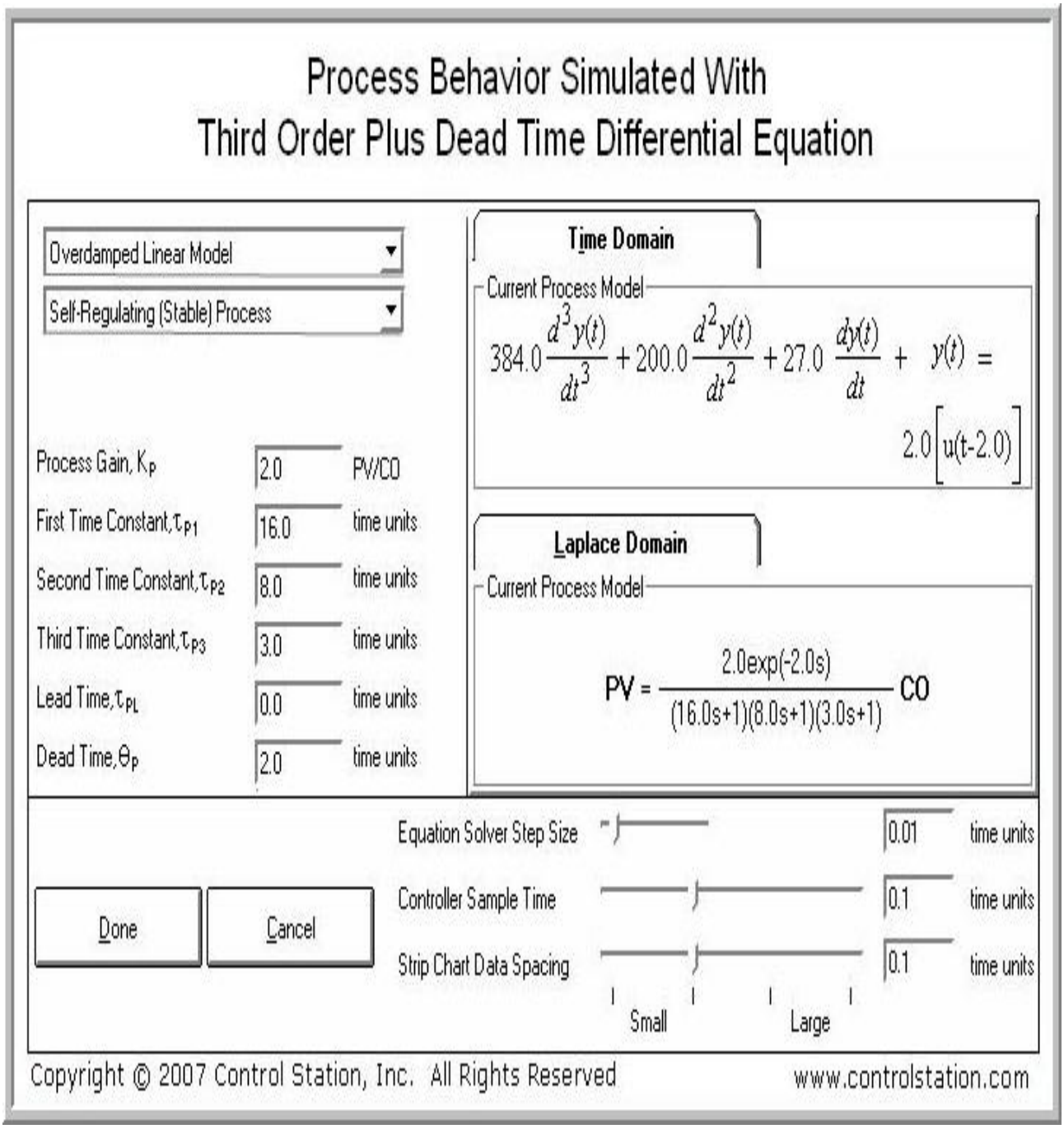
The Process
Like all articles on this site, the CO and PV data we consider are the wire out to wire in samples collected at the controller interface. Thus, as shown below, the equipment (e.g., actuator, valve, process unit, sensor, transmitter) and analog or digital manipulations (e.g., scaling, filtering, linearization) are all lumped as a single "process" that sits between the CO and PV values.



Copyright © 2007 by Douglas J. Cooper. All Rights Reserved

To provide the ability to manipulate and monitor all aspects of an orderly investigation, we use a differential equation (or transfer function) simulation utility to create the overall process.

It is not necessary to understand the simulation utility to appreciate and learn from the studies below. But for those interested, we provide a screen grab from the commercial software used for this purpose:



This same "process" is used in all examples in this article. Though expressed as a linear equation, it is sufficiently complex that the observations we make will be true for a broad range of process applications.

The Disturbance

Real processes have many disturbances that can disrupt operation and require corrective action from the controller. For the purposes of this investigation, we focus on only one generic disturbance (D) to our process.

The manner in which a PV responds to a disturbance is different for every application. Given this, here we choose a middle ground. Specifically, we specify that the impact of D on PV is exactly the same as the impact of CO on PV.

Thus, the D to PV behavior is simulated in the examples with the identical differential equation as shown above for the CO to PV behavior. This assumption is not right or wrong or good or bad. In fact, it is a rather common assumption in theoretical studies. Our goal is simply to provide a basis of comparison when we start exploring how sample time impacts controller performance.

The PI Controller
All examples in this article use the PI controller form:

$$CO = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{Ti}\int e(t)dt$$

where algorithm parameters are defined here.

The PI controller computes a CO action every loop sample time, T. To best highlight differences based on sample time issues, we choose an aggressive controller tuning as detailed in this example. Tuning parameters are thus computed based on an approximating first order plus dead time (FOPDT) model fit (step 3 of the recipe) as:

$$Kc = \frac{1}{Kp}\frac{Tp}{(\theta p + Tc)} \quad \text{and} \quad Ti = Tp$$

where: Tc is the larger of $0.1 \cdot Tp$ or $0.8 \cdot \Theta p$

Parameter Units
Time is expressed as generic "time units" and is not listed in the plots, calculations or tables. The conclusions we draw are independent of whether these are milliseconds, seconds, minutes or any other time units.

It is important to recognize, however, that when applying the observations from this article to other applications, we must be sure that all time-related parameters, including sample time, dead time, time constants and reset time, are expressed in consistent units. They should all be in seconds or all be in minutes, for example.

CO and PV both range from 0-100% in the examples. Process gain, Kp, thus has units of (% of PV)/(% of CO), while controller gain, Kc, units are (% of CO)/(% of PV).

For the remainder of this article, all parameters follow the above units convention. While something we normally avoid as bad practice, units are not explicitly displayed in the various plots and calculations.

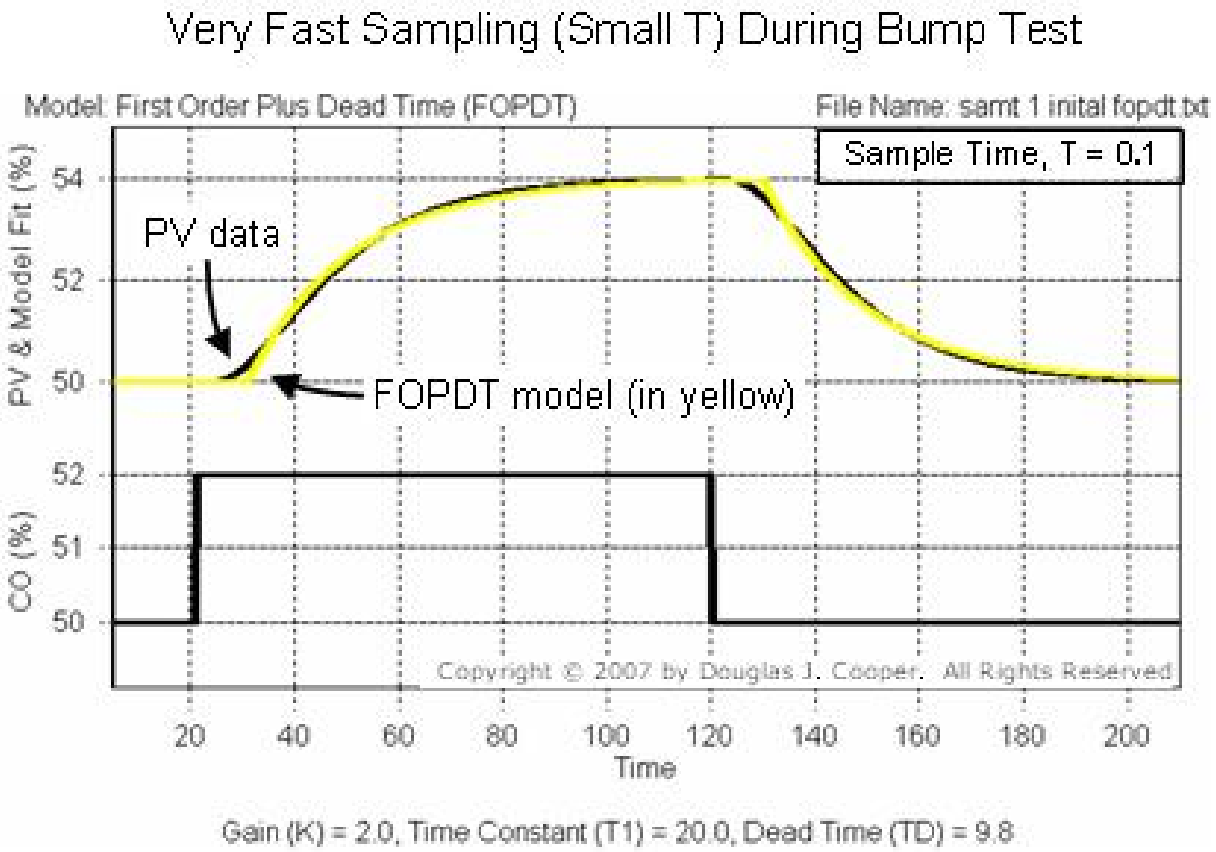
(1) Impact of Loop Sample Time When Controller Tuning is Constant
In this first study, we follow the recipe to design and tune a PI controller for the process above. Set point tracking performance is then explored as a function of control loop sample time, T.

• *Step 1: Establish the Design Level of Operation (DLO)*
We arbitrarily initialize CO, PV, SP and D all at 50% and choose this as the default DLO.

• *Step 2: Collect CO to PV bump test data around the DLO*
The plot in step 3 shows the bump test data used in this study. The sample time used

during data collection is T = 0.1, which we confirm in step 3 is a very fast T = 0.005*T*p.

• *Step 3: Approximate the Process Behavior with an FOPDT Dynamic Model*
We use commercial software to fit a first order plus dead time (FOPDT) model to the dynamic process data as shown below. This fit could also be performed by hand using a graphical analysis of the plot data following the methods detailed here.



The FOPDT model fit (in yellow) visually matches the PV data in the plot, so we accept the model as an appropriate approximation of the more complex dynamic process behavior. The model parameters are listed beneath the plot and are summarized here:

- Process gain (how far), Kp = 2.0
- Time constant (how fast), *T*p = 20
- Dead time (how much delay), Θp = 9.8

• *Step 4: Use the FOPDT Model Parameters to Complete the Design and Tuning*
Using the Kp, *T*p and Θp from step 3, we first compute the closed loop time constant, *T*c, for an aggressively tuned controller as:

Aggressive Tc = the larger of 0.1·Tp or 0.8·Θp
= larger of 0.1(20) or 0.8(9.8)
= 7.8

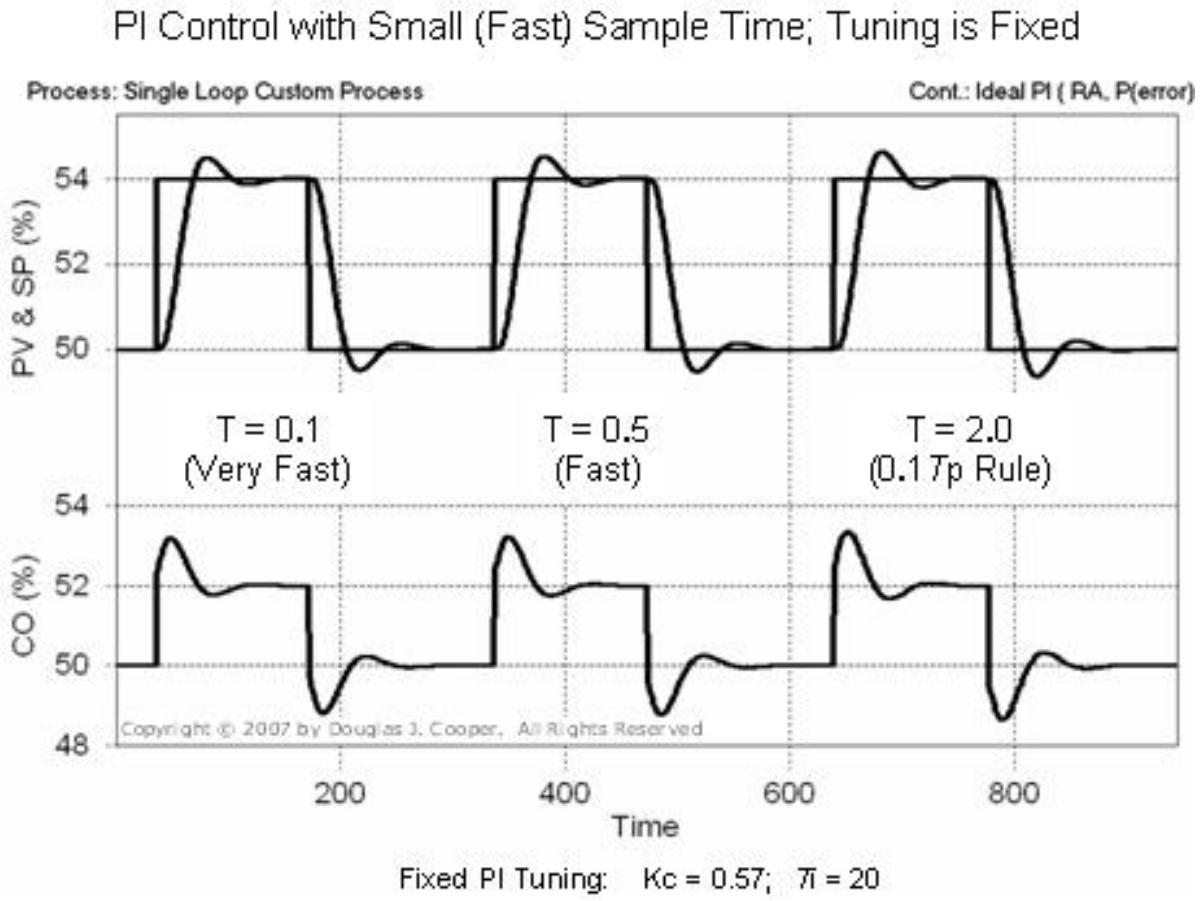
and then using the tuning correlations listed earlier, we compute PI tuning values:

$$Kc = \frac{1}{2.0}\frac{20}{(9.8 + 7.8)} = 0.57 \quad \text{and} \quad \mathit{Ti} = 20$$

We implement these Kc and *T*i values and hold them constant for the next three plots as we adjust control loop sample time, T.

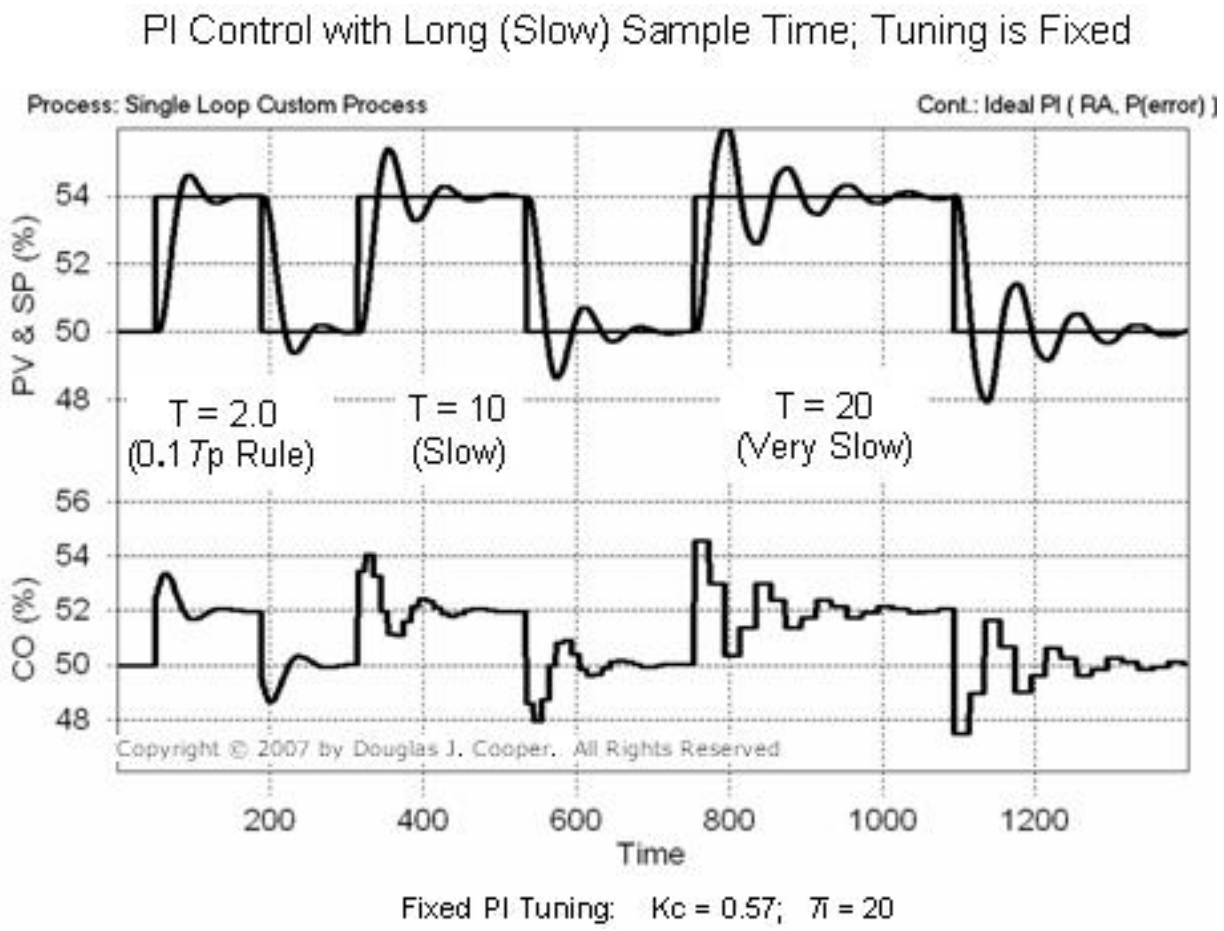
The "best practice" design rule at the top of this article states that sample time should be 10 times per process time constant or faster (T ≤ 0.1*T*p). Hence, the maximum sample time we should consider is: T_{max}= 0.1*T*p = 0.1(20) = 2.0

The plot below shows the set point tracking performance of the PI controller with fixed tuning when the sample time T is very fast (T = 0.1), fast (T = 0.5)and right at the maximum best practice value (T = 2.0).



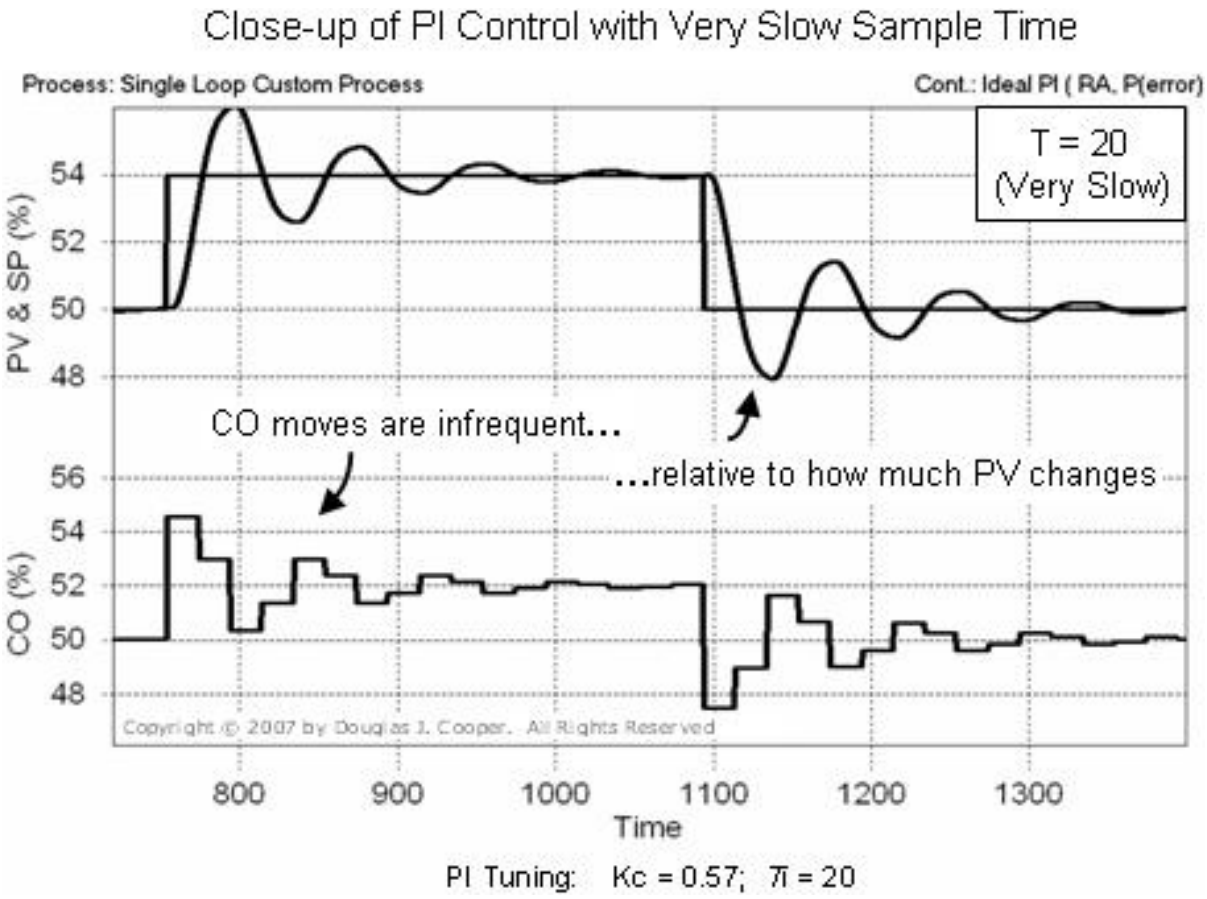
The plot above supports that a control loop sample time smaller (faster) than the "maximum best practice" 0.1*T*p rule has modest impact on performance.

In the plot below, we increase control loop sample time above the best practice limit. Because the scaling on the plot axes has changed from that above, we begin with the maximum best practice value of T = 0.1*T*p = 2.0, and then test performance when the loop sample time is slow (T = 10) and very slow (T = 20) relative to the rule.



Performance of a controller with fixed tuning clearly degrades as loop sample time, T, increases above the maximum best practice value of 0.1*T*p.

To better understand why, we zoom in on the control performance when loop sample time is very slow (T = 20). The plot below is a close-up from the plot above.



In the "very slow sample time" plot above, T = *T*p. That is, the controller measures and acts only once per process time constant.

We can see in the plot that PV moves a considerable amount between each corrective CO action. At this slow sample time, the controller simply cannot keep up with the action and the result is a degraded control performance.

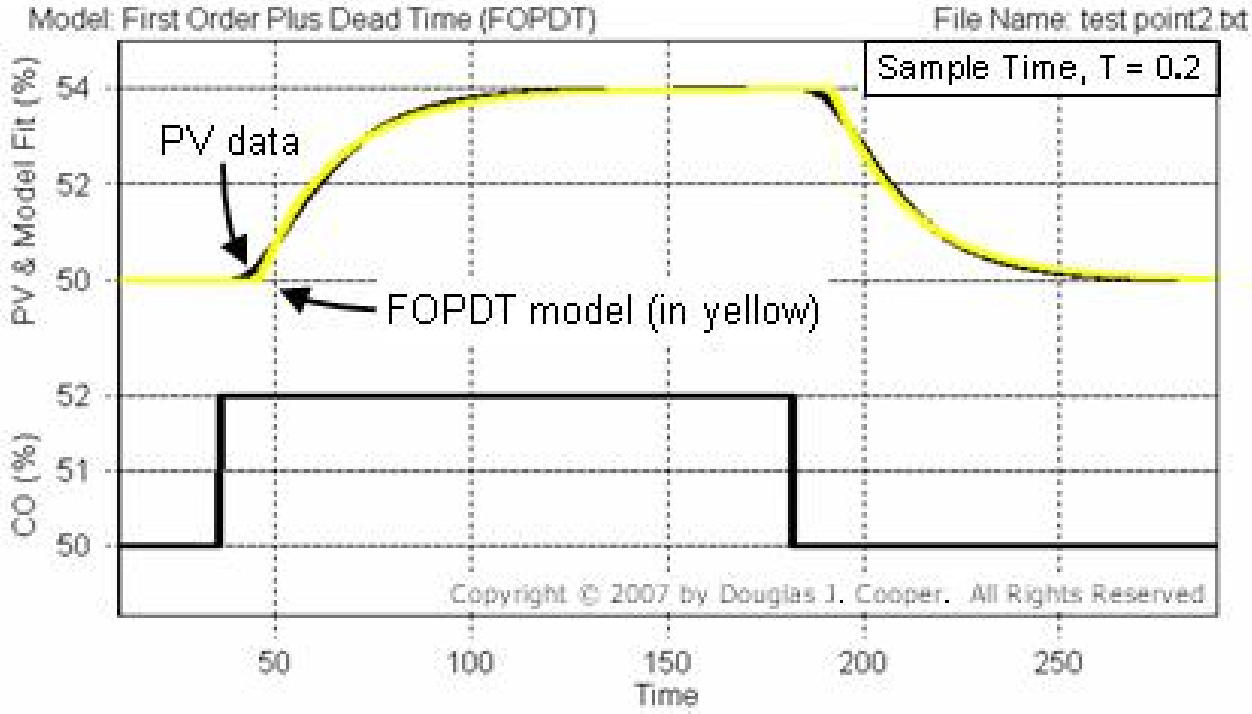
(2) Impact of Data Collection Sample Time During Bump Test
In this study we sample our process at different rates, T, during a bump test, complete the design and tuning, and then test the performance of the resulting controller using this same T as the control loop sample time. Following our recipe:

• *Step 1: Establish the Design Level of Operation (DLO)*
We again initialize CO, PV, SP and D all at 50% and choose this as the default DLO.

• *Step 2: Collect CO to PV bump test data around the DLO*
The plots in step 3 show identical bump tests. The only difference is the sample time used to collect and record data as the PV responds to the CO steps. We consider three cases:
- sample 10 times faster than rule: T = 0.01*T*p = 0.2
- sample at the maximum "best practice" rule: T = 0.1*T*p = 2
- sample 10 times slower than rule: T = *T*p = 20

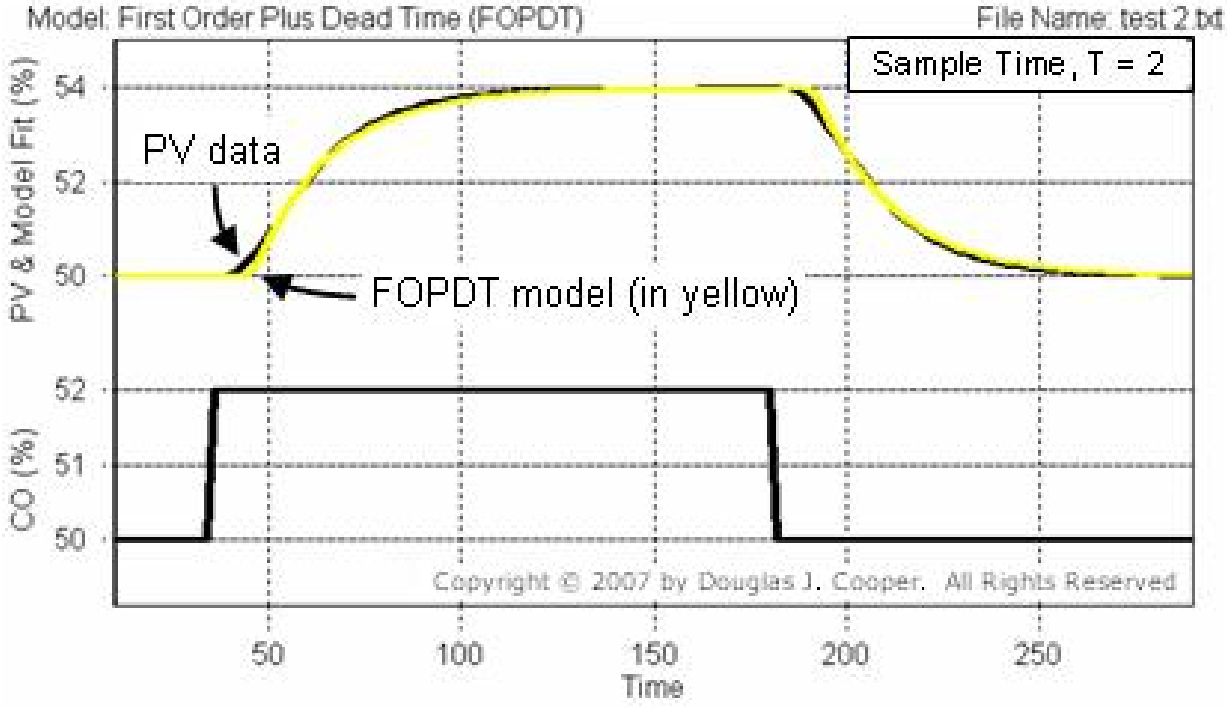
• *Step 3: Approximate the Process Behavior with an FOPDT Dynamic Model*
We again use Control Station software to fit a first order plus dead time model to each response plot (for a large view of the plots below, click T = 0.2; T = 2; T = 20)

Sampling 10 times **Faster** than T=0.1*T*p Rule during Bump Test

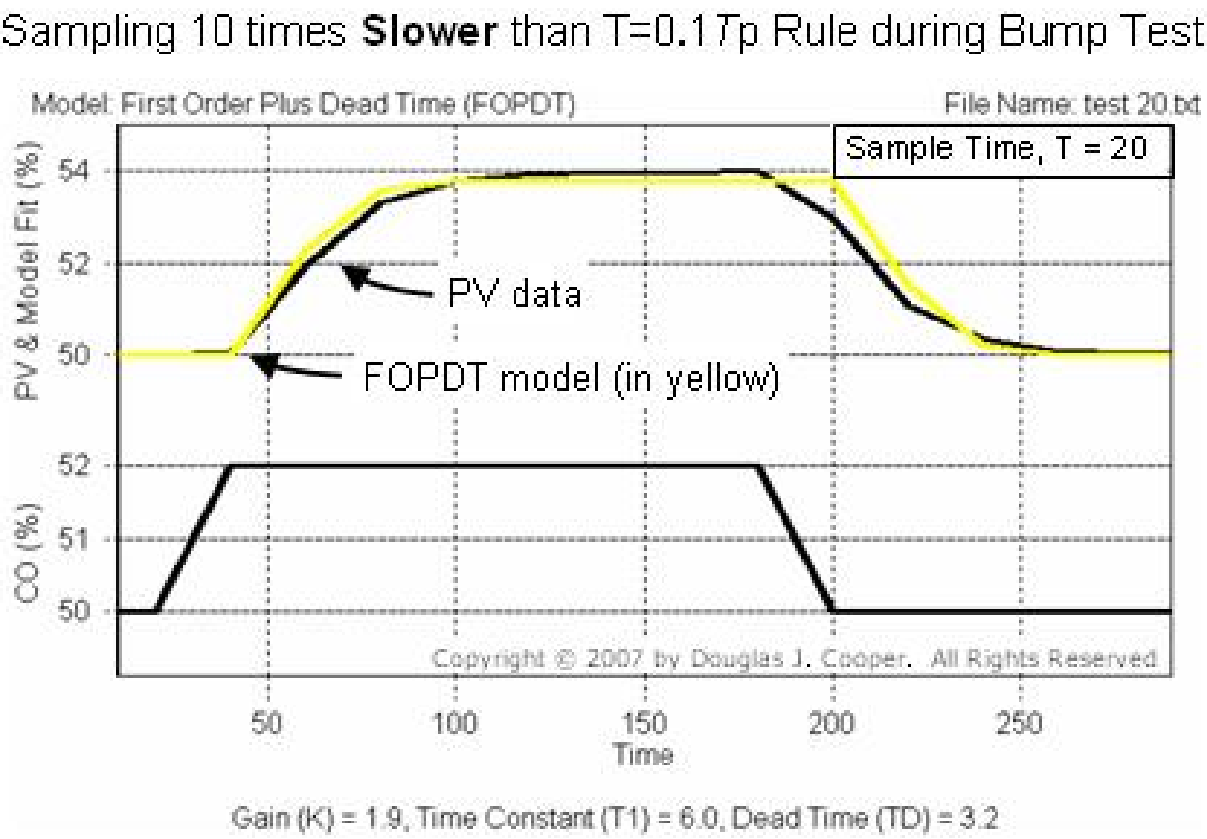


Gain (K) = 2.0, Time Constant (T1) = 20.0, Dead Time (TD) = 9.7

Sampling **at** T=0.1*T*p Rule during Bump Test



Gain (K) = 2.0, Time Constant (T1) = 20.0, Dead Time (TD) = 8.6



As the above plots reveal, an FOPDT model of the same process can be different if we sample and record data at different rates during a bump test.

Our best practice rule is based on the process time constant (*T*p), a value we may not even know until after we have conducted our data collection experiment. If we do not know what to expect from a process based on prior experience, we recommend a "faster is better" attitude when sampling and recording data during a bump test.

• *Step 4: Use the FOPDT Model Parameters to Complete the Design and Tuning*
We summarize the Kp, *T*p and Θp for the three cases in the table below. The first two columns ("sample fast" and "T = 0.1*T*p rule") show similar values. The "sample slow" column contains values that are clearly different.

FOPDT Model Values	Sample the **Same Process** At Different Rates During the Bump Test		
	Sample Fast T = 0.2	0.17p Rule T = 2	Sample Slow T = 20
Kp	2.0	2.0	1.9
*T*p	20	20	6.0
Θp	9.7	8.6	3.2
Closed Loop Time Constant (Aggressive) Rule: *T*c larger of 0.1·*T*p or 0.8·Θp			Use Rule: Θp,min = T
*T*c	7.8	6.9	16
Aggressive PI Tuning			
Kc	0.57	0.65	0.09
*T*i	20	20	6

Units: Kp [=] %PV/%CO; *T*p [=] time units; Θp [=] time units

Kc [=] %CO/%PV; *T*i [=] time units

The tuning values shown in the bottom two rows across the table are computed by substituting the FOPDT model values into the PI tuning correlations as discussed earlier in this article.

The "Θp,min = T" rule for controller tuning

We do confront one new tuning issue in this study that merits discussion.

Consider that all controllers measure, act, then wait until next sample time; measure, act, then wait until next sample time. This "measure, act, wait" procedure has a delay (or dead time) of one sample time built naturally into its structure.

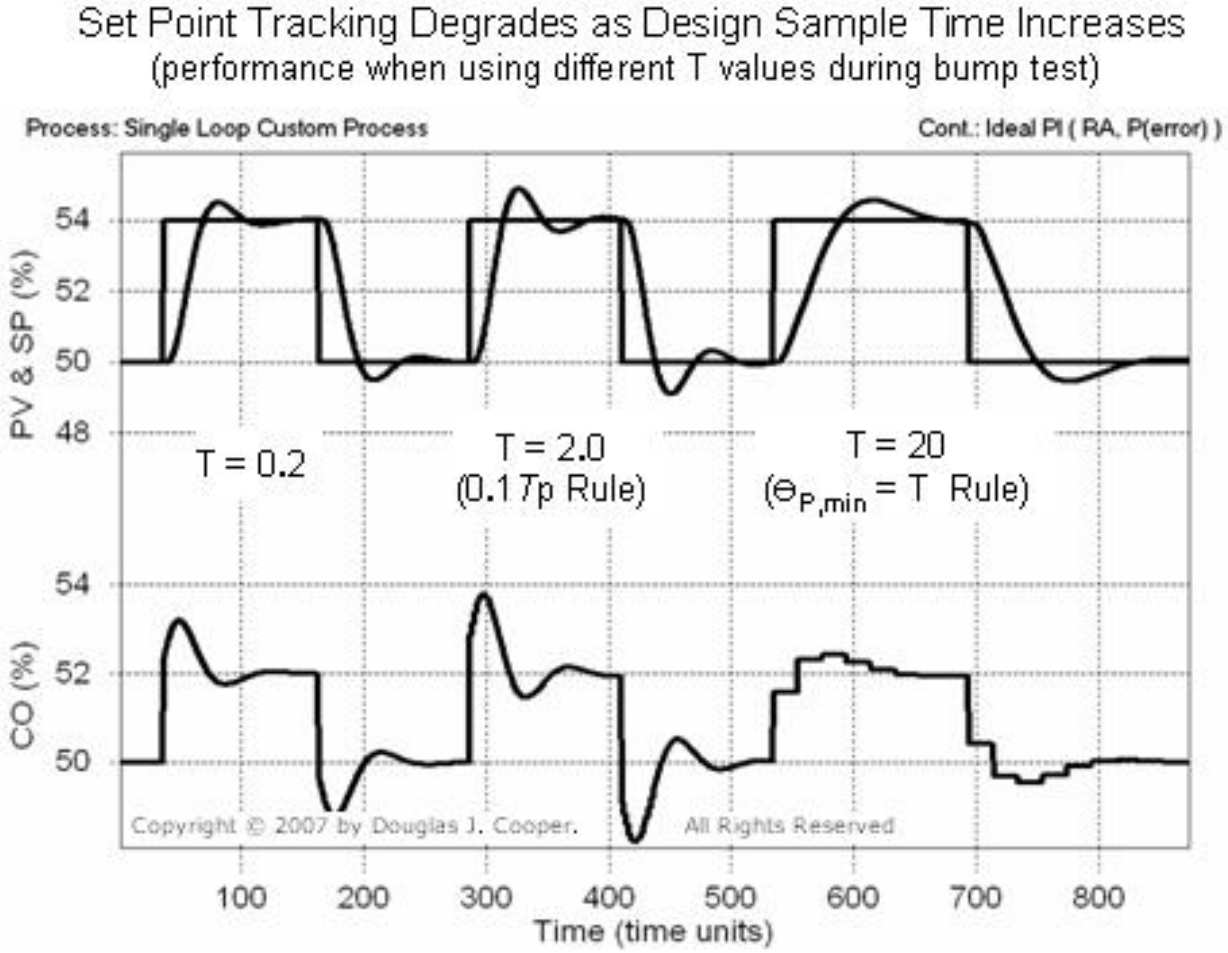
By definition, the minimum dead time, Θp, in a control loop is the loop sample time, T. Dead time can certainly be larger than T (and it usually is), but it cannot be smaller.

Thus, whether by software or graphical analysis, if we compute a Θp that is less than T, we must set Θp = T everywhere in our tuning correlations. This is the "Θp,min = T" rule for controller tuning.

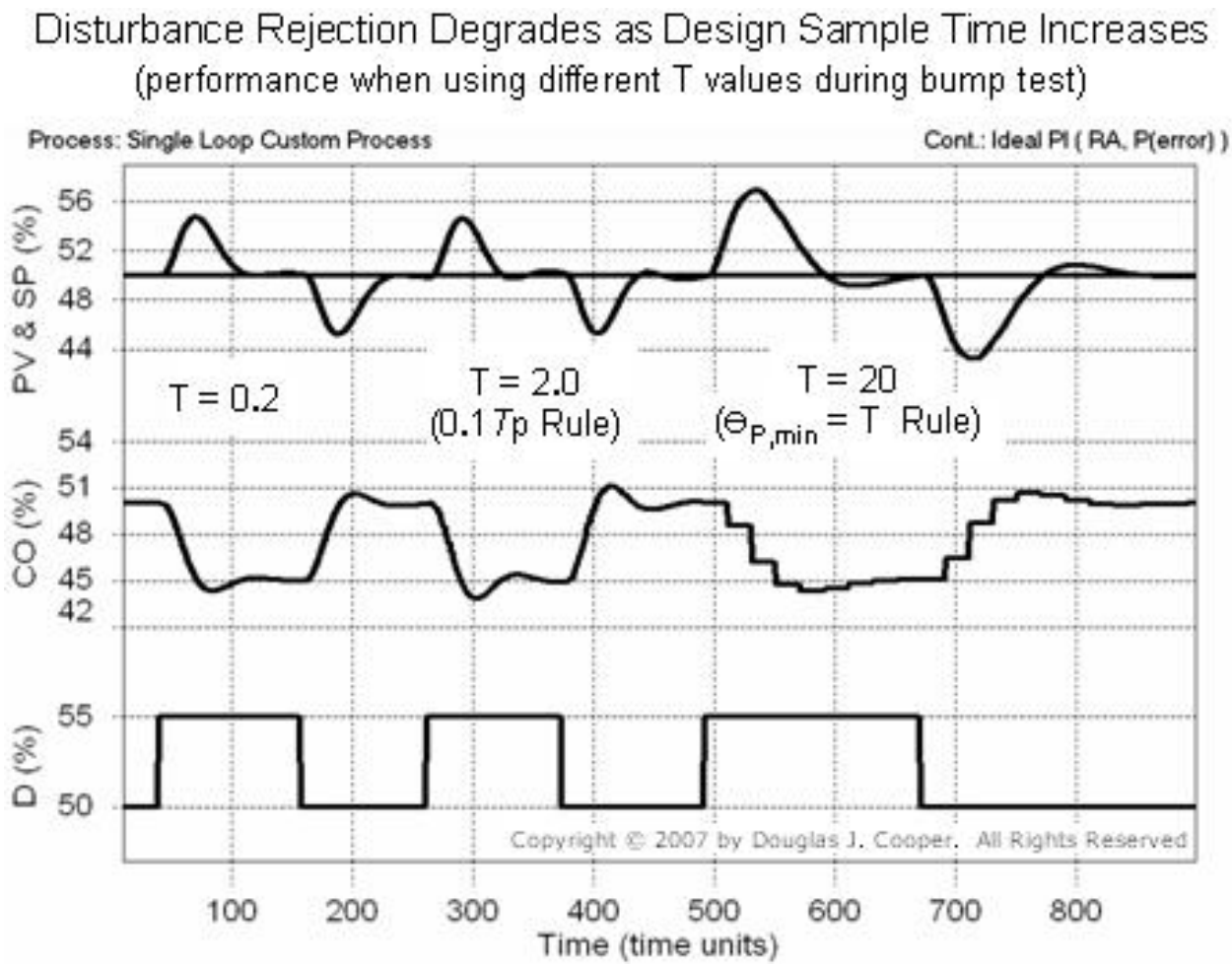
In the "sample slow" case, we know that we will be using T = 20 when we implement the controller. So even though the FOPDT model fit yields a Θp = 3.2, we use Θp = 20 when computing both *T*c and Kc as listed in the table.

The plot below shows the set point tracking performance of three controllers, each

designed and then implemented using three different sample times.



Below we show the performance of the three PI controllers in rejecting a disturbance. As discussed earlier in this article, recall that the D to PV dynamics are assumed to be identical to those of the CO to PV behavior.



We should note that if we were to use Θp = 3.2 when computing tuning values for the "sample slow" case, the controller would be wildly unstable. Even with the Θp,min = T rule, performance is noticeably degraded compared to the other cases.

Final Thoughts

1) The "best practice" rule that sample time should be 10 times per process time constant or faster (T ≤ 0.1*T*p) provides a powerful guideline for setting an upper limit on both control loop sample time and bump test data collection sample time.

2) Sampling as slow as once per *T*p during data collection and then controller implementation can produce a stable, though clearly degraded, controller. Be sure to follow the "Θp,min = T" rule when using the controller tuning correlations to achieve a stable result.

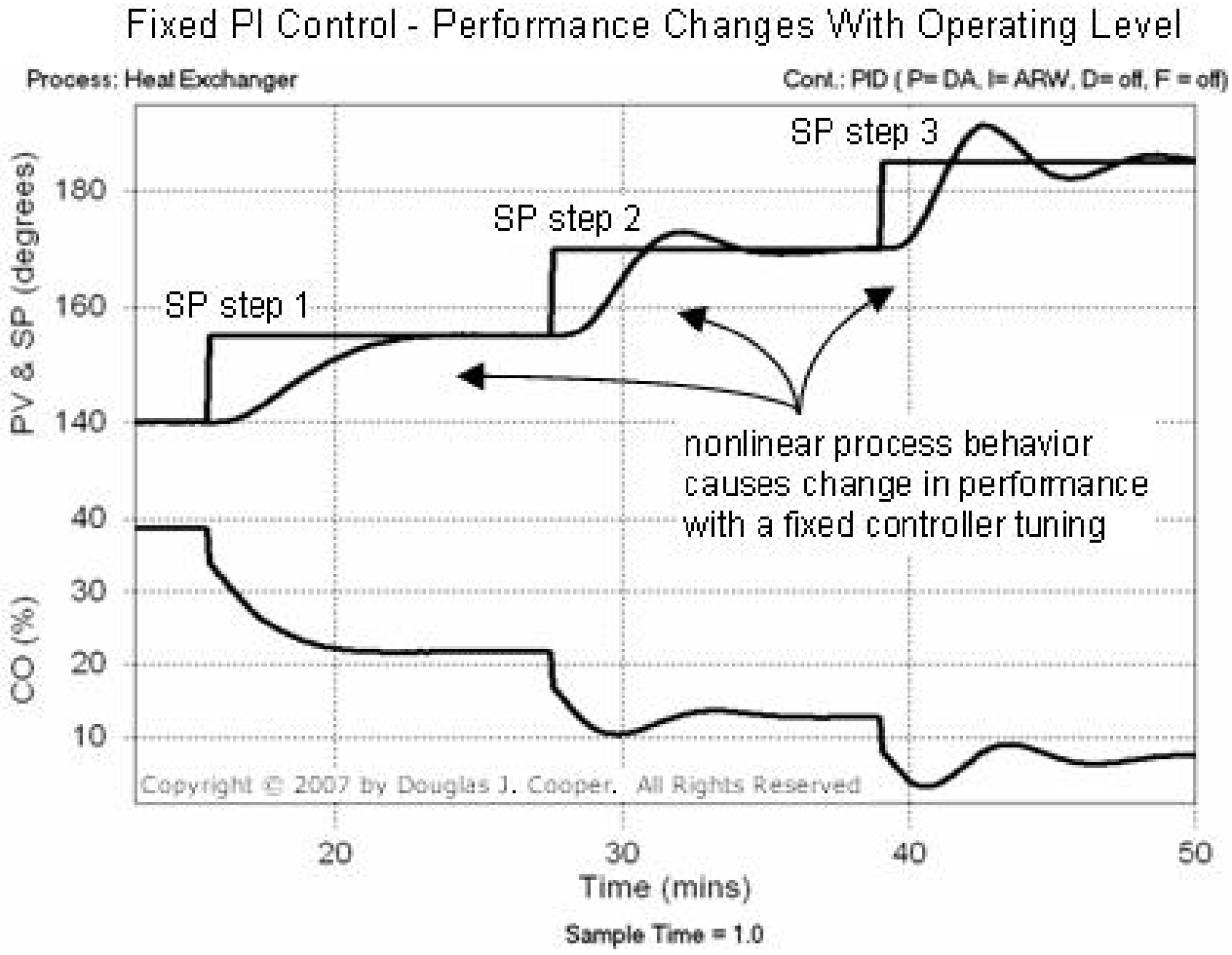
Parameter Scheduling and Adaptive Control of Nonlinear Processes

Processes with streams comprised of gases, liquids, powders, slurries and melts tend to

exhibit changing (or nonlinear) process behavior as operating level changes.

We discussed the nonlinear nature of the gravity drained tanks and heat exchanger processes in an earlier post. As we observed in that article and explore below, processes that are nonlinear with operating level will experience a degrading controller performance whenever the measured process variable (PV) moves away from the design level of operation (DLO).

We demonstrate this problem on the heat exchanger running with a PI controller tuned for a moderate response. As shown in the first set point step from 140 °C to 155 °C in the plot below, the PV responds in a manner consistent with our design goals. That is, the PV moves to the new set point (SP) in a deliberate fashion, but does not move so fast as to overshoot the set point.



The consequence of a nonlinear process behavior is apparent as the set point steps continue to higher temperatures. In the third SP step from 170 °C to 185 °C, the same PI controller that had given a desired moderate performance now produces an active PV response with a clear overshoot and slowly damping oscillations.

If we decide that such a change in performance with operating level is not acceptable, then parameter scheduled adaptive control may be an appropriate solution.

Developing an adaptive control strategy requires additional bump tests that may disrupt

production. Once sufficient process data is collected, adaptive controller design and implementation consumes more personnel time. Before we start the project, we should be sure that the loop has sufficient impact on our profitability to justify the effort and expense.

Parameter Scheduled Adaptive Control

The method of approach for parameter scheduled adaptive control is to:

a) Divide the total range of operation into some number of discrete increments or operating ranges.
b) Select a controller algorithm (P-Only, PI, PID or PID with CO Filter) for the application.
c) Specify loop sample time, action of the controller (reverse or direct acting), and other design values that will remain constant in spite of nonlinear behavior.
d) Apply our controller tuning recipe and compute tuning values for our selected controller at each of the operating increments as chosen in step a).

We require a computer based control system (a DCS or advanced PLC) to implement the adaptive logic. This is because the tuning values must be programmed as a look-up table (or schedule) where the measured PV indicates the current level of operation, and as such, "points" to appropriate controller tuning values in the table at any moment in time.

Once online, the computer reads a set of tuning values from the table as indicated by the current value of the PV. These are downloaded into the controller algorithm, which then proceeds to calculate the next controller output (CO) value.

Tuning updates are downloaded into the controller every loop sample time, T. As a result, the controller continually adapts as the operating level changes to maintain a reasonably consistent control performance across a range of nonlinear behavior.

Notes:
1) The set point (SP) is not appropriate to use as the operating level "pointer" because it indicates where we hope the PV will be, not necessarily where it actually is. The CO value can change both as operating level changes and as the controller works to reject disturbances. Since it reflects both disturbance load and operating level, the correspondence between current CO and current operating level is inconsistent. Current PV offers the most reliable indicator of expected process behavior.
2) "Gain scheduling" is a simplified variation of parameter scheduling, where, rather than updating all tuning values as operating level changes, only the controller gain is updated. All other tuning values remain constant with a pure gain scheduling approach. This simplification increases the chance that important process behaviors (such as a changing dead time, Θp) will be overlooked, thus decreasing the potential benefit of the adaptive strategy. With modern computing capability now widely available, we see no benefit from a "gain only" simplification unless it is the only choice offered by our vendor.

Interpolation Saves Time and Money
Ultimately, it is impractical to divide our range of operation into many increments and then tune a controller at each level. Such an approach requires that we bump the process at

least once in each operating increment. As we had alluded to in previous discussion, this can cause significant disruption to the production schedule, increase waste generation, use expensive materials and utilities, consume precious personnel time, and everything else that makes any loop tuning project difficult to sell in a production environment.

Thus, a popular variation on parameter scheduling, and the one explored in the case study below, is to design and tune only three controllers that span the range of expected operation. We then interpolate (fit a line) between the tuning values so we can update (or adapt) our controller to match any operating level at any time.

Normally, one controller is tuned to operate near the lower set point value we expect to encounter, while another is tuned for the expected high SP value. The third controller is then tuned for a strategically located mid range operation to give an appropriate shape to our interpolation curve. Our goal in choosing where to locate this midpoint is to reasonably approximate the complex nonlinear behavior of our process while keeping disruptive testing to a minimum. More discussion follows in the case study.

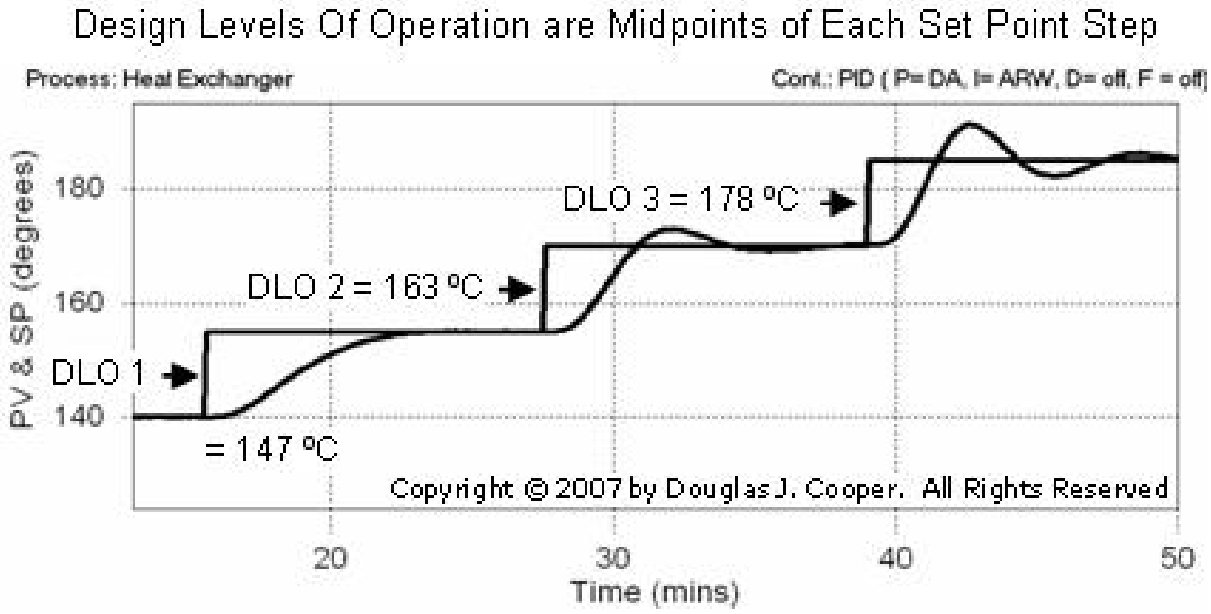
Case Study: Adaptive Control of the Heat Exchanger
We use the heat exchanger to illustrate and explore the ideas introduced above. As always, we follow our controller design and tuning recipe as we proceed. We choose a PI controller for the study and use the constant design values as detailed here (e.g., dependent PI algorithm form; loop sample time, T = 1.0 sec; controller is direct acting).

Step 1: Design Level of Operation (DLO)
Our adaptive schedule requires tuning values for three PI controllers that span the range of expected operation. Hence, we need to specify three design levels of operation (DLOs), one for each controller.

Since, as we detail here, set point driven data can be analyzed with commercial software for controller design and tuning, we will use the data from the plot at the top of this post as our bump test data.

As discussed in this article, a good bump test should generate dynamic process data both above and below the DLO. This is best practice because we then "average out" nonlinear effects when, in step 3 that follows, we approximate the bump test data with a simplifying first order plus dead time (FOPDT) dynamic model.

Since our test data has already been collected and plotted, we reverse this logic and pick the three DLOs as the midpoint of each SP step. Reading from the plot as shown below, we thus arrive at: DLO 1 = 147 °C; DLO 2 = 163 °C; DLO 3 = 178 °C

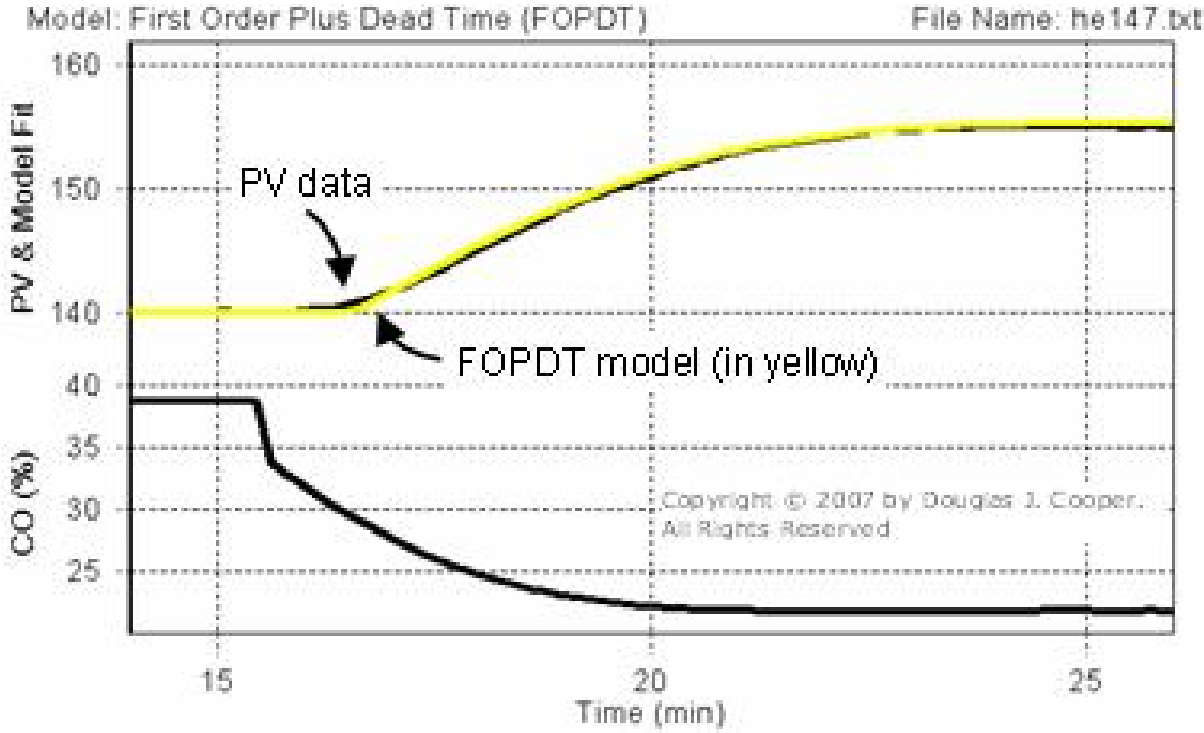


Step 2: Collect Process Data around the DLO
The plot data provides us with three complete CO to PV bump tests, each centered around its DLO. Hence, step 2 is complete.

Step 3: Fit a FOPDT Model to the Dynamic Process Data
We use Control Station's Loop-Pro software to divide the plot data into three CO to PV bump tests. We then use the software to fit a FOPDT model to each bump test following the same procedure as detailed here.

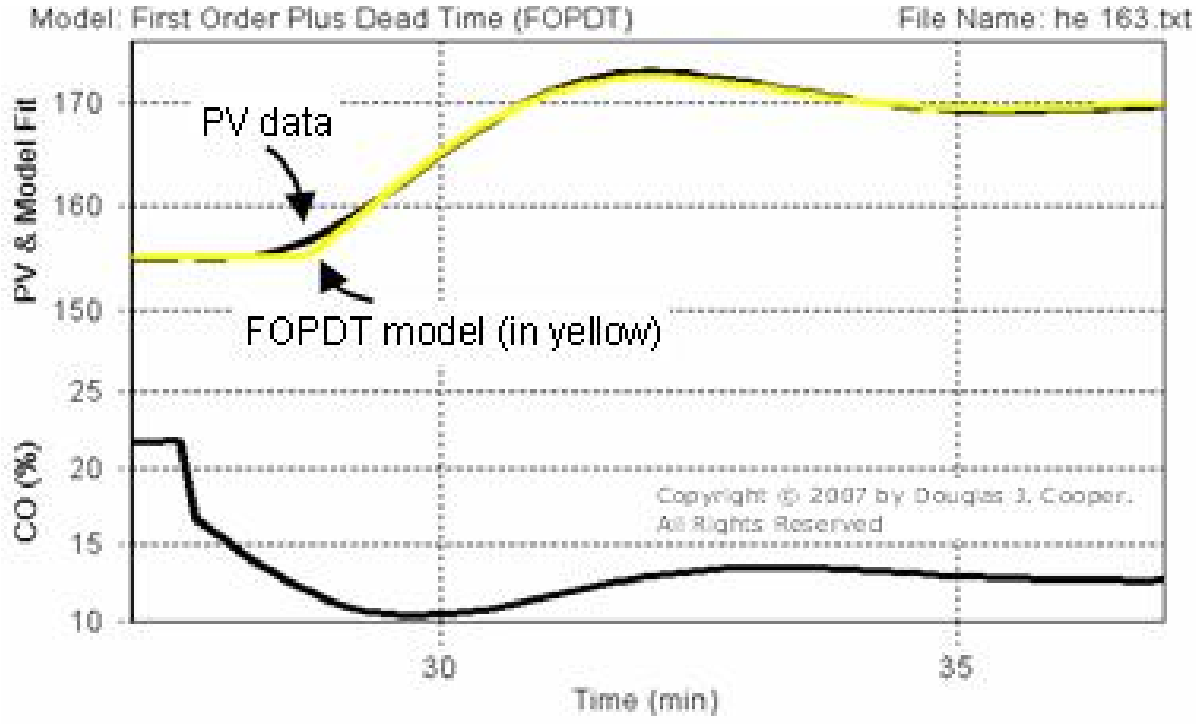
The plots below (see large view of step 1, step 2, or step 3) show the data and FOPDT model approximations. Because each FOPDT model visually matches its bump test data, we have confidence that the model parameters (listed below each plot and summarized in the table in Step 4) reasonably describe the dynamic process behavior at the three design levels of operation.

Model Fit of Set Point Step 1: 140 → 155 °C

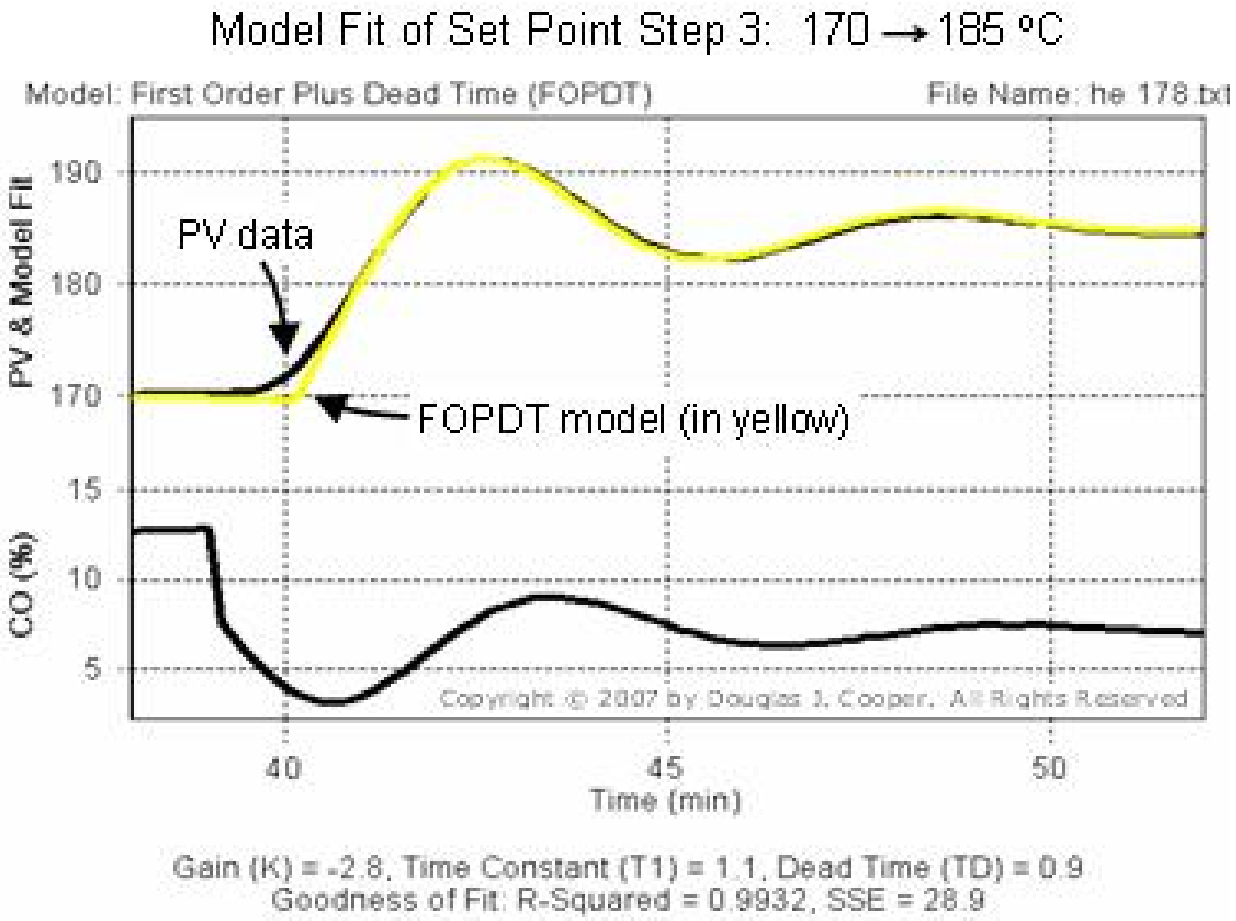


Gain (K) = -0.9, Time Constant (T1) = 1.2, Dead Time (TD) = 1.0
Goodness of Fit: R-Squared = 0.9983, SSE = 5.625

Model Fit of Set Point Step 2: 155 → 170 °C



Gain (K) = -1.7, Time Constant (T1) = 1.0, Dead Time (TD) = 1.0
Goodness of Fit: R-Squared = 0.9976, SSE = 8.697



Step 4: Use the FOPDT Model Parameters to Complete the Design
The table below summarizes the FOPDT model parameters from step 3 for the three DLOs.

FOPDT Model Values	SP Step 1 140 →155 °C DLO 1 = 147 °C	SP Step 2 155 →170 °C DLO 2 =163 °C	SP Step 3 170 →185 °C DLO 3 =178 °C
Kp	− 0.9	− 1.7	− 2.8
Tp	1.2	1.0	1.1
Θp	1.0	1.0	0.9
Moderate PI Tuning			
Kc	− 0.15	− 0.07	− 0.05
Ti	1.2	1.0	1.1
Aggressive PI Tuning			
Kc	− 0.74	− 0.33	− 0.24
Ti	1.2	1.0	1.1

Units: Kp [=] °C/%, Tp [=] min, Θp [=] min

Kc [=] %/°C, Ti [=] min

Note that the process gain, Kp, varies by 300% (from -0.9 to -2.8 °C/%) across the operating range. In contrast, process time constant, Tp, and process dead time, Θp, change by quite modest amounts.

For this investigation, we compute both moderate and aggressive PI tuning values for each DLO. The rules and correlations for this are detailed in this post, but briefly, we compute our closed loop time constant, Tc, as:

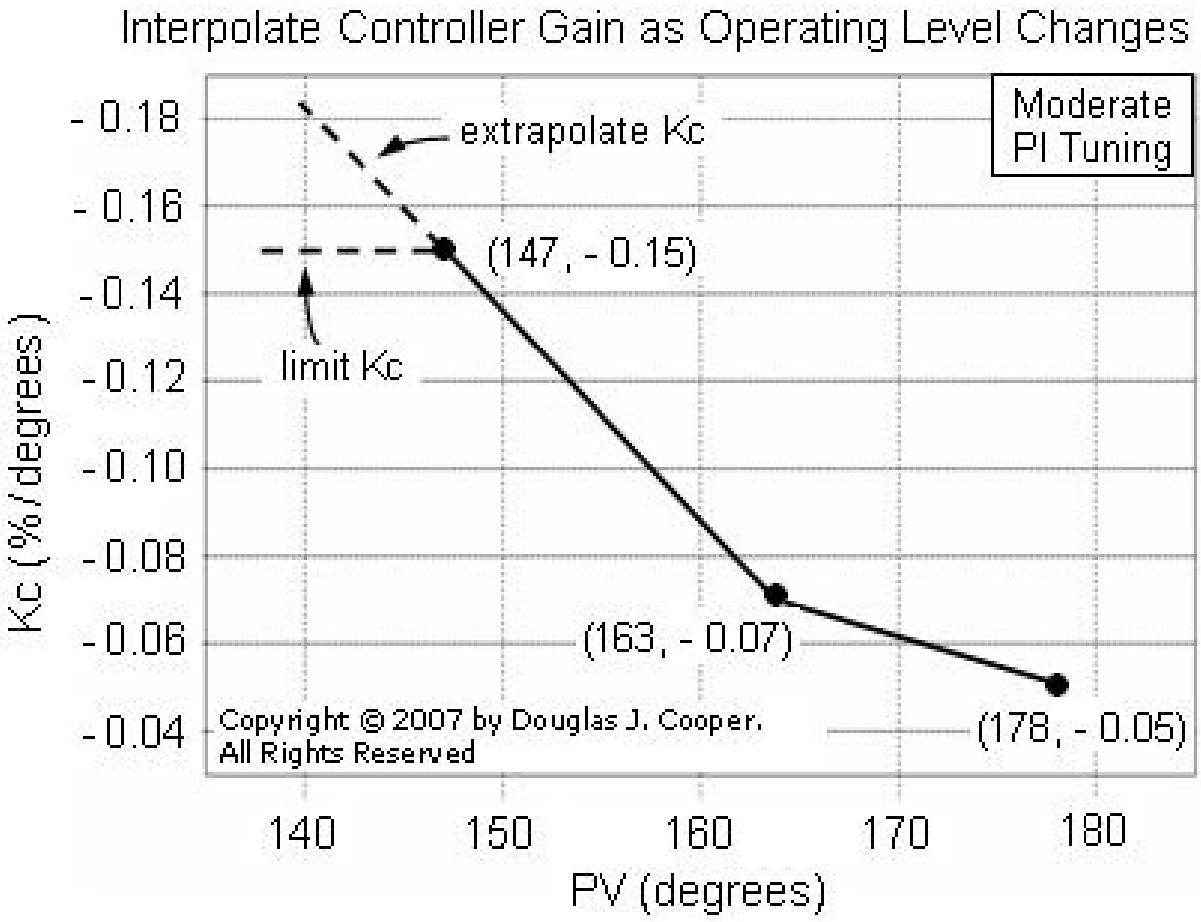
- *aggressive:* Tc is the larger of 0.1·Tp or 0.8·Θp
- *moderate*: Tc is the larger of 1·Tp or 8·Θp

With Tc computed, the PI correlations for controller gain, Kc, and reset time, Ti, are:

$$Kc = \frac{1}{Kp}\frac{Tp}{(\theta p + Tc)} \quad \text{and} \quad Ti = Tp$$

The moderate and aggressive PI tuning values for each of the DLOs are also summarized in the table above.

Below we illustrate how to interpolate controller gain, Kc, for the moderate tuning case.



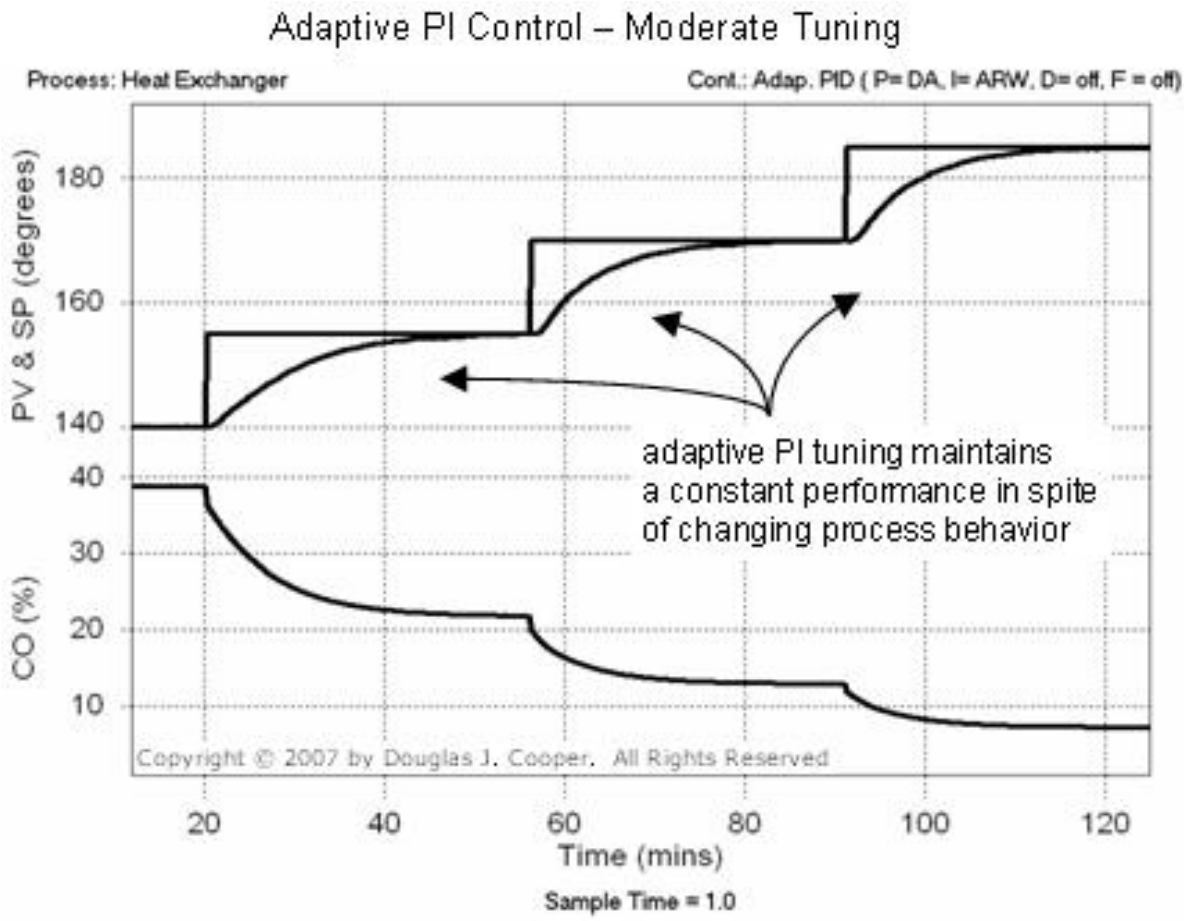
As shown in the plot, the three moderate Kc values are plotted as a function of PV. Lines of interpolation are fitted between each Kc value. The equations for these lines must be programmed into the control computer.

Now, as PV moves anywhere from 147 to 178 ^{o}C, we can use these equations to compute a unique value for Kc. This concept must also be applied to obtain interpolating equations for reset time, *T*i, thus producing a fully adaptive controller.

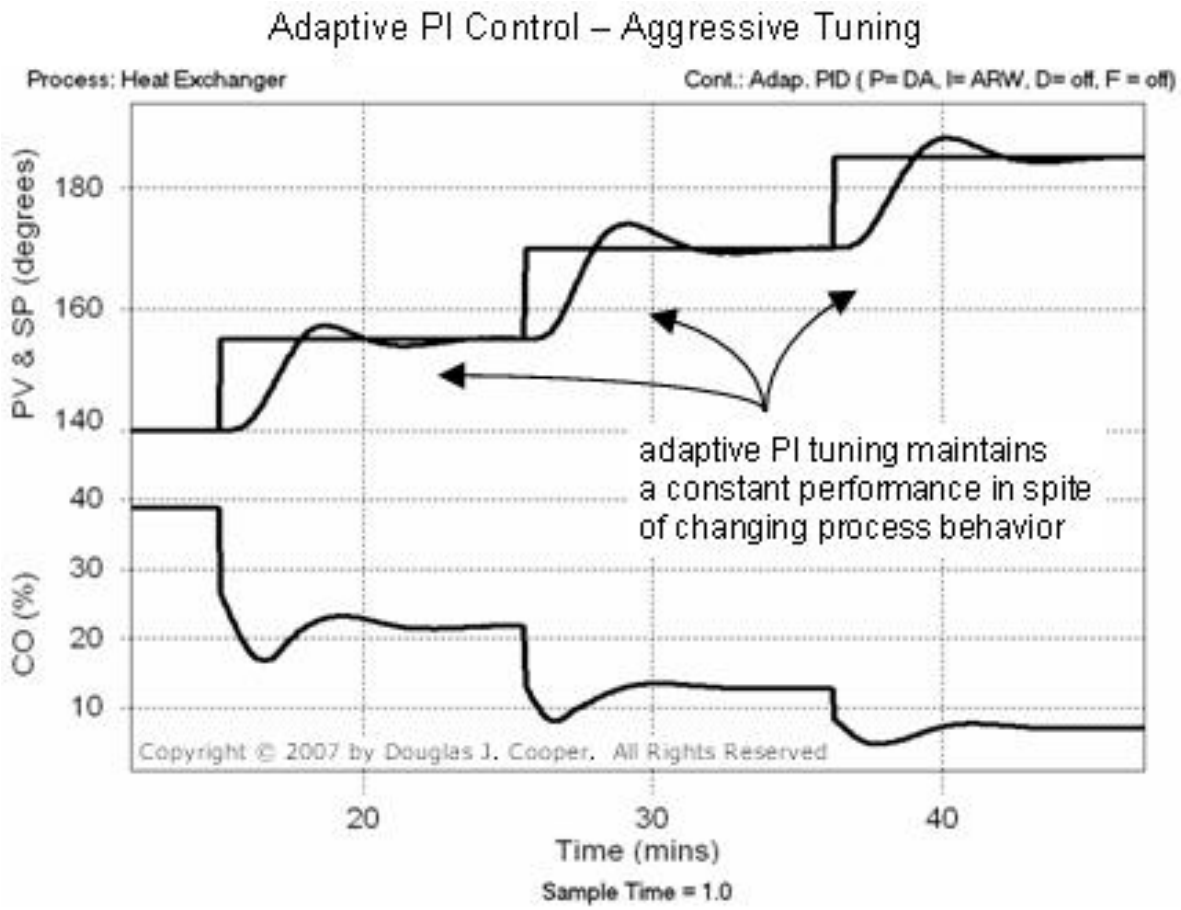
As shown in the plot above, one decision that must be made is whether to extrapolate the line and have the parameter continue the trend past the actual maximum or minimum data point. Alternatively, we could choose to limit Kc and have it stay constant for all PV values beyond the maximum or minimum.

Unless we are confident that we understand the true nature of a process, extrapolation into the unknown is more often a bad idea than a good one. In this case study, we choose not to extrapolate. Rather, we limit the tuning parameters to the maximum and minimum DLO values in the table. That is, Kc remains constant at -0.15 %/°C when PV moves below 147 ^{o}C, and remains constant at -0.05 when Kc moves above 178 ^{o}C. In between, Kc tracks the interpolating lines in the plot above.

The capability of this parameter scheduled adaptive control is shown for a moderate PI controller in the plot below. To appreciate the difference, compare this constant performance to the varied response in the plot at the top of this post.



The result of an aggressively tuned PI controller is shown in the plot below. The performance response is again quite consistent (though admittedly not perfect) across the range of operation.



A Proven Strategy
As these plots illustrate, a parameter scheduled adaptive controller can achieve consistent performance on processes that are nonlinear with operating level. This adaptive strategy has been widely employed in industrial practice and, as shown in this case study, is quite powerful in addressing a challenging and important problem.

Again, however, design and implementation requires extra effort and expense. We should be sure the loop is important enough to warrant such an investment before we begin

Plant-Wide Control Requires a Strong PID Foundation

The term "plant-wide control" is used here to describe the use of advanced software that sits above (or on top of) the individual PID controllers running a number of process units in a plant.

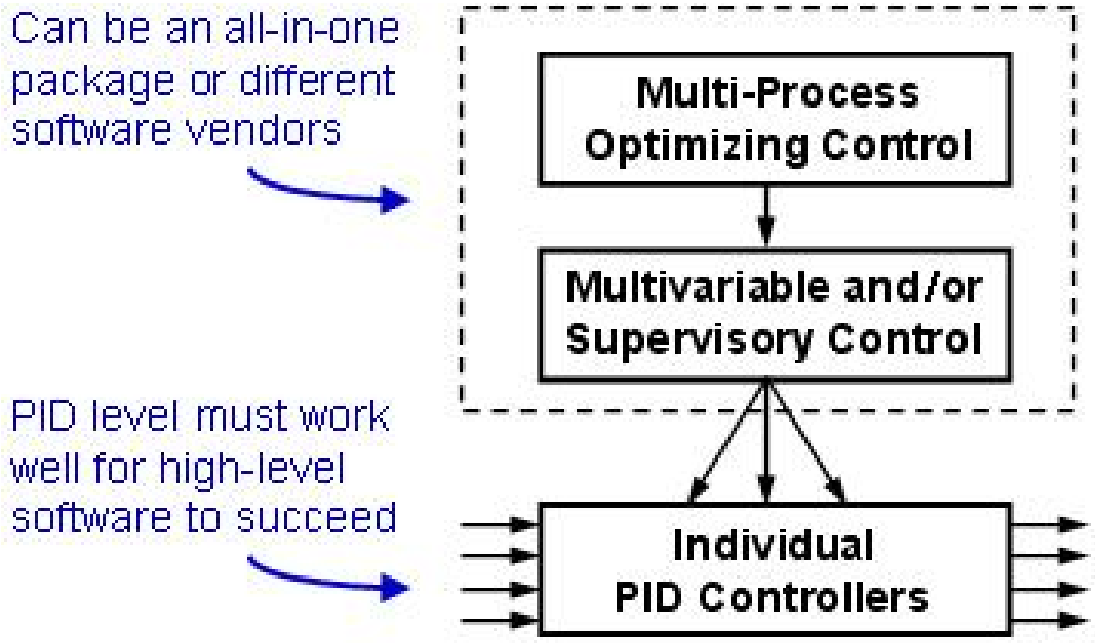
Depending on the technology employed, this advanced process control software can perform a variety of predictive, scheduling, supervisory and/or optimizing computations. The most common architecture in industrial practice has the plant-wide software compute and transmit set point updates to a traditional platform of individual PID controllers.

Notes:
☐ Plant-wide control, where one package computes updates for an entire facility, is widely discussed in scholarly articles but is rarely found in industrial practice.
☐ While it is possible for an advanced control package to completely eliminate the

need for individual PID controllers, this is also a rare practice. One reason is that advanced software is often an add-on to a plant already in operation. The existing PID controllers provide a distributed backup infrastructure that enable continued operation, including an orderly shutdown, in the event of a computer problem. Even in new construction, plants are normally built with a traditional platform of PID controllers underneath the advanced control software.

Plant-wide process control holds great allure. The promise is an orchestrated operation for addressing upsets, maximizing throughput, minimizing energy and environmental impact, making scheduling more flexible, and keeping production to a tighter specification.

The figure below illustrates the hierarchy of a more complex multi-level implementation.



Copyright © 2006 by Douglas J. Cooper. All Rights Reserved

Higher Level Software Uses Longer Time Scales
The individual PID loops of a traditional control system provide the firm foundation for advanced process control. They are designed, implemented and validated as discussed in the dozens of articles on this site. Typical loop sample times for PID controllers on processes with streams comprised of gases, liquids, powders, slurries and melts are often on the order of once per second.

Advanced software sitting above the individual controllers computes and transmits values less frequently, perhaps on the order of once every ten seconds to several minutes. High level optimizers output commands even less often, ranging from once per hour to even once per day. The frequency depends on the time constants of the process units in the hierarchy, the complexity of the plant-level control objectives, the numerical solution methods employed, and the capabilities of the installed hardware.

Project Stages For a Retrofit
Though each implementation is different, projects on existing plants tend to follow a standard progression:

1) Validate all sensors and associated instrumentation; replace where necessary.
2) Service all valves, pumps, compressors and other final control elements to ensure proper function. Replace or upgrade where necessary.
3) Upgrade the DCS (distributed control system) with latest software releases; update hardware if it is aging.
4) Tune all low level PID loops, including cascade, ratio and similar architectures.
5) Upgrade the computers in the control room to modern standards; ensure enough computing power to handle the plant-level software.
6) Design and deploy the plant-level control software.

Note that step 6 is presented as a simplistic single step. In reality, the design and implementation of plant-level software is a complex procedure requiring tremendous experience and sophistication on the part of the project team. Step 6 is presented as a summary bullet item because the purpose of this post is to separate and highlight the vital importance of a properly operating platform of individual PID controllers in any control project.

Steps 1-4 Provide Profitability
Software vendors who suggest that a prospective company consider all steps 1-6 as part of an "advanced process control" project are not being completely transparent.

The return on investment (ROI) may appear attractive for the complete project, but it is appropriate to determine what portion of the return is provided by steps 1-4 alone. The profit potential of these first steps, reasonably characterized as traditional control tasks, can be responsible for well over half of the entire revenue benefit on some projects!

Arguably, it is a better business practice to work through steps 1-4 and then reevaluate the situation before making a decision about the need for and profit potential from plant-level control software.

When the base level instrumentation is in proper working order, the PID loops can be tuned to provide improved plant performance. Rather than using historical tunings, the project team should use commercial software to quickly analyze process data and compute appropriate tuning parameters.

PID Control is The Foundation
As the figure at the top of this post illustrates, the PID loops provide the strong foundation upon which the plant-level software sits.

Plant-wide process control software cannot hope to improve plant operation if the PID loops it is orchestrating are not functioning properly. It is folly to proceed with step 6 above before having worked through steps 1 - 4.

In a great many situations, by the time step 4 is completed, indeed, the plant will be running significantly better. The orderly maintenance and tuning of the first level PID loops will provide a fast payback and begin making money for the company. Quickly.

12) Controller Tuning Using Closed-Loop (Automatic Mode) Data

Ziegler-Nichols Closed-Loop Method a Poor Choice for Production Processes

Ziegler and Nichols first proposed their method in 1942. It is a trial-and-error loop tuning technique that is still widely used today. The automatic mode (closed-loop) procedure is as follows:

- Set our controller to P-Only action and switch it to automatic when the process is at the design level of operation.
- Guess an initial controller gain, Kc, that we expect (hope) is conservative enough to keep the loop stable.
- Bump the set point a small amount and observe the response behavior.
- If the controller is not causing the measured process variable (PV) to sustain an oscillating pattern, increase the Kc (or decrease the proportional band, PB) and wait for the new response.
- Keep adjusting and observing by trail and error until we discover the value of Kc that causes sustained uniform oscillations in the PV. These oscillations should neither be growing nor dying out, and the controller output (CO) should remain unconstrained.
- The controller gain at this condition is called the ***ultimate gain***, Ku. The period of the PV oscillation pattern at the ultimate gain is called the ***ultimate period***, Pu.

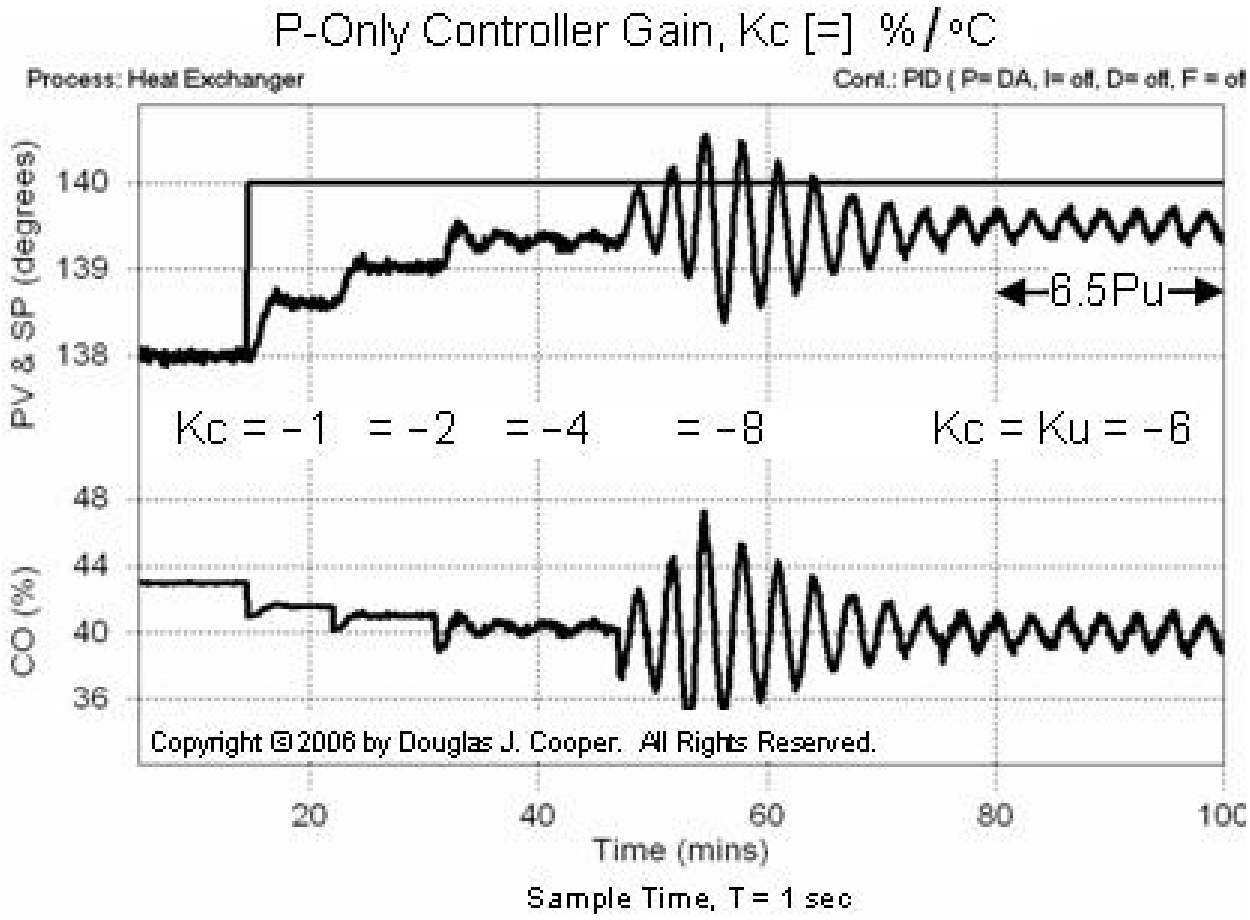
After using the procedure above to determine the ultimate gain and ultimate period, the Ziegler-Nichols (Z-N) correlations to compute our final tuning for a PI controller are:

• Ziegler-Nichols PI Tuning: $K_c = 0.45 \cdot K_u \quad T_i = P_u/1.2$

Many process control books and articles propose a variety of tuning correlations using Ku and Pu that provide improved performance over that provided by the original Z-N correlations listed above. Of course, the definition of "improved" can vary widely depending on the process being controlled and the operations staff responsible for a safe and profitable operation.

Ziegler-Nichols Applied to the Heat Exchanger
To illustrate the procedure, we apply it to the same heat exchanger process explored in numerous articles in this e-book.

Below is a plot () showing the heat exchanger under P-Only control. The controller gain, Kc, is initially set to Kc = –1.0 %/°C, a value we hope provides conservative performance as we start our guess and test investigation.



As shown above, the process is perturbed with a set point step and we wait to see if the controller yields sustained oscillations. The process variable (PV) displays a sluggish response at Kc = –1.0 %/°C, so we double the controller gain to Kc = –2.0 %/°C, and then double it again to Kc = –4.0 %/°C, waiting each time for the response pattern to establish itself.

When changing from a Kc = –4.0 up to Kc = –8.0 %/°C as shown in the above plot at about time t = 45 min, the heat exchanger goes unstable as evidenced by the rapidly growing oscillations in the PV. This indicates we have gone too far and must back down on the P-Only controller gain.

A compromise of Kc = –6.0 %/°C seems to create our desired goal of a process teetering on the brink of stability. Hence, we record our ultimate gain as:

Ku = –6 %/°C

As indicated in the plot above from time t = 80 min through t = 100 min, when under P-Only control at our ultimate gain, the PV experiences 6.5 complete cycles or periods of oscillation over 20 minutes. The ultimate period, Pu, is thus computed as:

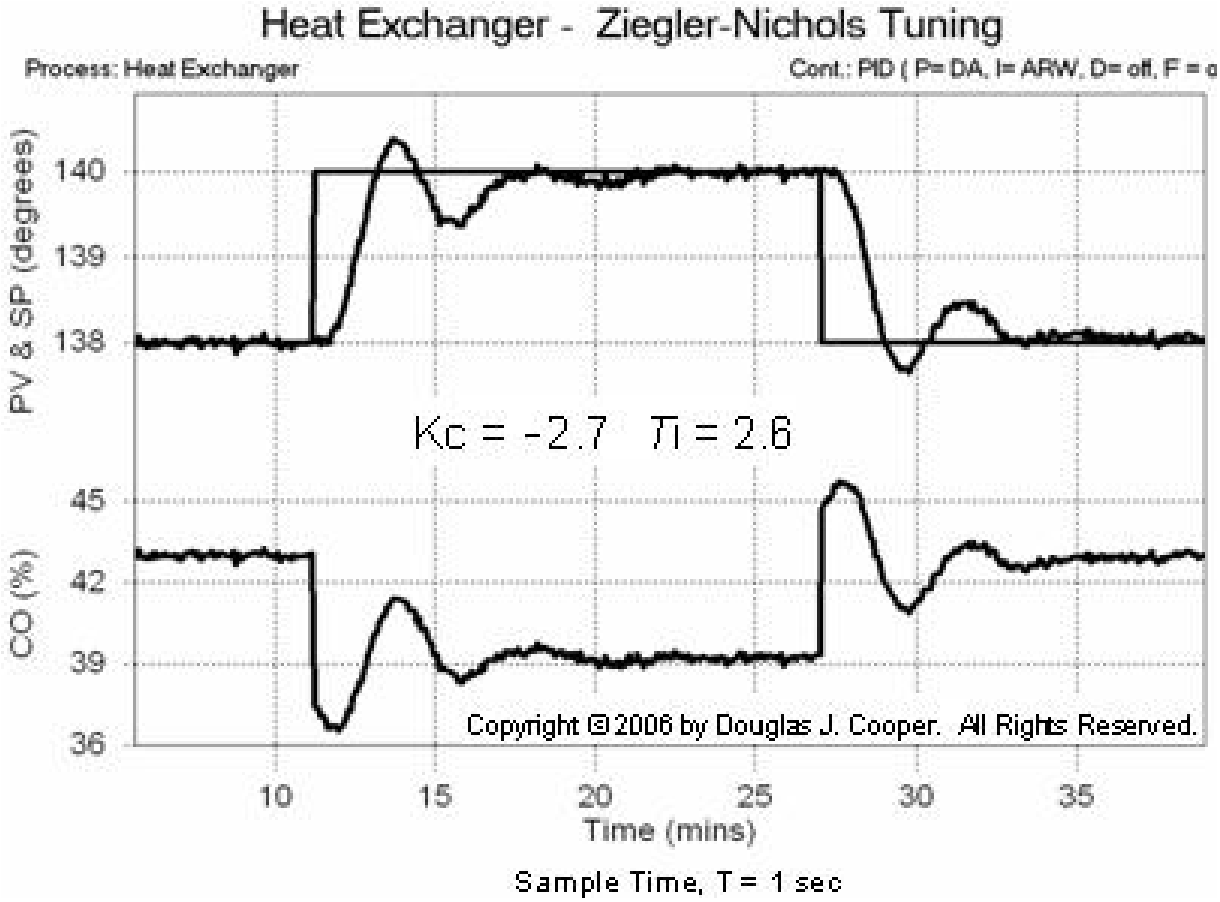
Pu = 20 min/6.5 = 3.1 min

We employ these Ku and Pu values in the Z-N correlations above to obtain our PI tuning values:

• Kc = 0.45·Ku = 0.45 (–6 %/°C) = –2.7 %/°C

• Ti = Pu/1.2 = 3.1 min/1.2 = 2.6 min

Below () is the performance of this Z-N tuned PI controller in tracking a set point step from 138 °C up to 140 °C. For comparison, the heat exchanger responding to the same set point steps using an IMC tuned PI controller is shown here.



Concerns in a Production Environment
Suppose we were using the Z-N closed-loop method to tune the above controller in a production environment:

1) The trial and error search for the ultimate gain took about 80 minutes in the test above. For comparison, consider that the single step test for the IMC design on the heat exchanger took only 5 minutes.

Even if there is suspicion that we skewed the process testing to make Z-N look bad, the fact remains that it will always take significantly longer to find the ultimate gain by trial and error then it will to perform a simple bump test.

And in a production situation, this means we are losing time and wasting money as we make bad product. Why even consider such a time-consuming and wasteful tuning method when it is not necessary?

2) Perhaps more significant, the Z-N method requires that we literally bring our process to

the brink of instability as we search for the ultimate gain. Creeping up on the ultimate gain can be very time consuming (and hence very expensive), but if we try to save time by making large adjustments in our search for Ku, it becomes much more likely that we will actually go unstable, at least for a brief period.

In production situations, especially involving reactors, separators, furnaces, columns and such, approaching an unstable operation is an alarming proposition for plant personnel.

Final thoughts
This e-book focuses on the control of processes with streams composed of liquids, gases, powders, slurries and melts. The challenging control problems for such process are most always found in a production environment.

Our controller design and tuning recipe is designed specifically for such situations. It's fast and efficient, saves time and money, and provides consistent and predictable performance.

Controller Tuning Using Set Point Driven Data

The controller design and tuning recipe we have used so successfully on this site requires that we bump our process and collect dynamic data as the process responds.

For the heat exchanger and gravity drained tanks study, we generated dynamic data using a step test while the controller was in manual mode. One benefit of this "open loop" step test is that we can analyze the response graph by hand to compute first order plus dead time (FOPDT) dynamic model parameters. These parameter values are then used in rules and correlations to complete the controller design and tuning.

In a production environment, operations personnel may not be willing to open a loop and switch to manual mode “just” so we can perform a bump test on a process. In these situations, we must be prepared to perform our dynamic testing when the controller is in automatic.

Arguably, such closed-loop testing offers potential benefits. Presumably, operation is safer while under automatic control. And closed loop testing should enable us to generate dynamic data and return to steady operation faster than other bump test method.

Software Required
Unfortunately, once we deviate from the pure manual mode step test, we must use a software tool to fit an FOPDT model to the data. Fortunately, inexpensive software is available that helps us perform our data analysis and controller tuning chores quickly and reliably.

In theory, closed loop dynamic testing can be problematic because the information contained in the process data will reflect both the character of the controller as well as that of the process. If we remain conscious of this and take modest precautions, this theoretical concern rarely causes real world problems.

Generating Dynamic Data in Automatic Mode
For closed loop studies, dynamic test data is generated by stepping, pulsing or otherwise

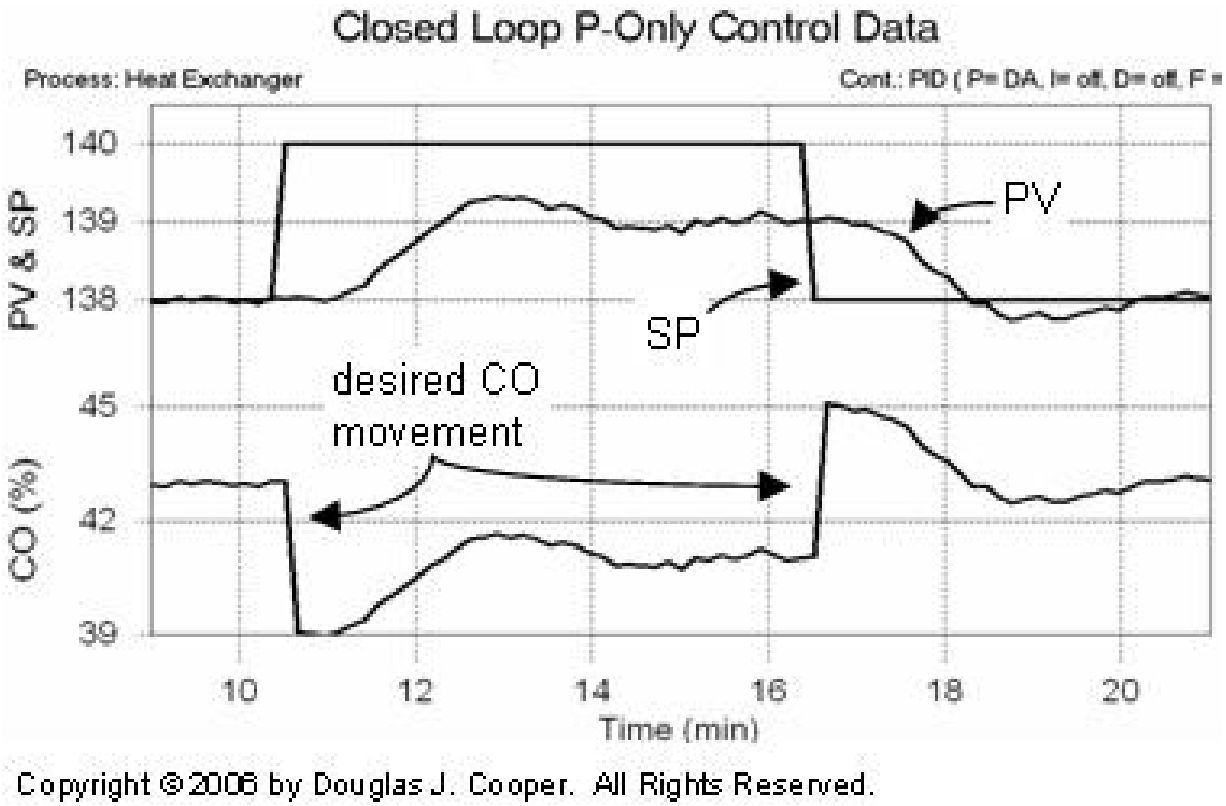
bumping the set point.

For model fitting purposes, the controller, when working to track this set point bump, must take actions that are energetic enough to generate a proper data set, but not be so aggressive that the PV oscillates wildly during data collection.

Similar to manual mode testing, a proper data set is generated when the controller output (CO) is moved far enough and fast enough to force a clear response in the measured process variable (PV).

Also, similar to manual mode testing, the process must be at a steady operation before first bumping the set point. The point of a bump test is to learn about the cause-and-effect relationship between the CO and PV. With the plant at a steady operation, we are starting with a clean slate and the dynamic character of the process will be clearly revealed as the PV responds. Just be sure to have the data capture routine collect the entire event, starting from before the initial bump.

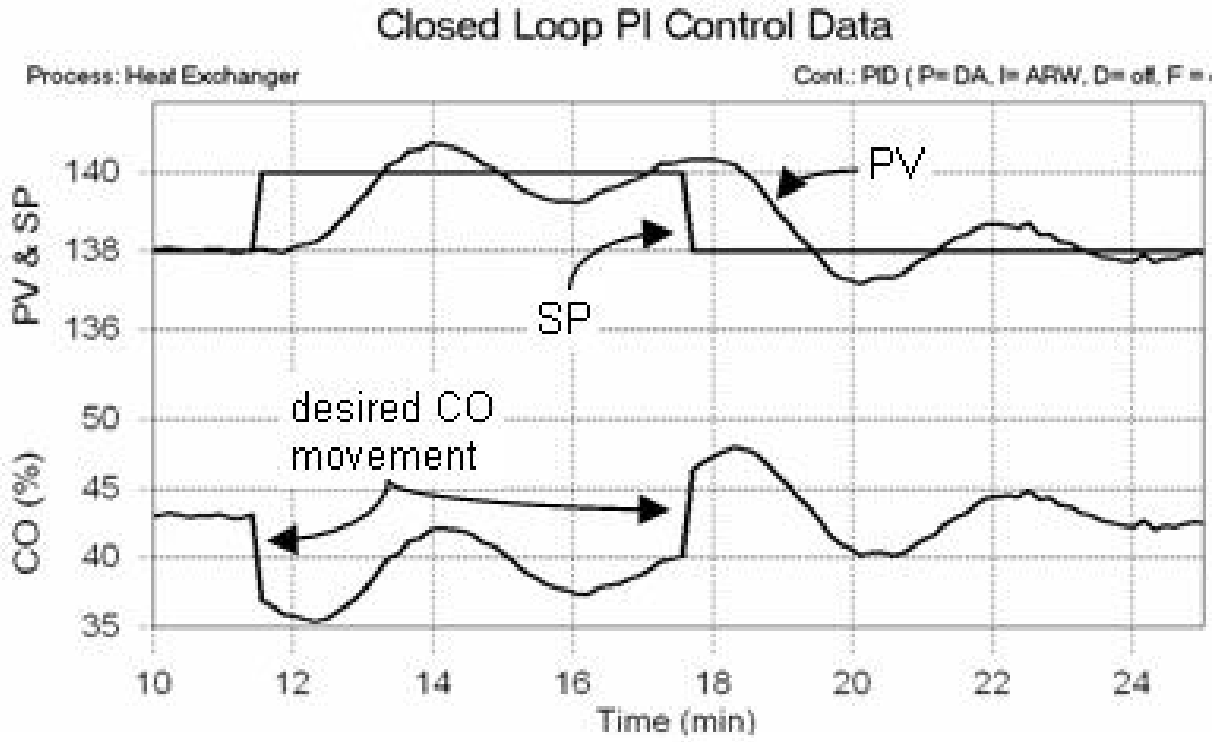
Set Point Bumps to the Heat Exchanger
Below is the heat exchanger process under P-Only control. As shown (), the process is initially at a steady operation. The set point is stepped from 138 °C up to 140 °C and back again. The P-Only controller produces a moderate set point response, with offset displayed as expected from this simple controller form.



Copyright © 2006 by Douglas J. Cooper. All Rights Reserved.

What is important in the above test is that when the set point is stepped, the P-Only controller is active enough to move the CO in the desired "far enough and fast enough" manner to force a clear response in the PV trace. This obvious CO to PV cause-and-effect relationship is exactly what we require in a good data set.

Below is the heat exchanger process with a poorly tuned PI controller. Again, the process is initially at a steady operation and the set point is stepped from 138 °C up to 140 °C and back ().

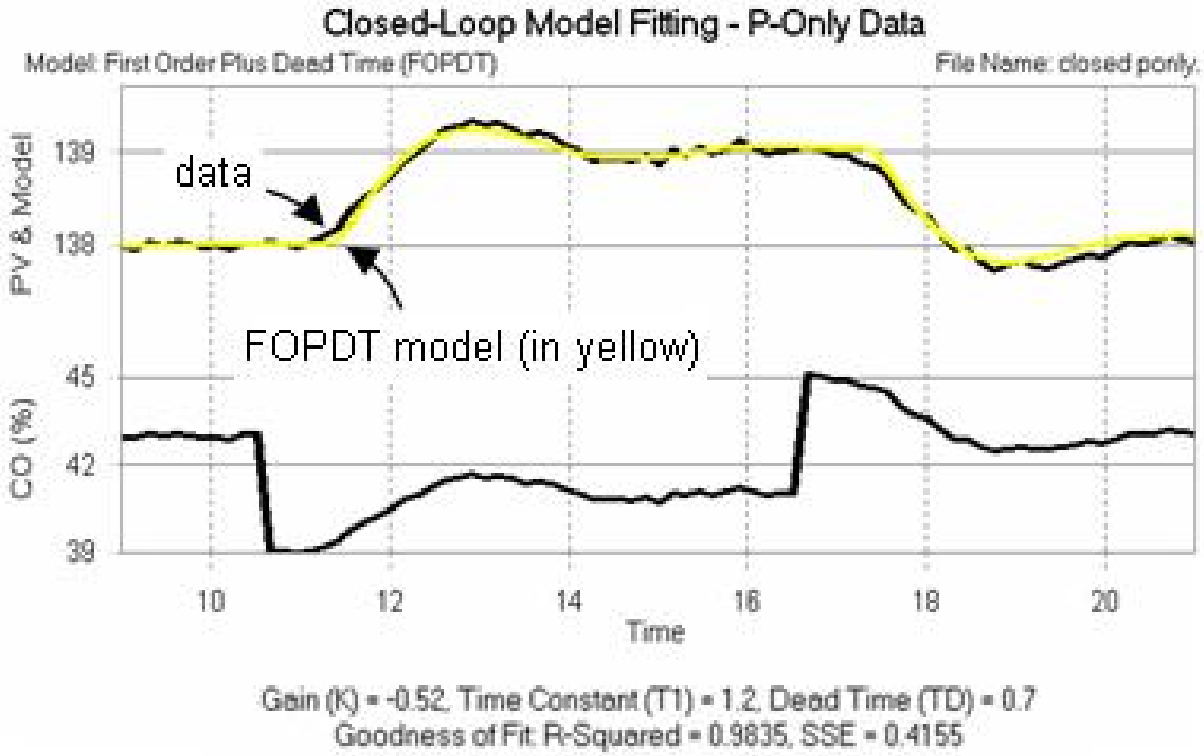


Copyright © 2006 by Douglas J. Cooper. All Rights Reserved.

This process is approaching the upper extreme of energetic oscillations. But as noted on the plot, we can still see the desired clear and sudden CO movement that forces the PV dynamics needed for a good FOPDT model fit.

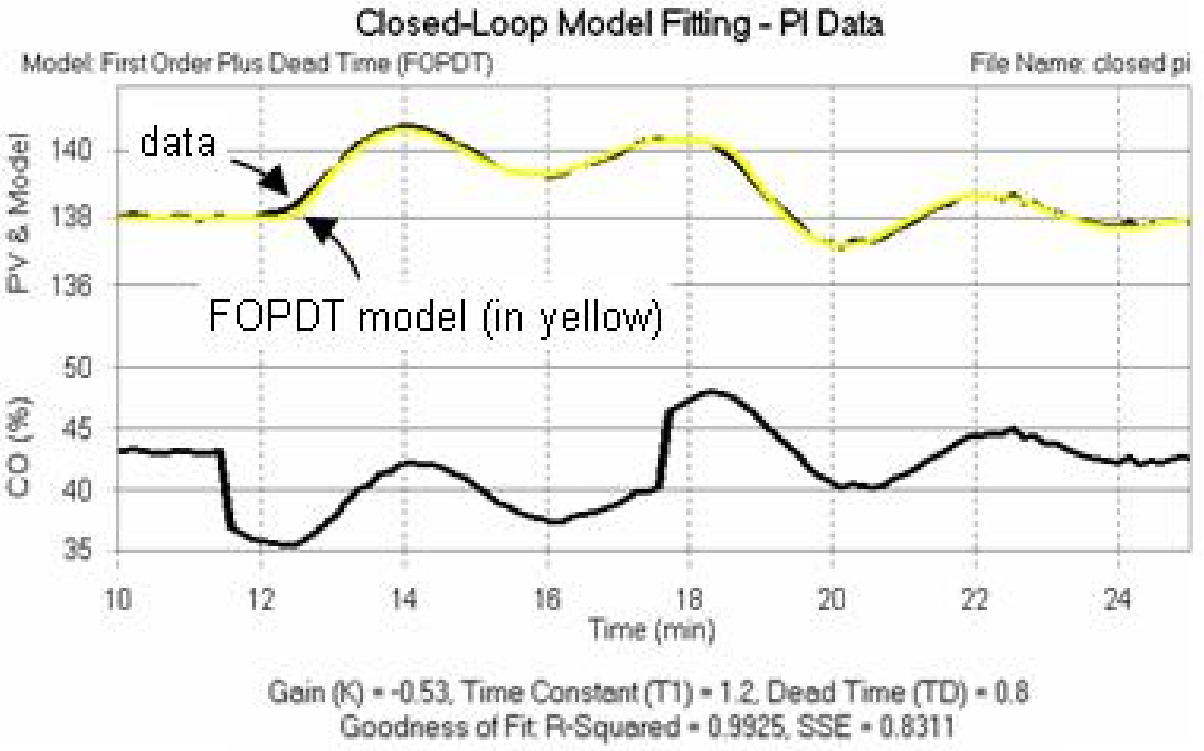
Automated Model Fitting
Below we fit a FOPDT model to the process data from the above two set point bump tests. We use the Control Station *Engineer* software in these examples.

Here is an automated model fit of the above P-Only control data () using the Control Station software:



Copyright © 2006 by Douglas J. Cooper. All Rights Reserved.

And here is an automated model fit of the above PI control data () using the software:



Copyright © 2006 by Douglas J. Cooper. All Rights Reserved.

Comparing Results

The table below compares the FOPDT model parameters from the above set point driven

bump test against the open loop step test results presented in a previous article:

	Open Loop Step Test	Closed Loop P-Only Data	Closed Loop PI Data
Process Gain, Kp	-0.53	-0.52	-0.53
Time Constant, Tp	1.3	1.2	1.2
Dead Time, θp	0.8	0.7	0.8

The table confirms what we can see visually in the model fit plots; it is certainly possible to obtain an accurate FOPDT dynamic model from closed loop data.

Implications for Control
The small differences in parameter values in the above table will have negligible impact on controller design and tuning. In fact, real world process data is imperfect and differences should be expected, even when repeating the identical test on the same process.

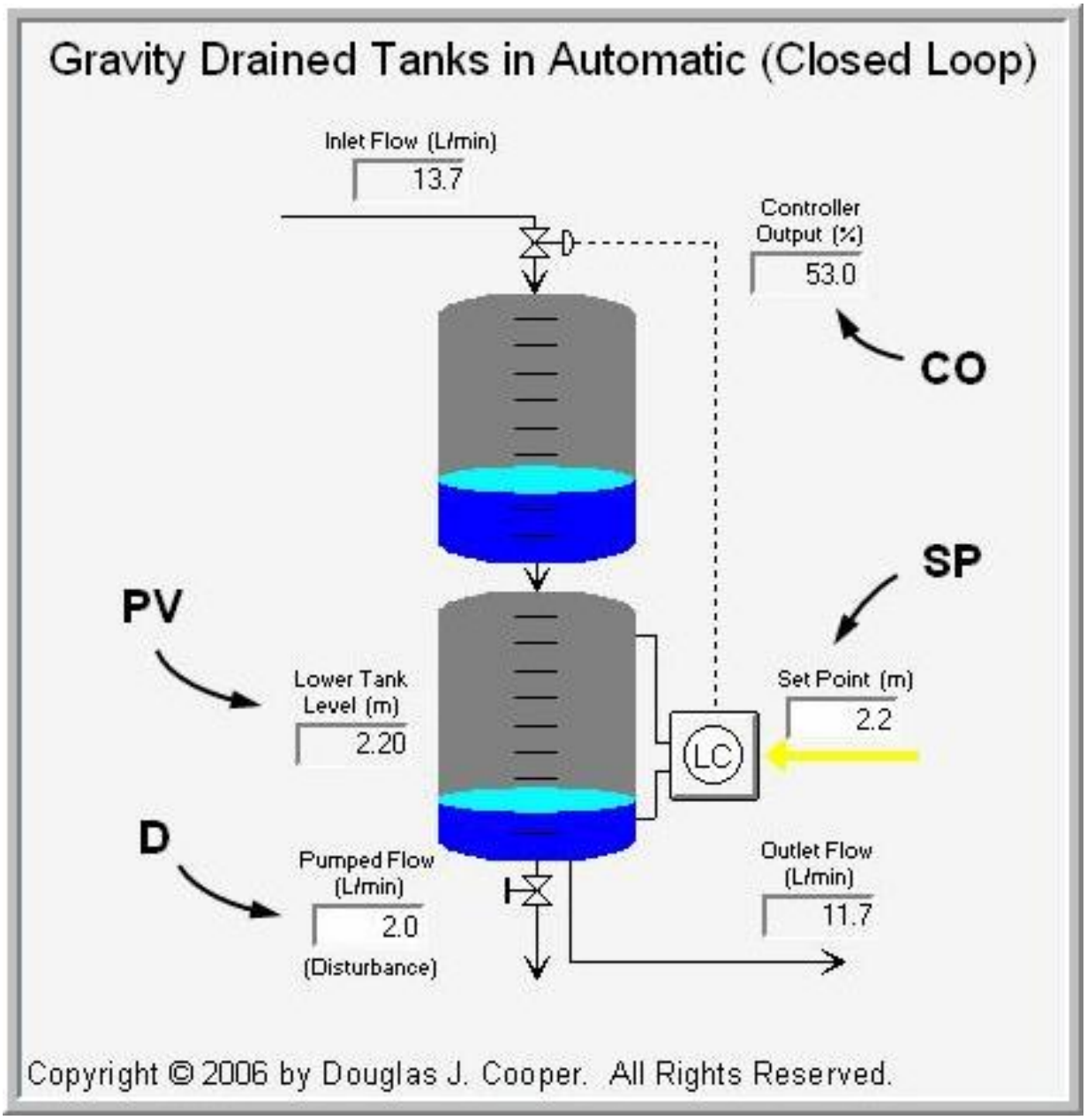
The use of these model parameters for controller design and tuning of the heat exchanger can be found at these links for P-Only, PI, PID and PID with Filter control.

Do Not Model Disturbance Driven Data for Controller Tuning

A fairly common stumbling block for those new to controller tuning relates to step 2 of the controller design and tuning recipe. Step 2 says to "collect controller output (CO) to process variable (PV) dynamic process data around the design level of operation."

But suppose disturbance rejection is our primary control objective (example study here). Shouldn't we then step or pulse (or "bump") our disturbance variable to generate the step 2 dynamic process test data?

As shown below for the gravity drained tanks process, that would involve bumping D, the flow rate of the pumped stream exiting the bottom tank () and modeling the D to PV dynamic relationship:



The short answer is, no. Tuning a feedback controller based on the D to PV dynamic behavior is a path to certain failure.

Wire Out to Wire In

A controller's "world" is wire out to wire in. The CO is the only thing a controller can adjust. The PV is the only thing a controller can "see." The controller sends a CO signal out on one wire. The impact of that action returns as a PV measurement signal on the other wire.

Disturbances, by their very nature, are often unmeasured. Unless a feed forward architecture has been implemented, the controller is only aware of a disturbance when it has already forced the PV from set point (SP). The CO is then the only handle the controller has to correct the problem.

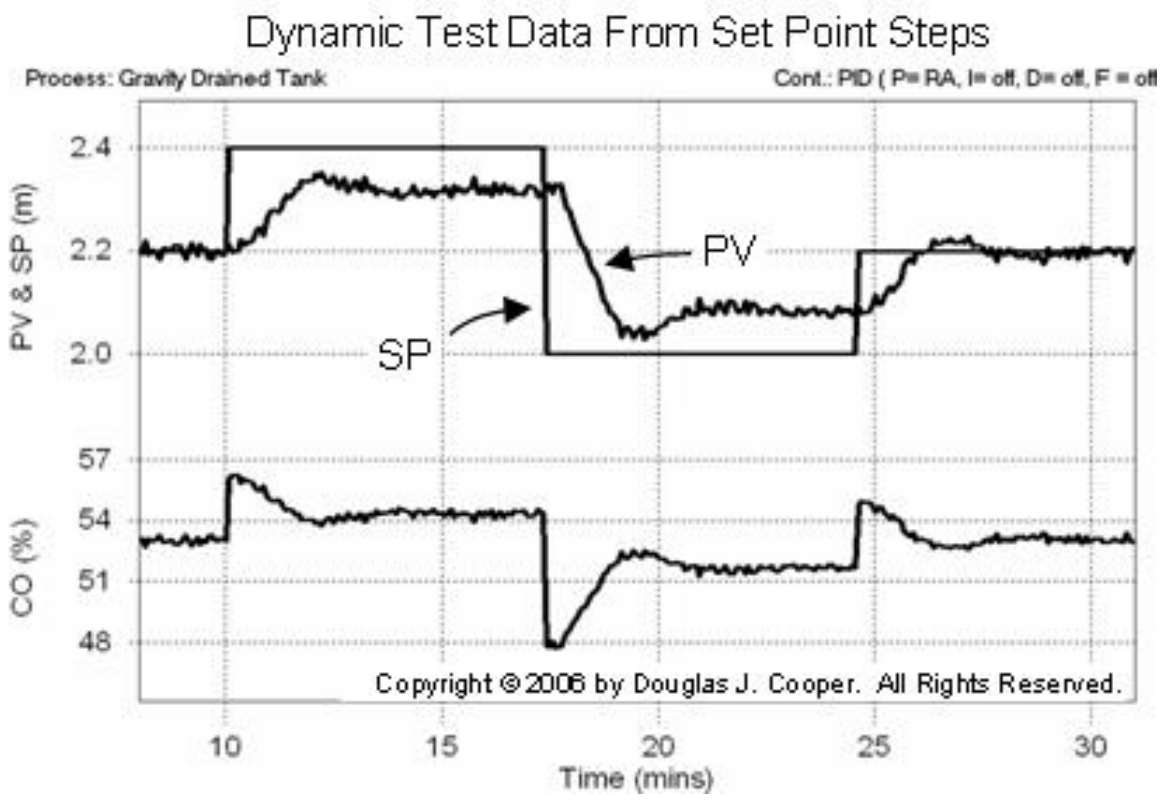
For a controller to take appropriate corrective actions, it must "know" how the PV will respond when it changes the CO. Thus, regardless of the control objective, it is CO to PV relationship that must always be the foundation for controller design and tuning.

Closed Loop Testing
As discussed here, we must use a software tool to fit a model to dynamic process test data that has been collected in automatic mode.

As always, the process must be steady before beginning the test. Also, the controller must be tuned so that the CO actions are energetic enough to force a clear response in the PV, but not be so aggressive that the PV oscillates wildly during data collection.

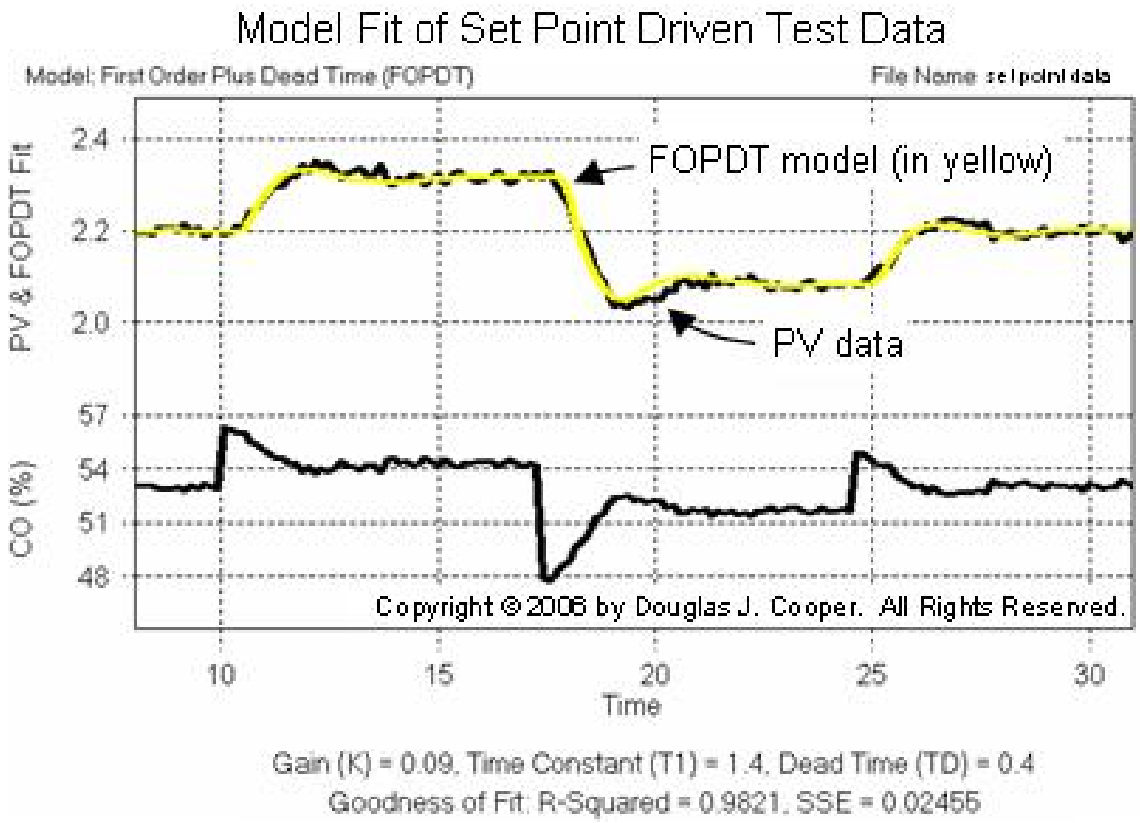
◊ ***SP Driven Dynamic Data is Good***
Useful data can be generated by bumping the set point enough to force a clear dynamic response. Below is data from the gravity drained tanks process under P-Only control using a controller gain, Kc = 16 %/m.

As shown (), the process is initially at a steady operation. The set point is stepped in a doublet, from 2.2 m up to 2.4 m, then down to 2.0 m, and back to the initial 2.2 m. The P-Only controller produces a moderate set point response, with offset displayed as expected from this simple controller. While not shown, the pumped flow disturbance, D, remains constant throughout the experiment.



What is important in the above test is that when the set point is stepped, the P-Only controller is sufficiently active to move the CO in the desired "far enough and fast enough" manner to force a clear response in the PV trace. This obvious CO to PV cause-and-effect relationship is exactly what we require in a good data set.

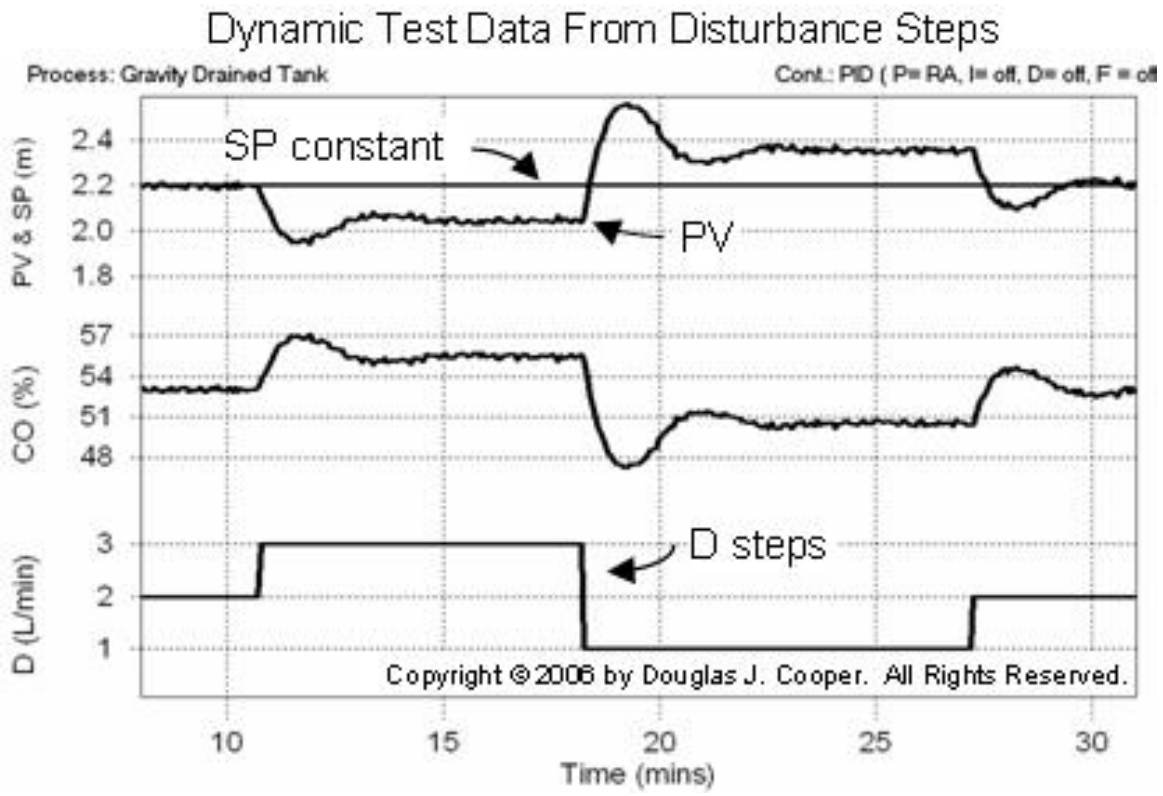
Step 3 of the controller design and tuning recipe is to fit a FOPDT (first order plus dead time) model to the dynamic process test data. Below is the results of this fit () using the above set point driven test data and the Control Station software.



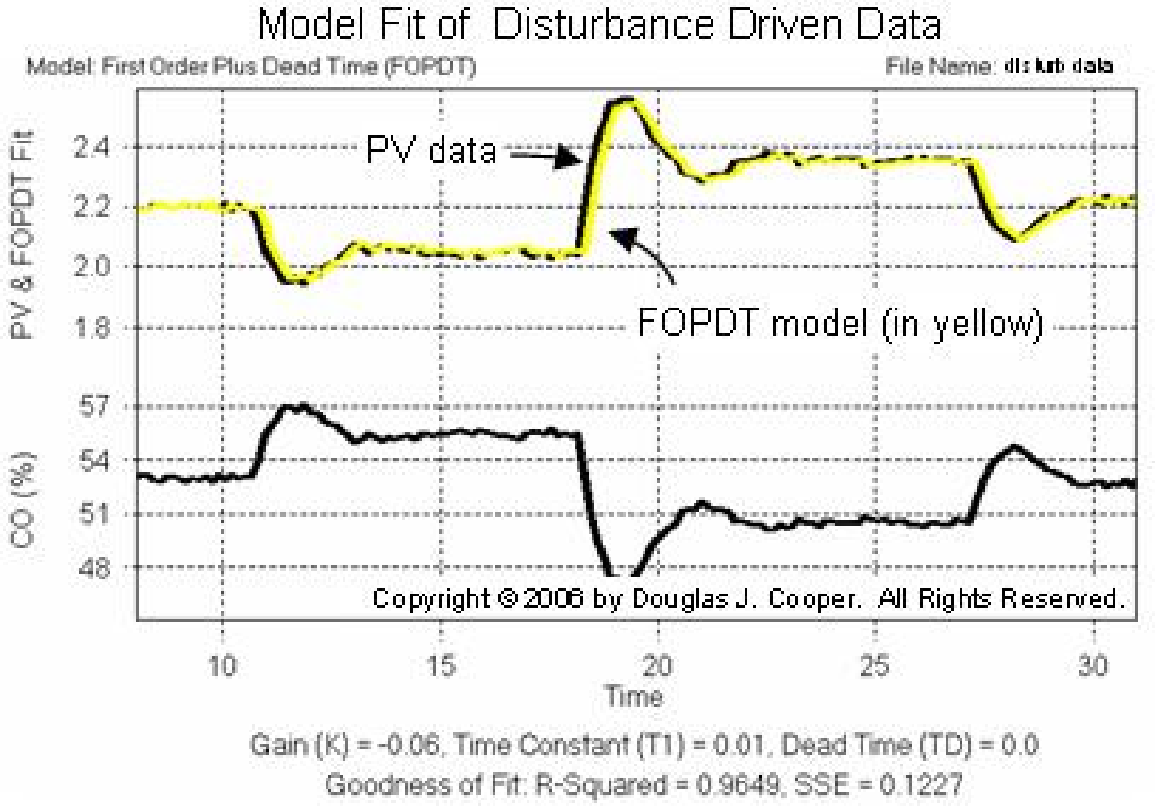
These results will be discussed later in this article.

◊ ***D Driven Dynamic Data is NOT Good***
Next we conduct a dynamic test where the set point remains constant and the dynamic event is forced by changes in D, the pumped flow disturbance.

As shown below (), D is stepped from 2 L/min up to 3 L/min, down to 1 L/min, and back to 2 L/min. The same P-Only controller used above produces a moderate disturbance rejection response with offset (more discussion on P-Only control, disturbance rejection and offset for the gravity drained tanks can be found here).



As per step 3 of the recipe, shown below () is a FOPDT model fit of the dynamic process test data from the above experiment:



It is unfortunate that the model fit looks so good, because it may give us confidence that the design is proceeding correctly.

Comparing Model Fit Results
The table below summarizes our FOPDT model parameters resulting from:

- an open loop step test as described here,
- the closed loop set point driven test shown above,
- the closed loop disturbance driven test shown above.

		--- closed loop tests ---	
	Open Loop Step	**Set Point Driven**	**Disturbance Driven**
Process gain, Kp (m/%)	0.09	0.09	-0.06
Time constant, τp (min)	1.4	1.4	0.01
Dead time, Θp (min)	0.5	0.4	0.0

As expected, the set point driven test produces model parameters that are virtually identical to those of the open loop test. And therefore, the controller design and tuning based on either of these two tests will provide the same desirable performance (PI control study using these parameters presented here).

But the disturbance driven model is distressing. The modeling fitting software succeeds in accurately describing the data (and this is a wonderful capability for feed forward control element design), but the parameters of the disturbance driven model are very different from those needed for proper control of the gravity drained tanks.

Perhaps most striking is that the disturbance driven test data yields a negative process gain, Kp. A controller designed from this data would have the wrong action (direct acting or reverse acting). And as a result, the controller would move the valve in the wrong direction, compounding errors rather than correcting for them.

Disturbances Are Always Bad
When generating dynamic process test data, it is essential that the influential disturbances remain quiet. If we are not familiar enough with our process to be sure about such disturbances, it would be best that we not adjust any controller settings.

13) Evaluating Controller Performance

Comparing Controller Performance Using Plot Data

When considering the range of control challenges found across the process industries, it becomes apparent that very different controller behaviors can be considered "good" performance. While one process may be best operated with a fast and aggressive control action, another may be better suited for a slow and gentle response.

Since there is no common definition of what is good or best performance, we are the judge and jury of goodness for our own process.

Performance is a Matter of Application

Suppose our process throughput is known to change quite suddenly because of the unreliable nature of our product filling/packaging stations at the end of our production line. When one of the container filling stations goes down, the upstream process throughput must be ramped down quickly to compensate. And as soon as the problem is corrected, we seek to return to full production as rapidly as possible.

In this application, we may choose to tune our controllers to respond aggressively to sudden changes in throughput demand. We must recognize and accept that the consequence of such aggressive action is that our process variables (PVs) may overshoot and oscillate as they settle out after each event.

Now suppose we work for a bio-pharma company where we grow live cell cultures in large bioreactors. Cells do not do well and can even die when conditions change too quickly. To avoid stressing the culture, it is critical that the controllers move the process variables slowly when they are counteracting disturbances or working to track set point changes.

So good performance can sometimes mean fast and aggressive, and other times slow and gentle.

Sometimes Performance is a Matter of Location

Distillation columns are towers of steel that can be as tall as a 20 story building. They are designed to separate mixtures of liquids into heavier and lighter components. To achieve this, they must run at precise temperatures, pressures and flow rates.

Because of their massive size, they have time constants measured in hours. A disturbance upset in one of the these behemoths can cause fluctuations (called "swings") in conditions that can take the better part of a day to settle out. If the plant is highly integrated with streams coming and going from process units scattered across the facility, then a distillation column swinging back and forth for hours will cause the operation of units throughout the plant to be disrupted.

So the rule of thumb in such an integrated operation is that the columns are king. If we are involved in operating a process upstream of a distillation column, we do everything in our power to contain all disruptions within our unit. Good control means keeping our problems to ourselves and avoiding actions that will impact the columns. We do this no matter what havoc it creates for our own production.

And if we are involved in operating a process downstream from a column, our life can be similarly miserable. The column controllers are permitted to take whatever action is necessary to keep the columns running smoothly. And this means that streams flowing out of the column can change drastically from moment to moment as the controllers fight to maintain balance.

A Performance Checklist

As these examples illustrate, good control performance is application specific. Ultimately, we define "best" based on our own knowledge and experience. Things we should take into account as we form our judgments include what our process is physically able to achieve, how this fits into the bigger safety and profitability picture of the facility, and what management has planned.

Thus, our performance checklist requires consideration of the:

- goals of production
- capabilities of the process
- impact on down stream units
- desires of management

Controller Performance from Plot Data

If we limit ourselves to judging the performance of a specific controller on one of our own processes, we can compare response plots side-by-side and make meaningful statements about which result is "better" as we define it.

There are more precise terms beyond “oscillates a lot” or “responds quickly.” We explore below several criteria that are computed directly from plot data. The more common terms in this category include:

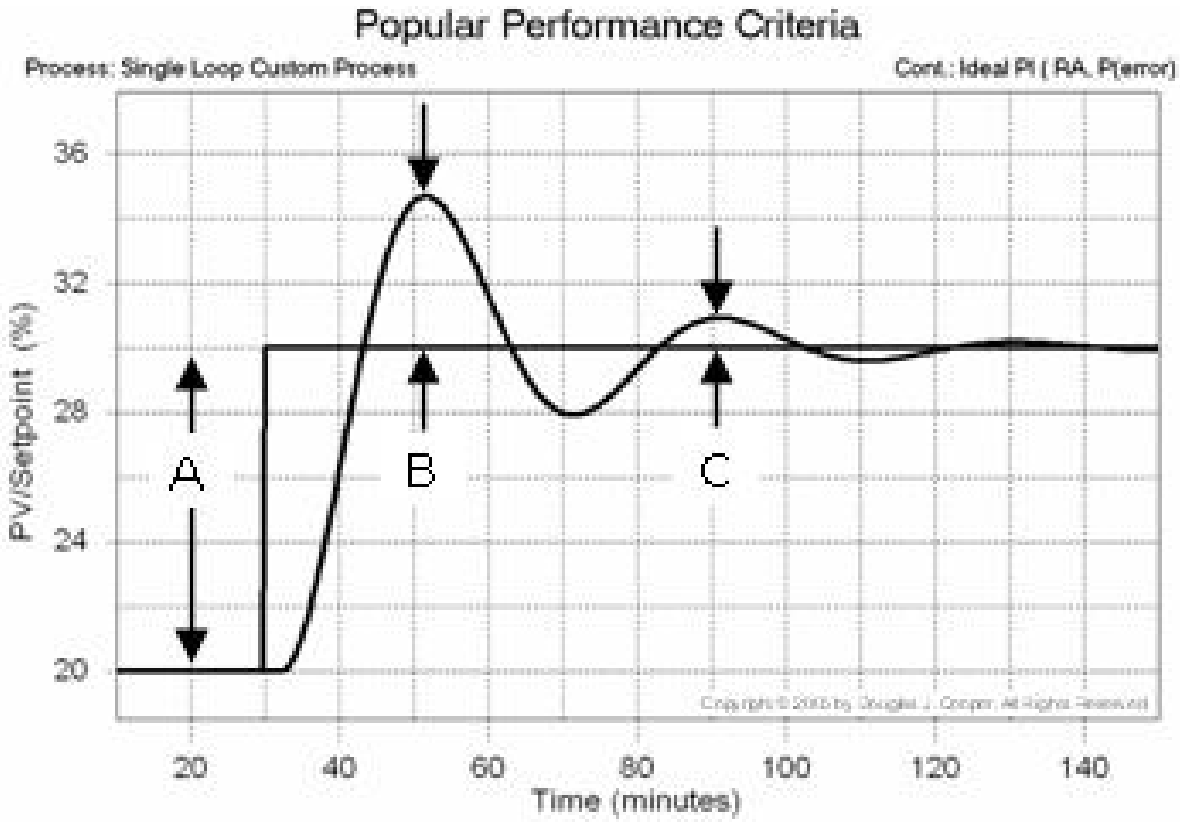
- rise time
- peak overshoot ratio
- settling time
- decay ratio

These and like terms permit us to make orderly comparisons among the range of performance available to us.

Peak Related Criteria

Below is a set point step response plot () with labels indicating peak features:

A = size of the set point step
B = height of the first peak
C = height of the second peak



The popular peak related criteria include:
- Peak Overshoot Ratio (POR) = B/A
- Decay Ratio = C/B

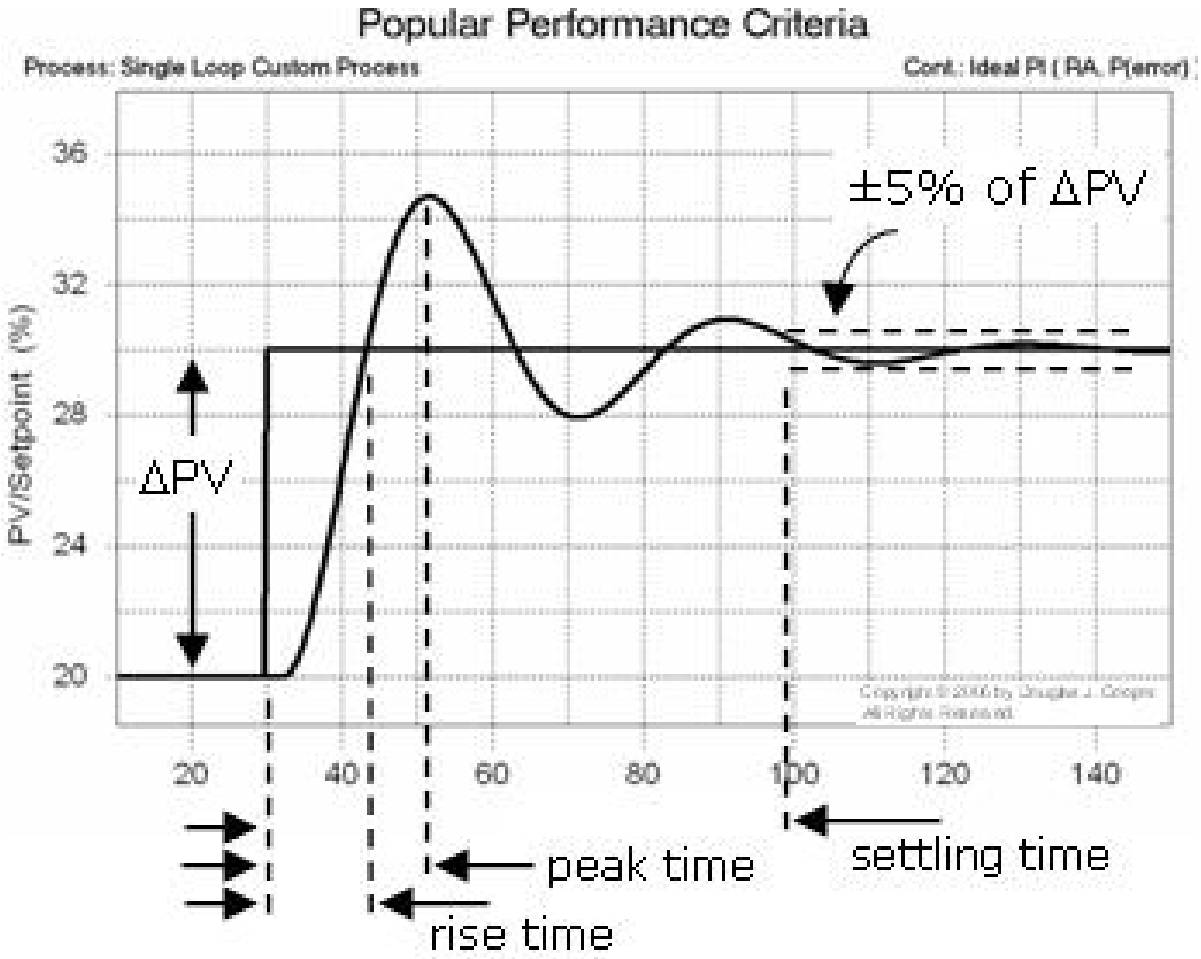
In the plot above, the PV was initially at 20% and a set point step moves it to 30%. Applying the peak related criteria by reading off the PV axis:
A = (30 - 20) = 10%
B = (34.5 – 30) = 4.5%
C = (31 – 30) = 1%

And so for this response:
POR = 4.5/10 = 0.45 or 45%
Decay ratio = 1/4.5 = 0.22 or 22%

An old rule of thumb is that a 10% POR and 25% decay ratio (sometimes called a quarter decay) are popular values. Yet in today's industrial practice, many plants require a "fast response but no overshoot" control performance.

No overshoot means no peaks, and thus, B = C = 0. This increasingly common definition of "good" performance means the peak related criteria discussed above are not useful or sufficient as performance comparison measures.

Time Related Criteria
An additional set of measures focus on time-related criteria. Below is the same set point response plot () but with the time of certain events labeled:



The clock for time related events begins when the set point is stepped, and as shown in the plot, include:

- Rise Time = time until the PV first crosses the set point
- Peak Time = time to the first peak
- Settling Time = time to when the PV first enters and then remains within a band whose width is computed as a percentage of the total change in PV (or □PV).

The 5% band used to determine settling time in the plot above was chosen arbitrarily. As we will see below, other percentages are equally valid depending on the situation.

From the plot, we see that the set point is stepped at time t = 30 min. The time related criteria are then computed by reading off the time axis as:

Rise Time = (43 – 30) = 13 min
Peak Time = (51 – 30) = 21 min
Settling Time = (100 – 30) = 70 min for a ±5% of □PV band

When There is No Overshoot

We should recognize that the peak and time criteria are not independent:

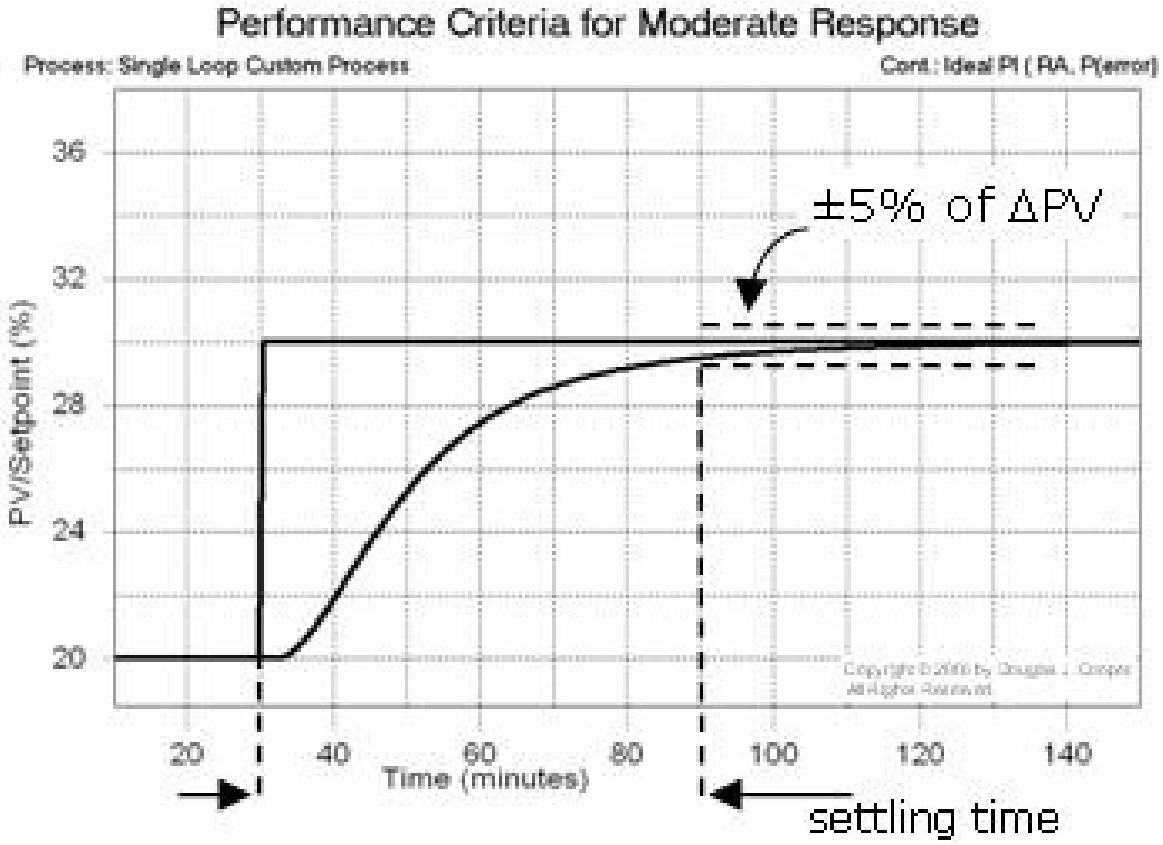
- a process with a large decay ratio will likely have a long settling time.
- a process with a long rise time will likely have a long peak time.

And in situations where we seek moderate tuning with no overshoot in our response plots, there is no peak overshoot ratio, decay ratio or peak time to compute. Even rise time, with

its asymptotic approach to the new steady state, is a measure of questionable value.

In such cases, settling time, or the time to enter and remain within a band of width we choose, still remains a useful measure.

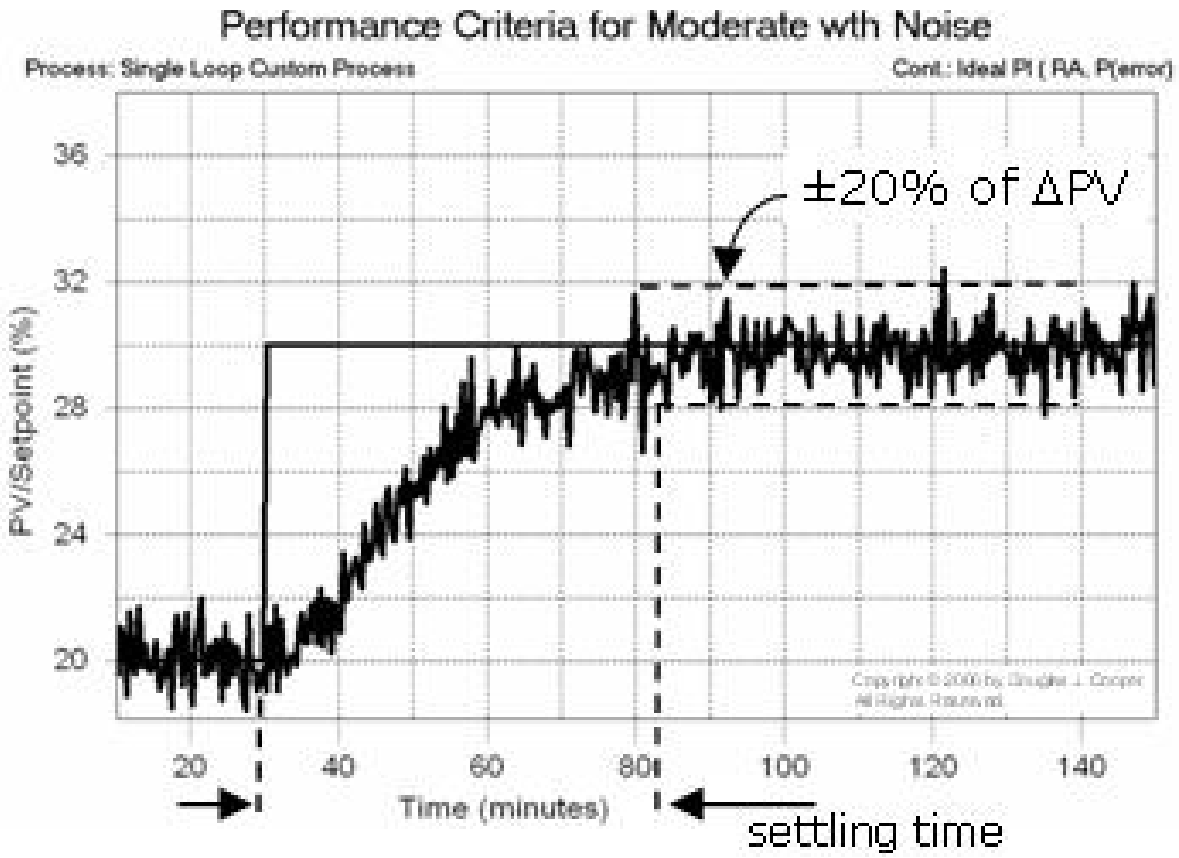
The plot below () shows the identical process as that in the previous plots. The only difference is that in this case, the controller is tuned for a moderate response.



We compute for this plot:
Settling Time = (90 – 30) = 60 min for a ±5% of □PV band

Isn't it interesting that a more moderately tuned controller has a faster settling time then the more active or aggressive response shown previously where settling time was 70 minutes? (In truth, this is not a surprising result because for the previous plots, the controller was deliberately mistuned to provide the sharp peaks we needed to clearly illustrate the performance definitions.)

Settling Band is Application Specific
Measurement noise, or random error in the PV signal, can be one reason to modify the width of the settling band. As shown below (), for the identical process and tuning as used in the above moderate response plot, but with significant noise added to the PV, the settling band must to be widened considerably to provide a meaningful measure.



Widening the band in this case is not a bad thing. It is simply necessary. And since we will (presumably) use the same settling band criteria when exploring alternatives for this loop, then it remains a useful tool for comparing performance.

Other Measures

The methods discussed in this article use data from a set response plot to make decisions about the performance of a specific loop in our plant. The methods require some judgment and we must be sure to be consistent for our evaluations to have value.

There are other performance tools and measures that require software to compute. These include moving average, moving variance, cross and auto correlation, power spectrum and more. These will be discussed in a future article.

IV. Control of Integrating Processes (by Doug Cooper & Bob Rice)

14) Integrating (Non-Self Regulating) Processes

Recognizing Integrating (Non-Self Regulating) Process Behavior

By Bob Rice[1] and Doug Cooper

The case studies on this site largely focus on the control of self regulating processes. The principal characteristic that makes a process self regulating is that it naturally seeks a steady state operating level if the controller output and disturbance variables are held constant for a sufficient period of time.

Cruise control of a car is a self regulating process. If we keep the fuel flow to the engine constant while traveling on flat ground on a windless day, the car will settle out at some constant speed. If we increase the fuel flow rate a fixed amount, the car will accelerate and then steady out at a different constant speed.

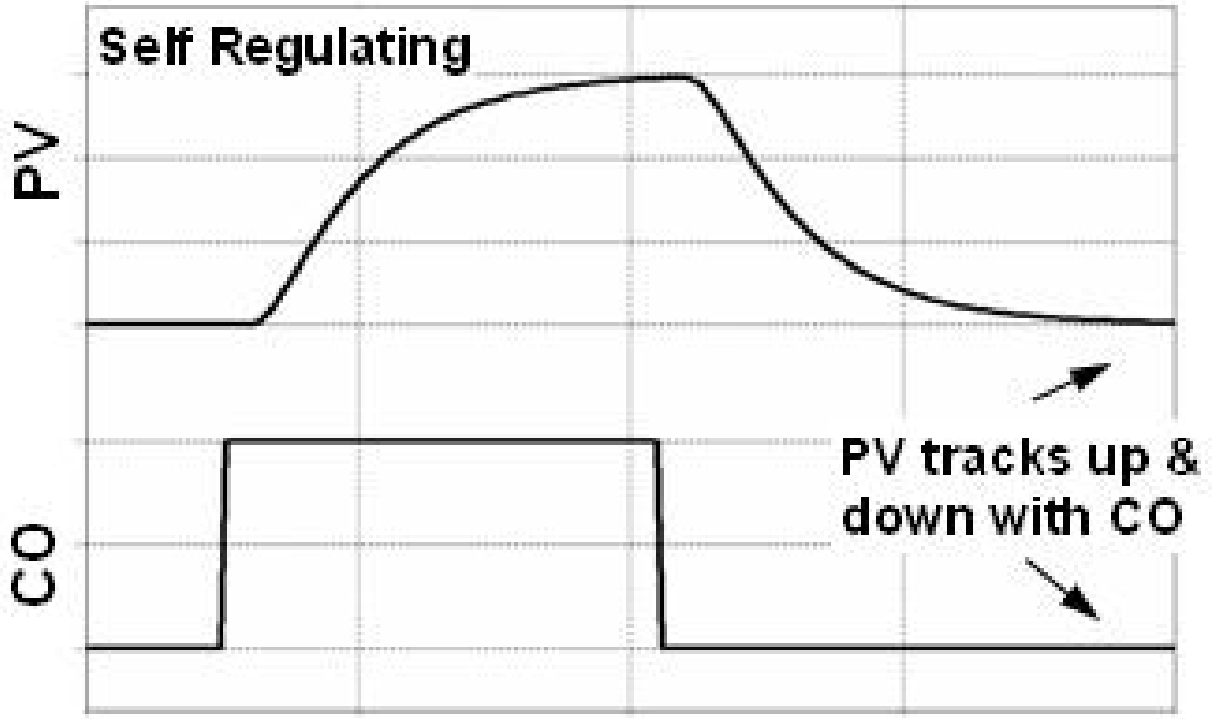
The heat exchanger process that has been studied on this site is self regulating. If the exchanger cooling rate and disturbance flow rate are held constant at fixed values, the exit temperature will steady at a constant value. If we increase the cooling rate, wait a bit, and then return it to its original value, the exchanger exit temperature will respond during the experiment and then return to its original steady state.

But some processes where the streams are comprised of gases, liquids, powders, slurries and melts do not naturally settle out at a steady state operating level. Process control practitioners refer to these as non-self regulating, or more commonly, as integrating processes.

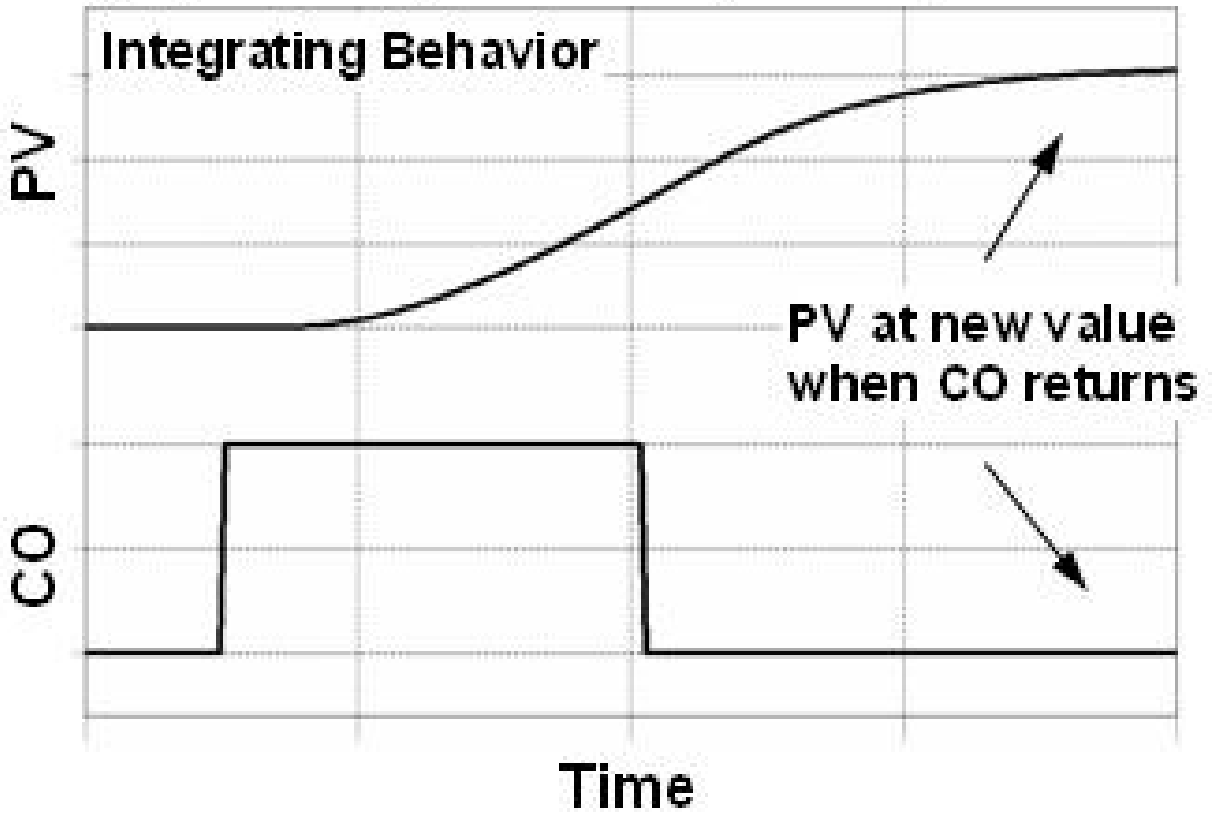
Integrating (non-self regulating) processes can be remarkably challenging to control. After exploring the distinctive behaviors illustrated below, you may come to realize that some of the level, temperature, pressure, pH and other loops you work with have such a character.

Integrating (Non-Self Regulating) Behavior in Manual Mode

The upper plot below shows the open loop (manual mode) behavior of the more common self regulating process. In this idealized response, the controller output (CO) and process variable (PV) are initially at steady state. The CO is stepped up and back from this steady state. As shown, the PV responds to the step, but ultimately returns to its original operating level.



Copyright © 2006 by Control Station, Inc. All Rights Reserved.



The lower plot above shows the open loop response of an ideal integrating process. The distinctive behavior is that the PV settles at a new operating level when the CO returns to its original value.

In truth, the integrating behavior plot above is misleading in that it implies that for such processes, a steady CO will produce a steady PV. While possible with idealized

simulations like that used to generate the plot, such a "balance point" behavior is rarely found in open loop (manual mode) for integrating processes in an industrial operation.

More realistically, if left uncontrolled, the lack of a balance point means the PV of an integrating process will naturally tend to drift up or down, possibly to extreme and even dangerous levels. Consequently, integrating processes are rarely operated in manual mode for very long.

Aside: the behavior shown in the integrating plot can also appear across small regions of operation of a self-regulating process if there is a significant dead-band in the final control element (FCE). This might result, for example, from loose mechanical linkages in a valve. For this investigation, we assume the FCE operates properly.

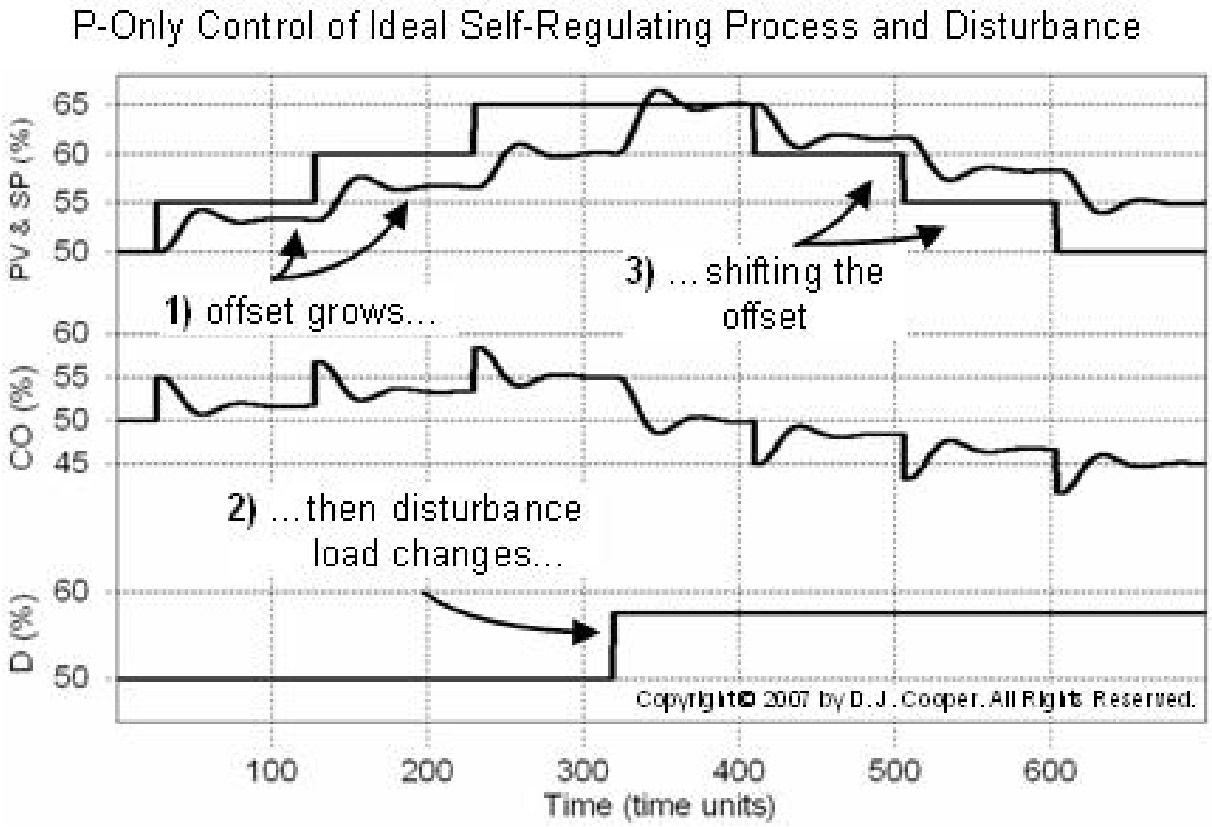
P-Only Control Behavior is Different

To appreciate the difference in behavior for integrating processes, we first recall P-Only control of a self regulating process (theory discussion here and application case studies here and here). We summarize some of the lessons learned in those articles by presenting the P-Only control of an ideal self regulating simulation.

As shown below (), when the set point (SP) is initially at the design level of operation (DLO) in the first moments of operation, then PV equals SP. Recall that the DLO is where we expect the SP and PV to be during normal operation when the major disturbances are at their normal or typical values.

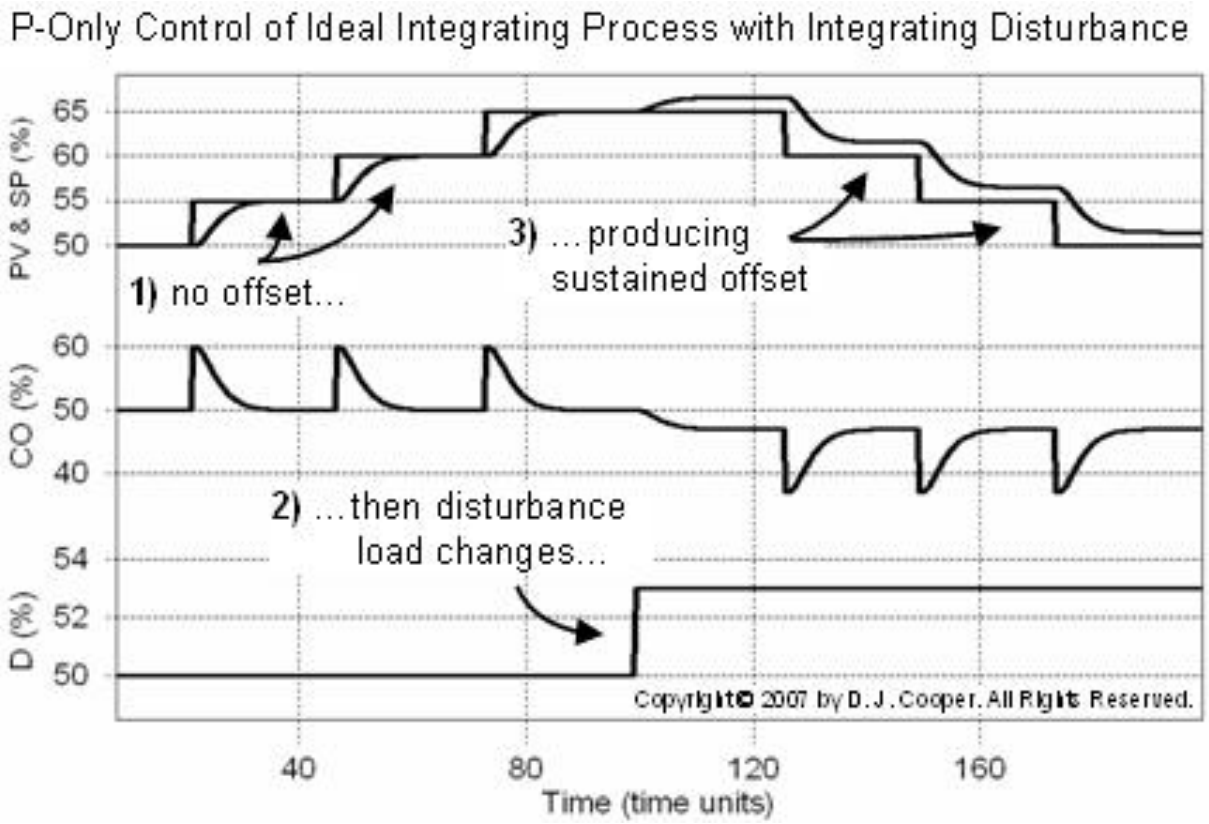
The set point is then stepped up from the DLO on the left half of the plot. The simple P-Only controller is unable to track the changing SP and a steady error, called offset, results. The offset grows as each step moves the SP farther away from the DLO.

Midway through the plot, a disturbance occurs. Its size was pre-calculated in this ideal simulation to eliminate the offset. The SP is then stepped back down to the right in the plot and we see that offset shifts, but again grows in a similar and predictable pattern.



With this as background, we next consider an ideal integrating process simulation under P-Only control as shown below ().

Even under simple P-Only control as shown in the left half of the plot, the PV is able to track the SP steps with no offset. This behavior can be quite confusing as it does not fit the expected behavior we have just seen above for the more common self regulating process.



The reason this happens is that integrating processes have a natural accumulating character. In fact, this is why “integrating process” is used as a descriptor for non-self regulating processes. Since the process integrates, then it appears that the controller does not need to.

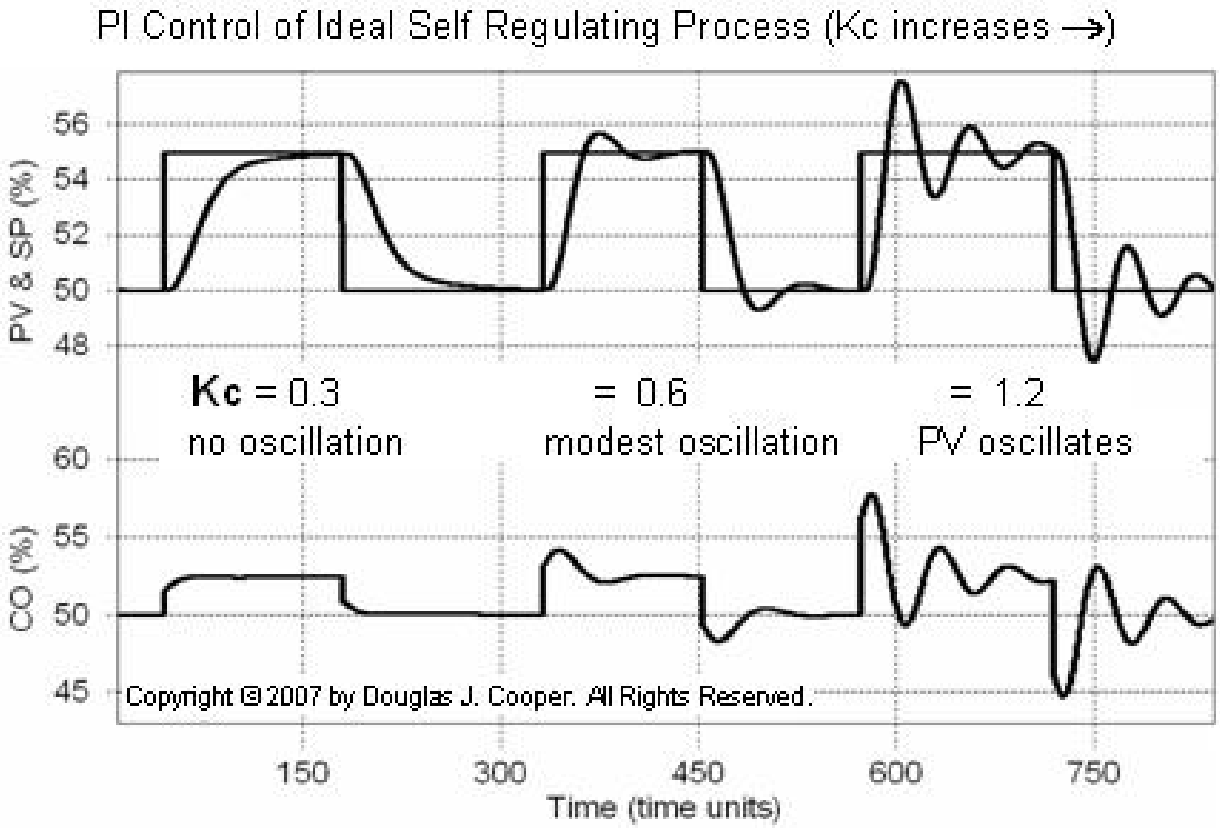
Yet the set point steps to the right in the above plot show that this is not completely correct. Once a disturbance shifts the baseline operation of the process, shown roughly at the midpoint in the above plot, an offset develops and remains constant, even as SP returns to its original design value.

Controller Output Behavior is Telling
If we study the CO trace in the two plots above, we see one feature that distinguishes self regulating from integrating process behavior.

In the self regulating process plot above, the average CO value tracks up and then down as the SP steps up and then down.

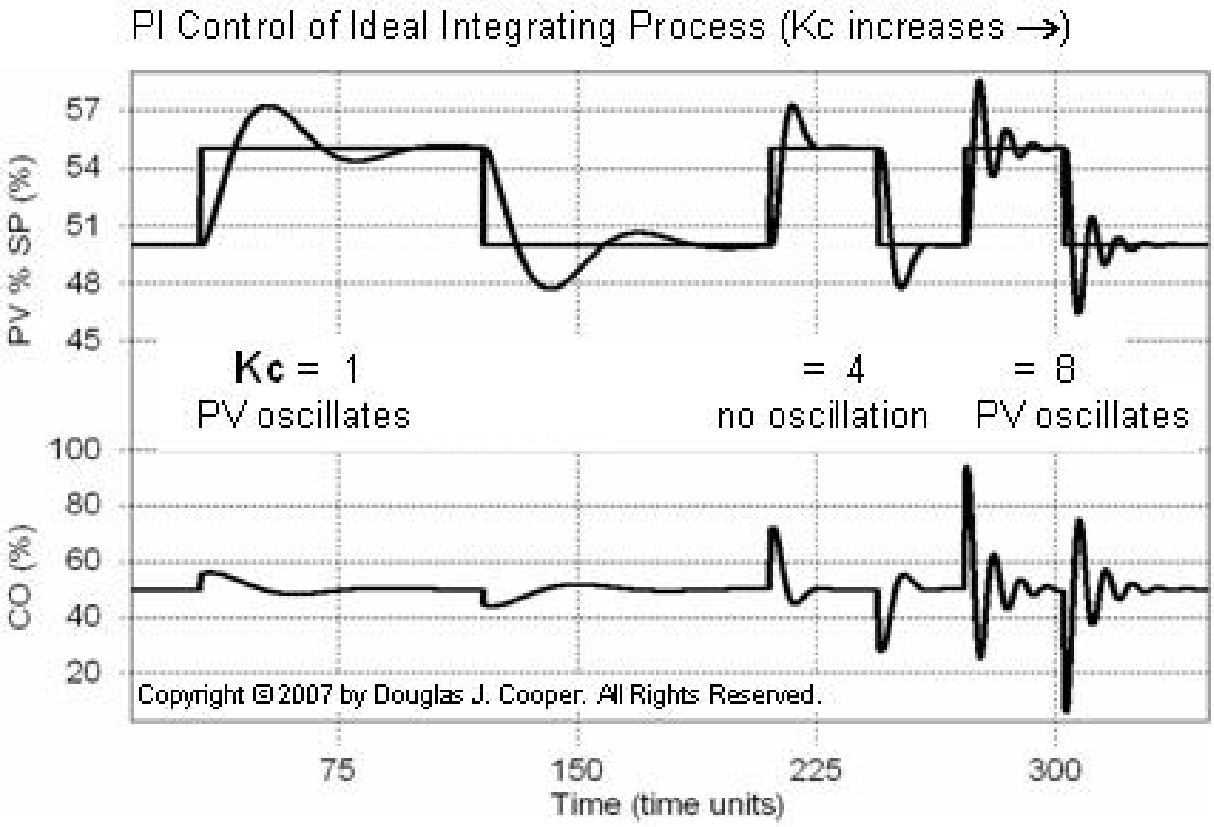
In the integrating process plot above, the CO spikes with each SP step, but then in a most unintuitive fashion, returns to the same steady value. It is only the change in the disturbance flow that causes the average CO to shift midway through the plot, though it then remains centered around the new value for the remainder of the SP steps.

PI Control Behavior is Different
The plot below () shows the ideal self regulating process controlled using the popular dependent ideal PI algorithm. Reset time, *T*i, is held constant throughout the experiment while controller gain, Kc, is doubled and then doubled again.



As Kc increases, the controller becomes more active, and as we have grown to expect, this increases the tendency of the PV to display oscillating (or underdamped) behavior.

For comparison, now consider PI control of an ideal integrating process simulation as shown below (). As above, reset time, *T*i, is held constant throughout the experiment while controller gain, Kc, is increased across the plot.



A counter-intuitive result is that as Kc becomes small ***and*** as it becomes large, the PV begins displaying an underdamped (oscillating) response behavior.

While the frequency of the oscillations is clearly different between a small and large Kc when seen together in a single plot as above, it is not always obvious what direction controller gain needs to move to settle the process when looking at such unacceptable performance on a control room display.

Tuning Recipe Required
One of the biggest challenges for practitioners is recognizing that a particular process shows integrating behavior prior to starting a controller design and tuning project. This, like most things, comes with training, experience and practice.

Once in automatic, closed loop behavior of an integrating process can be unintuitive and even confounding. Trial and error tuning can lead us in circles as we try to understand what is causing the problem.

A formal controller design and tuning recipe for integrating processes helps us overcome these issues in an orderly and reliable fashion.

A Design and Tuning Recipe for Integrating Processes

By Bob Rice[1] and Doug Cooper

As has been discussed elsewhere in this e-book, it is best practice to follow a formal recipe when designing and tuning a PID controller. A recipe lets us move a controller into operation quickly. And perhaps most important, the performance of the controller will be

superior to one tuned using an intuitive approach or trial-and-error method.

Additionally, a recipe-based approach overcomes many of the concerns that makes control projects challenging in an industrial operating environment. Specifically, a recipe approach causes less disruption to the production schedule, wastes less raw material and utilities, requires less personnel time, and generates less off-spec product.

The Recipe for Integrating Processes
Integrating (or non-self regulating) processes display counter-intuitive behaviors that make them surprisingly challenging to control. In particular, they do not naturally settle out to a steady operating level if left uncontrolled.

So while the controller design and tuning recipe is generally the same for both self regulating and integrating processes, there are important differences. Specifically, step 3 of the recipe uses a different dynamic model form and step 4 employs different tuning correlations.

Yet the design and tuning recipe maintains the familiar four step structure:

1. Establish the design level of operation (the normal or expected values for set point and major disturbances).
2. Bump the process and collect controller output (CO) to process variable (PV) dynamic process data around this design level.
3. Approximate the process data behavior with a ***first order plus dead time integrating*** (FOPDT Integrating) dynamic model.
4. Use the model parameters from step 3 in rules and correlations to complete the controller design and tuning.

It is important to recognize that real processes are more complex than the simple FOPDT Integrating model form used in step 3. In spite of this, the FOPDT Integrating model succeeds in providing an approximation of process behavior that is sufficiently accurate to yield reliable and predictable control performance when used with the rules and correlations in step 4 of the recipe.

The FOPDT Integrating Model
We recall that the familiar first order plus dead time (FOPDT) dynamic model used to approximate self regulating dynamic process behavior has the form:

$$Tp\frac{dPV(t)}{dt} + PV(t) = Kp \cdot CO(t - \theta p)$$

FOPDT Form

Yet this model cannot describe the kind of integrating process behavior shown in these examples. Such behavior is better described with the ***FOPDT Integrating model*** form:

$$\frac{dPV(t)}{dt} = Kp^{*} \cdot CO(t - \theta p)$$

FOPDT Integrating Form

It is interesting to note when comparing the two models above that the FOPDT Integrating form does not have the lone "+ PV" term found on the left hand side of the FOPDT dynamic model.

Also, individual values for the familiar process gain, Kp, and process time constant, *T*p, are not separately identified for the FOPDT Integrating model. Instead, an integrator gain, Kp*, is defined that has units of the ratio of the process gain to the process time constant, or:

$$Kp^* [=] \frac{Kp}{Tp} \quad \text{or} \quad Kp^* [=] \ PV/(CO \cdot time)$$

Tuning Correlations for Integrating Processes
Analogous to the FOPDT investigations on this site (e.g., here and here), we will see that the FOPDT Integrating model parameters Kp* and Θp of Step 3 can be computed using a graphical analysis of plot data or by automated analysis using commercial software.

Step 4 then provides tuning values for controllers such as the dependent, ideal PI form:

$$CO = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{Ti} \int e(t)dt$$

and the dependent, ideal PID form:

$$CO = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{Ti} \int e(t)dt - Kc \cdot Td \frac{dPV}{dt}$$

One important difference about integrating processes is that since there is no identifiable process time constant in the FOPDT Integrating model, we use dead time, Θp, as the baseline marker of time in the design and tuning rules.

Specifically, Θp is used as the basis for computing sample time, T, and the closed loop time constant, *T*c. Following the procedures widely discussed on this site for self regulating processes (e.g. here and here), we employ a rule to compute the closed time constant, *T*c, as:

$$Tc = 3\Theta p$$

The controller tuning correlations for integrating processes use this *T*c, as well as the Kp* and Θp from the FOPDT integrating model fit, as:

	Controller Gain Kc	Reset Time T_i	Deriv Time T_d
PI	$\frac{1}{K_p}\frac{2T_c+\theta_p}{(T_c+\theta_p)^2}$	$2T_c+\theta_p$	
PID	$\frac{1}{K_p}\frac{2T_c+\theta_p}{(T_c+0.5\theta_p)^2}$	$2T_c+\theta_p$	$\frac{0.25\theta_p^2+T_c\theta_p}{2T_c+\theta_p}$

Loop Sample Time, T
Determining a proper sample time, T, for integrating processes is somewhat more challenging than for self regulating processes.

As discussed in this article, there are two sample times, T, used in process controller design and tuning. One is the control loop sample time that specifies how often the controller samples the measured process variable (PV) and computes and transmits a new controller output (CO) signal. The other is the rate at which CO and PV data are sampled and recorded during a bump test (step 2 of the recipe).

Also discussed in that article is that all controllers measure, act, then wait until next sample time before repeating the loop. This "measure, act, wait" procedure has a delay (or dead time) of one sample time built naturally into its structure. Thus, the minimum dead time ($\Theta_{p,min}$) in any control loop is the loop sample time, T.

With this information, we recognize a somewhat circular argument in defining sample time for integrating processes:

- Our time basis for controller design is θ_p, and as such, the loop sample time, T, be small relative to dead time, or:

$$T \le 0.1\Theta_p$$

- But the minimum that dead time can be is one sample time, T, or

$$\Theta_{p,min} = T$$

Thus, T is based on Θ_p, and if the process is sampled too slowly during a bump test, then Θ_p can be based on T.

To avoid this issue, it is best practice to sample the process as fast as reasonably possible during bump tests so accurate model parameters can be determined during analysis. Loop sample time, T, can then be computed from dead time for controller implementation.

If there is concern about a particular analysis, an alternative and generally conservative way to compute sample time is:

$$T = 0.1 \frac{(PV_{max} - PV_{min})}{(CO_{max} - CO_{min})} \cdot \frac{1}{|Kp^*|}$$

where the subscripts max and min refer to the maximum and minimum values for CO and PV across the signal span of the instrumentation.

Using the Recipe
The tuning recipe for integrating processes has important differences from that used for self regulating process. When designing and tuning controllers for such processes, we should:

- use an FOPDT Integrating model form when approximating dynamic model behavior,
- note that the closed loop time constant, *T*c, and sample time, T, are based on model dead time, Θp.
- employ PI and PID tuning correlations specific to integrating processes.

Analyzing Pumped Tank Dynamics with a FOPDT Integrating Model

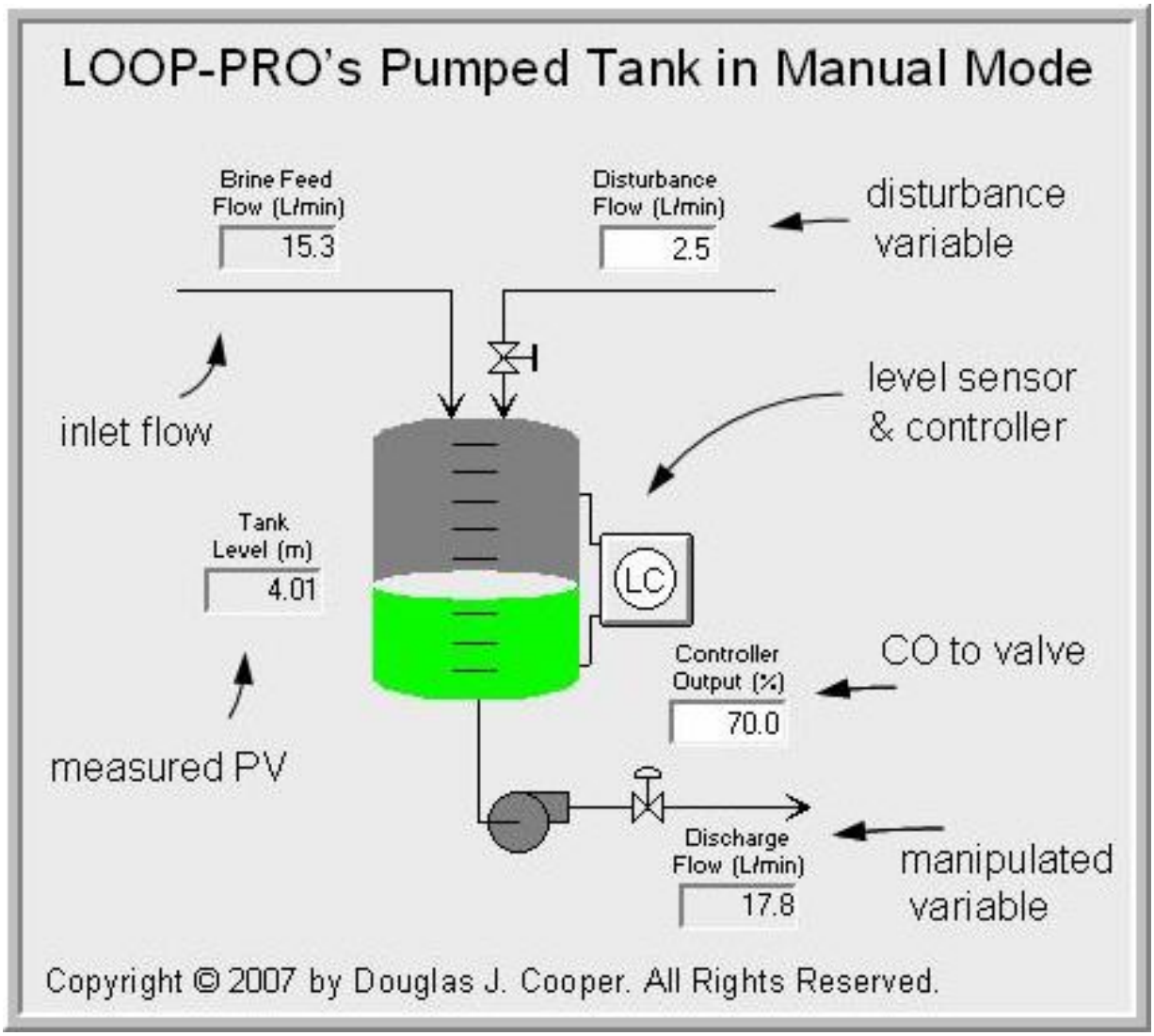
By Bob Rice[1] and Doug Cooper

Integrating (or non-self regulating) processes display counter-intuitive behaviors that make them surprisingly challenging to control. In particular, they do not naturally settle out to a steady operating level if left uncontrolled.

To address this distinctive dynamic character, we modify the controller design and tuning recipe to include a FOPDT Integrating model and slightly different design rules and tuning correlations as discussed here.

The Pumped Tank Process
To better understand the design and tuning of a PID controller for an integrating process, we explore the pumped tank case study from Control Station's Loop-Pro software.

As shown below in manual mode (), the process has two liquid streams feeding the top of the tank and a single exit stream pumped out the bottom.



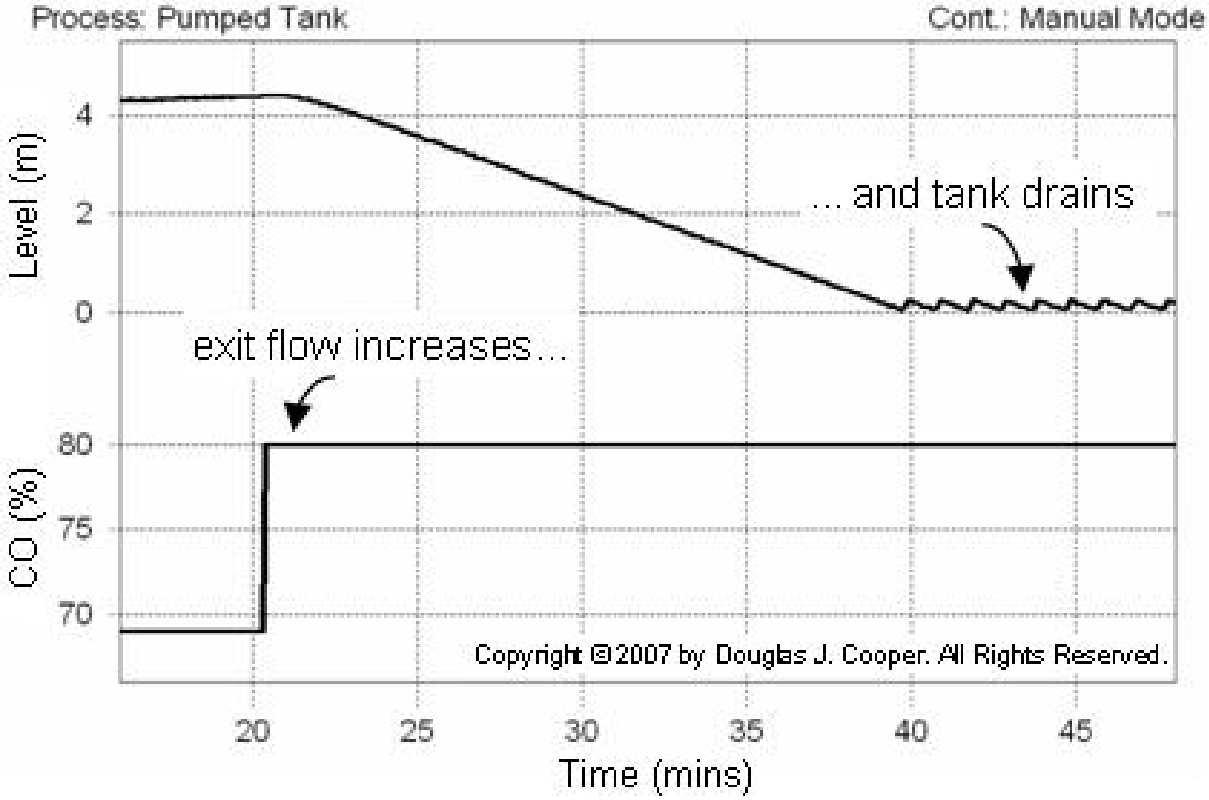
As labeled in the figure, the measured process variable (PV) is liquid level in the tank. To maintain level, the controller output (CO) signal adjusts a throttling valve at the discharge of a constant pressure pump to manipulate flow rate out of the bottom of the tank. This approximates the behavior of a centrifugal pump operating at relatively low throughput.

Unlike the gravity drained tanks case study where the exit flow rate increases and decreases as tank level rises and falls, the discharge flow rate here is strictly regulated by a pump. As a consequence, the physics do not naturally work to balance the system when any of the stream flow rates change.

This lack of a natural balancing behavior is why the pumped tank is classified as an integrating process. If the total flow into the tank is greater than the flow pumped out, the liquid level will rise and continue to rise until the tank fills or a stream flow changes. If the total flow into the tank is less than the flow pumped out, the liquid level will fall and continue to fall.

Below is a plot of the pumped tank behavior with the controller in manual mode (open loop). The CO signal is stepped up, increasing the discharge flow rate out of the bottom of the tank. The flow out becomes greater that the total feed into the top of the tank and as

shown, liquid level begins to fall.

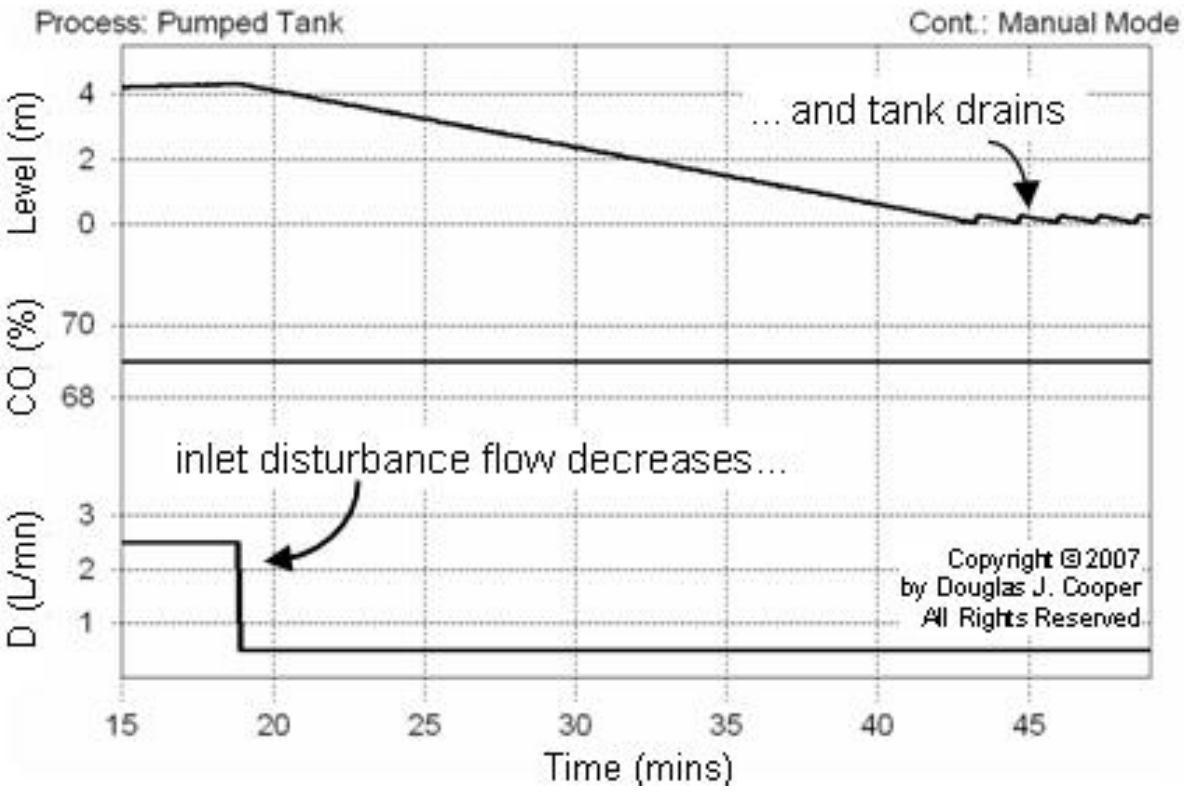


As the situation persists, liquid level continues to fall until the tank is drained. The saw-toothed effect shown when the tank is empty is because the pump briefly surges every time enough liquid accumulates for it to regain suction.

Not shown is that if the controller output were to be decreased enough to cause flow rate out to be less than flow rate in, the tank level would rise until full. If this were a real process, the tank would overflow and spill, creating safety and profitability issues.

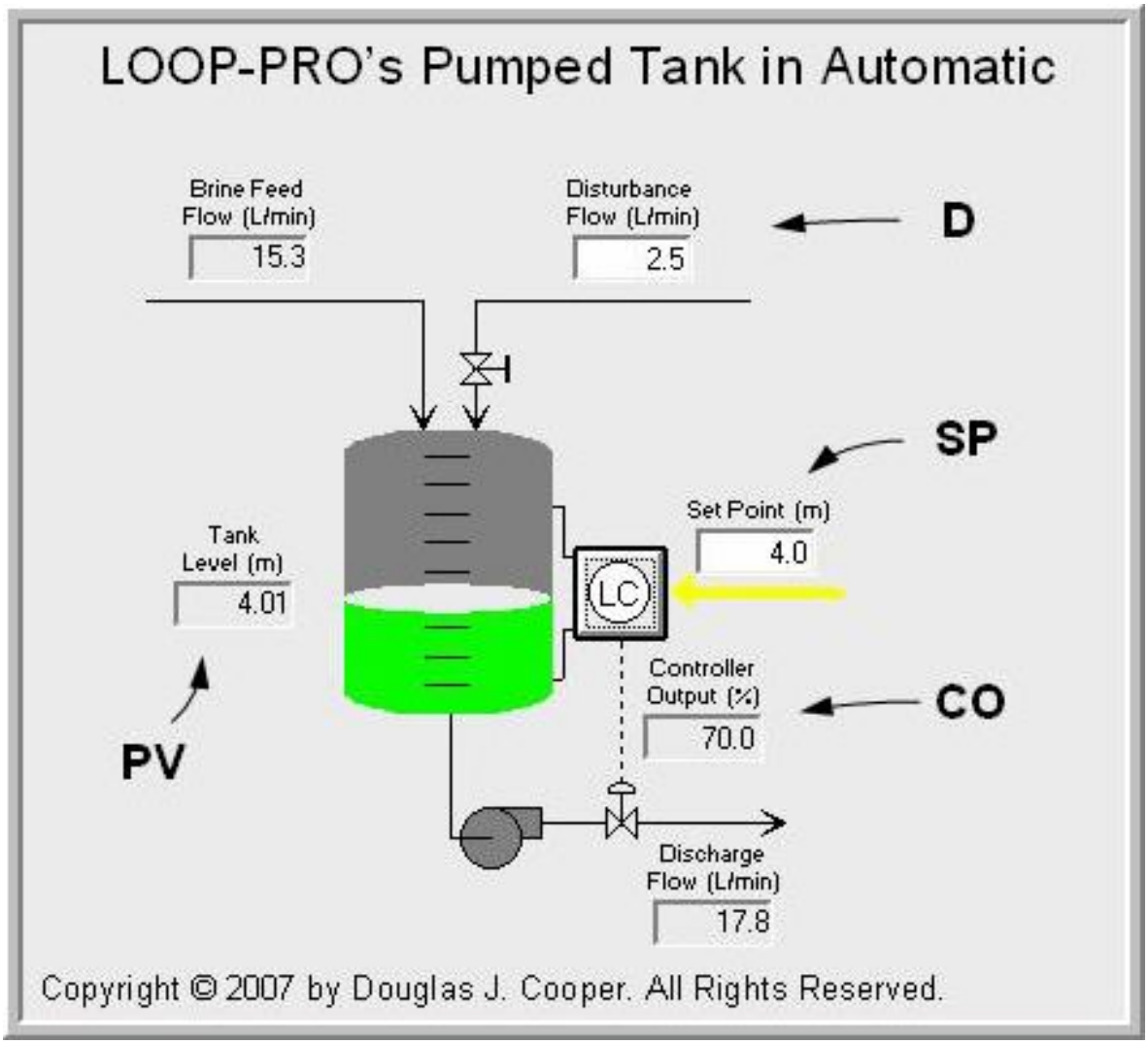
The Disturbance Stream
As shown in the process graphic at the top of this article, the disturbance variable is a flow rate of a secondary feed into the top of the tank. This disturbance flow (D) is controlled independently, as if by another process (which is why it is a disturbance to our process).

When D decreases (or increases), the measured PV level falls (or rises) in response. To illustrate, the plot below shows that the CO is held constant and D is decreased. Characteristic of an integrating process, the PV (tank level) starts falling because total flow into the tank is less than that pumped out. Like the case discussed above, the level continues to fall until the tank is drained.



Pumped Tank in Closed Loop

The process graphic below () shows the pumped tank in automatic mode (closed loop). The two streams feeding into the top of the tank total 17.8 L/min and level is steady because the controller regulates the discharge flow to this same value.



Graphical Modeling of Integrating Process Data

When collecting and analyzing data as detailed in steps 1 and 2 of the design and tuning recipe for integrating processes, we begin by specifying a design level of operation (DLO). In this study, we specify:

- Design PV = 4.8 m
- Design D = 2.5 L/min

Note that while not shown in the plots, D is held constant at 2.5 L/min throughout this study.

Just as with self regulating processes, step 3 of the recipe uses a simplifying dynamic model approximation to describe the complex behavior of a real process. The FOPDT Integrating model is simple in form yet it provides information sufficiently accurate for controller design and tuning.

The FOPDT Integrating model is:

$$\frac{dPV(t)}{dt} = Kp^{*} \cdot CO(t - \theta p)$$

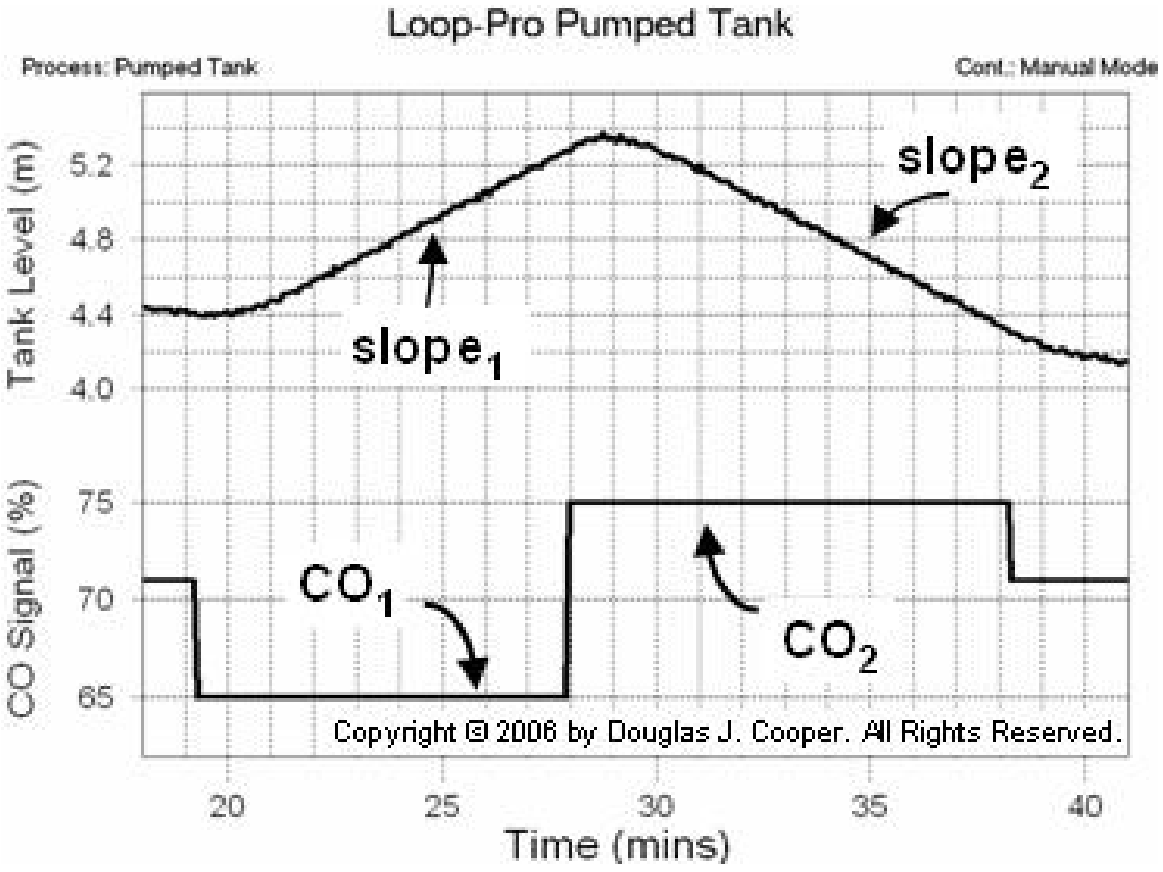
where the integrator gain, Kp*, has units of the ratio of the process gain to the process

time constant, or:

$$Kp^* [=] \frac{Kp}{Tp} \quad \text{or} \quad Kp^* [=] \ PV/(CO \cdot time)$$

The graphical method of fitting a FOPDT Integrating model to process data requires a data set that includes at least two constant values of controller output, CO_1 and CO_2.

As shown below for the pumped tank, both must be held constant long enough so that a slope trend in the PV response (tank liquid level) can be visually identified.



An important difference between the graphical technique for self-regulating processes and integrating processes as discussed here is that integrating processes need not start at a steady value (steady-state) before a bump is made to the CO. The graphical technique discussed here is only concerned with the slopes (or rates of change) in PV and the controller output signal that caused each PV slope.

The FOPDT Integrating model describes the PV behavior at each value of constant controller output CO_1 and CO_2 as:

$$\left.\frac{dPV}{dt}\right|_1 = Kp^* \; CO_1(t - \theta p)$$

and

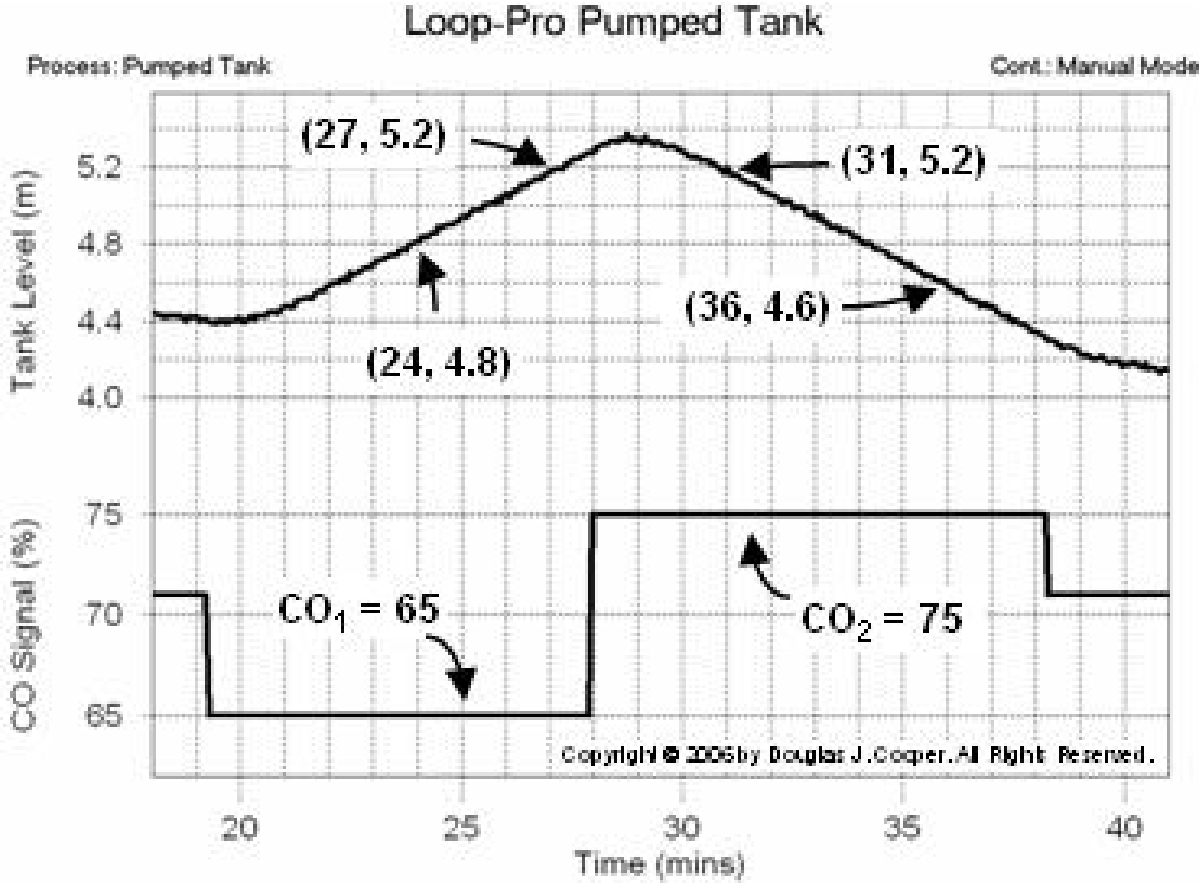
$$\left.\frac{dPV}{dt}\right|_2 = Kp^* \; CO_2(t - \theta p)$$

Subtracting and solving for Kp* yields:

$$Kp^* = \frac{\left.\frac{dPV}{dt}\right|_2 - \left.\frac{dPV}{dt}\right|_1}{CO_2 - CO_1} = \frac{slope_2 - slope_1}{CO_2 - CO_1}$$

Graphical Modeling of Pumped Tank Data

• *Computing Integrator Gain*
Below () is the same open loop data from the pumped tank simulation as shown above. The CO is stepped from 71% down to 65%, causing the liquid level (the PV) to rise. The controller output is then stepped from 65% up to 75%, causing a downward slope in the liquid level.



The slope of each segment is calculated as the change in tank liquid level divided by the change in time. From the plot data we compute:

$$\text{slope}_1 = \left.\frac{dPV}{dt}\right|_1 = \frac{\Delta PV_1}{\Delta t_1} = \frac{5.2 - 4.8}{27 - 24} = 0.13 \ \frac{m}{min}$$

and

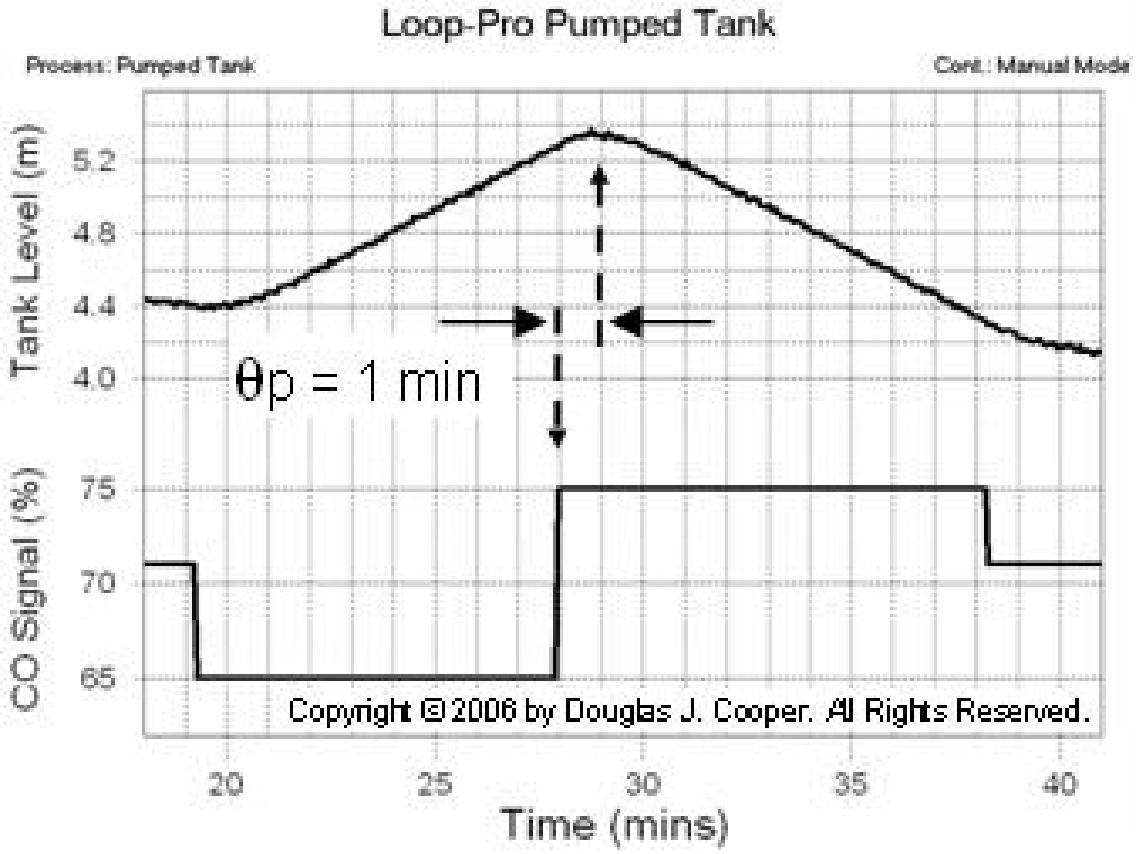
$$\text{slope}_2 = \left.\frac{dPV}{dt}\right|_2 = \frac{\Delta PV_2}{\Delta t_2} = \frac{4.6 - 5.2}{36 - 31} = -0.12 \ \frac{m}{min}$$

Using the two slopes computed above along with their respective CO values from the plot yields the integrator gain, Kp*, for the pumped tank:

$$Kp^* = \frac{\text{slope}_2 - \text{slope}_1}{CO_2 - CO_1} = \frac{(-0.12) - (0.13)}{75 - 65} = -0.025 \ \frac{m}{\%\,min}$$

• *Computing Dead Time*

The dead time is estimated from the plot using the same method described for the heat exchanger. That is, dead time, Θp, is computed as the difference in time from when the CO signal was stepped and when the measured PV starts a clear response to that change.



As shown above, the pumped tank dead time is estimated from the plot as:

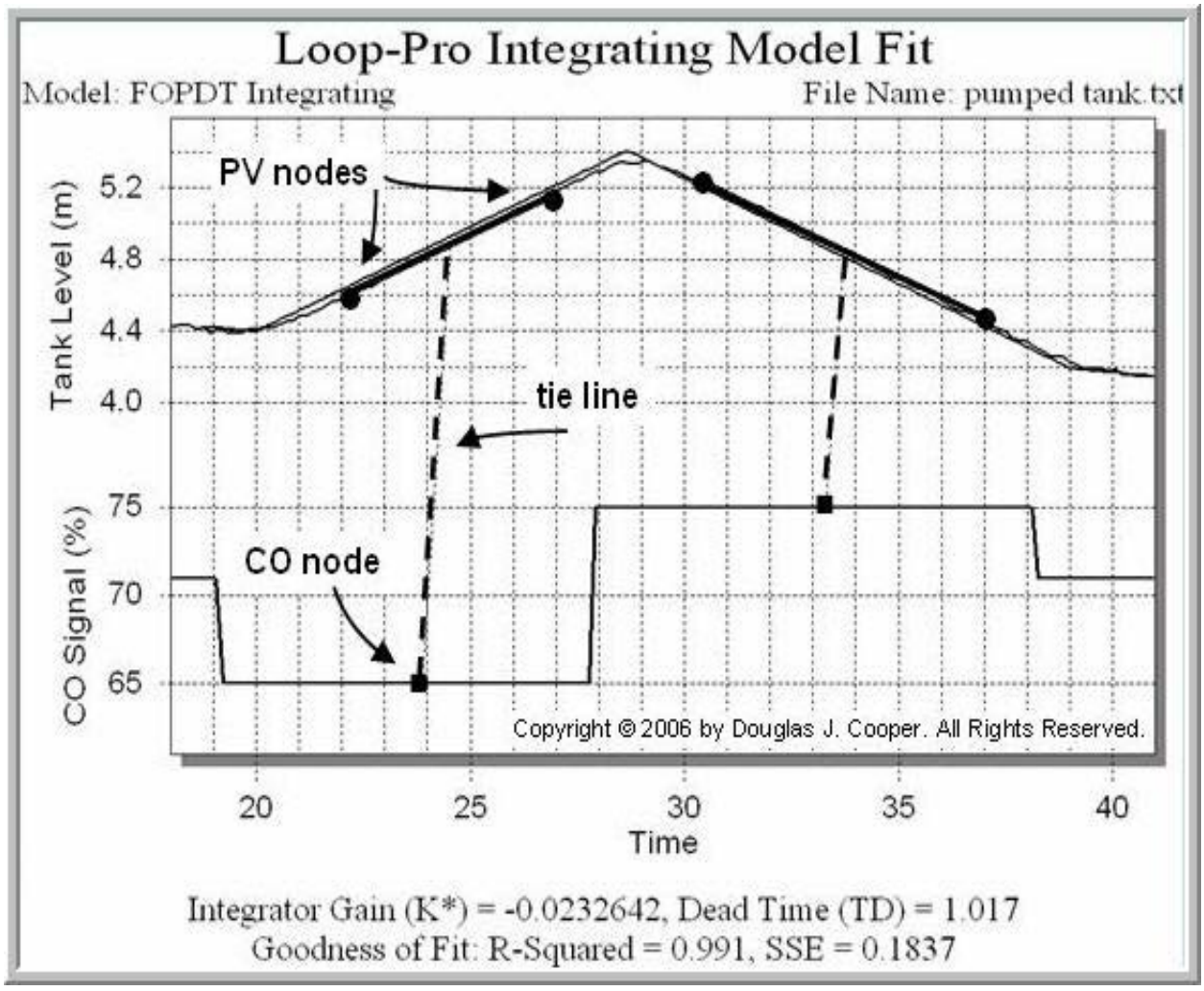
$\Theta p = 1.0$ min

Thus, the FOPDT Integrator model information needed to proceed with controller tuning using the correlations presented here is complete.

Automating the Model Fit

In today's world, there is no need to perform the model fit with graph paper and calculator. Commercial software offers analysis tools that makes fitting an FOPDT Integrating model quite simple.

Below, Control Station's Loop-Pro software is used to analyze the pumped tank data. As shown (), the software displays the data as a plot that includes adjustable nodes and tie lines.



To compute a fit, click on the two CO nodes at the bottom of the plot and drag them to match the two values of constant controller output expected in the data. Both have tie lines to identify their associated PV slope bar. Each slope bar has two end point nodes. Click and drag these so that each bar approximates the sloping segments on the graph.

With the six nodes (two CO and four PV) properly positioned, the FOPDT Integrating model parameters are automatically calculated and the model fit is displayed over the raw data.

We can see that the model PV line matches the measured PV data, so we have confidence that the model fit is good. The image above also shows the FOPDT Integrator values computed by the software:

$K_p^* = -0.023$ m/(% min), $\Theta_p = 1.0$ min

Recall that earlier in this article we computed by hand:

$K_p^* = -0.025$ m/(% min), $\Theta_p = 1.0$ min

In the dynamic modeling world, these values are virtually identical. We note that software offers additional benefits in that it performs the computation quickly, reduces the chance of computational error, and provides a visual confirmation that the model used for controller

design and tuning reasonably matches the process data.

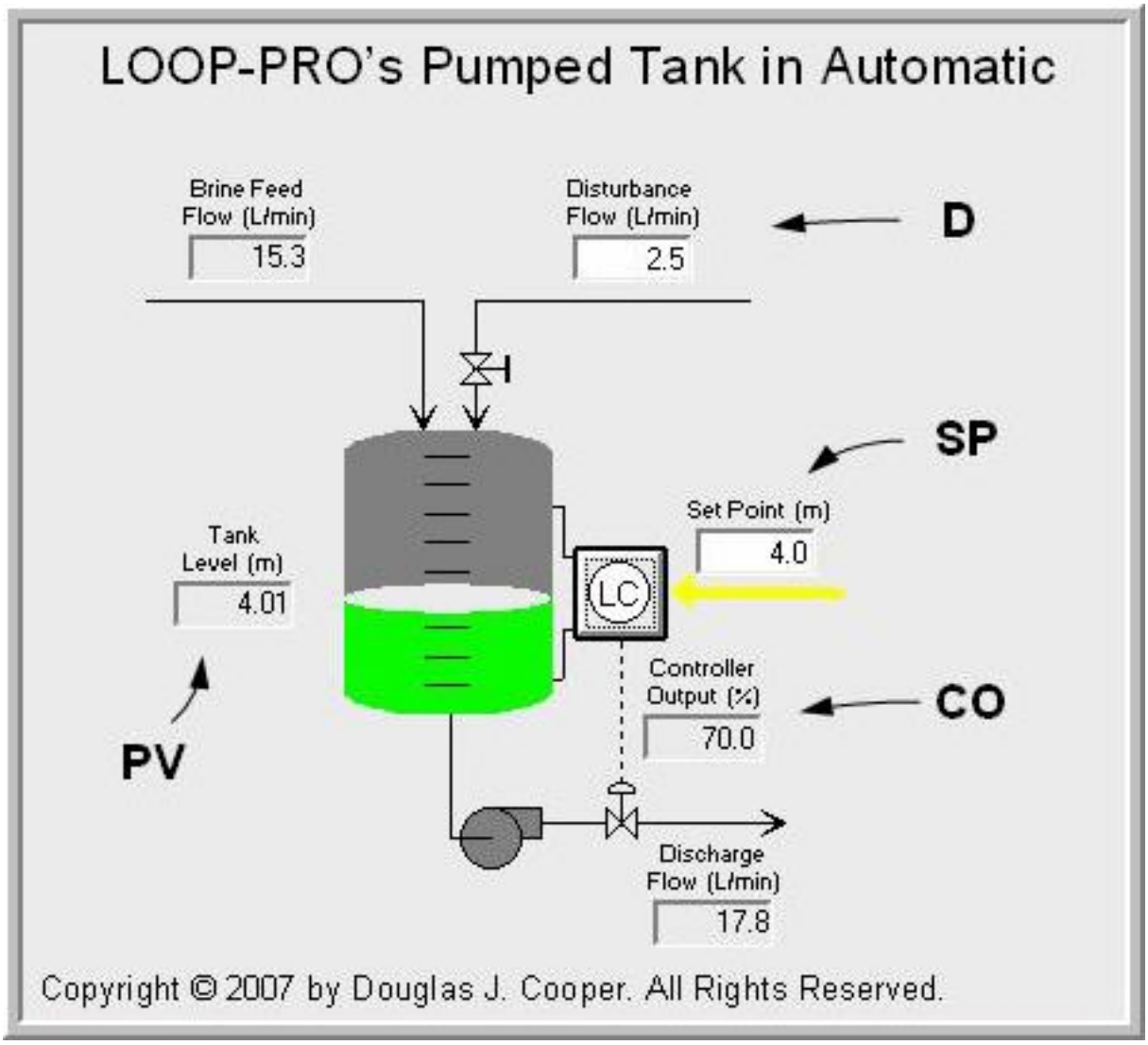
PI Control Study
With bump test data centered around our design level of operation, and with an approximating FOPDT Integrating model of that data computed, we have completed steps 1-3 of the controller design and tuning recipe for this integrating process.

PI Control of the Integrating Pumped Tank Process

By Bob Rice[1] and Doug Cooper

The control objective for the pumped tank process is to maintain liquid level at set point by adjusting the discharge flow rate out of the bottom of the tank. This process displays the distinctive integrating (or non-self regulating) behavior, and as such, presents an interesting control challenge.

The process graphic below () shows the pumped tank in automatic mode (also called closed loop):



The important variables for this study are labeled in the above graphic:

CO = signal to valve that adjusts discharge flow rate of liquid (controller output, %)
PV = measured liquid level signal from the tank (measured process variable, m)
SP = desired liquid level in tank (set point, m)
D = flow rate of liquid entering top of tank (major disturbance, L/min)

We follow the controller design and tuning recipe for integrating processes in this study as we design and test a PI controller. Please recall that there are subtle yet important differences between this procedure and the design and tuning recipe used for the more common self regulating process.

Step 1: Design Level of Operation (DLO)
We choose the DLO to be the same as that used in this article so we can build upon our previous modeling efforts:

- design value for PV and SP = 4.8 m
- design value for D = 2.5 L/min

Characteristic of real integrating processes, the pumped tank PV does not naturally settle at a steady operating level if CO is held constant. This lack of a natural "balance point" means we will not specify a CO as part of our DLO.

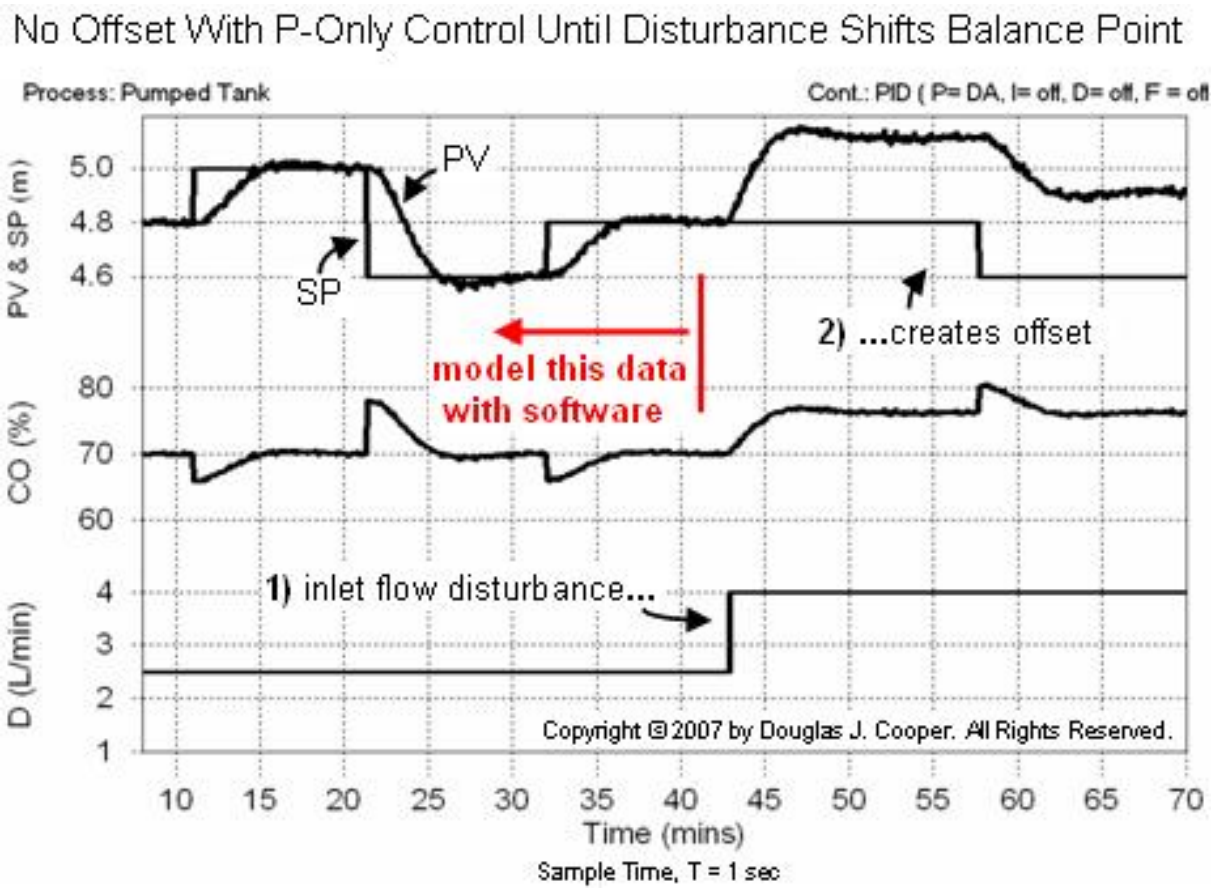
Step 2: Collect Process Data around the DLO
When PV and D are near the design level of operation and D is substantially quiet, we perform a dynamic test and generate CO-to-PV cause and effect dynamic data.

Because the PV of integrating processes tends to drift in manual mode (open loop), one alternative is to perform an open loop dynamic test that does not require bringing the process to steady state. As detailed in this article, the procedure is to maintain CO at a constant value until a slope trend in the PV can be visually identified. We then move the CO to a different value and hold it until a second PV slope is established. The article shows how to analyze the resulting plot data with hand calculations to obtain the FOPDT Integrating model for step 3 of the design and tuning recipe.

An alternative approach, presented below, is to use automatic mode data. When in closed loop, dynamic data is generated by bumping the SP. For model fitting purposes, the controller must be tuned such that the CO takes clear and sudden actions in response to the SP changes, and these must force PV movements that dominate the measurement noise.

Because a closed loop approach makes it possible to generate integrating process dynamic test data that begins at steady state, we can use model fitting software much like we did in this closed-loop set point driven study.

Below () we see the pumped tank process under P-Only control. As shown in the left half of the plot, while the major disturbance is quiet and at its design level, we are able to obtain good set point tracking performance with P-Only control.



As we discuss here and as shown above, a P-Only controller is able to provide good SP tracking performance with no offset as long the major disturbances are quiet and at their design values. Industrial processes can have many disturbances that impact operation. If any one of them changes, as happens in the above plot at roughly 43 minutes, then the simple P-Only controller is incapable of eliminating what becomes a sustained offset (i.e., incapable of making e(t) = SP – PV = 0)

This is why the integral action of a PI controller offers value even though the process itself possesses a naturally integrating behavior.

Note: in a surge tank where exit flow smoothing is more important than maintaining the measured level at SP, offset may not be considered a problem to be solved. Each situation must be considered on its own merits.

The red label in the above plot indicates that the left half contains dynamic response data that begins at steady state and that is not corrupted by disturbance changes. We isolate this data and model it as described in step 3 below.

Step 3: Fit a FOPDT Integrating Model to the Dynamic Process Data
We obtain an approximating description of the closed loop CO to PV dynamic behavior by fitting the process data with a first order plus dead time integrating (FOPDT Integrating) model of the form:

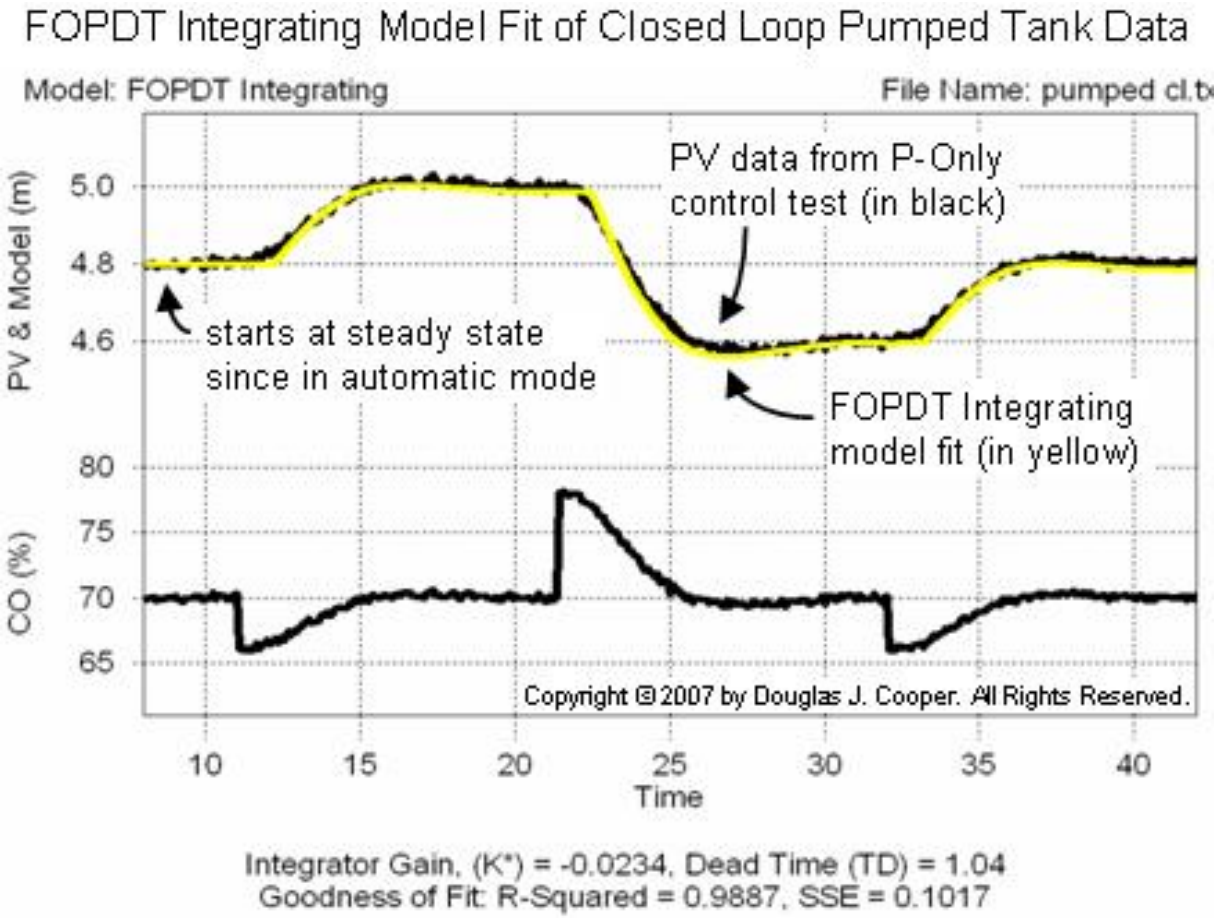
$$\frac{dPV(t)}{dt} = Kp^{*} \cdot CO(t - \theta p)$$

where

Kp* = integrator gain, with units [=] PV/(CO·time)
Θp = dead time, with units [=] time

Cropping the data and fitting the FOPDT Integrator model takes but a few mouse clicks with a commercial software tool (recall that an alternative graphical hand calculation method is described here).

The results of the automated model fit are shown below (). The visual similarity between the model and data gives us confidence that we have a meaningful description of the dynamic behavior of this process. The Kp* and Θp for this approximating model are shown at the bottom of the plot.



The model fitting software performs a systematic search for the combination of model parameters that minimizes the sum of squared errors (SSE), computed as:

$$SSE = \sum_{i=1}^{N} [\text{Measured PV}_i - \text{Model PV}_i]^2$$

The Measured PV is the actual data collected from our process. The Model PV is

computed using the model parameters from the search routine and the actual CO data from the file. N is the total number of samples in the file. In general, the smaller the SSE, the better the model describes the data.

The table below summarizes the model parameters from the above closed loop model fit as well as the results from the open loop slope driven analysis detailed in this article.

	- hand fit -	*- software model fits -*	
	Open Loop	**Open Loop**	**P-Only**
Integrator gain, Kp* (m/%·min)	-0.025	-0.023	-0.023
Dead time, Θp (min)	1.0	1.0	1.0

For the control studies in step 4, we will use:
- Kp* = – 0.023 m/%·min
- Θp = 1.0 min

Step 4: Use the FOPDT Integrating Parameters to Complete the Design

• *Sample Time,* T
The design and tuning recipe for integrating processes suggests setting the loop sample time, T, at one-tenth the process dead time or faster (i.e., T ≤ 0.1Θp). Faster sampling provides equally good, but not better, performance.

In this study, T ≤ 0.1(1.0 min), so T should be 6 seconds or less. We meet this with the sample time option available from virtually all commercial vendors:
◊ sample time, T = 1 sec

• *Control Action (Direct/Reverse)*
The pumped tank has a negative Kp*, so when CO increases, PV decreases in response. Since a controller must provide negative feedback, if the process is reverse acting, the controller must be direct acting. Thus, if the PV is too high, the controller must increase the CO to correct the error. Since the controller moves in the same direction as the problem, we specify:
◊ controller is direct acting

• *Computing Controller Error, e(t)*
Set point, SP, is manually entered into a controller. The measured PV comes from the sensor (our wire in). Since SP and PV are known values, then at every loop sample time, T, controller error can be directly computed as:
◊ error, e(t) = SP - PV

• *Determining Bias Value,* CO_{bias}
The lack of a natural balance point with integrating processes makes the determination of a design CO_{bias} problematic. The solution is to use bumpless transfer. That is, when switching to automatic, initialize SP to the current value of PV and CO_{bias} to the current value of CO (most commercial controllers are already programmed this way). By choosing our current operation as our design state at switchover, there is no corrective actions needed by the controller and it can smoothly engage, thus:
◊ controller bias, CO_{bias} = CO that exists at switch over

• *Controller Gain,* Kc*, and Reset Time, T*i

We use our FOPDT Integrating model parameters in the industry-proven Internal Model Control (IMC) tuning correlations to compute PI tuning values. Though all PI forms are equally capable, we use the dependent, ideal form of the PI algorithm in this study:

$$CO = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{Ti}\int e(t)dt$$

The first step in using the IMC correlations is to compute *T*c, the closed loop time constant. *T*c describes how active our controller should be in responding to a set point change or in rejecting a disturbance. For integrating processes, the design and tuning recipe suggests:

*T*c = 3Θp = 3(1.0 min) = 3 min

With *T*c computed, the PI controller gain, Kc, and reset time, *T*i, are computed as:

$$Kc = \frac{1}{Kp^*}\frac{2Tc+\theta p}{(Tc+\theta p)^2} \qquad Ti = 2Tc+\theta p$$

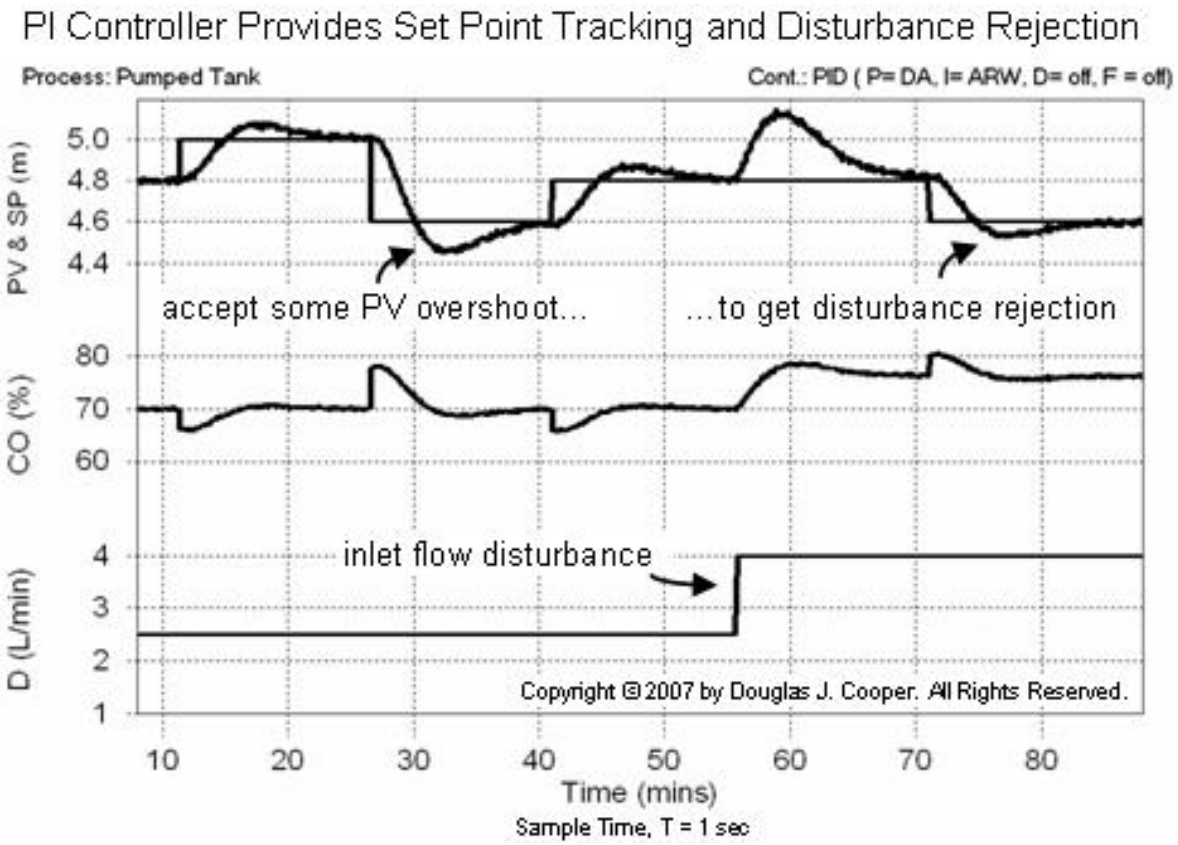
Substituting the Kp*, Θp and *T*c identified above into these tuning correlations, we compute:

$$Kc = \frac{1}{-0.023}\frac{2(3)+1}{(3+1)^2} \qquad Ti = 2(3)+1$$

or

- Kc = – 19 m/%
- *T*i = 7 min

Below is the performance of this PI controller (with Kc = – 19 and *T*i = 7) on the pumped tank (). The plot includes the same set point tracking and disturbance rejection test conditions as were used in the P-Only controller plot near the top of this article.



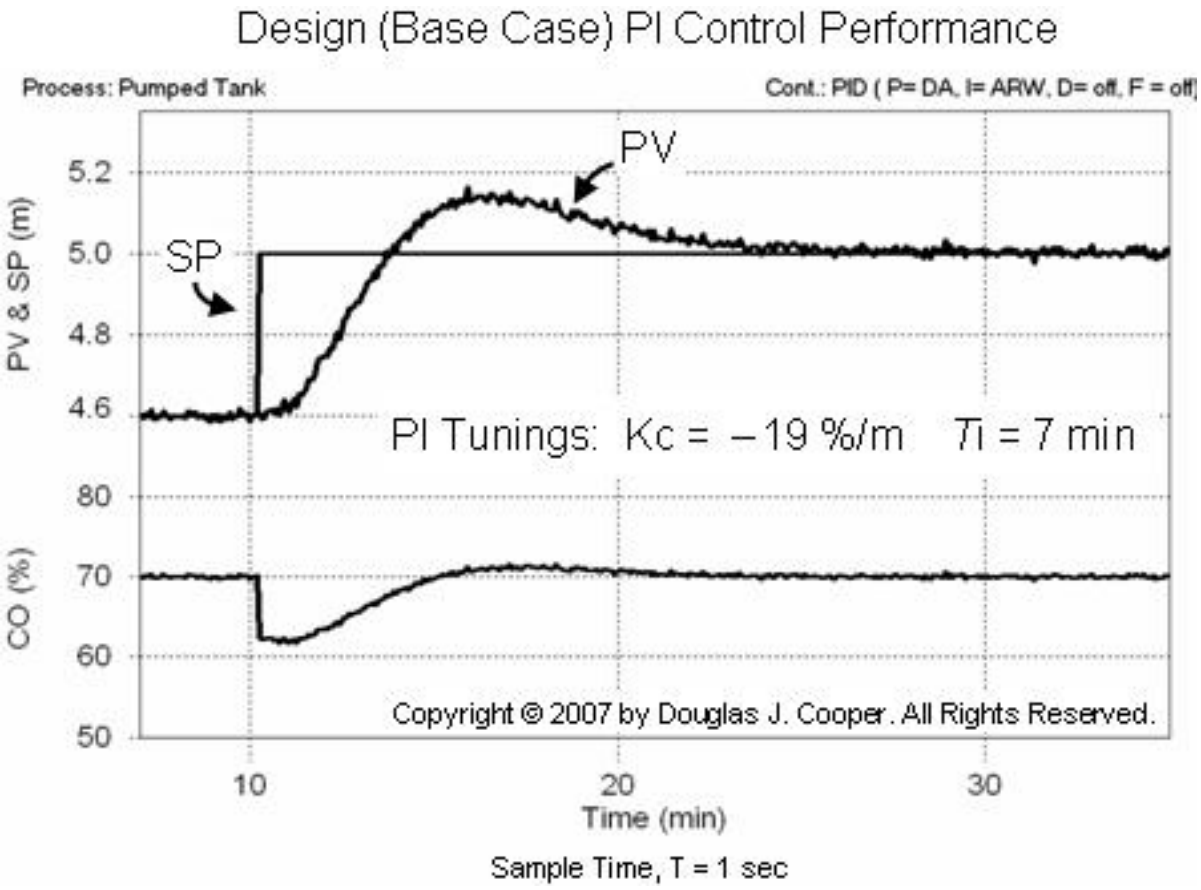
As labeled in the plot, our PI control set point response now includes some overshoot. Recall that the P-Only controller, shown near the top of this article, can provide a rapid set point response with no overshoot, that is, until a disturbance changes the balance point of the process.

The benefit of integral action is that when a disturbance occurs, a PI controller can reject the upset and return the PV to set point. This is because the constant summing of integral action continues to move the CO until controller error is driven to zero.

Thus, PI control requires that we accept some overshoot during set point tracking in exchange for the ability to reject disturbances. In many industrial applications, this is considered a fair trade.

Tuning Sensitivity Study

Below is the set point tracking performance of our PI controller on the pumped tank. The controller tuning (Kc = – 19 m/%, T_i = 7 min) is determined as detailed in the step-by-step recipe above.

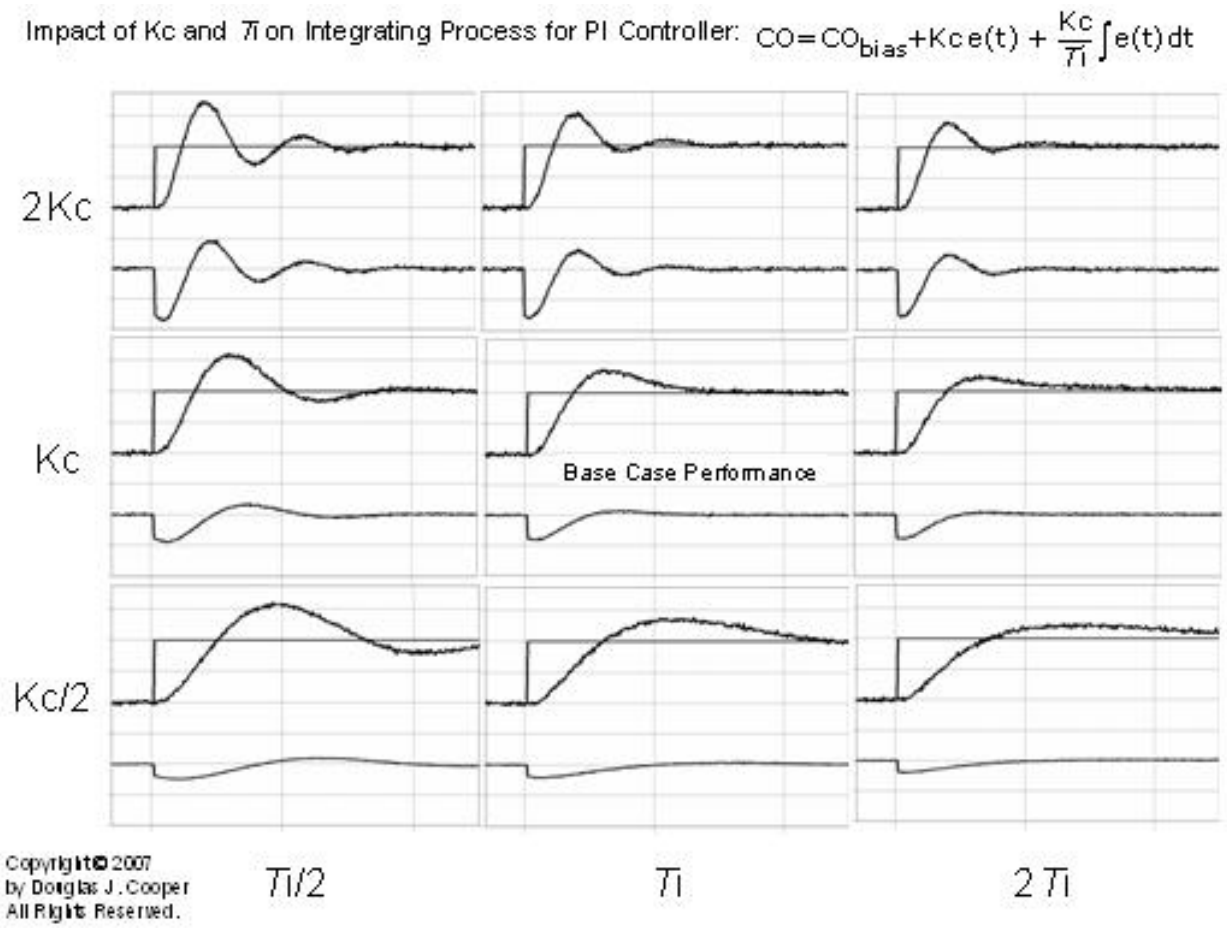


Some questions to consider are:

- How does performance vary as the tuning values change?
- How can we avoid overshoot if we find such behavior undesirable?

Below () is a tuning map for a PI controller implemented on the pumped tank integrating process. The center plot is the identical base case performance plot shown above.

The complete map shows set point tracking performance when controller gain (Kc) and reset time (*T*i) are individually doubled and halved.



While "good" or "best" performance is a matter best decided by the operations staff, the above map makes it clear that our recipe does an excellent job of meeting the desire for a reasonably rapid rise, a modest overshoot and a quick settling time.

Unfortunately, eliminating overshoot altogether does not appear to be one of our options for PI control of integrating processes.

Final Thoughts

The design and tuning recipe for integrating processes provides the above base case performance with minimal testing on our process. In a manufacturing environment where we need a fast solution with minimal disruption, this recipe is certainly one to have in our tool box.

V. Advanced Classical Control Architectures (by Doug Cooper & Allen Houtz)

15) Cascade Control For Improved Disturbance Rejection

The Cascade Control Architecture

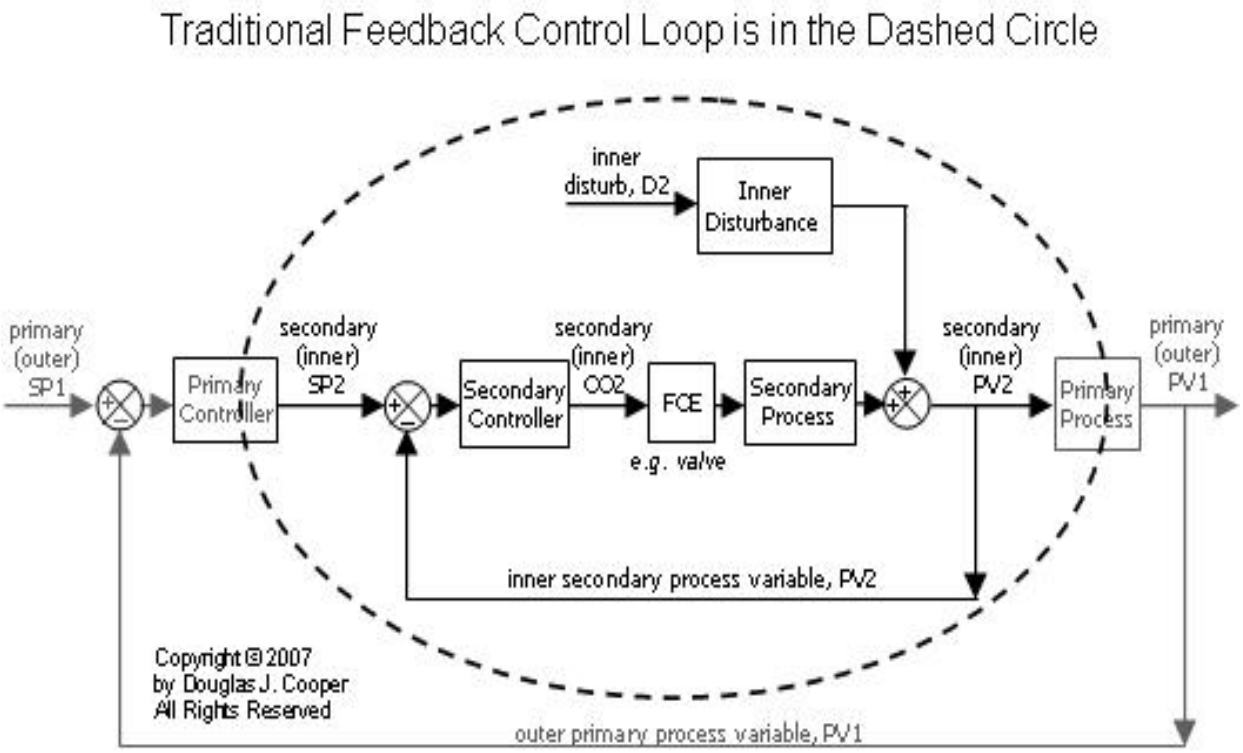
Two popular control strategies for improved disturbance rejection performance are cascade control and feed forward with feedback trim.

Improved performance comes at a price. Both strategies require that additional instrumentation be purchased, installed and maintained. Both also require additional engineering time for strategy design, tuning and implementation.

The cascade architecture offers alluring additional benefits such as the ability to address multiple disturbances to our process and to improve set point response performance.

In contrast, the feed forward with feedback trim architecture is designed to address a single measured disturbance and does not impact set point response performance in any fashion (explored in a future article).

The Inner Secondary Loop
The dashed line in the block diagram below () circles a feedback control loop like we have discussed in dozens of articles on *controlguru.com*. The only difference is that the words "inner secondary" have been added to the block descriptions. The variable labels also have a "2" after them.

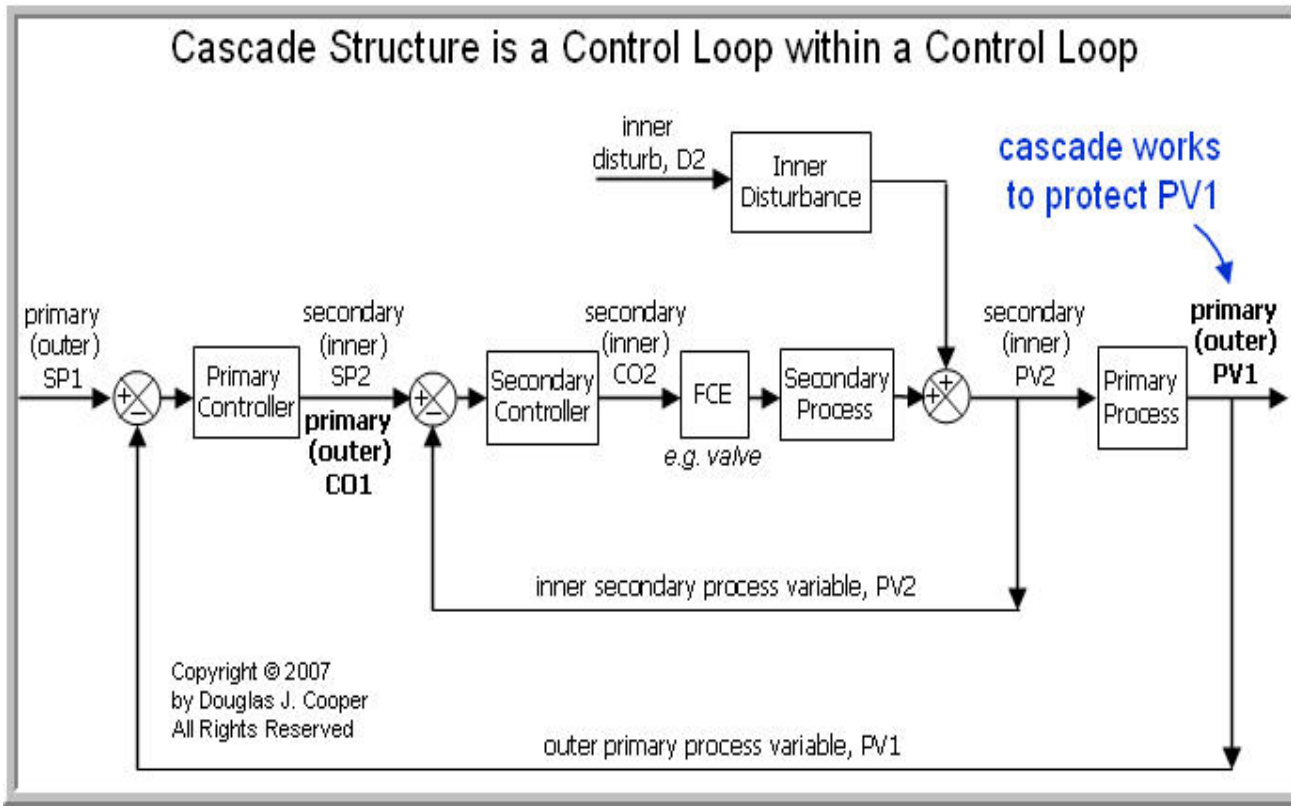


So,
SP2 = inner secondary set point
CO2 = inner secondary controller output signal
PV2 = inner secondary measured process variable signal

And
D2 = inner disturbance variable (often not measured or available as a signal)
FCE = final control element such as a valve, variable speed pump or compressor, etc.

The Nested Cascade Architecture
To construct a cascade architecture, we literally nest the secondary control loop inside a primary loop as shown in the block diagram below ().

Note that outer primary PV1 is our process variable of interest in this implementation. PV1 is the variable we would be measuring and controlling if we had chosen a traditional single loop architecture instead of a cascade.



Because we are willing to invest the additional effort and expense to improve the performance response of PV1, it is reasonable to assume that it is a variable important to process safety and/or profitability. Otherwise, it does not make sense to add the complexity of a cascade structure.

Naming Conventions

Like many things in the PID control world, vendor documentation is not consistent. The most common naming conventions we see for cascade (also called nested) loops are:

- secondary and primary
- inner and outer
- slave and master

In an attempt at clarity, we are somewhat repetitive in this article by using labels like "inner secondary" and "outer primary."

Two PVs, Two Controllers, One Valve

Notice from the block diagrams that the cascade architecture has:

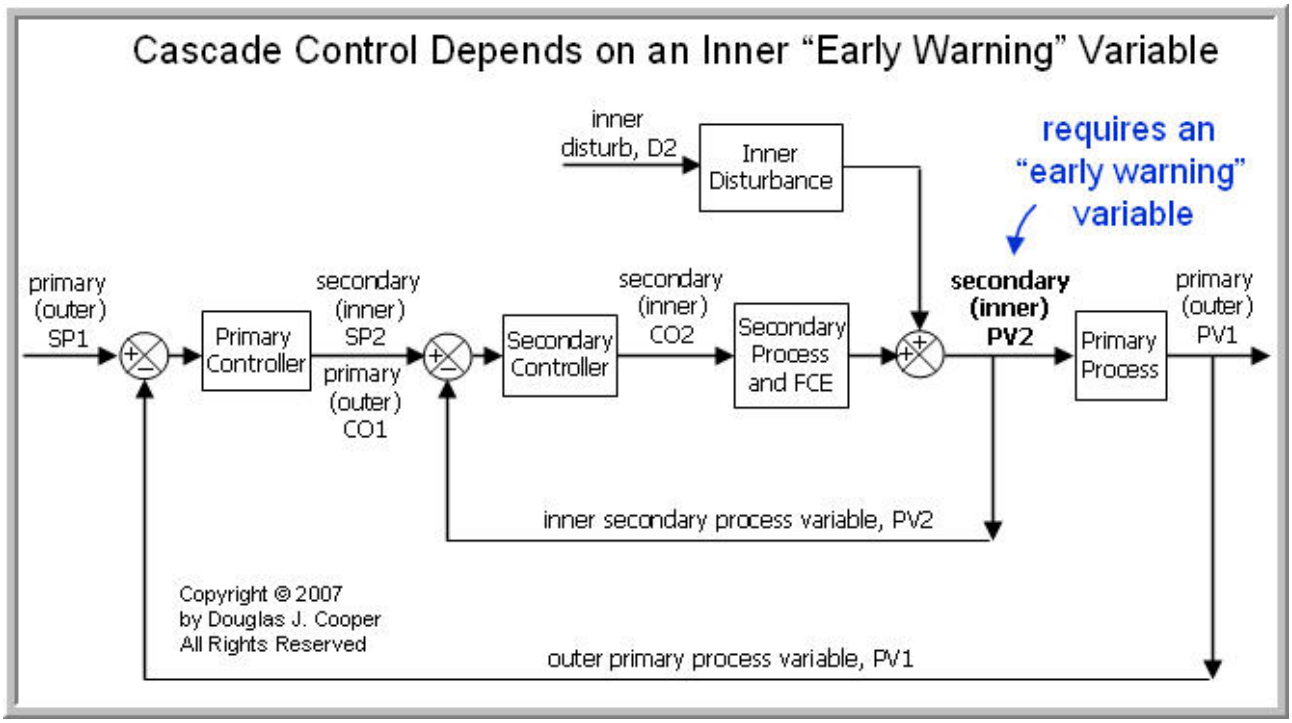
- two controllers (an inner secondary and outer primary controller)
- two measured process variable sensors (an inner PV2 and outer PV1)
- only ***one*** final control element (FCE) such as a valve, pump or compressor.

How can we have two controllers but only one FCE? Because as shown in the diagram above, the controller output signal from the outer primary controller, CO1, becomes the set point of the inner secondary controller, SP2.

The outer loop literally commands the inner loop by adjusting its set point. Functionally, the controllers are wired such that SP2 = CO1 (thus, the master and slave terminology referenced above).

This is actually good news from an implementation viewpoint. If we can install and maintain an inner secondary sensor at reasonable cost, and if we are using a PLC or DCS where adding a controller is largely a software selection, then the task of constructing a cascade control structure may be reasonably straightforward.

Early Warning is Basis for Success
As shown below (), an essential element for success in a cascade design is the measurement and control of an "early warning" process variable.



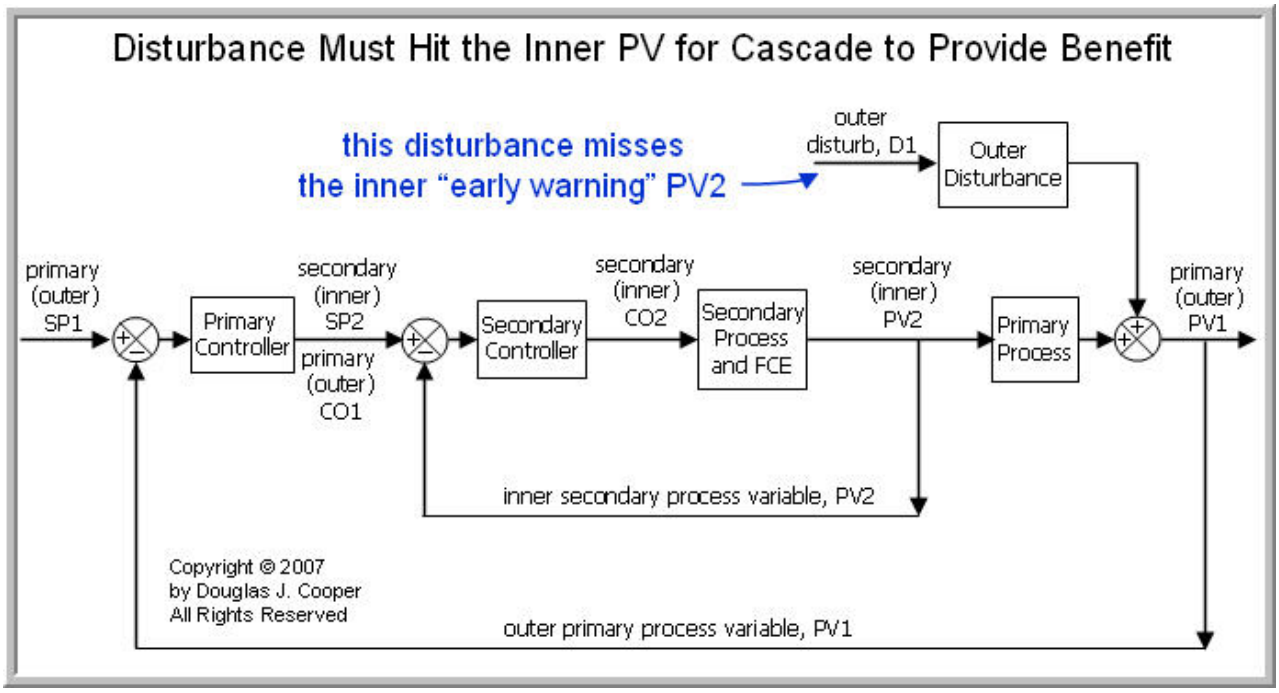
In the cascade architecture, inner secondary PV2 serves as this early warning process variable. Given this, essential design characteristics for selecting PV2 include that:

- it be measurable with a sensor,
- the same FCE (e.g., valve) used to manipulate PV1 also manipulates PV2,
- the same disturbances that are of concern for PV1 also disrupt PV2, and
- PV2 responds *before* PV1 to disturbances of concern and to FCE manipulations.

Since PV2 sees the disruption first, it provides our "early warning" that a disturbance has occurred and is heading toward PV1. The inner secondary controller can begin corrective action immediately. And since PV2 responds first to final control element (e.g., valve) manipulations, disturbance rejection can be well underway even before primary variable PV1 has been substantially impacted by the disturbance.

With such a cascade architecture, the control of the outer primary process variable PV1 benefits from the corrective actions applied to the upstream early warning measurement PV2.

Disturbance Must Impact Early Warning Variable PV2
As shown below (), even with a cascade structure, there will likely be disturbances that impact PV1 but do not impact early warning variable PV2.

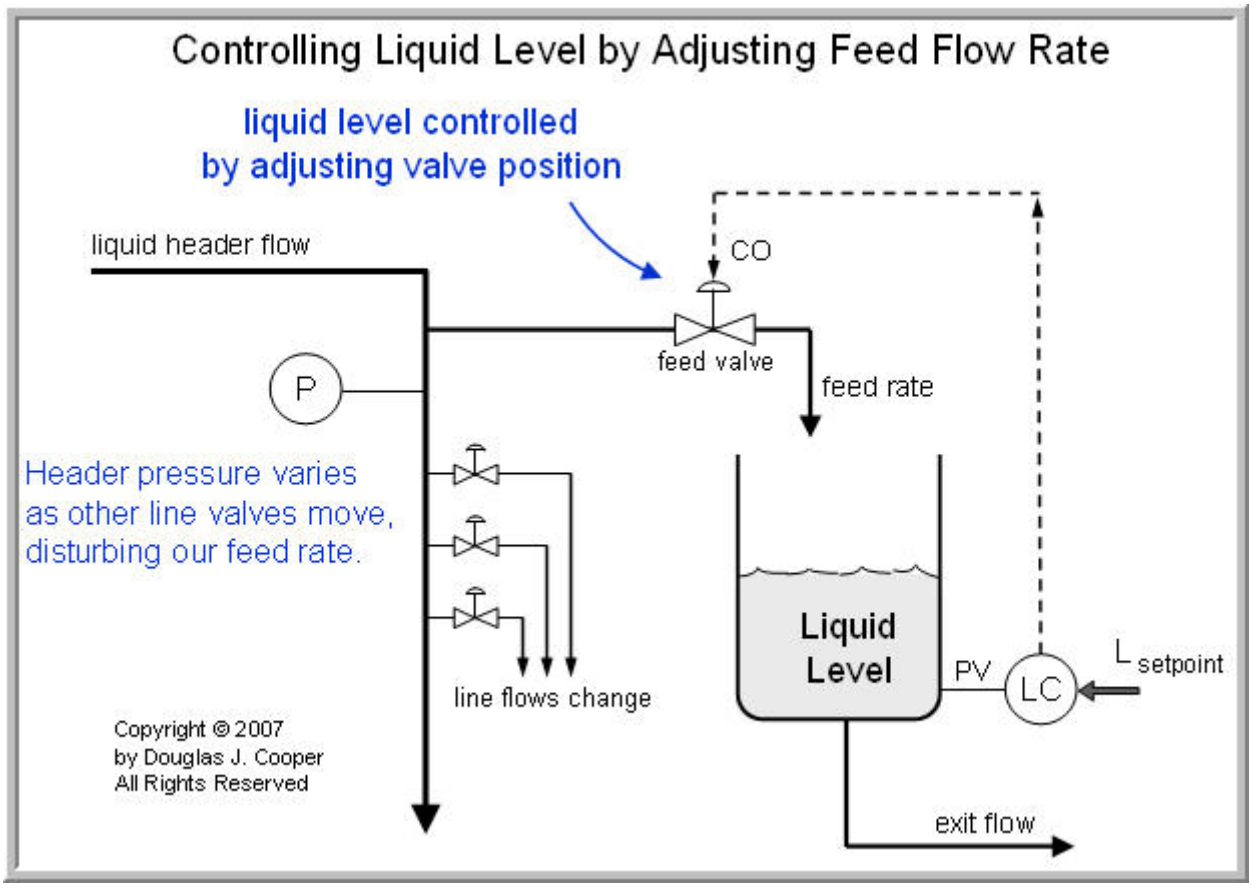


The inner secondary controller offers no "early action" benefit for these outer disturbances. They are ultimately addressed by the outer primary controller as the disturbance moves PV1 from set point.

On a positive note, a proper cascade can improve rejection performance for any of a host of disturbances that directly impact PV2 before disrupting PV1.

An Illustrative Example

To illustrate the construction and value of a cascade architecture, consider the liquid level control process shown below (). This is a variation on our gravity drained tanks, so hopefully, the behavior of the process below follows intuitively from our previous investigations.



As shown above, the tank is essentially a barrel with a hole punched in the bottom. Liquid enters through a feed valve at the top of the tank. The exit flow is liquid draining freely by the force of gravity out through the hole in the tank bottom.

The control objective is to maintain liquid level at set point (SP) in spite of unmeasured disturbances. Given this objective, our measured process variable (PV) is liquid level in the tank. We measure level with a sensor and transmit the signal to a level controller (the LC inside the circle in the diagram).

After comparing set point to measurement, the level controller (LC) computes and transmits a controller output (CO) signal to the feed valve. As the feed valve opens and closes, the liquid feed rate entering the top of the tank increases and decreases to raise and lower the liquid level in the tank.

This "measure, compute and act" procedure repeats every loop sample time, T, as the controller works to maintain tank level at set point.

The Disturbance
The disturbance of concern is the pressure in the main liquid header. As shown in the diagram above, the header supplies the liquid that feeds our tank. It also supplies liquid to several other lines flowing to different process units in the plant.

Whenever the flow rate of one of these other lines changes, the header pressure can be impacted. If several line valves from the main header open at about the same time, for example, the header pressure will drop until its own control system corrects the

imbalance. If one of the line valves shuts in an emergency action, the header pressure will momentarily spike.

As the plant moves through the cycles and fluctuations of daily production, the header pressure rises and falls in an unpredictable fashion. And every time the header pressure changes, the feed rate to our tank is impacted.

Problem with Single Loop Control
The single loop architecture in the diagram above attempts to achieve our control objective by adjusting valve position in the liquid feed line. If the measured level is higher than set point, the controller signals the valve to close by an appropriate percentage with the expectation that this will decrease feed flow rate accordingly.

But feed flow rate is a function of two variables:
- feed valve position, and
- the header pressure pushing the liquid through the valve (a disturbance).

To explore this, we conduct some thought experiments:

Thought Experiment #1: Assume that the main header pressure is perfectly constant over time. As the feed valve opens and closes, the feed flow rate and thus tank level increases and decreases in a predictable fashion. In this case, a single loop structure provides acceptable level control performance.

Thought Experiment #2: Assume that our feed valve is set in a fixed position and the header pressure starts rising. Just like squeezing harder on a spray bottle, the valve position can remain constant yet the rising pressure will cause the flow rate through the fixed valve opening to increase.

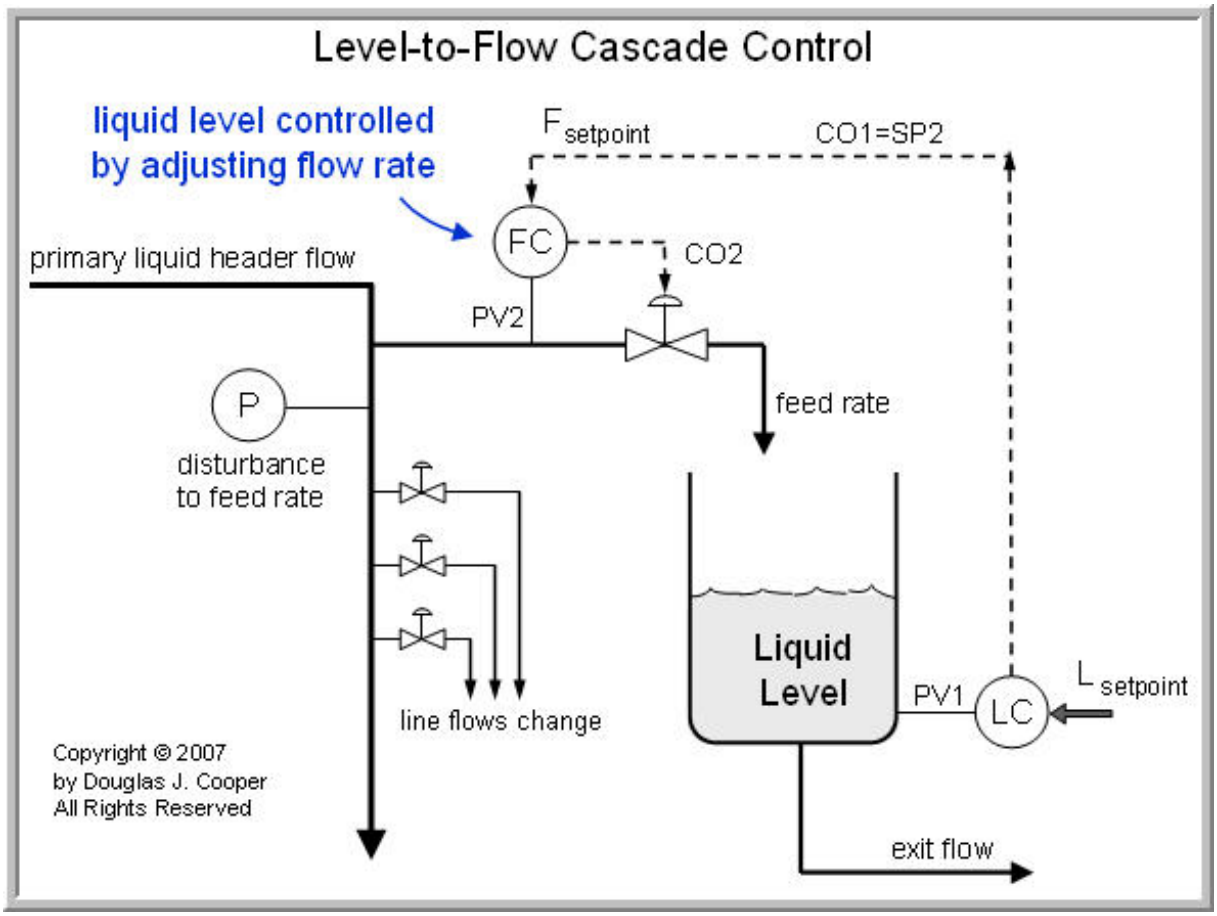
Thought Experiment #3: Now assume that the header pressure starts to rise at the same moment that the controller determines that the liquid level in our tank is too high. The controller can be **closing** the feed valve, but because header pressure is rising, the flow rate through the valve can actually be **increasing**.

As presented in Thought Experiment #3, The changing header pressure (a disturbance) can cause a contradictory outcome that can confound the controller and degrade control performance.

A Cascade Control Solution
For high performance disturbance rejection, it is not valve position, but rather, feed flow rate that must be adjusted to control liquid level.

Because header pressure changes, increasing feed flow rate by a precise amount can sometimes mean opening the valve a lot, opening it a little, and because of the changing header pressure, perhaps even closing the valve a bit.

Below is a classic level-to-flow cascade architecture (). As shown, an inner secondary sensor measures the feed flow rate. An inner secondary controller receives this flow measurement and adjusts the feed flow valve.



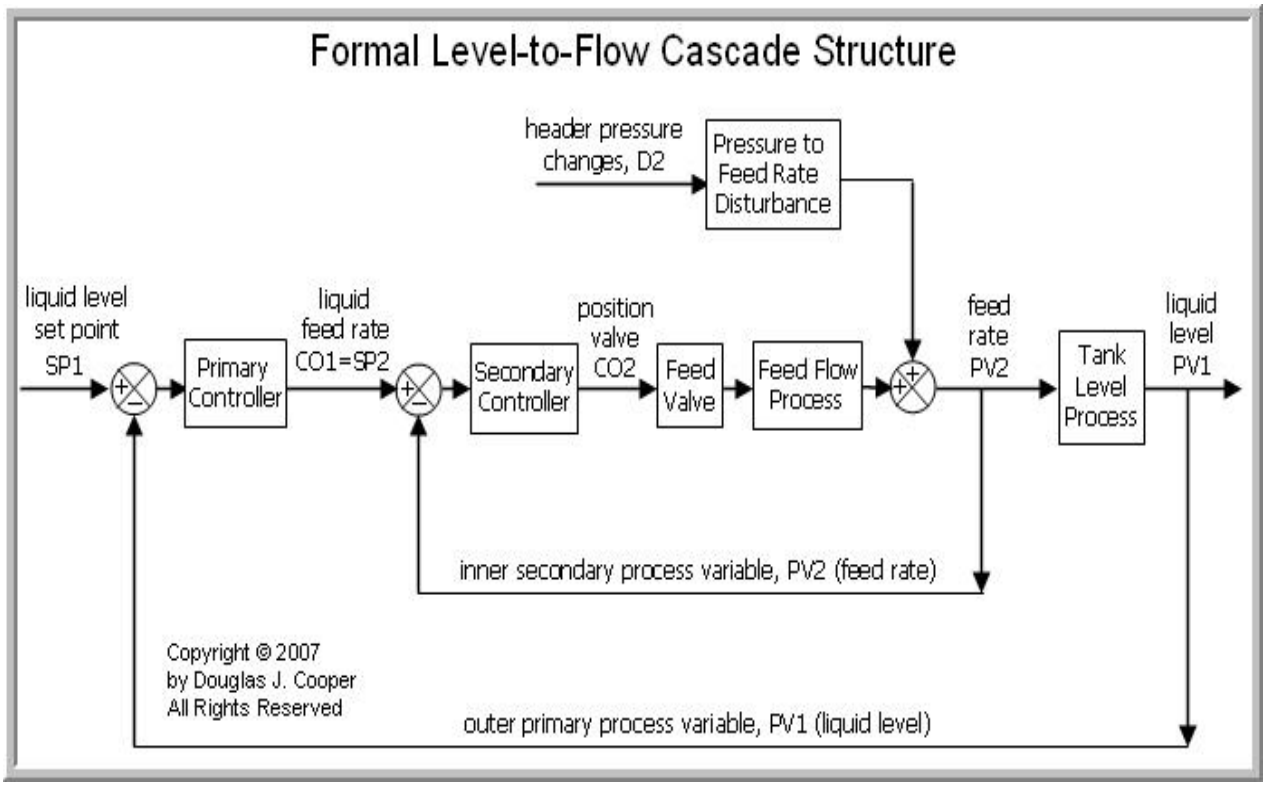
With this cascade structure, if liquid level is too high, the primary level controller now calls for a decreased liquid feed flow rate rather than simply a decrease in valve opening. The flow controller then decides whether this means opening or closing the valve and by how much.

Note in the diagram that, true to a cascade, the level controller output signal (CO1) becomes the set point for the flow controller (SP2).

Header pressure disturbances are quickly detected and addressed by the secondary flow controller. This minimizes any disruption caused by changing header pressure to the benefit of our primary level control process.

The Level-to-Flow Cascade Block Diagram
As shown in the block diagram below (), our level-to-flow cascade fits into our block diagram structure. As required, there are:

- Two controllers - the outer primary level controller (LC) and inner secondary feed flow controller (FC)
- Two measured process variable sensors - the outer primary liquid level (PV1) and inner secondary feed flow rate (PV2)
- One final control element (FCE) - the valve in the liquid feed stream



As required for a successful design, the inner secondary flow control loop is nested inside the primary outer level control loop. That is:

- The feed flow rate (PV2) responds ***before*** the tank level (PV1) when the header pressure disturbs the process or when the feed valve moves.
- The output of the primary controller, CO1, is wired such that it becomes the set point of the secondary controller, SP2.
- Ultimately, level measurement, PV1, is our process variable of primary concern. Protecting PV1 from header pressure disturbances is the goal of the cascade.

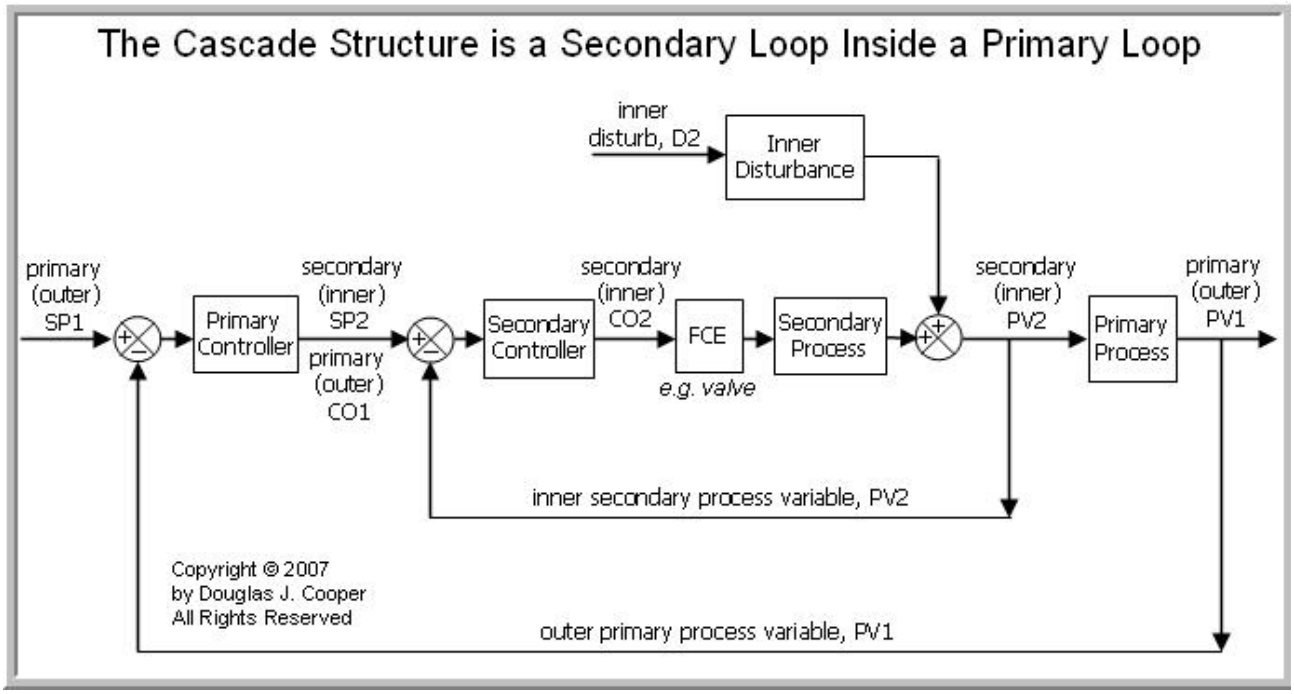
Design and Tuning
The inner secondary and outer primary controllers are from the PID family of algorithms. We have explored the design and tuning of these controllers in numerous articles, so as we will see, implementing a cascade builds on many familiar tasks.

There are a number of issues to consider when selecting and tuning the controllers for a cascade. We explore next an implementation recipe for cascade control.

An Implementation Recipe for Cascade Control

When improved disturbance rejection performance is our goal, one benefit of a cascade control (nested loops) architecture over a feed forward strategy is that implementing a cascade builds upon our existing skills.

The cascade block diagram is presented in the graphic below () and discussed in detail in this article. As shown, a cascade has two controllers. Implementation is a familiar task because the procedure is essentially to employ our controller design and tuning recipe twice in sequence.



The cascade architecture variables listed in the block diagram include:
CO2 = inner secondary controller output signal to the FCE
PV2 = the early warning inner secondary process variable
SP2 = CO1 = inner secondary set point equals the outer primary controller output
PV1 = outer primary process variable
SP1 = outer primary set point
D2 = disturbances that impact the early warning PV2 before they impact PV1
FCE = final control element (e.g., a valve) is continuously adjustable between on/open and off/closed, and that impacts PV2 before it impacts PV1

Two Bump Tests Required
Two bump tests are required to generate the dynamic process response data needed to design and tune the two controllers in a cascade implementation.

• *First the Inner Secondary Controller*
A reliable procedure begins with the outer primary controller in manual mode (open loop) as we apply the design and tuning recipe to the inner secondary controller.

Thus, with our process steady at (or as near as practical to) its design level of operation (DLO), we generate dynamic CO2 to PV2 process response data with either a manual mode (open loop) bump test or a more sophisticated SP driven (closed loop) bump test.

The objective for the inner secondary controller is timely rejection of disturbances D2 based on the measurement of an "early warning" secondary process variable PV2. Good disturbance rejection performance is therefore of fundamental importance for the inner secondary controller.

Yet as shown in the block diagram above and detailed in this article, the output signal of

the outer primary controller becomes a continually updated set point for the inner secondary controller (SP2 = CO1). Since we expect the inner secondary controller to respond crisply to these rapidly changing set point commands, it must also be tuned to provide good SP tracking performance.

In the perfect world, we would balance disturbance rejection and set point tracking capability for the inner secondary controller. But we cannot shift our attention to the outer primary controller until we have tested and approved the inner secondary controller performance.

In production processes with streams comprised of gases, liquids, powders, slurries and melts, disturbances are often unmeasured and beyond our ability to manipulate at will. So while we desire to balance disturbance rejection and set point tracking performance for the inner secondary controller, in practice, SP tracking tests tend to provide the most direct route to validating inner secondary controller performance.

• *Then the Outer Primary Controller*
Once implemented, the inner secondary controller literally becomes part of the “process” from the outer primary controller’s view.

As a result, *any* alteration to the inner secondary controller (e.g., tuning parameter adjustments, algorithm modifications, sample time changes) can change the process gain, Kp, time constant, *T*p, and/or dead time, Ɵp, of the outer loop CO1 to PV1 dynamic response behavior. This, in turn, impacts the design and tuning of the outer primary controller.

Thus, we must design, tune, test, accept and then "lock down" the inner secondary controller, leaving it in automatic mode with a fixed configuration. Only then, with our process steady and at (or very near) its DLO, can we proceed with the second bump test to complete the design and tuning of the outer primary controller.

Software Provides Benefit
Given that the outer primary controller design and tuning is based on the specifics of the inner secondary loop, a guess and test approach to a cascaded implementation can prove remarkably wasteful, time consuming and expensive.

In production operations, a commercial software package that automates the controller design and tuning tasks will pay for itself as early as a first cascade tuning project

Minimum Criteria for Success
A successful cascade implementation requires that early warning process variable PV2 respond *before* outer primary PV1 both to disturbances of concern (D2) *and* to final control element (FCE) manipulations (e.g., a valve).

Responding first to disturbances means that the inner secondary D2 to PV2 dead time, Ɵp, must be smaller than the overall D2 to PV1 dead time, or:

$$\text{Ɵp (D2} \rightarrow \text{PV2)} < \text{Ɵp (D2} \rightarrow \text{PV1)}$$

Responding first to FCE manipulations means that the inner secondary CO2 to PV2 dead

time must be smaller than the overall CO2 to PV1 dead time, or:

$$\Theta p\ (CO2 \rightarrow PV2) < \Theta p\ (CO2 \rightarrow PV1)$$

If these minimum criteria are met, then a cascade control architecture can show benefit in improving disturbance rejection.

P-Only vs. PI for Inner Secondary Controller
A subtle design issue relates to the choice of control algorithm for the inner secondary controller. While perhaps not intuitive, an inner secondary P-Only controller will provide better performance than a PI controller in many cascade implementations.

• *Defining Performance*
We focus all assessment of control performance on outer primary process variable PV1. Performance is "improved" if the cascade structure can more quickly and efficiently minimize the impact of disturbances D2 on PV1. Given the nature of the cascade structure, it is assumed that D2 first disrupts PV2 as it travels to PV1.

Since PV2 was selected because of its value as an early warning variable, our interest in PV2 control performance extends only to its ability to provide protection to outer primary process variable PV1. We are otherwise unconcerned if PV2 displays offset, shows a large response overshoot, or any other performance characteristic that might be considered undesirable in a traditional measured PV.

• *Is the Inner Loop "Fast" Relative to the Overall Process?*
A cascade architecture with a P-Only controller on the inner secondary loop will provide improved disturbance rejection performance over that achievable with a traditional single loop controller if the minimum criteria for success as discussed above are met.

A PI controller on the inner loop *may* provide even better performance than P-Only, but only if the dynamic character of the inner secondary loop is "fast" relative to that of the overall process.

If the inner secondary loop dynamic character is not sufficiently fast, then a PI controller on the inner loop, even if properly designed and tuned, will not perform as well as P-Only. It is even possible that a PI controller could degrade performance to an extent that the cascade architecture performs worse than a traditional (non-cascade) single loop controller.

• *Why PI controllers Need a "Fast" Inner Loop*
At every sample time T, the outer primary controller computes a controller output signal that is fed as a new set point to the inner secondary controller (CO1 = SP2). The inner secondary controller continually makes CO2 moves as it works to keep PV2 equal to the ever-changing SP2.

The cascade will fail if the inner loop cannot keep pace with the rapid-fire stream of SP2 commands. If the inner secondary controller "falls behind" (or more specifically, if the CO2 actions induce dynamics in the inner loop that do not settle quickly relative to the dynamic behavior of the overall process), the benefit of the early warning PV2 measurement is lost.

A P-Only controller can provide energetic control action when tracking set points and

rejecting disturbances. Its very simplicity can be a useful attribute in a cascade implementation because a P-Only controller quickly completes its response actions to any control error (E2 = SP2 – PV2). While P-Only controllers display offset when operation moves from the DLO, this is not considered a performance problem for inner secondary PV2.

With two tuning parameters, a PI controller has a greater ability to track set points and reject disturbances. However, this added sophistication yields a controller with a greater tendency to "roll" or oscillate. And the ability to eliminate offset, normally considered a positive attribute for PI controllers, can require a longer series of control actions that extends how quickly a loop settles.

Thus, a PI controller generally needs more time (a faster inner loop) to exploit its enhanced capability relative to that of a P-Only controller. If the dynamic nature of a particular cascade does not provide this time, then an inner-loop P-Only controller is the proper choice.

The Cascade Implementation Recipe
Below is a generally conservative approach for cascade control implementation. Note that the recipe assumes that:

- Early warning PV2 responds *before* PV1 both to D2 *and* CO2 changes as discussed above in the minimum criteria for cascade success.
- A first order plus dead time (FOPDT) model of dynamic process data yields a process time constant, *T*p, that is much larger than the dead time, Θp. This permits us to focus on the time constant as the marker for "fast" dynamic process behavior.

1) Starting from a steady operation, step, pulse or otherwise perturb CO2 around the design level of operation (DLO). Collect CO2, PV2 and PV1 dynamic data as the process variables respond. The inner secondary loop can be in automatic (closed loop) if the controller is sufficiently active to force a clear dynamic response in PV2. The outer primary loop is normally in manual (open loop).

2) Fit a FOPDT model to the inner secondary process (CO2 −› PV2) data and another to the overall process (CO2 −› PV1) data, yielding:

	Inner Secondary	**Overall Process**
Process Gain (*how far*)	Kp (CO2 −› PV2)	Kp (CO2 −› PV1)
Time Constant (*how fast*)	*T*p (CO2 −› PV2)	*T*p (CO2 −› PV1)
Dead Time (*how much delay*)	Θp (CO2 −› PV2)	Θp (CO2 −› PV1)

Note from the block diagram at the top of the article that the inner secondary process dynamics are contained within the overall process dynamics. Therefore, the physics of a cascade implies that the time constant and dead time values for the inner process will not be greater than those of the overall process, or:

*T*p (CO2 −› PV2) ≤ *T*p (CO2 −› PV1) and Θp (CO2 −› PV2) ≤ Θp (CO2 −› PV1)

3) Use the time constants to decide whether the inner secondary dynamics are fast

enough for a PI controller on the inner loop.

Case 1: If the inner secondary process is not at least 3 times faster than the overall process, it is not fast enough for a PI controller.
• If $3 \square T_p$ (CO2 −› PV2) > T_p (CO2 −› PV1) => Use P-Only controller

Case 2: If the inner process is 3 to 5 times faster than the overall process, then P-Only will perform similar to PI control. Use our own preference.
• If $3 \square T_p$ (CO2 −› PV2) ≤ T_p (CO2 −› PV1) ≤ $5 \square T_p$ (CO2 −› PV2) => Use either P-Only or PI controller

Case 3: If the inner process is more than 5 times faster than the overall process, it is "fast" and a PI controller will provide improved performance.
• If $5 \square T_p$ (CO2 −› PV2) < T_p (CO2 −› PV1) => Use PI controller

4) When we have determine whether the inner secondary controller should be a P-Only or PI algorithm, we tune it and test it. Once acceptable performance has been achieved, leave the inner secondary controller in automatic; it now literally becomes part of the outer primary process.

5) Select an algorithm with integral action for the outer primary controller (PI, PID or PID with CO filter) to ensure that offset is eliminated. Tune the primary controller using the design and tuning recipe. Note that bumping the outer primary process requires stepping, pulse or otherwise perturbing the set point (SP2) of the inner secondary controller.

6) With both controllers in automatic, tuning of the cascade is complete.

Cascade Control of the Jacketed Stirred Reactor
Once a cascade control architecture is put in service, we must remember that every time the inner secondary controller is changed in any way, the outer primary controller should be reevaluated for performance and retuned as necessary.

We should also be aware that the recipe presented above is a general procedure intended for broad application. Thus, there will be occasional exceptions to the rules.

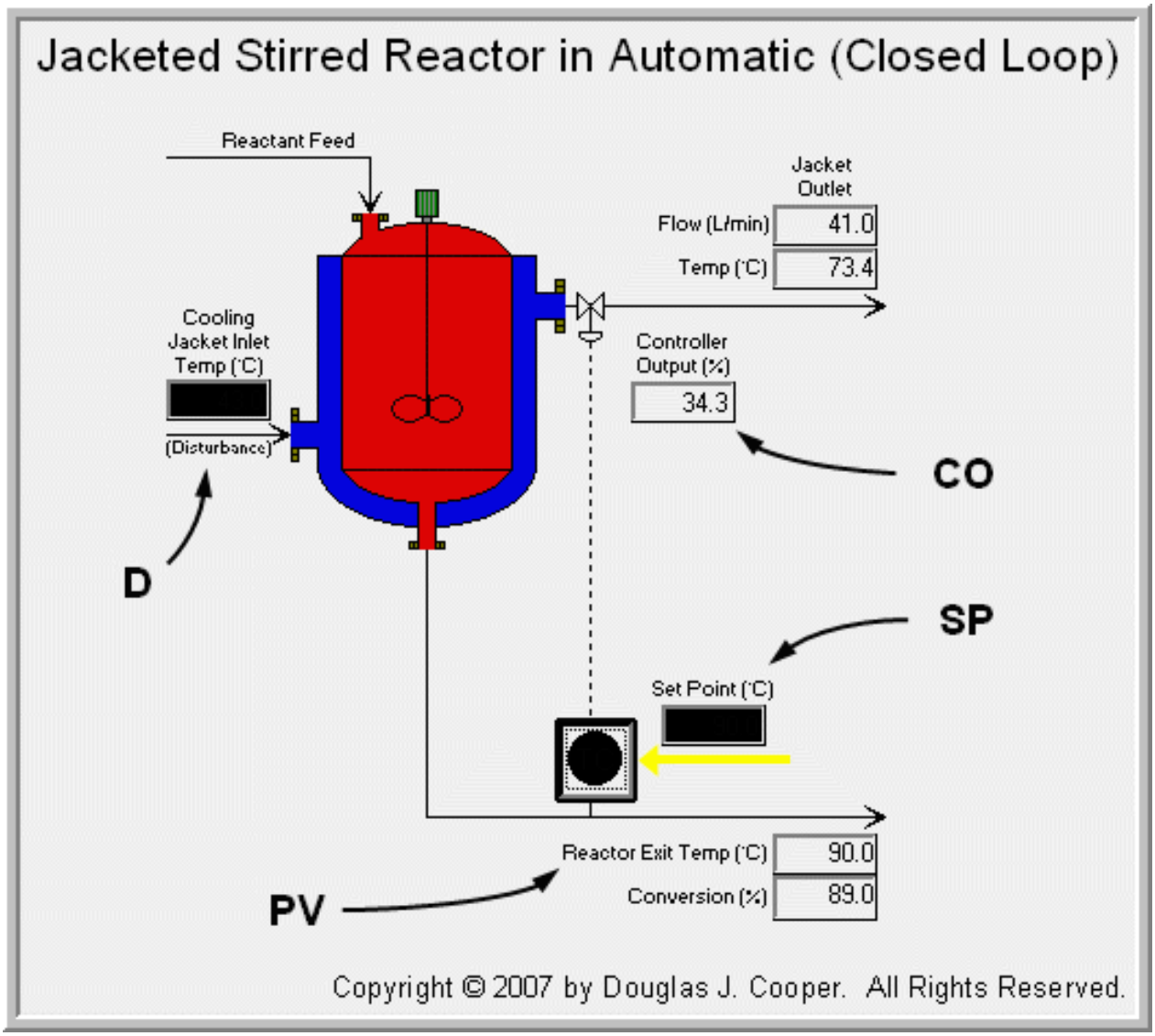
We next explore a case study on the cascade control of the jacketed stirred reactor

A Cascade Control Architecture for the Jacketed Stirred Reactor

Our control objective for the jacketed stirred reactor process is to minimize the impact on reactor operation when the temperature of the liquid entering the cooling jacket changes. We have previously explored the modes of operation and dynamic CO-to-PV behavior of the reactor. We also have established the performance of a single loop PI controller and a PID with CO Filter controller in this disturbance rejection application.

Here we consider a cascade architecture as a means for improving the disturbance rejection performance in the jacketed stirred reactor.

The Single Loop Jacketed Stirred Reactor
As shown in the process graphic below, the reactor exit stream temperature is controlled by adjusting the flow rate of cooling liquid through an outer shell (or cooling jacket) surrounding the main vessel.



As labeled above for the single loop case:
CO = signal to valve that adjusts cooling jacket liquid flow rate (controller output, %)
PV = reactor exit stream temperature (measured process variable, ^{o}C)
SP = desired reactor exit stream temperature (set point, ^{o}C)
D = temperature of cooling liquid entering the jacket (major disturbance, ^{o}C)

The control objective is to maintain the reactor exit stream temperature (PV) at set point (SP) in spite of unmeasured changes in the temperature of cooling liquid entering the jacket (D).

We measure exit stream temperature with a sensor and transmit the signal to a temperature controller (the TC inside the circle in the diagram). After comparing SP to PV, the temperature controller computes and transmits a CO signal to the cooling jacket liquid

flow valve.

As the valve opens and closes, the flow rate of liquid through the jacket increases and decreases. Like holding a hot frying pan under a water faucet, higher flow rates of cooling liquid remove more heat. Thus, a higher flow rate of cooling liquid through the jacket cools the reactor vessel, lowering the reactor exit stream temperature.

Problems with Single Loop Control
The single loop architecture in the diagram above attempts to achieve our control objective by adjusting the flow rate of cooling liquid through the jacket.

If the measured temperature is higher than set point, the controller signals the valve to increase cooling liquid flow by an appropriate percentage with the expectation that this will decrease reactor exit stream temperature accordingly.

A concern discussed in detail in this article is that the temperature of the cooling liquid entering the jacket (D) can change, sometimes rather quickly. This can disrupt reactor operation as reflected in the measured reactor exit stream temperature PV.

So reactor exit stream temperature PV is a function of two variables:
- cooling liquid flow rate, and
- the temperature of the cooling liquid entering the cooling jacket (D).

To explore this, we conduct some thought experiments:

Thought Experiment #1: Assume that the temperature of the cooling liquid entering the jacket (D) is constant over time. If the cooling liquid ***flow rate increases*** by a certain amount, the reactor exit stream ***temperature will decrease*** in a predictable fashion (and vice versa). Thus, a single loop structure should provide good temperature control performance.

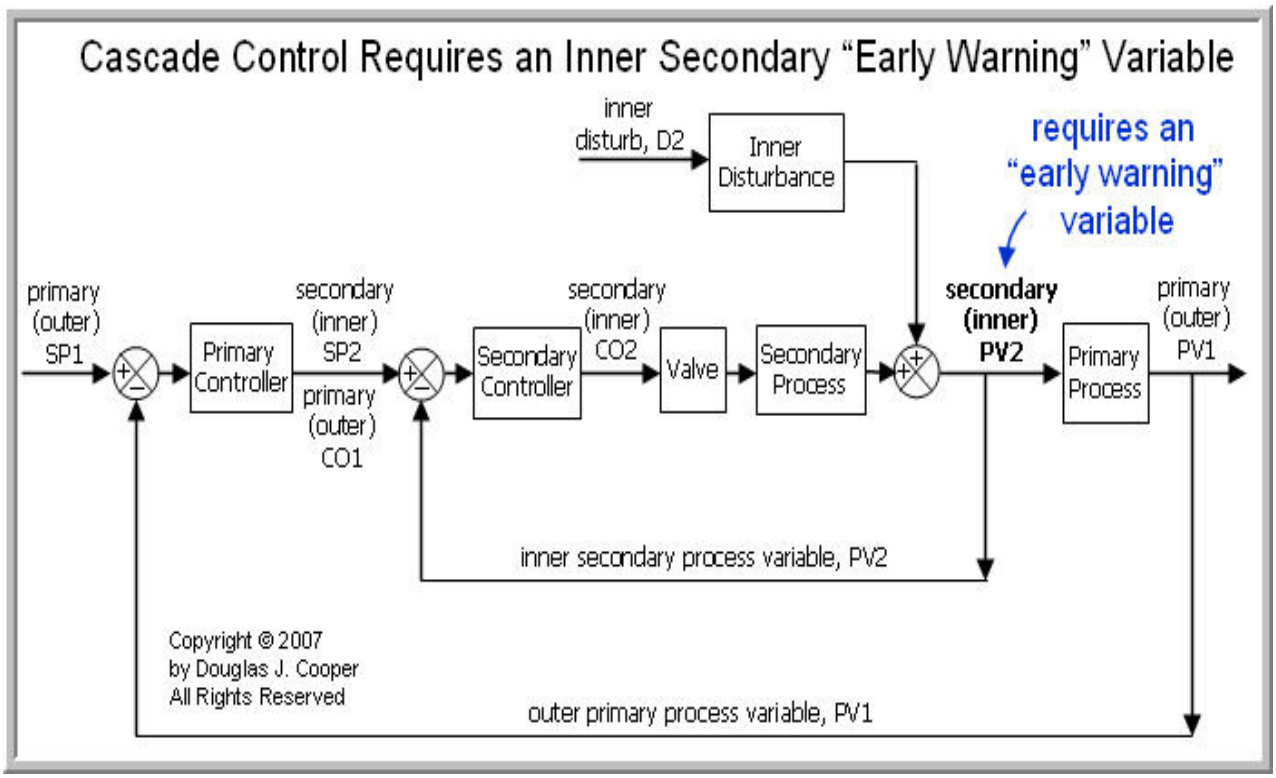
Thought Experiment #2: Assume that the temperature of cooling liquid entering the jacket (D) starts rising over time. A warmer cooling liquid can carry away less heat from the vessel. If the cooling liquid ***flow rate is constant*** through the jacket, the reactor will experience less cooling and the exit stream ***temperature will increase***.

Thought Experiment #3: Now assume that the temperature of cooling liquid entering the jacket (D) starts to rise at the same moment that the reactor exit stream temperature moves above set point. The controller will signal for a cooling liquid ***flow rate increase***, yet because the cooling liquid temperature is rising, the heat removed from the reactor vessel can actually decrease. Until further corrective action is taken, the reactor exit stream ***temperature can increase***.

As presented in Thought Experiment #3, the changing temperature of cooling liquid entering the jacket (a disturbance) can cause a contradictory outcome that can confound the controller and degrade control performance.

Cascade Control Improves Disturbance Rejection
As we established in our study of the cascade control architecture, an essential element for success in a cascade (nested loops) design is the measurement and control of an

"early warning" process variable, PV2, as illustrated in the block diagram below ().



Since disruptions impact PV2 first, it provides our "early warning" that a disturbance is heading toward our outer primary process variable, PV1. The inner secondary controller can begin corrective action immediately. And since PV2 responds first to valve manipulations, disturbance rejection can begin before PV1 has been visibly impacted.

A Reactor Cascade Control Architecture

The thought experiments above highlight that it is problematic to control exit stream temperature by adjusting the cooling liquid flow rate.

An approach with potential for "tighter" control is to adjust the temperature of the cooling jacket itself. This provides a clear process relationship in that, if we seek a higher reactor exit stream temperature, we know we want a higher cooling jacket temperature. If we seek a lower reactor exit stream temperature, we want a lower cooling jacket temperature.

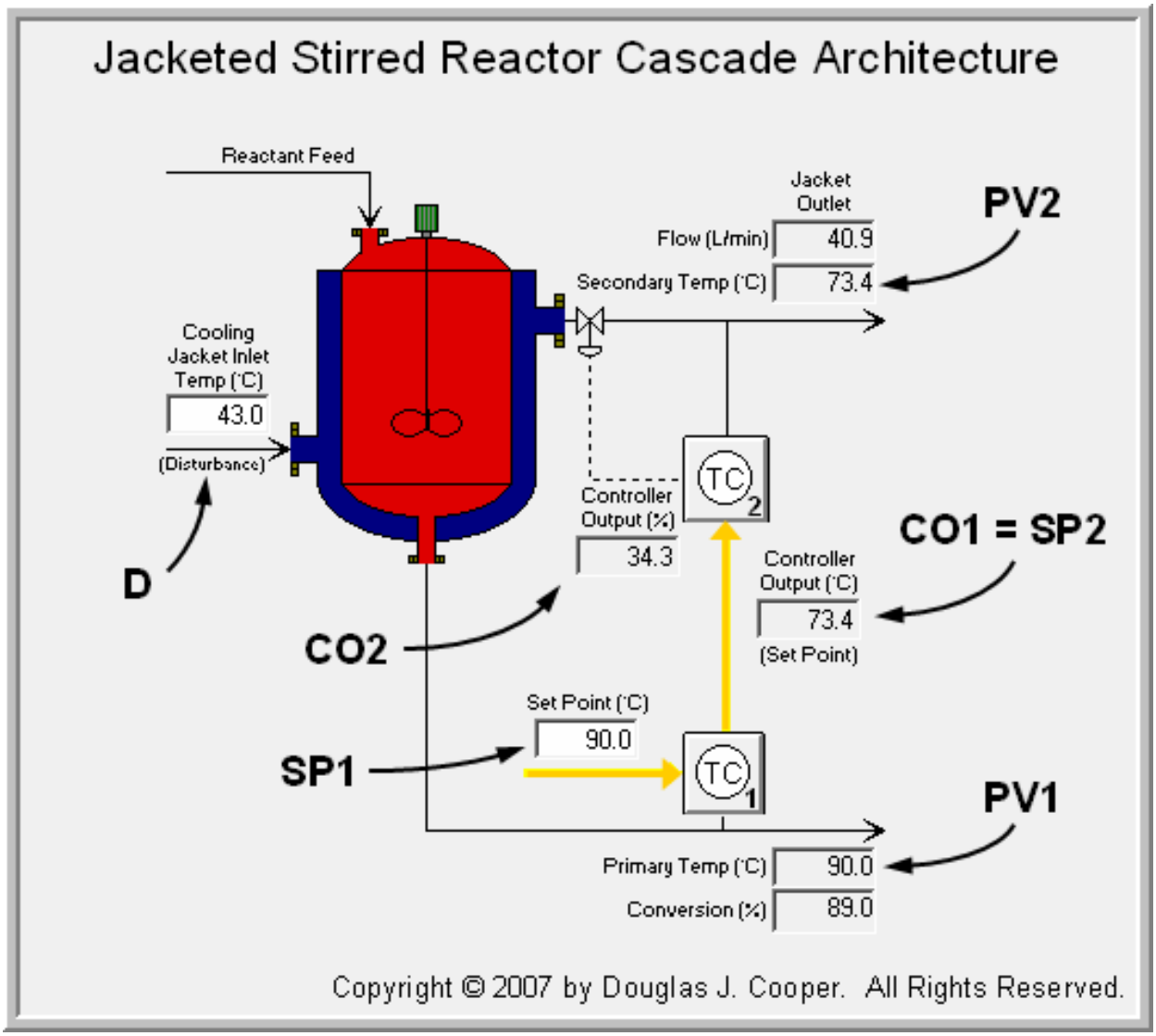
Because the temperature of cooling liquid entering the jacket changes, increasing cooling jacket temperature by a precise amount may mean decreasing the flow rate of cooling liquid a lot, decreasing it a little, and perhaps even increasing the flow rate a bit.

A "cheap and easy" proxy for the cooling jacket temperature is the temperature of cooling liquid exiting at the jacket outlet. Hence, we choose this as our inner secondary process variable, PV2, as we work toward the construction of a nested cascade control architecture.

Adding a temperature sensor that measures PV2 provides us the early warning that changes in D, the temperature of cooling liquid entering the jacket, are about to impact the

reactor exit stream temperature, PV1.

The addition of a second temperature controller (TC2) completes construction of a jacketed reactor control cascade as shown in the graphic below ().



Now, our inner secondary control loop measures the temperature of cooling liquid exiting at the jacket outlet (PV2) and sends a signal (CO2) to the valve adjusting cooling jacket flow rate. The valve increases or decreases the flow rate of cooling liquid if the jacket temperature needs to fall or rise, respectively.

Our outer loop maintains reactor exit stream temperature (our process variable of primary interest and concern) as PV1. Note in the graphic above that the controller output of our primary controller, CO1, becomes the set point of our inner secondary controller, SP2.

If PV1 needs to rise, the primary controller signals a higher set point for the jacket temperature (CO1 = SP2). The inner secondary controller then decides if this means opening or closing the valve and by how much.

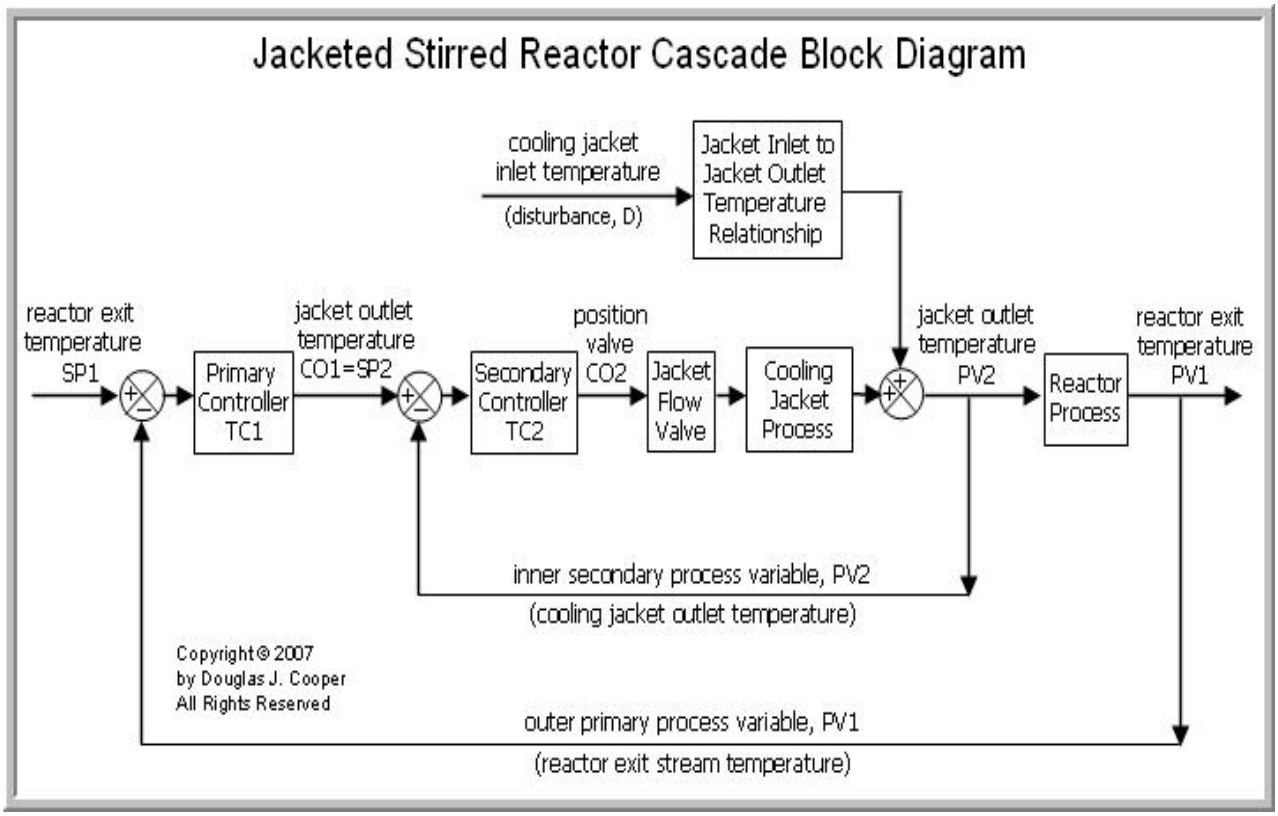
Thus, variations in the temperature of cooling liquid entering the jacket (D) are addressed

quickly and directly by the inner secondary loop to the benefit of PV1.

The cascade architecture variables are identified on the above graphic and listed below:
PV2 = cooling jacket outlet temperature is our "early warning" process variable (^{o}C)
CO2 = controller output to valve that adjusts cooling jacket liquid flow rate (%)
SP2 = **CO1** = desired cooling jacket outlet temperature (^{o}C)
PV1 = reactor exit stream temperature (^{o}C)
SP1 = desired reactor exit stream temperature (^{o}C)
D = temperature of cooling liquid entering the jacket (^{o}C)

The inner secondary PV2 (cooling jacket outlet temperature) is a proper early warning process variable because:
- PV2 is measurable with a temperature sensor.
- The same valve used to manipulate PV1 also manipulates PV2.
- The same disturbance that is of concern for PV1 also disrupts PV2.
- PV2 responds before PV1 to the disturbance of concern and to valve manipulations.

Reactor Cascade Block Diagram
The jacketed stirred reactor block diagram () for this nested cascade architecture is shown below.



As expected for a nested cascade, this architecture has:
- two controllers (an inner secondary and outer primary controller)
- two measured process variable sensors (an inner PV2 and outer PV1)
- only one valve (to adjust cooling liquid flow rate)

Tuning a Cascade

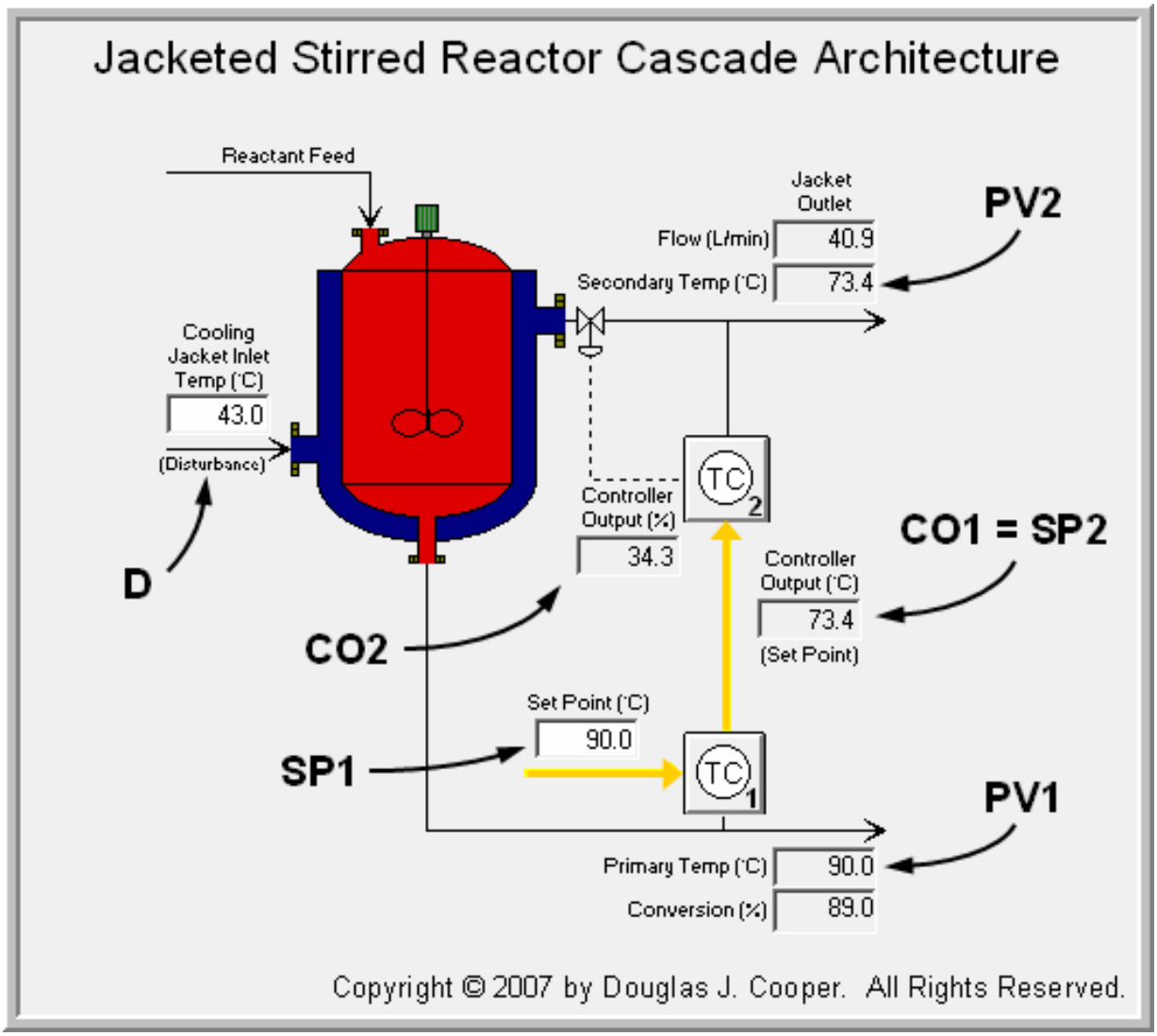
With a cascade architecture established, we apply our implementation recipe for cascade control and explore the disturbance rejection capabilities of this structure.

Cascade Disturbance Rejection in the Jacketed Stirred Reactor

Our control objective for the jacketed stirred reactor is to maintain reactor exit stream temperature at set point in spite of disturbances caused by a changing cooling liquid temperature entering the vessel jacket. In previous articles, we have established the design level of operation for the reactor and explored the performance of a single loop PI controller and a PID with CO Filter controller in meeting our control objective.

We also have proposed a cascade control architecture for the reactor that offers potential for improving disturbance rejection performance. We now apply our proposed architecture following the implementation recipe for cascade control. Our goal is to demonstrate the implementation procedure, understand the benefits and drawbacks of the method, and explore cascade disturbance rejection performance for this process.

The Reactor Cascade Control Architecture
The reactor cascade architecture has been detailed in a previous article and is shown in the graphic below ():



where:

CO2 = controller output to valve that adjusts cooling jacket liquid flow rate (%)
PV2 = cooling jacket outlet temperature is our "early warning" process variable (^{o}C)
SP2 = **CO1** = desired cooling jacket outlet temperature (^{o}C)
PV1 = reactor exit stream temperature (^{o}C)
SP1 = desired reactor exit stream temperature (^{o}C)
D = temperature of cooling liquid entering the jacket (^{o}C)

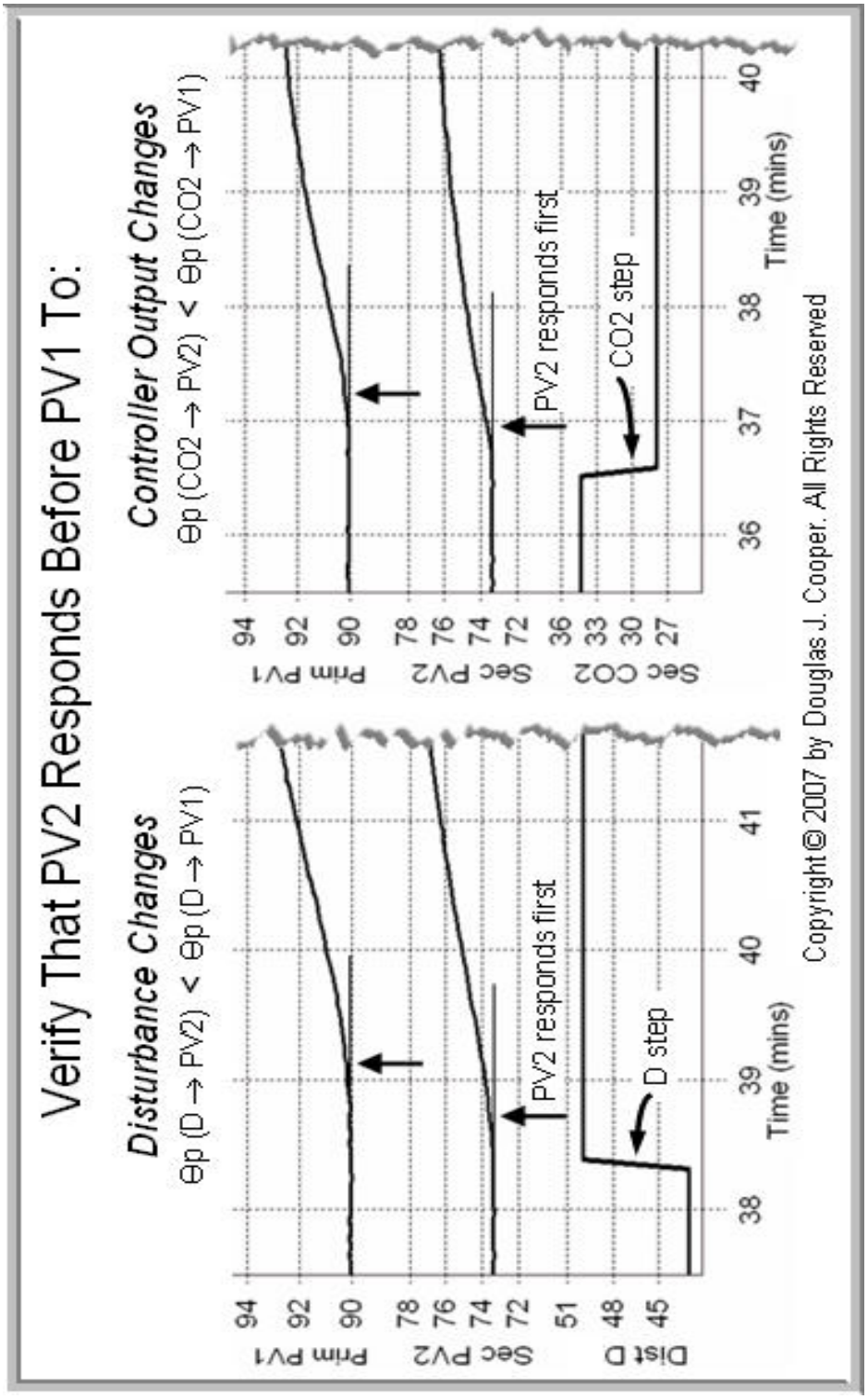
Design Level of Operation (DLO)

The details and discussion for our DLO are presented in this article and are summarized:

- Design PV1 and SP1 = 90 ^{o}C with approval for brief dynamic testing of ±2 ^{o}C
- Design D = 43 ^{o}C with occasional spikes up to 50 ^{o}C

Minimum Criteria for Success

A successful cascade implementation requires that early warning process variable PV2 respond *before* outer primary PV1 both to changes in the jacket cooling liquid temperature disturbance (D) *and* to changes in inner secondary controller output signal CO2. The plots below () verify that both of these criteria are met with the architecture shown in the graphic above.



Expressed concisely, the plots show that the delay in response (or dead time, Θp) follows the rule:

$$\Theta p\,(D \rightarrow PV2) < \Theta p\,(D \rightarrow PV1) \quad \text{and} \quad \Theta p\,(CO2 \rightarrow PV2) < \Theta p\,(CO2 \rightarrow PV1)$$

Thus, a cascade control architecture should improve disturbance rejection performance

relative to a single loop architecture.

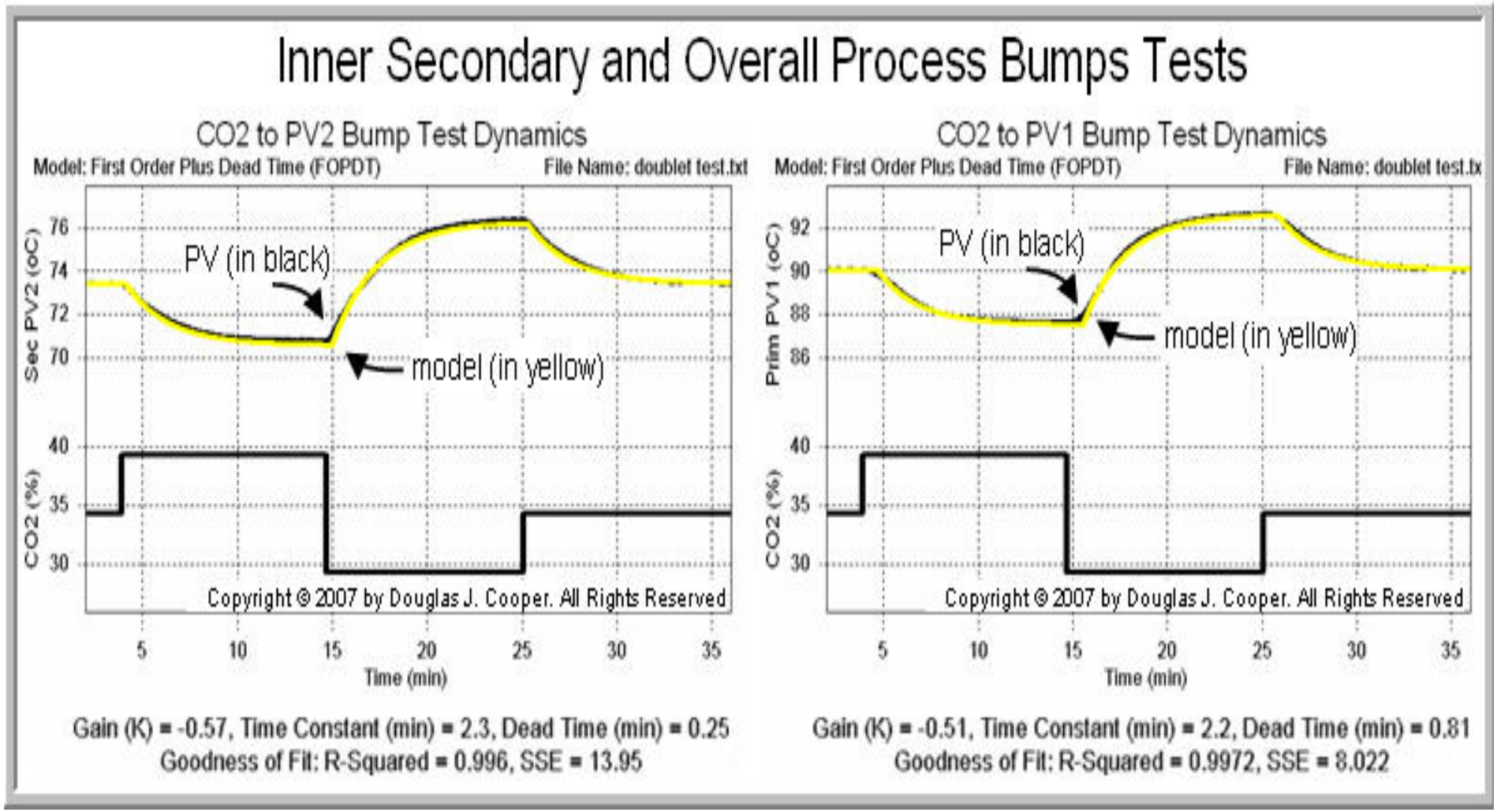
P-Only vs. PI for Inner Secondary Controller
The cascade implementation recipe first helps us decide if the inner secondary controller is fast enough for a PI algorithm or if it is better suited for a P-Only algorithm. The decision is based on the size of the inner secondary time constant relative to that of the overall process time constant.

To compute these time constants, we need to bump the process and analyze the dynamic process response data. In this example we choose to place both inner secondary and outer primary controllers in manual mode (open loop) during the bump test. An alternative not explored here is to have the inner secondary controller in automatic mode with tuning sufficiently active to force a clear dynamic response in PV2.

Following the steps of the cascade implementation recipe:

1) With the process steady at the design level of operation (DLO), we perform a doublet test, choosing here to move CO2 from 34.3% −› 39.3% −› 29.3% −› 34.3%. We record both PV2 and PV1 dynamic data as the process responds. We use this "step 1 data set" as we proceed with the implementation recipe.

2) As shown in the plots below, we use commercial software to fit a first order plus dead time (FOPDT) model to the inner secondary process (CO2 −› PV2) dynamic data and another to the overall process (CO2 −› PV1) dynamic data ():


Inner Secondary and Overall Process Bumps Tests
CO2 to PV2 Bump Test Dynamics
Model: First Order Plus Dead Time (FOPDT)
File Name: doublet test.txt
Sec PV2 (oC)
CO2 (%)
PV (in black)
model (in yellow)
Copyright © 2007 by Douglas J. Cooper. All Rights Reserved
Time (min)
Gain (K) = -0.57, Time Constant (min) = 2.3, Dead Time (min) = 0.25
Goodness of Fit: R-Squared = 0.996, SSE = 13.95
CO2 to PV1 Bump Test Dynamics
Model: First Order Plus Dead Time (FOPDT)
File Name: doublet test.tx
Prim PV1 (oC)
CO2 (%)
PV (in black)
model (in yellow)
Copyright © 2007 by Douglas J. Cooper. All Rights Reserved
Time (min)
Gain (K) = -0.51, Time Constant (min) = 2.2, Dead Time (min) = 0.81
Goodness of Fit: R-Squared = 0.9972, SSE = 8.022

e FOPDT model parameters from these bump tests are summarized:

	Inner Secondary (CO2 −› PV2)	**Overall Process** (CO2 −› PV1)
Process gain, Kp =	– 0.57 °C/%	– 0.51 °C/%
Time constant, *T*p =	2.3 min	2.2 min
Dead time, Θp =	0.25 min	0.81 min

The cascade block diagram implies that the time constant and dead time values for the inner process are contained within and contribute to the dynamics of the overall process. Thus, we can surmise that:

$$T_p\,(CO2 \text{ −› } PV2) \le T_p\,(CO2 \text{ −› } PV1) \quad \text{and} \quad \Theta_p\,(CO2 \text{ −› } PV2) \le \Theta_p\,(CO2 \text{ −› } PV1)$$

Yet the values in the table above indicate that this seemingly fundamental relationship does not hold true for the jacketed stirred reactor. In fact, when using a FOPDT model approximation of the dynamic response data, the time constant of the inner secondary process is slightly larger (longer) than that of the overall process.

The process graphic at the top of this article shows a main reactor vessel with a large volume of liquid relative to that of the cooling jacket. The function of the cooling jacket is to remove heat energy to regulate reactor vessel temperature. It seems logical that because of the volume differences, as long as the liquid temperature in the large reactor is changing, the temperature of the liquid in the small cooling jacket must follow. And the liquid in the reactor acts as a heat source because the chemical reaction inside the vessel generates heat energy faster as reactor temperature rises (and vice versa).

This intertwined relationship of heat generation and removal combined with relative sizes of the reactor and jacket offers one physical rationalization as to why the jacket (inner secondary) *T*p might reasonably be longer than that of the vessel (overall process) *T*p.

A simpler explanation is that the sensor used to measure temperature in the cooling jacket outlet flow was improperly specified at the time of purchase, and unfortunately, it responds slowly to actual cooling liquid changes. This additional response time alone could account for the observed behavior.

In any case, the dead time of the overall process is three times that of the inner secondary controller, and thus, PV2 provides a clear early warning that we can exploit for a cascade design. So while the simple cascade recipe has limitations that require our judgment, it provides valuable insights that enable us to proceed.

3) The cascade implementation recipe uses a time constant comparison to decide whether the inner secondary loop is fast enough for a PI controller. We reach a true statement with case 1 of the decision tree in step 3 of the recipe. That decision is:

$$\text{if } 3 \square T_p\,(CO2 \text{ −› } PV2) > T_p\,(CO2 \text{ −› } PV1) \Rightarrow \text{Use P-Only controller}$$

or using the parameters in the table above:

if 3(2.3 min) > 2.2 min => Use a P-Only controller

Inner Secondary Controller

4) Our cascade implementation recipe states that a P-Only algorithm is the best choice for the inner secondary controller. To explore this decision, we run four trials and compare P-Only and PI algorithms side-by-side. We are able to use the same "step 1 data set" for the design and tuning of all four of these inner secondary controllers:

TRIAL 1: moderate P-Only
TRIAL 2: aggressive P-Only
TRIAL 3: moderate PI
TRIAL 4: aggressive PI

As listed in the table in step 2 above, a first order plus dead time (FOPDT) model fit of the "step 1 data set" collected around our design level of operation (DLO) yields inner secondary FOPDT model parameters:

CO2 −› PV2 model: Kp = – 0.57 °C/%; Tp = 2.3 min; Θp = 0.25 min

Following our controller design and tuning recipe, we use these FOPDT model parameters in rules and correlations to complete the secondary controller design.

P-Only Controller design and tuning, including the use of the ITAE tuning correlation for computing controller gain, Kc, from FOPDT model parameters is summarized in the example in this article. Following those details:

P-Only algorithm: $CO = CO_{bias} + Kc \cdot e(t)$

TRIAL 1: Moderate P-Only

$$Kc = \frac{0.2}{Kp}\left(\frac{Tp}{\theta p}\right)^{1.22} = \frac{0.2}{-0.57}\left(\frac{2.3}{0.25}\right)^{1.22} = -5.3\ \%/^{o}C$$

TRIAL 2: Aggressive P-Only

Kc = 2.5 (Moderate Kc) = 2.5 (–5.3) = – 13.3 %/^{o}C

PI Controller design and tuning, including computing controller gain, Kc, and reset time, Ti, from FOPDT model parameters is summarized in the example in this article. Following those details:

PI algorithm: $$CO = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{Ti}\int e(t)dt$$

TRIAL 3: Moderate PI

Moderate Tc = the larger of 1·Tp or 8·Θp
= larger of 1(2.3 min) or 8(0.25 min)
= 2.3 min

$$Kc = \frac{1}{Kp}\frac{\tau p}{(\theta p + \tau c)} = \frac{1}{-0.57}\frac{2.3}{(0.25+2.3)} = -1.6\ \%/^{o}C$$

$$\tau i = \tau p = 2.3\ min$$

TRIAL 4: Aggressive PI

Aggressive τc = the larger of $0.1 \cdot \tau p$ or $0.8 \cdot \theta p$
= larger of 0.1(2.3 min) or 0.8(0.25 min)
= 0.23 min

$$Kc = \frac{1}{Kp}\frac{\tau p}{(\theta p + \tau c)} = \frac{1}{-0.57}\frac{2.3}{(0.25+0.23)} = -8.4\ \%/^{o}C$$

$$\tau i = \tau p = 2.3\ min$$

The "step 1 data set" model parameters and the four inner secondary controllers designed from this data are summarized in the upper half of the table below:

Inner Secondary CO2 Bump Test: from 34.3% → 39.3% → 29.3% → 34.3%
FOPDT CO2→ PV2 Model: Kp = −0.57 °C/%; τp = 2.3 min; θp = 0.25 min

	TRIAL 1	TRIAL 2	TRIAL 3	TRIAL 4
Inner Secondary P-Only and PI Controllers from above inner FOPDT model				
Kc [=] %/°C τi [=] min	Moderate P-Only Kc = –5.3	Aggressive P-Only Kc = –13.3	Moderate PI Kc = –1.6; τi = 2.3	Aggressive PI Kc = –8.4; τi = 2.3
Outer Primary FOPDT CO1=SP2 → PV1 Models using above inner controllers				
Kp (°C/°C)	0.70	0.82	0.90	0.93
τp (min)	0.61	0.43	2.2	0.50
θp (min)	0.63	0.48	0.82	0.62
Outer Primary PI Controllers with moderate tuning using above FOPDT models				
Kc [=]°C/°C τi [=] min	Moderate PI Kc =0.15 τi =0.61	Moderate PI Kc =0.12 τi =0.43	Moderate PI Kc =0.33 τi =2.2	Moderate PI Kc =0.10 τi =0.50

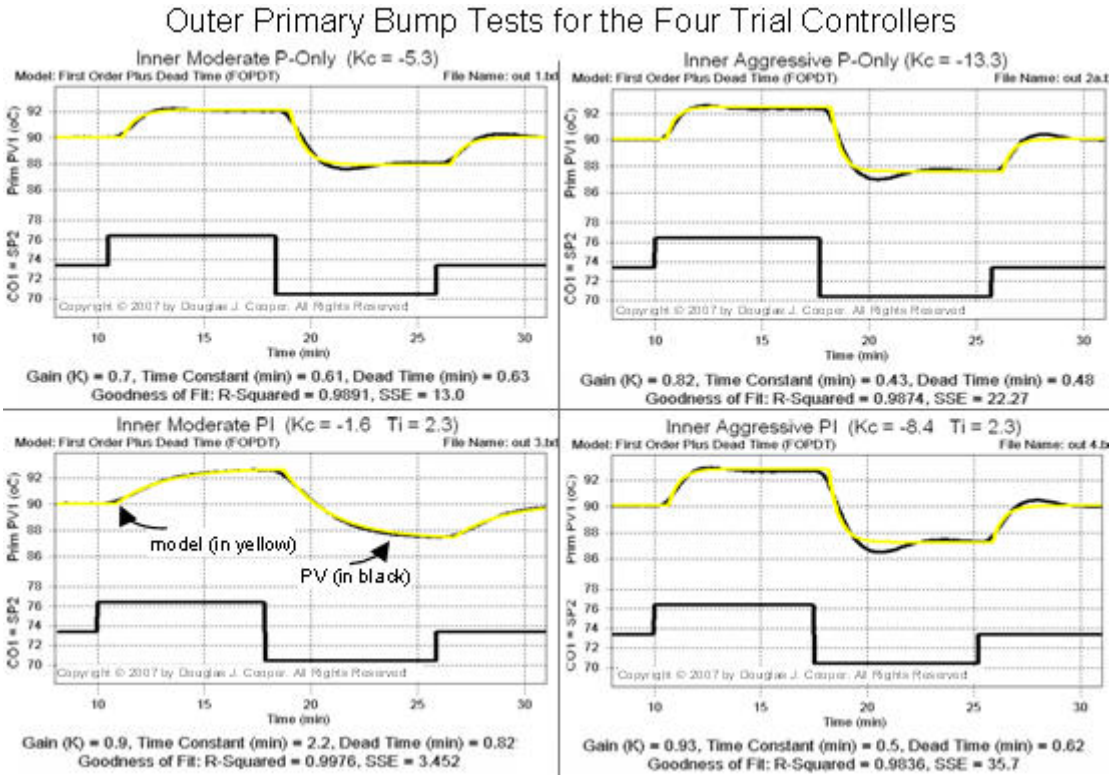
Outer Primary Controller

5) We normally would implement one inner secondary controller and test it for acceptable performance. Here, we "accept" each of the four trial controllers and turn our attention to the outer primary loop.

The four inner secondary controllers are thus implemented one after the other in a series of studies. When in automatic mode, each literally becomes part of the overall process. And because each is different, we must perform four separate bump tests and compute four sets of FOPDT model parameters to describe the four different outer primary (overall) dynamic process behaviors.

Recall that the output of the outer primary controller output, CO1, becomes the set point of the inner secondary controller, SP2. Thus, bumping CO1 is the same as bumping SP2 (i.e., CO1=SP2).

With each of the four inner secondary trial controllers in automatic, we choose to bump CO1=SP2 from 73.4 ^{o}C −› 76.4 ^{o}C −› 70.4 ^{o}C −› 73.4 ^{o}C. We again use commercial software to fit a FOPDT dynamic model to the CO1=SP2 −› PV2 dynamic response data sets as shown in the plots below (). The FOPDT model parameters (Kp, *T*p, Θp) from each bump test fit are summarized in the table above.



We find that industry practitioners, when designing a strategy for an application as challenging as reactor temperature control, generally seek a moderate response performance. Integral action will always be included to eliminate offset issues. Thus, we pair each inner secondary trial controller with a moderately tuned outer primary PI controller.

Following the identical procedure detailed in trial 3 of step 4 above, we compute four sets of moderate PI controller tuning parameters (one for each inner secondary controller). These are listed in the last row of the above table.

Compare Disturbance Rejection Performance

6) With both controllers in automatic, design and implementation of the cascade is complete. The objective of this cascade is to minimize the disruption to primary process variable PV1 when disturbance D changes.

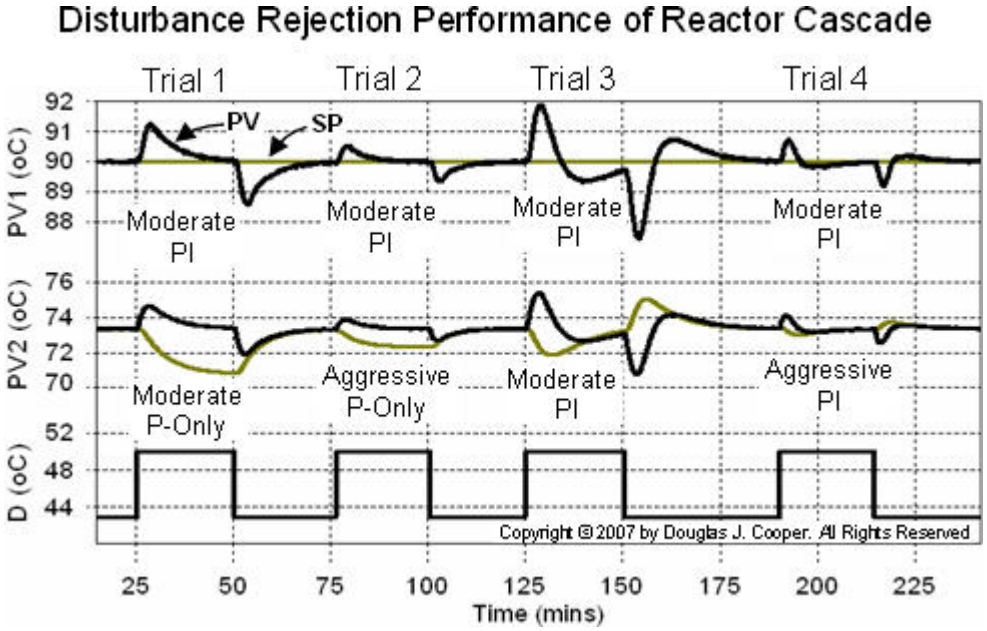
The specific D of concern in this study is that the temperature of cooling liquid entering the jacket, normally at 43 ^{o}C, is known to spike occasionally to 50 ^{o}C.

The lower trace in the plot below () shows disturbance steps (temperature changes) from

43 ^{o}C up to 50 ^{o}C and back. There are four trials shown, one for each of the inner secondary and outer primary controller pairs listed in the table above.

The middle portion of the plot shows the constantly moving inner secondary SP2=C01 in gold and the ability of early warning PV2 to track this ever-changing set point in black.

Recall that offset for early warning variable PV2 is not a concern in many cascade implementations. Here, for example, PV2 is cooling liquid temperature and cooling liquid is not a product destined for market. So our central focus is on how control actions based on this early warning variable help us minimize disturbance disruptions to PV1 and not on how precisely PV2 tracks set point.



Our success is shown on the upper portion of the plot. The outer primary set point, SP1 (in gold) remains constant throughout the study. Our interest is in the ability of each of the cascade implementations to maintain reactor exit temperature PV1 at SP1 in spite of the abrupt disturbance changes.

Some Observations

In general, a more successful (or "better") cascade performance is one where:

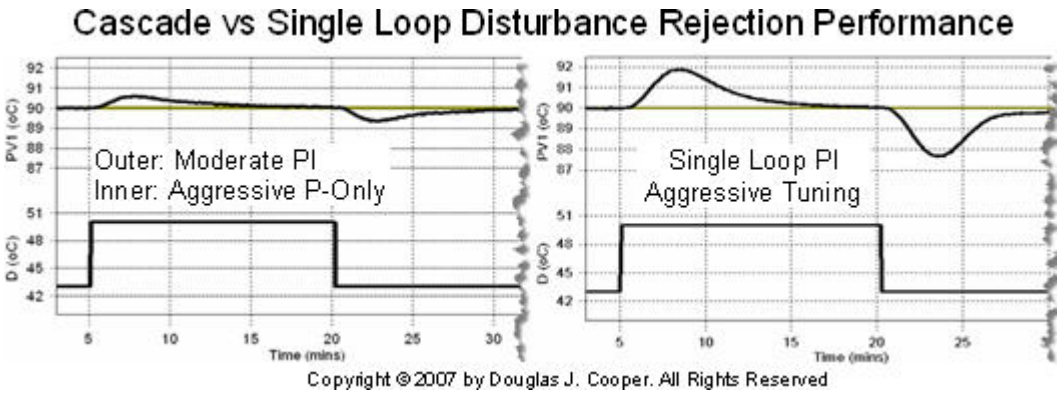
- there is a smaller maximum deviation from set point during the disturbance, and
- PV1 most rapidly returns to and settles out at set point after a disturbance.

• Trials 2 and 4 both have aggressively tuned inner secondary controllers, and these two implementations both have the smallest deviations from set point and settle most rapidly back to SP. This supports the notion that *inner secondary controllers should energetically attack early warning PV2 disruptions* for best cascade performance.

• While an aggressively tuned inner secondary P-Only and PI controller (trials 2 and 4) performed with similar success, the moderately tuned inner secondary PI controller (trial 3) displayed markedly degraded performance. This high sensitivity to inner loop PI tuning strengthens the "use P-Only" conclusion made in step 3 of our cascade implementation recipe.

Comparing Cascade to Single Loop
The central question is whether the extra effort associated with cascade control provides sufficient payoff in the form of improved disturbance rejection performance.

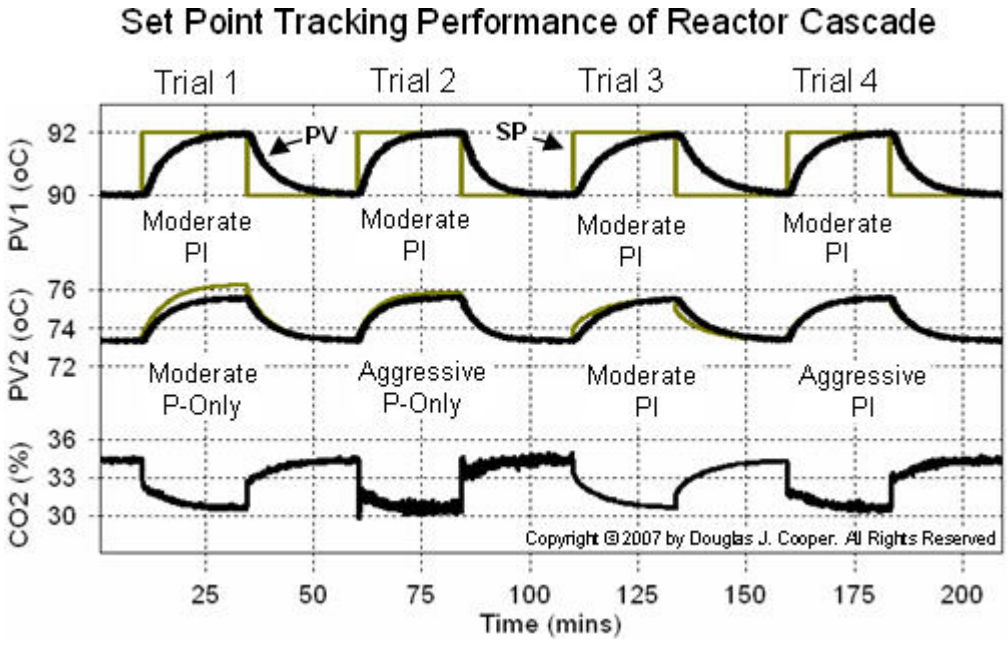
To the left in the plot below () is the performance of our trial 2 cascade implementation. The performance of an aggressively tuned single loop PI controller (design details in this article) in rejecting the same disturbance is shown to the right.



The cascade architecture reduces the maximum deviation from SP during the disturbance from ±2 ^{o}C for the single loop controller down to ±0.5 ^{o}C for the cascade. Settling time is shortened from about 10 minutes for the single loop controller down to about 8 minutes for the cascade.

If the financial return from such improved performance is greater than the cost to install and maintain the cascade, then choose cascade control.

Set Point Tracking Performance
While not our design objective, presented below is the set point tracking performance of the four cascade implementations ():



Cascade control is best suited for improved disturbance rejection. As shown above, its

impact on set point tracking performance is minimal. While one might argue that our "best" cascade design for disturbance rejection (trial 2) also provides the most rapid SP tracking response, this same improvement can be obtained with more aggressive tuning of a single loop PI controller.

16) Feed Forward with Feedback Trim For Improved Disturbance Rejection

The Feed Forward Controller

The most popular architectures for improved disturbance rejection performance are cascade control and the "feed forward with feedback trim" architecture introduced below.

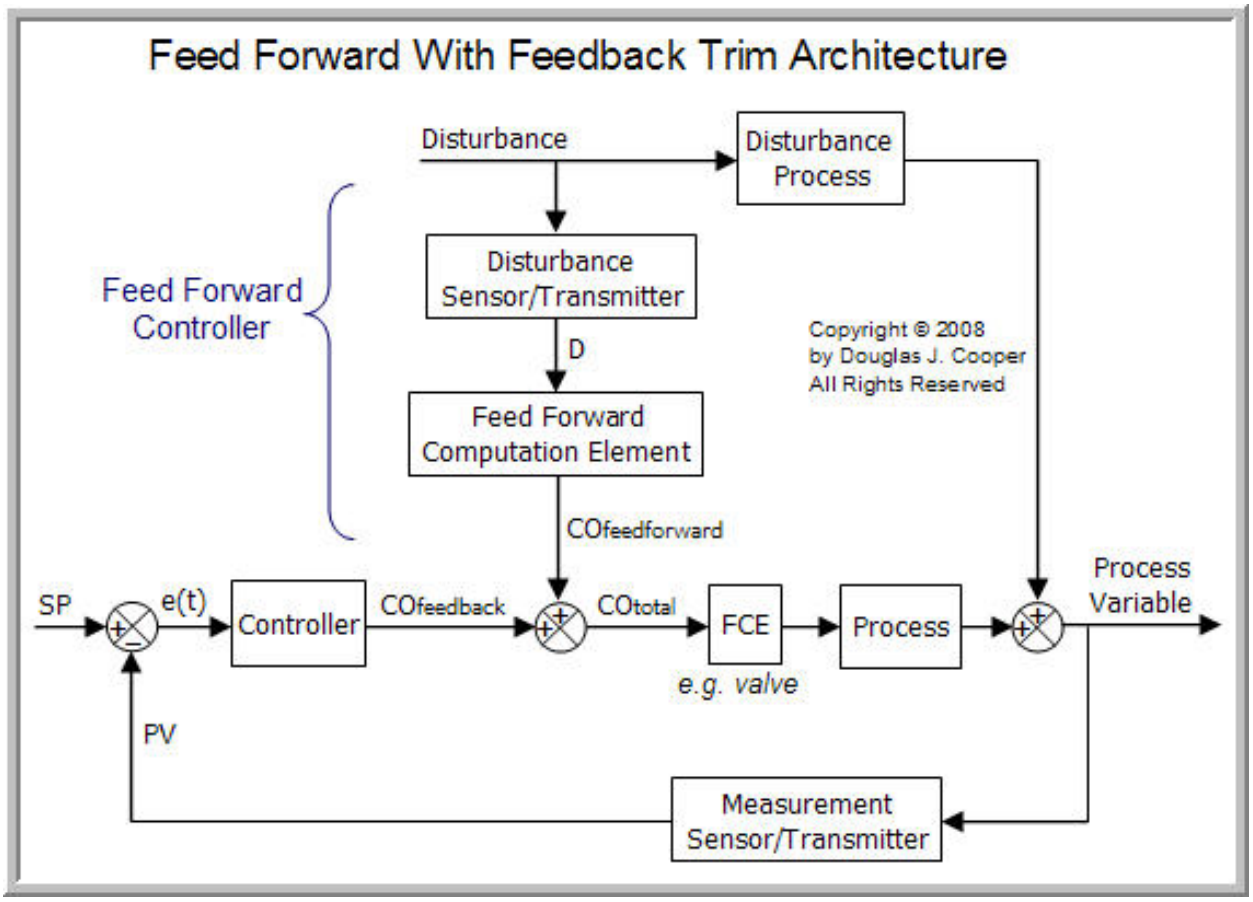
Like cascade, feed forward requires that additional instrumentation be purchased, installed and maintained. Both architectures also require additional engineering time for strategy design, implementation and tuning.

Cascade control will have a small impact on set point tracking performance when compared to a traditional single-loop feedback design and this may or may not be considered beneficial depending on the process application. The feed forward element of a "feed forward with feedback trim" architecture does not impact set point tracking performance in any way.

Feed Forward Involves a Measurement, Prediction and Action
Consider that a process change can occur in another part of our plant and an identifiable series of events then leads that “distant” change to disturb or disrupt our measured process variable, PV.

The traditional PID controller takes action only when the PV has been moved from set point, SP, to produce a controller error, e(t) = SP – PV. Thus, disruption to stable operation is already in progress before a feedback controller first begins to respond. From this view, a feedback strategy simply starts too late and at best can only work to minimize the upset as events unfold.

In contrast, a feed forward controller measures the disturbance, D, while it is still distant. As shown below (), a feed forward element receives the measured D, uses it to predict an impact on PV, and then computes preemptive control actions, COfeedforward, that counteract the predicted impact as the disturbance arrives. The goal is to maintain the process variable at set point (PV = SP) throughout the disturbance event.



where:
CO = controller output signal
D = measured disturbance variable
e(t) = controller error, SP – PV
FCE = final control element (e.g., valve, variable speed pump or compressor)
PV = measured process variable
SP = set point

To appreciate the additional components associated with a feed forward controller, we can compare the above to the previously discussed traditional feed back control loop block diagram.

When to Consider Cascade Control

The cascade architecture requires that an "early warning" secondary measured process variable, PV2, be identified that is inside (responds before) the primary measured process variable, PV1. Essential elements for success include that:

- PV2 is measurable with a sensor.
- The same final control element (FCE) used to manipulate PV1 also manipulates PV2.
- The same disturbances that are of concern for PV1 also disrupt PV2.
- PV2 responds *before* PV1 to disturbances of concern and to FCE manipulations.

One benefit of a cascade architecture is that it uses two traditional controllers from the PID family, so implementation is a familiar task that builds upon our existing skills. Also, cascade control will help improve the rejection of *any* disturbance that first disrupts the early warning variable, PV2, prior to impacting the primary process variable, PV1.

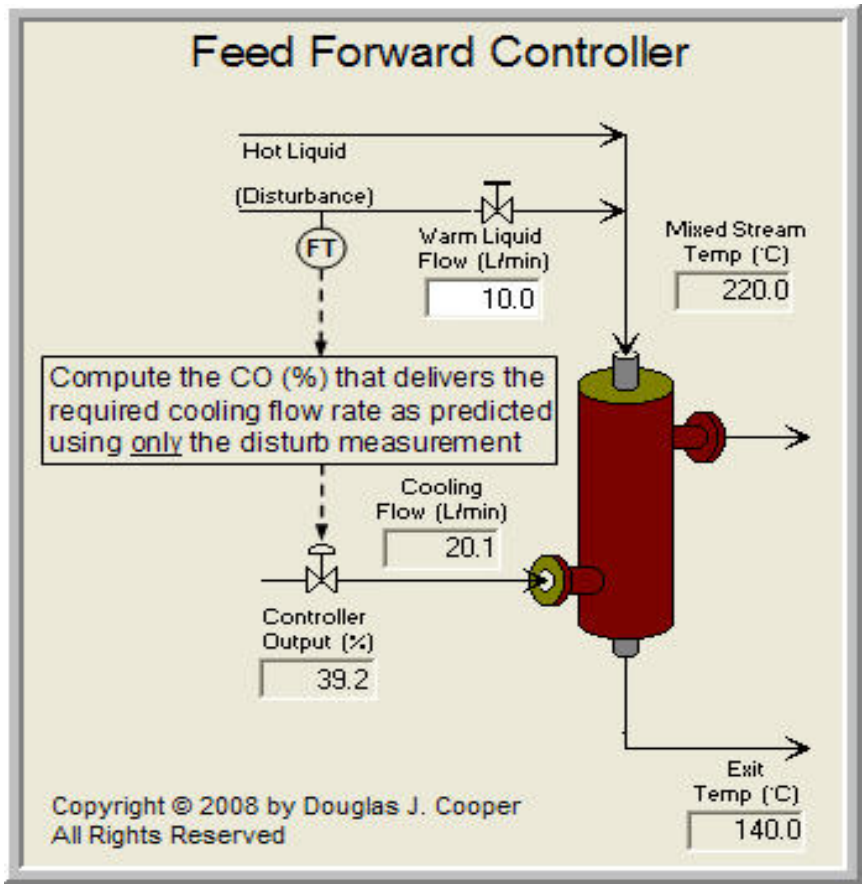
When to Consider Feed Forward with Feedback Trim
Feed forward anticipates the impact of a measured disturbance on the PV and deploys control actions to counteract the impending disruption in a timely fashion. This can significantly improve disturbance rejection performance, but only for the particular disturbance variable being measured.

Feed forward with feedback trim offers a solution for improved disturbance rejection if no practical secondary process variable, PV2, can be established (*i.e.*, a process variable cannot be located that is measureable, provides an early warning of impending disruption, and responds first to FCE manipulations).

Feed forward also has value if our concern is focused on one specific disturbance that is responsible for repeated, costly disruptions to stable operation. To provide benefit, the additional measurement must reveal process disturbances *before* they arrive at our PV so we have time to compute and deploy preemptive control actions.

The Feed-Forward-Only Controller
Pure feed-forward-only controllers are rarely found in industrial applications where the process flow streams are composed of gases, liquids, powders, slurries or melts. Nevertheless, we explore this idea using a thought experiment on the shell-and-tube heat exchanger simulation detailed in a previous article and available for exploration and study in commercial software.

The architecture of a feed-forward-only controller for the heat exchanger is illustrated below ():



As detailed in the referenced article, the PV to be controlled is the exit temperature on the tube side of the exchanger. To regulate this exit temperature, the CO signal adjusts a valve to manipulate the flow rate of cooling liquid on the shell side. A side stream of warm liquid combines with the hot liquid entering the exchanger and acts as a measured disturbance, D, to our process.

Because there is no feedback of a PV measurement in our controller architecture, feed-forward-only presents the interesting notion of open loop control. As such, it does not have a tendency to induce oscillations in the PV as can a poorly tuned feedback controller.

If we could mathematically describe how each change in D impacts PV (D −› PV) and how each change in CO impacts PV (CO −› PV), then we could develop a math model that predicts what manipulations to make in CO to maintain PV at set point whenever D changes.

But this would only be true if:
- we have perfect understanding of the D −› PV and CO −› PV dynamic relationships,
- we can describe these perfect dynamic relationships mathematically,
- these relationships never change,
- there are no other unmeasured disturbances impacting PV, and
- set point, SP, is always held constant.

The reality, however, is that with only a single measured D, a predictive model cannot account for many phenomena that impact the D −› PV and CO −› PV behavior. These may include changes in:

- the temperature and flow rate of the hot liquid feed that mixes with our warm disturbance stream on the tube side,
- the temperature of the cooling liquid on the shell side,
- the ambient temperature surrounding the exchanger that drives heat loss to the environment,
- the shell/tube heat transfer coefficient due to corrosion or fouling, and
- valve performance and capability due to wear and component failure.

Since all of the above are unmeasured, a fixed or stationary model cannot account for them when it computes control action predictions. Installing additional sensors and enhancing the feed forward model to account for each would improve performance but would lead to an expensive and complex architecture. And since there are more potential disturbances and external influences then those listed above, that still would not be sufficient.

This highlights that feed-forward-only control is problematic and should only be considered in rare instances. One situation where it may offer value is if a PV critical to process operation simply cannot be measured or inferred using currently available technology. Feed-forward-only control, in spite of its weaknesses and pitfalls, then offers some potential for improved operation.

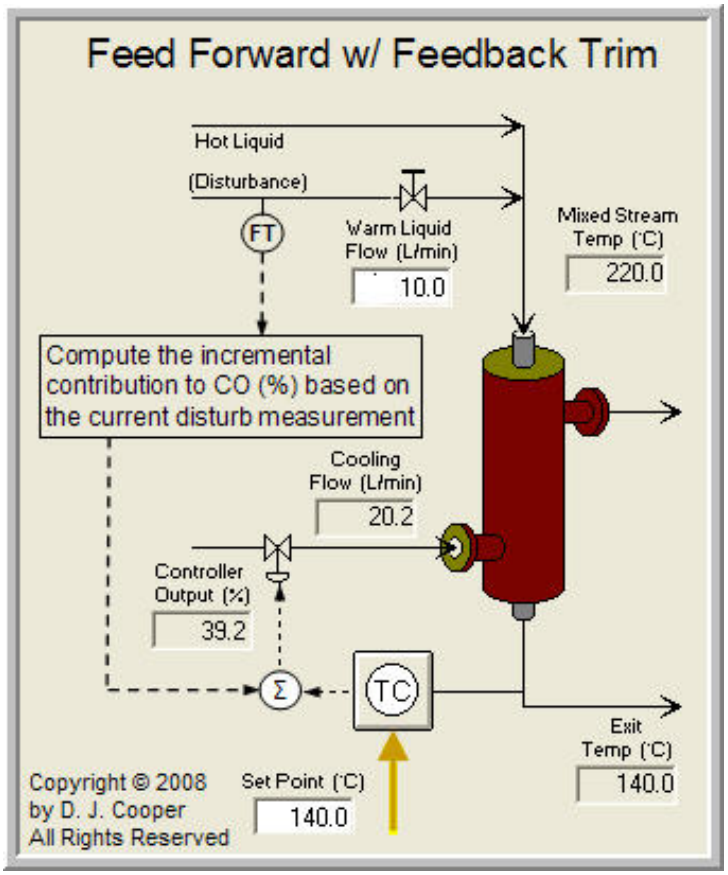
Feed Forward with Feedback Trim

The "feed forward with feedback trim" control architecture is the solution widely employed in industrial practice. It balances the capability of a feed forward element to take

preemptive control actions for *one* particularly disruptive disturbance while permitting a traditional feedback control loop to:

- reject all other disturbances and external influences that are not measured,
- provide set point tracking capability, and
- correct for the inevitable simplifying approximations in the predictive model of the feed forward element that make preemptive disturbance rejection imperfect.

A feed forward with feedback trim control architecture for the heat exchanger process is shown below ():



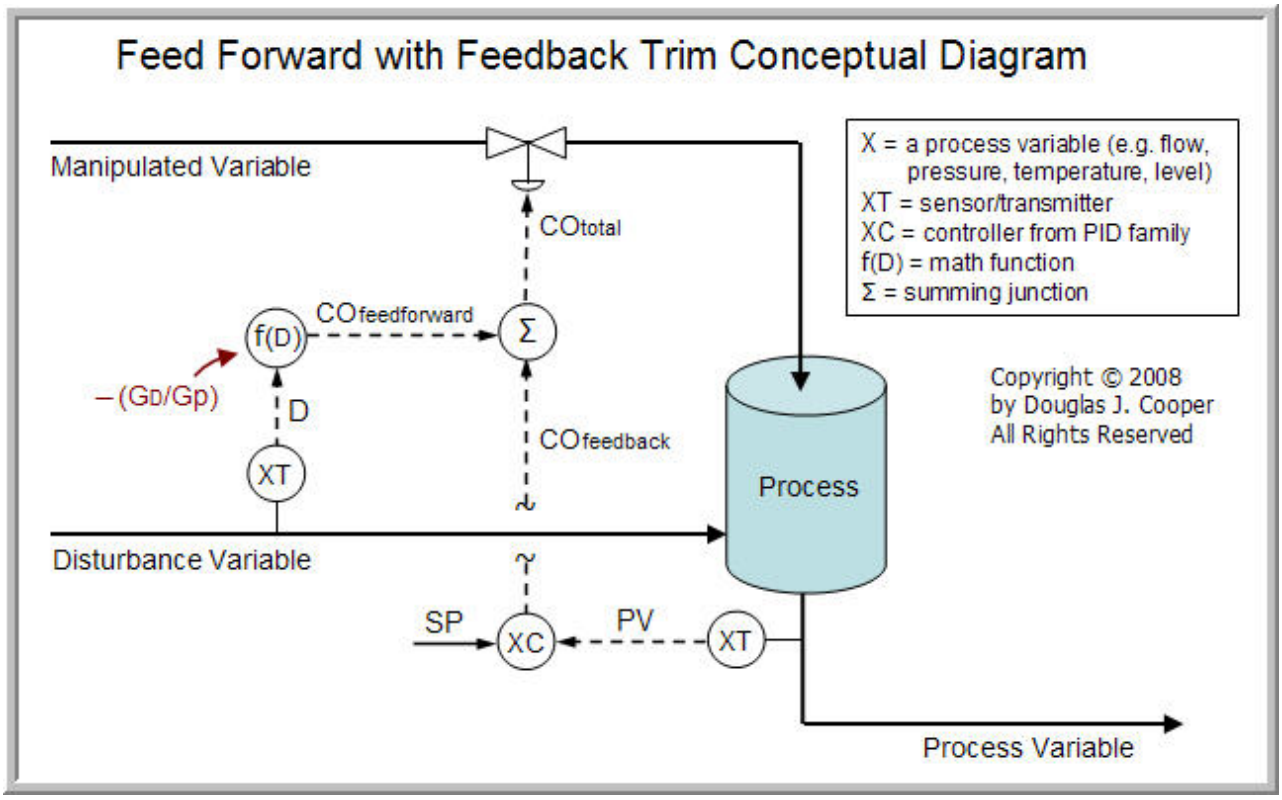
To construct the architecture, a feedback controller is first implemented and tested following our controller design and tuning recipe as if it were a stand-alone entity. The feed forward controller is then designed based on our understanding of the relationship between the D −› PV and CO −› PV variables. This is generally expressed as a math function that can range in complexity from a simple multiplier to complex differential equations.

With the architecture completed, the disturbance flow is measured and passed to a feed forward element that is essentially a combination disturbance/process model. The model uses changes in D to predict an impact on PV, and then computes control actions, COfeedforward to compensate for the predicted impact.

Conceptual Feed Forward with Feedback Trim Diagram
As shown in the first block diagram above, the COfeedforward control actions are combined with COfeedback to create an overall control action, COtotal, to send to the final

control element (FCE).

To illustrate the control strategy in a more tangible fashion, we present the feed forward with feedback trim architecture in a conceptual diagram below ():



The basis for the feed forward element math function, f(D) = — (GD/Gp) shown in the diagram is discussed in detail in this next article.

The more accurate the feed forward math function is in computing control actions that will counteract changes in the measured disturbance in a timely fashion, the less impact those disturbances will have on our measured process variable.

> Practitioner's note: a potential application of feed forward control exists if we hear an operator say something like, “Every time event *A* happens, process variable *X* is upset. I can usually help the *X* controller by switching to manual mode and moving the controller output.” If the variable associated with event *A* is already being measured and logged in the process control system, sufficient data is likely available to allow the implementation of a feed forward element to our feedback controller.

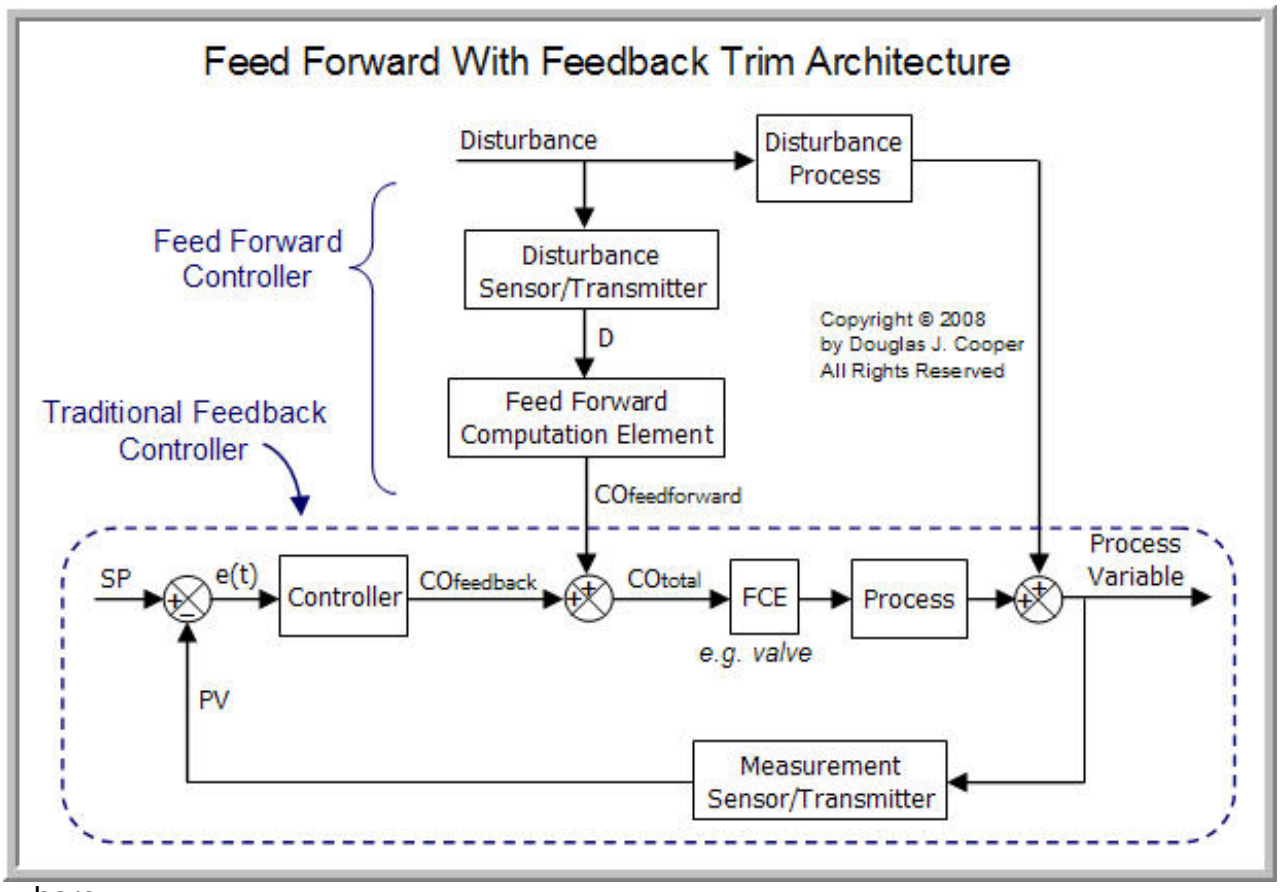
Improved disturbance rejection performance comes at a price in terms of process engineering time for model development and testing, and instrument engineering time for control logic programming. Like all projects, such investment decisions are made on the basis of cost and benefit.

Feed Forward Uses Models Within the Controller Architecture

Both "feed forward with feedback trim" and cascade control can provide improved disturbance rejection performance. They have different architectures, however, and choosing between the two depends on our specific control objective and the ability to obtain certain process measurements.

Feed Forward and Feedback Trim are Largely Independent
As illustrated below (), the feed forward with feedback trim architecture is constructed by coupling a feed-forward-only controller to a traditional feedback controller.

The feed forward controller seeks to reject the impact of one specific disturbance, D, that is measured *before* it reaches our primary process variable, PV, and starts its disruption to stable operation. Typically, this D is one that has been identified as causing repeated and costly upsets, thus justifying the expense of both installing a sensor to measure it, and developing and implementing the feed forward computation element to counteract it.

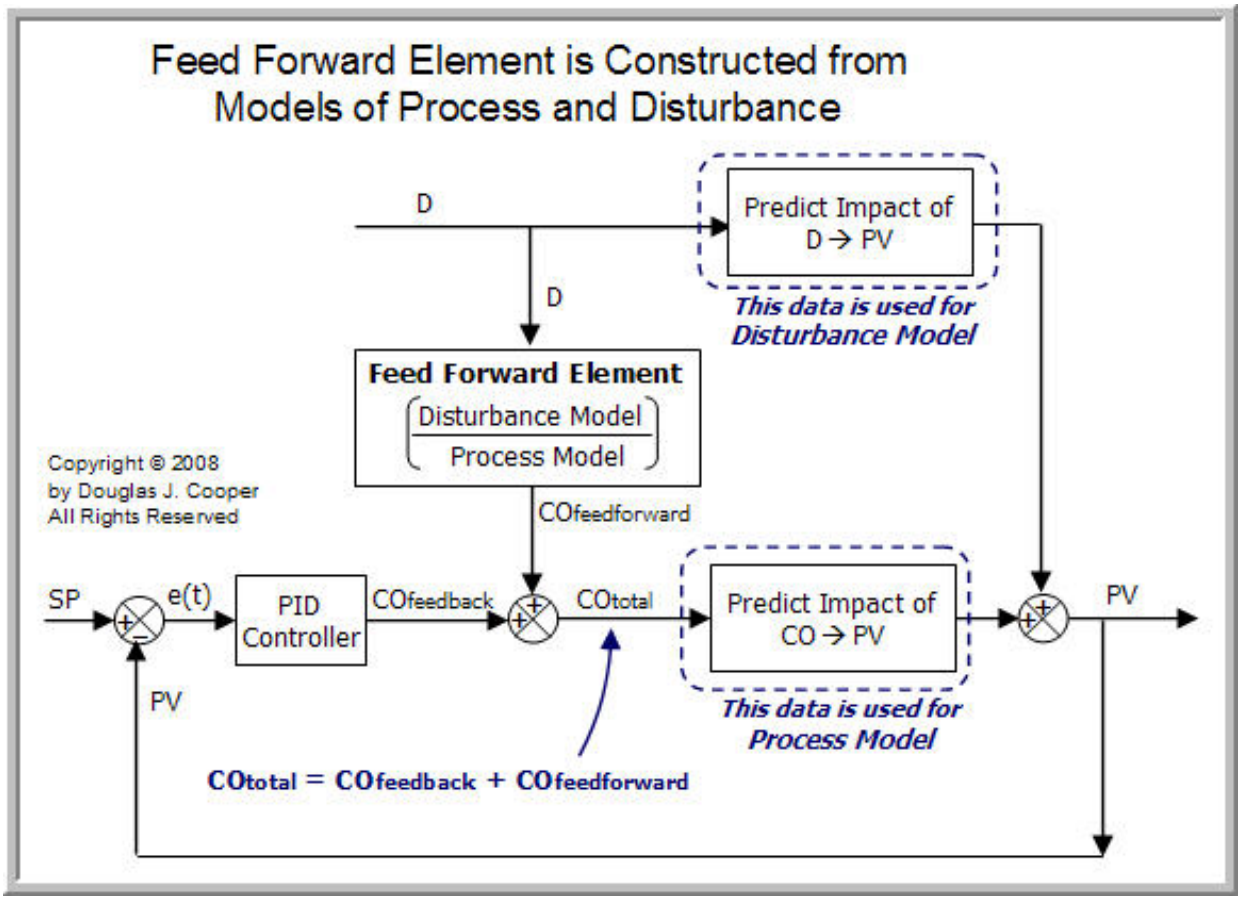


where:
CO = controller output signal
D = measured disturbance variable
e(t) = controller error, SP – PV
FCE = final control element (e.g., valve, variable speed pump or compressor)
PV = measured process variable
SP = set point

The feedback controller is designed and tuned like any stand-alone controller from the PID family of algorithms. The only difference is that it must allow for its controller output signal, COfeedback, to be combined with a feed forward controller output signal, COfeedforward, to arrive at a total control action, COtotal.

Feed Forward Element Uses a Process and Disturbance Model
The diagram below () shows that the function block element that computes the feed forward controller output signal, COfeedforward, is constructed by combining a process model and disturbance model.

The blocks circled with dotted lines below show where data is collected when developing the process and disturbance models. The feed forward controller ***does not*** include these circled blocks as separate elements in its architecture.



The process model (CO −› PV) in the feed forward element describes or predicts how each change in CO will impact PV. The disturbance model (D −› PV) describes or predicts how each change in D will impact PV.

In practice, these models can range from simple scaling multipliers (***static*** feed forward) through sophisticated differential equations (***dynamic*** feed forward). Sophisticated dynamic models can better describe actual process and disturbance behaviors, often resulting in improved disturbance rejection performance. Such models can also be challenging to derive and implement, increasing the time and expense of a project.

Dynamic Feed Forward Based on the FOPDT Model
We first develop a general feed forward element using dynamic models (differential

equations). Later, we will explore how we can simplify this general construction into a static feed forward element. Static feed forward is widely employed in industrial practice, in part because it can be implemented with an ordinary multiplying relay that scales the disturbance signal.

A dynamic feed forward element accounts for the "how far" gain, the "how fast" time constant and the "how much delay" dead time behavior of both the process (CO −› PV) and disturbance (D −› PV) relationships.

The simplest differential equation that describes such "how far, how fast, and with how much delay" behavior for either the process or disturbance dynamics is the familiar first order plus dead time (FOPDT) model.

● *The* CO −› PV *Process Model*
Describing the CO −› PV process behavior with a FOPDT model is not a new challenge. For example, we presented all details as we developed the FOPDT dynamic CO −› PV model for the gravity drained tanks process from step test data as:

$$1.4\frac{dPV(t)}{dt} + PV(t) = 0.09 \cdot CO(t - 0.5)$$

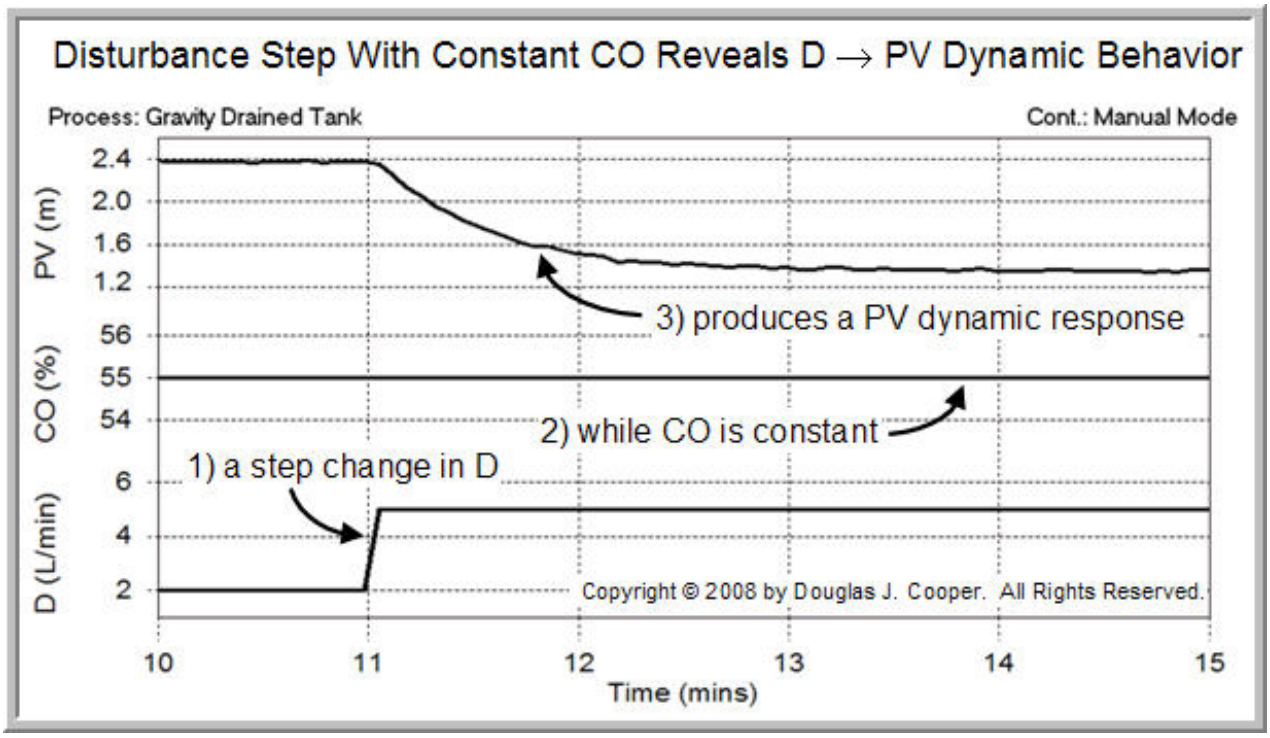
Which matches the general FOPDT (first order plus dead time) dynamic model form:

$$Tp\frac{dPV(t)}{dt} + PV(t) = Kp \cdot CO(t - \theta p)$$

where for a change in CO, the FOPDT model parameters are:
- Kp = process gain (the direction and how far PV will travel)
- *T*p = process time constant (how fast PV moves after it begins its response)
- Θp = process dead time (how much delay before PV first begins to respond)

This equation describes how each change in CO causes PV to respond (CO −› PV) as time, t, passes. Our past modeling experience also includes developing and documenting FOPDT CO −› PV models for the heat exchanger and jacketed reactor processes.

● *The* D −› PV *Disturbance Model*
The procedure used to develop the FOPDT (first order plus dead time) process models referenced above can be used in an identical fashion to develop a dynamic D −› PV disturbance model. While we do not show the graphical calculations at this point, consider the plot below from the gravity drained tanks process ().



Instead of analyzing a plot where a CO step forces a PV response while D remains constant, here we would analyze a D step forcing a PV response while CO remains constant.

We presume that an analogous graphical modeling procedure can be followed to determine the "how far, how fast, and with how much delay" dynamic D −› PV disturbance model:

$$T_D \frac{dPV(t)}{dt} + PV(t) = K_D \cdot D(t - \theta_D)$$

where for a step change in D:

- KD = disturbance gain (the direction and how far PV will travel)
- *T*D = disturbance time constant (how fast PV moves after it begins its response)
- ΘD = disturbance dead time (how much delay before PV first begins to respond)

Dynamic Feed Forward as a Thought Experiment

A feed forward element typically performs a model computation every sample time, T, to address any changes in measured disturbance, D. To help us visualize events, we discuss this as if it occurs as a two step "prediction and corrective action" procedure for a single disturbance:

1. The D −› PV disturbance model receives a change in the measured value of D and predicts an open-loop or uncontrolled “impact profile” on PV. This includes a prediction of how much delay will pass before the disruption first arrives at PV, the direction PV will travel for this particular D once it begins to respond, and how fast and how far PV will travel before it settles out at a predicted new steady state.
2. The CO −› PV process model then uses this PV impact profile to back-calculate a

series of corrective control actions, COfeedforward. These are CO moves sent to the final control element (FCE) to cause an "equal but opposite" response in PV such that it remains at set point, SP, throughout the event. Thus, the CO model seeks to exactly counteract the disruption profile predicted in step 1. The first CO actions are delayed as needed so they meet D upon arrival at PV. A series of CO actions are then deployed to counteract the predicted disruption over the life of the event.

Even sophisticated dynamic models are too simple to precisely describe the behavior of real processes. So although a feed forward element can dramatically reduce the impact of a disturbance on our PV, it will not provide a perfect "prediction and corrective action" disturbance rejection.

To account for model inaccuracies, the feed forward signal is combined with traditional feedback control action, COfeedback, to create a total controller output, COtotal. Whether it be a P-Only, PI, PID or PID w/ CO Filter algorithm, the feedback controller plays the important role of:

- minimizing the impact of disturbance variables other than D that can disrupt the PV,
- providing set point tracking capability to the overall strategy, and
- correcting for the simplifying approximations used in constructing the feed forward computation element that ultimately makes it imperfect in its actions.

The Sign of COfeedforward Requires Careful Attention
Notice in the block diagram above that COfeedforward is added as:

COtotal = COfeedback + COfeedforward

We write the equation this way because it is consistent with typical vendor documentation and standard piping/process & instrumentation diagrams (P&IDs).

The "plus" sign in the equation above requires our careful attention. To understand the caution, consider a case where the D −› PV disturbance model predicts that D will cause the PV to ***move up*** in a certain fashion or pattern over a period of time. According to our thought experiment above, the CO −› PV process model must compute feed forward CO actions that cause the PV to ***move down*** in an identical pattern.

But the sign of both the process gain, Kp, and disturbance gain, KD, together determine whether we need to send an increasing or decreasing signal to the FCE to compensate for a particular D. If Kp and KD are both positive or both negative, for example, then as a disturbance D moves up, the COfeedforward signal must move down to compensate. If Kp and KD are of opposite sign, then D and COfeedforward move in the same direction to counteract each other.

We show a standard "plus" sign in the equation above. But the computed feed forward signal, COfeedforward, will be positive or negative depending on the signs of the process and disturbance gains as just described. This ensures that the impact of the disturbance and the compensation from the feed forward element move in opposite directions to provide improved disturbance rejection performance.

Dynamic Feed Forward in Math
Suppose we define a generic process model, Gp, and generic disturbance model, GD, as:

Gp = generic CO −› PV process model (*describing how a CO change will impact PV*)
GD = generic D −› PV disturbance model (*describing how a D change will impact PV*)

We allow Gp and GD to have forms that can range from simple to the sophisticated.

For example, they can both be pure gain values Kp and KD and nothing more. They can be full FOPDT differential equations that include Kp, *T*p and Θp, and KD, *T*D and ΘD. One or the other (or both) can be non-self regulating (integrating), or perhaps self regulating but second or third order.

Leaving the exact form of the models undefined for now, we develop our feed forward element with the following steps:

1) Our generic CO −› PV process model, Gp, allows us to compute a PV response to changes in CO as:

PV = Gp·CO

With the generic model approach, we can rearrange the above to compute controller output actions that would reproduce a known or specified PV as:

CO = (1/Gp)·PV

2) Our generic D −› PV disturbance model, GD, lets us compute a PV response to changes in D as:

PV = GD·D

3) Following the logic in the above thought experiment, we use the D −› PV model of step 2 to predict an impact profile on PV for any measured disturbance D:

PVimpact = GD·D

4) We then use our rearranged equation of step 1 to back-calculate a series of corrective feed forward control signals that will move PV in a pattern that is opposite (and thus negative in sign) to the predicted PV impact profile from Step 3:

COfeedforward = − (1/Gp)·PVimpact

5) We finish by substituting the "PVimpact = GD·D" equation of step 3 into the COfeedforward equation of step 4:

COfeedforward = − (1/Gp)·(GD·D)

and rearrange to arrive at our final feed forward computational element composed of a disturbance model divided by a process model:

COfeedforward = − (GD/Gp)·D

Note: Above is a math argument that we hope seems reasonable and easy to follow.

But please be aware that for such manipulations to be proper, all variables and equations must first be mapped into the Laplace domain using Laplace transforms. The ease with which complex equations can be manipulated in the Laplace domain is a major reason control theorists use it for their derivations.

So, to be mathematically correct, we must first cast all variables and models into the Laplace domain as:

$PV(s) = Gp(s) \cdot CO(s)$ when the disturbance is constant

and

$PV(s) = GD(s) \cdot D(s)$ when the controller output signal is constant

where the Laplace domain models Gp(s) and GD(s) are called transfer functions.

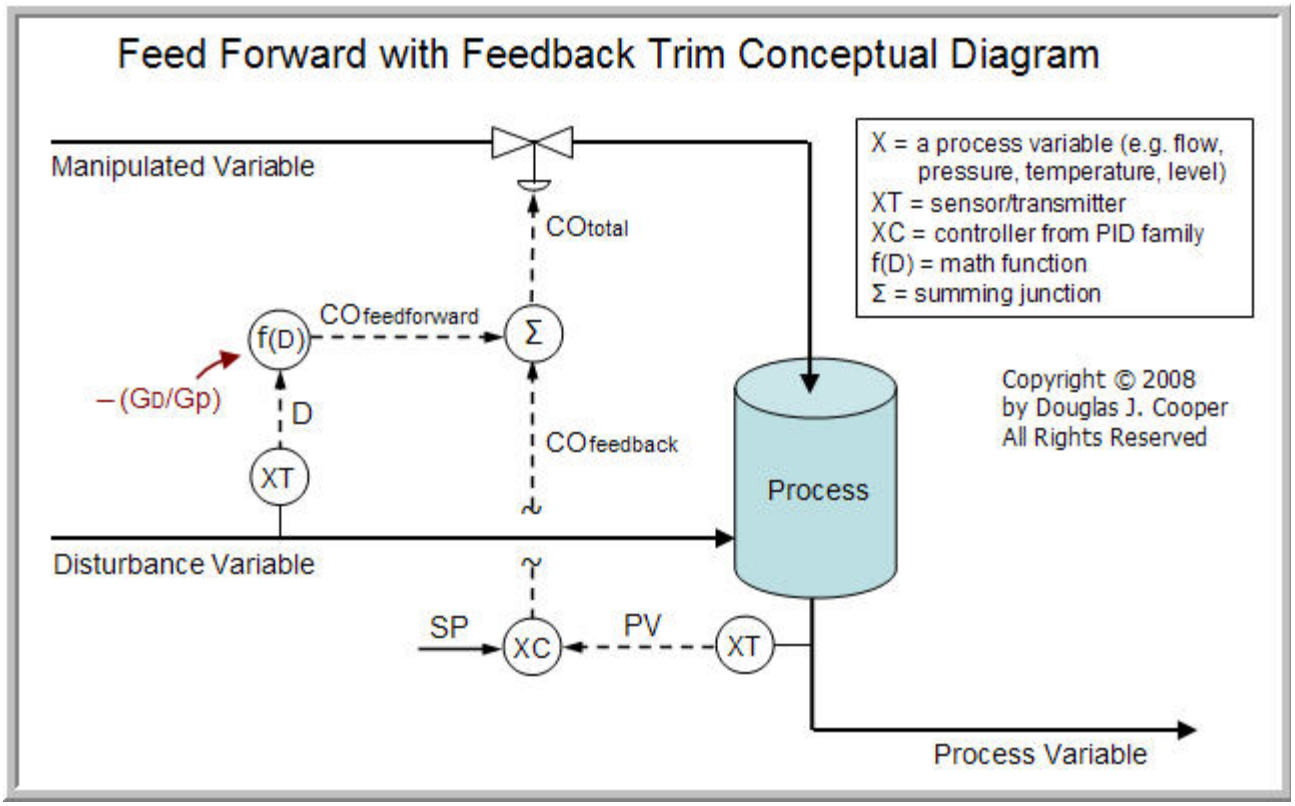
At the end of our derivation, our final feed forward computational element should be expressed as:

$COfeedforward(s) = -[GD(s)/Gp(s)] \cdot D(s)$

Thus, while we had omitted important details, our feed forward equation is indeed correct and we will use it going forward. We will continue to downplay the complexities of the math for now as we focus on methods of use to industry practitioners.

Conceptual Feed Forward with Feedback Trim Diagram

As promised in our introductory article on the feed forward architecture, we now have the basis for why we express the feed forward element math function, $f(D) = -(GD/Gp)$ as shown in our generalize feed forward with feedback trim conceptual diagram below ():



Implementation and Testing of Feed Forward with Feedback Trim
We next explore the widely used and surprisingly powerful static feed forward controller. We will discover the ease with which we can develop such an architecture, and also explore some of the limitations of this simplified approach.

Static Feed Forward and Disturbance Rejection in the Jacketed Reactor

By Doug Cooper and Allen Houtz[1]

As discussed in previous articles, the purpose of the feed forward controller of the feed forward with feedback trim architecture is to reduce the impact of one specific disturbance, D, on our primary process variable, PV. An additional sensor must be located upstream in our process so we have a disturbance measurement that provides warning of impending disruption. The feed forward element uses this D measurement signal to compute and implement corrective control actions so the disturbance has minimal impact on stable operation.

Here we build on the mathematical foundation of this previous material as we explore the popular and surprisingly powerful **static** feed forward computational element for this disturbance rejection architecture.

Static Feed Forward Uses the Simplest Model Form
If we define a generic process model, Gp, and generic disturbance model, GD, as:

Gp = generic CO −› PV process model (*describing how a CO change will impact PV*)
GD = generic D −› PV disturbance model (*describing how a D change will impact PV*)

then we can show details to derive a general feed forward computational element as a ratio of the disturbance model divided by the process model:

COfeedforward = − (GD/Gp)·D

Models Gp and GD can range from the simple to the sophisticated. With static feed forward, we limit Gp and GD to their constant "which direction and how far" gain values:

Gp = Kp (the CO −› PV process gain)
GD = KD (the D −› PV disturbance gain)

And the static feed forward element is thus a simple gain ratio multiplier:

COfeedforward = − (KD/Kp)·D (*static feed forward element*)

The static feed forward controller does not consider how the controller output to process variable (CO −› PV) dynamic behavior differs from the disturbance to process variable (D −› PV) dynamic behavior.

- We do not account for the size of the process time constant, *T*p, relative to the disturbance time constant, *T*D. As a consequence, we cannot compute and deploy a series of corrective control actions over time to match how fast the disturbance event is causing the PV to move up or down.
- We do not consider the size of the process dead time, Θp, relative to the disturbance dead time, ΘD. Thus, we cannot delay the implementation of corrective actions to coordinate their arrival with the start of the disturbance disruption on PV.

For processes where the CO −› PV dynamic behavior is very similar to the D −› PV dynamic behavior, like many liquid level processes for example, static feed forward will perform virtually the same as a fully dynamic feed forward controller in rejecting our measured disturbance.

Visualizing the action of this static feed forward element as a two step "prediction and corrective action" procedure for a single disturbance:

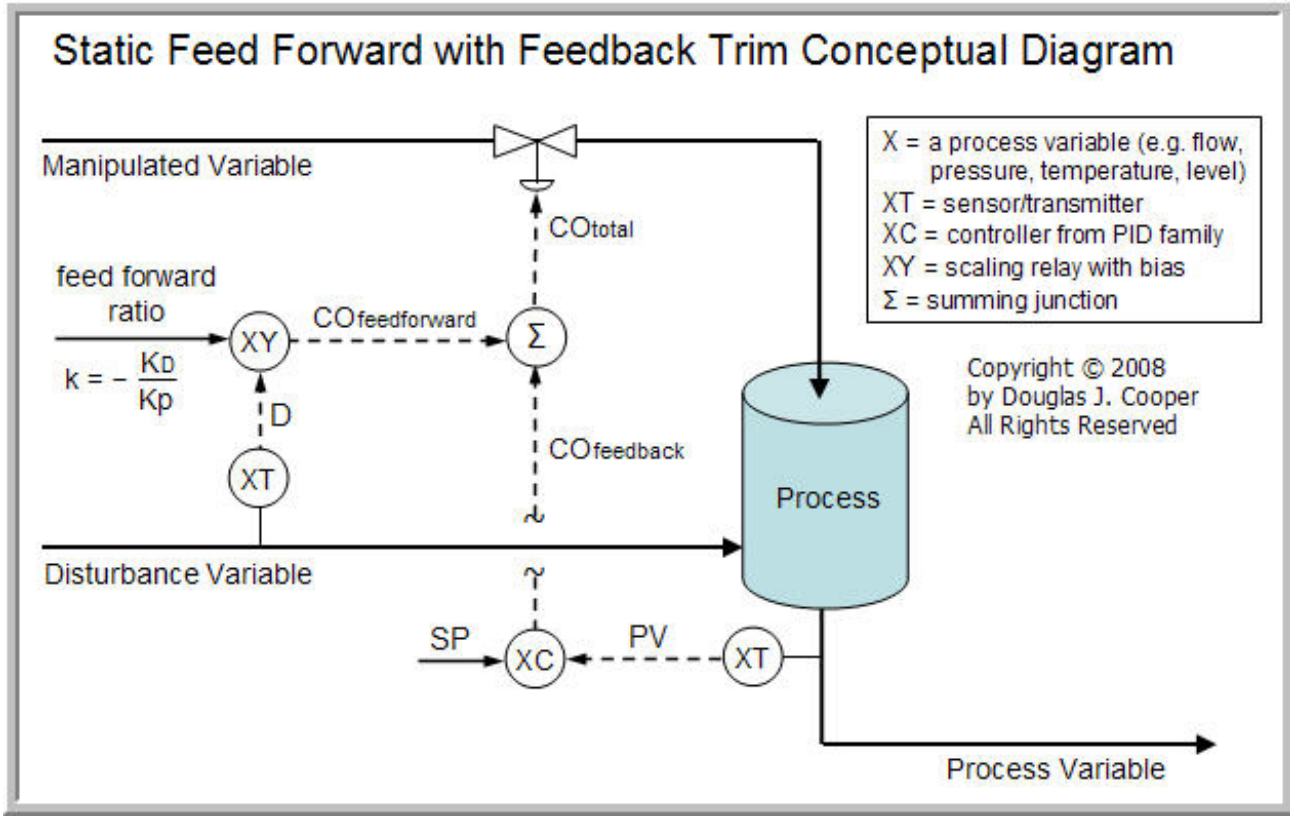
1. The D −› PV disturbance gain, KD, receives a change in D and predicts the total final impact on PV. The computation can only account for information contained in KD, which includes the direction and how far PV will ultimately travel in response to the measured D before it settles out at a new steady state.
2. The CO −› PV process gain, Kp, then uses this disturbance impact prediction of "which direction and how far" to back-calculate one CO move as a corrective control action, COfeedforward. This COfeedforward move is sent immediately to the final control element (FCE) to cause an "equal but opposite" response in PV.

Limited in Capability but (Reasonably) Easy to Implement
The static feed forward element makes one complete and final corrective action for every measured change in D. It does not delay the feed forward signal so it will meet the D

impact when it arrives at PV. It does not compute and deploy a series of CO actions to try and counteract a predicted disruption pattern over an event life.

Even with this limited performance capability, the benefit of the static form that makes it popular with industry practitioners is that it is reasonably straightforward to implement in a production environment. As shown below (), we can construct a static feed forward element with:

- a sensor/transmitter to measure disturbance D
- a scaling relay that multiplies signal D by our static feed forward ratio, (− KD/Kp)
- a summing junction that adds COfeedforward to COfeedback to produce COtotal



COfeedforward is Normally Zero

An important implementation issue is that COfeedforward should equal zero when D is at its design level of operation (DLO) value. Thus, the D used in our calculations is actually the disturbance signal from the sensor/transmitter (Dmeasured) that has been shifted or biased by the design level of operation disturbance value (DDLO), or:

D = Dmeasured − DDLO

With this definition, both D and COfeedforward will be zero when the disturbance is at its normal or expected value. Such a biasing capability is included with most all commercial scaling relay function blocks.

Static Feed Forward and the Jacketed Reactor Process

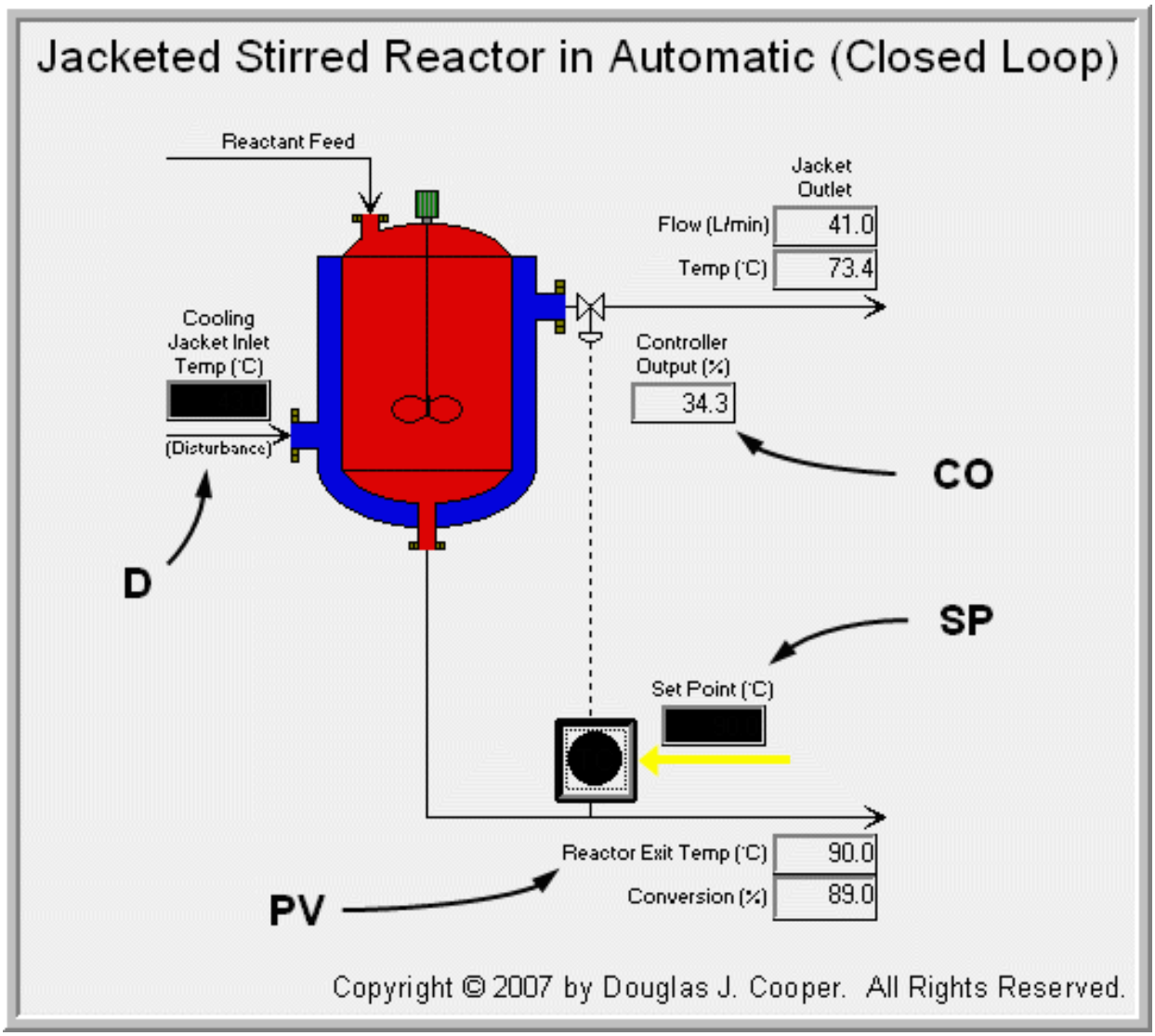
We have previously explored the modes of operation and dynamic CO −› PV behavior of the jacketed stirred reactor process. We also have established the performance of a single

loop PI controller, a PID with CO Filter controller and a cascade control implementation when our control objective is to minimize the impact of a disturbance caused by a change in the temperature of the liquid entering the cooling jacket.

Here we explore the design, implementation and performance of a static feed forward with feedback trim architecture for this same disturbance rejection objective.

Limitations of the Single Loop Architecture
The control objective is to maintain the reactor exit stream temperature (PV) at set point (SP) in spite of changes in the temperature of cooling liquid entering the jacket (D) by adjusting controller output (CO) signals to the cooling liquid flow valve.

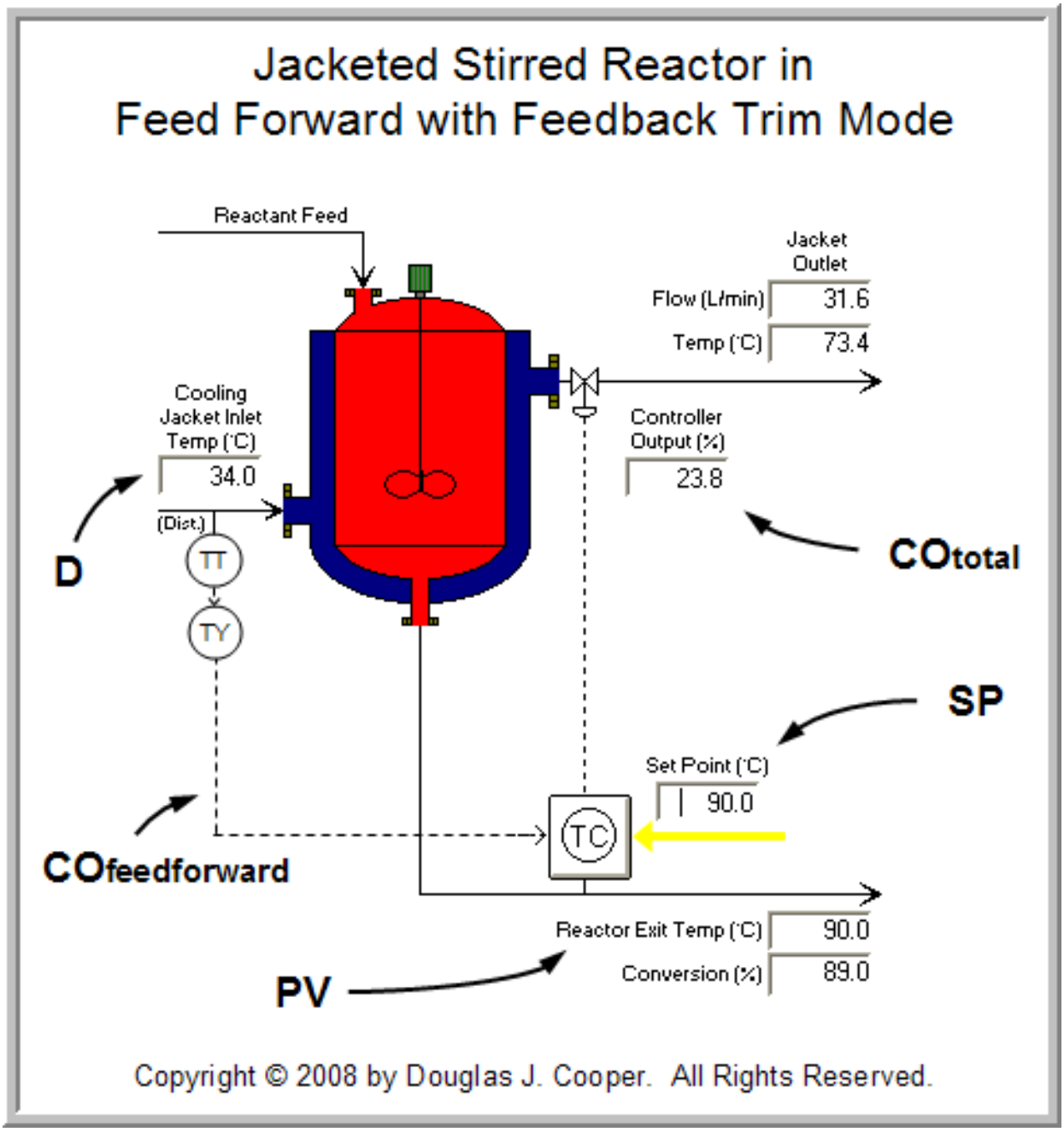
The nature of this process and the performance limitations of a single loop architecture as shown below have been detailed in a previous article.



where:
CO = signal to valve that adjusts cooling jacket liquid flow rate (controller output, %)
PV = reactor exit stream temperature (measured process variable, ^{o}C)
SP = desired reactor exit stream temperature (set point, ^{o}C)

D = temperature of cooling liquid entering the jacket (major disturbance, ^{o}C)

A Feed Forward with Feedback Trim Reactor Architecture
Below is the jacketed stirred reactor process with a feed forward with feedback trim controller architecture ().



The loop architecture from this commercial software simulation shows that D is measured, scaled and transmitted as COfeedforward to the controller. There it is combined with the traditional feedback signal to produce the COtotal sent to the cooling jacket flow valve. This is a simplified representation of the same feed forward with feedback trim conceptual diagram shown earlier in this article.

Design Level of Operation (DLO)

The details and discussion of the DLO used in our previous disturbance rejection studies for the jacketed stirred reactor are presented in a previous article and are summarized:

- Design PV and SP = 90 ^{o}C with approval for brief dynamic testing of ±2 ^{o}C
- Design D = 43 ^{o}C with occasional spikes up to 50 ^{o}C

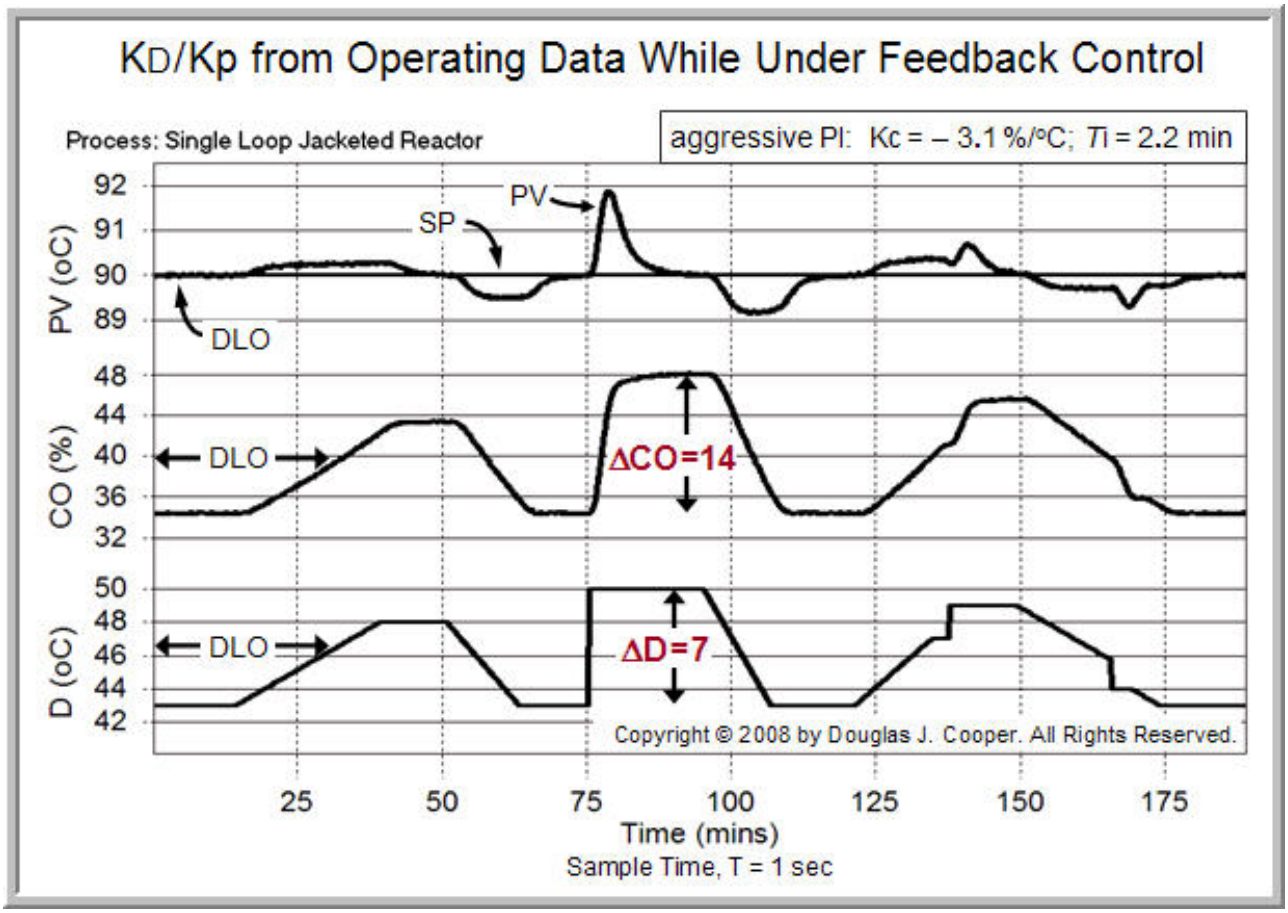
We note that D moves between 43 ^{o}C and 50 ^{o}C. We seek a single DLO value that lets us conveniently compare results from two different design methods explored below.

We choose here a DDLO as the average value of (43+50)/2 = 46.5 ^{o}C and acknowledge that other choices (such as simply using 43 ^{o}C) are reasonable. As long as we are consistent in our methodology, the conclusions we draw when comparing the two methods will remain unchanged.

When D = 46.5 ^{o}C and CO = 40%, then our measured process variable settles at the design value of PV = 90 ^{o}C. This relationship between the three variables explains the DLO values indicated on the plots that follow.

Design Method 1: Compute KD/Kp From Historic Data

Below () is a trend from our data historian showing approximately three hours of operation from the jacketed reactor process under PI control. No feed forward controller is active. The set point (SP) is constant and a number of disturbance events force the PV from SP. All variables are near our DLO as described above.



If we recall the definition that Kp = ΔPV/ΔCO and KD = ΔPV/ΔD, then for our static feed

forward design:

$$\begin{aligned} CO_{feedforward} &= -(K_D/K_p)\cdot D \\ &= -[(\Delta PV/\Delta D)/(\Delta PV/\Delta CO)]\cdot D \\ &= -[(\Delta CO/\Delta D)]\cdot D \end{aligned}$$

With this last equation, our design challenge is reduced to finding a disturbance change that lasts long enough for the controller output response to settle. If this event occurs reasonably close to our DLO, then we can directly compute our gain ratio feed forward element by measuring the disturbance and controller output changes from a plot.

▪ *About the Feedback Controller*
The plot shows disturbance rejection performance when the process is using a PI controller tuned for an aggressive response action. The details of the design, tuning and testing of this process and controller combination are presented in a previous article.

Note that with this "measure from a plot" approach of Method 1, the process can be controlled by any algorithm from the PID family, though integral action must be included to return the PV to SP (eliminate offset) after a disturbance. Also, our feedback controller tuning can range from conservative (sluggish) through an aggressive (active) response without affecting the applicability of the method.

Our interest is limited to finding a ΔD disturbance event with a corresponding ΔCO controller output signal response that lasts long enough for the PV to be returned to SP. Then as shown above, the ΔD and ΔCO relationship can be measured directly from the plot data.

▪ *Accounting for Negative Feedback*
If we are using automatic mode (closed loop) data as shown in the plot above, we must account for the negative feedback of our controller in our calculations. A controller always takes action that moves the CO signal in a direction that counteracts the developing controller error. Thus, when using automatic mode (closed loop) data as above, we must consider that a negative sign has been introduced into the signal relationship.

On the plot above, we have labeled a ΔD disturbance change with corresponding ΔCO controller output signal response. We introduce the sign change from negative feedback of our controller and compute:

$$\begin{aligned} CO_{feedforward} &= -[(\Delta CO/\Delta D)]\cdot D\cdot(-1 \text{ for negative feedback}) \\ &= [(14\%)/(7\ ^{o}C)]\cdot D \\ &= [2\ \%/^{o}C]\cdot D \end{aligned}$$

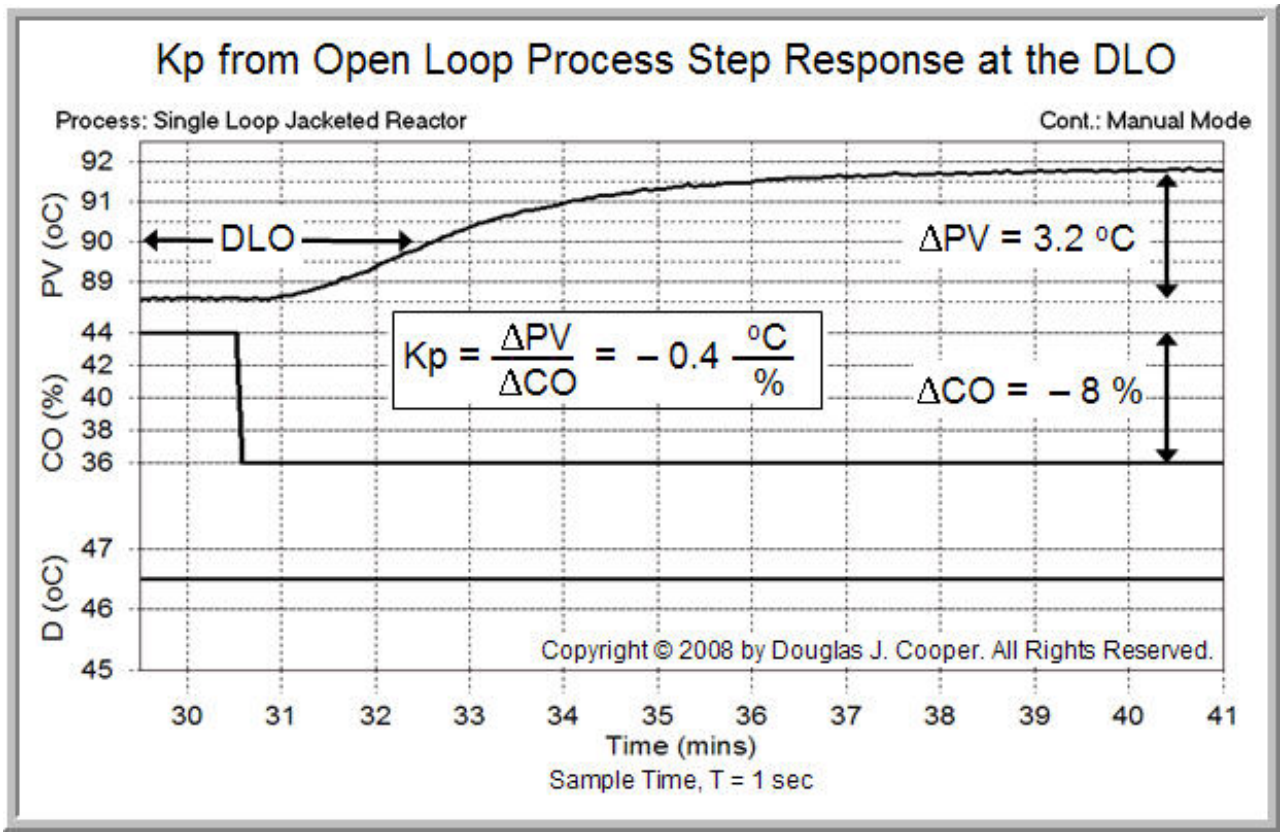
Design Method 2: Perform Two Independent Bump Tests
To validate that a feed forward gain ratio of 2 %/^{o}C is a reasonable number as determined from automatic mode (closed loop) data, here we perform two independent step tests, compute individual values for KD and Kp, and then compare the ratio results to Method 1.

This approach is largely academic because the challenges of steadying a real process

and then stepping individual parameters in such a perfect fashion is unrealistic in the chaotic world of most production environments. This exercise holds value, however, because it provides an alternate route that confirms the results presented in Method 1.

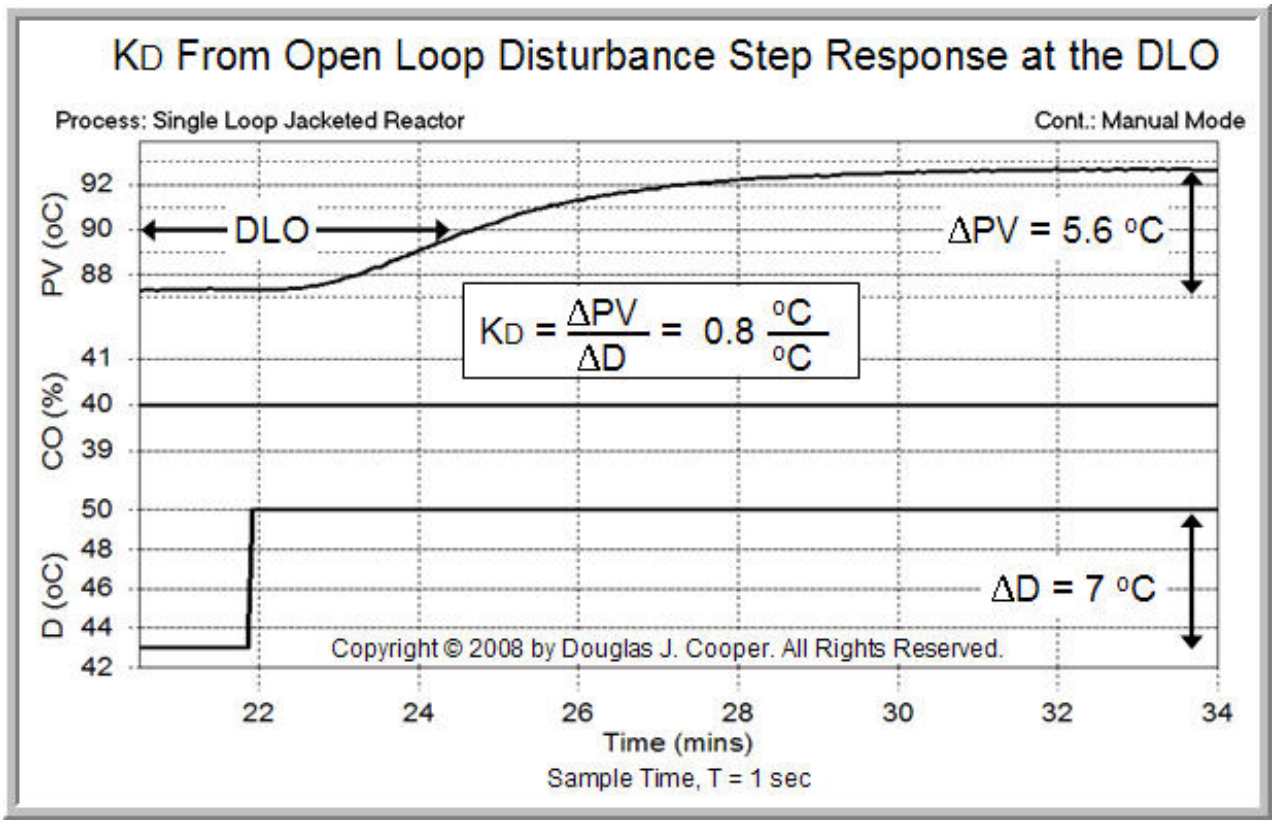
We follow the established procedure for computing a process gain, Kp, from a manual mode (open loop) step test response plot. That is, we set our disturbance parameter at DDLO, set the controller output at a constant CO value and wait until the PV is steady. We then step CO to force a PV response that is centered around our DLO.

Such a step response plot is shown below () for the jacketed stirred reactor. We measure and compute Kp = ΔPV/ΔCO = − 0.4 ^{o}C/% as indicated on the plot.



We repeat the procedure to compute a disturbance gain, KD, from an open loop step response plot. Here, we set our CO signal at the DLO value of 40%, set the disturbance parameter at a constant D value and wait until the PV is steady. We then step D to force a PV response that is again centered around our DLO.

Such a disturbance step response plot is shown below (). We measure and compute KD = ΔPV/ΔD = 0.8 ^{o}C/^{o}C as labeled on the plot.



With values for Kp and KD, we compute our gain ratio feed forward multiplier. Since we are in manual mode (open loop), we need not account for any sign change due to negative feedback in our calculation.

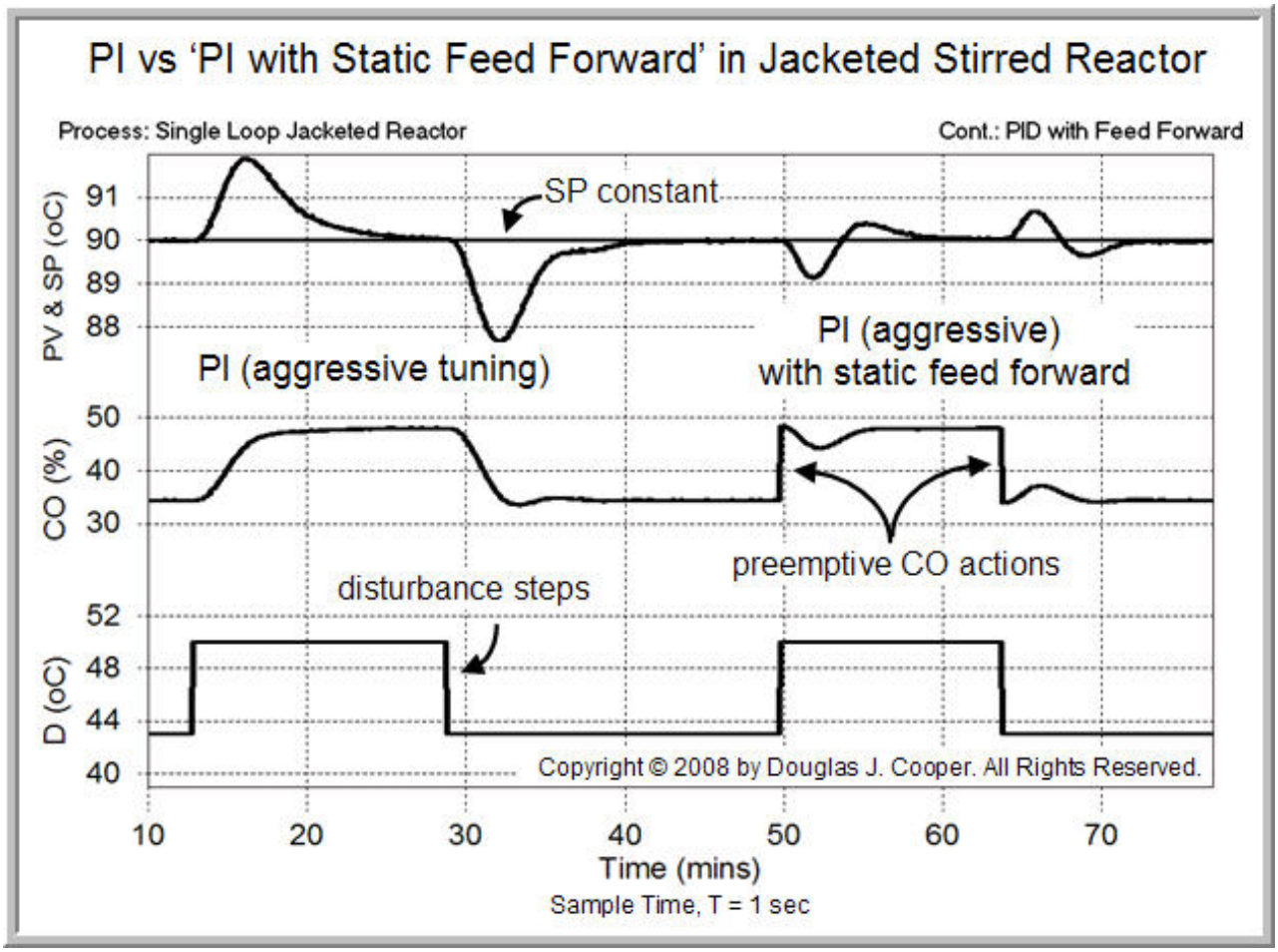
$$\begin{aligned} \text{COfeedforward} &= -(KD/Kp)\cdot D \\ &= -[(0.8\ ^{o}C/^{o}C)/(-0.4\ ^{o}C/\%)]\cdot D \\ &= [2\ \%/^{o}C]\cdot D \end{aligned}$$

Thus, with careful testing using a consistent and repeatable commercial process simulation, we observe that the practical approach of Method 1 provides the same results as the academic approach of Method 2.

Implement and Test

The all-important question we now consider is whether the extra effort associated with designing and implementing a "static feed forward with feedback trim" architecture provides sufficient payoff in the form of improved disturbance rejection performance.

To the left in the plot below () is the performance of a dependent, ideal PI controller with aggressive tuning values for controller gain Kc = − 3.1 %/^{o}C and reset time *T*i = 2.2 min as detailed in this article.



To the right in the plot above is the disturbance rejection performance of our static feed forward with feedback trim architecture. The feed forward gain ratio multiplier used is the 2 %/^{o}C as determined by two different methods described earlier in this article. The feedback controller remains the aggressively tuned PI algorithm as described above.

The static feed forward controller makes one complete preemptive corrective CO action whenever a change in D is detected as noted in the plot above. There is no delay of the feed forward signal based on relative dead time considerations, and there is no series of CO actions computed and deployed based on relative time constant considerations.

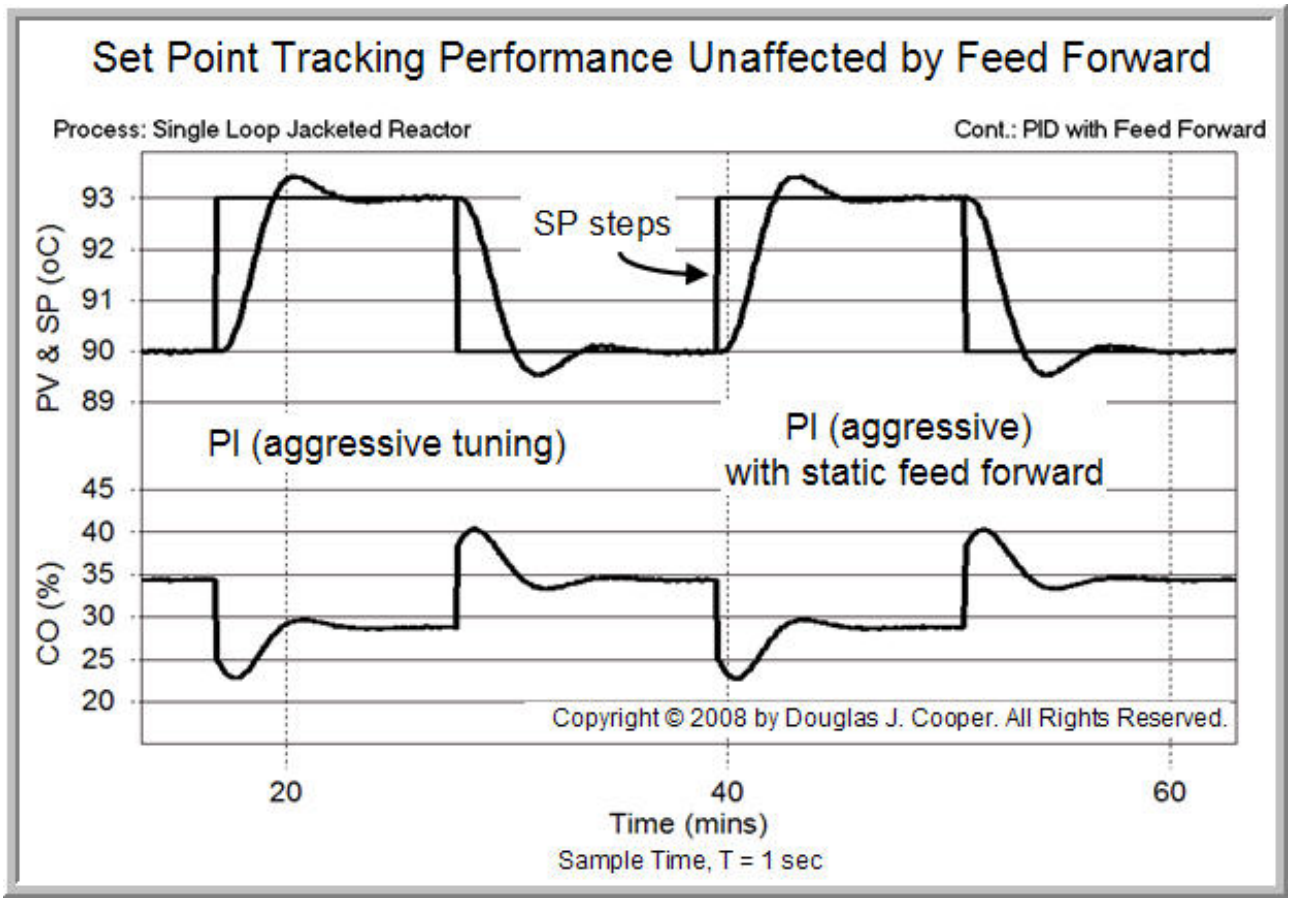
Nevertheless, the static feed forward controller is able to reduce the maximum deviation from SP during a disturbance event to half of its original value. The settling time is also reduced, though less dramatically. Like any control project, the operations staff must determine if this represents a sufficient payback for the effort and expenses required.

> Practitioner's note: Our decision to add feed forward to a feedback control loop is driven by the character of the disturbance and its effect on our PV. If the controller can react more quickly than the D can change, feed forward is not likely to significantly improve control. However, if the disturbance changes rapidly and fairly often, feed forward control can be a powerful tool to stabilize our process.

No Impact on Set Point Tracking Performance

While not our design objective, presented below is the set point tracking performance of the single loop PI controller compared to that of the static feed forward with feedback trim

architecture (). The same aggressive PI tuning values used above are maintained for this study.



Feed forward with feedback trim is designed for the improved rejection of one measured disturbance. As shown above, a feed forward controller has no impact on set point tracking performance when the disturbance is constant. This makes sense since the computed COfeedforward signal does not change unless D changes. Indeed, with no change in the measured disturbance, both architectures provide identical performance.

17) Ratio, Override and Cross-Limiting Control

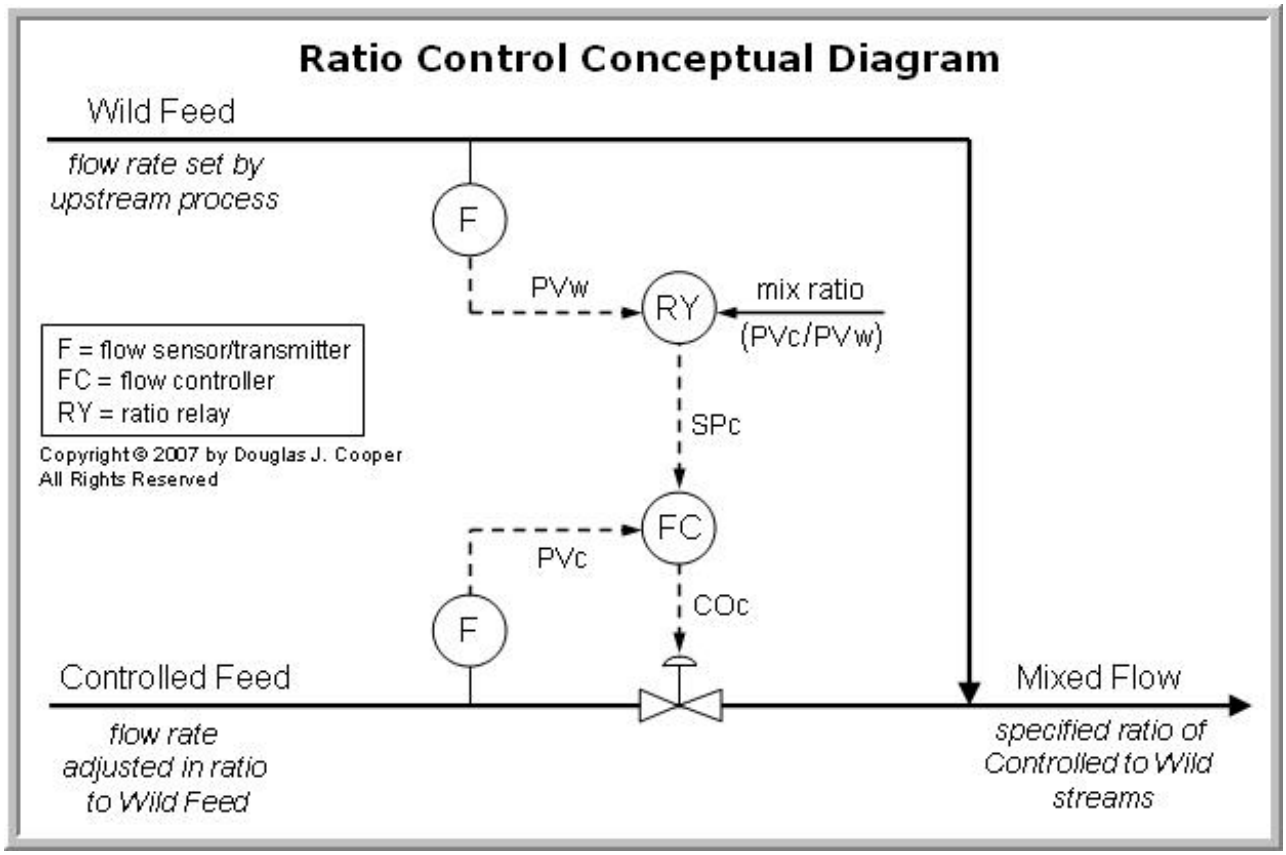
The Ratio Control Architecture

By Allen Houtz[1] and Doug Cooper

The ratio control architecture is used to maintain the flow rate of one stream in a process at a defined or specified proportion relative to that of another. A common application for ratio control is to combine or blend two feed streams to produce a mixed flow with a desired composition or physical property. Consistent with other articles in this e-book, applications of interest are processes with streams comprised of gases, liquids, powders, slurries or melts.

The conceptual diagram below () shows that the flow rate of one of the streams feeding the mixed flow, designated as the ***wild feed***, can change freely. Its flow rate might change based on product demand, maintenance limitations, feedstock variations, energy availability, the actions of another controller in the plant, or it may simply be that this is the stream we are least willing to manipulate during normal operation.

The other stream shown feeding the mixed flow is designated as the ***controlled feed***. A final control element (FCE) in the controlled feed stream receives and reacts to the controller output signal, COc, from the ratio control architecture. While the conceptual diagrams in this article show a valve as the FCE, we note that other flow manipulation devices such as variable speed pumps or compressors may also be used in ratio control implementations.



Relays in the Ratio Architecture

As the above diagram illustrates, we measure the flow rate of the wild feed and pass the signal to a relay, designated as RY in the diagram. The relay is typically one of two types:

- A ***ratio relay***, where the mix ratio is entered once during configuration and is generally not available to operations staff during normal operation.
- A ***multiplying relay*** (shown), where the mix ratio is presented as an adjustable parameter on the operations display and is thus more readily accessible for change.

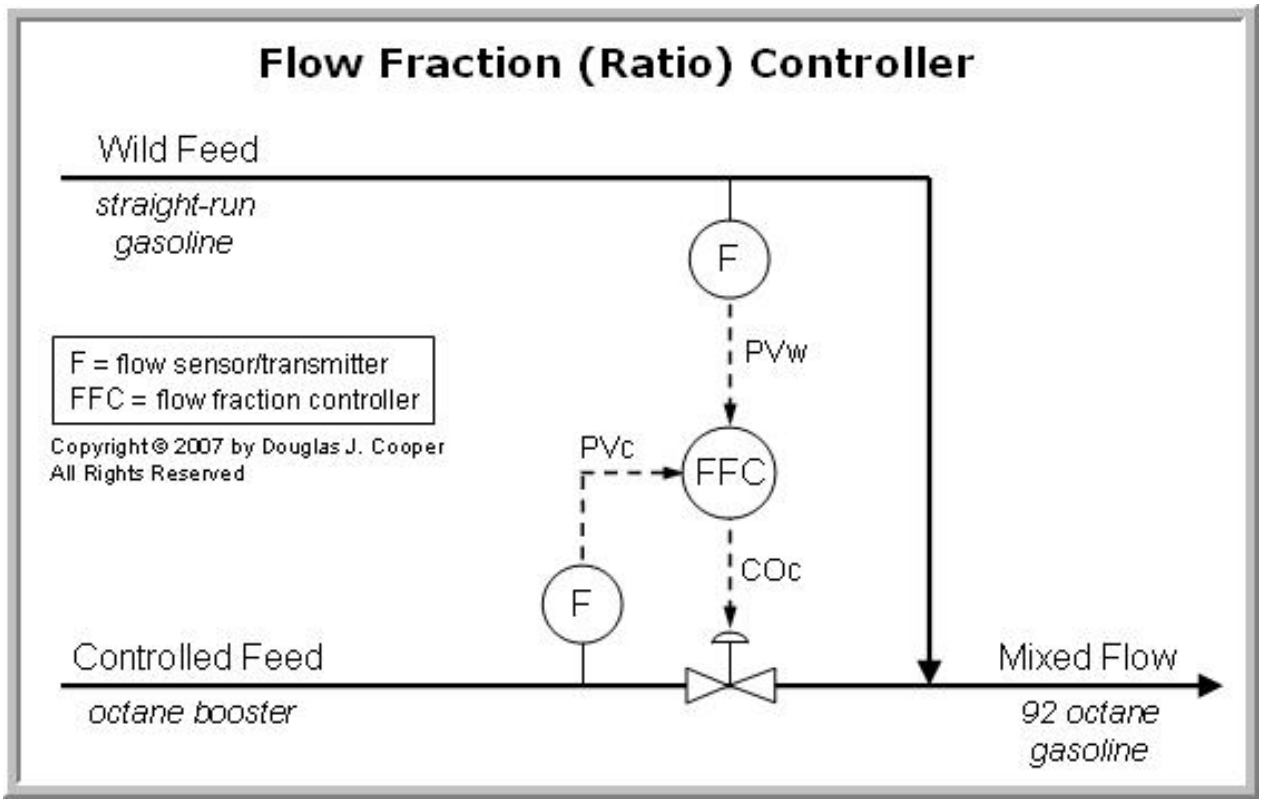
In either case, the relay multiplies the measured flow rate of the wild feed stream, PVw, by the entered mix ratio to arrive at a desired or set point value, SPc, for the controlled feed stream. A flow controller then regulates the controlled feed flow rate to this SPc, resulting in a mixed flow stream of specified proportions between the controlled and wild streams.

Linear Flow Signals Required

A ratio controller architecture as described above requires that the signal from each flow sensor/transmitter change linearly with flow rate. Thus, the signals from the wild stream process variable, PVw, and the controlled stream process variable, PVc, should increase and decrease in a straight-line fashion as the individual flow rates increase and decrease.

Turbine flow meters and certain other sensors can provide a signal that changes linearly with flow rate. Unfortunately, a host of popular flow sensors, including inferential head flow elements such as orifice meters, do not. Additional computations (function blocks) must then be included between the sensor and the ratio relay to transform the nonlinear signal into the required linear flow-to-signal relationship.

Flow Fraction (Ratio) Controller
A classic example of ratio control is the blending of an additive into a process stream. As shown below (), an octane booster is blended with straight-run gasoline stream being produced by an atmospheric distillation column. For any number of reasons, the production rate of straight-run gasoline will vary over time in a refinery. Therefore, the amount of octane booster required to produce the desired octane rating in the mixed product flow must also vary in a coordinated fashion.



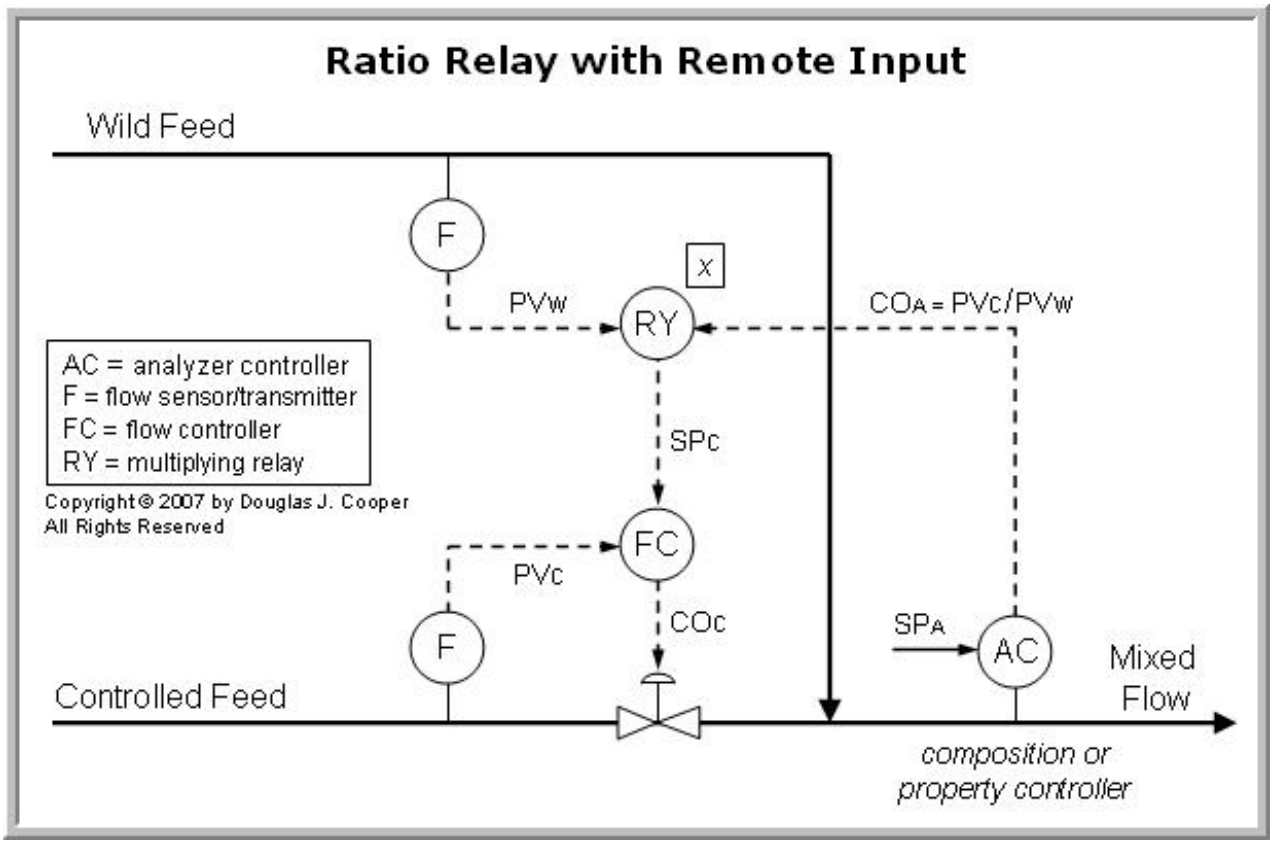
Rather than using a relay, we present an alternative ratio control architecture based on a flow fraction controller (FFC). The FFC is essentially a "pure" ratio controller in that it receives the wild feed and controlled feed signals directly as inputs. A ratio set point value is entered into the FCC, along with tuning parameters and other values required for any controller implementation.

Ratio Relay or Flow Fraction Controller
The flow fraction (ratio) controller is a preconfigured option in many modern computer based DCS or advanced PLC control systems. It provides exactly the same functionality as the ratio relay combined with a single-input single-output controller as discussed above.

The choice of using a relay or an FFC is a practical matter. The entered ratio multiplier value in a relay is not a readily accessible parameter. It therefore requires a greater level of permission and access to adjust. Consequently, the use of the ratio relay has the advantage (or disadvantage depending on the application) of requiring a higher level of authorization before a change can be made to the ratio multiplier.

Multiplying Relay With Remote Input
The ratio controller shown below () presents an additional level of complexity in that, like the cascade architecture, our ratio controller is contained within and is thus part of a larger control strategy.

In the example below, an analyzer sensor measures the composition or property we seek to maintain in the mixed flow stream. The measured value is compared to a set point value, SPA, and a mix ratio controller output signal, COA, is generated based on the difference. Thus, like a cascade, the outer analyzer controller continually sends mix ratio updates to the inner ratio control architecture.



The updated mix ratio COA value enters the multiplying relay as an external set point. The objective of this additional complexity is to correct for any unmeasured changes in the wild feed or controlled feed, thus maintaining the mixed flow composition or property at the set point value, SPA.

The term "analyzer" is used broadly here. Hopefully, we can indentify a fast, inexpensive and reliable sensor that allows us to infer the mixed flow composition or property of interest. Examples might include a capacitance probe, an in-line viscometer, or a pH meter.

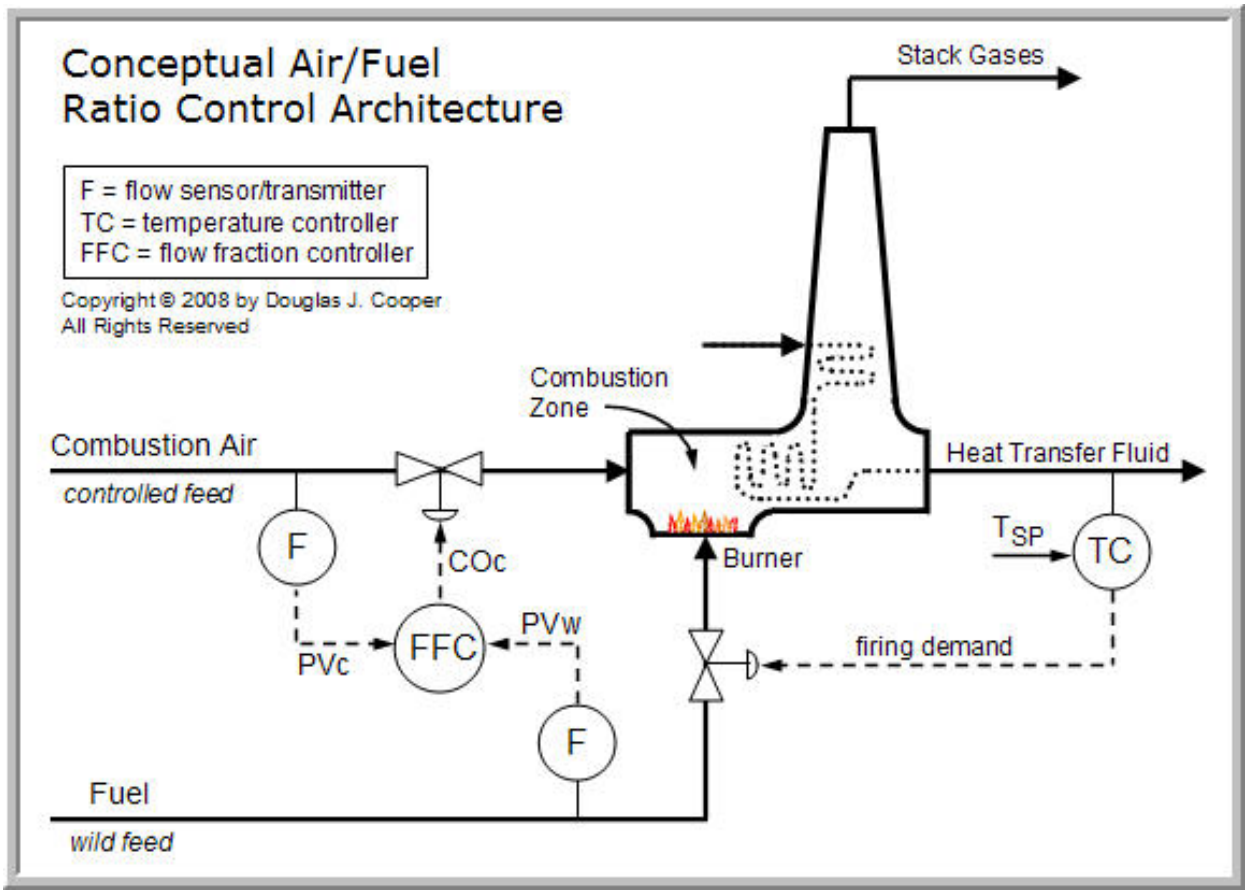
If we are required to use a chromatograph, spectrometer or other such instrument, we must allow for the increased maintenance and attention such devices often demand. Perhaps more important, the time to complete a sample and analysis cycle for these devices can introduce a long dead time into our feedback loop. As dead time increases, best attainable control performance decreases.

Ratio Control and Metered-Air Combustion Processes

By Allen Houtz[1] and Doug Cooper

A ratio control strategy can play a fundamental role in the safe and profitable operation of fired heaters, boilers, furnaces and similar fuel burning processes. This is because the air-to-fuel ratio in the combustion zone of these processes directly impacts fuel combustion efficiency and environmental emissions.

A requirement for ratio control implementation is that both the fuel feed rate and combustion air feed rate are measured and available as process variable (PV) signals. Shown below () is a conceptual air/fuel ratio control strategy.



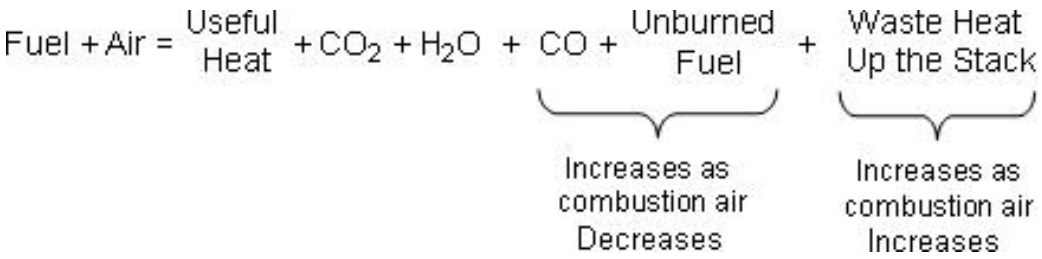
In this representative architecture, the fuel flow rate is adjusted to maintain the temperature of a heat transfer fluid exiting a furnace. On other processes, fuel flow rate might be adjusted to maintain the pressure in a steam header, the duct temperature downstream of the burner, or similar variable that must be regulated for efficient operation.

The combustion air feed rate is then adjusted by a flow fraction (ratio) controller to maintain a desired air/fuel ratio. While a simple sensor and valve is shown above, we will expand and modify this conceptual architecture as we progress in this discussion because:

▪ The final control element (FCE) for the combustion air stream, rather than being a valve, is more commonly a variable speed blower, perhaps with adjustable dampers or louvers.
▪ Measuring combustion air flow rate is challenging and can involve measuring a pressure drop across a portion of the combustion gas exhaust flow path.
▪ In different applications, the air flow rate can be the wild feed while fuel flow rate is the controlled feed.
▪ Stack gas analyzers add value and sophistication as they monitor the chemistry associated with combustion efficiency and environmental emissions.

Why Air/Fuel Ratio is Important
In combustion processes, air/fuel ratio is normally expressed on a mass basis. We get maximum useful heat energy if we provide air to the combustion zone at a mass flow rate (e.g., lb/min, Kg/hr) that is properly matched to the mass flow rate of fuel to the burner.

Consider this generic equation for fuel combustion chemistry:



Where:
CO_2 = carbon dioxide
CO = carbon monoxide
H_2O = water
Air = 21% oxygen (O_2) and 79% nitrogen (N_2)
Fuel = hydrocarbon such as natural gas or liquid fuel oil

Air is largely composed of oxygen and nitrogen. It is the oxygen in the air that combines with the carbon in the fuel in a highly energetic reaction called combustion. When burning hydrocarbons, nature strongly prefers the carbon-oxygen double bonds of carbon dioxide and will yield significant heat energy in an exothermic reaction to achieve this CO_2 form.

Thus, carbon dioxide is the common green house gas produced from the complete combustion of hydrocarbon fuel. Water vapor (H_2O) is also a normal product of hydrocarbon combustion.

Aside: nitrogen oxide (NOx) and sulfur oxide (SOx) pollutants are not included in our combustion chemistry equation. They are produced in industrial combustion processes principally from the nitrogen and sulfur originating in the fuel. As the temperature in the combustion zone increases, a portion of the nitrogen in the air can also convert to NOx. NOx and SOx combustion chemistry is beyond the scope of this article but a detailed discussion can be found here.

Too Little Air Increases Pollution and Wastes Fuel
The oxygen needed to burn fuel comes from the air we feed to the process. If the air/fuel ratio is too small in our heater, boiler or furnace, there will not be enough oxygen available to completely convert the hydrocarbon fuel to carbon dioxide and water.

A too-small air/fuel ratio leads to incomplete combustion of our fuel. As the availability of oxygen decreases, noxious exhaust gases including carbon monoxide will form first. As the air/fuel ratio decreases further, partially burned and unburned fuel can appear in the exhaust stack, often revealing itself as smoke and soot. Carbon monoxide, partially burned and unburned fuel are all poisons whose release is regulated by the government (the Environmental Protection Agency in the USA).

Incomplete combustion also means that we are wasting expensive fuel. Fuel that does not burn to provide useful heat energy, including carbon monoxide that could yield energy as it converts to carbon dioxide, literally flows up our exhaust stack as lost profit.

Too Much Air Wastes Fuel

The issue that makes the operation of a combustion process so interesting is that if we feed too much air to the combustion zone (if the air/fuel ratio is too high), we also waste fuel, though in a wholly different manner.

Once we have enough oxygen available in the burn zone to complete combustion of the hydrocarbon fuel to carbon dioxide and water, we have addressed the pollution portion of our combustion chemistry equation. Any air fed to the process above and beyond that amount becomes an additional process load to be heated.

As the air/fuel ratio increases above that needed for complete combustion, the extra nitrogen and unneeded oxygen absorb heat energy, decreasing the temperature of the flame and gases in the combustion zone. As the operating temperature drops, we are less able to extract useful heat energy for our intended application.

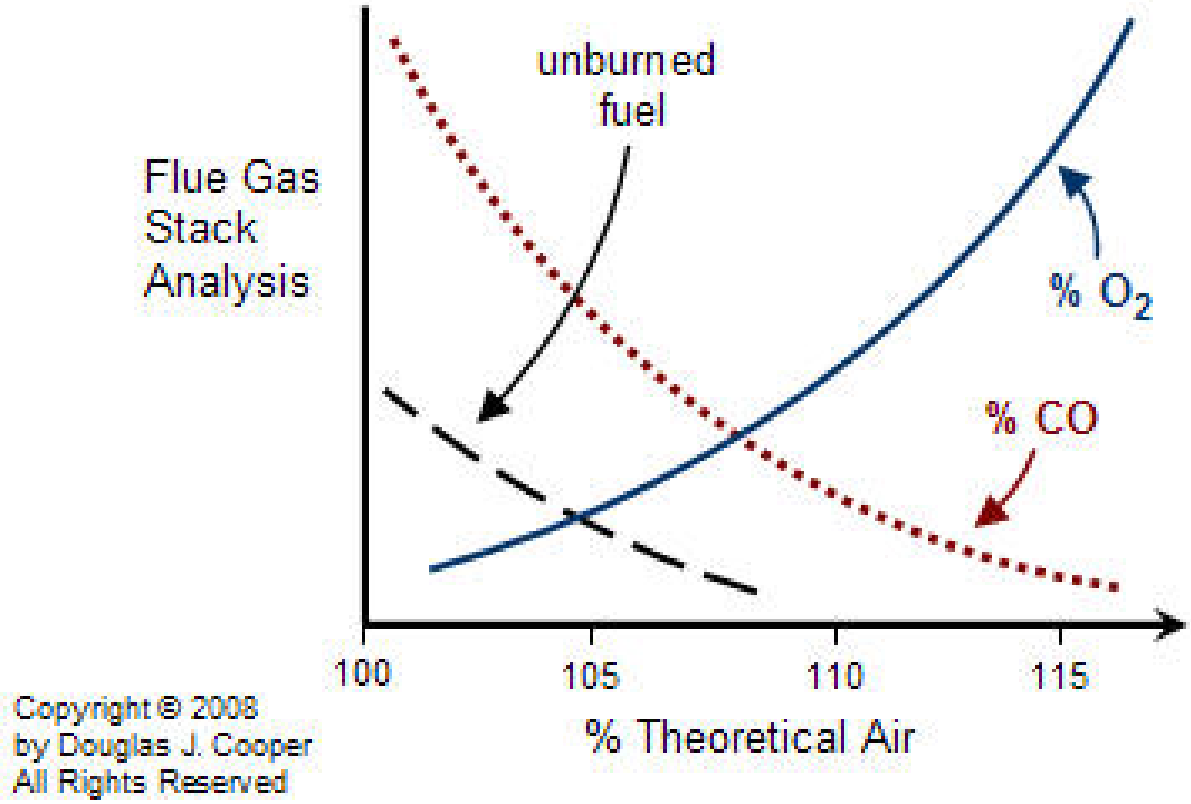
So when the air/fuel ratio is too high, we produce a surplus of hot air. And this hot air simply carries its heat energy up and out the exhaust stack as lost profit.

Theoretical (Stoichiometric) Air

The relationship between the air/fuel ratio, pollution formation and wasted heat energy provides a basis for control system design. In a meticulous laboratory experiment with exacting measurements, perfect mixing and unlimited time, we could determine the precise amount of air required to just complete the conversion of a hydrocarbon fuel to carbon dioxide and water. This minimum amount is called the “theoretical” or “stoichiometric" air.

Unfortunately, real combustion processes have imperfect mixing of the air with the fuel. Also, the gases tend to flow so quickly that the air and fuel mix have limited contact time in the combustion zone. As such, if we feed air in the exact theoretical or stoichiometric proportion to the fuel, we will still have incomplete combustion and lost profit.

Real burners generally perform in a manner similar to the graph below. The cost associated with operating at increased air/fuel ratios is the energy wasted in heating extra oxygen and nitrogen. Yet as the air/fuel ratio is decreased, losses due to incomplete combustion and pollution generation increase rapidly.



Copyright © 2008
by Douglas J. Cooper
All Rights Reserved

For any particular burner design, there is a target air/fuel ratio that balances the competing effects to minimize the total losses and thus maximize profit. As the graph above suggests (note that there is no scale on the vertical axis), a gas or liquid fuel burner generally balances losses by operating somewhere between 105% to 120% of theoretical air. This is commonly referred to as operating with 5% to 20% excess air.

Sensors Should be Fast, Cheap and Easy

Fired heaters, boilers and furnaces in processes with streams composed of gases, liquids, powders, slurries and melts are found in a broad range of manufacturing, production and development operations. Knowing that the composition of the fuel, the design of the burners, the configuration of the combustion zone, and the purpose of the process can differ for each implementation hints at a dizzying array of control strategy design and tuning possibilities.

To develop a standard control strategy, we require a flexible method of measuring excess air so we can control to a target air/fuel ratio. As discussed in this article, we normally seek sensors that are reliable, inexpensive, easy to install and maintain, and quick to respond. If we cannot get these qualities with a direct measurement of the process variable (PV) of interest, then an effective alternative is to measure a related variable if it can be done with a "fast, cheap and easy" sensor option.

Excess air is an example of a PV that is very challenging to directly measure in the combustion zone, yet oxygen and energy content in the stack gases is an appropriate alternative. As it turns out, operating with 5% to 20% excess air equates to having about 1% to 3% oxygen by volume in the stack gases.

Measuring the Stack Gases

By measuring exhaust stack gas composition, we obtain information we need to properly

monitor and control air/fuel ratio in the combustion zone. Stack analyzers fall into two broad categories:

- *Dry Basis Extractive Analyzers* pull a gas sample from the stack and cool it to condense the water out of the sample. Analysis is then made on the dry stack gas.
- *Wet Basis In Situ Analyzers* are placed in very close proximity to the stack. The hot sample being measured still contains the water vapor produced by combustion, thus providing a wet stack gas analysis.

A host of stack gas (or flue gas) analyzers can be purchased that measure O_2. The wet basis analyzers yield a lower oxygen value than dry basis analyzers by perhaps 0.3% – 0.5% by volume.

Instruments are widely available that also include a carbon monoxide measurement along with the oxygen measurement. A common approach is to pass the stack gas through a catalyst chamber and measure the energy released as the carbon monoxide and unburned fuel converts to carbon dioxide. The analyzer results are expressed as an equivalent percent CO in the sample. The single number, expressed as a CO measurement but representing fuel wasted because of insufficient air, simplifies control strategy design and process operation.

With a measurement of O_2 and CO (representing all lost fuel) in the stack of our combustion process, we have critical PV measurements needed to implement an air/fuel ratio control strategy. Note that it is the responsibility of the burner manufacturer and/or process design staff to specify the target set point values for a particular combustion system prior to controller tuning.

Air Flow Metering

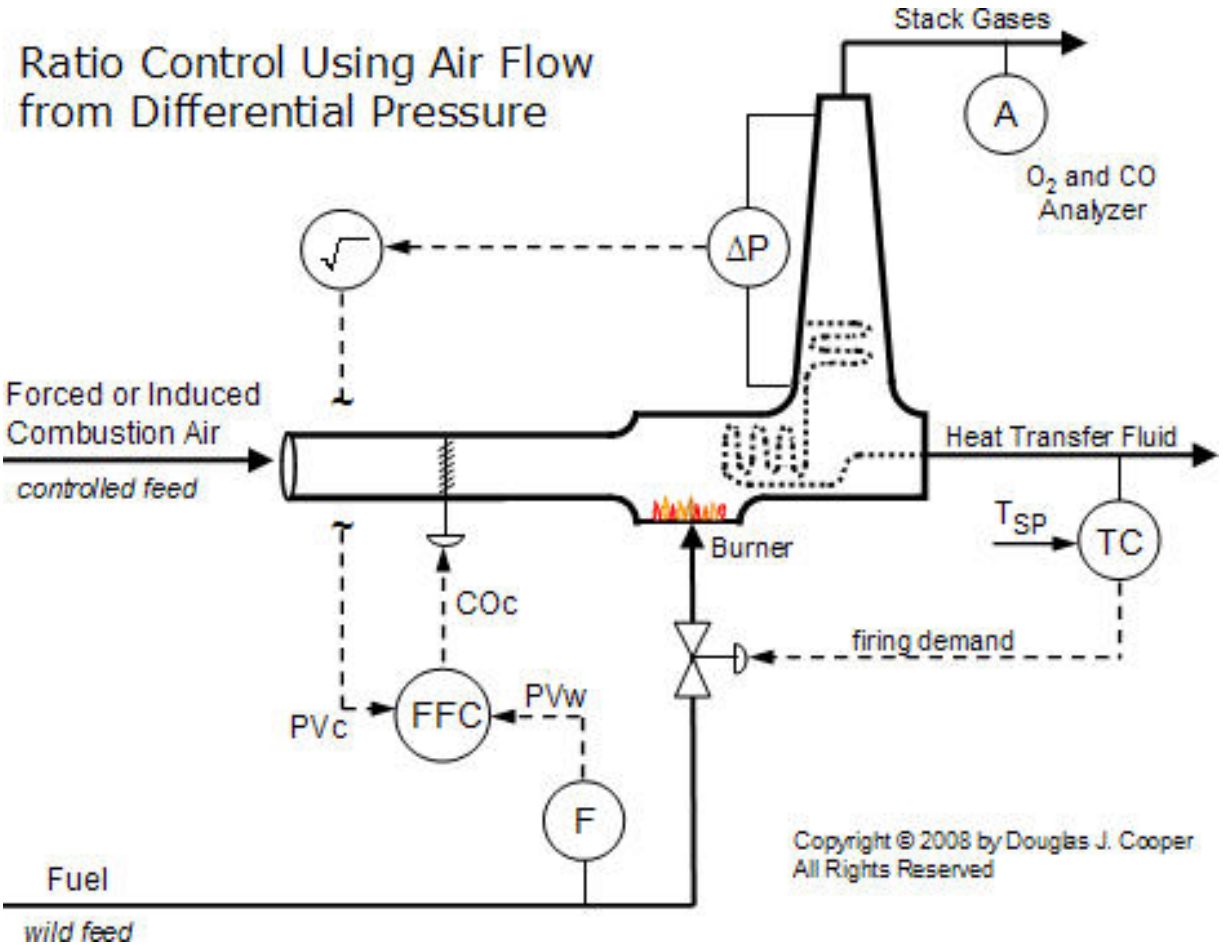
Combustion processes generally have combustion air delivered in one of three ways:

- A forced draft process uses a blower to feed air into the combustion zone.
- An induced draft process has a blower downstream of the burner that pulls or draws air through the combustion zone.
- A natural draft process relies on the void left as hot exhaust gases naturally rise up the stack to draw air into the combustion zone.

For this discussion, we assume a blower is being used to either force or induce combustion air feed because natural draft systems are not appropriately designed for active air flow manipulation.

Even with a blower, measuring the air feed rate delivered at low pressure through the twists and turns of irregular ductwork and firebrick is not cheap or easy. A popular alternative is to measure the pressure drop across some part of the exhaust gas stream. The bulk of the exhaust gas is nitrogen that enters with the combustion air. As long as the air/fuel ratio adjustments are modest, the exhaust gas flow rate will track the combustion air feed rate quite closely.

Thus, a properly implemented differential pressure measurement is a "fast, cheap and easy" method for inferring combustion air feed rate. The figure below () illustrates such a measurement across a heat transfer section and up the exhaust stack.



Also shown is that the controller output signal from the flow fraction (ratio) controller, COC, adjusts louvers to modulate the flow through the combustion zone. As the louvers open and close to permit more or less flow, the differential pressure measurement will increase or decrease, respectively.

As discussed in the ratio controller architecture article, the signal from the wild and controlled flow sensors must change linearly with flow rate. The differential pressure transmitter connected across a portion of the exhaust gas path becomes a linear gas flow sensor by recognizing that total gas flow, F, is proportional to the square root of the pressure differential (ΔP), or $F = \alpha\sqrt{\Delta P}$. Thus, the controlled feed process variable signal, PVC, is linear with flow when the square root of the differential pressure signal is extracted as shown in the diagram. Practitioner's Note: The differential pressure measurement must not be connected across the portion of the gas flow path that includes the adjustable louvers. Each change in louver position changes the F vs. ΔP relationship. Success would require that during calibration, we somehow determine a different coefficient □ for each louver position. This unrealistic task is easily avoided by proper location of the differential pressure taps.
Calibrating the differential pressure signal to a particular air feed rate is normally achieved while the fired heater, boiler or furnace is operating with the air/fuel ratio controller in manual mode. The maximum or full scale differential pressure calibration is determined by bringing the fuel flow firing rate to maximum (or as close as practical) and then adjusting the air feed flow rate until the design O_2 level is being measured in the stack gas.

The differential pressure being measured by these sensors is very small and the exhaust gas contains water vapor that can condense in sensing lines. Even one or two inches of

condensate in one side of the differential pressure transmitter can dramatically corrupt the measurement signal.

Choosing Air or Fuel for Firing Rate Control
With a means of measuring both the combustion air flow and the fuel flow, and with a stack analyzer to permit calibration and monitoring, we can implement the simple air/fuel ratio control as shown in the diagram above.

Because it can be measured accurately, fuel feed rate is a popular choice for the firing rate control variable. Yet in certain applications it is more desirable to employ the combustion air flow rate in this capacity.

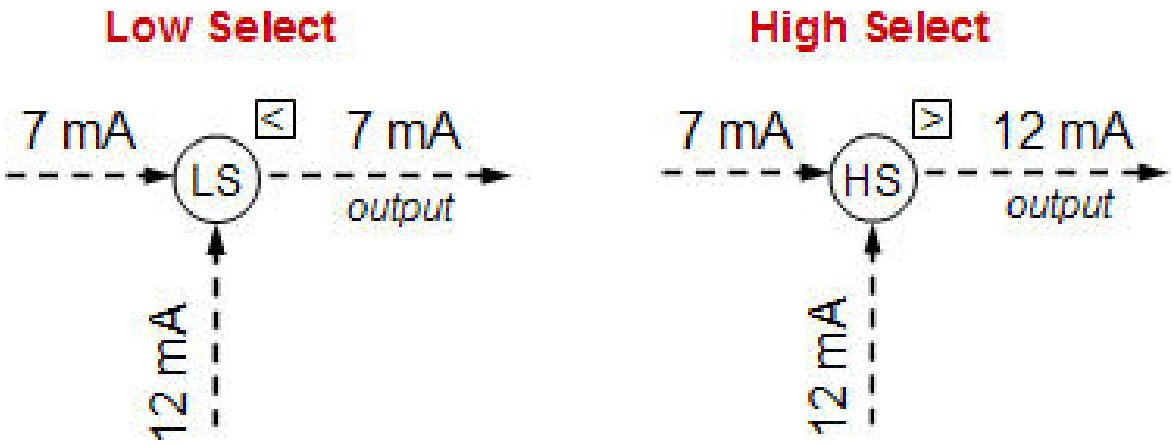
If fuel is the firing rate control variable, a rapid increase in firing rate with air following behind in time will lead to incomplete combustion as discussed above. On the other hand, if air is made the firing rate control variable, a rapid decrease in firing rate will lead to the same situation.

We study this issue in another article using a commercial furnace simulation. We focus on the benefits of a select override element to enhance safety, limit emissions and maximize useful energy from fuel.

Override (Select) Elements and Their Use in Ratio Control

By Allen Houtz[1] and Doug Cooper

A select element receives two input signals and forwards one of them onward in the signal path. A ***low select***, shown below to the left, passes the lowest of the two signals, while a ***high select***, shown to the right, passes the larger value onward.



A select element can be implemented as a DCS or PLC function block, as a few lines of computer code, or as a simple hardware circuit. And while the elements above are using electrical current, they can also be designed to select between high and low voltage or digital (discrete) counts.

The above pictures are not meant to imply that the selected output value has anything to

do with signal location. If the 12 mA signal shown entering on the lower input were to drop down to 5 mA while the 7 mA input entering from the left side remained constant, then the low select output would be 5 mA while the high select output would be 7 mA.

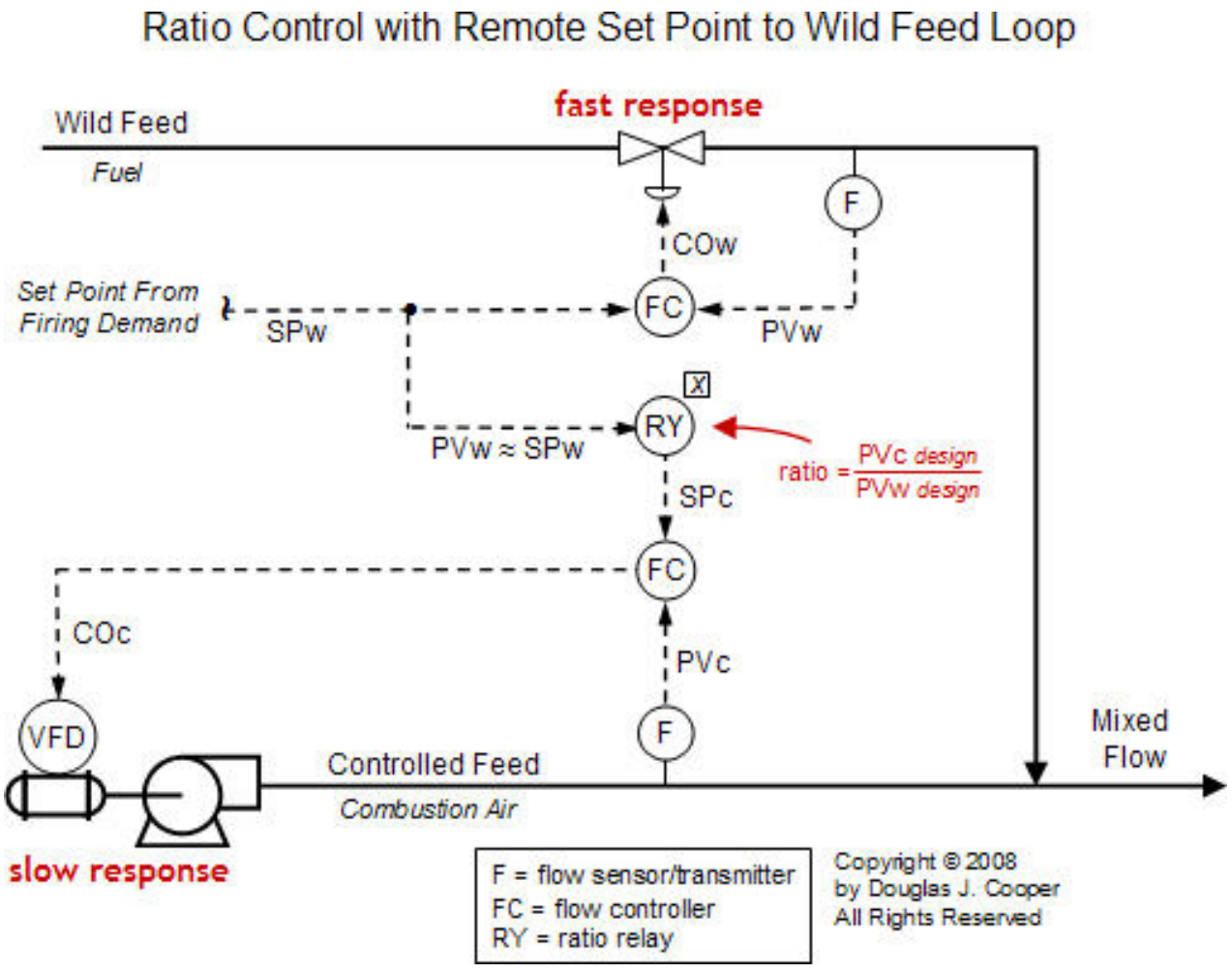
Logic Permits Increased Sophistication

The simple select element enables decision-making logic to be included in a control strategy, which in turn provides a means for increasing strategy sophistication. One such example is to use a select element to construct an architecture designed to control to a maximum or minimum limit or constraint.

Another popular application, and the one explored here, is to employ a select as an ***override element*** in a ratio control architecture. In particular, we explore how a select override might be included in an air/fuel ratio combustion control strategy to enhance safety, limit emissions and maximize useful energy from fuel.

Ratio Strategy Without Override

Before demonstrating the use of a select override, we consider a variation on our previously discussed ratio control of a metered-air combustion process. Shown below () is a ratio architecture much like that used in the referenced article except here we choose to employ a ratio relay with remote input rather than a flow fraction controller.



In this design, the fuel mass flow rate is regulated by a flow controller whose set point,

SPw, arrives as a firing demand from elsewhere in the plant. SPw might be generated, for example, by a controller adjusting the duct temperature downstream of a burner, the temperature of a heat transfer fluid exiting a furnace or the pressure in a steam header.

There is an implicit assumption in this architecture that the fuel mass flow rate closely tracks the firing demand set point, that is, PVw ≈ SPw. Thus, an integral term (e.g., PI control) is required in the wild feed flow control algorithm.

Since SPw is set elsewhere, we are not free to adjust fuel flow rate separately to maintain a desired air/fuel ratio. It is thus appropriately designated as the "wild feed" in this construction.

As SPw (and thus PVw) increases and decreases, a ratio relay shown in the control diagram multiplies the incoming signal by a design air/fuel ratio value (or in the general case, a controlled/wild feed ratio value) to compute the combustion air set point, SPc, for the controlled feed flow loop.

Practitioner's Note: A ratio controller architecture requires that the signal from each mass flow sensor/transmitter change linearly with flow rate. Thus, the signals from the wild stream process variable, PVw, and the controlled stream process variable, PVc, should increase and decrease in a straight-line fashion as the individual mass flow rates increase and decrease. If the flow sensor is not linear, additional computations (function blocks) must be included between the sensor and the ratio relay to transform the nonlinear signal into the required linear flow-to-signal relationship.

If the fuel flow control loop and the combustion air control loop both respond quickly to flow commands COw and COc respectively, then the architecture above should maintain the desired air/fuel ratio even if the demand set point signal, SPw, moves rapidly and often.

Problem if the Combustion Air Loop is Slow

The diagram shows a valve as the final control element (FCE) adjusting the fuel mass flow rate, and a variable frequency drive (VFD) and blower assembly as the FCE adjusting the combustion air mass flow rate. Valves generally respond quickly to controller output signal commands, so we expect the fuel mass flow rate to closely track changes in COw.

In contrast, air blower assemblies vary in capability. Here we consider a blower that responds slowly to control commands, COc, relative to the valve (the time constant of the blower "process" is much larger than that of the valve).

While we desire that the mass flow rates of the two streams move together to remain in ratio, the different response times of the FCEs means that during a firing demand change (a change in SPw), the feed streams may not be matched at the desired air/fuel ratio for a period of time.

To illustrate, consider a case where the firing demand, SPw, suddenly increases. The fuel flow valve responds quickly, increasing fuel feed to the burner. The ratio relay will receive SPw and raise the set point of the combustion air mass flow rate, SPc, so the two streams can remain in ratio.

If the air blower response is slow, however, a fuel rich environment can temporarily

develop. That is, there will be a period of time when we are below the desired 5% to 20% of excess air (below the 105% to 120% of theoretical or stoichiometric air) as we wait for the blower to ramp up and deliver more air to the burner.

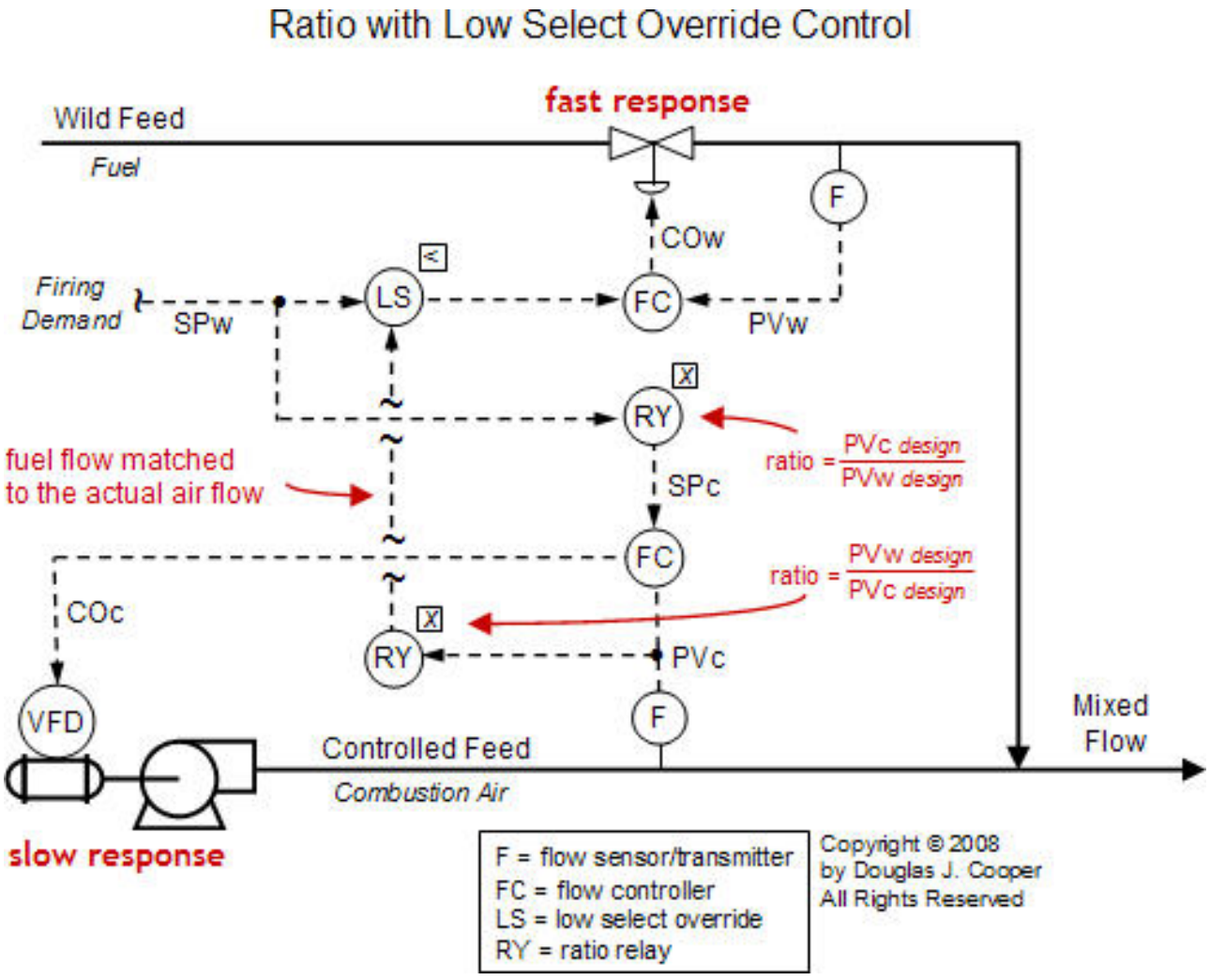
If there is insufficient air for complete combustion, then carbon monoxide and partially burned fuel will appear in the exhaust stack. As such, we have a situation where we are wasting expensive fuel and violating environmental regulations.

◊ **Solution 1: Detune the Fuel Feed Controller**
One solution is to enter conservative or sluggish tuning values into the fuel feed controller. By detuning (slowing down) the wild feed control loop so it moves as slowly as the combustion air blower, the two feed streams will be able to track together and stay in ratio. We thus avoid creating the fuel rich environment as just described.

Unfortunately, however, we also have made the larger firing demand control system less responsive, and this diminishes overall plant performance. In some process applications, a slow or sluggish ratio control performance may be acceptable. In the particular case of combustion control, it likely is not.

◊ **Solution 2: Use a Low Select Override**
The addition of an override to our control architecture is shown below (). The diagram is the same as that above except a second ratio relay feeding a low select element has been included in the design.

The second ratio relay receives the ***actual*** measured combustion air mass flow rate, PVc, and computes a matching fuel flow rate based on the design air/fuel ratio. This "fuel flow matched to the actual air flow" value is transmitted to the low select element. As shown below, the low select element also receives the firing demand fuel flow rate, SPw, set elsewhere in the plant.



A low select element passes the lowest of the two input signals forward. In this case, if SPw is a fuel rate that exceeds the availability of combustion air required to burn it, the select element will ***override*** the demand signal and forward the lower "fuel flow matched to the actual air flow" signal.

The override strategy shown above thus ensures that the feed streams remain in ratio for a rapid increase in firing demand, SPw, but it has no effect when there is a rapid decrease in firing demand.

When SPw rapidly decreases, the fuel flow rate will respond quickly and we will be in a "lean" environment (too much combustion air) until the blower slows to match the decreased fuel rate. When there is more air than that needed for complete combustion, the extra nitrogen and unneeded oxygen absorb heat energy, decreasing the temperature of the flame and gases in the combustion zone.

So while the select override element has eliminated pollution concerns when firing demand rapidly *increases*, we produce a surplus of hot air that simply leaves the exhaust stack as lost profit when firing demand rapidly *decreases*. In effect, we have solved only half the air/fuel balance problem with a single select override element.

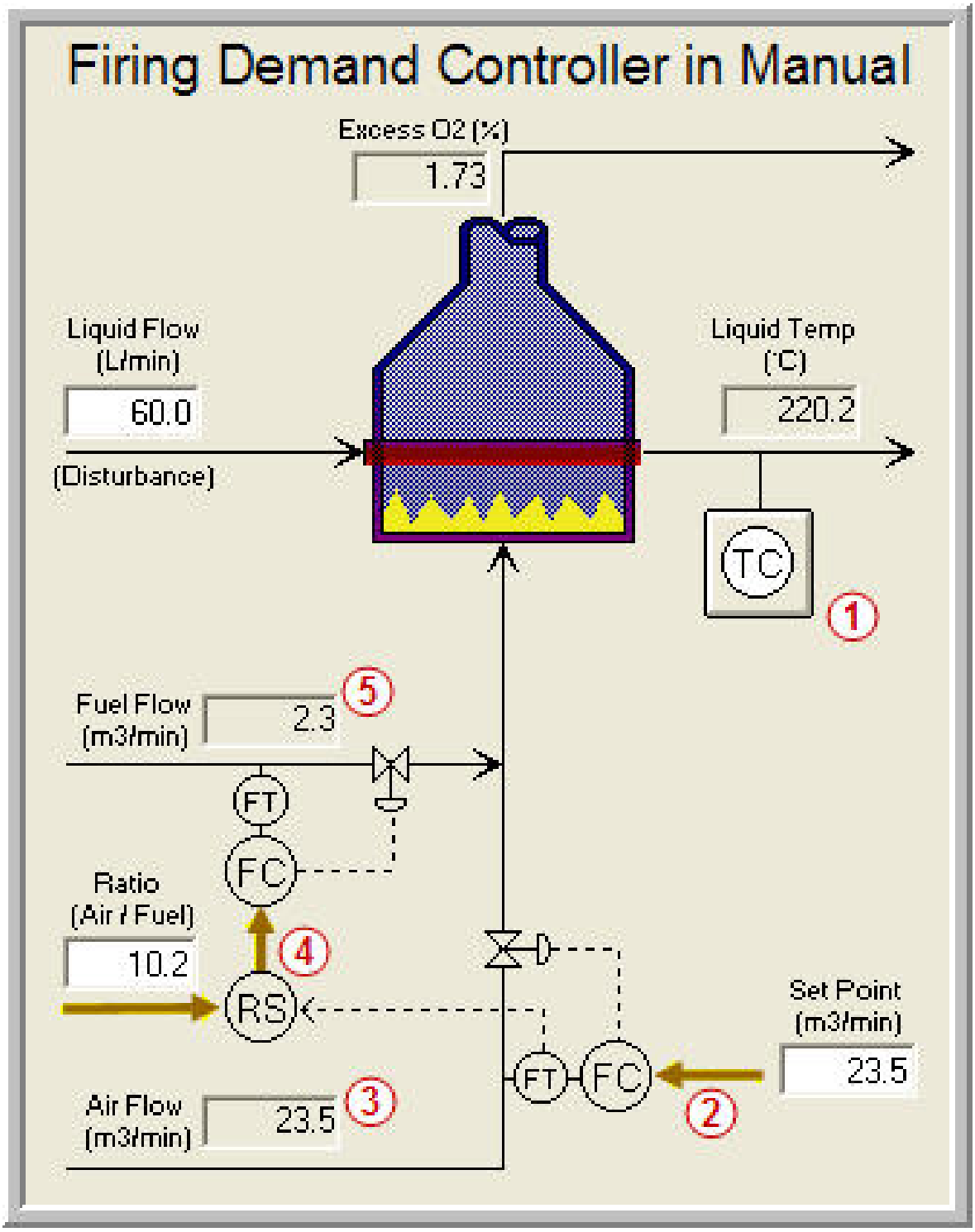
A Simulated Furnace Air/Fuel Ratio Challenge
To further our understanding of the select override in an air/fuel ratio strategy, we consider

a furnace simulation (images shown below) available in commercial software. The furnace burns natural gas to heat a process liquid flowing through tubes in the fire box. Firing demand is determined by a temperature controller located on the process liquid as it exits the furnace.

Because the output of the firing demand temperature controller becomes the set point to the wild feed of the air/fuel ratio strategy, it is in fact a primary (or outer) controller in a cascade control architecture. If the temperature of the process liquid is too hot (greater than set point), the firing demand controller seeks to reduce energy input. If the temperature is below set point, it seeks to add energy.

Unlike the example above, combustion air is the wild feed in this furnace simulation. Thus, when the firing demand temperature controller is in automatic (the cascade is enabled) set point changes are transmitted to the air flow controller. If the temperature controller is in manual, the set point of the combustion air flow controller must be entered manually by operations staff.

● *Firing Demand in Manual*
We first consider furnace operation when the firing demand temperature controller is in manual mode as shown below ().



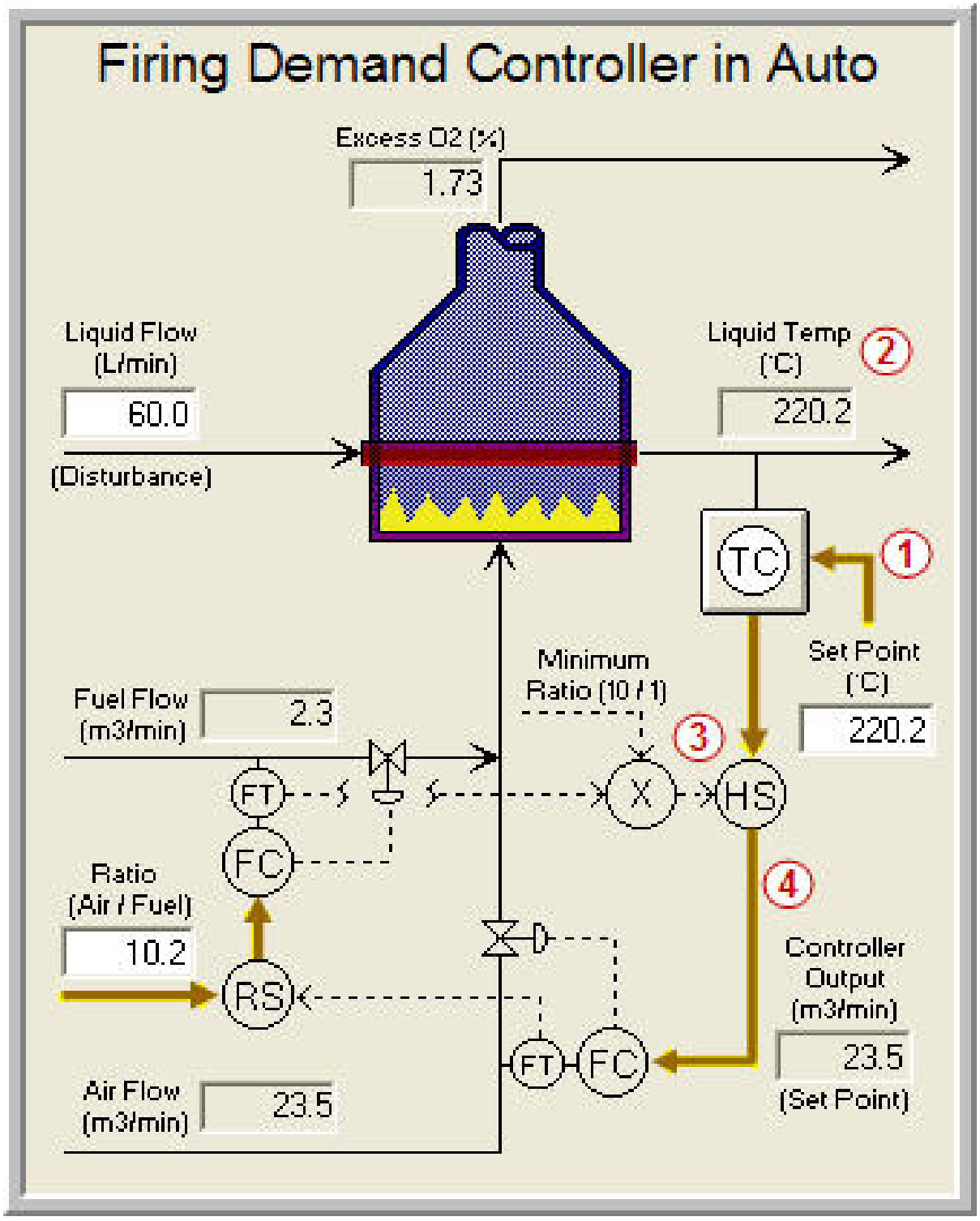
Following the number labels on the above diagram:

1. The firing demand temperature controller on the process liquid exiting the furnace is in manual mode. As such, it takes no control actions.
2. With the firing demand controller in manual, operations staff enter a set point (SP) into the combustion air flow controller. The controller adjusts the air flow valve to ensure the measured combustion air feed equals the SP value.
3. The display is in volume units (m3/min), though ratio controllers traditionally employ a mass flow basis (see Notes below).

4. Operations staff enter a desired air/fuel ratio into the ratio station (RS). The entered value is much like a set point. The ratio station receives the combustion air flow signal and forwards a fuel flow set point to maintain the desired ratio.
5. The flow controller adjusts the fuel valve to maintain the desired ratio. Here, the air/fuel ratio is: 23.5/2.3 = 10.2

Notes:
▪ The fuel flow controller has its set point established by the signal from the ratio station (RS), which could be constructed, for example, by inverting the desired air/fuel ratio with a division function and then using a multiplying relay to compute a desired fuel flow rate.
▪ The flow transmitters for the combustion air and fuel rate must be linearized as discussed in the introductory ratio control architecture article.
▪ Ratio control traditionally uses a mass flow basis. The use of volumetric flow units implies that the air and fuel are delivered at fixed temperature and pressure, thus making the volume flows proportional to the mass flows. Alternatively, a sophisticated function could translate mass flows to volume flows for display purposes. An air/fuel ratio in mass units (kg for example) would have a different value from the volume ratio because of the difference in molecular weights for the two streams.

● *Firing Demand in Automatic*
With operation steady, we switch the firing demand temperature controller to automatic as shown below ().



Following the number labels on the above diagram:

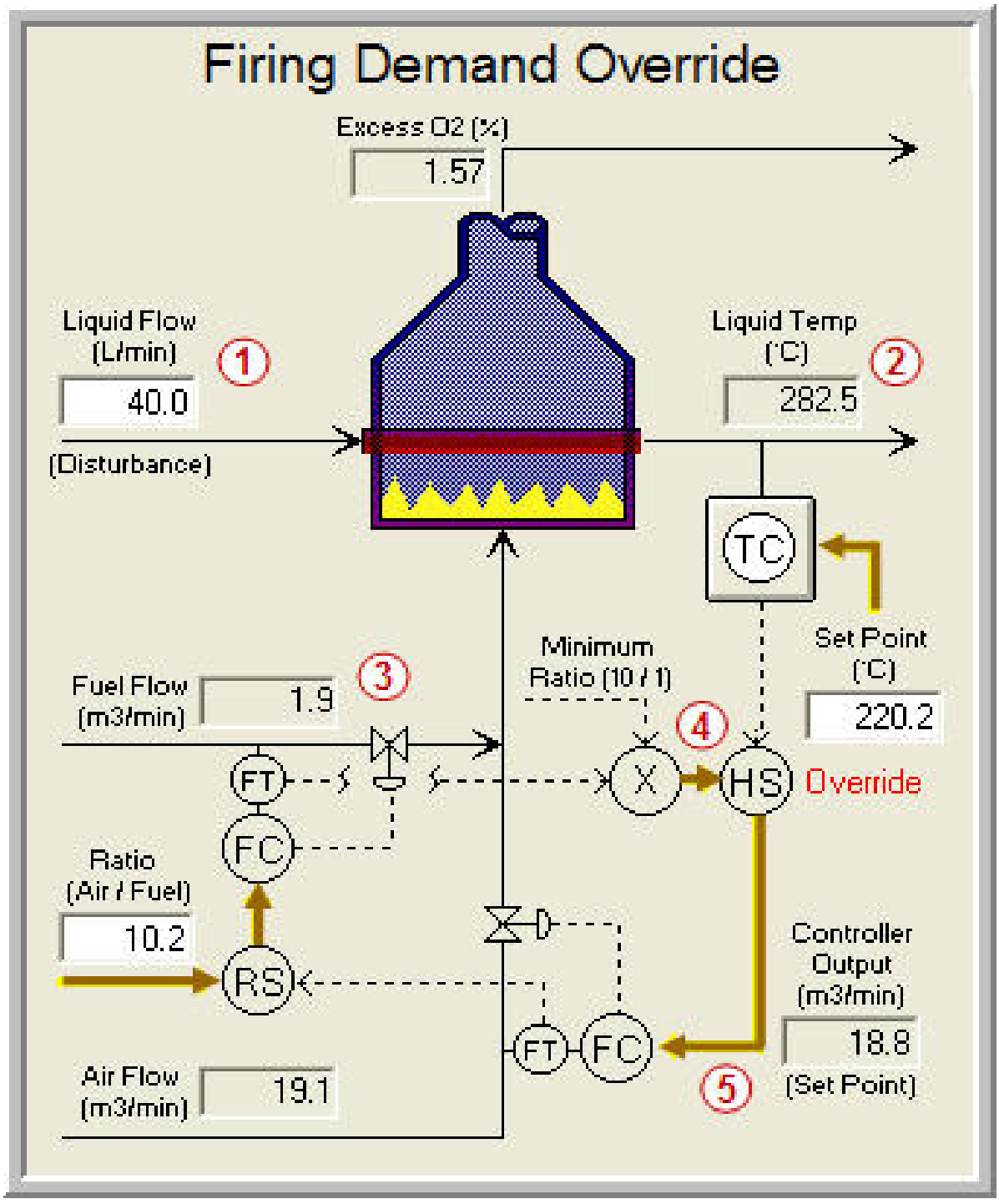
1. The firing demand controller measures the temperature of the process liquid exiting the furnace (the measured process variable, PV), compares it to the set point (SP) value, and computes as its controller output signal an air feed rate set point.
2. In the figure above, the measured PV equals the SP of the temperature controller, so the air feed set point from the firing demand controller is the same as when it was in manual mode.
3. A high select element receives the air feed set point from the firing demand controller

and a minimum permitted air feed set point based on the current flow of fuel.

4. The larger of the two air feed set points is forwarded by the high select element to the air flow controller, ensuring that there is always sufficient air to completely combust the fuel in the firebox. Because the firing demand controller generates an air feed set point that is above the minimum 10/1 ratio specified by the designers, then the high select element passes it onward to the combustion air flow controller.

● *Firing Demand Override*

A process upset requires that the high select element override the firing demand controller as shown below ().



Following the number labels on the above diagram:

1. The flow rate of the process liquid drops from 60 to 40 L/min, reducing the heat energy demand on the furnace.
2. As flow rate drops, the excess energy in the furnace raises the exit temperature of the process liquid. The measured PV temperature moves well above the SP, causing the firing demand controller to decrease energy input by rapidly lowering the set point to the air flow controller.
3. The current flow rate of fuel is 1.9 (actually 1.88 but the display has round off).
4. The minimum ratio that ensures enough air to complete combustion in the firebox of this furnace is a 10/1 ratio of air/fuel, or 1.88 x 10 = 18.8 m3/min. The high select element receives a combustion air feed rate from the firing demand controller that is below this minimum.
5. The high select element overrides the air flow set point from the firing demand controller and forwards the minimum 18.8 m3/min air flow sufficient for complete combustion.

The first example in this article employed fuel as the wild feed. This second furnace example used combustion air as the wild feed. Yet in both cases, an override element was required to implemented a control strategy that enhanced safety, limited emissions and maximized the useful energy from fuel.

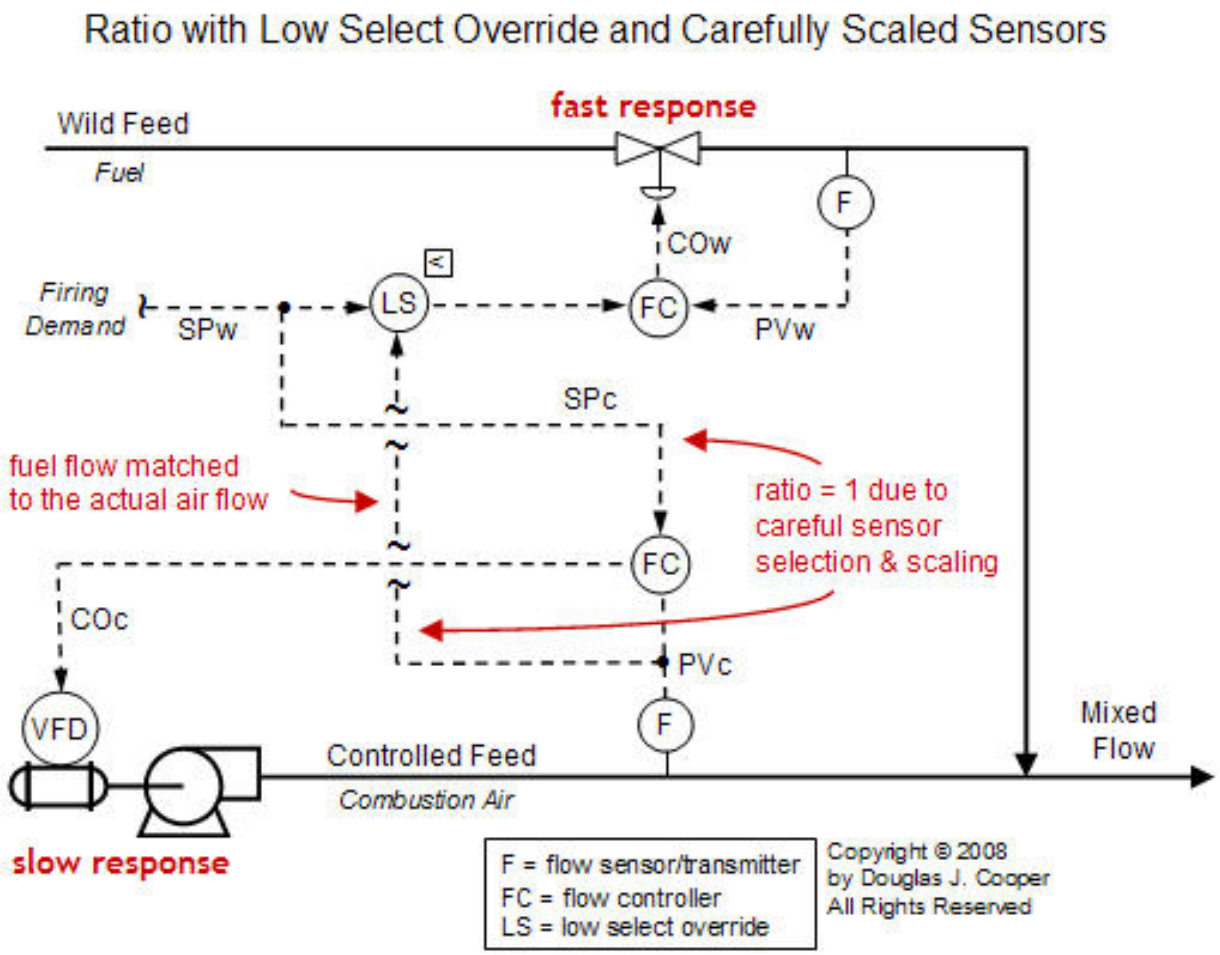
Cross-Limiting Ratio Control Strategy

If we were to increase the process liquid flow rate through the furnace, the firing demand controller would quickly ramp up the combustion air feed rate to provide more energy. Temporarily, there would be more air than that needed for complete combustion. That temporary surplus of hot air will carry its heat energy up and out the exhaust stack as lost profit.

So similar to the first example, a single select override element provides only half the solution depending on the direction that the upstream demand is moving.

The next article presents the use of two select override elements in a cross-limiting ratio control strategy. While fairly complex, the cross limiting structure offers benefit in that it provides protection in an air/fuel ratio strategy both when firing demand is increasing *and* decreasing.

We will simplify the diagrams in that article as shown below () by assuming that the two linearized flow sensor/transmitters have been carefully scaled so that both signals at the desired air/fuel ratio are exactly one to one.



By careful sensor selection and scaling, we can maintain the "ratio with low select override" strategy as presented earlier in this article while eliminating the multiplying relays from our design. As long as we use control algorithms with an integrating term (PI or PID), the upstream demand signal becomes the set point for both controllers and the desired ratio will be maintained.

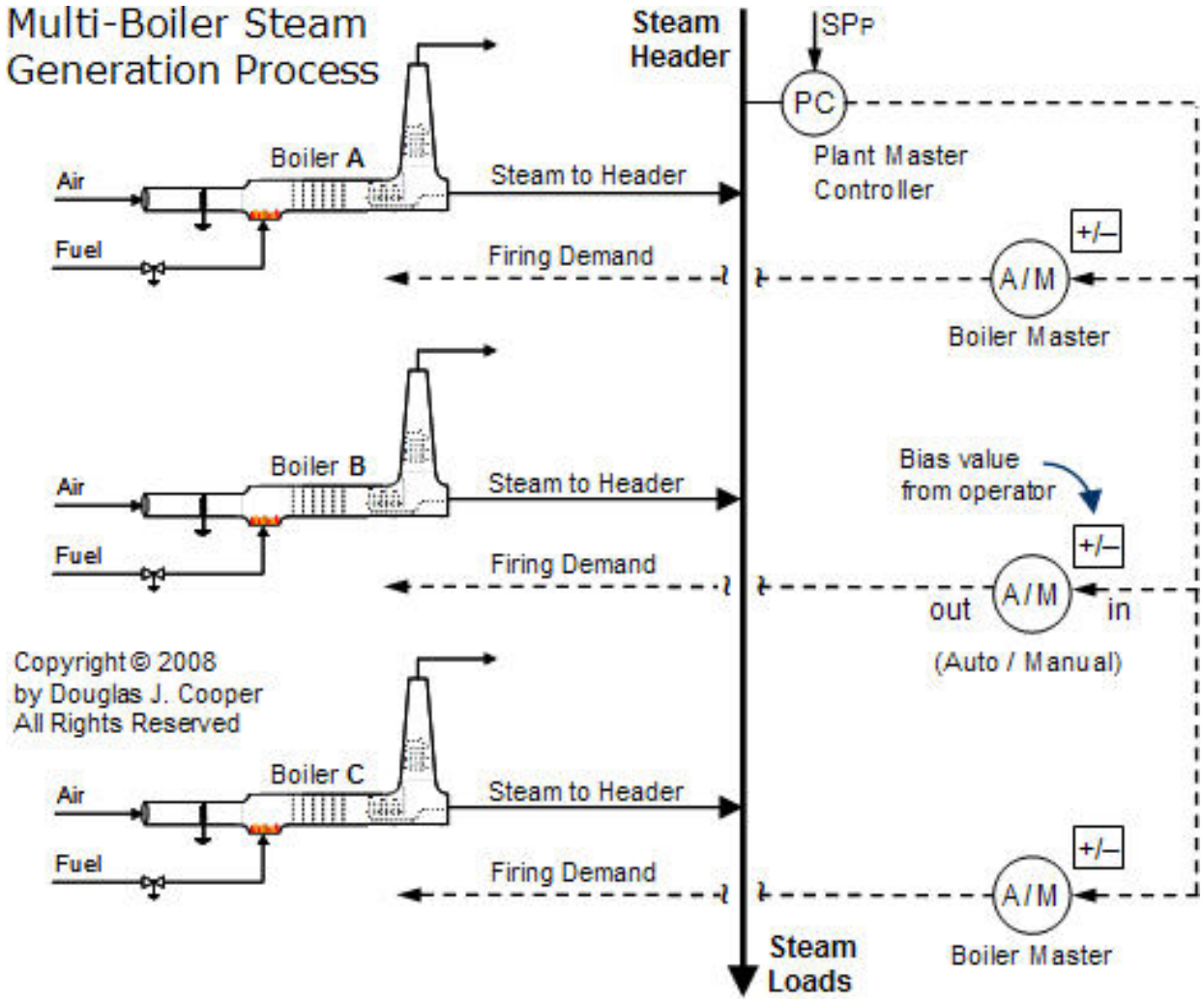
Ratio with Cross-Limiting Override Control of a Combustion Process

We explored override control using select elements in a previous article and learned that environmental and energy efficiency concerns for metered-air combustion processes can be partially addressed with a single select override element. Examples illustrated how a select override can either prevent having too much fuel *or* too much air in the air/fuel mixture fed to the burner of a combustion process, but one override element alone is not capable of preventing both scenarios.

In this article we explore the addition of a second select override element to create a cross-limiting architecture that prevents the air/fuel ratio fed to the burner from becoming overly rich (too much fuel) or lean (too much air) as operating conditions change. Variations on this cross-limiting architecture are widely employed within the air/fuel ratio

logic of a broad range of industrial combustion control systems.

Steam Boiler Process Example
To provide a larger context for this topic, we begin by considering a multi-boiler steam generation process as shown below ():



Steam generation processes often have multiple boilers that feed a common steam header. When steam is needed anywhere in the plant, the load is drawn from this common header. Steam turbines, for example, drive generators, pumps and compressors. Steam is widely used for process heating, can be injected into production vessels to serve as a reactant or diluent, and may even be used to draw a vacuum in a vessel via jet ejectors.

With so many uses, steam loads can vary significantly and unpredictably over time in a plant. The individual boilers must generate and feed steam to the common header at a rate that matches these steam load draws. Controlling the steam header to a constant pressure provides an important stabilizing influence to plant-wide operation.

● *Plant Master Controller*

A popular multi-boiler architecture for maintaining header pressure is to use a single pressure controller on the common header that outputs a firing demand signal for ***all*** of the boilers in the steam plant. This steam header pressure controller is widely referred to as the ***Plant Master***.

Based on the difference between the set point (SPP) and measured pressure in the header, the Plant Master controller computes a firing demand output that signals all of the boilers in the plant to increase or decrease firing, and thus, steam production.

● *Boiler Master Controller*
The Boiler Masters in the above multi-boiler process diagram are Auto/Manual selector stations with biasing (+/–) values. If all three of the Boiler Masters are in automatic, any change in the Plant Master output signal will pass through and create an associated change in the firing demand for the three boilers.

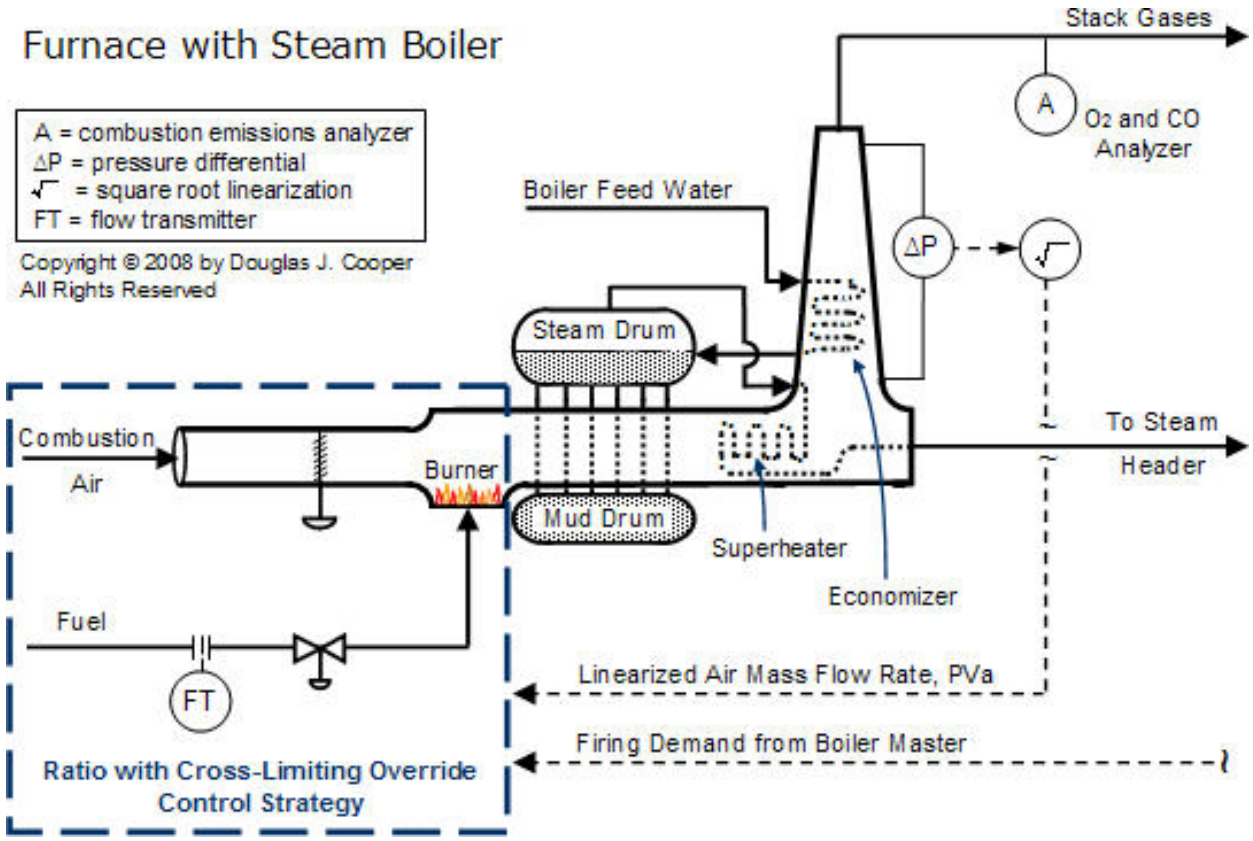
If a Boiler Master is in automatic, that boiler is said to be operating as a ***swing boiler***. As such, its firing demand signal will vary (or swing) directly as the Plant Master signal varies. If each of the fuel flow meters are scaled so that 100% of fuel flow produces maximum rated steam output, then each boiler will swing the same amount as the Plant Master calls for variations in steam production.

But suppose Boiler B has cracked refractory brick in the fire box or some other mechanical issue that, until repaired, requires that it be operated no higher than, for example, 85% of its design steam production rate. That is, Boiler B has been *derated* and its maximum permissible steam generating capacity has been lowered from the original design rating. Two options we can consider include:

1. When a Boiler Master is in automatic, then: signal out = signal in + bias
   where the bias value is set by the operator. If the bias value of Boiler Master B is set in this example to –15%, then no matter what output is received from the Plant Master (0% to 100%), the firing demand signal will never exceed 85% (100% plus the negative 15% bias). In this mode of operation, Boiler B will still swing with Boiler A and Boiler C in response to the Plant Master, but it will operate at a firing rate 15% below the level of the other two boilers (assuming their bias values are zero).

2. If a boiler is suffering from refractory problems, then allowing the firing rate to swing can accelerate refractory degradation. Thus, Boiler Master B might alternatively be switched to manual mode where the output firing demand signal is set to a constant value. In manual mode, Boiler B is said to provide a ***base load*** of steam production. With the firing rate of Boiler B set manually from the Boiler Master, it is unresponsive to firing demand signal variations from the Plant Master. We then would have two swing boilers (Boiler A and Boiler C) and one base loaded boiler (Boiler B).

Combustion Control Process
As shown below (), each furnace and steam boiler has its own control system. Of particular interest here is the maintenance of a specified air/fuel mass ratio for efficient combustion at the burners.



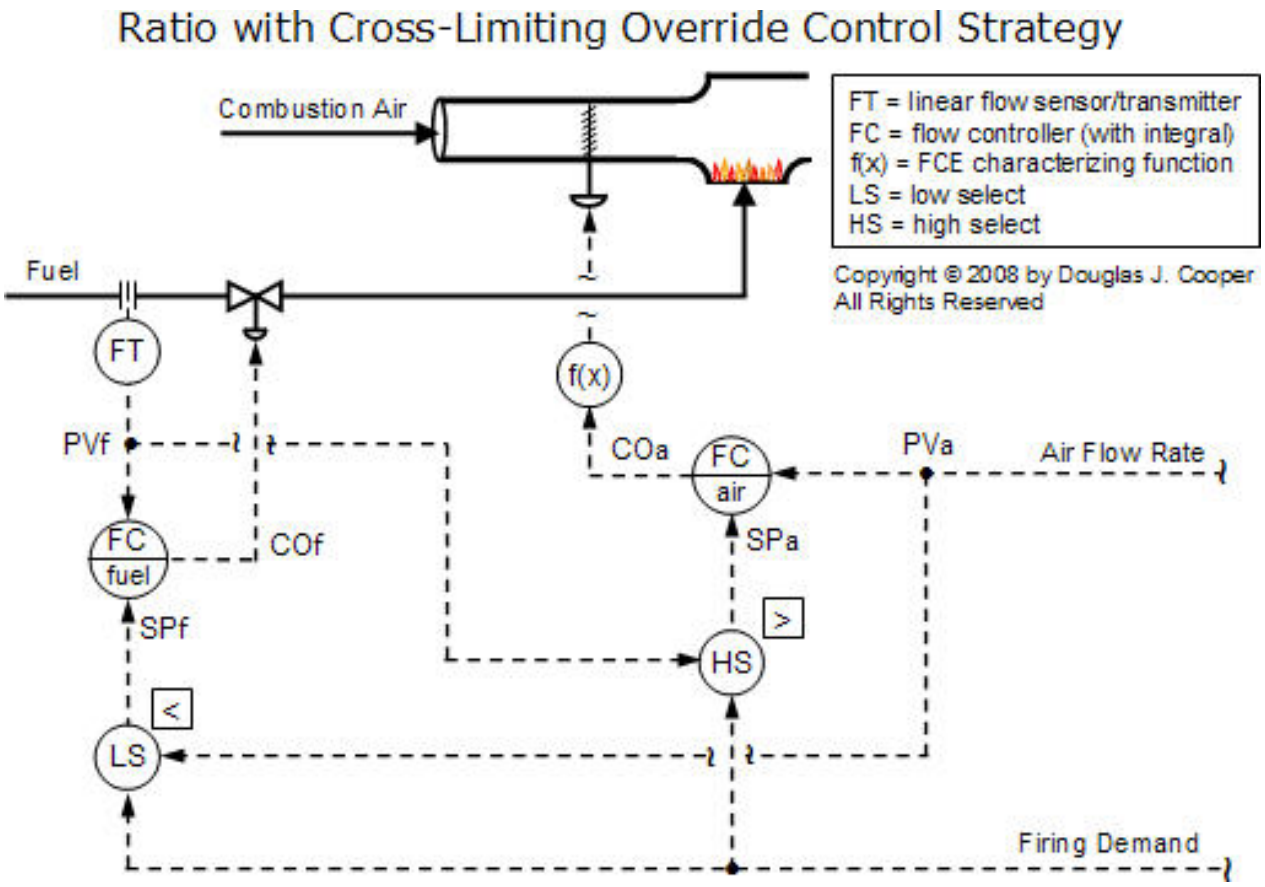
As shown above, the air/fuel ratio control strategy receives a firing demand from the Boiler Master. Air mass flow rate may be measured downstream of the combustion zone and is thus shown as an input to the ratio control strategy.

The boiler feed water and steam drum level controls are not discussed here but can be found in this 3-Element Level Control article.

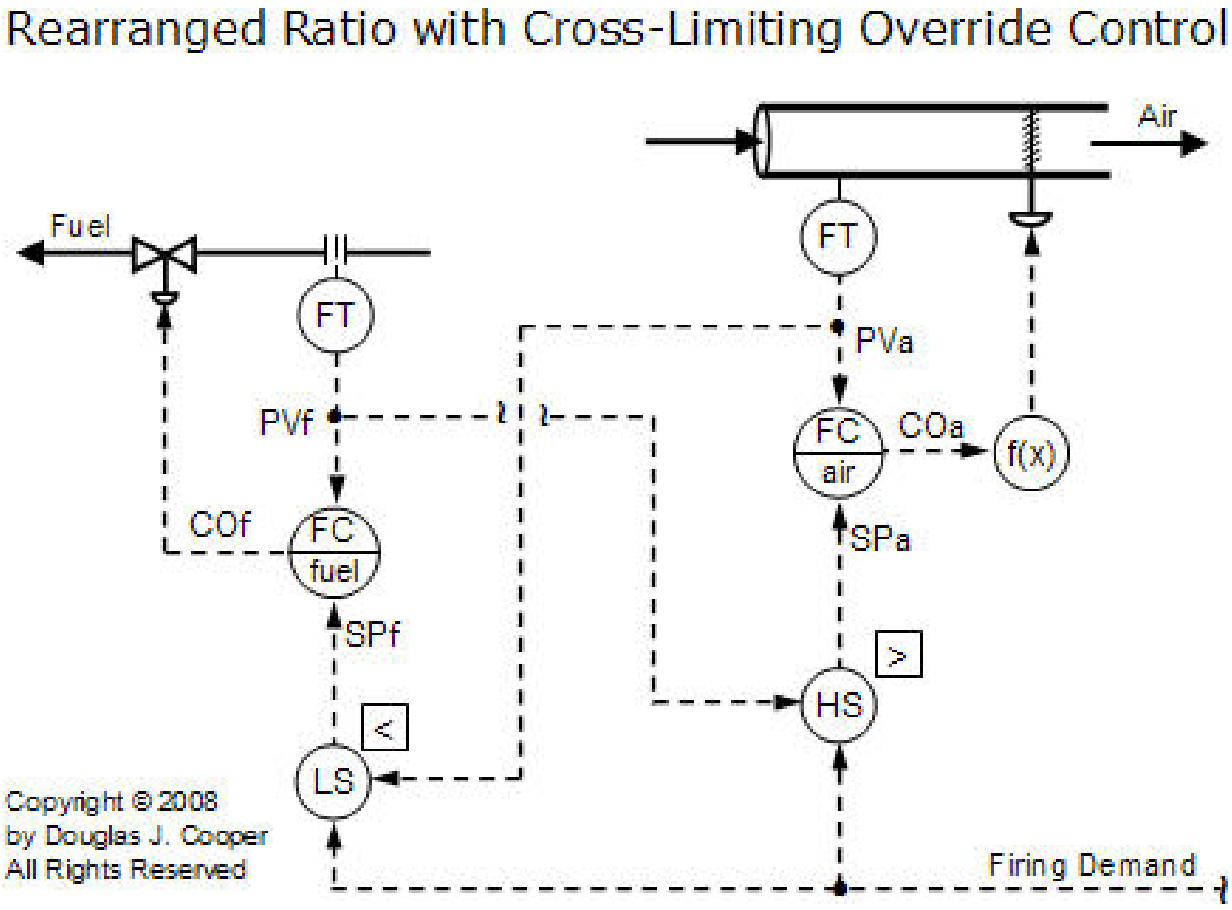
Ratio with Cross-Limiting Override Control
Certain assumption are used in the presentation that follows:

1. Air/fuel ratio is normally expressed as a mass flow ratio of air to fuel.
2. The air and fuel flow transmitter signals are linear with respect to the mass flow rate and have been scaled to range from 0-100%.
3. The flow transmitters have been carefully calibrated so that both signals at the design air/fuel ratio are one to one. That is, if the fuel flow transmitter signal, PVf, is 80%, then an air flow signal, PVa, of 80% will produce an air flow rate that meets the design air/fuel mass ratio. This enables us to implement the ratio strategy without using multiplying relays as discussed at the end of this article.

Shown below () are the sensors, controllers, final control elements (FCEs) and function blocks that might be included in the above dashed box labeled "ratio with cross-limiting override control strategy."



Before discussing the details of the strategy, we rearrange the loop layout to make the symmetry of the design more apparent (). Specifically, we reverse the fuel flow direction (fuel now flows from right to left below) and show the air mass flow rate transmitter as a generic measurement within the control architecture. The control diagram above is otherwise identical to that below.



Practitioner's Note: In any real process, different flow loops will have different process gains (the same change in controller output signal, CO, will produce a different change in flow rate) and each loop itself will display a nonlinear behavior over its range of operation (the process gain, time constant and/or dead time will change as operating level changes). The purpose of the characterizing function block, f(x), is to match the process gain of one loop over the range of operation with that of the other loop. With matching signal-to-flow gains, this optional function block simplifies the tuning of a ratio control strategy with two flow control loops. The characterizing function block, f(x), also simplifies manual operation because the two flow CO signals will be approximately equal at the design air/fuel ratio.

As shown above, the firing demand signal enters the high select override as a candidate for the set point of the air flow controller (SPa). In this cross-limiting strategy, the same firing demand signal enters the low select override as a candidate for the set point of the fuel flow controller (SPf).

As discussed in assumption 3 above, the flow transmitters have been calibrated so that when both signals match, we are at the design air/fuel mass flow ratio. Thus, because of the high select override, SPa is always the greater of the the firing demand signal or the value that matches the current fuel flow signal. And because of the low select override, SPf is always the lesser of the firing demand signal or the value that matches the current air flow signal.

The result is that if firing demand moves up, the high select will pass the firing demand signal through as SPa, causing the air flow to increase. Because of the low select

override, the fuel set point, SPf, will not match the firing demand signal increase, but rather, will follow the increasing air flow rate as it responds upward.

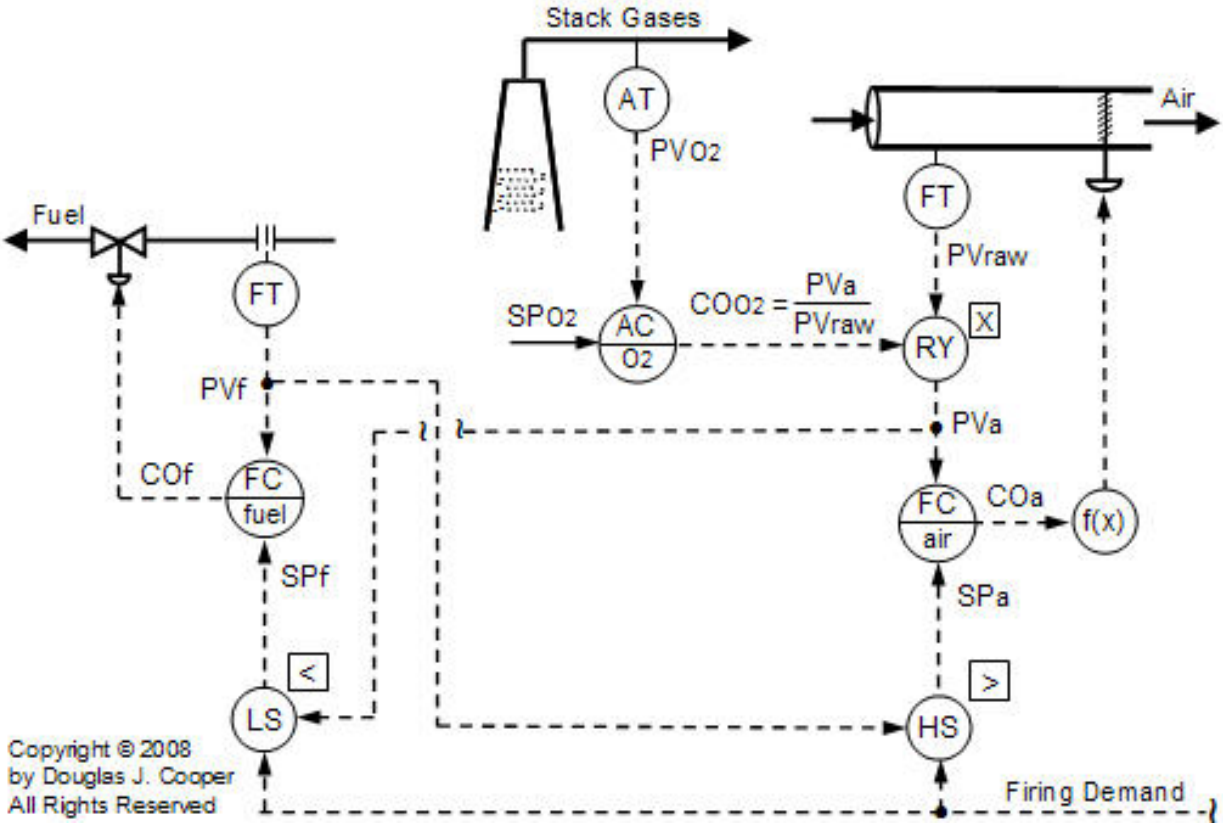
And if the firing demand moves down, the low select will pass the firing demand signal through as SPf, causing the fuel flow to decrease. Because of the high select override, the air set point, SPa, will not match the firing demand signal decrease, but rather, will track the decreasing fuel flow rate as it moves downward.

In short, the control system ensures that during sudden operational changes that move us in either direction from the design air/fuel ratio, the burner will temporarily receive extra air until balance is restored (we will be temporarily lean). While a lean air/fuel ratio means we are heating extra air that then goes up and out the stack, it avoids the environmentally harmful emission of carbon monoxide and unburned fuel.

Variable Air/Fuel Ratio
The basic cross-limiting strategy we have described to this point provides no means for adjusting the air/fuel ratio. This may be necessary, for example, if the composition of our fuel changes, if the burner performance changes due to corrosion or fouling, or if the operating characteristics of the burner change as firing level changes.

Shown below () is a cross-limiting override control strategy that also automatically adjusts the air/fuel ratio based on the oxygen level measured in the exhaust stack.

Ratio with Cross-Limiting Override & Oxygen Trim (Remote Input)



As shown in the diagram, the signal from the air flow transmitter, PVraw, is multiplied by the output of the analyzer controller, COO2, and the product is forwarded as the

measured air flow rate process variable, PVa.

With this construction, if the measured exhaust oxygen, PVO2, matches the oxygen set point, SPO2, then the analyzer controller (AC) output, COO2 will equal one and PVa will equal PVraw.

But if the oxygen level in the stack is too high, COO2 will become greater than one. By multiplying the raw air flow signal, PVraw, by a number greater than one, PVa appears to read high. And if the oxygen level in the stack is too low, we multiply PVraw with a number smaller than one so that PVa appears to read low.

The ratio strategy reacts based on the artificial PV values, adjusting the air/fuel ratio until the measured oxygen level, PVO2, is at set point SP02.

This manipulation to the air/fuel ratio based on measured exhaust oxygen is commonly called oxygen trim control. By essentially changing the effective calibration of the air flow transmitter to a new range, the signal ratio of the carefully scaled air and fuel transmitters can remain 1:1.

Practitioner's Note: Analyzers fail more often than other components in the strategy, so when designing and tuning the analyzer controller, it is important to limit how far COO2 can move from its baseline value of one. Also, the analyzer controller is effectively the primary (outer) controller in a cascade loop. The secondary (inner) loop is the same air flow control loop being driven by the Plant Master. As a result, it is advisable to tune the oxygen (or combustibles) trim controller significantly more conservatively than the Plant Master to minimize loop interactions.

18) Cascade, Feed Forward and Three-Element Control

Cascade, Feed Forward and Boiler Level Control

By Allen D. Houtz [1]

One common application of cascade control combined with feed forward control is in level control systems for boiler steam drums.

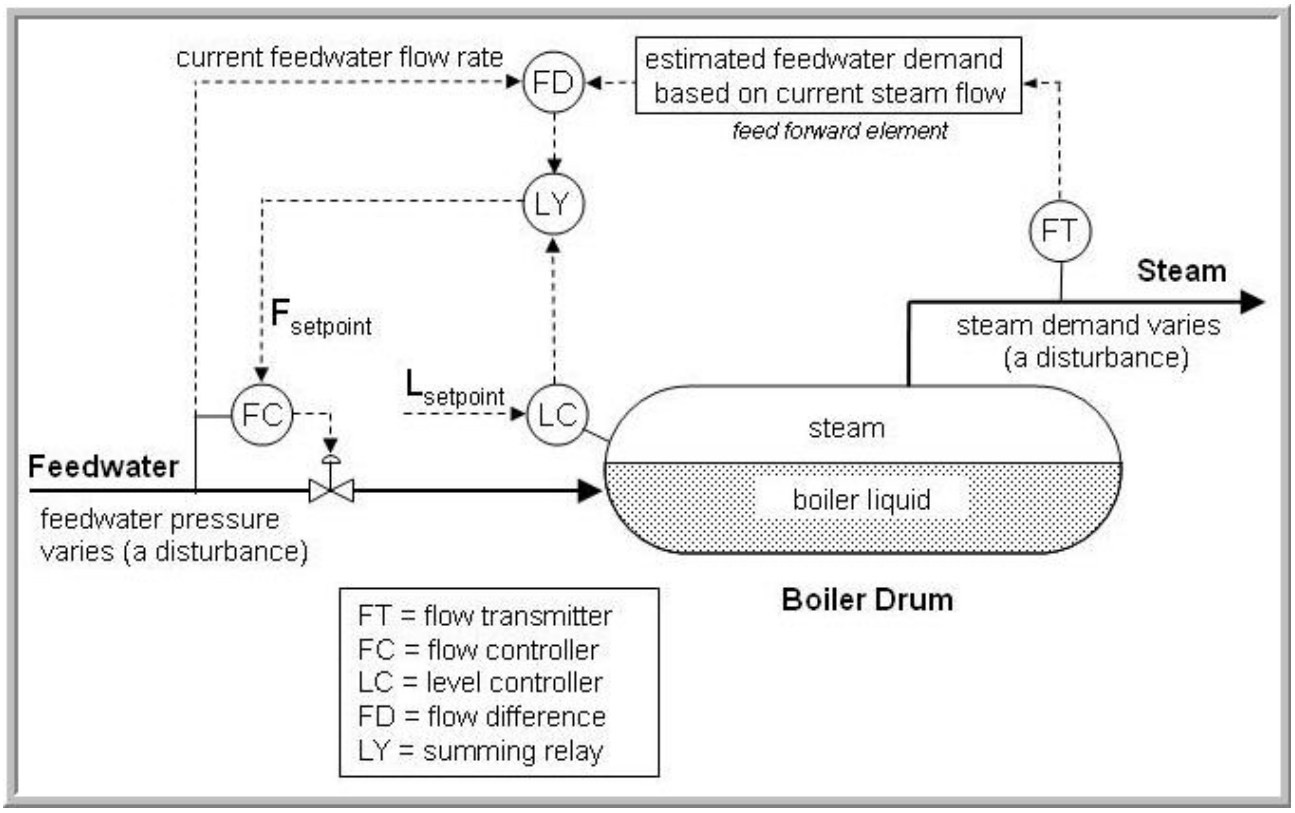
The control strategies now used in modern industrial boiler systems had their beginnings on shipboard steam propulsion boilers. When boilers operated at low pressure, it was reasonably inexpensive to make the steam drum large. In a large drum, liquid level moves relatively slowly in response to disturbances (it has a long time constant). Therefore, manual or automatic adjustment of the feedwater valve in response to liquid level variations was an effective control strategy.

But as boiler operating pressures have increased over the years, the cost of building and installing large steam drums forced the reduction of the drum size for a given steam production capacity.

The consequence of smaller drum size is an attendant reduction in process time constants, or the speed with which important process variables can change. Smaller time constants mean upsets must be addressed more quickly, and this has led to the development of increasingly sophisticated control strategies.

3 Element Strategy
As shown below, most boilers of medium to high pressure today use a “3-element” boiler control strategy. The term “3-element control” refers to the number of process variables (PVs) that are measured to effect control of the boiler feedwater control valve. These measured PVs are:

- liquid level in the boiler drum,
- flow of feedwater to the boiler drum, and
- flow of steam leaving the boiler drum.



Maintaining liquid level in the boiler steam drum is the highest priority. It is critical that the liquid level remain low enough to guarantee that there is adequate disengaging volume above the liquid, and high enough to assure that there is water present in every steam generating tube in the boiler. These requirements typically result in a narrow range in which the liquid level must be maintained.

The feedwater used to maintain liquid level in industrial boilers often comes from multiple sources and is brought up to steam drum pressure by pumps operating in parallel. With multiple sources and multiple pumps, the supply pressure of the feedwater will change over time. Every time supply pressure changes, the flow rate through the valve, even if it remains fixed in position, is immediately affected.

So, for example, if the boiler drum liquid level is low, the level controller will call for an increase in feedwater flow. But consider that if at this moment, the feedwater supply pressure were to drop. The level controller could be opening the valve, yet the falling supply pressure could actually cause a decreased flow through the valve and into the drum.

Thus, it is not enough for the level controller to directly open or close the valve. Rather, it must decide whether it needs more or less feed flow to the boiler drum. The level controller transmits its target flow as a set point to a flow controller. The flow controller then decides how much to open or close the valve as supply pressure swings to meet the set point target.

This is a “2-element” (boiler liquid level to feedwater flow rate) cascade control strategy. By placing this feedwater flow rate in a fast flow control loop, the flow controller will immediately sense any variations in the supply conditions which produce a change in feedwater flow. The flow controller will adjust the boiler feedwater valve position to restore

the flow to its set point before the boiler drum liquid level is even affected. The level controller is the primary controller (sometimes referred to as the master controller) in this cascade, adjusting the set point of the flow controller, which is the secondary controller (sometimes identified as the slave controller).

The third element in a “3-element control” system is the flow of steam leaving the steam drum. The variation in demand from the steam header is the most common disturbance to the boiler level control system in an industrial steam system.

By measuring the steam flow, the magnitude of demand changes can be used as a feed forward signal to the level control system. The feed forward signal can be added into the output of the level controller to adjust the flow control loop set point, or can be added into the output of the flow control loop to directly manipulate the boiler feedwater control valve. The majority of boiler level control systems add the feed forward signal into the level controller output to the secondary (feedwater flow) controller set point. This approach eliminates the need for characterizing the feed forward signal to match the control valve characteristic.

Actual boiler level control schemes do not feed the steam flow signal forward directly. Instead, the difference between the outlet steam flow and the inlet water flow is calculated. The difference value is directly added to the set point signal to the feedwater flow controller. Therefore, if the steam flow out of the boiler is suddenly increased by the start up of a turbine, for example, the set point to the feedwater flow controller is increased by exactly the amount of the measured steam flow increase.

Simple material balance considerations suggest that if the two flow meters are exactly accurate, the flow change produced by the flow control loop will make up exactly enough water to maintain the level without producing a significant upset to the level control loop. Similarly, a sudden drop in steam demand caused by the trip of a significant turbine load will produce an exactly matching drop in feedwater flow to the steam drum without producing any significant disturbance to the boiler steam drum level control.

Of course, there are losses from the boiler that are not measured by the steam production meter. The most common of these are boiler blow down and steam vents (including relief valves) ahead of the steam production meter. In addition, boiler operating conditions that alter the total volume of water in the boiler cannot be corrected by the feed forward control strategy. For example, forced circulation boilers may have steam generating sections that are placed out of service or in service intermittently. The level controller itself must correct for these unmeasured disturbances using the normal feedback control algorithm.

Notes on Firing Control Systems

In general, firing control is accomplished with a Plant Master that monitors the pressure of the main steam header and modulates the firing rate (and hence, the steam production rate) of one or more boilers delivering steam to the steam header. The firing demand signal is sent to all boilers in parallel, but each boiler is provided with a Boiler Master to allow the Plant Master demand signal to be overridden or biased. When the signal is overridden, the steam production rate of the boiler is set manually by the operator, and the boiler is said to be base-loaded. Most boilers on a given header must be allowed to be driven by the Plant Master to maintain pressure control. Boilers that have the Boiler Master set in automatic mode (passing the steam demand from the Plant Master to the boiler firing control system) are said to be swing boilers as opposed to base-loaded

boilers.

The presence of heat recovery steam boilers on a steam header raises new control issues because the steam production rate is primarily controlled by the horsepower demand placed on the gas turbine providing the heat to the boiler. If the heat recovery boiler operates at a pressure above the header pressure, a separate pressure control system can be used to blow off excess steam from the heat recovery boiler when production is above the steam header demand. Note that for maximum efficiency, most heat recovery boilers are fitted with duct burners to provide additional heat to the boiler. The duct burner is controlled with a Boiler Master like any other swing boiler. As long as there are other large swing boilers connected to the steam header, the other fired boilers can reduce firing as required when output increases from the heat recovery boiler.

Dynamic Shrink/Swell and Boiler Level Control

My objective is to continue the discussion we began in the article Cascade, Feed Forward and Boiler Level Control. Here we explore the causes and cures of the dynamic shrink/swell phenomena in boilers.

Boiler Start-up
As high pressure boilers ramp up to operating temperature and pressure, the volume of a given amount of saturated water in the drum can expand by as much as 30%. This natural expansion of the water volume during start-up is not dynamic shrink/swell as discussed later in this article, though it does provide its own unique control challenges.

The expansion (or more precisely, decrease in density) of water during start-up of the boiler poses a problem if a differential pressure or displacer instrument is used for level measurement. Such a level transmitter calibrated for saturated water service at say, 600 psig, will indicate higher than the true level when the drum is filled with relatively cool boiler feedwater at a low start-up pressure.

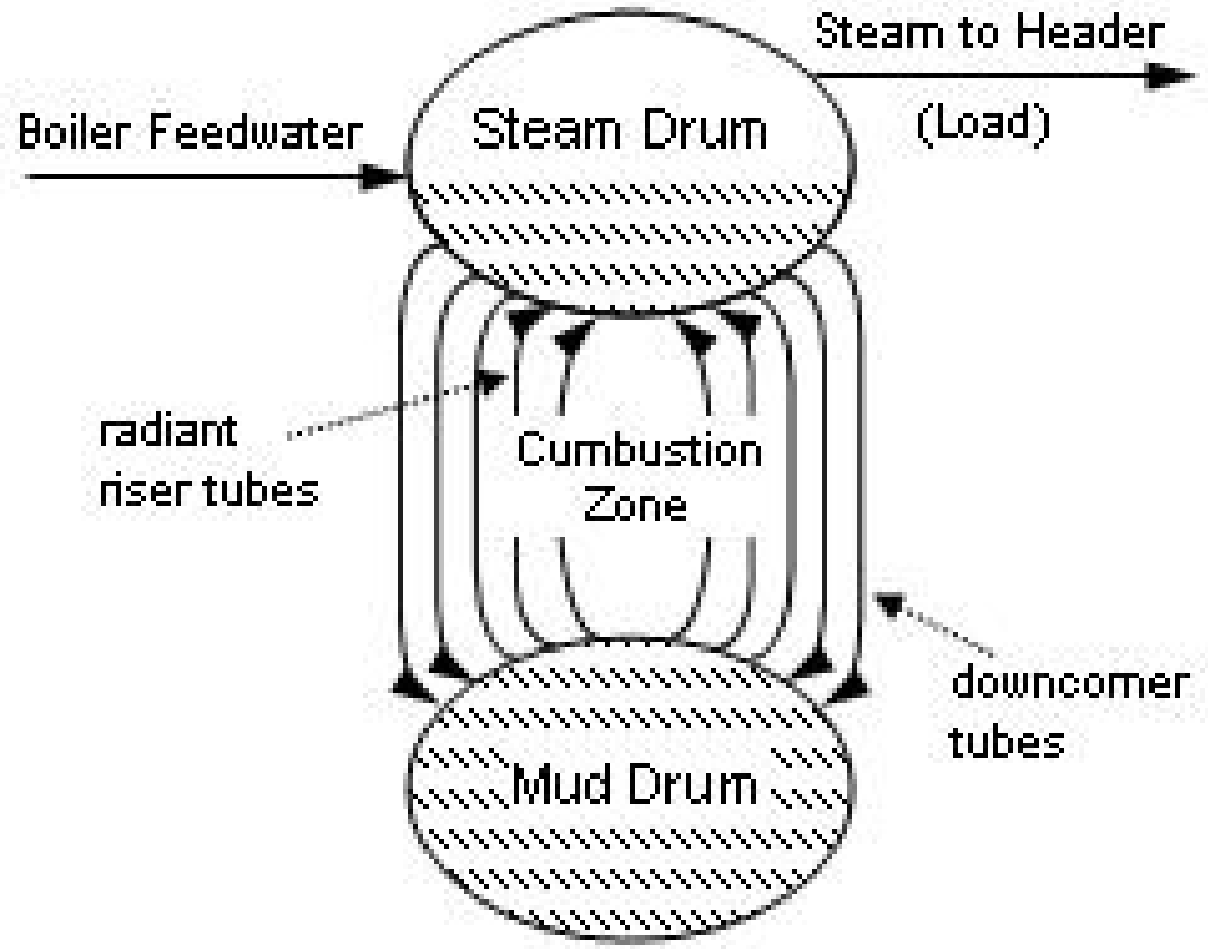
If left uncompensated at low pressure conditions, the “higher than true level” indication will cause the controller to maintain a lower than desired liquid level in the drum during the start-up period. If the low level trip device is actually sensitive to the interface (e.g. conductance probes or float switches), troublesome low level trip events become very likely during start-up.

This variation in the sensitivity of the level transmitter with operating conditions can be corrected by using the drum pressure to compensate for the output of the level transmitter. The compensation can be accomplished with great accuracy using steam table data. The compensation has no dynamic significance and can be used independent of boiler load or operating pressure.

Dynamic Shrink/Swell
Dynamic shrink/swell is a phenomenon that produces variations in the level of the liquid surface in the steam drum whenever boiler load (changes in steam demand) occur. This behavior is strongly influenced by the actual arrangement of steam generating tubes in the

boiler.

I have significant experience with “water wall” boilers that have radiant tubes on three sides of the firebox. There is a steam drum located above the combustion chamber and a mud drum located below the combustion chamber.



During operation, the tubes exposed to the radiant heat from the flame are always producing steam. As the steam rises in the tubes, boiler water is also carried upward and discharged into the steam drum. Tubes that are not producing significant steam flow have a net downward flow of boiler water from the steam drum to the mud drum.

The tubes producing large quantities of steam are termed risers and those principally carrying water down to the mud drum from the steam drum are termed downcomers. Excluding the tubes subject to radiant heat input from the firebox flame, a given tube will serve as a riser at some firing rates and a downcomer at other firing rates.

The mechanics of the natural convection circulation of boiler water within the steam generator is the origin of the dynamic shrink/swell phenomenon. Consider what happens to a boiler operating at steady state at 600 psig when it is subjected to a sudden increase in load (or steam demand).

A sudden steam load increase will naturally produce a drop in the pressure in the steam

drum, because, initially at least, the firing rate cannot increase fast enough to match the steam production rate at the new demand level. When the pressure in the drum drops, it has a dramatic effect on the natural convection within the boiler. The drop in pressure causes a small fraction of the saturated water in the boiler to immediately vaporize, producing a large amount of boil-up from most of the tubes in the boiler. During the transient, most of the tubes temporarily become risers. The result is that the level in the steam drum above the combustion chamber rises.

However, this rise in level is actually an inverse response to the load change. Since, the net steam draw rate has gone up, the net flow of water to the boiler needs to increase, because the total mass of water in the boiler is falling. However, the level controller senses a rise in the level of the steam drum and calls for a reduction in the flow of feedwater to the boiler.

This inverse response to a sudden load increase is dynamic swell. Dynamic shrink is also observed when a sudden load decrease occurs. However, the dynamic shrink phenomenon does not disrupt the natural convection circulation of the boiler as completely as the dynamic swell effect. Consequently, the reduction in level produced by a sudden decrease in load is typically much smaller and of shorter duration than the effect produced by dynamic swell.

Control Strategy for Shrink/Swell

What control strategies are used to deal with this unpleasant inverse system response? The basic three-element control system we have previously discussed in the article Cascade, Feed Forward and Boiler Level Control provides the most important tool.

When a sudden load (steam demand) increase occurs, the feed forward portion of the strategy will produce an increase in the set point for the feedwater flow controller. This increase in feedwater flow controller set point will be countered to varying degrees by the level controller response to the temporary rise in level produced by the dynamic swell.

The standard tool used to minimize the impact of the swell phenomenon on the level in a three-element level control system is the lead-lag relay in the feed forward signal from the flow difference relay. This is the traditional means of dealing with mismatched disturbance and manipulated variable dynamics in feed forward systems, and is certainly applicable in this control strategy. When used in the three-element level control strategy, the lead-lag relay is commonly termed the “shrink/swell relay.”

There are two significant limitations to the use of the lead-lag relay for shrink/swell compensation. To begin with, the response of most boilers to a load increase (swell event) is much more dramatic than the response to a load decrease (shrink event). In other words, the system response is very asymmetric. The lead-lag relay is perfectly symmetrical in responding to load changes in each direction and cannot be well matched to both directions.

Furthermore, the standard method of establishing the magnitudes of the lead time constant and lag time constant involves open loop tests of the process response to the disturbance (steam load) and to the manipulated variable (feedwater flow). A step test of the manipulated variable is generally not too difficult to conduct. However, changing the firing rate upward fast enough to actually produce significant swell is difficult without seriously upsetting the steam system, an event that is to be avoided in most operating

plants. Therefore, the practitioner's only choice is to gather accurate data continuously and wait for a disturbance event that will exercise the shrink/swell relay's function.

When a lead-lag relay is to be added to an existing three-element boiler control scheme, operator knowledge of the boiler behavior in sudden load increase situations can guide the initial settings. For example, if the operators indicate that they must manually lead the feedwater valve by opening it faster than the control system will open it automatically, it is clear that a lead time constant larger than the lag time is required. Similarly, if the operator must retard the valve response to prevent excessively high level, the lead time constant must be less than the lag time. The lag time constant will typically fall in the range of one minute to three minutes. The ratio of the lead time constant to the lag time constant determines the magnitude of the initial response to the disturbance. If the ratio is one to one, the system behaves the same as a system with no lead-lag relay.

Ultimately, the system must be adjusted by watching the response to actual steam system upsets that require sudden firing increases. If the initial observed response of level to an upset is a rising level, the ratio of lead time to lag time should be decreased. The inverse is similarly true. If the recovery from an initial rise in level is followed by significant overshoot below the level target, the lag time should be reduced. If the recovery from an initial level drop is followed by a large overshoot above the level target, the lag time should be increased.

VI. Process Applications in Control

19) Distillation Column Control

Distillation: Introduction to Control

Background
Approximately 40,000 distillation columns are operated in the U.S. chemical process industries and they comprise 95% of the separation processes for these industries. Because distillation operation directly affects product quality, process production rates and utility usage, the economic importance of distillation control is clear. Distillation control is a challenging problem because of the following factors:

- Process nonlinearity
- Multivariable coupling
- Severe disturbances
- Nonstationary behavior

Distillation columns exhibit static nonlinearity because impurity levels asymptotically approach zero. The impurity level in the overhead product is the concentration of the heavy key, and the impurity level in the bottoms product is the concentration of the light key. Nonlinear dynamics, i.e., variations in time constants with the size and direction of an input change, and static nonlinearity are much more pronounced for columns that produce high-purity products, e.g., columns that have impurity levels less than 1%.

Coupling is significant when the composition of both overhead and bottoms products are being controlled. Columns are affected by a variety of disturbances, particularly feed composition and flow upsets. Nonstationary behavior stems from changes in tray efficiencies caused by entertainment or fouling.

Improved distillation control is characterized by a reduction in the variability of the impurities in the products. Meeting the specification requirements on the variability of final products can make the difference between the product being a high value-added product with large market demand and being a low-valued product with a small market demand.

For customers who purchase the products produced by distillation columns as feedstock for their processes, the variability of the feedstock can directly affect the quality of the products they produce, e.g., the variability in the monomer feed to a polymerization process can directly affect the mechanical properties of the resulting polymer produced.

In addition, control performance can affect plant processing rates and utility usage. After the variability of a product has been reduced, the set point for the impurity in the product can be increased, moving the set point closer to the specification limit. If this column is the bottleneck for the process, then increasing the average impurity level, i.e., moving the impurity set point closer to the specification limit, allows greater plant processing rates.

Even if the column in question is not a bottleneck, moving the impurity set point closer to the specification limit reduces the utility usage for the column. While each of these factors

can be economically important for large-scale processes, the order of economic importance is usually product quality first, followed by process throughput and finally utility reductions.

Column Schematic
A schematic of a binary distillation column with one feed and two products is shown in Figure 1:

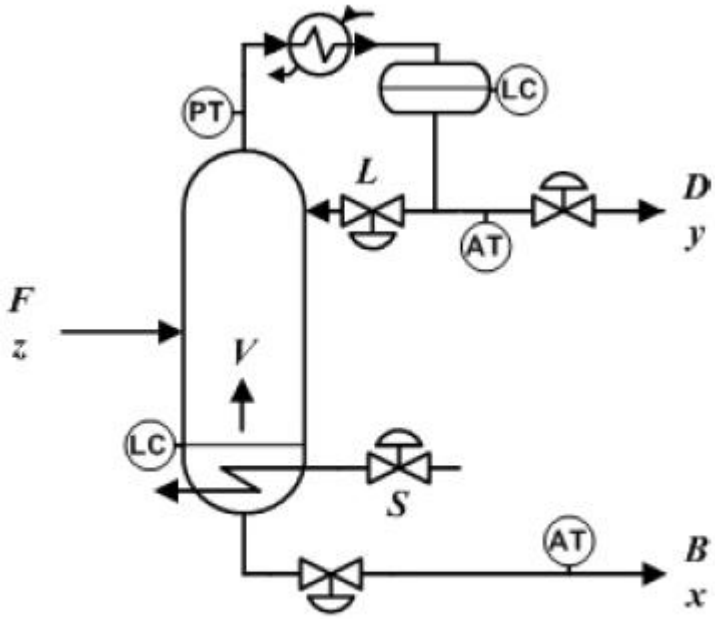


Figure 1 Schematic of a two product distillation column. Note that the valves on this figure represent flow control loops.

Material balance and energy balance effects
Combining an overall steady-state material balance with the light component material balance for a binary separation yields:

$$\frac{D}{F} = \frac{z-x}{y-x}$$

Rearranging results in:

$$y = x + \frac{z-x}{D/F}$$

This equation indicates that as the flow rate of the distillate product, D, decreases while keeping F, z and x constant, the purity of the overhead product, y, increases. Likewise, as D increases, its purity decreases.

Because the sum of the product flows must equal the feed rate at steady state, when one product becomes more pure, the other product must get less pure. This is shown graphically in Figure 2. This is an example of the material balance effect in which the product impurity level varies directly with the flow rate of the corresponding product.

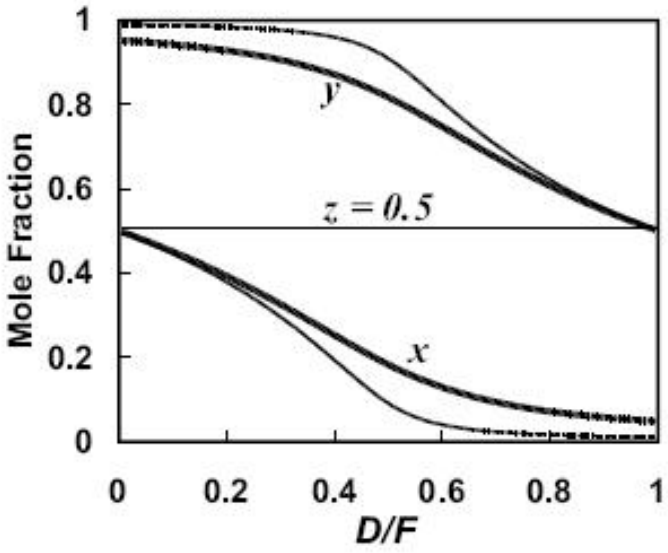


Figure 2 Effect of *D/F* and energy input on product purities. The thin line corresponds to a case in which the energy input is increased.

Another key factor that affects product purities is the energy input to the column, which determines the vapor rate, V, up the column. As the energy input to the column increases, the separation of the light and heavy components usually increases (Figure 2). One measure of the separation is the separation factor, S, which is given by

$$S = \frac{y}{1-y}\frac{1-x}{x}$$

As the impurity levels in the products decrease (i.e., y□1 and/or x□0), S increases.

Another way to understand the effect of an increase in energy input to the column is to consider the vapor/liquid traffic inside the column. If V increases while D and B are kept constant, the reflux, L, increases by the same amount as V. As a result, the reflux ratio, L/D, increases. This increase in vapor/liquid traffic inside the column causes a decrease in the impurities in the products for the same D/F ratio (Figure 2). When evaluating how a column responds in a control situation, it is helpful to remember that *the energy input to the column generally determines the degree of separation that the column can achieve while the material balance (i.e., D/F) determines how the separation is allocated between the two products.*

Vapor and Liquid Dynamics

The difference between vapor and liquid dynamics in a distillation column can contribute to interesting composition dynamic behavior. For all but very-high-pressure columns, i.e., those operating near the critical pressure of the light key, a change in V in the reboiler can be observed in the overhead in just a few seconds while a change in the reflux flow rate requires several minutes to reach the reboiler.

The hydraulic response of a tray depends on the accumulation or depletion of liquid on it. The hydraulic time constant for flow from a tray typically ranges between 3 and 10

seconds. As a result, for a column with 50 or more trays, the overall hydraulic response time is on the order of several minutes.

As an example of the effect of the difference between liquid and vapor dynamics, consider the overhead product purity for an increase in V for a column in which the accumulator level sets the reflux flow rate and the distillate rate is held constant. Initially, the increase in vapor flow moves up the column rapidly while the liquid flow down the column remains relatively constant because the reflux rate is set by the level controller on the accumulator.

In the rectifying section, the L/V ratio determines the separating power of that section. As a result of the increase in V, the concentration of the impurity in the overhead increases initially. The increase in V begins to increase the level in the accumulator, which, after some time, leads to an increase in the reflux flow. As the increased reflux flow slowly makes its way down the rectifying section, L/V increases, causing a decrease in the impurity level in the overhead product. Therefore, for this scenario, an increase in V results in an inverse response in the concentration of the overhead product due to the difference in vapor and liquid dynamic in the column.

Entrainment

For columns operating at pressures less than about 165 psia, as V increases above 80% of flood conditions, droplets of liquid from the active area of the tray are blown in the vapor to the tray above, thus reducing the separation efficiency of the column. For certain vacuum columns, the tray efficiency can drop as much as 30% as the boilup rate increases above 80% of flood. 100% of flood corresponds to the condition for which an increase in vapor rate results in a decrease in separation for the column.

As the tray efficiency decreases because of increased entrainment, the process gain decreases, requiring larger changes in the manipulated variables to obtain the same change in the product impurity levels.

Structure Packed Columns

Columns that use sections of structured packing offer significant efficiency advantages over trayed columns for low-pressure applications because there is less pressure drop across the structured packing than across a corresponding set of trays. Because of the low liquid holdup on structured packing, these columns have faster composition dynamics than trayed columns.

The liquid holdup on the structured packing is low enough that the composition profile through the packing reaches its steady-state profile much more quickly than the reboiler and accumulator. For a column with structured packing, the dynamic lag of the accumulator and the reboiler primarily determine the dynamic response of the product compositions.

More on Distillation Control

This is the first of a series on distillation control. The next article presents the major disturbances affecting composition control and the importance of properly functioning regulatory controls.

_____

1. This material is reprinted from *Chemical Process Control, 2nd Ed.* with the permission of the publisher: Ferret Publishing (806 747 3872).

Distillation: Major Disturbances & First-Level Control

In the first article of this series, discussed the importance and challenges associated with distillation control and the control relevant issues of distillation operations. Here he continues the discussion by presenting the major disturbances affecting composition control and the importance of properly functioning regulatory controls. Please refer to the previous article for terminology and variable definitions.

Disturbances

The type and magnitude of disturbances affecting a distillation column have a direct effect on the resulting product variability. An analysis of the major types of disturbances encountered in distillation columns follows.

• *Feed composition upsets*

Changes in the feed composition represent the most significant upsets with which a distillation control system must deal on a continuous basis. A feed composition change shifts the composition profile through the column resulting in a large upset in the product compositions. Most industrial columns do not have a feed composition analyzer; therefore, feed composition upsets usually appear as unmeasured disturbances. When a feed composition analyzer is available, a feedforward controller can be applied using the on-line measurements of the feed composition. Because feed composition changes represent a major disturbance for distillation control, the sensitivity of potential control configurations to feed composition upsets is a major issue for configuration selection.

• *Feed flow rate upsets*

The flow rates in a steady-state model of a column with constant tray efficiencies scale directly with column feed rate. Therefore, ratio control (using L/F, D/F, V/F or B/F as composition controller output) is an effective means of handling feed flow rate upsets. Dynamic compensation is normally required to account for the dynamic mismatch between the response of the product compositions to feed flow rate changes and the response to changes in the MVs. When certain ratios (e.g., L/D, V/B) are used as MVs, these ratios, combined with the level control, automatically compensate for feed flow rate changes.

• *Feed enthalpy upsets*

For columns that use a low reflux ratio, feed enthalpy changes can significantly alter the vapor/liquid rates inside the column, causing a major shift in the internal composition profile and, therefore, a significant upset in the product compositions. This upset can be difficult to identify because (1) most industrial columns do not have feed temperature measurements and (2) even if a feed temperature measurement is available, it does not detect feed enthalpy changes for a two-phase feed. This disturbance may be difficult to distinguish from feed composition upsets without a more detailed analysis. It may be necessary to install a feed preheater or cooler to maintain a constant feed enthalpy to a column.

• *Subcooled reflux changes*

When a thundershower passes over a plant, the reflux temperatures for the columns can drop sharply. Columns that use finned-fan coolers as overhead condensers are particularly susceptible to rapid changes in ambient conditions. If internal reflux control is not applied, severe upsets in the operation of the columns result because of major shifts in the composition profiles of the columns. When internal reflux control is correctly applied,

the impact of a thunderstorm on column operations can be effectively eliminated.

• *Loss of reboiler steam pressure*
When a steep drop in steam header pressure occurs, certain columns (those operating with control valves on the reboiler steam that are nearly fully open) experience a sharp drop in reboiler duty. This results in a sharp increase in the impurity levels in the products. When the steam header pressure returns to its normal level, the composition control system for the column attempts to return to the normal product purities. Because of the severity of this upset, if the composition controllers are not properly tuned, the upset can be amplified by the composition controllers, requiring the operators to take these controllers off-line to stabilize the column, greatly extending the duration of the period of production of off-specification products. This disturbance is, in general, the most severe disturbance that a control system on a distillation column must handle and may require invoking overrides that gradually bring the operation of the column to its normal operating window instead of expecting the composition controllers to handle this severe upset by themselves.

• *Column pressure upsets*
Column pressure has a direct effect on the relative volatility of the key components in the column. Thus, changes in the column pressure can significantly affect product compositions. A properly implemented pressure control scheme maintains column pressure close to its set point, with only short-term and low-amplitude departures. A large class of columns (e.g., refinery columns) is operated at maximum condenser duty to maximize column separation, which minimizes steam usage. For these cases, the column pressure increases during the day, when the cooling water or ambient air temperature is the greatest, and decreases at night, but the resulting pressure changes are usually slow enough that the composition controller can efficiently reject this disturbance.

Regulatory Controls
Improperly functioning flow, level or pressure controllers can undermine the effectiveness of the product composition controllers.

• *Flow controllers*
Flow controllers are used to control the flow rates of the products, the reflux and the heating medium used in the reboiler and their set points are determined by the various level and composition controllers. To assess the performance of a flow control loop, you can applying block sine waves and comparing these results for the dead band and time constant with the expected performance levels.

• *Level controllers*
Level controllers are used to maintain the level in the accumulator, the reboiler and the intermediate accumulator of a stacked column (i.e., a distillation column composed of two separate columns when there are too many trays for one column). Loose level control on the accumulator and reboiler has been shown to worsen the composition control problem for material balance control configurations (when either D or B is used as a MV for composition control). When D or B is adjusted, the internal vapor/liquid traffic changes only after the corresponding level controller acts as a result of the change in D or B. On the other hand, if a level controller is tuned too aggressively, it can result in oscillations passed back to the column and contribute to erratic operation. When the reboiler duty is set by the level controller on the reboiler, a level controller that causes oscillation in the

reboiler can also cause cycling in the column pressure.

• *Column pressure controllers*
The column overhead pressure acts as an integrator and is determined by the net accumulation of material in the vapor phase. Column pressure is controlled by directly changing the amount of material in the vapor phase of the overhead or by changing the rate of condensation of the overhead, which converts low-density vapor to high-density liquid. A variety of approaches can be used to control column pressure including:

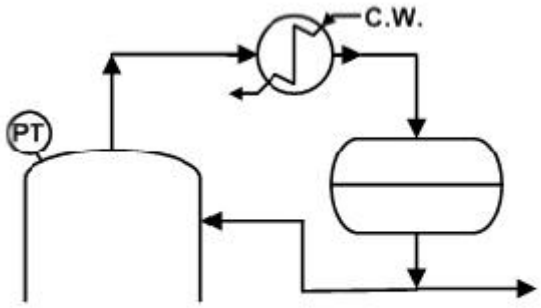


Figure 3 Minimum column pressure operation using maximum cooling water flow rate.

1. using the maximum cooling water flow rate and allowing the column pressure to float at the minimum pressure level (Figure 3).

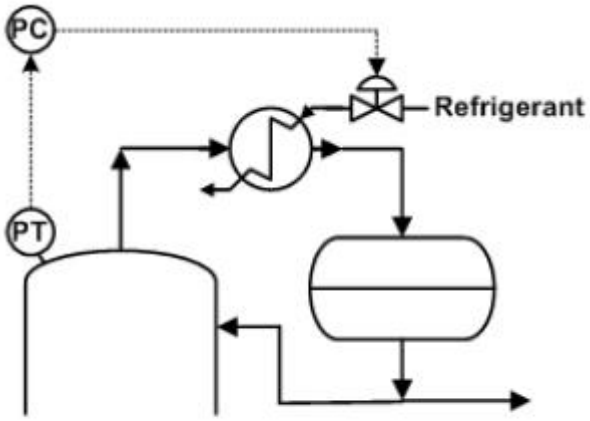


Figure 4 Column pressure control configuration using refrigerant flow as the MV.

2. adjusting the flow rate of a refrigerant to the condenser (Figure 4).

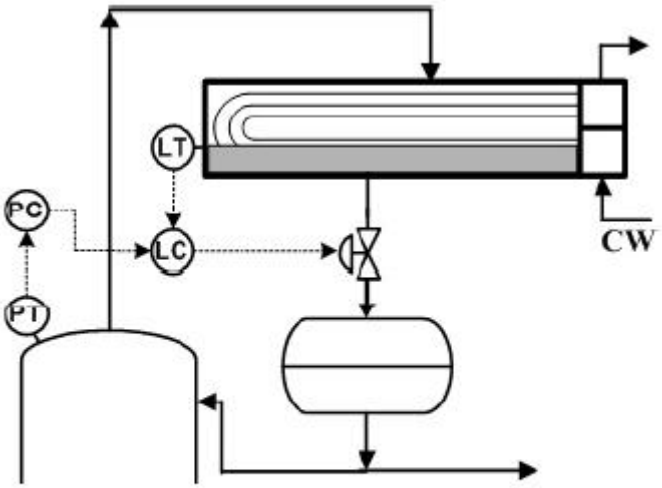


Figure 5 Column pressure configuration using the setpoint for the liquid level in the condenser as the MV for the pressure controller.

3. adjusting the level of liquid in the condenser to change the effective heat-transfer area (Figure 5).

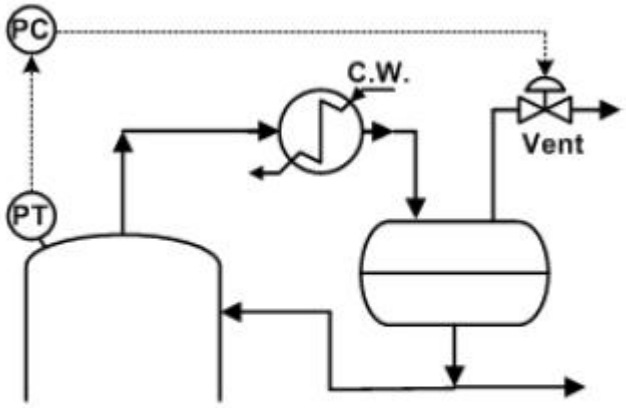


Figure 6 Column pressure control configuration using vent flow as the MV.

4. venting vapor from the accumulator (Figure 6).

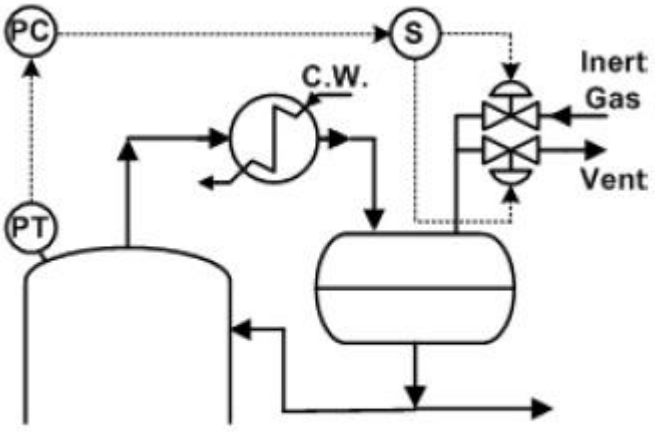


Figure 7 Column pressure control configuration using vent flow or inert gas injection as the MV.

5. venting vapor from or injecting inert gas into the vapor space in the accumulator (Figure 7).

Note that approaches 1 - 3 directly affect the rate of vapor condensation to control pressure while approaches 4 and 5 directly adjust the amount of vapor in the overhead of the column for pressure control.

The fastest-responding pressure control configurations (i.e., the approaches that should provide the tightest control to set point) are vent flow (Figure 6) and vent flow or inert injection (Figure 7).

The override/select controller in Figure 7 uses vent flow when the measured pressure is above set point and uses the injection of an inert gas when the pressure is below set point.

The speed of the pressure control loops based on manipulating the flow of a refrigerant (Figure 4) and adjusting the effective heat-transfer area (Figure 5) respond considerably more slowly because both of these approaches make changes in the rate of heat transfer to change the column pressure.

Operating at minimum column pressure (Figure 3) allows the column pressure to swing with the maximum pressure normally occurring during the afternoon and the minimum pressure occurring early in the morning.

More on Distillation Control

The application of the best high-level approach to distillation control (e.g., model predictive control) will generally be ineffective if the process is not thoroughly understood and the regulatory controls are not implemented properly.

The next article in this series on distillation control discusses the use of product composition measurements in distillation column control and explores single composition control strategies.

Distillation: Inferential Temperature Control & Single-Ended Control

In the first article of this series, discussed control relevant issues associated with distillation columns. The second article presented the major disturbances affecting composition control and the importance of properly functioning regulatory controls.

In this article, discusses the use of product composition measurements in distillation column control. He also explores single composition strategies where either the top or bottom product composition is controlled while the other is allowed to float.

Product Composition Measurements

Product impurity levels (measured either on-line or in the laboratory) are used by feedback controllers to adjust column operation to meet product specifications. In addition, tray temperatures are used to infer product compositions, where feasible.

On-line analyzers

There is a range of on-line analyzers commonly used in the chemical process industries (e.g., gas chromatographs). A key issue with analyzers is their associated analyzer delay.

For columns that have slow-responding composition dynamics, analyzer delay is usually less of an issue. For a propylene/propane splitter (i.e., a high reflux ratio column), the composition dynamics for the primary product have a time constant of about 2 hours. As the cycle time for the analyzer increases from 5 minutes to 10 minutes, it does not significantly affect the feedback control performance.

For an ethane/propane splitter (i.e., a low reflux ratio column), even a five-minute analyzer delay seriously undermines composition control performance because the time constant for this process is less than five minutes. Fortunately, most fast-acting columns have a significant temperature drop across them so that product composition can be effectively inferred from tray temperature measurements.

For columns that have a low relative volatility (i.e., less than 1.4), inferential temperature control is not feasible and feedback based on an on-line analyzer is required, but these columns generally have slow composition dynamics compared to the analyzer delay. For columns with a high relative volatility (i.e., greater than 2.0), inferential temperature control is usually effective.

Inferential temperature control

Inferential temperature control is an effective means of maintaining composition control for columns from a control and economic standpoint. That is, it greatly reduces the analyzer dead time for feedback control and is much less expensive to install and maintain than an on-line composition analyzer. Because of their superior repeatability, RTDs (resistance temperature detectors) and thermistors are usually used for inferential temperature applications.

For multicomponent separations, tray temperatures do not uniquely determine the product composition. As a result, for these cases it is essential that an on-line analyzer or, at least, periodic laboratory analysis be used to adjust the tray temperature set point to the proper

level.

If feedback based on laboratory analysis is not used, offset between the desired product composition and the actual product composition can result. Column pressure also significantly affects tray temperature measurements. For most systems, a simple linear correction compensates for variations in column pressure:

$$T_{pc} = T_{meas} - K_{pr}(P - P_0)$$

where T_{pc} is the pressure-compensated temperature that should be used for feedback control, T_{meas} is the measured tray temperature, K_{pr} is pressure correction factor, P is the operating pressure and P_0 is the reference pressure.

K_{pr} can be estimated by applying a steady-state column simulator for two different pressures within the normal operating range and using the following equation:

$$K_{pr} = \frac{T_i(P_1) - T_i(P_2)}{P_1 - P_2}$$

where T_i is the value of the tray temperature for tray i predicted by the column simulator.

A steady-state column model can also be used to determine the best tray locations for inferential control by finding the trays whose temperatures show the strongest correlation with product composition. The following procedure is recommended:

1. Apply the steady-state column model at the base-case conditions (i.e., x^{BC} and y^{BC}) and record the temperature of each tray as T_i^{BC}.

2. Run the column model for an increase in the impurity level in the bottoms product (i.e., $x^{BC} + \Delta x$, y^{BC}) and record the tray temperatures as $T_i^{\Delta x}$. Δx should be approximately 25-50% of the impurity level at the base-case conditions.

3. Run the column model for an increase in the impurity level in the overhead product (i.e., x^{BC}, $y^{BC} + \Delta y$) and record the tray temperatures as $T_i^{\Delta y}$. Δy should be approximately 25-50% of the impurity level at the base-case conditions.

4. To locate the best tray for inferential control for the stripping section, determine tray number i that maximizes

$$\Delta T_i^{net} = \left| T_i^{\Delta x} - T_i^{BC} \right| - \left| T_i^{\Delta y} - T_i^{BC} \right|$$

Because the proper tray should not be particularly sensitive to variations in the composition of the overhead product, the effect of Δy is subtracted from the temperature difference caused by changes in the bottom product composition.

5. To locate the best tray for inferential control for the rectifying section, determine the tray number i that maximizes

$$\Delta T_i^{net} = \left| T_i^{\Delta y} - T_i^{BC} \right| - \left| T_i^{\Delta x} - T_i^{BC} \right|$$

6. Finally, repeat this procedure for a representative range of feed compositions. Then, select the tray that provides the best overall results. Remember that it may not be possible to locate the tray temperature measurement precisely at the optimum tray location.

Table 1 () shows an example of this approach used to locate the best tray temperature for inferential temperature control in the stripping section of a depropanizer.

Table 1

Example for Choosing the Proper Tray Location for Inferential Temperature Control

	BC	Δx	Δy	
x	0.04	0.02	0.04	
y	0.03	0.03	0.15	
	T_i^{BC}	$T_i^{\Delta x}$	$T_i^{\Delta y}$	ΔT_i^{NET}
T_1	112.4	114.1	111.8	1.1
T_2	105.1	107.5	104.6	1.9
T_3	102.8	105.6	102.2	2.3
T_4	100.8	103.9	100.2	2.5
T_5	99.0	102.4	98.5	2.9
T_6	97.3	101.1	96.8	3.3
T_7	95.6	99.7	95.0	3.5
T_8	94.0	98.4	93.6	4.0
T_9	92.4	97.1	91.7	**4.2**
T_{10}	90.9	95.8	90.2	**4.2**
T_{11}	89.4	94.4	88.6	**4.2**
T_{12}	87.4	93.0	85.6	**4.2**
T_{13}	86.5	91.6	87.0	**4.2**
T_{14}	85.1	90.0	85.6	4.0
T_{15}	83.6	88.4	84.6	3.8
T_{16}	82.2	86.7	81.2	3.5
T_{17}	80.7	84.9	79.7	3.2
T_{18}	79.2	83.0	78.1	2.7

Note that a temperature measurement anywhere between tray 7 and tray 16 should work well for this application for the assumed feed composition.

For certain columns, the bulk of the temperature change occurs in a few trays, resulting in

Distillation: Dual Composition Control & Constraint Control

In the first article of this series, discussed control relevant issues associated with distillation columns. The second article presented the major disturbances affecting composition control and the importance of properly functioning regulatory controls. The third article discussed the use of product composition measurements in distillation column control and explored single composition control strategies.

This final segment of the series considers advanced issues including dual composition control and constraint control.

Dual Composition Control

The choice of the proper configuration for dual composition control is a more challenging problem than for single composition control because there are more viable approaches and the analysis of performance is more complex.

There is a variety of choices for the manipulated variables, or MVs, including L, D, L/D, V, B, V/B, B/L, and D/V, that can be paired to the four control objectives (x, y, reboiler level and accumulator level). As a result, there are a large number of possible configuration choices although most of them are not practical.

It is assumed that the choice for the control configuration for the column pressure (Figures 3-7) is made separately from the selection of the composition control configuration.

If we limit our choices to L, D and L/D for controlling y and V, B and V/B for controlling x, there are nine possible configurations to consider: (L,V), (D,V), (L/D,V), (L,B), (D,B), (L/D,B), (L,V/B), (D,V/B) and (L/D,V/B). In each configuration, the first term is the MV used to control y and the second term is used to control x.

Figure 10 shows the (L,V) configuration. The set point for the reflux flow controller is set by the overhead composition controller and the set point for the flow controller on the reboiler duty is set by the bottom composition controller. This leaves D to control the accumulator level and B to control the reboiler level.

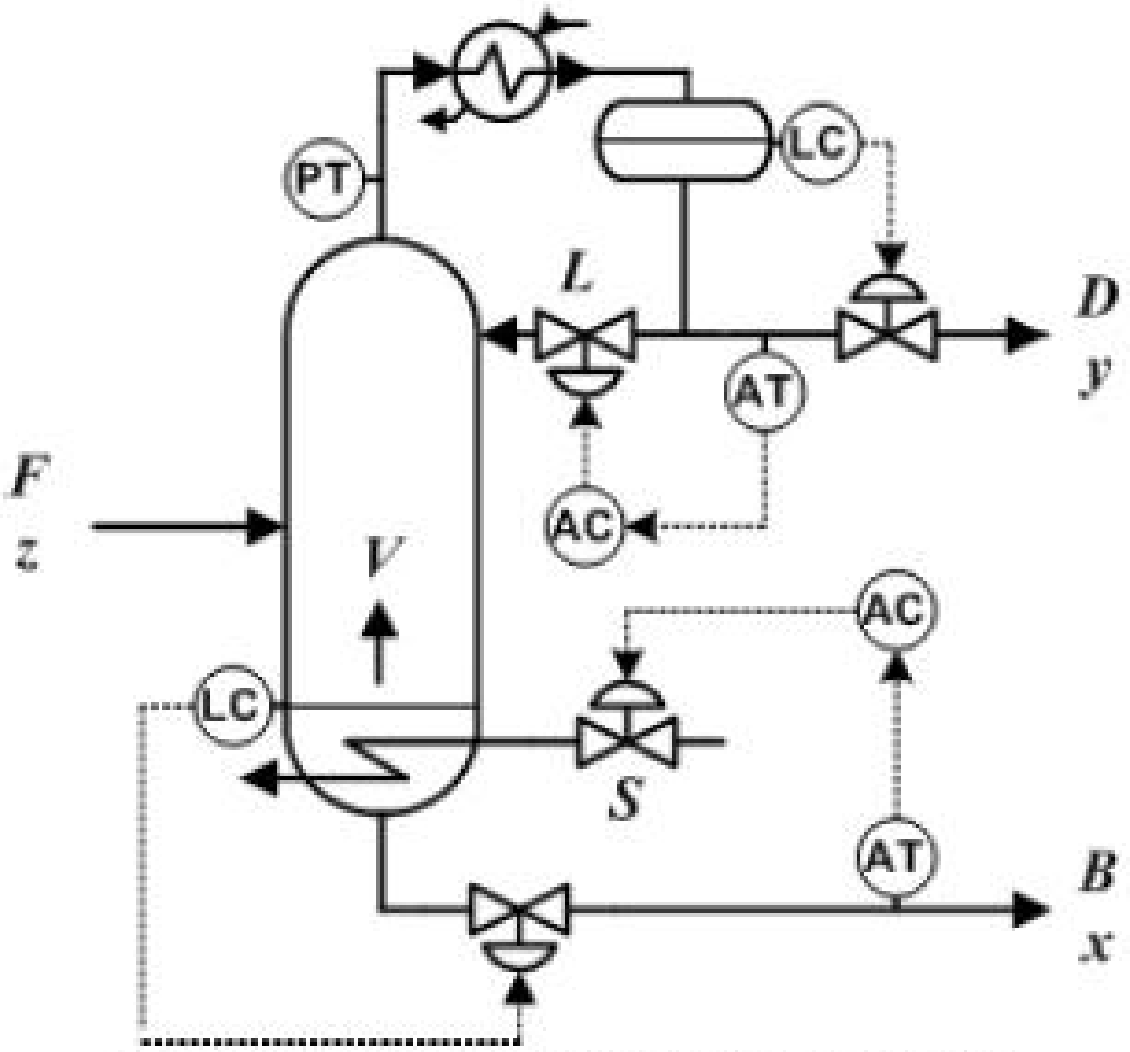


Figure 10 Control diagram for the (*L*,*V*) configuration for dual composition control. The symbol for a control valve represents a flow control loop.

Figure 11 shows the (D,B) configuration where D is adjusted to control y and B is changed to control x, which leaves L for the accumulator level and V for the reboiler level.

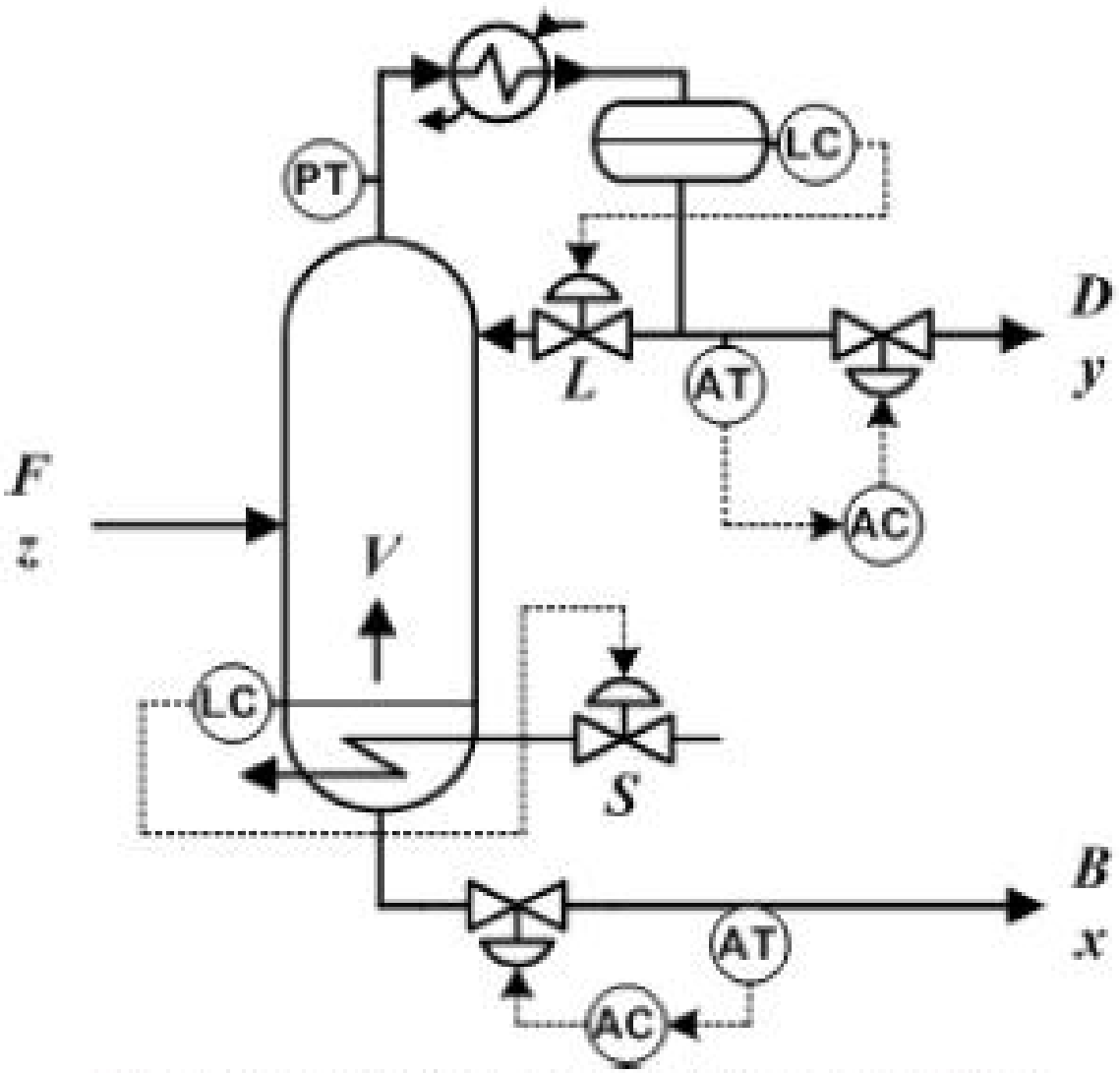


Figure 11 Control diagram for the (*D*,*B*) configuration for dual composition control. The symbol for a control valve represents a flow control loop.

Consider the classifications of these nine control configurations. The five configurations that use either D or B as a MV for composition control are referred to as “material balance configurations” because they use the overall material balance for the column to adjust product compositions. In fact, the (D,B) configuration has been referred to as the super material balance configuration.

The four configurations that do not use D or B as MVs are known as “energy balance configurations” because they directly adjust the vapor/liquid traffic in the column for composition control. The (L/D,V/B) configuration is known as the “double ratio configuration.”

The major factors affecting the composition control performance of a particular configuration are coupling, sensitivity to disturbances and the response time for changes in the MV. The most commonly used configuration is the (L,V) configuration because it

provides good dynamic response, is the least sensitive to feed composition disturbances and is the easiest to implement, even though it is highly susceptible to coupling.

On the other hand, the (L/D,V/B) configuration is, in general, the least affected by coupling and has good dynamic response, but is quite sensitive to feed composition disturbances and is more difficult to implement. The (D,B) configuration has advantages for certain high-purity columns if the levels are tuned tightly, but is open-loop unstable, i.e., non-self-regulating.

As a result, there is no clear choice for the best configuration for dual composition control of distillation columns. In fact, there are specific cases for which each of the nine potential configurations listed earlier provides the best control performance.

While it is not possible to *a priori* choose the optimum configuration, there are some guidelines that can reduce the possibility of choosing a poor configuration for a particular column. In general, for high reflux ratio cases (L/D > 8), configurations that use material balance MVs (D and B) or ratios (L/D and V/B) are preferred while for low reflux ratio cases (L/D < 5), configurations that use energy balance MVs (L and V) or ratios are preferred.

In many cases, the control of one of the two products is more important than control of the other. For these cases, when the overhead product is more important, L is usually the best MV. When the bottoms product is more important, V is the proper MV. If the column is a high reflux ratio column, the MV for the less important product should be a material balance knob (D or B) or a ratio (L/D or V/B).

Because a C3 splitter is a high reflux ratio column and control of the overhead product is more important, the (L,B) or (L,V/B) configuration is preferred, which is consistent with simulation studies that have been performed. If the column is a low reflux ratio column, the less important product should be controlled by an energy balance knob (V or L) or a ratio (L/D or V/B). For example, for a low reflux column for which the bottom product is more important, the (L,V) or (L/D,V) configuration is preferred.

Table 2 summarizes the recommended control configurations for columns in which one product is more important than the other.

Table 2

Recommended Control Configurations When One Product is More Important than the Other

	x more important	*y* more important
Low reflux ratio columns ($L/D < 5$)	$(L/D,V)$; (L,V)	$(L,V/B)$; (L,V)
High reflux ratio columns ($L/D > 8$)	$(L/D,V)$; (D,V)	$(L,V/B)$; (L,B)

Composition Controller Tuning

For most distillation columns, the effective dead-time-to-time constant ratio is relatively small. Therefore, derivative action is not necessary and PI composition controllers are

commonly used. When inferential temperature control is used, fast-acting temperature loops with significant sensor lag may require derivative action because of their effective dead time-to-time constant ratios. Because most composition and temperature control loops are relatively slow responding, ATV identification with on-line tuning is recommended.

When one product is more important than the other, it is best to first tune the less important loop loosely (i.e., less aggressively tuned, e.g., critically damped) and then tune the important loop to control to set point tightly (e.g., a 1/6 decay ratio). This dynamically decouples the multivariable control problem by providing relatively fast closed-loop dynamics for the important loop and considerably slower closed-loop dynamics for the less important loop.

In this manner, the coupling effects of the less important loop are slow enough that the important loop can easily absorb them. As a result, the variability in the important loop can be maintained consistently at a relatively low level. In effect, this approach approximates the performance of single composition control without allowing the less important product composition to suffer large offsets from set point.

When the importance of the control of both products is approximately equal, both loops need to be tuned together. In this case, both loops must be detuned equally to the point where the effects of coupling are at an acceptable level.

Constraint Control
Some of the most common column constraints include:

Maximum reboiler duty
This constraint can result from (1) an increase in the column pressure that reduces the temperature difference for heat transfer in the reboiler; (2) fouled or plugged heat-exchanger tubes in the reboiler that reduce the maximum heat transfer-rate; (3) an improperly sized steam trap that causes condensate to back up into the reboiler tubes; (4) an improperly sized control valve on the steam to the reboiler that limits the maximum steam flow to the reboiler; or (5) an increase in the column feed rate such that the required reboiler duty exceeds the maximum duty of the reboiler.

Maximum condenser duty
This constraint can be due to (1) an increase in the ambient air temperature that decreases the temperature difference for heat transfer; (2) fouled or plugged tubes in the condenser that reduce its maximum heat duty; (3) an improperly sized coolant flow control valve; (4) an increase in coolant temperature; or (5) an increase in column feed rate such that the required condenser duty exceeds the maximum duty of the condenser.

Flooding
Kister discusses in detail the three types of flooding, showing that each type results from excessive levels of vapor/liquid traffic in the column.

Weeping
Weeping results when the vapor flow rate is too low to keep the liquid from draining through a tray onto the tray below.

Maximum reboiler temperature

For certain systems, elevated temperatures in the reboiler can promote polymerization reactions to the level that excessive fouling of the reboiler results. A reboiler duty constraint can be identified when the steam flow control valve remains fully open and the requested steam flow is consistently above the measured flow rate.

Condenser duty constraints are usually identified when the column pressure reaches a certain level or when the reflux temperature rises to a certain value. The onset of flooding or weeping is generally correlated to the pressure drop across the column or across a portion of the column. It should be emphasized that it is the reboiler duty that is adjusted to honor each of these constraints, i.e., prevent the process from violating the constraints.

Three approaches can be used for constraint control of a distillation column:
1. Convert from dual composition control to single composition control.
2. Reduce the column feed rate to maintain the purity of the products.
3. Reduce the product impurity setpoints for both products.

Figure 12 shows how a low select can be used to switch between constraint control and unconstrained control for a maximum reboiler temperature constraint when the overhead is the more important product. When the bottom product is more important, the combined constrained and unconstrained configuration is more complicated, with more potential configuration choices.

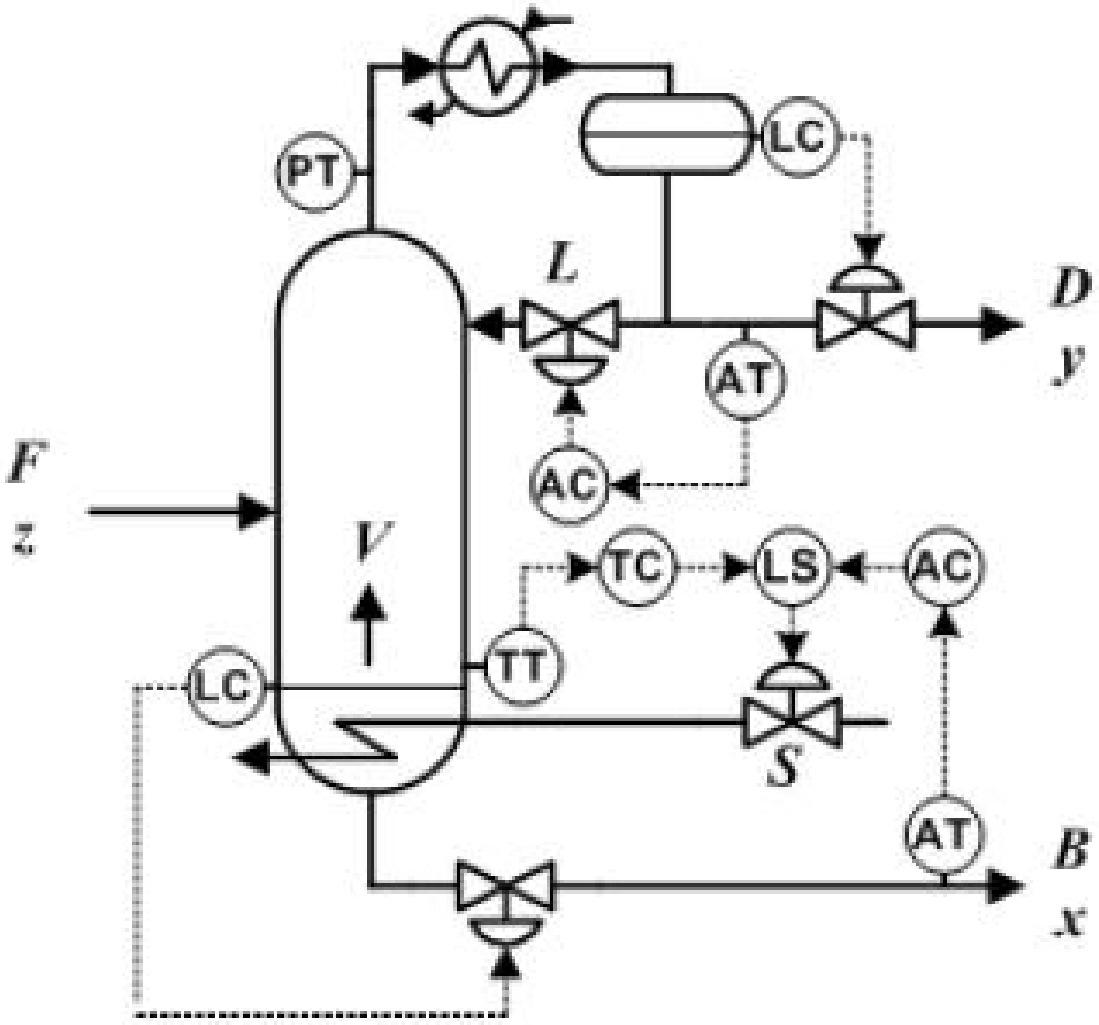


Figure 12 Schematic of the (*L*,*V*) configuration with LS constraint controls designed to maintain control of the overhead purity when a maximum reboiler temperature constraint is encountered.

Figure 13 shows a control configuration for observing the maximum reboiler temperature constraint when the bottoms product is more important. Note that in this case, the reflux flow rate is used to maintain the bottom product composition while the reboiler duty is used to maintain the reboiler temperature. In this configuration, the reboiler temperature control loop acts relatively quickly while the bottoms composition loop is slower acting because reflux flow is used as the MV.

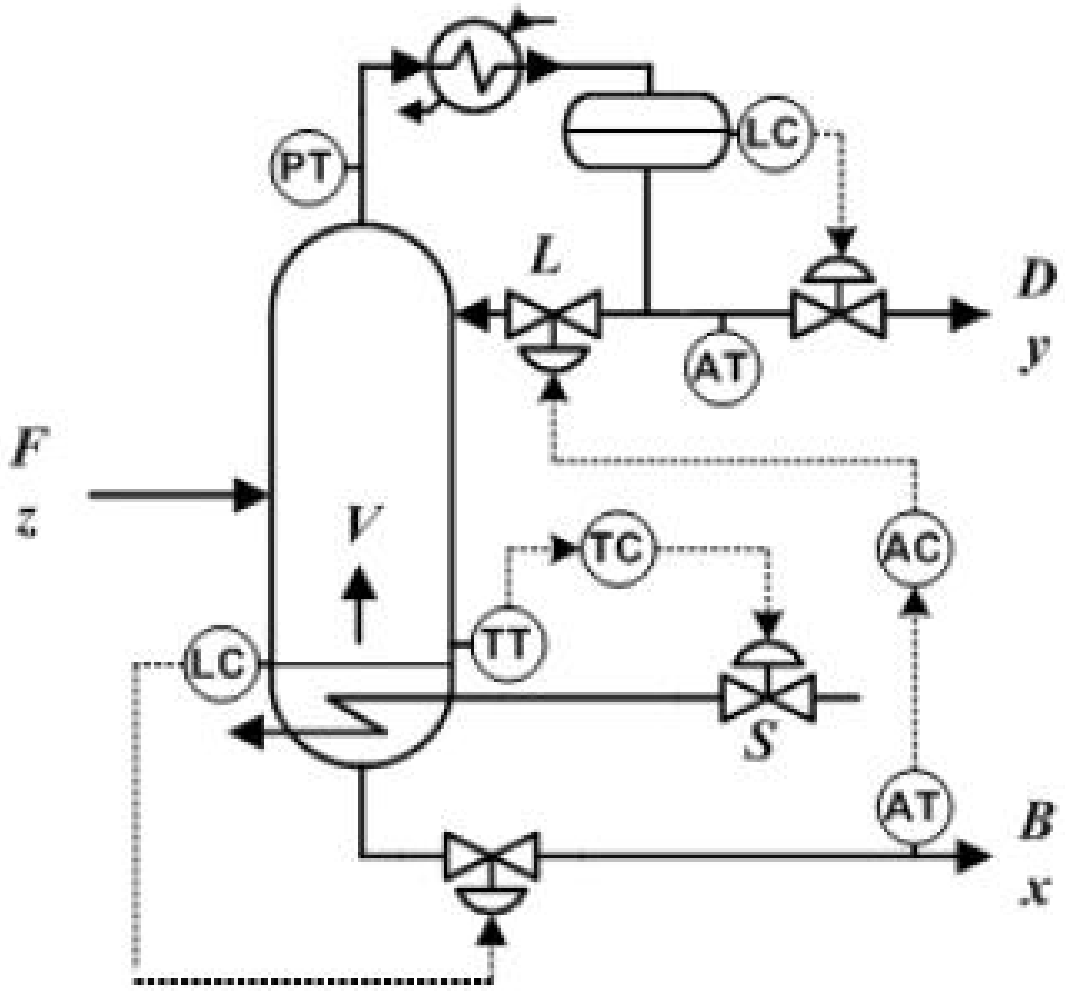


Figure 13 Schematic for the configuration of a constraint controller for a maximum reboiler constraint for which the bottom product is most important.

Multivariable Control

The advantage of model predictive control (MPC) applied to distillation columns is greatest when MPC is applied to a series of columns, e.g., an entire separation train. This results because the MPC controller can efficiently operate the system of columns against the operative constraints to maximize the throughput for the system.

For a typical industrial separation train, there is a large complex set of constraints that limit the overall process throughput. As a result, there is a large number of combinations of operative constraints that must be controlled if the controller is to maximize the throughput for the full range of operation. Advanced PID constraint controllers applied to this problem require a separate control configuration for each combination of operative constraints, resulting in an excessively large number of separate control configurations.

On the other hand, an MPC controller can directly handle the full range of combinations of operative constraints with a single MPC controller. In addition, the MPC controller is much easier to maintain than a custom-built advanced PID controller for such a complex system.

MPC can provide significant control improvements over PID control for a single column in most cases, but these improvements pale in comparison to the economic advantages offered by applying MPC to large-scale processes.

Keys to Effective Distillation Control
For effective distillation control, it is imperative to take care of the basics first.

1. Ensure that the regulatory controls are functioning properly.
2. Evaluate the analyzer dead time, reliability and accuracy.
3. Check that RTDs or thermistors are being used to measure tray temperatures for composition inference and that they are correctly located. Also, ensure that pressure-corrected tray temperatures are used.
4. Use internal reflux controls for changes in reflux temperature.
5. When L, D, V and B are used as MVs for composition control, ratio them to the measured feed rate when column feed rate changes are a common disturbance.

For configuration selection, use the (L,V) configuration for single composition control. For dual composition control, use an energy balance configuration for low reflux ratio cases and use material balance or ratio configurations for high reflux ratio columns. For many dual composition control cases, the control of one product is much more important than the other. For such cases, you should use L as the MV when the important product is produced in the overhead and V when it is produced in the bottoms.

Additionally, the less important product should be controlled using an energy balance knob (L or V) or a ratio knob (L/D or V/B) for low reflux cases or D, L/D, B or V/B for high reflux columns. For these cases, it is important to tune the important loop tightly and tune the less important loop much less aggressively.

Finally, override and select control should be applied to ensure that all column constraints are satisfied when they become operative.

Return to the Table of Contents to learn more.

1. This material is reprinted from *Chemical Process Control, 2nd Ed.* with the permission of the publisher: Ferret Publishing (806 747 3872).

20) Discrete Time Modeling of Dynamic Systems (by Peter Nachtwey)

A Discrete Time Linear Model of the Heat Exchanger

Since most of us don't have our own heat exchanger to test on, I thought it would be good to show how one can be simulated. This would allow us to test the equations that calculate the PI tuning values over the range from a conservative to aggressive response. In addition, the sample period can be changed to see the affect that the sample period has on the response.

Building the Discrete Time Model
Learning how to model a first order plus dead time (FOPDT) system is the easiest place to start. This model simply uses the last process variable (PV) and the new CO (control output) to estimate what the new value of PV will be in the next time period.

The form of the discrete time model is:

$$PV(n+1) = A \cdot PV(n) + B \cdot Kp \cdot CO(n - \Theta p) + C$$

The FOPDT model is basically a simple single pole filter similar to an RC circuit in electronics, except here:
1. the system has gain, Kp
2. the input at time n affects the output at time n+1
3. the system has an initial steady state, C, that is not 0
4. the system has dead time, Θp

The first thing that must be done is to build the basic model by specifying the A and B constants. Note that A and B are constants for a first order system and are arrays for higher order systems. So in this study:

$$A = \exp(-T/Tp)$$
$$B = (1 - A)$$

T is the sample period and *T*p is the plant time constant. A and B are coefficients in the range from 0 to 1, and coefficient A tends to be close to 1.

Calculating coefficient C in the above model is the tricky part. The heat exchanger steady state is not 0 °C. If coefficient C is left out the above model equation or it is set to zero, than PV will approach 0. This does not match the observed behavior of the heat exchanger. Therefore, a value for C must be calculated that will provide the correct value of PV when CO is set to 0.

This is done as follows. From the heat exchanger data plot, we can see that the heat

exchanger temperature (PV) is 140 °C degrees when the controller output (CO) is 39%. If these values are plugged into the formula y=mx+b, we can solve for the steady state temperature, PVss:

140 = 39Kp + PVss

And this lets us calculate coefficient C:

C = (1 – A) PVss

The data below is from the Validating Our Heat Exchanger FOPDT Model tutorial. The heat exchanger is modeled as a reverse acting FOPDT system with Laplace domain transfer function:

$$Gp = \frac{Kp\, e^{-\theta p s}}{\tau p\, s + 1}$$

The model parameter values for Kp, *T*p and Θp where provided in the tutorial as:
• Plant gain, Kp = –0.533 °C/%
• Time constant, *T*p = 1.3 min
• Dead time, Θp = 0.8 min

Also,
• Sample time, T = 1/60 min
• Dead time in samples = nΘp = Θp/T = 48

Steady state PV can now be computed assuming the system is linear:

PVss = 140 – 39Kp = 160.8 °C

Next generate the discrete time transition matrix. In this simple case it is a single element:

A = exp(–T/Tp) = 0.987

Also generate the discrete time input coupling matrix, which again is a single element. Notice that the plant gain is negative:

B Kp = (1 – A) Kp
= (1 – 0.987)(–0.533)
= –0.00679

Using the above information, coefficient C becomes:

C = (1 – A) PVss = 2.048

Substitute our values for A, B and C into the discrete time model form above to calculate temperature (PV) as a function of controller output (CO) delayed by dead time, or:

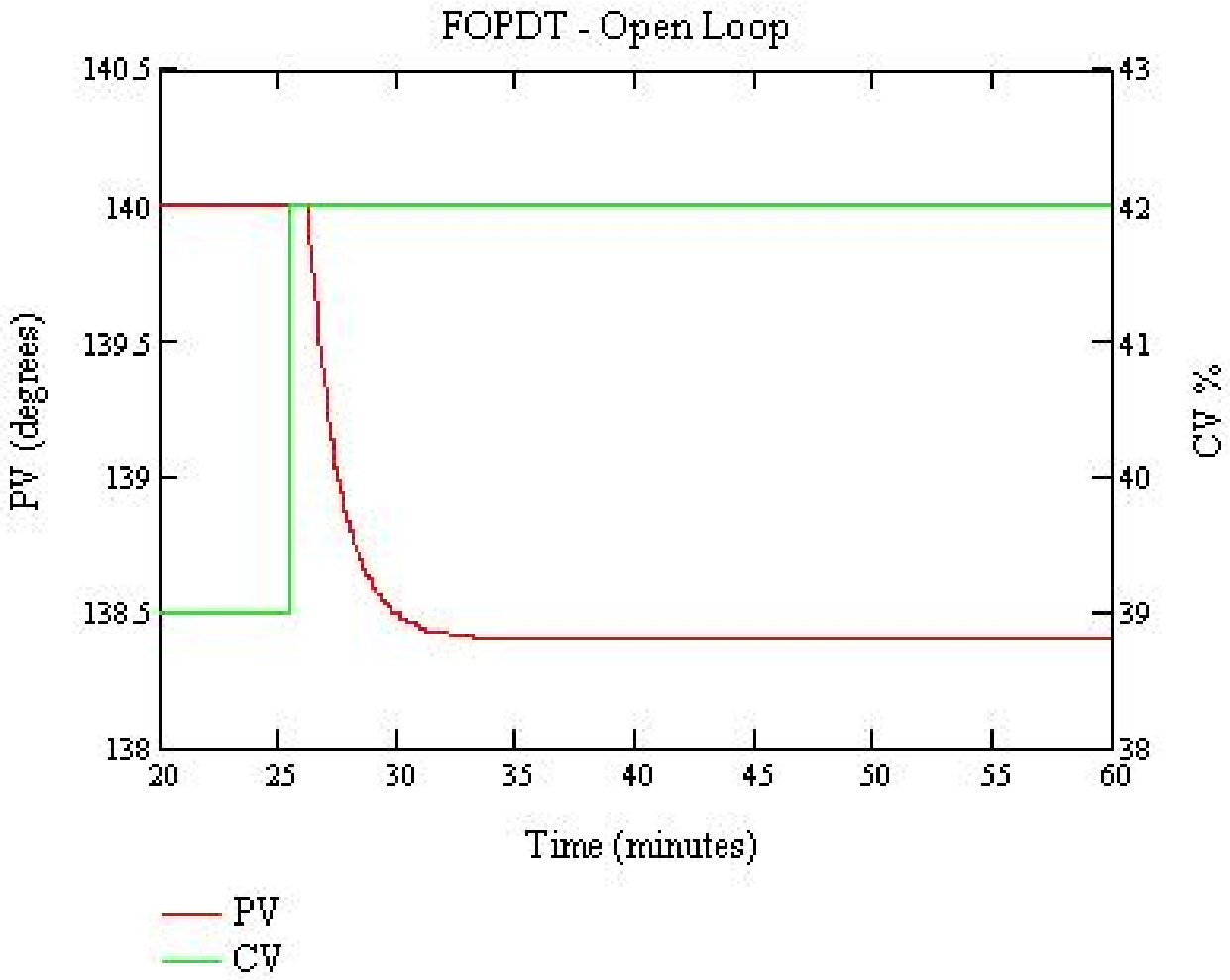
• PV(n+1) = 0.987 PV(n) – 0.00679 CO(n–48) + 2.048

Open Loop Dynamics
The easiest way to test this discrete time model is to execute an open loop step test that is similar to the example given in a previous tutorial (see Validating Our Heat Exchanger FOPDT Model).

Initialize:
temperature as: PV(0) = 140
controller output as: CO = 39 for n ≤ 48

Implement controller output step at time t = 25.5 min, or n = 1530:
CO = 39 for n < 1530
CO = 42 for n ≥ 1531

Simulate for 60 min, or: n = 0,1,…,3600. Display results in the plot below after 20 min:



Controller Tuning
The formulas for calculating the PI tuning values can now be tested using the discrete time process model that was generated above.

The tuning values are for the PI controller that is expressed in the continuous time or analog form as:

$$CO = CO_{bias} + Kc \cdot e(t) + \frac{Kc}{Ti}\int e(t)dt$$

The PI controller tuning values are calculated using the equations provided in previous tutorials (see PI Control of the Heat Exchanger). Following that procedure, the desired closed loop time constant, *T*c, must be chosen to determine how quickly or how aggressive the desired response should be.

For an aggressive response, *T*c should be the larger of 0.1*T*p or 0.8Θp. As listed above, *T*p = 1.3 min and Θp = 0.8 min for the heat exchanger. Using these model parameters, then:

Tc = max(0.1*T*p, 0.8Θp) = 0.64

A moderate *T*c of the larger of 1.0*T*p or 8.0Θp will produce a response with no overshoot. We will use this rule in the closed loop simulation below:

Tc = max(1.0*T*p, 8.0Θp) = 6.4

Using this moderate *T*c, we can now compute the controller gain, Kc, and integrator term, *T*i:

- $Kc = \frac{Tp}{Kp(Tc+\theta p)} = \frac{1.3}{-0.533(0.8+6.4)} = -0.339\frac{\%}{^oC}$
- $Ti = Tp = 1.3 \text{ min}$

Closed Loop PI Control
The function logic that follows implements the PI controller and the discrete time FOPDT model's response. The controller uses the PV to calculate a CO that is used later by the plant model. How much later is determined by the dead time. The model then responds by changing the PV. This new PV is then used in the next iteration.

- compute controller error:

ERR = SP – PV

- compute COp, the proportional contribution to CO:

COp = Kc·ERR

- compute COi, the integral contribution to CO:

COi = COi + (Kc·T·ERR)/*T*i

- compute the total controller output:

CO = COp + COi

- compute the process response at the next step n using the time-delayed CO:

PV(n+1) = A·PV(n) + B·Kp·CO(n–Θp) + C

Aside: The integral contribution to CO can include the integrator limiting technique talked about in the Integral (Reset) Windup and Jacketing Logic post with the expression:

$$COi = \max\left(\min\left(COi + \frac{Kc \cdot T}{Ti} ERR,\ 100 - COp\right),\ 0 - COp\right)$$

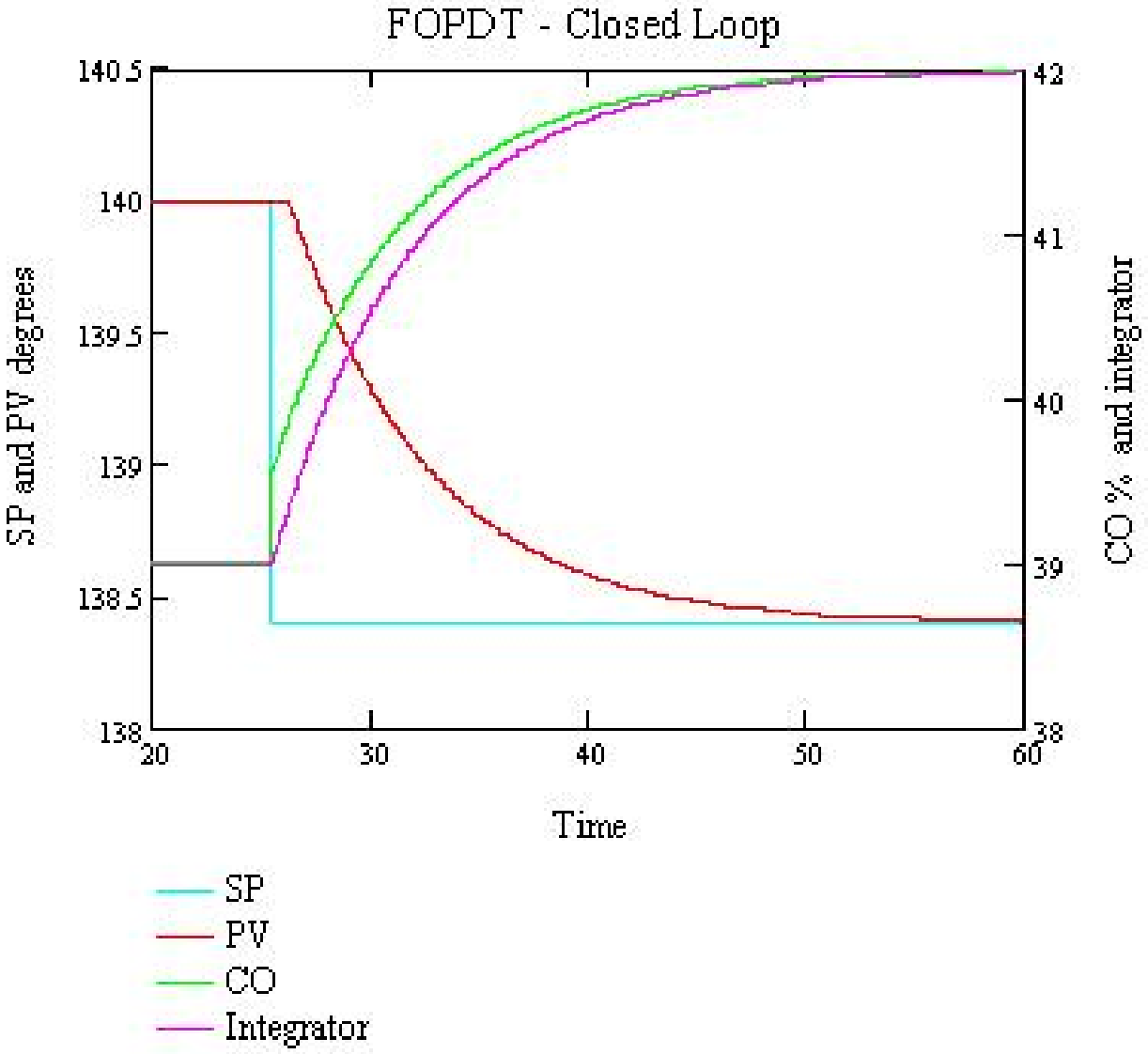
We initialize the variables at time t=0 by assuming that there is no error and the system is in a quiescent state. Thus, all of the control output, CO, is due only to the integrator term, COi:

PV(0) = 140; CO(0) = 39; COi(0) = CO(0)

The plot below shows the response of the closed loop system over a simulation time of 60 minutes. The set point is stepped from 140 to 138.4 at time t = 25.5 minutes.

The function logic above calculates the PV and CO for each time period in the 60 minute simulation. Note that there are three CO type variables. CO is the current control value, COi is the current integrator contribution to the CO, and CO(n-Θp) is the delayed CO that simulates the dead time.

When programming the functions, additional logic is required keep from using indexes less than 0.



Notice that to prevent overshoot, the rate at which the controller's integrator output, COi, responds or winds up must be limited so the total control output does not exceed the final steady state control output, and final steady state control output is not reached before the system starts to respond. The way I think about it is that the function is calculating the value for Kc so this goal can be achieved.

Also notice that with moderate tuning, the closed loop response is much slower than the open loop response.

21) Fuzzy Logic and Process Control (by Fred Thomassom)

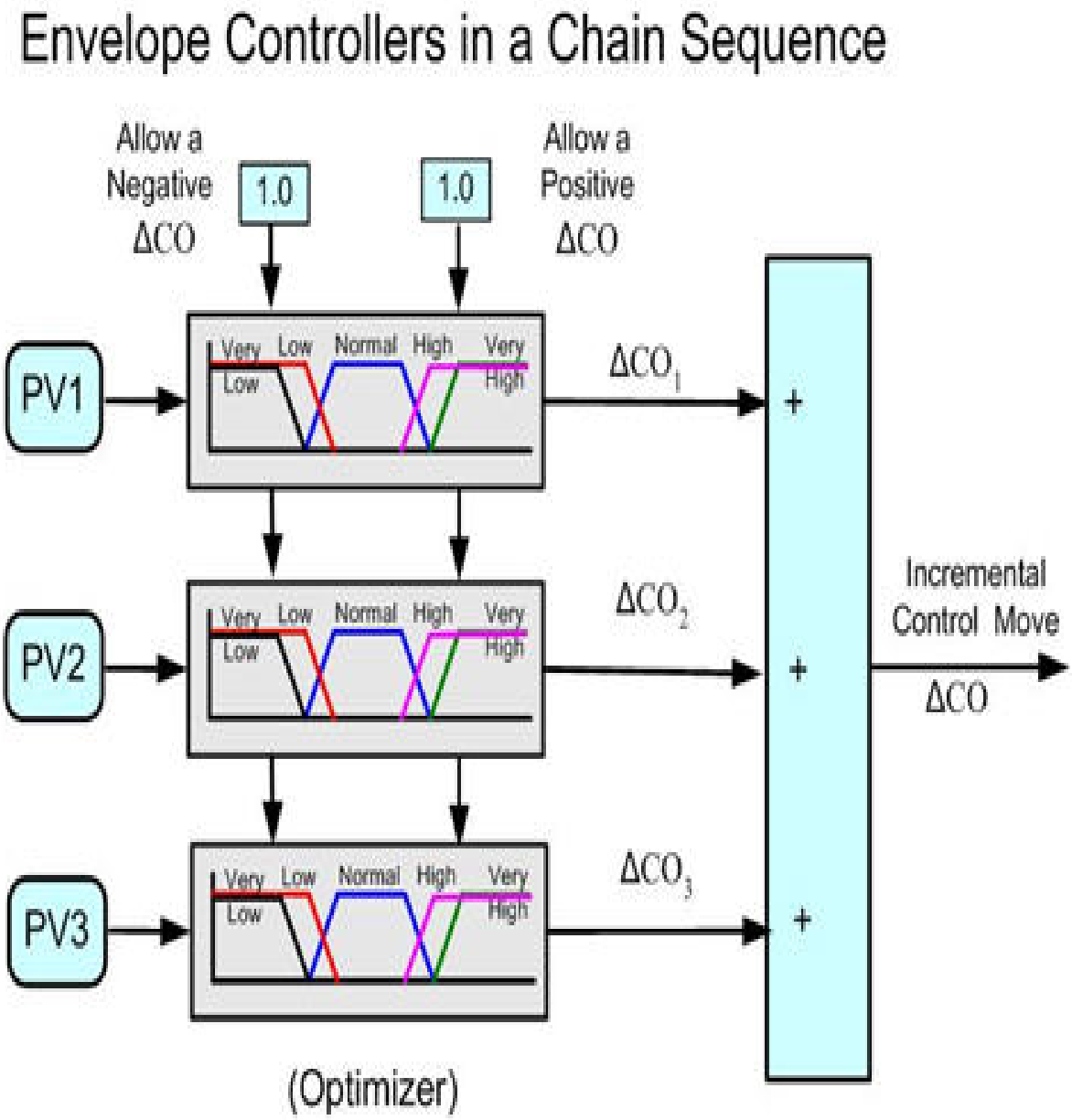
Envelope Optimization and Control Using Fuzzy Logic

In the *Control Magazine* article entitled Envelope Optimization by Bela Liptak (www.controlglobal.com; 1998), a multivariable control strategy is described whereby a process is “herded” to stay within a safe operating envelope while the optimization proceeds.

Building on that work, a method to implement Envelope Optimization using fuzzy logic is described in this article (a primer on fuzzy logic and fuzzy control can be found here).

When some of us think about optimization, we think about complex gradient search methods. This is something that a PhD dreams up but they rarely ever work in the plant. After the theoretician goes back to the R & D facility, the plant engineers are charged with keeping it running. After a few glitches that nobody understands, it is turned off and that marks the end of the optimization effort.

Background
The basic building tool for this optimization methodology is the fuzzy logic Envelope Controller (EC) function block. Such EC blocks are often chained (or sequenced) one after the other. As shown below, each EC in the chain sequence focuses on a single process variable, with the complete chain forming an efficient and sophisticated optimization strategy.



With such a chain sequence construction, it is possible to incorporate many constraints. It is not difficult to add or remove a constraint or change its priority, even in real time.

To better understand the operation of the chain sequence shown in the figure above, use the following analogy. Three operators are sitting side by side in front of their own operator's console. Each operator can only view a single PV. There are operating constraints for the PV [Very Low, Low, Normal, High, Very High]. Each operator can make incremental control moves to the single output variable. They can make a move once a minute.

Operator #1 monitors PV1. If PV1 is Very High, a fixed size negative incremental move will be made. If PV1 is Very Low, a positive move will be made. Otherwise no control move is made. Control moves are made to insure the constraints are not exceeded.

Operator #2 monitors PV2. If PV2 is Very High, a fixed size negative incremental move needs to be made. If PV2 is Very Low, a positive move needs to be made. Otherwise no

control move needs to be made. But first, permission from Operator #1 is required. If PV1 is Very High, a positive control move is not allowed. If PV1 is Very Low, a negative move is not allowed. Otherwise permission is granted for any move.

Permission is a fuzzy variable which varies from 0.0 to 1.0. Permission of 0.50 allows Operator # 2 to make half of the fixed control move.

Operator #3 monitors PV3. This operator performs the optimization function. In this example, it will be to maximize production rate. Therefore PV3 is the production rate. If PV3 is not High, a positive control move will be made. This operator only makes positive control moves. But first permission from both Operator #1 and Operator #2 is required. It will insure that the constraints are not violated. This requires that neither PV1 or PV2 are High. Operator #3 will stop making control moves when PV3 becomes High.

It is up to Operator #1 and Operator #2 to reduce the production rate if a constraint is exceeded.

A detailed description of the Envelope Controller block is given in the next section. However it may be clearer to go directly to the two optimization examples at the end of the article and then come back and read the rest of the article. The description of the Envelope Controller Block is included for completeness.

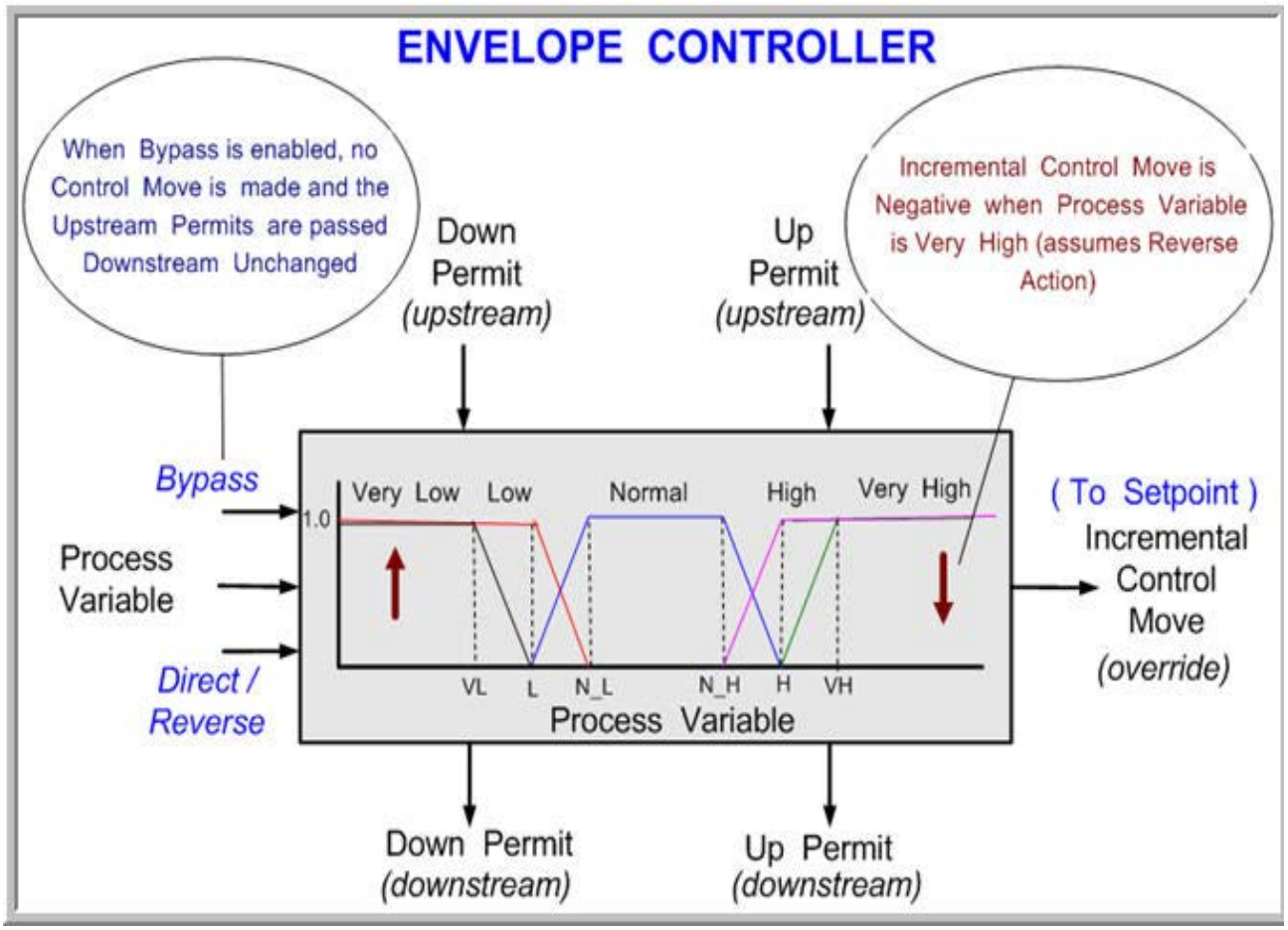
Two optimization examples are shown later in this article. The first deals with minimizing the amount of combustion air for an industrial power boiler. The second deals with minimizing the outlet pressure of a forced draft fan of an industrial power boiler to minimize energy losses.

This controller is best suited to applications where there are multiple measured input variables that must stay within prescribed limits and there is a single output variable.

The Envelope Controller Block

Before discussing how the fuzzy logic Envelope Controller (EC) function block can be chained in a sequence to form an optimization strategy, we first consider the function of a single EC block.

A schematic of the basic EC block is shown in the figure below:



As will be illustrated in the examples later in this article, the terms "upstream" and "downstream" refer to the flow of information in the chain sequence of EC's in the optimization strategy and not the flow of material through processing units.

The basic construction is to use the chain of ECs to oversee constraints in order of priority. The last EC in the chain is used as an optimizer.

Envelope Controller Inputs, Outputs and Parameters:

Inputs:

1. Process Variable (continuous PV)
2. Down_Permit (0 – 1.0)
3. Up_Permit (0 – 1.0)

Outputs:

1. Incremental Control Output (continuous CO)
2. Down_Permit (0 – 1.0)
3. Up_Permit (0 – 1.0)

Parameters to Specify:

1. NEG - (size of negative move)
2. POS - (size of positive move)
3. Six breakpoints (VL, L, N_L, N_H, H, VH) on the PV axis which uniquely identifies all five membership functions
4. G - (Overall Gain)

Switches to Set:
1. Bypass (on/off)
2. Control Action (Reverse or Direct)

Membership Functions:
Five membership functions are shown superimposed in the Envelope Controller (EC) figure above. Each membership function is a fuzzifier. A continuous process variable (PV) is mapped to 5 sets of fuzzy variables whose values are between 0.0 and 1.0. They are:

1. PV_VL = μ_1(PV) VL: Very Low
2. PV_L = μ_2(PV) L: Low
3. PV_N = μ_3(PV) N: Normal
4. PV_H = μ_4(PV) H: High
5. PV_VH = μ_5(PV) VH: Very High

Where μ means degree of membership. When PV is midway between VL and L in the above EC diagram, for example, then μ_1 = 0.5 and μ_2 = 1.0. All the rest are 0.0.

The fuzzy variables, such as PV_N, are called linguistic variables. Here, PV_N means “PV is Normal.” Similarly, PV_VH means "PV is very high."

Control Action Switch
This switch has two positions as shown in the figure above: “Reverse” and “Direct.” In most processes, when the process variable (PV) exceeds the high limit, the controller output (CO) must be decreased to bring it back into the safe operating envelope. The control action switch must be in the “Reverse” position for this to occur.

However, there are processes where the CO must be increased to bring it back into the safe operating envelope if the PV exceeds the high limit. The position of the control action switch must be “Direct” for these processes.

Thus, the control action switch of the EC is analogous to the direct and reverse action of PID controllers.

Bypass Selector Switch
As shown in the figure above, an EC has a bypass switch. When the EC is in bypass mode, the EC is switched out of the chain sequence and the Up Permit_In and Down Permit_In signals pass through the EC unchanged as if it does not exist.

Controller Functionality
The membership functions of each EC define the safe operating region for the incoming PV. When the PV is in the “PV Normal Region,” the incremental control move (override) produced by the EC is 0.0. The Up Permit_In and Down Permit_In values are passed unmodified to the next Envelope Controller in the chain sequence.

To simplify the discussion, we assume the EC uses reverse action in the following logic. A direct action EC uses analogous, though opposite, logic.

When the PV entering an EC is in the “High” or “Low” regions, the Up Permit_Out or the Down Permit_Out variable is modified by the fuzzy variable Correction Factors (CF):

1. Up Permit_CF = NOT (H)
2. Down Permit_CF = NOT (L) where NOT (x) = 1–x

The product operator (μ_A[PV]*μ_B[PV]) is used in the EC computation. This is one version of the fuzzy “AND.” The Up Permit_In and Down Permit_In are multiplied by the appropriate correction factors to become:

1. Up Permit_Out = (Up Permit_In)* (Up Permit_CF)
2. Down Permit_Out = (Down Permit_In)*(Down Permit_CF)

When the PV enters the “Very High” or “Very Low” regions, an incremental control move (override) is made to nudge the PV back into the safe region:

1. ΔCO = (Down Permit_In)*(PV_VH)*NEG*G
2. ΔCO = (Up Permit_In)*(PV_VL)*POS*G

This is a simple two-rule controller. For a EC with reverse action, these rules can be summarized:

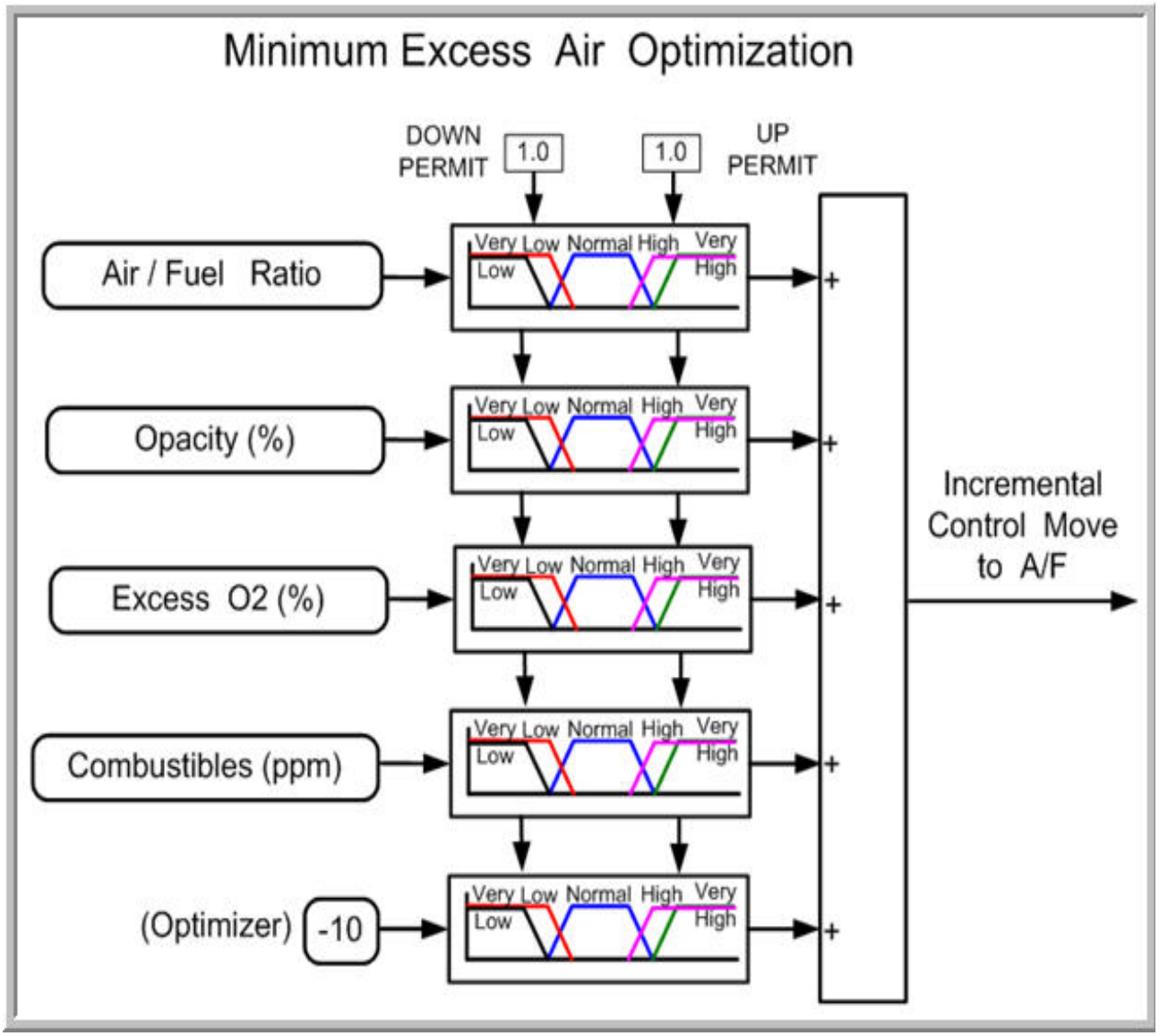
[**R1**] if there is permission to make a negative control move **and** the PV is very high (VH), then make a negative incremental control move.

[**R2**] if there is permission to make a positive control move **and** the PV is very low (VL), then make a positive incremental control move.

NEG*G and POS*G are singletons. ΔCO is the defuzzified value. It is simply the product of the singleton and the truth of the rule. There are many defuzzification techniques. A simple but highly effective method uses singletons and a simple center of gravity calculation. This is the one used in these applications.

Chaining in Sequence
ECs are chained in sequence to form an efficient Envelope Optimization controller. With a proper number of constraints in proper priority order, they form a safe operating envelope for the process. ECs can be implemented in many modern process control system to create Envelope Optimization logic.

□□ Example: Minimum Excess Air Optimization
This application of Envelope Optimization is for an industrial power boiler. As shown below (), it uses five ECs chained in sequence. Like many optimization problems, the solution for minimum excess air optimization is found on the constraint boundary.



Control Objectives

1) Maintain combustion air at minimum flow required for safe operation and maximum boiler efficiency.
2) Stay within the safe operating envelope as defined by the limits of the first four ECs.
3) Take immediate corrective action when any of the prioritized constraints are violated.
4) Optimization must proceed slowly to avoid upsetting the process.

Optimization Functionality

The air-to-fuel (A/F) ratio is slowly reduced by the negative incremental moves made by the Optimizer EC (the last in the chain sequence) until a limit is reached on one of the first four ECs. This limit condition would then reduce the Down Permit entering the Optimizer EC to 0.0.

Notice that the Up and Down Permits are set to 1.0 at the top of the chain. If the PVs of the first four ECs are all in the “Normal” regions. The Up Permit and Down Permit will pass down the chain unmodified as 1.0.

The first EC sets the high and low limits for the A/F ratio. Note that this EC has the highest priority. If the A/F reaches its low limit, the Down Permit goes to 0.0. The second EC sets the high limit for Opacity (no low limit). The third EC sets the high and low limits for O_2. The fourth EC sets the high limit for combustibles (unburned hydrocarbons). There is no low limit. Normally this control operates at the low limit for A/F ratio with low O_2 and very low combustibles.

Aside: The optimization function is automatically disabled when the combustibles meter goes into self-calibration and resumes normal operation when the calibration is finished. The operator can also turn the optimization "off" at any time. The Minimum Excess Air Optimization can function with just O_2 if the combustibles meter is out of service. The Combustibles EC would need to be placed on "bypass" for this to occur.

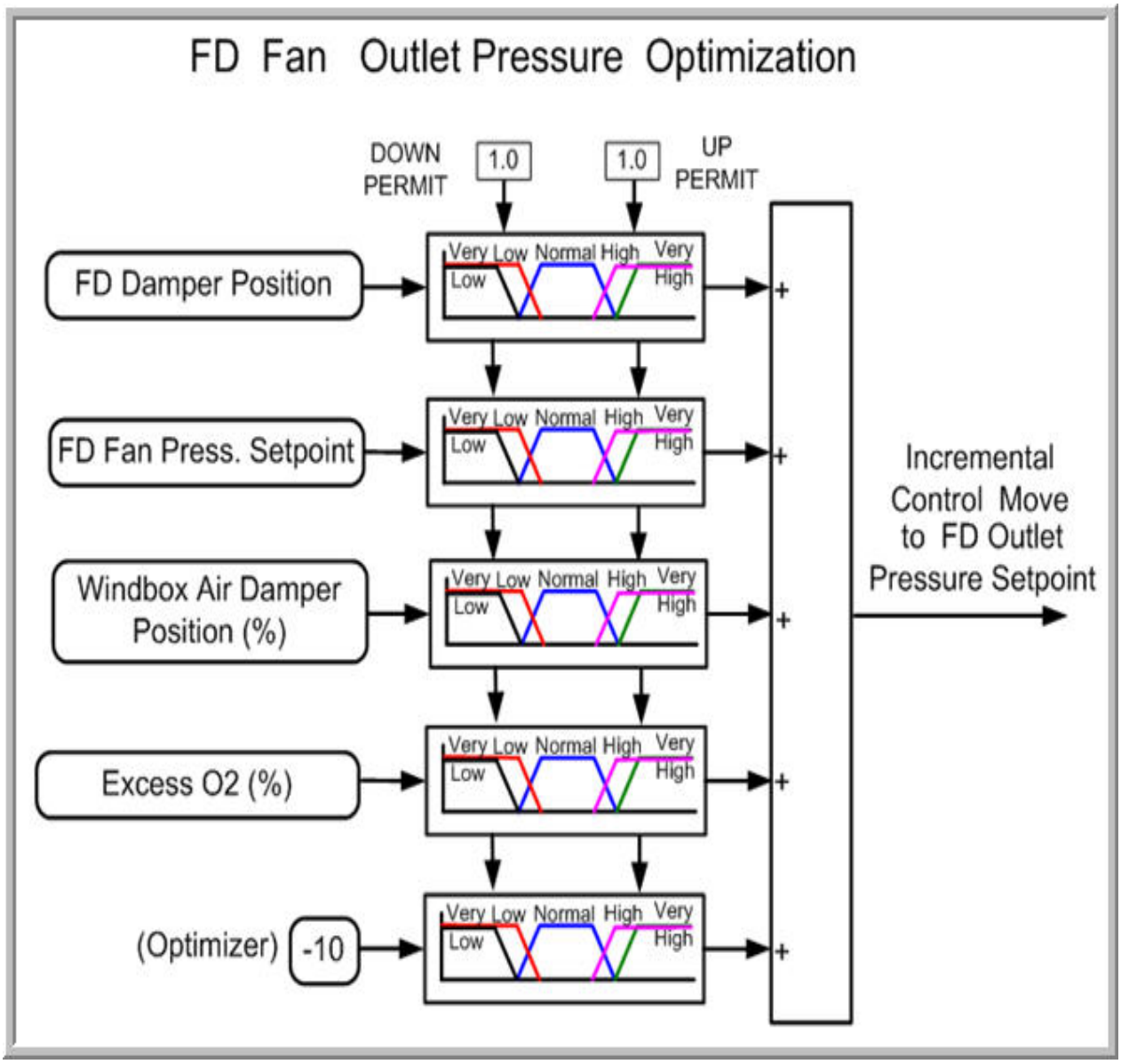
While the optimizing function is slow, that is not the case for the constraint override action. For example, if the O_2 fell below the low limit, the A/F ratio would quickly increase to bring O_2 back within the "Normal" region.

Tuning this control is not simple. It requires selecting numerous gains and membership breakpoints. The slope of the membership functions are part of the loop gain when override action is taking place.

While tuning the control is difficult, the good news is that once it is tuned it does not seem to require any tweaking afterwards. This particular application has used the same tuning parameters since it was commissioned several years ago with no apparent degradation in performance.

Example: Forced Draft Fan Outlet Pressure Optimization

This application of Envelope Optimization is for an industrial power boiler. It uses five ECs chained together as shown in the graphic below ().



On this power boiler, the Forced Draft (FD) Fan Outlet Pressure is controlled with a PID controller. The PID controller accomplishes this by adjusting the inlet vanes. This provides a constant pressure source for the windbox air dampers. At high steaming rates the pressure needs to be fairly high (about 10.0 inches). At lower steaming rates it doesn't need to be this high and results in a large pressure drop across the windbox dampers. This results in more energy being used than is necessary.

Control Objectives

1) Maintain the FD fan outlet pressure at the minimum pressure required to provide the amount of air needed for combustion.
2) Maintain the windbox air dampers in a mid-range position for maximum controllability.
3) Stay within the "Safe Operating Envelope" as defined by the limits of the first four Envelope Controllers.

Optimization Functionality

The FD fan outlet pressure set point is slowly reduced by negative incremental moves made by the last Envelope Controller (Optimizer) until a limit is reached on one of the first four ECs. Normally it is the FD Fan Pressure set point EC that reaches its low limit and

reduces the Down Permit to 0.0 entering the Optimizer EC.

The Down Permit does not suddenly go to 0.0 as the low limit is reached. It gradually decreases because of the slope of the PV_L membership function. When the truth of PV_L is 0.5, the Down Permit becomes 0.5 and only half of the Optimizer's regular incremental move is made. Likewise, when PV_L is 0.1 only one tenth of the Optimizer's regular move is made.

The first EC sets the high and low limits for the FD damper position. The actuator always has the highest priority. The second EC sets the high and low limits for the FD outlet pressure set point. The third EC sets the high limit for the windbox air dampers (no low limit). The fourth EC sets the low limit for O_2 (no high limit).

At low boiler steaming rates the FD fan outlet pressure is held at minimum by the second EC. As the boiler steaming rate increases, the windbox dampers open further to provide more combustion air. The damper position reaches its very high limit causing a positive incremental override control move from the third EC. This causes the pressure set point to increase. As the FD fan outlet pressure rises, the damper moves back to a lower value.

Note that the total incremental control move is made up of the sum of the moves from all of the ECs. In this situation, the new equilibrium occurs when the sum of the third and fifth ECs equal 0.0. The third EC is making a positive move and the Optimizer EC is making a negative move. If the gains and slopes of the membership functions of the third EC are not carefully selected, cycling (otherwise known as instability) will occur. In control jargon this feature could be called mid-ranging. It is desired to always have the windbox damper operating in its normal range.

If for some reason the O_2 goes below its very low limit, positive incremental override control moves will be made by the fourth EC raising the FD fan outlet pressure. This increases air flow to the windbox. This is a safety feature.

Tuning this control is not simple. This is particularly true for the interaction between the third EC and the Optimizer. In this application none of the PVs have significant time constants or delays. Therefore predicted PVs were not used. While tuning the control is difficult, the good news is that once it is tuned it does not seem to require any tweaking afterwards. This particular application has used the same tuning parameters since it was commissioned several years ago with no apparent degradation in performance.

Final Thoughts

An interesting method to implement Envelope Optimization using fuzzy logic has been presented in this article. Constraints can be prioritized with those affecting safety and environmental concerns at the top of the list. The Envelope Controller is at the heart of the approach. Linking these controllers together in a chain allows one to form a "Safe Operating Envelope" which is the constraint boundary. Optimization proceeds as the process is kept within this envelope. Two examples were shown that use this methodology.

The reader should study some of the fundamental concepts of fuzzy logic if they are interested in an in-depth understanding of this approach. Understanding how to tune these controls is the most difficult part of the application. This can only be done during the

startup phase or with a dynamic simulation.

The method described and the examples presented are valid for situations where there is only one variable to manipulate. This approach can be extended to problems where there are multiple variables to manipulate. The author has worked on problems where two variables were manipulated. It usually requires a sophisticated predictive model to provide inputs to some of the Envelope Controllers. It actually becomes a type of Model Predictive Control (MPC).